Học Knowledge Distillation từ đầu

Lộ trình 48 bài

Ghi chú học tập bằng tiếng Việt

Tháng 11, 2026

Mục lục

Học Knowledge Distillation từ đầu: lộ trình 48 bài . . . 4
Nhãn mềm và nhiệt độ: học từ cả phân phối của thầy, không chỉ từ đáp án đúng . . . 8
Chưng cất đặc trưng, attention và quan hệ: dạy trò không chỉ đáp án mà cả cách thầy nhìn . . . 15
Quan hệ thầy-trò: thầy lớn hơn chưa chắc dạy tốt hơn . . . 23
Chưng cất cho dữ liệu dạng chuỗi: học từng token, học cả chuỗi và khớp bên trong BERT . . . 31
KL thuận, KL ngược và chưng cất on-policy: trò nhỏ nên bắt chước thầy ở đâu . . . 39
Chưng cất bằng dữ liệu và lập luận: thầy chỉ cần viết ra văn bản là học trò đã học được . . . 47
Đo chưng cất, scaling và khi nào nên chưng cất: tính đủ chi phí rồi mới kết luận . . . 55
00 Preface: Toán nền tảng và hướng dẫn đọc về Knowledge Distillation . . . 65
01 Do Deep Nets Really Need to be Deep? (2014) . . . 75
02 Distilling the Knowledge in a Neural Network (2015) . . . 80
03 FitNets: Hints for Thin Deep Nets (2015) . . . 87
04 Paying More Attention to Attention: Improving the Performance of Convolutional Neural Networks via Attention Transfer (2017) . . . 92
05 Knowledge Distillation: A Survey (2021) . . . 97
06 Born Again Neural Networks (2018) . . . 104
07 Deep Mutual Learning (2017) . . . 110
08 Improved Knowledge Distillation via Teacher Assistant (2020) . . . 115
09 On the Efficacy of Knowledge Distillation (2019) . . . 120
10 Does Knowledge Distillation Really Work? (2021) . . . 126
11 Knowledge distillation: A good teacher is patient and consistent (2022) . . . 131
12 Relational Knowledge Distillation (2019) . . . 136
13 Contrastive Representation Distillation (2020) . . . 142
14 Decoupled Knowledge Distillation (2022) . . . 148
15 Sequence-Level Knowledge Distillation (2016) . . . 154
16 DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter (2019) . . . 160
17 Patient Knowledge Distillation for BERT Model Compression (2019) . . . 165
18 TinyBERT: Distilling BERT for Natural Language Understanding (2020) . . . 171
19 MiniLM: Deep Self-Attention Distillation for Task-Agnostic Compression of Pre-Trained Transformers (2020) . . . 176
20 Causal Distillation for Language Models (2022) . . . 181
21 Training data-efficient image transformers & distillation through attention (2021) . . . 185
22 Emerging Properties in Self-Supervised Vision Transformers (2021) . . . 190
23 AM-RADIO: Agglomerative Vision Foundation Model -- Reduce All Domains Into One (2024) . . . 195
24 CLIP-KD: An Empirical Study of CLIP Model Distillation (2024) . . . 201
25 LLaVA-MoD: Making LLaVA Tiny via MoE Knowledge Distillation (2024) . . . 207
26 LLaVA-KD: A Framework of Distilling Multimodal Large Language Models (2024) . . . 212
27 A Survey on Knowledge Distillation of Large Language Models (2024) . . . 217
28 MiniLLM: On-Policy Distillation of Large Language Models (2023) . . . 223
29 On-Policy Distillation of Language Models: Learning from Self-Generated Mistakes (2024) . . . 230
30 DistiLLM: Towards Streamlined Distillation for Large Language Models (2024) . . . 235
31 Rethinking Kullback-Leibler Divergence in Knowledge Distillation for Large Language Models (2024) 240

32 Speculative Knowledge Distillation: Bridging the Teacher-Student Gap Through Interleaved Sampling (2025) . . . 245
33 TAID: Temporally Adaptive Interpolated Distillation for Efficient Knowledge Transfer in Language Models (2025) . . . 250
34 DistiLLM-2: A Contrastive Approach Boosts the Distillation of LLMs (2025) . . . 255
35 Self-Instruct: Aligning Language Models with Self-Generated Instructions (2023) . . . 261
36 Distilling Step-by-Step! Outperforming Larger Language Models with Less Training Data and Smaller Model Sizes (2023) . . . 265
37 Orca: Progressive Learning from Complex Explanation Traces of GPT-4 (2023) . . . 270
38 DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning (2025) — phần distillation . . . 276
39 Small Models Struggle to Learn from Strong Reasoners (2025) . . . 282
40 Gemma 2: Improving Open Language Models at a Practical Size (2024) . . . 287
41 Towards the Law of Capacity Gap in Distilling Language Models (2023) . . . 291
42 Distillation Scaling Laws (2025) . . . 296
43 MiniPLM: Knowledge Distillation for Pre-Training Language Models (2025) . . . 303
44 Qwen3 Technical Report (2025) . . . 308
45 Sparse Logit Sampling: Accelerating Knowledge Distillation in LLMs (2025) . . . 312
46 Distil-Whisper: Robust Knowledge Distillation via Large-Scale Pseudo Labelling (2023) . . . 318
47 ShortOPD: Recovering Pruned LLMs with Short-to-Long On-Policy Distillation (2026) . . . 324

Học Knowledge Distillation từ đầu: lộ trình 48 bài

Tập sách giới thiệu **knowledge distillation (KD, chưng cất tri thức)** bằng tiếng Việt, từ các ý tưởng nền tảng đến ứng dụng cho mô hình ngôn ngữ và thị giác. Các bài được sắp xếp thành lộ trình dành cho người mới bắt đầu.

Các bài là ghi chú diễn giải nghiên cứu, không phải bản dịch; thông tin nguồn được ghi ở đầu từng chương.

> Đây là ghi chú diễn giải, không phải bản dịch.

Giai đoạn 0: Chuẩn bị

#	File	Nội dung
00	`00_preface_toan_va_huong_dan.md`	KD là gì, phân loại, toán (temperature, hệ số T^2, forward/reverse KL, f-divergence, feature loss, exposure bias, estimator top-k), cạm bẫy, bảng thuật ngữ, câu tự kiểm tra. **Đọc trước tiên.**
01	`nen01_nhan_mem_nhiet_do_va_kl.md`	Chương nền: Nhãn mềm và nhiệt độ. học từ cả phân phối của thầy, không chỉ từ đáp án đúng.
02	`nen02_chung_cat_dac_trung_attention_va_quan_he.md`	Chương nền: Chưng cất đặc trưng, attention và quan hệ. dạy trò không chỉ đáp án mà cả cách thầy nhìn.
03	`nen03_thay_tro_khoang_cach_nang_luc.md`	Chương nền: Quan hệ thầy-trò. thầy lớn hơn chưa chắc dạy tốt hơn.
04	`nen04_chung_cat_cho_chuoi_va_nlp.md`	Chương nền: Chưng cất cho dữ liệu dạng chuỗi. học từng token, học cả chuỗi và khớp bên trong BERT.
05	`nen05_kl_thuan_nguoc_va_chung_cat_on_policy.md`	Chương nền: KL thuận, KL ngược và chưng cất on-policy. trò nhỏ nên bắt chước thầy ở đâu.
06	`nen06_chung_cat_bang_du_lieu_va_lap_luan.md`	Chương nền: Chưng cất bằng dữ liệu và lập luận. thầy chỉ cần viết ra văn bản là học trò đã học được.
07	`nen07_do_chung_cat_scaling_va_khi_nao_nen.md`	Chương nền: Đo chưng cất, scaling và khi nào nên chưng cất. tính đủ chi phí rồi mới kết luận.

Giai đoạn 1: Nền tảng

#	File	Bài	Ý chính
01	`01_2014_do_deep_nets_need_to_be_deep.md`	Ba & Caruana	Model nông học theo logit của model sâu (gốc từ Model Compression 2006).
02	`02_2015_distilling_knowledge_hinton.md`	Hinton et al.	Soft target với temperature: định nghĩa KD kinh điển.
03	`03_2015_fitnets.md`	FitNets	Học theo **hidden state** (hint) của teacher.
04	`04_2017_attention_transfer.md`	Attention Transfer	Học theo bản đồ attention / kích hoạt.
05	`05_2021_kd_survey.md`	KD: A Survey	Bản đồ toàn cảnh KD.

Giai đoạn 2: Hiểu KD (vì sao, khi nào nó hoạt động)

#	File	Bài	Ý chính
06	`06_2018_born_again_networks.md`	Born-Again Networks	Student cùng kích cỡ **vượt** teacher.
07	`07_2018_deep_mutual_learning.md`	Deep Mutual Learning	Các student học lẫn nhau, không cần teacher lớn.
08	`08_2020_teacher_assistant.md`	Teacher Assistant	Capacity gap: teacher quá lớn thì kém; dùng model trung gian.
09	`09_2019_efficacy_of_kd.md`	On the Efficacy of KD	Teacher tốt hơn không phải lúc nào cũng dạy tốt hơn; early stopping teacher.
10	`10_2021_does_kd_really_work.md`	Does KD Really Work?	Student thường **không khớp** được teacher; vấn đề tối ưu.
11	`11_2022_patient_consistent_teacher.md`	Patient and Consistent Teacher	Cùng augmentation cho teacher và student, train rất lâu.

Giai đoạn 3: Feature, relation và loss mới

#	File	Bài	Ý chính
12	`12_2019_relational_kd.md`	Relational KD	Học quan hệ giữa các mẫu (khoảng cách, góc).
13	`13_2020_contrastive_representation_distillation.md`	CRD	KD bằng contrastive learning trên biểu diễn.
14	`14_2022_decoupled_kd.md`	Decoupled KD	Tách KL thành phần lớp đúng và phần các lớp khác.

Giai đoạn 4: KD cho NLP và chuỗi

#	File	Bài	Ý chính
15	`15_2016_sequence_level_kd.md`	Sequence-Level KD	KD ở mức **chuỗi**: học theo output beam search của teacher.
16	`16_2019_distilbert.md`	DistilBERT	Khởi tạo từ teacher, bỏ nửa số layer, loss ba thành phần.
17	`17_2019_patient_kd_bert.md`	Patient KD	Học hidden state của nhiều layer trung gian.
18	`18_2020_tinybert.md`	TinyBERT	Ánh xạ layer; loss trên attention, hidden, embedding, prediction.
19	`19_2020_minilm.md`	MiniLM	Distill quan hệ self-attention của layer cuối.
20	`20_2022_causal_distillation.md`	Causal Distillation	Distill **cấu trúc nhân quả** (interchange intervention). Nối với interpretability.

Giai đoạn 5: KD cho vision transformer và VLM

#	File	Bài	Ý chính
21	`21_2021_deit.md`	DeiT	Distillation token cho ViT; hard vs soft distillation.
22	`22_2021_dino.md`	DINO	Self-distillation không cần nhãn cho ViT.
23	`23_2024_am_radio.md`	AM-RADIO	Distill nhiều teacher vào một vision encoder. Liên quan việc nén ViT.
24	`24_2024_clip_kd.md`	CLIP-KD	Nghiên cứu thực nghiệm về distill CLIP.
25	`25_2024_llava_mod.md`	LLaVA-MoD	Làm LLaVA nhỏ lại bằng MoE và KD.
26	`26_2024_llava_kd.md`	LLaVA-KD	Framework distill MLLM (output và relation).

Giai đoạn 6: KD cho LLM: hàm loss và on-policy

#	File	Bài	Ý chính
27	`27_2024_survey_kd_of_llms.md`	Survey on KD of LLMs	Bản đồ KD cho LLM.
28	`28_2023_minillm.md`	MiniLLM	Reverse KL, tối ưu bằng policy gradient.
29	`29_2024_gkd_on_policy.md`	GKD	On-policy distillation: student học trên output của chính nó.
30	`30_2024_distillm.md`	DistiLLM	Skew KL và dữ liệu student sinh ra một cách thích nghi.
31	`31_2024_rethinking_kl_llm_kd.md`	Rethinking KL in LLM KD	Forward hay reverse KL: thực tế khác lý thuyết.
32	`32_2025_speculative_kd.md`	Speculative KD	Xen kẽ token của student và teacher khi sinh.
33	`33_2025_taid.md`	TAID	Teacher "nội suy" thay đổi dần theo thời gian để giảm capacity gap.
34	`34_2025_distillm2.md`	DistiLLM-2	Loss contrastive giữa output của teacher và student.

Giai đoạn 7: Distill bằng dữ liệu và distill reasoning

#	File	Bài	Ý chính
35	`35_2023_self_instruct.md`	Self-Instruct	Model tự sinh dữ liệu instruction: KD ở mức dữ liệu.
36	`36_2023_distilling_step_by_step.md`	Distilling Step-by-Step	Distill cả rationale; model nhỏ vượt model lớn.
37	`37_2023_orca.md`	Orca	Học từ explanation trace của GPT-4.
38	`38_2025_deepseek_r1_distillation.md`	DeepSeek-R1	Distill reasoning vào model dense nhỏ; so distill với RL.
39	`39_2025_small_models_strong_reasoners.md`	Small Models Struggle to Learn from Strong Reasoners	Model nhỏ học từ teacher nhỏ hơn lại tốt hơn.

Giai đoạn 8: Quy mô, teacher và thực hành

#	File	Bài	Ý chính
40	`40_2024_gemma2_distillation.md`	Gemma 2	KD lúc pretraining từ teacher lớn hơn.
41	`41_2023_law_of_capacity_gap.md`	Law of Capacity Gap	Kích thước teacher tối ưu tăng theo kích thước student.
42	`42_2025_distillation_scaling_laws.md`	Distillation Scaling Laws	Khi nào distill hơn supervised; khi nào teacher lớn gây hại.
43	`43_2025_miniplm.md`	MiniPLM	KD pretraining bằng cách teacher "định hình" lại dữ liệu offline.
44	`44_2025_qwen3_distillation.md`	Qwen3	Strong-to-weak distillation: off-policy rồi on-policy; so chi phí với RL.
45	`45_2025_sparse_logit_sampling.md`	Sparse Logit Sampling	Top-k logit đã renormalise bị bias; lấy mẫu ngẫu nhiên thì không.
46	`46_2023_distil_whisper.md`	Distil-Whisper	Chưng cất nhận dạng tiếng nói: pseudo-label, lọc theo WER, transcription dài.
47	`47_2026_shortopd.md`	ShortOPD	On-policy distillation để phục hồi LLM đã prune.

Cách đọc gợi ý

- **Người mới:** 00 → 01–04 → 06, 08, 10 → 15 → 18 → 28, 29 → 42.

Liên kết với các thư mục khác

- `../pruning content/`: cách cắt model trước khi distill.
- `../interp content/`: interpretability, để hiểu distill phục hồi được gì bên trong model.
- `../rl content/`: reinforcement learning; on-policy distillation (28, 29, 47) dùng cùng bộ máy với RL (xem rl 00, 17, 39, 52).

Nền tảng 01

Nhãn mềm và nhiệt độ: học từ cả phân phối của thầy, không chỉ từ đáp án đúng

Loại: chương nền tảng (không phải ghi chú paper)

Đọc trước: logit, softmax và cross-entropy ở ML cơ bản nền tảng 02, 05; KL divergence ở math nền tảng 11; đạo hàm và gradient ở math nền tảng 06

Dùng ở: distillation 01 (khớp logit), distillation 02 (công thức chuẩn với nhiệt độ), distillation 05 (bản đồ phân loại) và mọi bài logit distillation về sau (06, 08, 14, 16, 31 và các bài chưng cất mô hình ngôn ngữ)

> Chương này giải thích từ đầu, không phải bản dịch một paper. Các phép tính ghi "tự tính" đã kiểm bằng numpy; ba hình đều tự sinh từ công thức.

1. Vì sao cần nhãn mềm và nhiệt độ

Nhãn mềm (soft label) là phân phối xác suất mà một mô hình đã học xong (thầy) gán cho các lớp, dùng làm mục tiêu để dạy mô hình nhỏ hơn (trò). Thay vì bảo trò "đáp án là chó, ngoài ra không có gì", ta bảo "đây là chó, hơi giống mèo, gần như không giống xe tải". Nhiệt độ T là nút vặn giúp ta nhìn rõ phần "hơi giống mèo" đó, vì ở $T = 1$ phần ấy thường quá nhỏ để trò cảm nhận được.

Chương này trả lời năm câu hỏi liên tiếp: softmax có nhiệt độ làm gì; vì sao xác suất nhỏ của các lớp sai lại chứa thông tin; loss distillation là gì và đo khoảng cách bằng KL ra sao; vì sao gradient bị co lại $1/T^2$ và vì sao ta nhân bù T^2; và cuối cùng, nó nối với các bài paper của cuốn này như thế nào.

2. Nhắc nền: logit, softmax, cross-entropy, KL

2.1 Logit và softmax

Mạng phân loại K lớp cho ra một vector K số thực $z = (z_1, \dots, z_K)$ gọi là **logit**, chưa phải xác suất. Softmax biến nó thành phân phối:

$$q_i = \frac{e^{z_i}}{\sum_{j=1}^{K} e^{z_j}}.$$

Ví dụ (tự tính): logit $(2, 0)$ cho $q = (0{,}8808;\ 0{,}1192)$ vì $e^2 = 7{,}389$, $e^0 = 1$, tổng $8{,}389$. Hai tính chất cần nhớ: cộng thêm cùng một hằng vào mọi logit không đổi kết quả (chỉ hiệu giữa các logit mới quan trọng), và hiệu logit càng lớn thì phân phối càng nhọn.

2.2 Cross-entropy với nhãn cứng

Nhãn cứng (hard label) là one-hot y: $y_i = 1$ ở đáp án đúng, còn lại bằng 0. Cross-entropy $\mathrm{CE}(y, q) = -\sum_i y_i \log q_i = -\log q_{\text{đúng}}$. Gradient của nó theo logit là $q - y$ (math nền tảng 07 và ML cơ bản nền tảng 06 đã dẫn). Lưu ý: tối thiểu hoá CE chỉ quan tâm $q_{\text{đúng}}$ lớn lên, hoàn toàn không phân biệt trò đặt 9% hay 0,1% cho "xe tải" khi nhãn là "chó".

2.3 KL divergence

Với hai phân phối p và q trên cùng K lớp:

$$\mathrm{KL}(p \,\|\, q) = \sum_i p_i \log \frac{p_i}{q_i} \;\geq 0,$$

bằng 0 khi và chỉ khi $p = q$. Nó **không đối xứng**: trong chưng cất, p là thầy (đích), q là trò (đang học), và ta tối thiểu hoá $\mathrm{KL}(p \,\|\, q)$ (forward KL). Ví dụ nhỏ (tự tính): $p = (0{,}7; 0{,}3)$, $q = (0{,}5; 0{,}5)$ cho KL $= 0{,}7 \ln 1{,}4 + 0{,}3 \ln 0{,}6 = 0{,}0823$ nat.

Một liên hệ quan trọng: $\mathrm{KL}(p\|q) = H(p, q) - H(p)$, trong đó $H(p, q)$ là cross-entropy và $H(p)$ là entropy của p. Vì thầy đứng yên, $H(p)$ là hằng số theo tham số trò, nên **tối thiểu hoá KL tương đương tối thiểu hoá cross-entropy với nhãn mềm**. Nhãn cứng chính là trường hợp p là one-hot (khi đó $H(p) = 0$ và KL trùng CE).

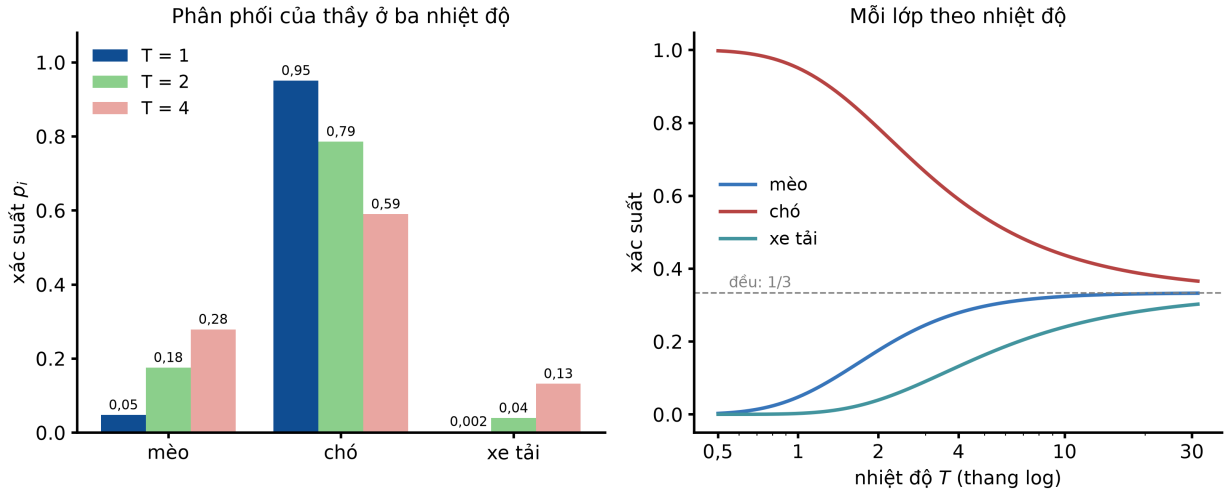
3. Trực giác bằng hình: nhiệt độ làm phẳng phân phối

Ta dùng một ví dụ xuyên suốt cả chương: ba lớp (mèo, chó, xe tải), ảnh là một con chó. Logit của thầy $z^T = (3, 6, 0)$, logit của trò $z^S = (2, 3, 1)$. Đây là số chọn cho đẹp, không phải đo từ mạng thật.

Softmax có nhiệt độ chia logit cho T trước khi qua softmax:

$$q_i^{(T)} = \frac{e^{z_i/T}}{\sum_j e^{z_j/T}}.$$

$T = 1$ là softmax bình thường; $T > 1$ làm phẳng; $T \to \infty$ cho phân phối đều $1/K$; $T \to 0^+$ tiến tới one-hot (argmax).



Hình 1: Phân phối của thầy (logit 3, 6, 0) ở các nhiệt độ T (minh hoạ, tự tính)

Cách đọc hình: bên trái, mỗi nhóm cột là một lớp, ba màu là ba nhiệt độ. Ở $T = 1$ (xanh đậm) thầy gần như chắc chắn "chó" (0,95) và cho "xe tải" chỉ 0,002; khi T lên 4 (hồng) đáp án "chó" xuống 0,59 còn "xe tải" lên 0,13. Bên phải là cùng thông tin với T liên tục (thang log): cả ba đường hội tụ dần về 1/3 khi T rất lớn, và "mèo" luôn cao hơn "xe tải" ở mọi T.

Điểm đáng chú ý nhất là **thứ tự và tỉ lệ giữa các lớp sai**. Ở $T = 1$, xác suất của mèo gấp khoảng 20 lần xe tải (0,0473 so với 0,0024, tự tính); ở $T = 4$ tỉ lệ này là 2,1 lần. Thứ tự "chó, rồi mèo, rồi xe tải" không đổi, nhưng ở $T = 1$ hai lớp sai bị nén sát 0 nên rất khó để một bộ tối ưu hoá nhìn ra khác biệt, còn ở $T = 4$ chúng đủ lớn để đóng góp vào loss.

4. Vì sao nhãn mềm chứa "tri thức tối"

Nhãn cứng nói: "chó = 1, mèo = 0, xe tải = 0". Nó coi mèo và xe tải sai **như nhau**. Nhãn mềm của thầy nói thêm rằng con chó này giống mèo hơn xe tải. Hinton et al. gọi phần thông tin ẩn trong xác suất nhỏ của các lớp sai là **dark knowledge** (tri thức tối); chính bài 02 trong cuốn này dùng ví dụ một chiếc xe BMW dễ bị nhầm với xe tải rác hơn rất nhiều so với củ cà rốt (theo ghi chú distillation 02).

Có thể nhìn bằng số. Entropy của phân phối thầy (tự tính, đơn vị nat): $0{,}207$ ở $T = 1$, $0{,}622$ ở $T = 2$, $0{,}934$ ở $T = 4$; mức tối đa với 3 lớp là $\ln 3 = 1{,}099$. Entropy một one-hot bằng 0. Phân phối càng "mềm", mỗi mẫu mang càng nhiều thông tin về quan hệ giữa các lớp, nhưng nếu quá mềm (T rất lớn) mọi lớp tiến về bằng nhau và thông tin lại mất. Đây là lý do T là siêu tham số cần chọn, thường nằm ở cỡ vài đơn vị chứ không cực đoan (đây là nhận xét chung, phải thử trên bài toán cụ thể).

Ba cách hiểu bổ sung, chưa phải kết luận chắc chắn và được các bài 09, 10 trong cuốn này bàn tiếp:

- **Thông tin mỗi mẫu:** một nhãn mềm mang nhiều bit hơn nhãn cứng nên trò học nhanh hơn từ ít dữ liệu.
- **Regularizer:** mục tiêu mềm ngăn trò quá tự tin, gần với label smoothing.
- **Thầy sạch hơn nhãn:** thầy có thể làm mượt các ví dụ nhãn nhiễu hoặc mơ hồ.

5. Loss distillation

5.1 Định nghĩa chính thức

Gọi z^T, z^S là logit của thầy và trò; $p^{(T)} = \text{softmax}(z^T/T)$ và $q^{(T)} = \text{softmax}(z^S/T)$. Loss chưng cất tiêu chuẩn (Hinton et al.) là

$$L = \alpha\, T^2\, \mathrm{KL}\Big(p^{(T)} \,\|\, q^{(T)}\Big) + (1-\alpha)\, \mathrm{CE}\Big(y,\, q^{(1)}\Big)\,,$$

với $\alpha \in [0, 1]$ cân bằng hai nguồn tín hiệu: **loss mềm** (khớp thầy ở nhiệt độ T) và **loss cứng** (khớp nhãn thật ở $T = 1$). Các điểm cần ghi nhớ:

- Cả thầy và trò dùng **cùng** T ở phần mềm. Khi suy luận (inference) trò quay về $T = 1$.
- Thầy đóng băng: gradient chỉ chảy vào tham số trò.
- Có nơi viết α cho phần cứng và $1 - \alpha$ cho phần mềm, hoặc dùng hai hệ số tự do độc lập. Khi đọc paper cần kiểm tra quy ước.
- Một số bài dùng cross-entropy mềm $H(p^{(T)}, q^{(T)})$ thay KL. Vì $H(p)$ không phụ thuộc trò, gradient giống nhau (mục 2.3).

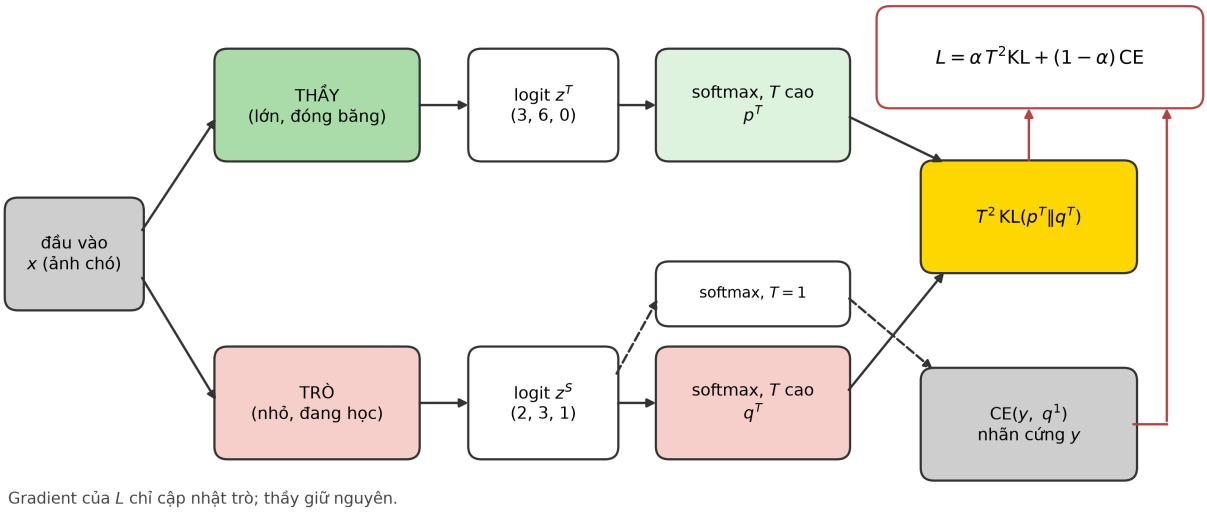
Cách đọc hình: đi từ trái sang phải. Cùng một đầu vào đi qua hai nhánh: thầy (xanh, đóng băng) và trò (hồng, đang học). Logit của cả hai qua softmax nhiệt độ cao rồi đi vào khối KL nhân T^2 (vàng). Một nhánh riêng của trò, softmax ở $T = 1$ (nét đứt), đi vào CE với nhãn cứng. Hai loss cộng lại với trọng số α và $1 - \alpha$ (khung đỏ). Dùng chung hình này cho mọi logit distillation: các bài sau chỉ thay "KL" bằng một loss khác hoặc thêm điểm khớp ở lớp giữa (chương nền 02).

5.2 Gradient và hệ số T^2

Đạo hàm của $\mathrm{KL}(p^{(T)} \| q^{(T)})$ theo logit trò z_i^S là

$$\frac{\partial\, \mathrm{KL}}{\partial z_i^S} = \frac{1}{T}\left(q_i^{(T)} - p_i^{(T)}\right).$$

Đây là dạng quen thuộc "dự đoán trừ mục tiêu" của softmax với cross-entropy, nhưng có thêm thừa số $1/T$ từ phép chia logit. Khi T lớn, $q_i^{(T)} - p_i^{(T)}$ bản thân cũng nhỏ cỡ $1/T$ (vì phân phối gần đều). Xấp xỉ khai triển bậc nhất: nếu T đủ lớn so với độ lớn logit, và logit đã trừ trung bình, thì

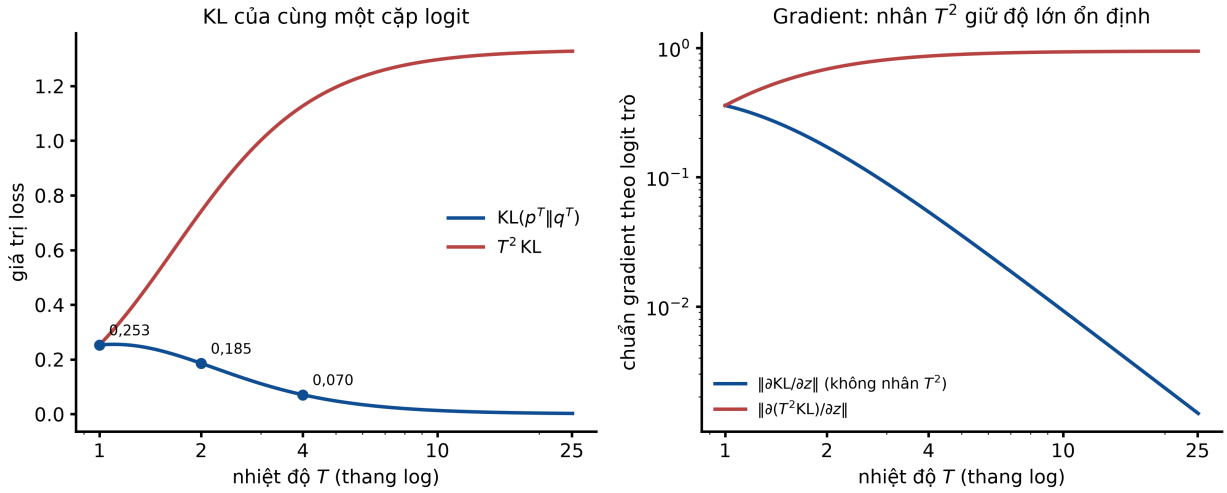


Hình 2: Sơ đồ thầy-trò và loss distillation (minh hoạ, tự tính)

$$\frac{\partial\,\mathrm{KL}}{\partial z_i^S} \approx \frac{1}{K\,T^2}\left(z_i^S - z_i^T\right).$$

(Đây là suy luận để lấy trực giác; ví dụ số ở mục 6 kiểm chứng độ lớn.) Hai hệ quả:

1. **Khi $T \to \infty$, KL tiến tới khớp logit trực tiếp:** gradient tỉ lệ với $z^S - z^T$, tức hồi quy bình phương tối thiểu trên logit, đúng là cách của Ba & Caruana và Buciluǎ et al. (theo ghi chú distillation 02, "khớp logit là trường hợp đặc biệt" của cách Hinton). Ở T vừa phải, KL nằm giữa: đủ mềm để lộ các lớp sai nhưng vẫn không bỏ qua hoàn toàn các logit rất âm.
2. **Gradient co lại $1/T^2$.** Nếu cộng loss mềm với loss cứng mà không bù, khi đổi T ta vô tình đổi cả tỉ trọng giữa hai loss. Nhân T^2 vào loss mềm giữ cho độ lớn gradient của hai phần xấp xỉ cùng cỡ khi ta thử nhiều T, nên α có ý nghĩa ổn định hơn.



Hình 3: KL và gradient theo nhiệt độ cho cùng một cặp logit (minh hoạ, tự tính)

Cách đọc hình: bên trái, đường xanh là KL giữa thầy và trò (logit $(3, 6, 0)$ và $(2, 3, 1)$) khi tăng T: nó giảm dần về 0 vì cả hai phân phối đều dần về phân phối đều. Đường đỏ là KL nhân T^2 và đi lên rồi phẳng ra. Bên

phải (thang log hai trục), chuẩn gradient không nhân T^2 (xanh) rơi gần như theo luật $1/T^2$, còn có nhân T^2 (đỏ) tiến tới một mức ổn định (khoảng 0,94 ở $T = 25$, tự tính). Hình nói về một cặp logit cố định, không nói về quá trình huấn luyện.

Cảnh báo: hệ số T^2 là quy ước để cân bằng độ lớn; nó **không** chứng minh T^2 là lựa chọn tối ưu. Một số bài về mô hình ngôn ngữ bỏ hoặc đổi nó; cần đọc từng bài.

5.3 Chọn T và α trong thực hành

Không có công thức chọn T và α đúng cho mọi bài toán; đây là kinh nghiệm chung, cần thử trên bài toán của bạn (không phải số liệu paper).

- **Nhiệt độ T:** thử một lưới nhỏ như $\{1, 2, 4, 8\}$ trên tập kiểm định. T quá nhỏ thì loss mềm gần như nhãn cứng, quá lớn thì chỉ còn khớp logit thô và mất tính chọn lọc.
- **Trọng số α:** khi dữ liệu có nhãn đáng tin, thường giữ một phần loss cứng. Khi thầy mạnh hơn hẳn nhãn (dữ liệu nhiễu), có thể đẩy α cao.
- **Số lớp K lớn:** với từ vựng hàng chục nghìn token, nhiệt độ cao làm đuôi xác suất đóng góp đáng kể vào KL; cần cẩn trọng về chi phí và độ ổn định (các bài 31, 45 trong cuốn).
- **Độ nhạy:** hai mô hình trò giống hệt nhau nhưng T khác nhau có thể cho kết quả khác một cách có ý nghĩa; hãy báo cáo T và α khi so sánh.

6. Ví dụ tay: logit ba lớp, $T = 1, 2, 4$, KL và gradient

Dữ liệu (số chọn cho đẹp): thầy $z^T = (3, 6, 0)$, trò $z^S = (2, 3, 1)$, thứ tự (mèo, chó, xe tải), nhãn thật là chó: $y = (0, 1, 0)$.

Bước 1: softmax của thầy. Ở $T = 1$: $e^3 = 20{,}09$, $e^6 = 403{,}4$, $e^0 = 1$, tổng $424{,}5$ nên $p^{(1)} = (0{,}0473;\ 0{,}9503;\ 0{,}0024)$ (tự tính). Ở $T = 2$ chia logit cho 2: $(1{,}5; 3; 0)$ nên $p^{(2)} = (0{,}1753;\ 0{,}7856;\ 0{,}0391)$. Ở $T = 4$: $(0{,}75; 1{,}5; 0)$ nên $p^{(4)} = (0{,}2786;\ 0{,}5898;\ 0{,}1316)$ (tự tính).

Bước 2: softmax của trò. $q^{(1)} = (0{,}2447;\ 0{,}6652;\ 0{,}0900)$; $q^{(2)} = (0{,}3072;\ 0{,}5065;\ 0{,}1863)$; $q^{(4)} = (0{,}3265;\ 0{,}4192;\ 0{,}2543)$ (tự tính). Trò ít tự tin hơn thầy ở mọi T, và đặc biệt đặt tới 9% cho xe tải ở $T = 1$ trong khi thầy chỉ cho 0,24%.

Bước 3: KL. Ví dụ $T = 1$: $\mathrm{KL} = 0{,}0473 \ln \frac{0{,}0473}{0{,}2447} + 0{,}9503 \ln \frac{0{,}9503}{0{,}6652} + 0{,}0024 \ln \frac{0{,}0024}{0{,}0900} = -0{,}0777 + 0{,}3390 - 0{,}0087 \approx 0{,}2526$ (tự tính, làm tròn trung gian). Đầy đủ:

T	KL	$T^2 \cdot$KL	$\partial \mathrm{KL}/\partial z^S$ (mèo, chó, xe tải)	$T\,(q - p) = \partial(T^2 \mathrm{KL})/\partial z^S$
1	0,2526	0,2526	(0,197; −0,285; 0,088)	(0,197; −0,285; 0,088)
2	0,1854	0,7418	(0,066; −0,140; 0,074)	(0,264; −0,558; 0,294)
4	0,0705	1,1274	(0,012; −0,043; 0,031)	(0,192; −0,682; 0,491)

(tự tính; làm tròn 3 chữ số). Hai cột gradient cho thấy điều đã nói ở mục 5.2: khi tăng T từ 1 lên 4, gradient thô giảm từ cỡ 0,29 xuống cỡ 0,04, còn gradient sau khi nhân T^2 vẫn cỡ 0,2 đến 0,7. Dấu của gradient mô tả rất tự nhiên: gradient âm ở "chó" nghĩa là đi theo hướng ngược gradient sẽ tăng logit chó của trò; gradient dương ở "xe tải" nghĩa là hạ logit xe tải. Ở $T = 4$ phần xe tải (0,491) lớn hơn nhiều so với ở $T = 1$ (0,088): nhiệt độ cao làm lớp sai có tiếng nói.

Kiểm tra xấp xỉ khi T lớn: với $T = 100$, $T(q - p)$ tính trực tiếp là $(0{,}009; -0{,}671; 0{,}662)$ (tự tính), rất gần giá trị dự đoán $(z^S - z^T)/K$ sau khi trừ trung bình logit, $(0; -0{,}667; 0{,}667)$. Ở $T = 4$ xấp xỉ còn thô, đừng dùng nó cho tính toán.

Bước 4: loss tổng với $\alpha = 0{,}5$, $T = 4$. Loss cứng $\mathrm{CE}(y, q^{(1)}) = -\ln 0{,}6652 = 0{,}4076$. Loss mềm sau nhân bù là $1{,}1274$. Vậy

$$L = 0{,}5 \cdot 1{,}1274 + 0{,}5 \cdot 0{,}4076 = 0{,}7675 \text{ (tự tính).}$$

Bước 5: gradient tổng theo z^S. Phần mềm: $0{,}5 \cdot T(q^{(4)} - p^{(4)}) = (0{,}096; -0{,}341; 0{,}245)$. Phần cứng: $0{,}5 \cdot (q^{(1)} - y) = 0{,}5 \cdot (0{,}2447; -0{,}3348; 0{,}0900) = (0{,}122; -0{,}167; 0{,}045)$. Cộng lại: $(0{,}218;\ -0{,}509;\ 0{,}290)$ (tự tính; đã kiểm bằng sai phân hữu hạn). Tổng ba thành phần xấp xỉ 0 (vì softmax không đổi khi cộng hằng vào mọi logit), điều này là phép kiểm nhanh khi tự tính.

7. Biến thể và chi tiết hay bị bỏ qua

- **Nhiệt độ ở thầy và trò có thể khác nhau,** và một số bài cho thầy "nóng" hơn trò hoặc dùng nhiệt độ thay đổi theo mẫu. Công thức chuẩn dùng chung T.
- **Thầy phải được hiệu chỉnh.** Nếu thầy quá tự tin (xác suất sát one-hot) thì nhãn mềm của $T = 1$ không mang thêm thông tin; nhiệt độ cao có thể cứu một phần, nhưng đây là một lý do khiến chất lượng thầy (không chỉ độ chính xác của nó) ảnh hưởng đến trò, câu hỏi được bàn ở các bài 09, 10 và chương nền 03.
- **Bỏ nhãn cứng ($\alpha = 1$)** vẫn dùng được khi dữ liệu không nhãn (transfer set). Khi dữ liệu có nhãn, thêm loss cứng thường giúp, nhưng mức giúp phụ thuộc bài toán.
- **Thay KL bằng loss khác** (L2 trên logit, KL ngược, Jensen-Shannon, nhiều biến thể cho mô hình ngôn ngữ) sẽ đổi hành vi, đặc biệt trên đuôi xác suất; chi tiết ở các bài 14, 31, 33.
- **Mô hình ngôn ngữ:** "lớp" là token trong từ vựng (hàng chục nghìn), KL được tính ở từng vị trí chuỗi và lấy trung bình. Công thức giữ nguyên, nhưng bài toán đuôi, bộ nhớ và độ lệch giữa huấn luyện và sinh khác hẳn (suy luận của chương, các bài 28 đến 34 đi sâu).
- **Ổn định số:** tính softmax bằng cách trừ max logit và dùng log-softmax để tính KL, kẻo $\log 0$ cho NaN (math nền tảng 12).

8. Nối sang các bài paper của cuốn

- **Bài 01 (Ba & Caruana):** khớp logit bằng L2 là giới hạn $T \to \infty$ của KL; mục 5.2 giải thích vì sao hai cách có liên hệ chặt chẽ chứ không chỉ là "đồ nghề khác nhau".
- **Bài 02 (Hinton et al.):** công thức mục 5.1 và hệ số T^2 đều xuất phát từ bài này; đọc bài với ví dụ số ở đây trong tay sẽ thấy dễ hơn.
- **Bài 05 (survey):** logit distillation là nhánh "response-based" trong bản đồ phân loại; chương nền 02 tiếp tục với nhánh "feature-based" và "relation-based".
- **Bài 06 (born-again networks):** bài phân rã gradient của KD thành phần "dark knowledge" và phần "trọng số hoá mẫu"; công thức $q - p$ ở mục 5.2 chính là nền cho phép phân rã đó.
- **Bài 14 (decoupled KD) và các bài về KL cho LLM (31, 33):** đều mổ xẻ chính các thành phần của KL ở mục 5; chương nền 02 và 03 sẽ giới thiệu ý tưởng, chương này đã cho bạn công cụ đọc gradient.

9. Sai lầm hay gặp

- **Quên bù T^2** hoặc bù hai lần khi lấy code từ nhiều nguồn; kiểm bằng cách đổi T và xem độ lớn loss mềm có đổi nghìn lần hay không.
- **Dùng T lúc suy luận.** Chỉ dùng $T > 1$ lúc huấn luyện phần mềm; khi đánh giá, trò chạy ở $T = 1$.
- **Cho gradient chảy vào thầy** (quên `detach` hoặc quên đặt `eval`), làm thầy bị kéo về phía trò.
- **Đảo chiều KL** (tính $\mathrm{KL}(q\|p)$ trong khi tài liệu nói $\mathrm{KL}(p\|q)$): hành vi ở đuôi khác nhau.
- **Tính softmax rồi lấy log** thay vì log-softmax, mất độ chính xác với xác suất rất nhỏ (đúng chỗ ta quan tâm nhất).
- **Nghĩ nhãn mềm luôn tốt hơn nhãn cứng.** Với thầy kém hoặc quá tự tin, nhãn mềm có thể không giúp; đó là câu hỏi thực nghiệm.

10. Tóm tắt công thức

Khái niệm	Công thức
Softmax nhiệt độ	$q_i^{(T)} = e^{z_i/T} \big/ \sum_j e^{z_j/T}$
KL thầy đến trò	$\mathrm{KL}(p\|q) = \sum_i p_i \log(p_i/q_i)$
Quan hệ KL và CE	$\mathrm{KL}(p\|q) = H(p,q) - H(p)$
Gradient KL theo logit trò	$\frac{1}{T}\left(q_i^{(T)} - p_i^{(T)}\right)$
Xấp xỉ khi T lớn	$\frac{1}{KT^2}(z_i^S - z_i^T)$ (logit trừ trung bình)
Loss distillation	$\alpha T^2 \mathrm{KL}(p^{(T)}\|q^{(T)}) + (1-\alpha)\mathrm{CE}(y, q^{(1)})$
Gradient của loss mềm đã nhân bù	$T\left(q_i^{(T)} - p_i^{(T)}\right)$

11. Bài tập tự tính

Bài 1. Logit $(2, 0)$. Tính softmax ở $T = 1$ và $T = 2$.

Đáp án (đã kiểm numpy): $T = 1$: $(0{,}8808;\ 0{,}1192)$. $T = 2$ tức logit $(1, 0)$: $(0{,}7311;\ 0{,}2689)$. Nhiệt độ cao làm phẳng.

Bài 2. Logit $(4, 1, 0)$. Tỉ lệ p_1/p_3 ở $T = 1$ và $T = 4$ là bao nhiêu? Tính cả hai phân phối.

Đáp án: tỉ lệ bằng $e^{(4-0)/T}$: $e^4 = 54{,}6$ ở $T = 1$ và $e^1 = 2{,}718$ ở $T = 4$. Phân phối: $T = 1$ là $(0{,}9362;\ 0{,}0466;\ 0{,}0171)$; $T = 4$ là $(0{,}5434;\ 0{,}2567;\ 0{,}1999)$.

Bài 3. $p = (0{,}7; 0{,}3)$, $q = (0{,}5; 0{,}5)$. Tính $\mathrm{KL}(p\|q)$ và $\mathrm{KL}(q\|p)$ (nat).

Đáp án: $\mathrm{KL}(p\|q) = 0{,}0823$; $\mathrm{KL}(q\|p) = 0{,}5 \ln \frac{0{,}5}{0{,}7} + 0{,}5 \ln \frac{0{,}5}{0{,}3} = 0{,}0872$. Hai chiều khác nhau, dù ở ví dụ này chỉ khác chút.

Bài 4. Ở $T = 2$, $p^{(2)} = (0{,}55; 0{,}45)$ và $q^{(2)} = (0{,}40; 0{,}60)$. Tính gradient của $T^2\mathrm{KL}$ theo logit trò, và gradient không nhân T^2.

Đáp án: không nhân: $(q - p)/T = (-0{,}075;\ 0{,}075)$. Nhân T^2: $T(q - p) = (-0{,}30;\ 0{,}30)$. Hai thành phần đối nhau vì tổng bằng 0.

Bài 5 (khó). Dùng $z^T = (3, 6, 0)$, $z^S = (2, 3, 1)$, $y = (0, 1, 0)$, $T = 4$, $\alpha = 0{,}5$ ở mục 6. Nếu đặt $\alpha = 1$ thì gradient tổng theo z^S là gì, và nó có đưa thành phần "chó" của trò vào đúng hướng ngược lại với trường hợp $\alpha = 0$ không?

Đáp án: với $\alpha = 1$, gradient là $T(q^{(4)} - p^{(4)}) = (0{,}192; -0{,}682; 0{,}491)$ (tự tính); với $\alpha = 0$ là $q^{(1)} - y = (0{,}245; -0{,}335; 0{,}090)$. Thành phần chó âm ở cả hai: cùng hướng (tăng logit chó), không ngược. Khác biệt ở xe tải: phần mềm đẩy mạnh hơn ($0{,}491$ so với $0{,}090$) vì thầy cho xe tải xác suất nhỏ hơn trò nhiều ở mọi nhiệt độ.

Tài liệu tham khảo nền: Hinton, Vinyals, Dean, "Distilling the Knowledge in a Neural Network" (2015) và ghi chú distillation 02; Ba & Caruana (2014), ghi chú distillation 01; Gou et al., khảo sát KD (ghi chú distillation 05); preface của cuốn này (mục 3, toán cốt lõi); math nền tảng 11 (entropy, cross-entropy, KL).

Nền tảng 02

Chưng cất đặc trưng, attention và quan hệ: dạy trò không chỉ đáp án mà cả cách thầy nhìn

Loại: chương nền tảng (không phải ghi chú paper)

Đọc trước: nen01 của cuốn này (nhãn mềm, KL, loss distillation); vector, ma trận và phép chiếu ở math nền tảng 02; chuẩn và khoảng cách ở math nền tảng 13; mạng nhiều lớp và CNN ở ML cơ bản nền tảng 05, 07

Dùng ở: distillation 03 (FitNets), distillation 04 (attention transfer), distillation 12 (chưng cất quan hệ), distillation 13 (học tương phản), distillation 14 (decoupled KD)

> Chương này giải thích từ đầu, không phải bản dịch một paper. Các phép tính ghi "tự tính" đã kiểm bằng numpy; hình bản đồ attention là mô phỏng từ tensor tự sinh, không phải ảnh chụp từ mạng thật.

1. Vì sao chỉ khớp logit thì chưa đủ

Chưng cất đặc trưng là cho trò khớp với thầy ở những chỗ bên trong mạng, không chỉ ở đầu ra. Ở nen01 ta chỉ khớp logit. Logit là một bản tóm tắt rất ngắn: với bài toán 10 lớp, thầy chỉ truyền cho trò 10 con số mỗi ảnh, dù bên trong mạng có hàng nghìn kích hoạt đã kết tinh "cách thầy nhìn ảnh".

Ba lý do khiến người ta thêm tín hiệu ở giữa (đây là động cơ chung, lập luận từ các paper và ghi chú, không phải kết luận chắc):

- **Tín hiệu dày hơn.** Mỗi lớp giữa cho thêm một mục tiêu trực tiếp, gần đầu vào hơn nên gradient không phải đi hết cả mạng.
- **Tối ưu dễ hơn với mạng sâu và hẹp.** Đây là động cơ gốc của FitNets (theo ghi chú distillation 03): mạng sâu nhưng mảnh khó huấn luyện, lớp giữa của thầy đóng vai trò gợi ý (hint).
- **Cấu trúc mà logit không thấy.** Thầy "nhìn vào đâu" (attention), hay "xếp các mẫu gần xa ra sao" (quan hệ) là những tính chất không nằm trong 10 con số logit.

Chương đi qua năm ý tưởng theo thứ tự: FitNets (khớp đặc trưng qua phép chiếu), attention transfer (khớp bản đồ tóm tắt), relational KD (khớp khoảng cách giữa các mẫu), học tương phản CRD và decoupled KD (ý tưởng). Mỗi ý chỉ nói mức ý tưởng, chi tiết nằm ở các bài paper.

2. Nhắc nền: kích hoạt, ma trận chiếu, chuẩn hoá

2.1 Tensor kích hoạt và vector đặc trưng

Một lớp tích chập cho ra tensor kích hoạt $A \in \mathbb{R}^{C\times H\times W}$: C kênh, mỗi kênh là bản đồ $H \times W$. Một lớp cuối của mạng thường cho một **vector đặc trưng** $f \in \mathbb{R}^d$ cho mỗi ảnh (số chiều d do kiến trúc quyết định). Thầy và trò thường có d khác nhau: ví dụ thầy $d = 3$, trò $d = 2$ trong ví dụ đồ chơi dưới đây.

2.2 Ma trận chiếu giải quyết lệch số chiều

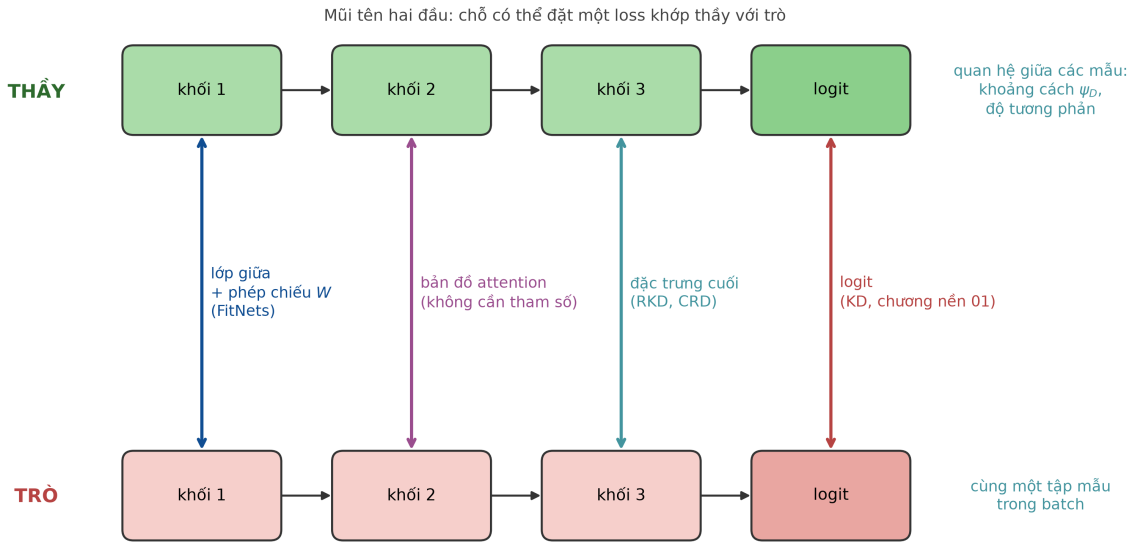
Không thể trừ trực tiếp vector 2 chiều cho vector 3 chiều. Ta thêm một ma trận học được $W \in \mathbb{R}^{3\times 2}$ (một phép biến đổi tuyến tính) để đưa đặc trưng trò lên 3 chiều: $Wf_S \in \mathbb{R}^3$, rồi so với f_T. Ma trận này gọi là **projector** hay **regressor** (tên trong FitNets), và chỉ dùng lúc huấn luyện; khi triển khai, trò bỏ nó đi.

2.3 Chuẩn L2 và chuẩn hoá

Chuẩn L2 của vector v là $\|v\|_2 = \sqrt{\sum_i v_i^2}$. Chia v cho chuẩn của nó cho vector có độ dài 1 (chỉ giữ "hình dạng", bỏ độ lớn). Ví dụ (tự tính): $v = (3, 4)$ có $\|v\|_2 = 5$ nên $v/\|v\|_2 = (0{,}6; 0{,}8)$. Ý này sẽ dùng ở attention

và quan hệ.

3. Trực giác bằng hình: ta có thể khớp ở những đâu



Hình 4: Các điểm có thể khớp giữa thầy và trò (minh hoạ, tự tính)

Cách đọc hình: hàng trên là thầy (xanh), hàng dưới là trò (hồng), mỗi hộp là một khối của mạng, ô cuối là logit. Mỗi mũi tên hai đầu là một chỗ có thể đặt một loss kéo hai bên về nhau. Cột phải nhất (logit) là KD của nen01. Cột trái (lớp giữa kèm phép chiếu W) là FitNets; cột thứ hai (bản đồ attention) là attention transfer; cột thứ ba (đặc trưng cuối) thường là chỗ đặt loss quan hệ hoặc tương phản. Các điểm khớp có thể dùng cùng lúc, và trong thực hành phần lớn phương pháp **cộng** loss ở giữa vào loss logit.

Chú ý rằng sơ đồ chỉ cho thấy "chỗ đặt loss", không nói loss nào tốt hơn. Cái nào hiệu quả phụ thuộc kiến trúc, dữ liệu và thầy.

4. FitNets: khớp lớp trung gian qua phép chiếu

4.1 Định nghĩa

Chọn một **lớp gợi ý** (hint) của thầy với đầu ra f_T và một **lớp được dẫn** (guided) của trò với đầu ra f_S. Loss gợi ý

$$L_{\text{hint}} = \frac{1}{2}\|W f_S - f_T\|_2^2,$$

trong đó W là phép chiếu học được (trong FitNets là một lớp nhỏ gắn lên lớp được dẫn). Theo ghi chú distillation 03, FitNets huấn luyện hai giai đoạn: trước hết dùng L_{hint} để khởi tạo phần đầu của trò, sau đó huấn luyện toàn bộ trò bằng KD. Nhiều phương pháp sau này cộng L_{hint} với loss logit trong một giai đoạn.

4.2 Ví dụ tay: khớp một vector 3 chiều qua W

Thầy $f_T = (2, 1, 3)$. Trò $f_S = (1, 2)$ (2 chiều). Phép chiếu cố định (cho dễ) là

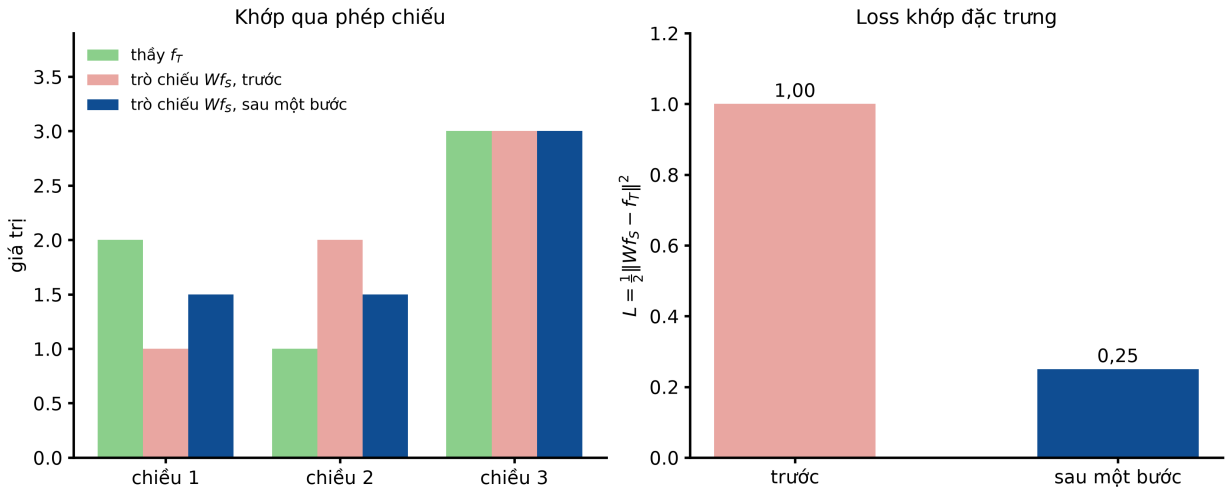
$$W = \begin{pmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \end{pmatrix}.$$

Bước 1: chiếu. $W f_S = (1,\ 2,\ 1+2) = (1, 2, 3)$.

Bước 2: sai lệch. $r = W f_S - f_T = (1-2,\ 2-1,\ 3-3) = (-1, 1, 0)$, nên $L_{\text{hint}} = \frac{1}{2}(1+1+0) = 1$ (tự tính).

Bước 3: gradient. Theo quy tắc chuỗi, $\partial L/\partial f_S = W^\top r$. $W^\top = \begin{pmatrix} 1 & 0 & 1 \\ 0 & 1 & 1 \end{pmatrix}$ nên $W^\top r = (-1,\ 1)$. Ngoài ra $\partial L/\partial W = r\, f_S^\top$ là ma trận 3×2 có hàng $(-1, -2)$, $(1, 2)$, $(0, 0)$ (tự tính).

Bước 4: một bước giảm gradient trên f_S với tốc độ $0{,}5$ (coi f_S như biến, đại diện cho các tham số phía trước): $f_S \leftarrow (1, 2) - 0{,}5(-1, 1) = (1{,}5; 1{,}5)$. Chiếu lại: $W f_S = (1{,}5; 1{,}5; 3)$, sai lệch $(-0{,}5; 0{,}5; 0)$, loss giảm từ 1 xuống $0{,}25$ (tự tính).



Hình 5: Khớp đặc trưng thầy ba chiều qua phép chiếu và loss trước, sau một bước (minh hoạ, tự tính)

Cách đọc hình: trái, ba cột mỗi chiều là thầy (xanh nhạt), trò chiếu trước bước (hồng), trò chiếu sau bước (xanh đậm). Chiều 3 đã khớp từ đầu; chiều 1 và 2 được kéo về phía thầy nhưng chưa tới (1,5 so với 2 và 1,5 so với 1). Phải: loss từ 1,00 xuống 0,25. Với W này, trò khớp hoàn toàn khi $f_S = (2, 1)$ (khi đó $W f_S = (2, 1, 3) = f_T$); các bước giảm gradient tiếp theo sẽ tiến về đó. Điều cần nhớ: **trò chỉ khớp được phần thầy nằm trong không gian ảnh của** W. Nếu f_T không có dạng $W f_S$ với mọi f_S (ví dụ $f_T = (2, 1, 5)$ với cùng W), loss không thể về 0: sai lệch tối thiểu còn lại là $\frac{1}{2}\|Wx^* - f_T\|^2 = 0{,}667$ tại $x^* = (2{,}667; 1{,}667)$ (tự tính, bình phương tối thiểu).

4.3 Chi tiết hay bị bỏ qua

- Phép chiếu có thể là lớp tuyến tính, tích chập 1×1 hay MLP nhỏ; chọn khác nhau cho kết quả khác nhau.
- Khớp **toàn bộ** tensor kích hoạt ép trò sao chép cả chi tiết vô ích. Chương sau (attention) giảm yêu cầu này.
- Chọn lớp gợi ý và lớp được dẫn là siêu tham số quan trọng; chọn lệch tầng có thể làm hại (đây là kinh nghiệm chung, cần thử).

5. Attention transfer: khớp bản tóm tắt thay vì toàn bộ tensor

5.1 Ý tưởng

Thay vì khớp tensor $C \times H \times W$ đầy đủ, ta tóm tắt nó thành một bản đồ $H \times W$ cho biết vùng nào của ảnh có kích hoạt mạnh, rồi khớp bản đồ ấy. Lưu ý: "attention" ở đây là bản đồ không gian của CNN, không phải self-attention của Transformer (theo ghi chú distillation 04).

5.2 Định nghĩa

Theo ghi chú distillation 04, bản đồ kích hoạt dạng $F^2_{\text{sum}}(A) = \sum_{c=1}^{C} |A_c|^2$ (tổng bình phương theo kênh). Gọi $Q = \text{vec}\big(F(A)\big)$ là bản đồ trải thành vector độ dài HW. Loss attention transfer cộng thêm vào cross-entropy

$$L_{\text{AT}} = \frac{\beta}{2} \sum_{j} \left\| \frac{Q_S^j}{\|Q_S^j\|_2} - \frac{Q_T^j}{\|Q_T^j\|_2} \right\|_p ,$$

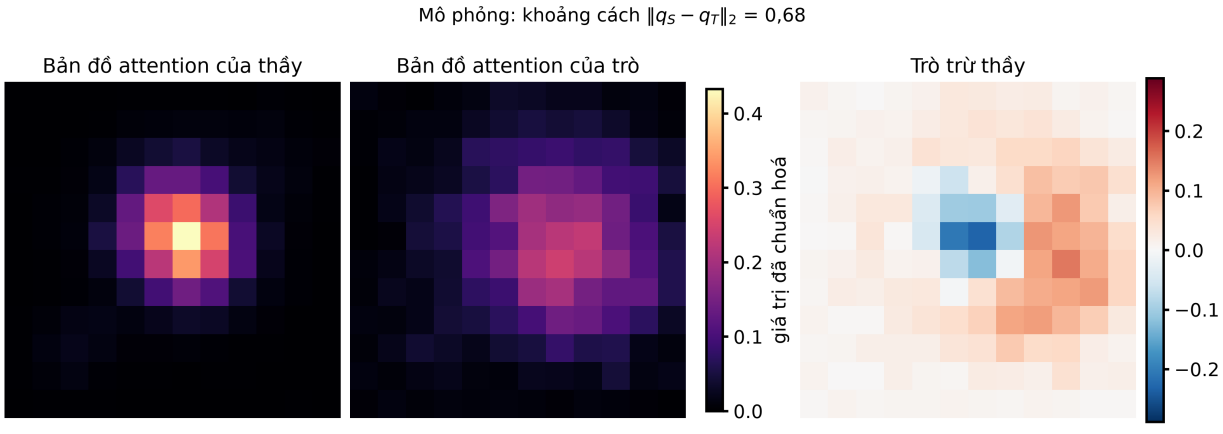
với j chạy qua các cặp lớp cùng độ phân giải, $p = 2$ trong các thí nghiệm của paper. Chuẩn hoá L2 từng bản đồ loại bỏ khác biệt về thang giá trị giữa thầy và trò; chỉ còn **hình dạng** (vùng nào nổi bật).

5.3 Ví dụ tay: hai bản đồ 2×2

Thầy có hai kênh 2×2: kênh 1 $= \begin{pmatrix} 1 & 2 \\ 0 & 0 \end{pmatrix}$, kênh 2 $= \begin{pmatrix} 1 & 0 \\ 2 & 0 \end{pmatrix}$. Tổng bình phương theo kênh, trải thành vector: $(1+1,\ 4+0,\ 0+4,\ 0+0) = (2, 4, 4, 0)$, chuẩn $\sqrt{4+16+16} = 6$, nên $q_T = (\frac{1}{3}, \frac{2}{3}, \frac{2}{3}, 0)$.

Trò có hai kênh giống nhau $\begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix}$: tổng bình phương $(2, 2, 2, 0)$, chuẩn $\sqrt{12} = 3{,}464$, $q_S = (0{,}577; 0{,}577; 0{,}577; 0)$.

Hiệu theo từng vị trí $q_S - q_T = (0{,}244;\ -0{,}089;\ -0{,}089;\ 0)$, và khoảng cách $\|q_S - q_T\|_2 = 0{,}275$ (tự tính). Trò dàn kích hoạt đều ba ô, thầy tập trung vào hai ô; loss phạt đúng sự khác nhau về "nơi nổi bật". Độ lớn tuyệt đối (thầy chuẩn 6, trò chuẩn 3,46) không hề ảnh hưởng.



Hình 6: Bản đồ attention mô phỏng của thầy và trò, và hiệu của chúng (minh hoạ, tự tính)

Cách đọc hình: dữ liệu là **mô phỏng**: tensor $6 \times 12 \times 12$ do numpy sinh (thầy là bốn vệt Gaussian hẹp quanh một "vật" ở giữa cộng hai vệt phụ, trò là các vệt rộng và lệch sang phải, cả hai có nhiễu nhỏ), sau đó tính $\sum_c A_c^2$ và chuẩn hoá L2. Trái và giữa là hai bản đồ cùng thang màu; phải là hiệu (trò trừ thầy): xanh là chỗ trò kém tập trung hơn, đỏ là chỗ trò lan rộng hơn. Khoảng cách mô phỏng là 0,68. Đây là hình để hiểu khái niệm, không phải ảnh chụp từ một mạng đã huấn luyện.

5.4 Chi tiết hay bị bỏ qua

- AT cần cặp lớp có độ phân giải không gian cùng cỡ (hoặc phải nội suy) nhưng **không cần** ma trận chiếu học được, vì số kênh C bị cộng mất đi.
- Cộng theo kênh giả định rằng giá trị kích hoạt lớn nghĩa là quan trọng (giả định ngầm của paper, chưa phải chân lý với mọi kiến trúc).
- AT thường dùng **cùng** KD logit; nó là bổ sung, không thay thế.

6. Chưng cất quan hệ: tri thức nằm ở khoảng cách giữa các mẫu

6.1 Ý tưởng

Nếu trò có đặc trưng 1 chiều còn thầy 512 chiều, buộc từng điểm trùng nhau là phi thực tế. Ý của relational KD (theo ghi chú distillation 12): giữ lại **hình học giữa các mẫu**, tức mẫu nào gần mẫu nào, nhóm ba mẫu tạo góc ra sao, chứ không phải tọa độ tuyệt đối.

Với cặp mẫu, thế quan hệ là khoảng cách chuẩn hoá bằng trung bình khoảng cách trong batch:

$$\psi_D(t_i, t_j) = \frac{1}{\mu}\|t_i - t_j\|_2, \qquad \mu = \text{trung bình của mọi } \|t_i - t_j\|_2.$$

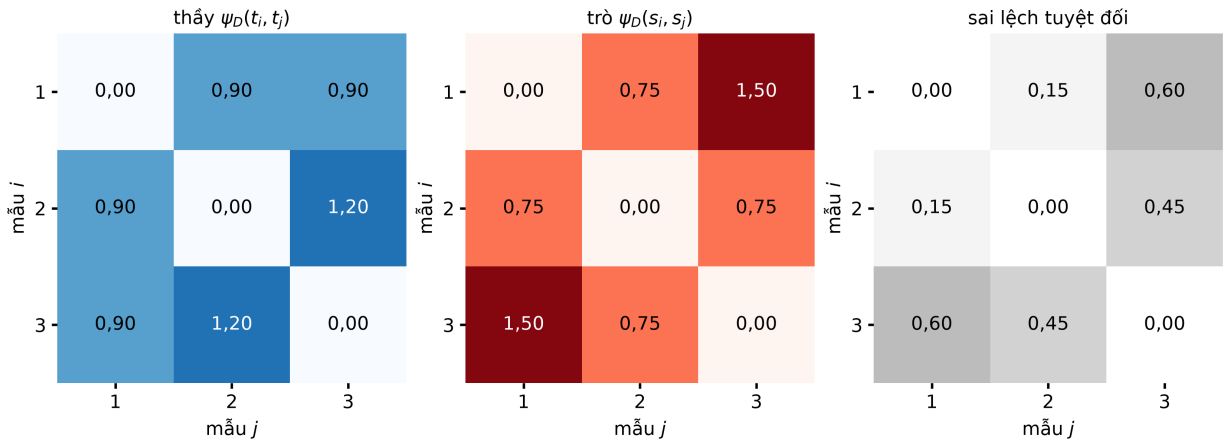
Loss RKD khoảng cách là tổng Huber của ψ_D thầy và ψ_D trò trên mọi cặp. (Huber là bình phương khi sai lệch nhỏ hơn 1, tuyến tính khi lớn hơn.)

6.2 Ví dụ tay: ba mẫu, thầy 3 chiều, trò 1 chiều

Thầy: $t_1 = (0, 0, 0)$, $t_2 = (2, 2, 1)$, $t_3 = (2, -2, 1)$. Khoảng cách: $d_{12} = \sqrt{4+4+1} = 3$, $d_{13} = 3$, $d_{23} = \sqrt{0+16+0} = 4$. Trung bình $\mu_T = (3+3+4)/3 = 3{,}333$, nên $\psi_D = (0{,}9;\ 0{,}9;\ 1{,}2)$ cho các cặp (12, 13, 23).

Trò chỉ có một chiều: $s_1 = 0$, $s_2 = 2$, $s_3 = 4$. Khoảng cách $d_{12} = 2$, $d_{13} = 4$, $d_{23} = 2$, trung bình $\mu_S = 2{,}667$, nên $\psi_D = (0{,}75;\ 1{,}5;\ 0{,}75)$.

Sai lệch từng cặp: $0{,}15$, $-0{,}6$, $0{,}45$. Với Huber $\delta = 1$ (đều trong vùng bình phương), mỗi cặp góp $\frac{1}{2}d^2$: $0{,}0113$; $0{,}18$; $0{,}1013$, tổng $0{,}2925$ và trung bình theo cặp $0{,}0975$ (tự tính). Trò đang đặt mẫu 1 và 3 quá xa nhau so với thầy (1,5 so với 0,9), nên loss lớn nhất ở cặp 13.



Hình 7: Ma trận khoảng cách chuẩn hoá của thầy, trò và sai lệch tuyệt đối giữa ba mẫu (minh hoạ, tự tính)

Cách đọc hình: mỗi ô là ψ_D giữa mẫu i và j (đường chéo bằng 0). Thầy (xanh) cho cặp 23 xa nhất (1,20), hai cặp còn lại bằng nhau (0,90). Trò (đỏ) lại cho cặp 13 xa nhất (1,50). Ma trận xám bên phải là sai lệch tuyệt

đối: ô sẫm nhất (0,60) là chỗ cần sửa. Chú ý trò khác chiều với thầy nhưng hai ma trận có cùng cỡ (3×3): đó là lý do khớp quan hệ không cần ma trận chiếu.

Đây là bản "một trò 1 chiều" hơi cực đoan để dễ tính; trong thực tế trò có nhiều chiều hơn, và có thể một số cặp không thể khớp được vì không gian chiều quá thấp (suy luận của chương).

6.3 Chi tiết hay bị bỏ qua

- RKD còn có thế góc giữa ba điểm (cosine), mạnh hơn nhưng tốn $O(n^3)$ trong batch (theo ghi chú distillation 12, paper nói thế bậc cao hơn mạnh hơn nhưng đắt hơn).
- Chuẩn hoá bằng μ làm loss bất biến với thang, nên trò không cần sao chép độ lớn khoảng cách.
- Quan hệ phụ thuộc cách chọn batch: batch toàn mẫu cùng lớp cho ít thông tin hơn batch trộn nhiều lớp.

7. Học tương phản (CRD) ở mức ý tưởng

Ý tưởng: với cùng một ảnh x_i, đặc trưng thầy và trò là một **cặp dương** (phải gần nhau); với hai ảnh khác nhau, chúng là **cặp âm** (phải xa nhau). Huấn luyện trò để một bộ chấm điểm phân biệt được cặp dương với cặp âm chính là tối đa hoá một cận dưới của thông tin tương hỗ giữa biểu diễn thầy và trò (theo ghi chú distillation 13). Khác RKD, CRD không so khoảng cách bằng số mà bằng **xếp hạng**: mẫu dương phải cao điểm hơn các mẫu âm.

Ví dụ tay nhỏ về cơ chế (số giả định): đặc trưng trò của ảnh 1 có độ tương đồng cosine với ba đặc trưng thầy là $0{,}9$ (đúng ảnh 1, dương), $0{,}2$ và $0{,}1$ (hai ảnh khác, âm). Với nhiệt độ $\tau = 0{,}5$, softmax trên sim/τ cho xác suất của cặp dương $0{,}6904$ (tự tính), loss $-\ln 0{,}6904 = 0{,}3705$. Nếu $\tau = 1$ xác suất chỉ $0{,}5139$ (loss $0{,}6657$); nếu $\tau = 0{,}1$ là $0{,}9988$ (loss $0{,}0012$). Nhiệt độ tương phản τ đóng vai trò giống T ở nen01: nhỏ thì nhọn.

Chi tiết (theo paper): CRD chạy ở tầng đặc trưng gần logit, dùng hai phép chiếu tuyến tính để thầy và trò có số chiều khác nhau, và dùng một "kho" (memory buffer) để có nhiều mẫu âm mà không tốn bộ nhớ batch. Chọn mẫu âm khác lớp có thể tránh đẩy xa các mẫu thuộc cùng lớp (ghi chú distillation 13 nêu kết quả này).

8. Decoupled KD ở mức ý tưởng

Ý tưởng: KL của nen01 gộp hai việc: (a) trò có đúng là lớp đích hay không (nhị phân), (b) trong các lớp **không** phải đích, thầy xếp thứ tự thế nào ("tri thức tối"). Theo ghi chú distillation 14:

$$\mathrm{KL}(p\|q) = \underbrace{\mathrm{KL}(b^T\|b^S)}_{\text{TCKD}} + (1-p_t^T)\,\underbrace{\mathrm{KL}(\hat{p}^T\|\hat{p}^S)}_{\text{NCKD}},$$

trong đó t là lớp đích, $b = (p_t,\, 1-p_t)$ là phân phối nhị phân "đích hay không", và $\hat{p}$ là phân phối thầy **chỉ trên các lớp không phải đích** (chuẩn hoá lại). Quan sát: NCKD bị nhân với $1-p_t^T$, nên khi thầy rất tự tin vào đúng lớp đích thì phần "tri thức tối" gần như bị tắt. DKD thay $1-p_t^T$ bằng hệ số độc lập β và thêm α cho TCKD, để cân chỉnh hai phần tự do.

Ví dụ tay, dùng ví dụ nen01 ($T = 1$, thầy $(0{,}0473;\ 0{,}9503;\ 0{,}0024)$, trò $(0{,}2447;\ 0{,}6652;\ 0{,}0900)$, đích là chó):

- TCKD: $b^T = (0{,}9503; 0{,}0497)$, $b^S = (0{,}6652; 0{,}3348)$, KL $= 0{,}2442$.
- NCKD: phân phối không-đích của thầy $\hat{p}^T = (0{,}9526; 0{,}0474)$, của trò $\hat{p}^S = (0{,}7311; 0{,}2689)$, KL $= 0{,}1698$.
- Hệ số $1-p_t^T = 0{,}0497$: phần NCKD thực sự đóng góp là $0{,}0497 \times 0{,}1698 = 0{,}0084$.
- Kiểm: $0{,}2442 + 0{,}0084 = 0{,}2526 = \mathrm{KL}(p\|q)$ ở nen01 (tự tính, khớp đến 4 chữ số).

Chỉ 0,0084 trên tổng 0,2526 (khoảng 3,3%) là phần "lớp sai". Ở $T = 4$ phần đó tăng lên: $1-p_t^T = 0{,}4102$, NCKD $= 0{,}0287$, đóng góp $0{,}0118$ trên tổng $0{,}0705$ (khoảng 17%, tự tính). Nhiệt độ cao thực sự mở "tiếng

nói" của lớp sai, đúng với trực giác ở nen01; DKD đi thêm một bước bằng cách tách hai phần.

9. Nối sang các bài paper của cuốn

- **Bài 03 (FitNets):** là mục 4; khi đọc phần "hai giai đoạn" hãy nhớ phép chiếu W chỉ dùng khi huấn luyện.
- **Bài 04 (attention transfer):** là mục 5; ví dụ 2×2 giúp đọc công thức F^p_{sum} và chuẩn hoá L2.
- **Bài 12 (relational KD):** là mục 6; ví dụ ba mẫu ở đây là phiên bản số khác với ví dụ trong ghi chú bài 12 (cùng ý, số khác).
- **Bài 13 (CRD):** là mục 7; chương này chỉ cho ý tưởng xếp hạng dương, âm, còn tối đa hoá thông tin tương hỗ và kho mẫu âm xem bài.
- **Bài 14 (DKD):** là mục 8; phân rã TCKD và NCKD kiểm lại bằng số ở nen01.

10. Sai lầm hay gặp

- **Không có phép chiếu khi số chiều lệch**, hoặc quên bỏ phép chiếu khi triển khai (nó không thuộc trò).
- **Khớp lớp giữa không cùng ngữ nghĩa** (ví dụ lớp rất nông của thầy với lớp sâu của trò), làm loss bị ép về chỗ vô nghĩa.
- **Cho AT nhận tensor chưa cùng độ phân giải** mà không nội suy.
- **Quên rằng loss giữa phải cân bằng với loss logit**: các hệ số β phải điều chỉnh, vì độ lớn khác nhau nhiều bậc.
- **Hiểu "relational" là so tọa độ**: nó so khoảng cách (hoặc góc) chuẩn hoá, bất biến với số chiều.
- **Nghĩ rằng càng nhiều điểm khớp càng tốt.** Nhiều phương pháp feature distillation cho lợi ích nhỏ so với KD logit tốt; cần ablation (suy luận của chương, bài 05 trong cuốn cho bản đồ so sánh).

11. Tóm tắt công thức

Phương pháp	Điểm khớp	Công thức chính
FitNets	lớp giữa qua W	$\frac{1}{2}\|W f_S - f_T\|_2^2$
Attention transfer	bản đồ $H \times W$	$\frac{\beta}{2}\|q_S - q_T\|_p$, $q = Q/\|Q\|_2$, $Q = \text{vec}\sum_c A_c^2$
Relational KD (khoảng cách)	giữa các mẫu	Huber của $\psi_D = \|t_i - t_j\|_2/\mu$
CRD	dương so với âm	softmax xếp hạng cặp dương trên sim/τ
Phân rã DKD	lớp đích và không đích	$\text{KL} = \text{TCKD} + (1 - p_t^T)\text{NCKD}$

12. Bài tập tự tính

Bài 1. $W = \begin{pmatrix} 1 & 0 \\ 0 & 2 \\ 1 & 1 \end{pmatrix}$, $f_S = (2, 1)$, $f_T = (1, 2, 4)$. Tính $W f_S$, sai lệch, $L = \frac{1}{2}\|W f_S - f_T\|^2$ và $\partial L/\partial f_S$.

Đáp án (đã kiểm numpy): $W f_S = (2, 2, 3)$; sai lệch $(1, 0, -1)$; $L = 1$; $\partial L/\partial f_S = W^\top r = (0, -1)$.

Bài 2. Thầy có $Q_T = (3, 4, 0)$ và trò có $Q_S = (0, 0, 5)$ (bản đồ đã trải thành vector). Tính khoảng cách AT với $p = 2$ sau chuẩn hoá.

Đáp án: $q_T = (0{,}6; 0{,}8; 0)$, $q_S = (0; 0; 1)$, khoảng cách $\sqrt{0{,}36 + 0{,}64 + 1} = \sqrt{2} = 1{,}414$: hai vector trực giao nên đạt xa nhất giữa hai vector đơn vị không âm.

Bài 3. Ba mẫu thầy nằm trên một đường thẳng cách đều theo tỉ lệ $(0, 0)$, $(3, 4)$, $(6, 8)$. Tính ψ_D của ba cặp.

Đáp án: khoảng cách $5, 5, 10$, trung bình $6{,}667$, $\psi_D = (0{,}75; 0{,}75; 1{,}5)$. Đây cũng chính là ma trận trò 1 chiều ở mục 6.2: một trò 1 chiều thẳng hàng $0, 2, 4$ sẽ có loss bằng 0 nếu thầy xếp thẳng hàng cách đều.

Bài 4. Cosine giữa đặc trưng trò và ba đặc trưng thầy (1 dương, 2 âm) là $(0{,}8; 0{,}4; 0)$, $\tau = 0{,}5$. Tính xác suất của cặp dương và loss.

Đáp án: $e^{1{,}6} = 4{,}953$, $e^{0{,}8} = 2{,}226$, $e^0 = 1$, tổng $8{,}179$; xác suất $0{,}6056$; loss $-\ln 0{,}6056 = 0{,}5015$.

Bài 5 (khó). Thầy $p = (0{,}6; 0{,}3; 0{,}1)$, trò $q = (0{,}4; 0{,}4; 0{,}2)$, lớp đích là lớp 1. Tính TCKD, NCKD, và kiểm $\mathrm{KL}(p\|q) = \mathrm{TCKD} + (1 - p_t)\mathrm{NCKD}$.

Đáp án: $b^T = (0{,}6; 0{,}4)$, $b^S = (0{,}4; 0{,}6)$, TCKD $= 0{,}0811$. Không-đích: $\hat{p}^T = (0{,}75; 0{,}25)$, $\hat{p}^S = (\frac{2}{3}, \frac{1}{3})$, NCKD $= 0{,}0164$. Tổng $0{,}0811 + 0{,}4 \times 0{,}0164 = 0{,}0877$, đúng bằng $\mathrm{KL}(p\|q) = 0{,}0877$.

Tài liệu tham khảo nền: Romero et al., FitNets (ghi chú distillation 03); Zagoruyko & Komodakis, attention transfer (ghi chú distillation 04); Park et al., Relational KD (ghi chú distillation 12); Tian et al., CRD (ghi chú distillation 13); Zhao et al., Decoupled KD (ghi chú distillation 14); khảo sát KD (ghi chú distillation 05); nen01 của cuốn này.

Nền tảng 03

Quan hệ thầy-trò: thầy lớn hơn chưa chắc dạy tốt hơn

Loại: chương nền tảng (không phải ghi chú paper)

Đọc trước: nen01 và nen02 của cuốn này (nhãn mềm, loss distillation, khớp đặc trưng); overfitting và bias-variance ở ML cơ bản nền tảng 03; hồi quy tuyến tính ở ML cơ bản nền tảng 02; xác suất và trung bình ở math nền tảng 08

Dùng ở: distillation 06 (born-again), distillation 07 (học tương hỗ), distillation 08 (trợ giảng), distillation 09, distillation 10, distillation 11 (vì sao KD hiệu quả hay không) và distillation 41 (quy luật khoảng cách năng lực)

> Chương này giải thích từ đầu, không phải bản dịch một paper. Các phép tính ghi "tự tính" đã kiểm bằng numpy. Hai hình đường cong và hàm số là **mô phỏng** hồi quy một chiều do mình tự sinh (seed cố định), không phải kết quả đo trên mạng thật; chương nói rõ chỗ nào là mô phỏng, chỗ nào là giả thuyết, chỗ nào lấy từ ghi chú paper.

1. Vì sao cần hiểu quan hệ thầy-trò

Chưng cất không chỉ là "thầy mạnh thì trò mạnh". Nếu thầy càng mạnh, trò càng giỏi thì chỉ cần luôn dùng thầy lớn nhất có thể. Thực tế, trong một số thí nghiệm, trò học từ thầy quá lớn lại kém hơn trò học từ thầy vừa phải. Hiện tượng này gọi là **khoảng cách năng lực** (capacity gap). Chương này dạy từ trực giác: vì sao nó có thể xảy ra, có những cách chữa nào (trợ giảng, tự chưng cất, nhiều thầy), và vì sao câu hỏi "tại sao chưng cất hiệu quả" vẫn còn để ngỏ.

Điều cần nói ngay từ đầu: các lời giải thích trong chương là **giả thuyết** được các bài paper đưa ra hoặc được suy ra ở đây, chưa phải kết luận chắc. Chương dùng ví dụ hồi quy một chiều nhỏ để làm trực giác, không để chứng minh cho mạng sâu.

2. Nhắc nền: dung lượng, xấp xỉ và nhiễu

2.1 Dung lượng (capacity)

Dung lượng của mô hình là mức phong phú của các hàm mà nó có thể biểu diễn. Một mô hình tuyến tính $y = ax + b$ chỉ biểu diễn được đường thẳng; đa thức bậc 10 biểu diễn được nhiều dạng cong hơn nhiều. Với mạng nơ-ron, dung lượng tăng theo độ rộng, độ sâu. Trò nhỏ có dung lượng thấp, nghĩa là có những hàm nó **không thể** biểu diễn, dù được huấn luyện lâu bao nhiêu.

2.2 Hai nguồn sai số

ML cơ bản nền tảng 03 đã nói: sai số của mô hình gồm sai số **xấp xỉ** (bias: mô hình không đủ dung lượng để biểu diễn hàm đúng) và sai số do **phương sai** (mô hình bắt theo nhiễu của dữ liệu huấn luyện). Mô hình nhỏ có thiên lệch cao, phương sai thấp; mô hình lớn thì ngược lại. Ta sẽ dùng chính hai khái niệm này để kể câu chuyện thầy-trò.

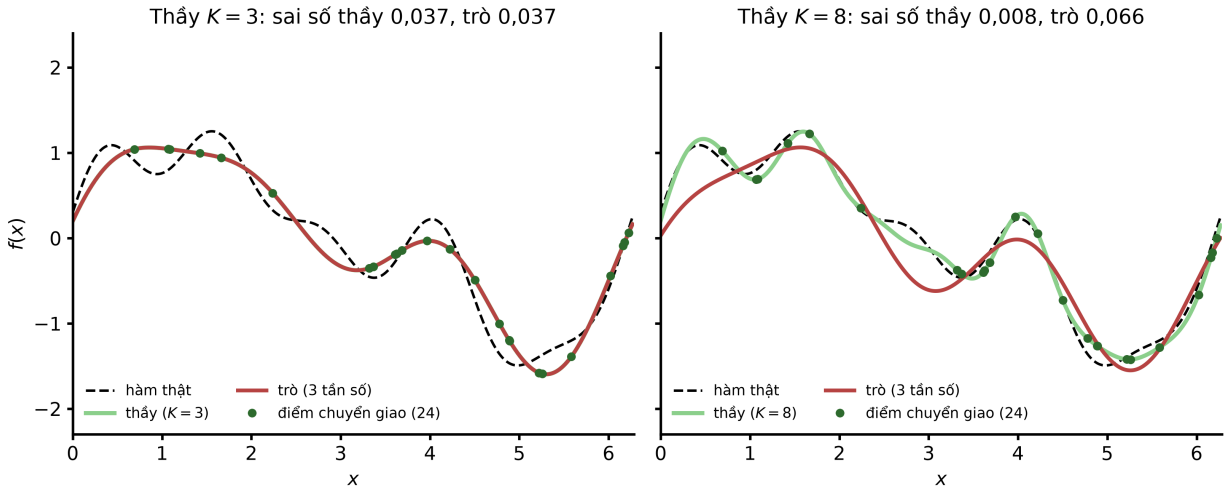
2.3 Ví dụ nhỏ: đường thẳng bắt chước đường cong

Cho $g(x) = x^2$ tại ba điểm $x = -1, 0, 1$, tức giá trị $(1, 0, 1)$. Đường thẳng tốt nhất theo bình phương tối thiểu có hệ số $a = 0$ (hệ số góc) và $b = \frac{2}{3}$ (trung bình của $1, 0, 1$). Nó dự đoán $\frac{2}{3}$ ở cả ba điểm, sai số bình phương trung bình là $\frac{1}{3}\left(\frac{1}{9} + \frac{4}{9} + \frac{1}{9}\right) = 0{,}2222$ (tự tính). Đây là **sai số xấp xỉ tối thiểu** của đường thẳng với hàm này: dù huấn luyện hoàn hảo cỡ nào, dung lượng của đường thẳng không cho nó thấp hơn.

3. Trực giác bằng hình

3.1 Trò bắt chước hàm của thầy

Ta dựng một thí nghiệm giả: hàm đúng $f(x) = \sin x + 0{,}5 \sin 2x + 0{,}3 \cos 3x + 0{,}25 \sin 5x$ trên $[0, 2\pi]$ (tổng bốn sóng, có một sóng tần số 5). Dữ liệu có nhãn: 200 điểm, nhiễu Gauss độ lệch chuẩn 0,3. **Thầy cỡ** K là hồi quy ridge trên các hàm $\sin(kx), \cos(kx)$ với $k = 1..K$ (cỡ càng lớn, hàm thầy càng uốn lượn được). **Trò** luôn là mô hình 3 tần số ($K = 3$, 7 hệ số). Trò không thấy nhãn, chỉ thấy **đầu ra của thầy** ở một tập chuyển giao (transfer set) gồm 24 điểm ngẫu nhiên, rồi khớp bình phương tối thiểu.



Hình 8: Mô phỏng: trò ba tần số bắt chước thầy K = 3 và thầy K = 8 từ 24 điểm chuyển giao (minh hoạ, tự tính)

Cách đọc hình: đường nét đứt đen là hàm thật; đường xanh là thầy; chấm xanh đậm là 24 điểm chuyển giao (giá trị của thầy tại các x đó); đường đỏ là trò. Trái: thầy $K = 3$ có cùng dung lượng với trò, nên đường đỏ chồng hẳn lên thầy (thầy bị che dưới trò), sai số hai bên bằng nhau (0,037). Phải: thầy $K = 8$ bám hàm thật tốt hơn nhiều (sai số 0,008) nhưng trò lại kém hơn ở trường hợp trái (0,066): đường đỏ lệch khỏi thầy ở những vùng thầy uốn mạnh, đặc biệt ở các đoạn không có điểm chuyển giao. Đây là kết quả **một lần chạy** (seed 0); chỉ mang tính minh hoạ.

Hiện tượng trong hình này, theo suy luận của chương: thầy $K = 8$ chứa thành phần tần số cao mà trò 3 tần số không biểu diễn nổi, và với chỉ 24 điểm, trò không thể tách thành phần ấy ra; phần tần số cao bị "gấp" (aliasing) vào các hệ số tần số thấp của trò và làm méo chúng. Đây là giải thích cho **mô hình đồ chơi**, không phải cơ chế đã chứng minh trong mạng sâu.

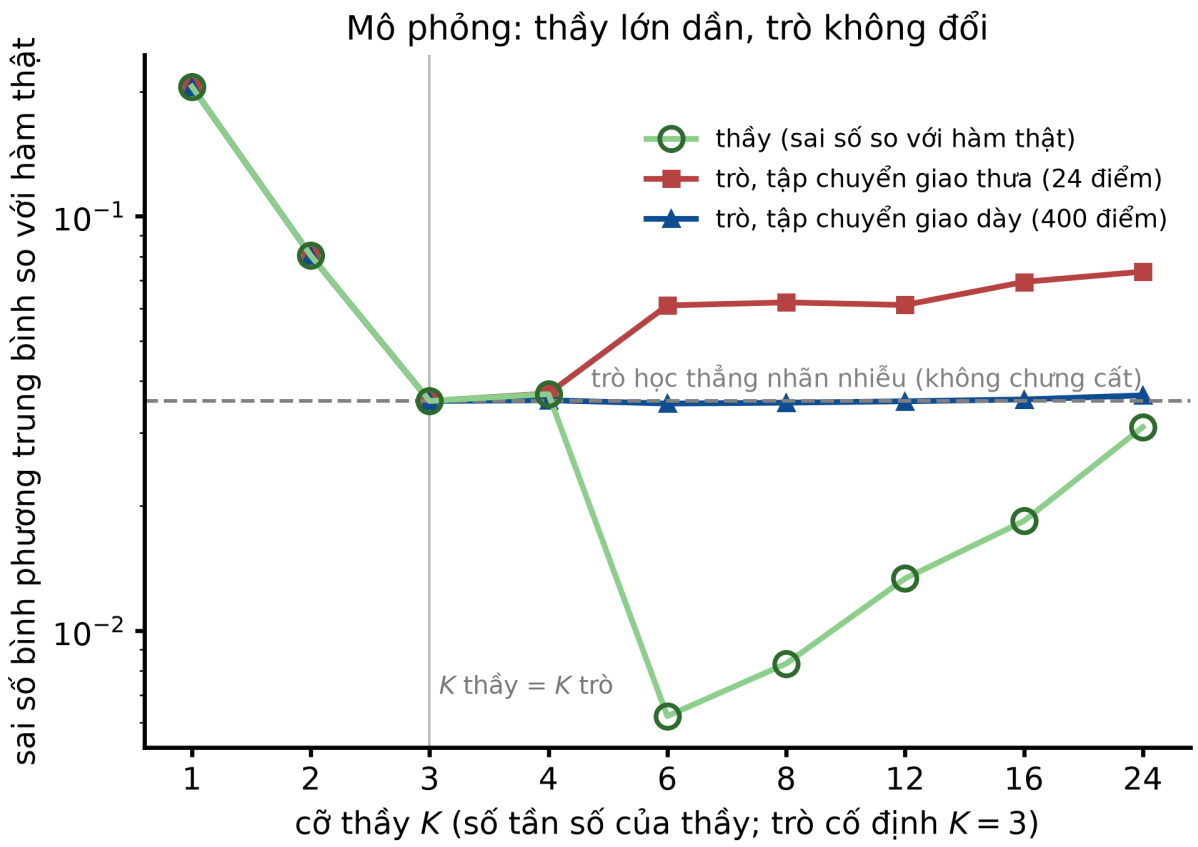
3.2 Chất lượng của trò theo cỡ thầy

Lặp lại thí nghiệm trên 200 hạt giống ngẫu nhiên và lấy trung bình, với cỡ thầy K từ 1 đến 24.

Cách đọc hình: trục dọc là sai số bình phương trung bình so với **hàm thật** (thang log, thấp là tốt), trục ngang là cỡ thầy. Với $K \leq 3$, thầy và hai đường trò trùng nhau (vòng xanh nằm đè lên). Đường xanh nhạt (thầy) giảm tới $K = 6$ rồi tăng trở lại: thầy quá lớn bị overfit 200 điểm nhiễu. Đường đỏ (trò, tập chuyển giao 24 điểm) lại tách ra từ $K = 4$: nó tăng và nằm trên mức đường nét đứt xám, tức **kém hơn cả trò học thẳng từ nhãn nhiễu**, dù thầy đang tốt hơn. Đường xanh đậm (tập chuyển giao dày 400 điểm) phẳng, bằng mức trò học từ nhãn. Số liệu mô phỏng (tự tính): với $K = 3$ thầy, trò đạt sai số 0,0358; với thầy $K = 6$ (thầy tốt nhất, sai số 0,0062) trò đạt 0,0609 trên tập thưa và 0,0353 trên tập dày; trò học thẳng nhãn đạt 0,0359.

Ba điều rút ra từ hình mô phỏng, và đây là điều chương muốn nhớ:

- Trò đạt chất lượng không theo thầy: thầy tốt nhất ($K = 6$) không cho trò tốt nhất (cho $K = 3$).



Hình 9: Mô phỏng: sai số của thầy và của trò (3 tần số) theo cỡ thầy K, với tập chuyển giao thưa và dày (minh hoạ, tự tính)

- Khoảng cách xuất hiện khi thầy chứa cấu trúc mà trò không biểu diễn được **và** dữ liệu chuyển giao không đủ để trò "lờ đi" cấu trúc đó. Với tập chuyển giao dày, trò chỉ học phần nó biểu diễn được và hiệu ứng biến mất.
- Mô phỏng này không nói rằng mọi trò đều mắc hiện tượng này; nó chỉ cho một **cơ chế khả dĩ** ở thế giới đồ chơi. Các paper trong cuốn đề nghị cơ chế khác ở mạng thật (mục 7).

4. Ví dụ tay: đường thẳng bắt chước thầy mượt và thầy nhiễu

Cùng hàm $g(x) = x^2$ tại $x = -1, 0, 1$, giá trị thật $(1, 0, 1)$. Trò là đường thẳng $y = ax + b$. Hai thầy:

- **Thầy mượt:** trả đúng g: $(1, 0, 1)$.
- **Thầy nhiễu:** bắt cả nhiễu của dữ liệu nhãn, trả $y = g + e$ với $e = (0{,}3; -0{,}3; 0{,}9)$, tức $(1{,}3;\ -0{,}3;\ 1{,}9)$ (số giả định cho dễ tính).

Bước 1: trò học từ thầy mượt. Với ba điểm đối xứng $x = -1, 0, 1$, bình phương tối thiểu cho b bằng trung bình các giá trị và $a = (y_3 - y_1)/2$ (suy ra từ công thức hồi quy). Thầy mượt: $b = \frac{2}{3}$, $a = 0$. Dự đoán $(0{,}667; 0{,}667; 0{,}667)$. Sai số so với hàm thật: $0{,}2222$ (đây là sai số xấp xỉ ở mục 2.3).

Bước 2: trò học từ thầy nhiễu. $b = (1{,}3 - 0{,}3 + 1{,}9)/3 = 0{,}9667$, $a = (1{,}9 - 1{,}3)/2 = 0{,}3$. Dự đoán $(0{,}667;\ 0{,}967;\ 1{,}267)$. So với hàm thật $(1, 0, 1)$: sai lệch $(-0{,}333;\ 0{,}967;\ 0{,}267)$, sai số bình phương trung bình $\frac{1}{3}(0{,}111 + 0{,}934 + 0{,}071) = 0{,}3722$ (tự tính).

Bước 3: phân rã. $0{,}3722 - 0{,}2222 = 0{,}1500$ là phần nhiễu mà trò đã "học theo". Tổng năng lượng nhiễu của thầy là $\frac{1}{3}(0{,}09 + 0{,}09 + 0{,}81) = 0{,}33$; trò chỉ chép lại $0{,}15$ và **bỏ** $0{,}18$, vì đường thẳng không thể biểu diễn thành phần nhiễu có dạng $(1, -2, 1)$ (thành phần này trực giao với mọi đường thẳng). Đây là phép phân rã trực giao quen thuộc của hồi quy tuyến tính (math nền tảng 03).

Hai nhận xét: một, **trò dung lượng thấp lọc bớt nhiễu của thầy** (đây là một cách hiểu vì sao trò nhỏ đôi khi vượt thầy hay overfit ít hơn, một giả thuyết, không phải quy luật); hai, phần nhiễu nằm trong không gian của trò thì trò **chép nguyên xi**, nên thầy nhiễu vẫn làm trò tệ hơn thầy mượt ($0{,}372$ so với $0{,}222$). Nếu nhiễu chỉ có dạng $e = (0{,}3; -0{,}6; 0{,}3)$ (hoàn toàn trực giao với đường thẳng) thì trò học từ thầy nhiễu cho **đúng** kết quả như học từ thầy mượt, sai số $0{,}2222$ (tự tính): trò không thể thấy nhiễu đó.

5. Ba cách chữa và một cách mở rộng

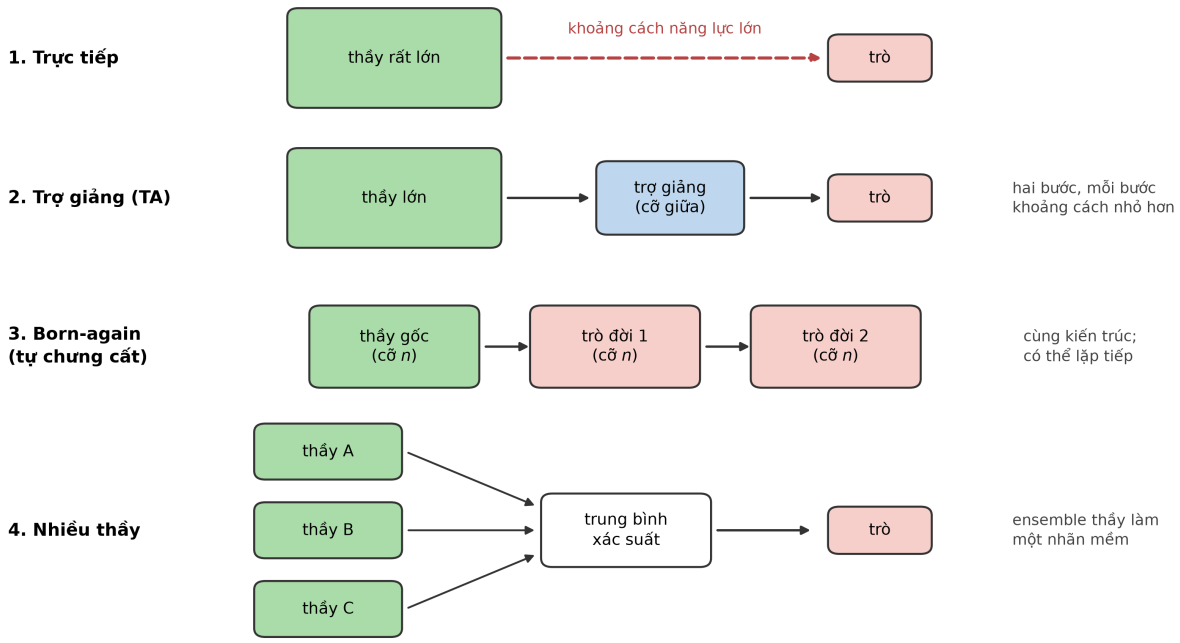
Cách đọc hình: kích thước hộp tượng trưng cho cỡ mô hình, màu xanh lá là mô hình đóng vai thầy, hồng là trò, xanh dương nhạt là mô hình trung gian. Hàng 1 là chưng cất trực tiếp từ thầy rất lớn (mũi tên đứt đỏ là khoảng cách năng lực lớn). Hàng 2 chèn một trợ giảng cỡ giữa. Hàng 3 là tự chưng cất (các đời cùng cỡ). Hàng 4 dùng nhiều thầy và lấy trung bình xác suất làm nhãn mềm. Hình là sơ đồ tổ chức, không có số liệu.

5.1 Trợ giảng (teacher assistant)

Ý tưởng: nếu khoảng cách giữa thầy và trò quá lớn, chèn một mô hình **cỡ giữa** làm cầu nối: thầy dạy trợ giảng, trợ giảng dạy trò. Mỗi bước, khoảng cách năng lực nhỏ hơn.

Theo ghi chú distillation 08 (TAKD): tác giả cố định trò CNN hai lớp tích chập, thay thầy từ 4 đến 10 lớp; độ chính xác của thầy tăng theo cỡ, nhưng độ chính xác của trò tăng rồi giảm. Một trợ giảng cỡ trung gian cải thiện nhẹ so với chưng cất trực tiếp; ví dụ CNN trên CIFAR-100 (trò 2 lớp, trợ giảng 4 lớp, thầy 10 lớp): NOKD 41,09%, chưng cất trực tiếp 44,57%, TAKD 44,92% (số lấy từ ghi chú 08, Bảng 1). Mức cải thiện nhỏ so với các lần lệch, và ghi chú 09 cho rằng chuỗi nhiều bước kiểu TA không giải quyết hẳn vấn đề.

Chọn **cỡ** trợ giảng là một siêu tham số: ghi chú 08 báo cáo trợ giảng gần trò hơn cho kết quả tốt hơn trong các bảng của họ, nhưng đây là kết quả trên bài toán cụ thể, không phải quy tắc chung.



Hình 10: Bốn cách tổ chức thầy và trò: trực tiếp, trợ giảng, born-again, nhiều thầy (minh hoạ, tự tính)

5.2 Born-again và tự chưng cất

Ý tưởng: bỏ luôn yếu tố "thầy lớn": trò **cùng kiến trúc** với thầy, khởi tạo ngẫu nhiên khác, học từ đầu ra của thầy (có thể kèm nhãn thật). Theo ghi chú distillation 06, trò (gọi là born-again network, BAN) thường **vượt** thầy trong các thí nghiệm của bài. Có thể lặp: trò đời 1 làm thầy cho đời 2, và ensemble các đời cho kết quả tốt hơn nữa.

Điều này khớp trực giác ở mục 4: không còn khoảng cách dung lượng, nên chỉ còn lợi ích **regularization** hoặc tối ưu hoá của nhãn mềm (không còn "chuyển giao năng lực"). Nhưng chương này chỉ đưa ra suy luận; bài 06 phân tích kỹ hơn.

Học tương hỗ (DML, bài 07): bỏ thầy cố định, nhiều trò học đồng thời và dạy lẫn nhau bằng KL (theo ghi chú 07). Về ý tưởng, mỗi trò vừa là thầy vừa là trò, nên không có khoảng cách dung lượng đáng kể.

5.3 Nhiều thầy

Một cách mở rộng: dùng **nhiều thầy** và gộp thông tin (trung bình xác suất, hoặc chọn ngẫu nhiên một thầy mỗi bước, hoặc thầy riêng cho nhãn mềm và đặc trưng; theo bản đồ phân loại ở ghi chú distillation 05). Nhãn mềm trung bình của nhiều thầy giống nhãn của một ensemble, thường **đa dạng và ít nhiễu** hơn mỗi thầy riêng, và ensemble là nguồn gốc ý tưởng của chưng cất (Hinton muốn nén ensemble vào một mạng, theo ghi chú 02).

Ví dụ tay: hai thầy trên hai lớp, $p_A = (0{,}8; 0{,}2)$, $p_B = (0{,}4; 0{,}6)$, trò $q = (0{,}5; 0{,}5)$. Trung bình hai thầy là $(0{,}6; 0{,}4)$. $\mathrm{KL}(\text{trung bình} \| q) = 0{,}6 \ln 1{,}2 + 0{,}4 \ln 0{,}8 = 0{,}0201$. Trong khi KL riêng từng thầy là $0{,}1927$ và $0{,}0201$, trung bình hai KL là $0{,}1064$ (tự tính). KL của trung bình **nhỏ hơn** trung bình các KL, đúng như tính lồi của KL: đích trung bình nằm gần "giữa" hơn so với từng đích lệch. (Hệ quả: học từ trung bình là đòi hỏi nhẹ hơn học từng thầy riêng lẻ.) Nhược điểm: chi phí chạy nhiều thầy, và nếu các thầy sai giống nhau thì trung bình không sửa được.

6. Hệ quả cho cách chọn thầy

Từ các mục trên, ta có những quy tắc **kinh nghiệm** (chưa phải định luật):

- Đừng mặc định chọn thầy lớn nhất. Thử một vài thầy, đặc biệt cỡ gần trò.
- Nếu bắt buộc phải dùng thầy rất lớn, cân nhắc trợ giảng hoặc dừng sớm KD cho trò (đề xuất ở ghi chú 09, xem mục 7).
- Chất lượng thầy đo bằng độ chính xác chưa đủ: thầy "dạy tốt" là thầy mà trò **học khớp được**.
- Có nghiên cứu nêu quy luật định lượng: ghi chú distillation 41 báo cáo quy luật thực nghiệm thầy tối ưu cỡ khoảng 2,5 lần trò trong thiết lập của họ (mô hình ngôn ngữ cỡ nhỏ, một cách cắt tỉa cụ thể), và ghi chú 42 sau đó đặt câu hỏi này lại một cách nghiêm ngặt hơn. Hãy coi hệ số 2,5 là kết quả của một thiết lập, không phải hằng số.

7. Vì sao chưng cất hiệu quả, hoặc không: các giả thuyết

Không có một câu trả lời duy nhất. Dưới đây là các giả thuyết xuất hiện trong cuốn này; mỗi giả thuyết giải thích được một phần và có thể cùng đúng.

- **Tri thức tối (nen01, bài 02):** nhãn mềm mang quan hệ giữa các lớp. Giải thích vì sao KD thắng nhãn cứng khi thầy đã học tốt.
- **Regularization và làm mượt (bài 06):** ngay cả thầy cùng cỡ cũng giúp trò, nên lợi ích không chỉ là chuyển giao năng lực. Ghi chú 06 còn khảo sát phần đóng góp của độ tin cậy ở lớp đúng so với cấu trúc các lớp còn lại.
- **Capacity mismatch (bài 09):** trò nhỏ không tìm được nghiệm khớp thầy lớn (KL tới thầy cao ngay cả trên tập huấn luyện); với bài khó, KD loss còn làm hại cross-entropy ở cuối huấn luyện. Cách chữa đề xuất: dừng KD sớm, hoặc dừng sớm quá trình huấn luyện thầy (theo ghi chú 09).
- **Khớp hàm và tối ưu hoá (bài 10, 11):** bài 10 đo độ khớp (fidelity) thấp bất ngờ giữa thầy và trò kể cả khi trò đủ lớn, và ủng hộ giả thuyết rằng khó khăn nằm ở **tối ưu hoá** (trò kẹt ở lòng chảo xấu), không hẳn ở thiếu dữ liệu chưng cất. Bài 11 xem KD như khớp hàm trên nhiều điểm hỗ trợ: thầy và trò phải thấy cùng một view, dữ liệu phải mở rộng bằng augmentation mạnh, và phải huấn luyện rất lâu (theo ghi chú 10, 11).
- **Aliasing và tập chuyển giao thưa (chỉ ở mô hình đồ chơi mục 3, suy luận của chương):** nếu thầy có thành phần mà trò không biểu diễn nổi và dữ liệu chuyển giao không đủ dày, thành phần ấy làm méo phần trò học được.

Điều chung: **khớp được thầy** (mức độ trò tái tạo thầy) và **tổng quát hoá tốt** (mức độ trò đúng với dữ liệu thật) là hai đại lượng khác nhau, và hai thí nghiệm ở mục 3.2 minh hoạ rằng cải thiện đại lượng thứ nhất ở thầy không bảo đảm đại lượng thứ hai ở trò.

8. Biến thể và chi tiết hay bị bỏ qua

- **Thầy không phải lúc nào cũng hơn trò về độ chính xác:** trò có thể vượt thầy khi tự chưng cất (mục 5.2), và nhãn mềm tự thân đã là regularizer.
- **Cỡ "dung lượng" khó đo:** số tham số, độ sâu, độ rộng hay số FLOP không phải thước đo đồng nhất; các bài paper dùng proxy khác nhau (ví dụ số lớp conv hay số block ở ghi chú 08).
- **Tập chuyển giao không chỉ là tập huấn luyện:** có thể dùng dữ liệu không nhãn, dữ liệu augment hoặc do mô hình sinh ra. Mô phỏng ở mục 3 cho thấy mật độ điểm có thể làm thay đổi kết luận.
- **Chưng cất nhiều thầy** không bắt buộc phải trung bình xác suất; có thể chọn trọng số theo độ tin cậy từng thầy.
- **Dừng sớm KD:** chuyển từ loss chưng cất sang cross-entropy ở cuối huấn luyện là một thủ thuật đơn giản mà ghi chú 09 đề xuất để chống capacity mismatch.

9. Nối sang các bài paper của cuốn

- **Bài 06 và 07:** là mục 5.2; chương giải thích vì sao không có khoảng cách năng lực khi thầy và trò cùng cỡ.
- **Bài 08:** là mục 5.1; ví dụ mục 4 cho trực giác vì sao "cầu nối" có thể giúp.
- **Bài 09, 10, 11:** là mục 7; ba bài đưa ba giả thuyết khác nhau về vì sao KD lúc được lúc không.
- **Bài 41:** là mục 6; quy luật khoảng cách năng lực biến câu hỏi "thầy cỡ bao nhiêu" thành một công thức thực nghiệm.

10. Sai lầm hay gặp

- **Cho rằng thầy chính xác hơn thì luôn dạy tốt hơn.** Mục 3 và ghi chú 08, 09 cho thấy không phải vậy.
- **Đọc mô phỏng đồ chơi như bằng chứng cho mạng sâu.** Hình mục 3 chỉ là một cơ chế khả dĩ ở hồi quy Fourier.
- **Báo cáo trò hơn thầy mà không kiểm tra nguồn lợi:** lợi ích có thể là regularization chứ không phải chuyển giao năng lực.
- **Chọn trợ giảng tuỳ tiện:** cỡ và số trợ giảng là siêu tham số, và lợi ích trên bài toán này không chắc có trên bài khác.
- **Dùng nhiều thầy mà không kiểm tra chi phí:** chạy k thầy là k lần suy luận của thầy mỗi bước.
- **So sánh tập chuyển giao khác nhau mà không nói:** kết luận về thầy tốt hay xấu có thể thay đổi theo dữ liệu chuyển giao.

11. Tóm tắt khái niệm

Khái niệm	Nội dung
Khoảng cách năng lực	thầy quá lớn so với trò có thể làm trò kém hơn thầy vừa phải
Trợ giảng (TA)	thầy dạy mô hình cỡ giữa, mô hình đó dạy trò
Born-again	trò cùng kiến trúc với thầy, có thể lặp nhiều đời
Học tương hỗ	nhiều trò dạy lẫn nhau, không có thầy cố định
Nhiều thầy	gộp (thường trung bình) xác suất các thầy làm nhãn mềm
Phân rã sai số	sai số trò so với thật = xấp xỉ + phần nhiễu thầy mà trò chép được
Tính lồi của KL	$\mathrm{KL}(\bar{p}\Vert q) \le \overline{\mathrm{KL}(p_k\Vert q)}$ khi $\bar{p} = \overline{p_k}$

12. Bài tập tự tính

Bài 1. Trò là đường thẳng, ba điểm $x = -1, 0, 1$, thầy trả $(2, 0, 2)$ (tức $2x^2$). Tìm đường thẳng trò học được và sai số bình phương trung bình giữa trò và thầy.

Đáp án (đã kiểm numpy): $b = \frac{4}{3}$, $a = 0$; dự đoán $\frac{4}{3}$ ở cả ba điểm; sai lệch so với thầy $(-0{,}667;\ 1{,}333;\ -0{,}667)$; MSE $= 0{,}8889$.

Bài 2. Hàm thật $g = x^2$ tại $x = -1, 0, 1$, thầy nhiễu trả $g + e$ với $e = (0{,}3; -0{,}6; 0{,}3)$. Sai số của trò đường thẳng so với g là bao nhiêu và vì sao?

Đáp án: $0{,}2222$, bằng sai số xấp xỉ khi học từ thầy mượt; vì $e \propto (1, -2, 1)$ trực giao với mọi đường thẳng ($\sum e_i = 0$ và $\sum x_i e_i = 0$), nên trò không "thấy" nhiễu.

Bài 3. Với $e = (0{,}3; -0{,}3; 0{,}9)$ (ví dụ mục 4), tính phần nhiễu trò chép lại và phần bỏ đi, rồi kiểm tổng bằng năng lượng nhiễu.

Đáp án: trò chép $0{,}15$, bỏ $0{,}18$, tổng $0{,}33 = \frac{1}{3}(0{,}09 + 0{,}09 + 0{,}81)$.

Bài 4. Hai thầy $p_A = (0{,}8; 0{,}2)$, $p_B = (0{,}4; 0{,}6)$, trò $q = (0{,}5; 0{,}5)$. Tính $\mathrm{KL}(p_A \| q)$, $\mathrm{KL}(p_B \| q)$ và KL của trung bình hai thầy; so sánh với trung bình hai KL.

Đáp án: $0{,}1927$; $0{,}0201$; KL của trung bình $= 0{,}0201$; trung bình hai KL $= 0{,}1064$. KL của trung bình nhỏ hơn, đúng với tính lồi.

Bài 5 (khó). Từ số liệu mô phỏng ở mục 3.2 (trò 3 tần số, tập thưa): trò học thẳng nhãn nhiễu đạt $0{,}0359$; trò học từ thầy $K = 6$ đạt $0{,}0609$; trò học từ thầy $K = 3$ đạt $0{,}0358$. Chưng cất từ thầy $K = 6$ kém hơn học thẳng nhãn bao nhiêu lần? Áp dụng quy luật "thầy tối ưu khoảng 2,5 lần trò" (ghi chú 41, kết quả của một thiết lập) cho một trò 1,2 tỉ tham số thì thầy gợi ý cỡ bao nhiêu?

Đáp án: $0{,}0609 / 0{,}0359 \approx 1{,}70$ lần (tệ hơn khoảng 70%, mô phỏng). Quy luật cho khoảng $1{,}2 \times 2{,}5 = 3$ tỉ tham số; đây là con số gợi ý theo hệ số của một thiết lập thực nghiệm, không phải cam kết cho mọi bài toán.

Tài liệu tham khảo nền: Mirzadeh et al., teacher assistant (ghi chú distillation 08); Furlanello et al., born-again networks (ghi chú 06); Zhang et al., deep mutual learning (ghi chú 07); Cho & Hariharan, efficacy of KD (ghi chú 09); Stanton et al., does KD really work (ghi chú 10); Beyer et al., function matching (ghi chú 11); ghi chú 41, 42; nen01 và nen02 của cuốn này; ML cơ bản nền tảng 03.

Nền tảng 04

Chưng cất cho dữ liệu dạng chuỗi: học từng token, học cả chuỗi và khớp bên trong BERT

Loại: chương nền tảng (không phải ghi chú paper)

Đọc trước: entropy, cross-entropy và KL (math nền tảng 11); mô hình dự đoán token kế tiếp ở mức trực giác (ML cơ bản nền tảng 07 và 08); soft label, nhiệt độ và loss distillation ở nen01 của cuốn này

Dùng ở: distillation 15 (chưng cất mức chuỗi), distillation 16 (DistilBERT), distillation 17 (PKD), distillation 18 (TinyBERT), distillation 19 (MiniLM), distillation 20 (chưng cất nhân quả)

> Chương này giải thích từ đầu, không phải bản dịch một paper. Các phép tính ghi "tự tính" đã kiểm bằng numpy; các hình ghi "giả định" hay "mô phỏng" dùng dữ liệu tự sinh, không phải số đo từ mô hình thật. Con số lấy từ paper luôn có chú "theo ghi chú distillation NN".

1. Vì sao cần chương này

Một câu: văn bản là chuỗi các lựa chọn nối nhau, và ở mỗi vị trí thầy đưa ra một phân phối trên cả từ vựng chứ không phải một nhãn; vì vậy "chưng cất" cho chuỗi có hai câu hỏi mà phân loại ảnh không có: khớp ở mức từng token hay mức cả chuỗi, và khớp cái gì bên trong Transformer.

Với bài phân loại ảnh 10 lớp, thầy cho một vectơ 10 số rồi trò khớp. Với mô hình ngôn ngữ, mỗi vị trí cần một vectơ dài cỡ 32.000 đến vài trăm nghìn số, và có hàng trăm vị trí trong một câu, mỗi vị trí phụ thuộc những token đứng trước. Chương này dạy bốn việc: (1) mô hình ngôn ngữ dự đoán token kế tiếp là gì; (2) vì sao phân phối trên từ vựng lớn khác nhãn lớp; (3) khớp mức token so với mức chuỗi; (4) ba cách chưng cất BERT ở mức ý tưởng, khớp logit, trạng thái ẩn và attention. Các bài 15 đến 20 của cuốn dùng các khái niệm này liên tục.

2. Nhắc nền: mô hình ngôn ngữ và KL

2.1 Mô hình dự đoán token kế tiếp

Một **mô hình ngôn ngữ** gán xác suất cho cả chuỗi bằng cách phân rã từng bước:

$$p(x_1, \ldots, x_T) = \prod_{t=1}^{T} p(x_t \mid x_{<t}),$$

trong đó $x_{<t}$ là các token đứng trước vị trí t (tiền tố). Mỗi nhân tử $p(\cdot \mid x_{<t})$ là một phân phối trên cả từ vựng V token: vectơ V số không âm có tổng bằng 1. Mô hình sinh văn bản bằng cách lấy mẫu một token từ phân phối đó, nối vào tiền tố, rồi lặp lại.

Ví dụ nhỏ với từ vựng 5 token. Sau tiền tố "the", thầy nghĩ token kế tiếp là:

token	the	cat	dog	sat	ran
thầy p	0,50	0,25	0,15	0,07	0,03

Dữ liệu huấn luyện thông thường chỉ cho một đáp án đúng ("cat", chẳng hạn), tức nhãn cứng. Thầy cho cả phân phối: "cat" hợp lý nhưng "dog" cũng hợp lý, "sat" ít, "ran" rất ít. Đó là thứ thông tin phong phú hơn nhãn cứng.

2.2 KL tại một vị trí

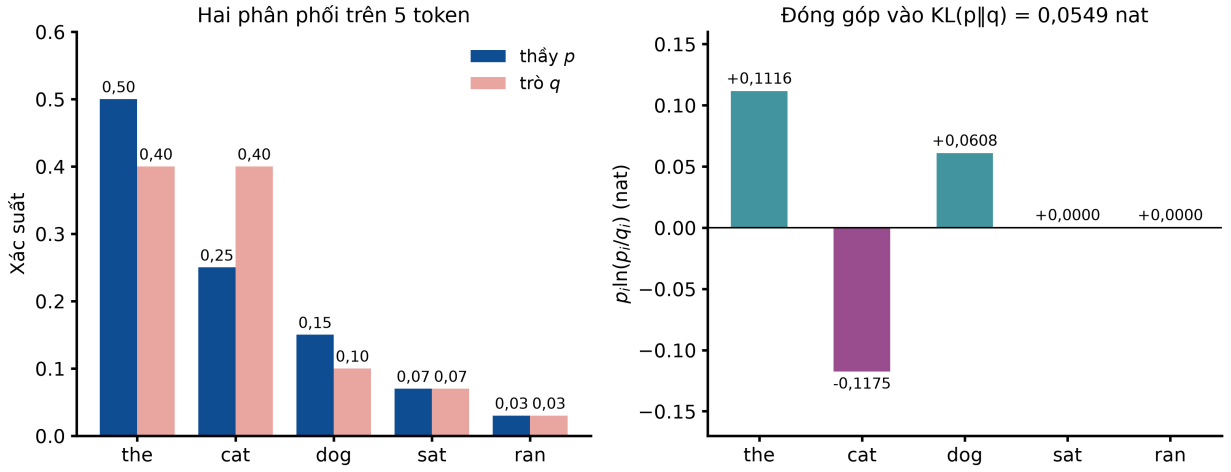
Nhắc lại từ math nền tảng 11: độ lệch KL từ thầy p sang trò q trên một vị trí là

$$\mathrm{KL}(p\|q) = \sum_{i=1}^{V} p_i \ln \frac{p_i}{q_i} = \underbrace{-\sum_i p_i \ln q_i}_{\text{cross-entropy}} - \underbrace{\Big(-\sum_i p_i \ln p_i\Big)}_{\text{entropy của thầy}}.$$

KL luôn không âm và bằng 0 khi $q = p$. Vì entropy của thầy không đổi theo trò, cực tiểu hoá KL tương đương cực tiểu hoá cross-entropy giữa thầy và trò, đúng như cách huấn luyện mô hình ngôn ngữ thông thường, chỉ khác là "nhãn" là cả phân phối thầy chứ không phải một one-hot.

3. Trực giác bằng hình: hai phân phối trên 5 token

Giả sử trò đoán sau "the" là $q = (0{,}40;\ 0{,}40;\ 0{,}10;\ 0{,}07;\ 0{,}03)$. Hình bên trái vẽ hai phân phối; hình bên phải chia KL thành đóng góp của từng token.



Hình 11: Thầy và trò trên từ vựng 5 token tại một vị trí (trái) và đóng góp của từng token vào KL(p‖q) (phải) (minh hoạ, tự tính)

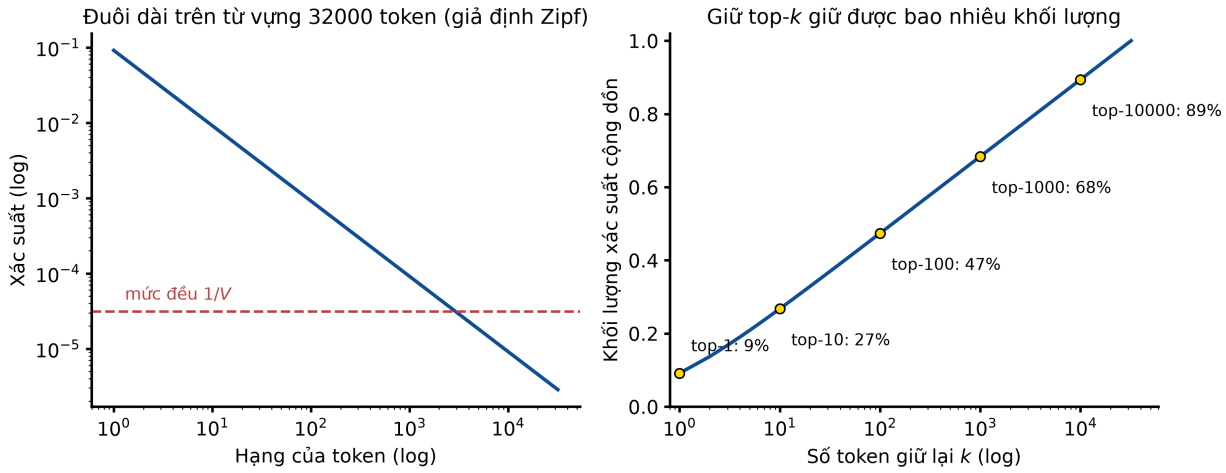
Cách đọc hình: bên trái, cột xanh đậm là thầy, cột hồng là trò; trò đánh giá thấp "the" (0,40 so với 0,50) và đánh giá cao "cat" (0,40 so với 0,25). Bên phải, mỗi cột là $p_i \ln(p_i/q_i)$: "the" đóng góp +0,1116, "dog" +0,0608, còn "cat" đóng góp âm (-0,1175) vì trò gán nhiều hơn thầy. Hai token cuối bằng 0 vì $p = q$. Tổng năm số là KL = 0,0549 nat (tự tính). Đóng góp từng token có thể âm, chỉ tổng mới không âm.

4. Vì sao phân phối trên từ vựng lớn khác nhãn lớp

4.1 Hình dạng: đuôi dài

Từ vựng của mô hình ngôn ngữ cỡ $V = 32.000$ đến hàng trăm nghìn. Phân phối của ngôn ngữ tự nhiên có "đuôi dài": một số ít token chiếm phần lớn xác suất, hàng nghìn token còn lại mỗi token một chút nhưng cộng lại đáng kể. Để thấy hình dạng, ta dùng phân phối **giả định** kiểu Zipf (xác suất tỉ lệ nghịch với hạng, $p_k \propto 1/k$) trên 32.000 token. Đây là giả định để vẽ, không phải đo từ mô hình thật.

Cách đọc hình: bên trái, trục log-log, xác suất giảm gần như tuyến tính theo hạng; đường đứt đỏ là mức đều $1/V$, tức mức xác suất nếu mọi token ngang nhau. Bên phải là khối lượng cộng dồn: trong giả định này token hạng 1 chỉ giữ 9%, top-10 giữ 27%, top-100 giữ 47%, top-1000 giữ 68% và top-10000 giữ 89% (tự tính). Giữ top-k nhỏ vẫn bỏ rơi một khối lượng lớn nằm ở đuôi.



Hình 12: Phân phối giả định kiểu Zipf trên từ vựng 32000 token (trái) và khối lượng cộng dồn khi giữ top-k token (phải) (minh hoạ, tự tính)

4.2 Ba hệ quả thực hành

- **Lưu và truyền thầy rất đắt.** Một câu 100 token cần $100 \times V$ số, hơn 3 triệu số cho $V = 32.000$, so với 100 số nguyên của nhãn cứng. Vì vậy nhiều phương pháp chỉ giữ top-k logit hoặc cho thầy chạy trực tuyến. Hệ quả này thuộc về độ phức tạp, không phải điều paper nào chứng minh.
- **Thông tin nằm ở cả đuôi.** Với nhãn lớp, "dark knowledge" nằm ở vài lớp sai gần giống đúng (xe tải gần xe hơi). Với từ vựng lớn, nó nằm ở hàng nghìn từ đồng nghĩa, biến thể chia động từ, tên riêng. Nhưng cũng nhiều đuôi là nhiễu; cắt đuôi ở top-k là một đánh đổi (cuốn này có một bài về lấy mẫu logit thưa, xem distillation 45).
- **Trò nhỏ có thể thiếu sức chứa.** Trò có ít tham số hơn không thể biểu diễn hết mọi phụ thuộc giữa các từ như thầy. Chương nền 05 của cuốn này đi sâu vào hệ quả: chọn KL thuận hay KL ngược khi trò không phủ hết thầy.

5. Chưng cất mức token và mức chuỗi

5.1 Hai cách hỏi "trò cần giống thầy ở đâu"

Chưng cất mức token (word-level KD): lấy một câu từ dữ liệu (hoặc do thầy sinh), với mỗi vị trí t ép phân phối token kế tiếp của trò giống thầy trên cùng tiền tố:

$$\mathcal{L}_{\text{token}} = \sum_{t=1}^{T} \mathrm{KL}\big(p(\cdot \mid x_{<t}) \,\|\, q_\theta(\cdot \mid x_{<t})\big).$$

Đây là Hinton KD áp dụng độc lập tại mỗi vị trí. Thầy "dạy" đúng tại các tiền tố trong dữ liệu.

Chưng cất mức chuỗi (sequence-level KD): thay vì khớp từng phân phối token, coi cả chuỗi là một "nhãn". Ý tưởng của Kim và Rush (2016): cho thầy sinh chuỗi đáp án $\hat{y}$ bằng beam search (xấp xỉ chuỗi có xác suất cao nhất), rồi huấn luyện trò như dữ liệu có nhãn cứng trên $\hat{y}$:

$$\mathcal{L}_{\text{seq}} = -\ln q_\theta(\hat{y} \mid x), \qquad \hat{y} \approx \arg\max_{y} p(y \mid x).$$

Đây là xấp xỉ: ta thay phân phối thầy trên mọi chuỗi bằng một điểm (mode) của nó.

5.2 Sao hai cách khác nhau

Mức token khớp *từng bước* nhưng tiền tố là của dữ liệu. Nếu trò không đủ sức chứa, nó "chia đều lỗi" ở mọi vị trí và có thể ghép các mảnh vốn không đi cùng nhau thành chuỗi mà thầy không bao giờ viết. Mức chuỗi hỏi thẳng: chuỗi nào thầy sẽ viết, rồi dạy trò viết nó, không bắt trò mô hình hoá mọi lựa chọn phụ. Cái giá là mất đa dạng (chỉ thấy mode), nên phân phối trò khác thầy nhiều.

6. Ví dụ tay

6.1 KL mức token trên 5 token

Dùng $p = (0{,}50; 0{,}25; 0{,}15; 0{,}07; 0{,}03)$ và $q = (0{,}40; 0{,}40; 0{,}10; 0{,}07; 0{,}03)$ như mục 3. Từng bước:

1. "the": $0{,}50 \ln(0{,}50/0{,}40) = 0{,}50 \times 0{,}2231 = 0{,}1116$.
2. "cat": $0{,}25 \ln(0{,}25/0{,}40) = 0{,}25 \times (-0{,}4700) = -0{,}1175$.
3. "dog": $0{,}15 \ln(0{,}15/0{,}10) = 0{,}15 \times 0{,}4055 = 0{,}0608$.
4. "sat", "ran": $p = q$ nên đóng góp 0.
5. Tổng: $0{,}1116 - 0{,}1175 + 0{,}0608 = 0{,}0549$ nat (tự tính).

Tách thành entropy và cross-entropy: entropy của thầy $H(p) = 1{,}2691$ nat, cross-entropy $-\sum p_i \ln q_i = 1{,}3240$ nat, hiệu chính là $0{,}0549$ (tự tính). KL theo chiều ngược $\mathrm{KL}(q\|p) = 0{,}0582$ nat, không bằng chiều thuận (tự tính); KL không đối xứng, ý này chương nền 05 khai thác.

So với nhãn cứng: nếu nhãn thật là "the", cross-entropy cứng là $-\ln 0{,}40 = 0{,}9163$ nat. Nhãn cứng chỉ nói "đáp án là the" và không nói gì về việc "cat" đáng tin hơn "ran". Thầy cho trò biết điều đó.

6.2 Chuỗi 3 token: mức token so với mức chuỗi

Từ vựng chỉ có hai token A và B, chuỗi dài 3, nên có 8 chuỗi. Thầy có điều kiện theo tiền tố, $P(x_t = A \mid \cdot)$:

tiền tố	rỗng	A	B	AA	AB	BA	BB
$P(A)$	0,6	0,8	0,1	0,9	0,3	0,5	0,2

Xác suất cả chuỗi là tích. Ví dụ $P(AAA) = 0{,}6 \times 0{,}8 \times 0{,}9 = 0{,}432$ và $P(BBB) = 0{,}4 \times 0{,}9 \times 0{,}8 = 0{,}288$. Tám xác suất (tự tính):

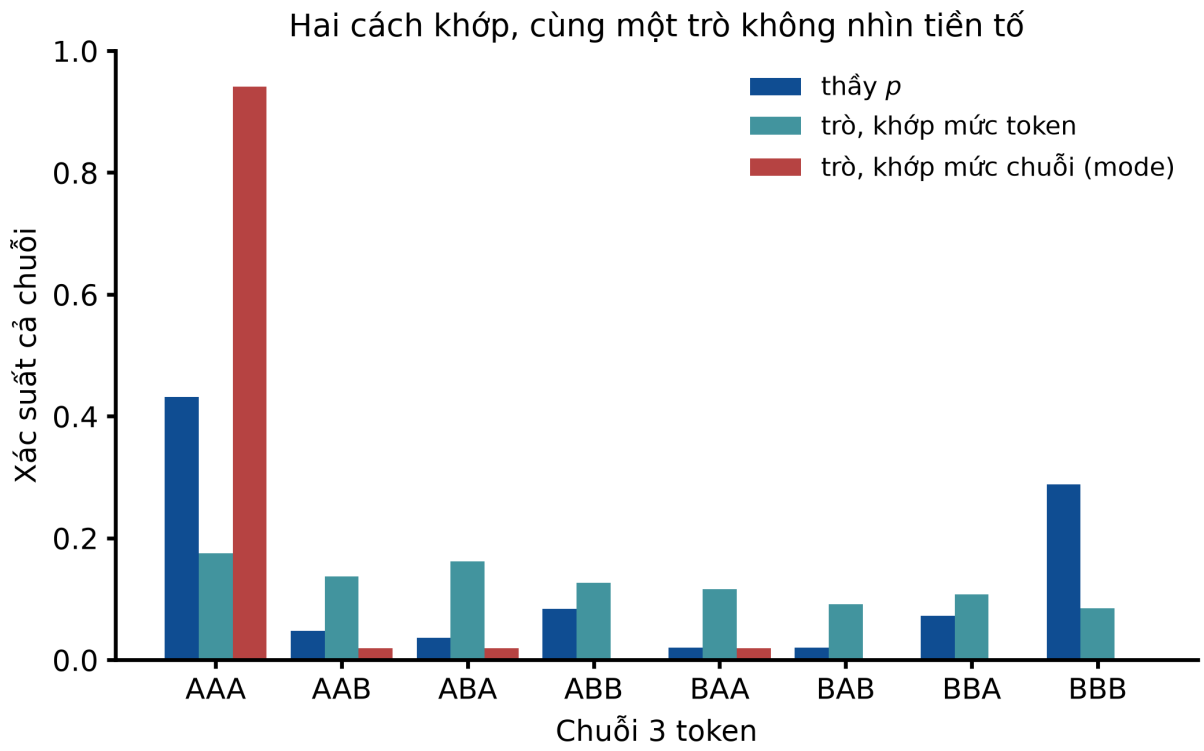
chuỗi	AAA	AAB	ABA	ABB	BAA	BAB	BBA	BBB
thầy p	0,432	0,048	0,036	0,084	0,020	0,020	0,072	0,288

Thầy có hai "đỉnh": AAA (0,432) và BBB (0,288), tức hai kiểu chuỗi nhất quán (toàn A hoặc toàn B). Ta tạo một trò cố tình rất yếu, **không nhìn tiền tố**: mỗi vị trí có một xác suất $q_t(A)$ riêng, độc lập các vị trí khác. Đây là giả định minh hoạ, để thấy rõ chuyện gì xảy ra khi trò thiếu sức chứa; trò thật mạnh hơn nhiều.

Khớp mức token. Cực tiểu tổng KL mức token trên các tiền tố của thầy cho nghiệm $q_t(A)$ bằng xác suất biên của thầy tại vị trí đó: vị trí 1 là 0,6; vị trí 2 là $0{,}6 \times 0{,}8 + 0{,}4 \times 0{,}1 = 0{,}52$; vị trí 3 là 0,56 (tự tính). Chuỗi của trò là tích các xác suất độc lập, ví dụ $q(AAA) = 0{,}6 \times 0{,}52 \times 0{,}56 = 0{,}1747$ và $q(AAB) = 0{,}6 \times 0{,}52 \times 0{,}44 = 0{,}1373$.

Khớp mức chuỗi. Mode của thầy là AAA. Trò học cho AAA gần như mọi xác suất; nếu làm mượt với $\varepsilon = 0{,}02$ mỗi vị trí (để KL còn hữu hạn) thì $q(AAA) = 0{,}98^3 = 0{,}9412$.

Cách đọc hình: cột xanh đậm là thầy với hai đỉnh AAA và BBB. Cột xanh lục (khớp mức token) trải khá đều trên cả tám chuỗi, đỉnh ở AAA chỉ 0,175 và BBB chỉ 0,085. Cột đỏ (khớp mức chuỗi) dồn gần hết khối lượng vào AAA, còn BBB gần như bằng 0.



Hình 13: Xác suất cả chuỗi 3 token của thầy, trò khớp mức token và trò khớp mức chuỗi (minh hoạ, tự tính)

Hai nhận xét định lượng (tự tính):

- **Mức token mất tính nhất quán.** Bốn chuỗi trộn A và B kiểu "AAB, ABA, BAA, BAB" chiếm 12,4% khối lượng của thầy nhưng 50,7% khối lượng của trò khớp mức token. Trò không nhìn tiền tố nên không biết "sau A thì ưu tiên A", và ghép các token thành chuỗi mà thầy ít viết. Đây là hệ quả của giả định trò yếu, là suy luận minh hoạ của chương.
- **Mức chuỗi mất đa dạng.** Trò dồn vào AAA nên đánh mất đỉnh BBB: $\mathrm{KL}(p\|q_{\text{seq}}) = 3{,}66$ nat, rất lớn vì thầy có 28,8% ở chỗ trò gần như bằng 0. Còn trò mức token có $\mathrm{KL}(p\|q_{\text{tok}}) = 0{,}51$ nat, đúng bằng tổng KL từng vị trí (0 + 0,262 + 0,249; quy tắc chuỗi cho KL).

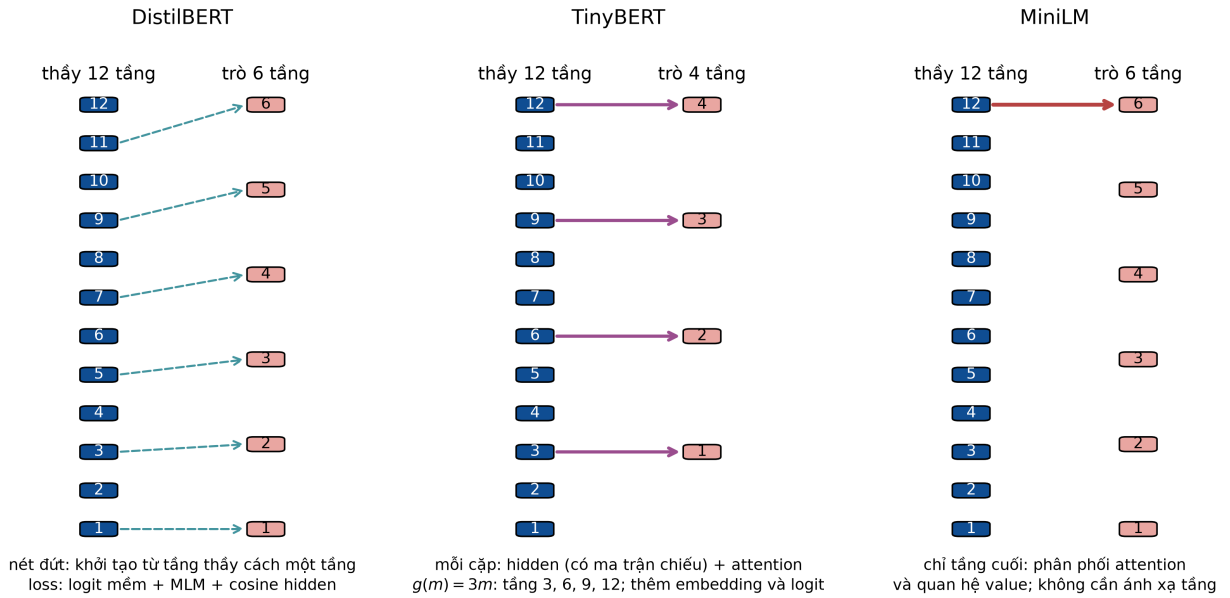
Không cách nào "đúng tuyệt đối": mức token giữ đa dạng nhưng cần trò đủ sức nhìn tiền tố; mức chuỗi cho chuỗi sạch nhưng thu hẹp phân phối về mode.

7. Chưng cất BERT: khớp logit, trạng thái ẩn, attention

BERT là mô hình Transformer đọc cả câu và tạo biểu diễn cho từng token ở mỗi tầng (không sinh từng token như mô hình ngôn ngữ). Nhưng ý tưởng chưng cất thì giống nhau: trò (ít tầng hoặc chiều ẩn nhỏ) bắt chước thầy. Có ba loại tín hiệu để khớp:

- **Logit hoặc xác suất đầu ra:** phân phối mềm ở tầng cuối (hoặc phân phối trên từ vựng ở tác vụ che từ, MLM). Rẻ, nhưng chỉ nhìn đầu ra.
- **Trạng thái ẩn (hidden state):** vectơ biểu diễn của từng token ở một tầng giữa. Cho trò thấy "thầy hiểu câu thế nào ở tầng thứ k". Nếu chiều ẩn khác nhau cần một ma trận chiếu học được.
- **Attention:** ma trận trọng số $\mathrm{softmax}(QK^{\top}/\sqrt{d_k})$ cho biết token nào chú ý token nào. Là thứ rất "cấu trúc": các head nhớ ngữ pháp, quan hệ gần xa.

Vấn đề kế tiếp: trò có ít tầng hơn, nên *tầng nào của trò học từ tầng nào của thầy*? Hình dưới vẽ ba lời đáp.



Hình 14: Ánh xạ tầng thầy sang tầng trò ở DistilBERT, TinyBERT và MiniLM, sơ đồ khái niệm (minh hoạ, tự tính)

Cách đọc hình: cột xanh đậm là 12 tầng thầy, cột hồng là tầng trò, mỗi mũi tên là một cặp "trò học từ thầy". DistilBERT (trái) lấy trọng số khởi tạo từ một tầng trong hai của thầy (nét đứt) rồi huấn luyện với loss ở đầu ra. TinyBERT (giữa) ánh xạ đều $g(m) = 3m$ cho trò 4 tầng nên khớp hidden và attention tại tầng thầy 3, 6, 9, 12. MiniLM (phải) chỉ khớp tầng cuối với tầng cuối nên không cần chọn ánh xạ.

7.1 DistilBERT: khởi tạo cách tầng, ba loss

Theo ghi chú distillation 16: student giảm một nửa số tầng, lấy trọng số khởi tạo từ một tầng trong hai của BERT; loss tổng là $L_{ce} + L_{mlm} + L_{cos}$ (cross-entropy với xác suất mềm của thầy, loss MLM với nhãn thật, và loss cosine kéo hướng hidden của trò gần thầy). Kết quả theo paper: nhỏ hơn 40%, nhanh hơn 60%, giữ khoảng 97% điểm GLUE của BERT (theo ghi chú distillation 16). Tín hiệu trung gian ở đây chỉ là cosine trên hidden cuối.

7.2 TinyBERT: khớp ở mọi chỗ

Theo ghi chú distillation 18: khớp embedding, attention từng head, hidden từng tầng được ánh xạ và logit. Vì chiều ẩn trò ($d' = 312$) khác thầy ($d = 768$), dùng ma trận chiếu học được. Ánh xạ đều $g(m) = 3m$. Trò 4 tầng (14,5M tham số) đạt Avg GLUE 77,0 (test) so với 79,5 của BERT-base; nhỏ hơn 7,5 lần và nhanh hơn 9,4 lần (theo ghi chú distillation 18). Cùng bảng, DistilBERT 4 tầng (52,2M tham số) đạt 71,9. Nên "khớp nhiều tầng" ở đây có lợi dù ít tham số hơn, nhưng paper còn dùng thêm chưng cất hai giai đoạn và tăng cường dữ liệu, nên không tách được riêng công của từng tín hiệu.

7.3 MiniLM: chỉ tầng cuối, khớp quan hệ

Theo ghi chú distillation 19: chỉ khớp tự-attention của tầng cuối: phân phối attention (so bằng KL) và quan hệ giữa các value, softmax$(VV^\top/\sqrt{d_k})$. Quan hệ là ma trận $|x| \times |x|$ nên không phụ thuộc chiều ẩn, không cần ma trận chiếu, và hết bài toán chọn ánh xạ tầng. Trong cùng kiến trúc 6x768 (bảng 2 của paper, tóm trong ghi chú), Avg: DistilBERT 75,2; TinyBERT 79,1; MiniLM 80,4; BERT-base 81,5 (theo ghi chú distillation 19), có chú ý các cơ sở so sánh không hoàn toàn cùng điều kiện.

7.4 Ví dụ tay nhỏ cho hai loại loss trung gian

Attention. Một token của thầy chú ý ba token trong câu với trọng số $(0{,}6;\ 0{,}3;\ 0{,}1)$, trò chú ý $(0{,}4;\ 0{,}4;\ 0{,}2)$. Đây là hai phân phối nên loss là KL: $0{,}6 \ln 1{,}5 + 0{,}3 \ln 0{,}75 + 0{,}1 \ln 0{,}5 = 0{,}2433 - 0{,}0863 - 0{,}0693 = 0{,}0877$ nat (tự tính). Mỗi head, mỗi token trong câu có một phân phối như vậy, và loss là trung bình.

Hidden. Hidden của thầy $t = (1;\ 2)$ và của trò (sau khi đã chiếu về cùng chiều) $s = (2;\ 1)$. MSE là $\big((1-2)^2 + (2-1)^2\big)/2 = 1$ (tự tính), còn cosine là $(2+2)/(\sqrt{5}\sqrt{5}) = 0{,}8$ nên loss cosine $1 - 0{,}8 = 0{,}2$ (tự tính). Hai thước đo trả lời hai câu hỏi khác nhau: MSE hỏi "có trùng cả độ dài lẫn hướng không", cosine chỉ hỏi "có cùng hướng không".

7.5 Vì sao nhiều lựa chọn như vậy

Nhiều tín hiệu hơn cho trò nhiều chỉ dẫn hơn, nhưng đòi hỏi ánh xạ tầng, ma trận chiếu và nhiều siêu tham số. Tín hiệu quá chặt cũng có thể gò trò vào cách thầy tổ chức bên trong. Đây là cân bằng giữa "giám sát dày" và "để trò tự do tìm cách biểu diễn", chủ đề của distillation 20.

8. Biến thể và chi tiết hay bị bỏ qua

- **Nhiệt độ.** Chia logit cho $T > 1$ trước softmax làm phân phối phẳng hơn, lộ đuôi. Loss mềm thường nhân T^2 để giữ độ lớn gradient. Khi suy luận trò dùng $T = 1$.
- **Teacher forcing.** Cả mức token lẫn huấn luyện LM thường đưa tiền tố đúng cho mô hình ở mọi vị trí, khác với lúc suy luận khi trò dùng token của chính mình. Chương nền 05 nói về độ lệch này.
- **Mask.** Ở BERT, loss chỉ tính ở các vị trí bị che (MLM) hoặc ở mọi token (hidden), nên phải chú ý token đệm (padding).
- **KL, MSE, cosine.** Phân phối so bằng KL; hidden so bằng MSE hoặc cosine; attention so bằng KL hoặc MSE. Mỗi lựa chọn có thang đo khác nhau, nên trọng số cần dò.
- **Mức chuỗi bằng mode xấp xỉ.** Beam search không cho chính xác mode. Trong ghi chú distillation 15, bản dịch En-De: trò 2x500 baseline 14,7 BLEU (greedy), thêm word-KD 15,4, Seq-KD 18,9; thầy 17,7 (theo ghi chú distillation 15). Seq-KD đẩy trò vượt thầy greedy; đồng thời mức chuỗi làm trò quá tự tin (xác suất của đầu ra greedy 16,9% so với 1,3% của thầy trong cùng bảng), nên đa dạng giảm.

9. Nối sang các bài paper của cuốn

- **distillation 15 (Seq-KD):** hiện thực hoá mục 5.1, nhắc cả word-KD và seq-KD, và thấy lợi ích cho mô hình dịch.
- **distillation 16 đến 19:** ba cách ánh xạ tầng của mục 7; đọc lần lượt theo mức giám sát tăng dần rồi quay lại "chỉ tầng cuối" của MiniLM.
- **distillation 20 (chưng cất nhân quả):** hỏi nên ép trò khớp *cái gì* bên trong mô hình.
- **distillation 28 đến 34:** cho mô hình sinh, thay KL mức token bằng các lựa chọn divergence và mẫu của trò; xem chương nền 05 của cuốn này. Ví dụ MiniLLM (theo ghi chú distillation 28), LLaMA 13B sang 7B, DollyEval Rouge-L: SFT 26,3; KD 27,4; SeqKD 27,5; MiniLLM 29,0; thầy 29,7.

10. Sai lầm hay gặp

- Nghĩ rằng nhãn của thầy là "một token". Thầy cho cả phân phối trên từ vựng, thông tin ở cả đuôi.
- Cắt đuôi (top-k) mà quên chuẩn hoá lại, làm tổng xác suất khác 1.
- Cho rằng khớp từng token thì cả chuỗi sẽ giống thầy. Với trò thiếu sức chứa, mỗi bước "gần đúng" vẫn có thể cộng thành chuỗi sai (mục 6.2).
- Nghĩ chưng cất mức chuỗi vô hại về đa dạng: nó dồn về mode, nên mất nhánh ít xác suất hơn.
- Quên ma trận chiếu khi chiều ẩn trò khác thầy, hoặc quên tính loss trên token đệm.
- Đọc bảng GLUE của DistilBERT, TinyBERT và MiniLM như thể cùng điều kiện huấn luyện; các paper khác nhau về dữ liệu, tăng cường dữ liệu và số bước.

11. Tóm tắt công thức

Tên	Công thức	Ghi chú
LM phân rã	$p(x_{1:T}) = \prod_t p(x_t \mid x_{<t})$	mỗi nhân tử là phân phối trên V token
KL một vị trí	$\sum_i p_i \ln(p_i/q_i)$	không âm, không đối xứng
Mức token	$\sum_t \mathrm{KL}(p(\cdot \mid x_{<t}) \| q_\theta(\cdot \mid x_{<t}))$	tiền tố từ dữ liệu
Mức chuỗi	$-\ln q_\theta(\hat{y} \mid x)$	$\hat{y}$ gần mode của thầy
Ánh xạ tầng TinyBERT	$g(m) = 3m$	trò 4 tầng, thầy 12 tầng
Quan hệ value MiniLM	$\mathrm{softmax}(VV^\top/\sqrt{d_k})$	không phụ thuộc chiều ẩn

12. Bài tập tự tính

Bài 1. Từ vựng 3 token, $p = (0{,}7; 0{,}2; 0{,}1)$, $q = (0{,}5; 0{,}3; 0{,}2)$. Tính $\mathrm{KL}(p\|q)$ và $\mathrm{KL}(q\|p)$ (nat).

Đáp án: $\mathrm{KL}(p\|q) = 0{,}7\ln 1{,}4 + 0{,}2\ln(2/3) + 0{,}1\ln 0{,}5 = 0{,}2355 - 0{,}0811 - 0{,}0693 = 0{,}0851$; chiều ngược $0{,}0920$ (tự tính).

Bài 2. $p = (0{,}5; 0{,}25; 0{,}15; 0{,}07; 0{,}03)$ và trò đều $q_i = 0{,}2$. Tính KL và cross-entropy của thầy với trò.

Đáp án: $H(p) = 1{,}2691$, cross-entropy $= \ln 5 = 1{,}6094$, nên KL $= 1{,}6094 - 1{,}2691 = 0{,}3404$ nat (tự tính).

Bài 3. Với phân phối Zipf giả định trên 32.000 token, cần ít nhất bao nhiêu token hàng đầu để giữ 50% khối lượng, và 90%?

Đáp án: 134 token cho 50% và 10.705 token cho 90% (tự tính; giả định $p_k \propto 1/k$).

Bài 4. Trong ví dụ chuỗi 3 token, trò mức token có $q_1(A) = 0{,}6$, $q_2(A) = 0{,}52$, $q_3(A) = 0{,}56$. Tính $q(BBB)$ và so với $p(BBB) = 0{,}288$.

Đáp án: $q(BBB) = 0{,}4 \times 0{,}48 \times 0{,}44 = 0{,}0845$; thầy cao hơn $0{,}288/0{,}0845 \approx 3{,}4$ lần (tự tính).

Bài 5. BERT-base có khoảng 110M tham số, DistilBERT khoảng 66M; thời gian suy luận 668 so với 410 (đơn vị như ghi chú distillation 16). Giảm bao nhiêu phần trăm tham số, và thời gian giảm bao nhiêu phần trăm?

Đáp án: tham số giảm $1 - 66/110 = 40\%$; thời gian $410/668 = 0{,}614$ nên giảm khoảng 39%, tức nhanh khoảng 1,6 lần (tự tính; paper nói "nhanh hơn 60%" theo cách tính tốc độ).

Tài liệu tham khảo nền: Hinton, Vinyals và Dean về distilling knowledge (ghi chú distillation 02); Kim và Rush về sequence-level KD (ghi chú distillation 15); ghi chú distillation 16 đến 19 cho DistilBERT, PKD, TinyBERT, MiniLM.

Nền tảng 05

KL thuận, KL ngược và chưng cất on-policy: trò nhỏ nên bắt chước thầy ở đâu

Loại: chương nền tảng (không phải ghi chú paper)

Đọc trước: KL, entropy và cross-entropy (math nền tảng 11); Gaussian và hỗn hợp (math nền tảng 09); xác suất và kỳ vọng (math nền tảng 08); chương nền 04 của cuốn này (chưng cất cho chuỗi, mức token và mức chuỗi)

Dùng ở: distillation 28 (MiniLLM, KL ngược on-policy), distillation 29 (GKD), distillation 30 (DistiLLM, skew KL), distillation 31 (xem lại KL thuận và ngược), distillation 32, distillation 33, distillation 34

> Chương này giải thích từ đầu, không phải bản dịch một paper. Các phép tính ghi "tự tính" đã kiểm bằng numpy; các hình là mô phỏng số hoặc mô hình giả định, không phải số đo từ mô hình thật. Con số lấy từ paper luôn có chú "theo ghi chú distillation NN".

1. Vì sao cần chương này

Một câu: KL không đối xứng, nên "làm cho trò giống thầy" có ít nhất hai cách đo (thuận và ngược) cho ra hai trò khác hẳn khi trò không đủ sức bắt chước hết thầy; và dữ liệu mà trò học (văn bản có sẵn) khác với dữ liệu mà trò gặp khi dùng (văn bản do chính nó sinh), điều này dẫn tới chưng cất on-policy.

Dùng để làm gì: các bài 28 đến 34 của cuốn đều xoay quanh hai câu hỏi: dùng divergence nào, và lấy mẫu từ đâu (từ thầy, từ dữ liệu, hay từ trò). Chương này dạy cả hai bằng ví dụ nhỏ nhất có thể tính tay: thầy hai đỉnh, trò một đỉnh.

2. Nhắc nền

2.1 KL có hai chiều

Với hai phân phối p (thầy) và q (trò):

$$\mathrm{KL}(p\|q) = \mathbb{E}_{x\sim p}\Big[\ln \frac{p(x)}{q(x)}\Big] \quad \text{(KL thuận)}, \qquad \mathrm{KL}(q\|p) = \mathbb{E}_{x\sim q}\Big[\ln \frac{q(x)}{p(x)}\Big] \quad \text{(KL ngược)}.$$

Khác nhau ở **kỳ vọng lấy theo phân phối nào**. KL thuận lấy theo thầy: phạt trò ở mọi nơi *thầy* có khối lượng. KL ngược lấy theo trò: chỉ phạt trò ở nơi *trò* đặt khối lượng.

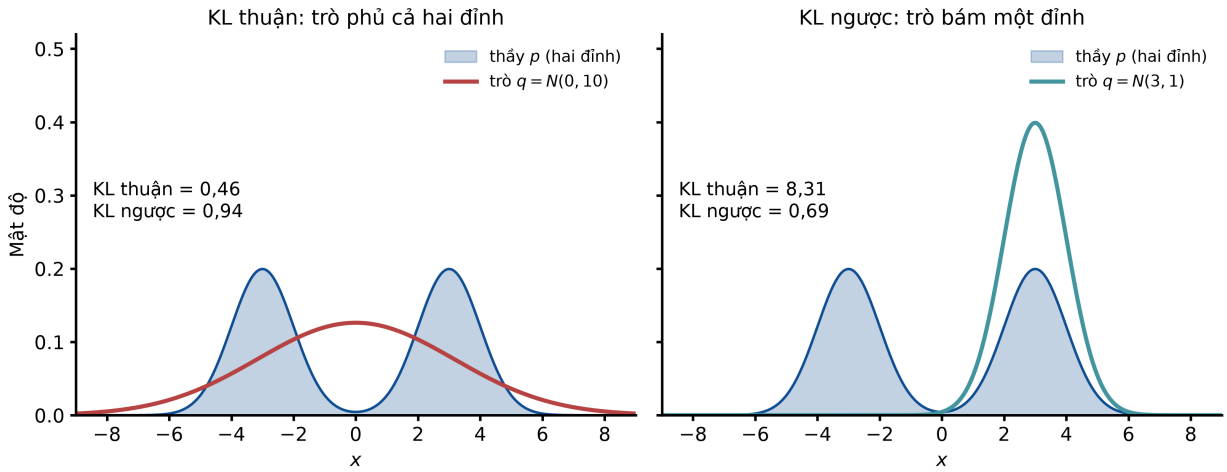
Ví dụ nhỏ với hai khả năng. Thầy $p = (0{,}5;\ 0{,}5)$, trò $q = (0{,}9;\ 0{,}1)$. Thì $\mathrm{KL}(p\|q) = 0{,}5\ln(0{,}5/0{,}9) + 0{,}5\ln(0{,}5/0{,}1) = -0{,}2939 + 0{,}8047 = 0{,}5108$ và $\mathrm{KL}(q\|p) = 0{,}9\ln 1{,}8 + 0{,}1\ln 0{,}2 = 0{,}5290 - 0{,}1609 = 0{,}3681$ (tự tính). Hai số khác nhau; thông thường không cái nào "đúng hơn", chúng đo hai thứ khác.

2.2 Hai nhiệm vụ của divergence trong KD

KD đưa trò về thầy bằng cách cực tiểu hoá một divergence. Nếu trò đủ mạnh để biểu diễn đúng p thì cả hai chiều đều cho nghiệm $q = p$. Chương này xét tình huống thực tế hơn: trò **không** biểu diễn được p. Khi đó chiều KL quyết định trò sẽ hy sinh điều gì.

3. Trực giác bằng hình: một Gauss khớp hai đỉnh

Thầy là hỗn hợp hai Gauss: $p(x) = 0{,}5\,N(x;-3,1) + 0{,}5\,N(x;3,1)$, nghĩa là hai "đáp án" đều hợp lý, đỉnh tại -3 và $+3$. Trò chỉ là một Gauss $N(\mu,\sigma^2)$ (một đỉnh). Câu hỏi: chọn (μ,σ) nào?



Hình 15: Hai trò tối ưu cho cùng một thầy hai đỉnh: theo KL thuận (trái) và theo KL ngược (phải) (minh hoạ, tự tính)

Cách đọc hình: vùng xanh là thầy với hai đỉnh. Bên trái, trò tối ưu theo KL thuận là một Gauss rộng $N(0,\ 10)$ trải qua cả hai đỉnh, nhưng cũng đặt nhiều khối lượng vào chỗ trũng ở giữa nơi thầy gần như không có gì. Bên phải, trò tối ưu theo KL ngược bám đúng một đỉnh, $N(3, 1)$, hẹp, bỏ trống đỉnh còn lại. Mỗi hình ghi sẵn cả hai KL của trò đó: trò trái có KL thuận 0,46 và KL ngược 0,94; trò phải có KL thuận 8,31 và KL ngược 0,69 (tự tính, tích phân số).

Hai tên gọi: KL thuận là **mode-covering** (phủ mọi nơi thầy có khối lượng), KL ngược là **mode-seeking** (bám lấy một đỉnh). Thực ra đó là cách gọi trực giác; paper distillation 31 phản biện nó cho LLM rời rạc (mục 8).

4. Vì sao hai chiều cho hai kết quả

4.1 Nhìn vào dấu hiệu phạt

- **KL thuận** có $\ln \frac{p(x)}{q(x)}$ lấy kỳ vọng theo p. Nếu tại một x có $p(x) > 0$ mà $q(x) \approx 0$, số hạng $\ln(p/q)$ rất lớn. Trò *không được phép* bỏ trống nơi thầy có khối lượng. Ngược lại, nơi thầy bằng 0 mà trò có khối lượng, số hạng chỉ bị nhân với $p \approx 0$ nên phạt nhẹ.
- **KL ngược** có $\ln \frac{q(x)}{p(x)}$ lấy kỳ vọng theo q. Nơi $q \approx 0$ thì trọng số q bằng 0 nên không bị phạt, dù thầy có khối lượng ở đó. Ngược lại, nếu trò đặt khối lượng ở chỗ $p \approx 0$ thì $\ln(q/p)$ rất lớn, bị phạt nặng.

Vì vậy KL thuận ép trò "đừng bỏ sót", và KL ngược ép trò "đừng bịa chỗ thầy không có".

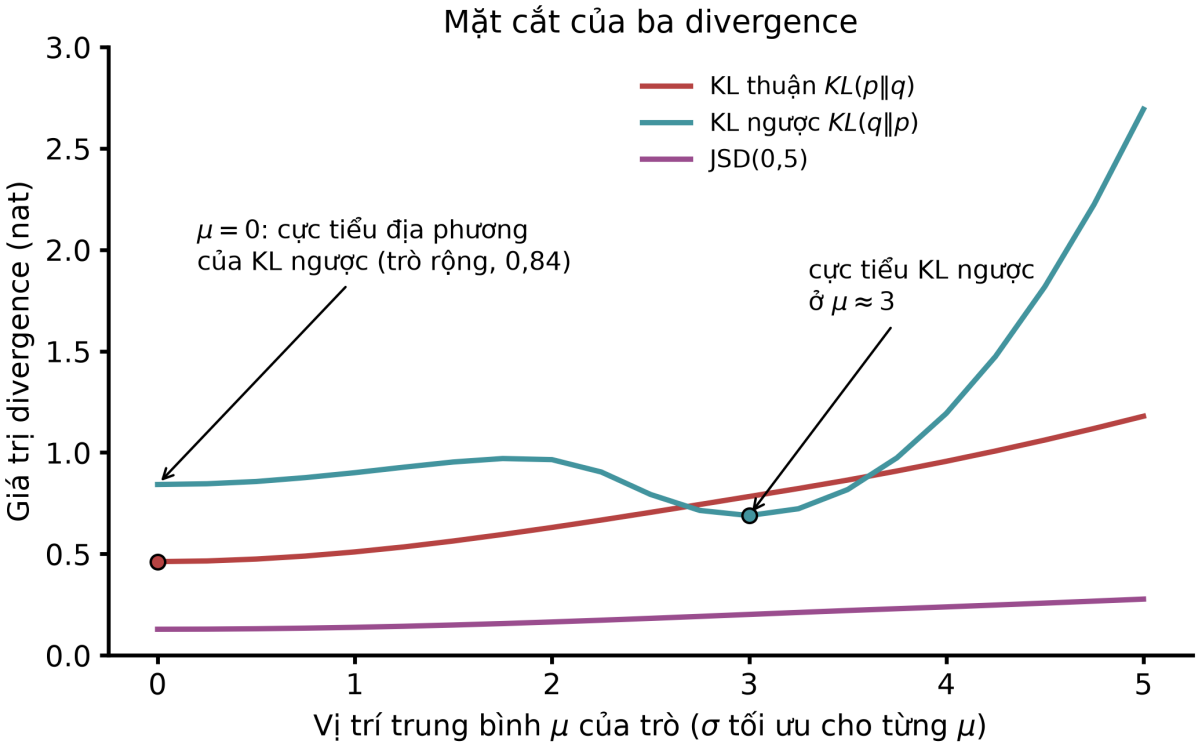
4.2 Khớp bằng một Gauss: công thức đóng

Với KL thuận, nghiệm Gauss là **khớp moment** (trung bình và phương sai bằng của thầy). Thầy có trung bình 0 (đối xứng) và phương sai $1 + 3^2 = 10$ (phương sai trong mỗi đỉnh cộng bình phương độ lệch của hai tâm so với trung bình). Vậy trò tối ưu là $N(0, 10)$, đúng như hình. Với KL ngược, không có công thức đóng; số cho thấy cực tiểu toàn cục là $N(\approx 3,\ \approx 1{,}02)$ hoặc ảnh đối xứng $N(\approx -3,\ \approx 1{,}02)$, KL ngược khoảng 0,689 (tự tính). Giá trị này gần $\ln 2 = 0{,}693$: tại đỉnh trò bám, thầy chỉ có một nửa khối lượng, nên $p \approx 0{,}5\, q$ và $\ln(q/p) \approx \ln 2$.

4.3 KL ngược có nhiều cực tiểu địa phương

Hình sau cố định μ rồi chọn σ tối ưu cho từng μ, vẽ giá trị ba divergence theo μ.

Cách đọc hình: đường đỏ (KL thuận) gần như đơn điệu, một "bát" duy nhất với đáy ở $\mu = 0$. Đường xanh lục (KL ngược) có **hai** chỗ trũng: một ở $\mu \approx 3$ (chấm xanh, tối ưu toàn cục, 0,689) và một nhỏ ở $\mu = 0$ với



Hình 16: Mặt cắt của KL thuận, KL ngược và JSD(0,5) theo vị trí trung bình mu của trò, độ rộng sigma chọn tối ưu cho từng mu (minh hoạ, tự tính)

giá trị 0,84 (trò rộng); giữa chúng là một gờ cao khoảng $\mu \approx 1{,}75$. Tối ưu từ đâu thì rơi vào hố nào. Đường tím (JSD(0,5), bị chặn) rất thấp và mượt trong dải này. Ở thực hành LLM, nghĩa là KL ngược nhạy với khởi tạo; đây là một lý do để khởi tạo trò từ mô hình đã tinh chỉnh có giám sát (suy luận của chương; theo ghi chú distillation 30, bảng 6, MiniLLM và GKD đều giảm khi bỏ bước tinh chỉnh này).

5. Vì sao trò nhỏ không thể phủ hết phân phối thầy

Phần này là suy luận của chương, không phải kết quả của paper. Trò có ít tham số hơn thầy, nên tập các phân phối trò biểu diễn được $\mathcal{Q}$ nhỏ hơn. Nếu $p \notin \mathcal{Q}$ thì không có q nào bằng p; mọi divergence chỉ cho "gần nhất" theo thước đo của nó. Ví dụ hai đỉnh: không có Gauss nào có hai đỉnh. Ở mô hình ngôn ngữ, hai đỉnh là hai cách trả lời hợp lý cho một câu hỏi (thầy 13 tỉ tham số biết cả hai, trò 1 tỉ thì không kham hết). Có ba hệ quả:

- Theo KL thuận, trò "chia nhỏ" xác suất ra cả hai cách và cả chỗ giữa, nên khi lấy mẫu có thể sinh câu lai tạp, vô nghĩa (nhưng không bỏ sót kiểu trả lời nào).
- Theo KL ngược, trò chọn một cách trả lời, nhưng đó là trả lời tốt, và mất đa dạng.
- Khi đáp án ngắn (không gian đầu ra nhỏ), trò phủ được mọi mode nên khác biệt giữa hai chiều biến mất; ở đáp án dài thì rõ hơn. Theo ghi chú distillation 28 (Hình 7 của paper), với đáp án tối đa 5 token mọi phương pháp tương đương, còn với đáp án dài MiniLLM hơn rõ.

6. Lệch phân phối giữa huấn luyện và suy luận: exposure bias

6.1 Hiện tượng

Khi huấn luyện mô hình ngôn ngữ theo kiểu thông thường (teacher forcing), ở mỗi vị trí t mô hình nhận **tiền tố đúng** từ dữ liệu và được chấm ở token kế tiếp. Khi suy luận, mô hình nhận **tiền tố do chính nó sinh**, có thể lệch. Một lỗi nhỏ đẩy tiền tố sang vùng mô hình chưa từng được huấn luyện; nó dễ sai thêm, lệch thêm.

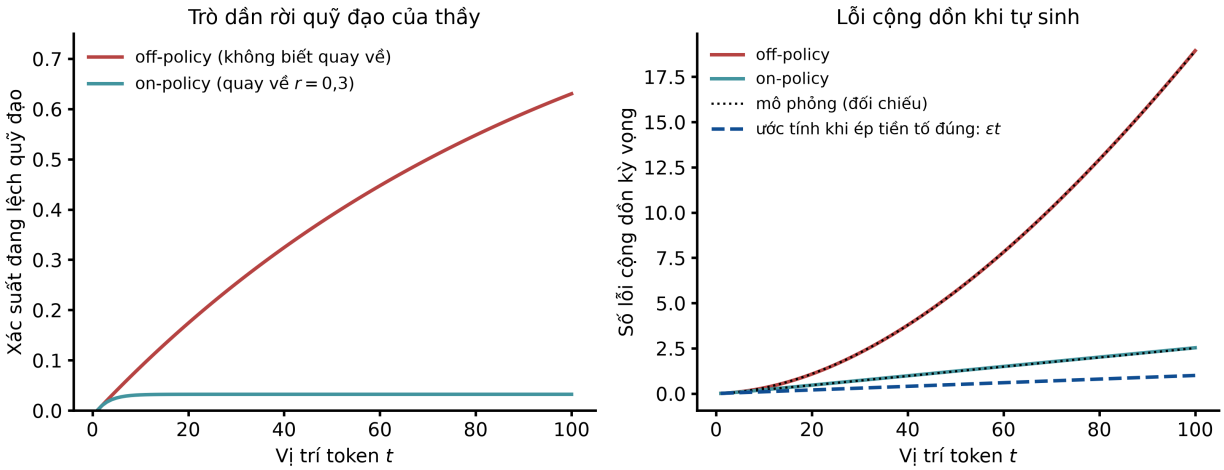
Hiện tượng gọi là **exposure bias** (độ lệch phơi bày): tập tiền tố lúc học và lúc dùng khác nhau.

Cần phân biệt: đây là chuyện *phân phối của tiền tố*, không phải chuyện thầy sai. KD mức token cổ điển (huấn luyện trò trên văn bản có sẵn) thừa hưởng nguyên điểm yếu này.

6.2 Mô hình giả định hai trạng thái

Để thấy cỡ độ lớn, ta dựng mô hình đơn giản (giả định, không đo từ mô hình thật):

- Có hai trạng thái: *trên quỹ đạo* (tiền tố giống dữ liệu) và *lệch quỹ đạo*.
- Khi trên quỹ đạo, mỗi token sai với xác suất $\varepsilon = 0{,}01$, và mỗi bước có xác suất ε rơi vào trạng thái lệch.
- Khi lệch, mỗi token sai với xác suất $\delta = 0{,}5$ (mô hình chưa từng gặp tiền tố kiểu này).
- Trò **off-policy** chưa từng thấy trạng thái lệch nên không biết quay về ($r = 0$). Trò **on-policy** đã học từ chính mẫu của mình nên quay về trên quỹ đạo với xác suất $r = 0{,}3$ mỗi bước.



Hình 17: Xác suất lệch quỹ đạo và số lỗi cộng dồn kỳ vọng theo vị trí token, trò off-policy và trò on-policy trong mô hình giả định hai trạng thái (minh hoạ, tự tính)

Cách đọc hình: bên trái là xác suất đang lệch tại vị trí t: đường đỏ (không biết quay về) leo mãi, tới khoảng 0,63 sau 100 token; đường xanh lục dừng ở khoảng 0,03 vì có quay về. Bên phải là số lỗi cộng dồn kỳ vọng: nếu giả định luôn có tiền tố đúng, ta sẽ ước tính $\varepsilon t = 1$ lỗi sau 100 token (đường xanh dương đứt). Trò off-policy thực tế tích luỹ khoảng 18,9 lỗi (đường đỏ), cong lên như t^2; trò on-policy khoảng 2,5 lỗi. Đường chấm đen là mô phỏng 20.000 chuỗi (seed cố định) khớp công thức đóng (tự tính). Hình dạng "εt so với εt^2" là bài học quen thuộc của học bắt chước (imitation learning): lỗi cộng dồn có thể tăng bậc hai theo độ dài nếu mô hình không được học trên các trạng thái do chính nó tạo ra.

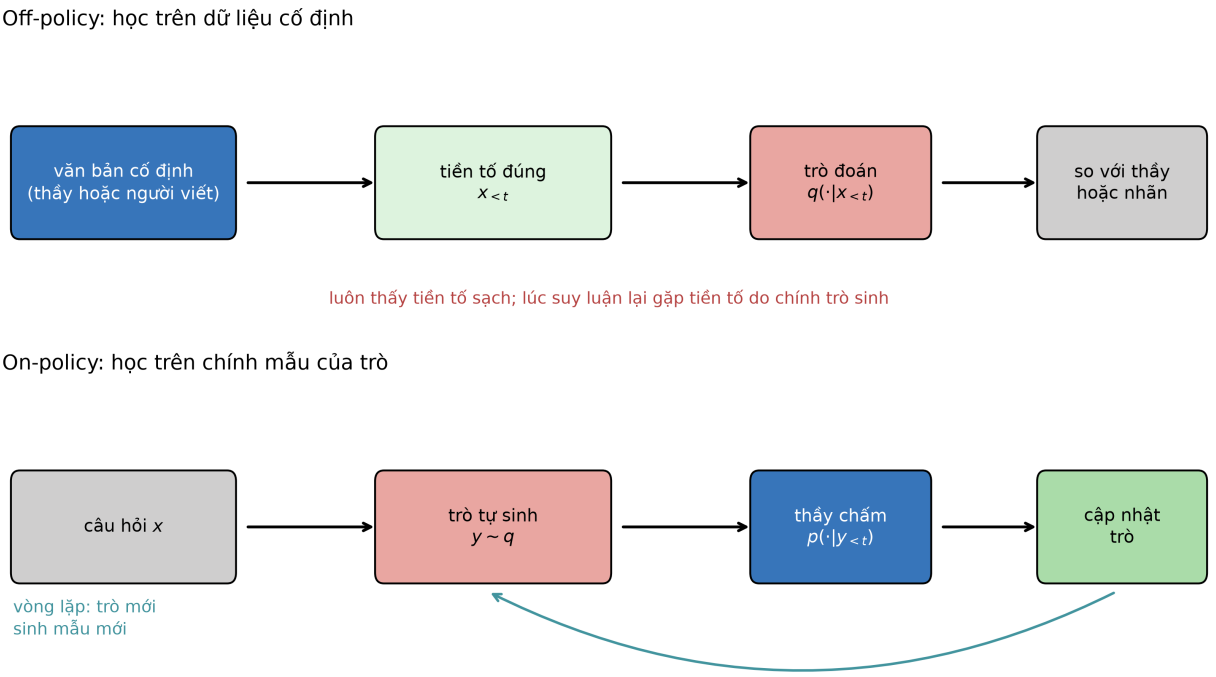
7. Chưng cất on-policy

7.1 Ý tưởng

On-policy là để *chính trò* sinh mẫu (tiền tố, đáp án), rồi *thầy chấm* trên đúng tiền tố đó. Nghĩa là phân phối của tiền tố trong lúc học bằng phân phối khi suy luận. Gọi $y \sim q_\theta(\cdot \mid x)$ là đáp án do trò sinh cho câu hỏi x; loss có dạng

$$\mathcal{L}_{\text{on}}(\theta) = \mathbb{E}_x \, \mathbb{E}_{y \sim q_\theta(\cdot|x)} \Big[\textstyle\sum_t D\big(p(\cdot \mid x, y_{<t}) \,\|\, q_\theta(\cdot \mid x, y_{<t})\big) \Big],$$

với D là một divergence nào đó. **Off-policy** thay y bằng mẫu cố định (văn bản dữ liệu hoặc đầu ra thầy), như ở chương nền 04.



Hình 18: Sơ đồ off-policy (học trên văn bản cố định) và on-policy (trò sinh, thầy chấm, cập nhật rồi lặp) (minh hoạ, tự tính)

Cách đọc hình: hàng trên là off-policy: tiền tố đúng đi vào trò, trò đoán rồi so với thầy hoặc nhãn; trò không bao giờ thấy tiền tố của chính nó. Hàng dưới là on-policy: trò sinh y, thầy tính p trên các tiền tố của trò, trò cập nhật; mũi tên cong là vòng lặp, trò sau mỗi bước sinh mẫu khác.

7.2 Hai cách dùng

- **KL ngược bằng policy gradient** (ý tưởng MiniLLM, theo ghi chú distillation 28): $\mathrm{KL}(q\|p)$ có kỳ vọng theo q, nên lấy mẫu từ q chính là cách ước lượng nó. Nhưng đạo hàm đi qua việc lấy mẫu, nên dùng gradient kiểu REINFORCE, phương sai cao. Paper thêm ba mẹo ổn định (tách từng bước, trộn mẫu thầy vào, chuẩn hoá độ dài); bỏ chuẩn hoá độ dài hoặc bỏ trộn mẫu thầy làm Rouge-L giảm mạnh (ví dụ 24,6 xuống 14,7 và 20,4 theo ghi chú distillation 28).
- **GKD** (theo ghi chú distillation 29): chia dữ liệu thành tỉ lệ λ mẫu của trò và $1-\lambda$ mẫu cố định, và cho phép chọn divergence tuỳ ý (KL thuận, KL ngược, JSD). Gradient không đi qua phép lấy mẫu; chỉ chấm lại mẫu trò sinh (nên là ước lượng gradient thiên lệch nhưng ổn định, theo ghi chú distillation 29). Kết quả: dữ liệu on-policy là yếu tố quan trọng nhất; trên XSum, WMT, GSM8K, biến thể on-policy và mixed luôn vượt biến thể chỉ dùng dữ liệu cố định.

7.3 Chi phí và đánh đổi

On-policy phải sinh mẫu trong lúc huấn luyện: theo ghi chú distillation 29, chi phí tăng khoảng 1,8x, 2x và 2,2x (trò nhỏ hơn thầy 38x, 12x, 3,8x) so với lấy mẫu từ dữ liệu cố định. Nên có nhánh nghiên cứu giảm chi phí (distillation 30 dùng bộ nhớ mẫu và lịch thích nghi; 32 cho thầy thay các token tệ ngay trong lúc trò sinh, lấy mẫu xen kẽ). Ngoài ra mẫu trò ở đầu huấn luyện thường kém, nên thầy chấm trên rác; vì vậy hay bắt đầu bằng trò đã tinh chỉnh có giám sát, hoặc trộn mẫu thầy.

8. Các cách dung hoà ở mức ý tưởng

- **Trộn on-policy và off-policy** (GKD, λ): mỗi lô lấy một phần mẫu từ trò, phần còn lại từ dữ liệu. Giảm chi phí và giảm rủi ro mẫu rác.

- **JSD(β):** $\mathrm{JSD}_\beta(p,q) = \beta\,\mathrm{KL}(p\|m) + (1-\beta)\,\mathrm{KL}(q\|m)$ với $m = \beta p + (1-\beta)q$. Bị chặn, nội suy giữa KL thuận (β nhỏ) và KL ngược (β gần 1), theo ghi chú distillation 29.
- **Skew KL** (DistiLLM, theo ghi chú distillation 30): so p với hỗn hợp $\alpha p + (1-\alpha)q$ thay vì q, để mẫu số không về 0 và gradient ổn định; $\alpha = 0{,}1$ tốt nhất theo paper. Ví dụ tay: $p = (0{,}5; 0{,}5)$, $q = (0{,}9; 0{,}1)$, $\alpha = 0{,}1$ cho hỗn hợp $(0{,}86;\ 0{,}14)$; $\mathrm{SKL} = 0{,}5\ln(0{,}5/0{,}86) + 0{,}5\ln(0{,}5/0{,}14) = 0{,}3653$ nat so với $\mathrm{KL} = 0{,}5108$ (tự tính); và tỉ số lớn nhất p/q là 5 trong khi p/hỗn hợp chỉ 3,57 (tự tính). Trên một mô hình đồ chơi hai đỉnh của chương này, với JSD, SKL và SRKL ($\alpha = 0{,}1$), cực tiểu toàn cục tìm bằng lưới vẫn là Gauss rộng ở tâm 0 (tự tính). Nghĩa là các divergence bị chặn không tự động khiến trò bám một đỉnh ở thang này; công dụng của chúng chủ yếu là ổn định tối ưu, không phải chọn mode.
- **Trộn thuận và ngược thích nghi** (AKL, theo ghi chú distillation 31): giả định "KL thuận phủ, KL ngược bám" không đúng hẳn cho LLM rời rạc, vì cả hai chỉ có một điểm cân bằng $q = p$; khác biệt nằm ở giai đoạn đầu: KL thuận ưu tiên khớp phần đầu (các token xác suất cao), KL ngược ưu tiên phần đuôi. Paper trộn hai loss theo trọng số thích nghi. Số liệu paper (Bảng 1 ghi chú distillation 31, GPT-2 120M, Dolly Rouge-L): FKL 23,46; RKL 22,62; FKL+RKL 23,36; AKL 23,88 (theo ghi chú distillation 31).
- **Thầy trung gian nội suy theo thời gian** (TAID, distillation 33), không dùng dữ liệu do trò sinh: đổi mục tiêu từng bước cho vừa sức trò.

Bảng nhanh một số số liệu trên GPT-2 (thầy GPT-2 1,5B sang trò 120M), Dolly Rouge-L (theo ghi chú distillation 30, bảng 1): KLD 23,52; RKLD 23,82; JSD tổng quát 24,34; SKL 24,80; SRKL 25,21. Biên độ nhỏ (vài phần mười đến một điểm), nên khó rút ra "luật" từ một thí nghiệm.

9. Ví dụ tay: tính KL thuận và KL ngược cho hai lựa chọn của trò

Thầy $p = 0{,}5\,N(-3,1) + 0{,}5\,N(3,1)$. Hai lựa chọn của trò:

- **Lựa chọn A:** $q_A = N(0,10)$, phủ cả hai.
- **Lựa chọn B:** $q_B = N(3,1)$, bám đỉnh phải.

KL tính bằng tích phân số trên lưới (numpy, bước 0,005; công thức $\sum p \ln(p/q)\,\Delta x$):

trò	$\mathrm{KL}(p\|q)$	$\mathrm{KL}(q\|p)$
A: $N(0,10)$	0,46	0,94
B: $N(3,1)$	8,31	0,69

(tự tính). Đọc: nếu thước đo là KL thuận, A thắng (0,46 so với 8,31) vì B bỏ trống đỉnh trái có khối lượng 50%; nếu thước đo là KL ngược, B thắng (0,69 so với 0,94). Lý do B tệ ở KL thuận: tại $x = -3$, $p \approx 0{,}2$ trong khi $q_B \approx 0$ (cỡ 10^{-8}), nên $\ln(p/q)$ rất lớn. Lý do A tệ ở KL ngược: trò A đặt 24,8% khối lượng vào $|x| < 1$, vùng thầy chỉ có 2,3% (tự tính), và đó là chỗ $\ln(q/p)$ lớn.

Về khối lượng: A đặt 50,0% bên phải $x = 0$ (cân đối) còn B đặt 99,9% (tự tính). Khi lấy mẫu từ A, gần một phần tư mẫu rơi vào vùng "chưa từng là đáp án của thầy"; khi lấy mẫu từ B, mọi mẫu là đáp án hợp lý nhưng luôn thuộc cùng một nhánh.

10. Biến thể và chi tiết hay bị bỏ qua

- **Khối lượng đa dạng là một chỉ số riêng.** Theo ghi chú distillation 29, khi lấy mẫu ở nhiệt độ 1, divergence mode-seeking cho chất lượng cao hơn nhưng đa dạng thấp hơn (đo bằng Self-BLEU); hạ nhiệt độ làm các divergence gần nhau.
- **Greedy và sampling.** Với giải mã greedy, chọn divergence ít ảnh hưởng; với lấy mẫu, ảnh hưởng nhiều hơn (theo ghi chú distillation 29). Vì mode-seeking đã hướng trò vào vùng xác suất cao.

- **Hiệu chuẩn (calibration).** Theo ghi chú distillation 28, trên SST2 (LLaMA-7B zero-shot) ECE: thầy 0,025; KD 0,191; SeqKD 0,243; MiniLLM 0,099. Con số cho thấy KL ngược on-policy không loại bỏ hết vấn đề, nhưng gần thầy hơn các baseline.
- **Student vượt teacher.** Trong một số ô, trò vượt thầy về Rouge-L; tác giả MiniLLM đoán do thầy cũng chịu exposure bias (theo ghi chú distillation 28), đây là giả thuyết, không kiểm chứng.
- **Không có divergence thắng ở mọi tác vụ.** Theo ghi chú distillation 29, divergence tối ưu phụ thuộc tác vụ và cách giải mã: trên dịch WMT, JSD tổng quát tốt hơn KL thuận và KL ngược, còn trên instruction tuning KL ngược tốt hơn nhiều so với KL thuận (tác giả nêu giả thuyết, chưa kiểm chứng, rằng tính mode-seeking giúp tập trung vào ý chính của chỉ dẫn). Trong thực hành, nên coi chiều KL là một siêu tham số cần thử, không phải chân lý.
- **Chi phí huấn luyện.** Theo ghi chú distillation 28, chưng cất LLaMA 13B sang 7B dưới 10 giờ trên 16 GPU V100 32GB.

11. Sai lầm hay gặp

- Nghĩ KL thuận và KL ngược "tương đương". Chỉ bằng nhau ở cực tiểu khi trò biểu diễn được thầy; ngoài ra chúng cho hai trò khác nhau.
- Đọc "mode-seeking" thành "luôn tốt hơn". Mất đa dạng, nhạy khởi tạo (nhiều cực tiểu địa phương), và theo distillation 31 trực giác này không áp dụng nguyên si cho từ vựng rời rạc.
- Dùng mẫu của thầy để suy ra hành vi của trò. Phân phối tiền tố khác, nên lỗi cộng dồn như mục 6.
- Chạy on-policy từ trò chưa được khởi tạo tốt. Mẫu rác cho thầy chấm rác, huấn luyện lệch.
- Bỏ qua chuẩn hoá độ dài trong KL ngược chuỗi: trò học sinh chuỗi lặp hoặc ngắn bất thường (theo ghi chú distillation 28, hiện tượng này xảy ra khi bỏ length normalization).
- So số liệu giữa các paper như thể cùng điều kiện: mỗi paper dùng dữ liệu, trò, thầy và số seed khác.

12. Tóm tắt công thức

Tên	Công thức	Ý nghĩa
KL thuận	$\mathbb{E}_p[\ln(p/q)]$	mode-covering: không bỏ sót chỗ thầy có
KL ngược	$\mathbb{E}_q[\ln(q/p)]$	mode-seeking: không bịa chỗ thầy không có
Khớp moment (Gauss, KL thuận)	$\mu = \mathbb{E}_p[x]$, $\sigma^2 = \mathrm{Var}_p(x)$	nghiệm đóng cho KL thuận
JSD(β)	$\beta\,\mathrm{KL}(p\|m) + (1-\beta)\,\mathrm{KL}(q\|m)$	bị chặn, nội suy hai chiều
Skew KL	$\mathrm{KL}(p\|\alpha p + (1-\alpha)q)$	mẫu số không về 0
On-policy	$y \sim q_\theta$ rồi thầy chấm	tiền tố lúc học bằng lúc dùng

13. Bài tập tự tính

Bài 1. $p = (0{,}5; 0{,}5)$, $q = (0{,}99; 0{,}01)$. Tính $\mathrm{KL}(p\|q)$ và $\mathrm{KL}(q\|p)$ (nat).

Đáp án: $\mathrm{KL}(p\|q) = 0{,}5\ln(0{,}5/0{,}99) + 0{,}5\ln(0{,}5/0{,}01) = 1{,}6145$; $\mathrm{KL}(q\|p) = 0{,}99\ln 1{,}98 + 0{,}01\ln 0{,}02 = 0{,}6371$ (tự tính). Khi trò gần như bỏ một đáp án, KL thuận rất lớn còn KL ngược vẫn dưới $\ln 2 = 0{,}693$.

Bài 2. Trong mô hình hai trạng thái của mục 6.2 với $\varepsilon = 0{,}01$ và không quay về, xác suất đã rơi vào trạng thái lệch sau 50 token là bao nhiêu? Sau 100 token?

Đáp án: $1 - 0{,}99^{50} = 0{,}395$ và $1 - 0{,}99^{100} = 0{,}634$ (tự tính).

Bài 3. Thầy là hỗn hợp $0{,}5\,N(-5, 1) + 0{,}5\,N(5, 1)$. Gauss tối ưu theo KL thuận có phương sai bao nhiêu? Và KL thuận của nó (số)?

Đáp án: moment: trung bình 0, phương sai $1 + 5^2 = 26$, nên $N(0, 26)$; $\mathrm{KL}(p\|q) \approx 0{,}94$ nat (tự tính, tích phân số).

Bài 4. $p = (0{,}5; 0{,}5)$, $q = (0{,}9; 0{,}1)$, $\alpha = 0{,}1$. Tính hỗn hợp $\alpha p + (1-\alpha)q$ và SKL.

Đáp án: hỗn hợp $(0{,}86; 0{,}14)$; SKL $= 0{,}5 \ln(0{,}5/0{,}86) + 0{,}5 \ln(0{,}5/0{,}14) = 0{,}3653$ (tự tính), nhỏ hơn $\mathrm{KL}(p\|q) = 0{,}5108$.

Bài 5. Với thầy hai đỉnh $0{,}5N(-3, 1) + 0{,}5N(3, 1)$ và trò $N(3, 1)$, vì sao $\mathrm{KL}(q\|p)$ xấp xỉ $\ln 2$? Giá trị chính xác?

Đáp án: trên vùng q có khối lượng, $p \approx 0{,}5\, q$ (đỉnh trái không đóng góp), nên $\ln(q/p) \approx \ln 2 = 0{,}6931$; giá trị số là 0,6893, hơi nhỏ hơn vì đuôi trái của q chồng lên một ít đuôi của đỉnh trái (tự tính).

Tài liệu tham khảo nền: ghi chú distillation 28, 29, 30 và 31 trong repo; math nền tảng 11 (KL); tài liệu về học bắt chước và exposure bias ở mức giáo trình.

Nền tảng 06

Chưng cất bằng dữ liệu và lập luận: thầy chỉ cần viết ra văn bản là học trò đã học được

Loại: chương nền tảng (không phải ghi chú paper)

Đọc trước: entropy, cross-entropy và KL (math nền tảng 11); mô hình sinh và cách một mô hình ngôn ngữ dự đoán token kế tiếp (ML cơ bản nền tảng 08); chưng cất bằng xác suất mềm (logit) và nhiệt độ ở nen01 của cuốn này và ở distillation 02; preface 00

Dùng ở: distillation 35 (Self-Instruct), distillation 36 (distilling step-by-step), distillation 37 (Orca), distillation 38 (chưng cất từ R1), distillation 39 (học trò nhỏ và lập luận mạnh)

> Chương này giải thích từ đầu, không phải bản dịch một paper. Các phép tính ghi "tự tính" đã kiểm bằng numpy; các hình ghi "mô phỏng" dùng dữ liệu tự sinh, không phải kết quả đo từ mô hình thật. Con số lấy từ paper luôn có chú "theo ghi chú distillation NN". Suy luận riêng của chương (không phải của paper) được ghi rõ là suy luận.

1. Vì sao cần chương này

Chưng cất không nhất thiết phải mở xác suất của thầy: chỉ cần thầy viết ra câu trả lời (và lời giải thích) là đã có thể dạy học trò, nhưng cái giá là phải kiểm soát chất lượng thứ văn bản ấy.

Hãy hình dung hai cách học nấu ăn. Cách thứ nhất: bạn đứng cạnh đầu bếp và thấy được mọi cân nhắc trong đầu họ ("muối thêm một chút, 70% chắc là đủ, 20% nên nêm thêm"). Cách thứ hai: đầu bếp chỉ nấu xong một nồi và đưa cho bạn nếm, kèm lời giải thích ngắn. Cách thứ nhất giàu thông tin hơn nhiều, nhưng cách thứ hai áp dụng được với mọi đầu bếp, kể cả đầu bếp chỉ trả lời qua cửa sổ nhỏ (một dịch vụ qua API chỉ trả văn bản).

Hầu hết các mô hình mạnh nhất hiện nay là mô hình đóng: ta chỉ gọi được và nhận văn bản, không có xác suất trên toàn từ vựng. Muốn chưng cất từ chúng, con đường duy nhất là **chưng cất dữ liệu**: nhờ thầy sinh ra dữ liệu huấn luyện, rồi huấn luyện học trò bằng cách giám sát bình thường (SFT, supervised fine-tuning). Chương này giúp bạn:

1. Phân biệt ba cách chưng cất (logit, dữ liệu, lập luận) và biết mỗi cách cần gì ở thầy.
2. Hiểu vì sao "học từ văn bản thầy sinh" thực chất là một ước lượng ngẫu nhiên của chưng cất logit.
3. Đếm được chi phí token và thông tin của một mẫu "nhãn ngắn" so với "giải thích từng bước".
4. Hiểu lọc theo đáp án đúng (rejection sampling) hoạt động ra sao, mất gì và còn lại gì.
5. Biết các rủi ro: học luôn lỗi và phong cách của thầy.

2. Nhắc nền cần thiết

2.1 Mô hình ngôn ngữ và hai cách dạy

Một mô hình ngôn ngữ cho ra, tại mỗi vị trí, một phân phối xác suất $p(\cdot \mid \text{ngữ cảnh})$ trên toàn từ vựng. Khi huấn luyện thông thường (SFT), mục tiêu là **one-hot**: chỉ token "đúng" có xác suất 1. Hàm mất mát là cross-entropy giữa mục tiêu one-hot và dự đoán q của học trò: $-\log q(\text{token đúng})$.

Chưng cất logit thay mục tiêu one-hot bằng phân phối mềm p_T của thầy: mất mát là $H(p_T, q) = -\sum_v p_T(v) \log q(v)$, hơn kém hằng số entropy $H(p_T)$ so với KL $D_{\mathrm{KL}}(p_T \| q)$ (xem math nền tảng 11).

2.2 Ví dụ số nhỏ: xác suất mềm so với một mẫu

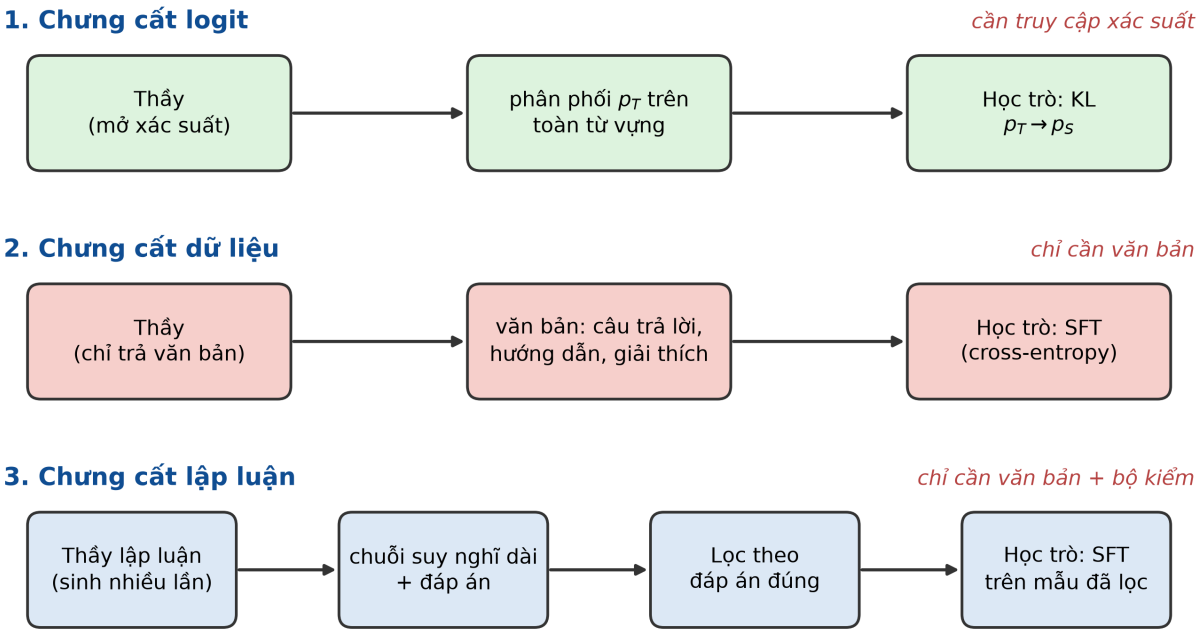
Giả sử tại một vị trí, thầy cho phân phối trên bốn token $p_T = (0{,}6;\ 0{,}25;\ 0{,}1;\ 0{,}05)$ và học trò hiện cho $q = (0{,}4;\ 0{,}3;\ 0{,}2;\ 0{,}1)$.

- **Có xác suất (logit):** cross-entropy mềm là $H(p_T, q) = 1{,}1268$ nat (tự tính); entropy của thầy $H(p_T) = 1{,}0331$ nat, nên $D_{\mathrm{KL}}(p_T \| q) = 0{,}0937$ nat (tự tính). Gradient theo logit của học trò là $q - p_T = (-0{,}20;\ 0{,}05;\ 0{,}10;\ 0{,}05)$ (tự tính).
- **Chỉ có một mẫu văn bản:** thầy sinh ra một token, giả sử là token thứ nhất. Mục tiêu là one-hot, gradient là $q - e_1 = (-0{,}60;\ 0{,}30;\ 0{,}20;\ 0{,}10)$ (tự tính), lệch xa gradient mềm ở mẫu này.

Nhưng nếu thầy sinh rất nhiều mẫu thì sao? Mất mát mỗi mẫu là $-\log q(v) = (0{,}916;\ 1{,}204;\ 1{,}609;\ 2{,}303)$ (tự tính) cho bốn token. Trung bình có trọng số xác suất của thầy: $0{,}6 \cdot 0{,}916 + 0{,}25 \cdot 1{,}204 + 0{,}1 \cdot 1{,}609 + 0{,}05 \cdot 2{,}303 = 1{,}1268$ (tự tính), **đúng bằng cross-entropy mềm**. Đây là điểm mấu chốt của chương: **huấn luyện trên văn bản thầy sinh là ước lượng Monte Carlo (lấy mẫu ngẫu nhiên) của chưng cất logit**. Mỗi mẫu rất nhiễu, nhưng trung bình đúng.

Độ nhiễu cụ thể: để ước lượng xác suất $0{,}25$ của token thứ hai, sai số chuẩn của tần suất là $\sqrt{0{,}25 \cdot 0{,}75 / N}$, tức $0{,}433$ với $N = 1$ mẫu, $0{,}137$ với $N = 10$, $0{,}043$ với $N = 100$ (tự tính). Muốn biết "thầy nghĩ gì" chính xác bằng văn bản phải sinh rất nhiều mẫu; đổi lại không cần mở xác suất.

3. Trực giác bằng hình: ba cách chưng cất



Từ trên xuống: lượng thông tin mỗi vị trí giảm, nhưng yêu cầu truy cập thầy cũng giảm

Hình 19: Sơ đồ ba cách chưng cất: chưng cất logit cần xác suất của thầy, chưng cất dữ liệu chỉ cần văn bản, chưng cất lập luận thêm bước lọc theo đáp án đúng (minh hoạ, tự tính)

Sơ đồ ba cách chưng cất: chưng cất logit cần xác suất của thầy, chưng cất dữ liệu chỉ cần văn bản, chưng cất lập luận thêm bước lọc theo đáp án đúng (minh hoạ, tự tính)

Cách đọc hình: mỗi hàng là một cách chưng cất, đi từ trái sang phải theo chiều tín hiệu chảy từ thầy sang học trò. Hàng trên cùng dùng cả phân phối trên từ vựng, nên giàu thông tin nhất nhưng đòi hỏi quyền truy cập xác

suất. Hàng giữa chỉ nhận văn bản và học bằng cross-entropy thông thường. Hàng dưới thêm một bước lọc: thầy sinh nhiều lần, ta chỉ giữ lời giải có đáp án đúng. Chữ đỏ nghiêng bên phải cho biết điều kiện cần ở thầy.

(Dòng chú thích dưới sơ đồ nói thông tin mỗi vị trí giảm dần từ trên xuống: đây là cách diễn đạt trực giác, vì ở hàng dưới cùng mỗi mẫu lại dài hơn rất nhiều, như phần 6 sẽ tính.)

4. Định nghĩa chính thức

Chưng cất logit (còn gọi chưng cất mềm, white-box): cực tiểu hoá

$$\mathcal{L}_{\text{logit}} = \mathbb{E}_x \Big[D_{\text{KL}}\big(p_T(\cdot \mid x) \,\|\, q_\theta(\cdot \mid x)\big) \Big].$$

Cần: xác suất (hoặc logit) của thầy tại mọi vị trí, và thường là cùng bộ tách từ (tokenizer) giữa thầy và trò.

Chưng cất dữ liệu (còn gọi chưng cất cứng, sequence-level, black-box): thầy sinh ra tập $\mathcal{D} = \{(x_i, y_i)\}$ với $y_i \sim p_T(\cdot \mid x_i)$ (hoặc giải mã tham lam). Học trò cực tiểu hoá

$$\mathcal{L}_{\text{data}} = -\frac{1}{|\mathcal{D}|} \sum_i \log q_\theta(y_i \mid x_i).$$

Cần: chỉ văn bản. Không cần cùng bộ tách từ, vì học trò tự tách lại y_i theo bộ tách từ của mình. Ý tưởng này là chưng cất mức chuỗi của Kim và Rush (xem distillation 15).

Chưng cất lập luận là chưng cất dữ liệu, trong đó y_i gồm cả chuỗi suy nghĩ trung gian (rationale, chain of thought, CoT) lẫn đáp án cuối, và tập dữ liệu thường được **lọc** bằng một bộ kiểm (đáp án đúng, chạy được mã, qua máy chấm) trước khi huấn luyện. Nếu đặt $c(y) \in \{0, 1\}$ là kết quả kiểm, tập sau lọc là $\mathcal{D}' = \{(x, y) \in \mathcal{D} : c(y) = 1\}$.

Nhắc thuật ngữ. *Teacher* (thầy), *student* (trò), *rationale* (đoạn giải thích trung gian dẫn từ đề tới đáp án), *CoT* (chuỗi suy nghĩ), *rejection sampling* (sinh nhiều lần, giữ lại mẫu đạt tiêu chí), *SFT* (tinh chỉnh có giám sát bằng cross-entropy).

5. Ví dụ tay 1: cùng một bài toán, nhãn ngắn và giải thích từng bước

Bài toán: **"Một cửa hàng bán 12 cây bút, mỗi cây 7 nghìn đồng, giảm 10% tổng hóa đơn. Phải trả bao nhiêu?"** Đáp án: $12 \times 7 = 84$, giảm 10% của 84 là 8,4, còn $84 - 8{,}4 = 75{,}6$ nghìn đồng.

Hai dạng mẫu huấn luyện:

- *Nhãn ngắn:* `75,6`
- *Giải thích từng bước:* `Bước 1: 12 × 7 = 84 (nghìn). Bước 2: giảm 10% của 84 là 8,4. Bước 3: 84 - 8,4 = 75,6. Đáp án: 75,6.`

Đếm token. Để tính đơn giản, ta quy ước một token là một từ, một số (kể cả số có dấu phẩy) hoặc một ký hiệu. Đây chỉ là quy ước để đếm cho dễ, không phải bộ tách từ thật của một mô hình; số token thật sẽ khác nhưng tỉ lệ cho cùng một thông điệp.

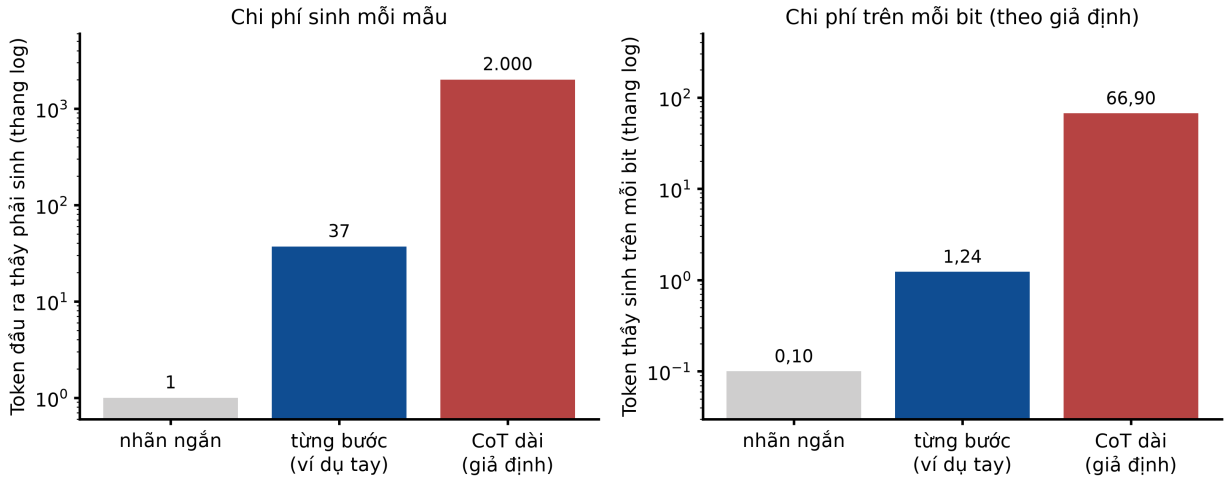
- Đề bài: 26 token (tự tính).
- Nhãn ngắn: 1 token (tự tính).
- Giải thích từng bước: 37 token (tự tính).

Mẫu giải thích dài gấp 37 lần về số token thầy phải sinh và số token học trò phải học (đo ở phần đầu ra).

Thông tin mỗi mẫu (giả định đơn giản). Giả sử mọi số trong bài là một giá trị đồng đều trong 1000 khả năng. Khi đó mỗi số chứa $\log_2 1000 \approx 9{,}97$ bit (tự tính). Nhãn ngắn chỉ cho học trò một số (đáp án): khoảng

9,97 bit. Lời giải từng bước cho ba số (84, 8,4, 75,6): khoảng 29,90 bit (tự tính), gấp ba. Đây là giả định để minh hoạ, không phải phép đo; số thật phụ thuộc vào điều học trò đã biết.

Chi phí trên mỗi bit. Nhãn ngắn: $1/9{,}97 = 0{,}10$ token mỗi bit. Giải thích: $37/29{,}90 = 1{,}24$ token mỗi bit, tức đắt gấp khoảng 12,3 lần mỗi bit thông tin (tự tính). Nếu là một CoT dài khoảng 2000 token (con số giả định) chứa cùng ba số thì tỉ lệ là $2000/29{,}9 \approx 66{,}9$ token mỗi bit (tự tính).



Hình 20: Chi phí sinh mỗi mẫu và chi phí trên mỗi bit cho nhãn ngắn, giải thích từng bước (ví dụ tay) và CoT dài (giả định 2000 token) (minh hoạ, tự tính)

Chi phí sinh mỗi mẫu và chi phí trên mỗi bit cho nhãn ngắn, giải thích từng bước (ví dụ tay) và CoT dài (giả định 2000 token) (minh hoạ, tự tính)

Cách đọc hình: cả hai trục tung đều ở thang log. Panel trái cho thấy số token thầy phải sinh cho một mẫu: 1, 37 và 2000. Panel phải chia số đó cho số bit giả định: mỗi bit thông tin "đắt" hơn khi lời giải càng dài. Cột thứ ba dựa trên giả định 2000 token và ba số quan trọng, nên chỉ để minh hoạ xu hướng.

Kết luận của ví dụ (suy luận của chương). Nếu chỉ tính bit của đáp án, giải thích từng bước đắt hơn nhãn ngắn. Vậy tại sao người ta vẫn dùng? Vì cái giá trị của lời giải không nằm ở ba con số của riêng bài này mà ở **thủ tục** (nhân giá với số lượng, rồi lấy phần trăm) có thể áp dụng cho bài toán khác. Nhãn ngắn dạy "đề này ra 75,6", không dạy cách làm. Đây là lý do distilling step-by-step cần ít mẫu hơn (xem phần 8).

6. Hai bài toán cụ thể của chưng cất bằng văn bản

6.1 Khác biệt giữa chưng cất logit và chưng cất dữ liệu

Tiêu chí	Chưng cất logit	Chưng cất dữ liệu
Thầy phải cung cấp	xác suất trên từ vựng	chỉ văn bản
Thông tin mỗi vị trí	cả phân phối	một token được lấy mẫu
Bộ tách từ	thường phải chung	không cần chung
Lưu trữ	rất lớn nếu lưu sẵn	nhỏ (chỉ văn bản)
Dùng được với API đóng	không	có
Mất mát có phương sai	thấp	cao (mỗi mẫu nhiễu)

Ví dụ lưu trữ (tự tính). Với 1 tỉ token và từ vựng 32000, lưu đủ logit dạng nửa chính xác (2 byte) mất $10^9 \times 32000 \times 2 = 6{,}4 \times 10^{13}$ byte, tức 64 TB; lưu chỉ id token (4 byte) mất 4 GB, chênh 16000 lần. Theo

ghi chú distillation 42, lưu đủ logit float32 cho khoảng 2 nghìn tỉ token C4 với từ vựng 32.168 tốn khoảng 260 PB; theo ghi chú distillation 43, 50 tỉ token với từ vựng 150 nghìn tốn khoảng 30 PB. Đó là lý do ra đời các cách lưu thưa (top-K, lấy mẫu) mà ghi chú distillation 45 bàn.

6.2 Lọc theo đáp án đúng: ví dụ tay 2

Ta có 5 bài toán, thầy sinh mỗi bài $k = 4$ lời giải (tổng 20 mẫu). Số lời giải đúng cho mỗi bài là $(4, 3, 2, 1, 0)$.

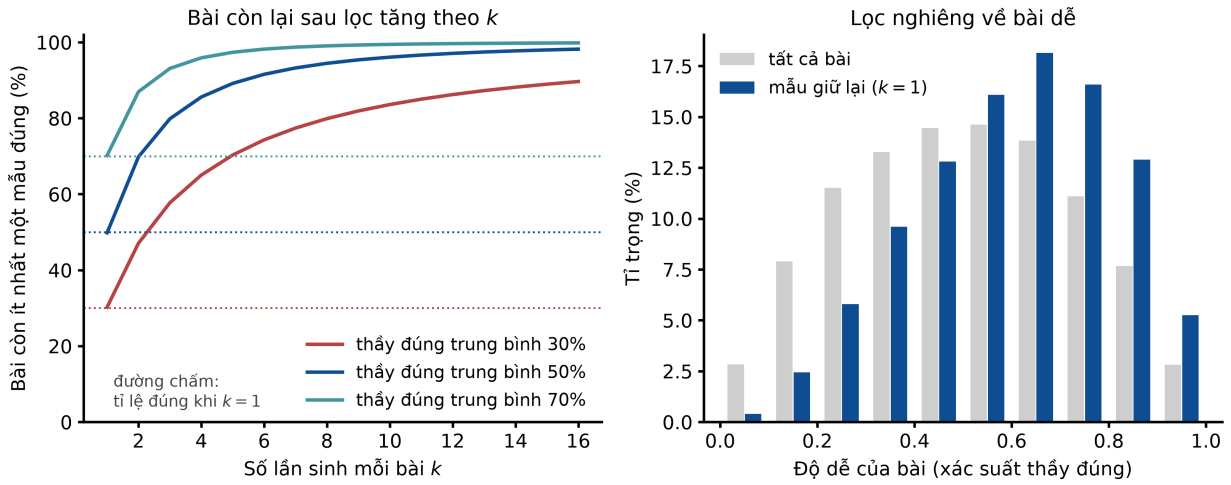
- Tổng số mẫu đúng: $4 + 3 + 2 + 1 + 0 = 10$, tức giữ lại $10/20 = 50\%$ số mẫu (tự tính).
- Số bài còn ít nhất một mẫu: 4 trên 5, tức 80% (tự tính).
- Thành phần tập sau lọc theo bài: $(40\%, 30\%, 20\%, 10\%, 0\%)$ (tự tính). Bài dễ nhất (thầy giải đúng cả 4 lần) chiếm 40% tập giữ lại dù chỉ là 1/5 số bài. Bài khó nhất (thầy luôn sai) biến mất hoàn toàn.
- Nếu giới hạn tối đa 2 mẫu mỗi bài để tránh lệch về bài dễ: giữ $\min(c, 2)$ mẫu mỗi bài, được $2 + 2 + 2 + 1 + 0 = 7$ mẫu (tự tính).

Điều này nêu ra hai hiệu ứng: **lọc làm tập dữ liệu nghiêng về bài dễ**, và **bài thầy giải không nổi thì học trò cũng không học được từ thầy**.

Lọc không hoàn hảo: đáp án đúng mà lập luận sai. Giả sử trong 1000 mẫu, 600 mẫu có lập luận đúng. Trong 400 mẫu còn lại, lập luận sai nhưng có 10% rơi vào đúng đáp án (đoán mò hoặc bù trừ sai), tức 40 mẫu. Bộ lọc "chỉ nhìn đáp án" giữ $600 + 40 = 640$ mẫu; độ chính xác của lập luận trong tập giữ lại là $600/640 = 93{,}75\%$ (tự tính). Với câu trắc nghiệm 4 lựa chọn, khả năng đoán trúng là 25%, độ chính xác tụt còn $600/(600 + 100) = 85{,}7\%$ (tự tính). Đây là suy luận của chương dựa trên số giả định; mức lẫn thật phụ thuộc vào loại bài.

6.3 Mô phỏng tỉ lệ mẫu còn lại sau lọc

Ta mô phỏng 20000 bài với độ khó khác nhau: xác suất thầy giải đúng bài i là p_i rút từ phân phối Beta (seed cố định, dữ liệu tự sinh). Với k lần sinh, xác suất bài đó còn ít nhất một mẫu đúng là $1 - (1 - p_i)^k$.



Hình 21: Mô phỏng: bài còn lại sau lọc theo số lần sinh k, và độ khó của mẫu giữ lại so với toàn bộ bài (minh hoạ, tự tính)

Mô phỏng: bài còn lại sau lọc theo số lần sinh k, và độ khó của mẫu giữ lại so với toàn bộ bài (minh hoạ, tự tính)

Cách đọc hình: trái là tỉ lệ bài còn ít nhất một lời giải đúng khi tăng số lần sinh k, cho ba thầy có độ đúng trung bình 30%, 50%, 70%; các đường chấm là tỉ lệ đúng khi $k = 1$. Với thầy 50%: khoảng 86% bài còn lại ở $k = 4$ và khoảng 98% ở $k = 16$ (mô phỏng). Với thầy 30%, $k = 16$ chỉ được khoảng 90%. Phải: với $k = 1$,

các cột xanh (mẫu giữ lại) lệch sang bài dễ so với các cột xám (mọi bài): cột bài dễ nhất (xác suất đúng 0,9 đến 1,0) tăng từ khoảng 2,8% lên 5,3% trong khi cột bài khó nhất (0 đến 0,1) giảm từ 2,8% xuống 0,4% (mô phỏng).

Bài học (suy luận): tăng k cứu được các bài "khó vừa" nhưng tốn thêm chi phí sinh gấp k lần; nó không cứu bài thầy gần như không bao giờ đúng.

7. Các hình thái chưng cất bằng văn bản

Self-Instruct: tự sinh cả đề bài. Thay vì chỉ sinh câu trả lời cho đề có sẵn, thầy còn sinh luôn đề (instruction), bắt đầu từ một nhóm ít ví dụ do người viết, sinh thêm, lọc trùng, rồi lặp. Theo ghi chú distillation 35, bắt đầu từ 175 tác vụ người viết, quy trình thu khoảng 52K instruction và 82K instance; kiểm tay một mẫu cho thấy 92% instruction hợp lệ nhưng chỉ khoảng 54% mẫu có mọi trường đều hợp lệ (đầu ra đúng khoảng 58%). Bộ lọc dựa trên độ trùng bề mặt (ROUGE-L dưới 0,7 so với các instruction đã có).

Orca: dạy cả cách nghĩ. Thầy được yêu cầu bằng "system message" giải thích từng bước ("think step-by-step and justify your steps"), nên mỗi mẫu chứa quá trình suy luận thay vì chỉ đáp án. Theo ghi chú distillation 37, dữ liệu gồm 5 triệu phản hồi của ChatGPT, trong đó 1 triệu cũng có phản hồi từ GPT-4, và học trò học tuần tự: trước từ ChatGPT, sau từ GPT-4, như một chương trình học (curriculum) vì lý do khoảng cách năng lực.

Distilling step-by-step: rationale là nhiệm vụ phụ. Thầy cho nhãn và rationale; học trò nhỏ học hai nhiệm vụ (dự đoán nhãn, và sinh rationale) phân biệt bằng tiền tố. Lúc suy luận chỉ cần nhãn, nên không phải sinh rationale. Theo ghi chú distillation 36, với nhãn người, chỉ 12,5% dữ liệu e-SNLI đã vượt tinh chỉnh chuẩn dùng 100%.

Chưng cất lập luận kiểu R1. Thầy là mô hình lập luận đã được huấn luyện bằng học tăng cường để sinh chuỗi suy nghĩ dài; thầy sinh nhiều lần mỗi đề, chỉ giữ lời giải đúng, rồi học trò SFT. Theo ghi chú distillation 38, tập chưng cất khoảng 800K mẫu, trong đó khoảng 600K là dữ liệu lập luận (đã lọc bằng rejection sampling, loại CoT trộn ngôn ngữ, đoạn quá dài, khối mã) và khoảng 200K là dữ liệu không lập luận.

8. Nối sang các bài paper của cuốn

- **distillation 35 (Self-Instruct).** Là chưng cất dữ liệu ở dạng "mầm": từ rất ít mẫu người viết, nhờ thầy tự sinh cả đề lẫn đáp án. Hãy chú ý bảng chất lượng của ghi chú (54% mẫu hợp lệ hoàn toàn) và tự hỏi bộ lọc có ngăn được lỗi không; nó cho thấy rủi ro phần 9 là thực.
- **distillation 36 (distilling step-by-step).** Là chính xác ví dụ tay 1 ở quy mô thực: lời giải kèm nhãn. Theo ghi chú, mô hình T5 nhỏ có rationale vượt cả mô hình lớn gọi few-shot ở một số tập; cơ chế được suy luận ở phần 5 (học thủ tục).
- **distillation 37 (Orca).** Thêm hai ý: system message đa dạng để lấy các kiểu giải thích khác nhau, và học tuần tự từ thầy trung gian rồi tới thầy mạnh. Ghi chú nêu rõ với mô hình đóng chỉ có văn bản, đây là loại chưng cất khả thi.
- **distillation 38 (R1).** Là chưng cất lập luận kèm lọc theo đáp án đúng, ở quy mô 800K mẫu. Ghi chú cũng nhấn mạnh so sánh "chưng cất thắng RL" chưa tính chi phí sinh dữ liệu và chi phí thầy.
- **distillation 39 (học trò nhỏ và lập luận mạnh).** Chỉ ra rằng không phải thầy lập luận dài nào cũng tốt cho mọi học trò: theo ghi chú, học trò nhỏ (dưới hoặc bằng 3B) học tốt hơn từ chuỗi ngắn, học trò lớn (từ 7B) học tốt hơn từ chuỗi dài; cùng với phần 6.3, đây là lý do nên chọn độ dài lập luận theo sức học trò.

9. Rủi ro, biến thể và chi tiết hay bị bỏ qua

Chép lỗi của thầy. Học trò học y_i như thể đúng. Nếu thầy sai và không có bộ lọc, học trò học luôn câu sai, thậm chí tự tin hơn. Với mẫu "đáp án đúng nhưng lập luận sai" (6.2), lỗi này lọt qua bộ lọc đáp án.

Chép phong cách. Học trò dễ bắt chước giọng văn, độ dài, cách mở đầu của thầy (ví dụ giọng "Tất nhiên, tôi sẽ giúp...") mà không học được năng lực tương ứng. Hãy đánh giá bằng bài toán có đáp án kiểm được, không chỉ bằng độ giống giọng thầy (suy luận của chương).

Đa dạng thấp. Khi thầy giải mã tham lam hoặc nhiệt độ thấp, dữ liệu rất giống nhau, học trò chỉ thấy một nhánh lập luận. Khi nhiệt độ cao thì tăng đa dạng nhưng cũng tăng lỗi. Cân bằng này là chi tiết hay bị bỏ qua.

Lệch về bài dễ. Như đã tính ở 6.2 và 6.3: lọc làm tập huấn luyện nghiêng về bài dễ, bài khó bị thiếu đúng chỗ ta muốn học trò tiến bộ.

Rò rỉ dữ liệu kiểm. Nếu đề lấy từ nguồn có chứa đề thi, thầy có thể đã thấy đáp án; mẫu sinh ra mang theo nhiễm bẩn và đánh giá học trò sẽ đẹp giả.

Giấy phép và điều khoản. Nhiều dịch vụ mô hình đóng cấm dùng đầu ra để huấn luyện mô hình cạnh tranh; hãy kiểm tra trước khi chưng cất (lưu ý thực hành).

Chất lượng lọc theo nhiều tầng. Ngoài đáp án đúng, các bộ lọc thường gặp gồm: độ dài hợp lý, không trộn ngôn ngữ, không lặp, định dạng đúng, độ trùng thấp với mẫu đã có, máy chấm hoặc thực thi mã. Theo ghi chú distillation 38, bài báo không có ablation riêng cho từng bước lọc, nên ta không biết mỗi bước đóng góp bao nhiêu.

Chi phí của thầy. Mỗi token thầy sinh tốn khoảng $2N_T$ phép tính (với N_T là số tham số của thầy), trong khi học trò học một token tốn khoảng $6N_S$. Với một ví dụ (tự tính): thầy 30B sinh 10^6 mẫu, mỗi mẫu 500 token, tốn $2 \times 30 \times 10^9 \times 5 \times 10^8 = 3 \times 10^{19}$ phép tính; học trò 3B huấn luyện một epoch trên đúng tập này tốn $6 \times 3 \times 10^9 \times 5 \times 10^8 = 9 \times 10^{18}$ phép tính. Chi phí sinh gấp $3{,}33$ lần chi phí học, và tổng gấp $4{,}33$ lần chi phí học thuần (tự tính). Chương nền 07 của cuốn này đi sâu vào phép tính ngân sách như thế.

10. Sai lầm hay gặp

- Nghĩ rằng chưng cất dữ liệu "không phải chưng cất thật". Thực ra nó là ước lượng ngẫu nhiên của chưng cất logit (phần 2.2), chỉ nhiễu hơn.
- Coi đáp án đúng là bằng chứng lập luận đúng. Mẫu lọt lưới (6.2) làm học trò học lập luận hỏng có đáp án đúng.
- Quên rằng lọc làm lệch phân phối bài khó/dễ, rồi ngạc nhiên vì học trò yếu ở bài khó.
- So sánh "nhãn ngắn" với "giải thích" mà không nói rõ số token và số mẫu: hai dạng mẫu khác nhau hàng chục lần về token nên phải so sánh ở cùng ngân sách.
- Dùng một độ dài CoT cho mọi học trò, trong khi học trò nhỏ có thể học tốt hơn từ chuỗi ngắn (theo ghi chú distillation 39).
- Quên tính chi phí sinh dữ liệu của thầy khi tuyên bố "chưng cất rẻ".

11. Tóm tắt công thức

Ý	Công thức hoặc kết luận
Chưng cất logit	$\mathcal{L}_{\text{logit}} = \mathbb{E}[D_{\text{KL}}(p_T \Vert q_\theta)]$
Chưng cất dữ liệu	$\mathcal{L}_{\text{data}} = -\frac{1}{\lvert\mathcal{D}\rvert} \sum \log q_\theta(y_i \mid x_i)$
Quan hệ	mẫu của thầy $y \sim p_T$ làm $\mathcal{L}_{\text{data}}$ là ước lượng Monte Carlo của cross-entropy mềm
Sai số chuẩn tần suất	$\sqrt{p(1-p)/N}$
Phần bài còn sau lọc	$1 - (1 - p_i)^k$
Chi phí sinh mỗi token	khoảng $2N_T$
Chi phí học mỗi token	khoảng $6N_S$

12. Bài tập tự tính

Bài 1. Thầy sinh $k = 4$ lời giải cho mỗi trong 10 bài; số lời giải đúng mỗi bài là $(4, 4, 3, 2, 1, 0, 0, 0, 1, 1)$. Tính tỉ lệ mẫu còn lại sau lọc theo đáp án đúng và tỉ lệ bài còn ít nhất một mẫu.

Đáp án: tổng mẫu đúng $= 16$ trên 40 mẫu, tỉ lệ 40%. Số bài còn ít nhất một mẫu: 6 trên 10, tức 60% (tự tính).

Bài 2. Trong 500 mẫu, 55% có lập luận đúng. Trong số lập luận sai, 20% vẫn cho ra đáp án đúng nên lọt bộ lọc đáp án. Độ chính xác của lập luận trong tập giữ lại?

Đáp án: đúng: 275; sai lọt lưới: $225 \times 0{,}2 = 45$. Độ chính xác $275/320 = 85{,}9\%$ (tự tính).

Bài 3. Tại một vị trí, thầy cho $p_T = (0{,}5;\ 0{,}3;\ 0{,}2)$ và học trò cho $q = (0{,}25;\ 0{,}25;\ 0{,}5)$. Tính cross-entropy mềm $H(p_T, q)$, entropy $H(p_T)$ và KL (đơn vị nat), rồi kiểm tra rằng trung bình của $-\log q(v)$ theo p_T đúng bằng cross-entropy mềm.

Đáp án: $-\log q = (1{,}386;\ 1{,}386;\ 0{,}693)$. Trung bình có trọng số: $0{,}5 \cdot 1{,}386 + 0{,}3 \cdot 1{,}386 + 0{,}2 \cdot 0{,}693 = 1{,}2477 = H(p_T, q)$. Entropy $H(p_T) = 1{,}0297$; KL $= 1{,}2477 - 1{,}0297 = 0{,}2180$ nat (tự tính).

Bài 4. Lưu đủ logit nửa chính xác (2 byte) cho 2 tỉ token với từ vựng 50000 tốn bao nhiêu, so với lưu id token 4 byte?

Đáp án: $2 \times 10^9 \times 50000 \times 2 = 2 \times 10^{14}$ byte, tức 200 TB; id token: 8×10^9 byte, tức 8 GB; chênh 25000 lần (tự tính).

Bài 5. Thầy 30B sinh 10^6 mẫu mỗi mẫu 500 token; học trò 3B học một epoch. Tính tỉ lệ chi phí sinh trên chi phí học, rồi tính tổng so với chi phí học thuần, bằng công thức $1 + N_T/(3N_S)$.

Đáp án: sinh: 3×10^{19}; học: 9×10^{18}; tỉ lệ $3{,}33$. Tổng so với học thuần: $1 + 30/(3 \cdot 3) = 4{,}33$ lần (tự tính).

Tài liệu tham khảo nền: Hinton, Vinyals, Dean 2015 và Kim, Rush 2016 (trong repo: distillation 02, distillation 15); các ghi chú distillation 35 đến distillation 39; distillation 27 cho bức tranh tổng quan về chưng cất LLM.

Nền tảng 07

Đo chưng cất, scaling và khi nào nên chưng cất: tính đủ chi phí rồi mới kết luận

Loại: chương nền tảng (không phải ghi chú paper)

Đọc trước: nen06 của cuốn này (chưng cất bằng dữ liệu và lập luận, chi phí sinh của thầy); số học FLOP và tỉ lệ (math nền tảng 12 và 13); loss, overfitting và khái niệm khái quát (ML cơ bản nền tảng 03); batch và ngân sách huấn luyện (optimization nền 07); preface 00

Dùng ở: distillation 40 (Gemma 2), distillation 41 (quy luật khoảng cách năng lực), distillation 42 (scaling law của chưng cất), distillation 43 (MiniPLM), distillation 44, distillation 45, distillation 46, distillation 47

> Chương này giải thích từ đầu, không phải bản dịch một paper. Các phép tính ghi "tự tính" đã kiểm bằng numpy. Các đường cong trong hình là **mô phỏng với tham số giả định** (ghi rõ trong chương), dùng để minh hoạ dạng lập luận, không phải số đo từ mô hình thật. Con số lấy từ paper luôn có chú "theo ghi chú distillation NN". Suy luận riêng của chương được ghi rõ là suy luận.

1. Vì sao cần chương này

Một bài báo chưng cất nào cũng có thể khoe "học trò chưng cất tốt hơn học trò huấn luyện từ đầu"; câu hỏi thật là: tốt hơn ở cùng cái giá nào, và có tính tiền của thầy chưa?

Hãy hình dung một lớp học thêm. Một em đến lớp học thêm và thi tốt hơn em tự học. Nhưng nếu lớp học thêm phải thuê giáo viên giỏi, thuê phòng, mua tài liệu, còn em tự học chỉ tốn tiền sách, thì ta có thật sự biết cách nào "hiệu quả" hơn? Chỉ khi cộng toàn bộ chi phí và so sánh ở cùng tổng ngân sách ta mới biết.

Chưng cất đặt ra đúng bài toán này. Thầy phải được huấn luyện (hoặc đã có sẵn), phải chạy suy luận để sinh xác suất hoặc văn bản, rồi học trò mới học. Chương này dạy bạn:

1. So sánh **công bằng**: học trò từ đầu và học trò chưng cất ở cùng ngân sách.
2. Tính **chi phí của thầy** (sinh logit hoặc sinh mẫu) so với chi phí của trò, bằng một công thức đơn giản.
3. Hiểu **khoảng cách năng lực** (capacity gap) và các **quy luật scaling** của chưng cất ở mức ý tưởng.
4. Có **quy tắc quyết định**: khi nào chưng cất đáng, khi nào từ đầu hoặc cắt tỉa tốt hơn.
5. Có **danh sách kiểm báo cáo trung thực** để đọc và viết bài chưng cất.

2. Nhắc nền cần thiết

2.1 Chi phí tính toán xấp xỉ theo số tham số và số token

Một công thức thô nhưng rất hữu ích: với mô hình N tham số, một lượt **suy luận** (forward) trên một token tốn khoảng $2N$ phép tính (FLOP), còn một bước **huấn luyện** (forward + backward) trên một token tốn khoảng $6N$ FLOP. Số $6N$ gồm $2N$ cho forward và khoảng $4N$ cho backward. Đây là xấp xỉ thông dụng (bỏ qua số hạng attention), không phải con số chính xác của một kiến trúc cụ thể.

Ví dụ tay. Mô hình 1 tỉ tham số huấn luyện trên 100 tỉ token tốn khoảng $6 \times 10^9 \times 10^{11} = 6 \times 10^{20}$ FLOP (tự tính).

2.2 Quy luật scaling của loss

Mô hình ngôn ngữ lớn có loss giảm đều đặn khi tăng số tham số N và số token D. Một dạng hàm quen thuộc (kiểu Chinchilla, xem ghi chú distillation 40 và 42):

$$L(N, D) = E + \frac{A}{N^{\alpha}} + \frac{B}{D^{\beta}},$$

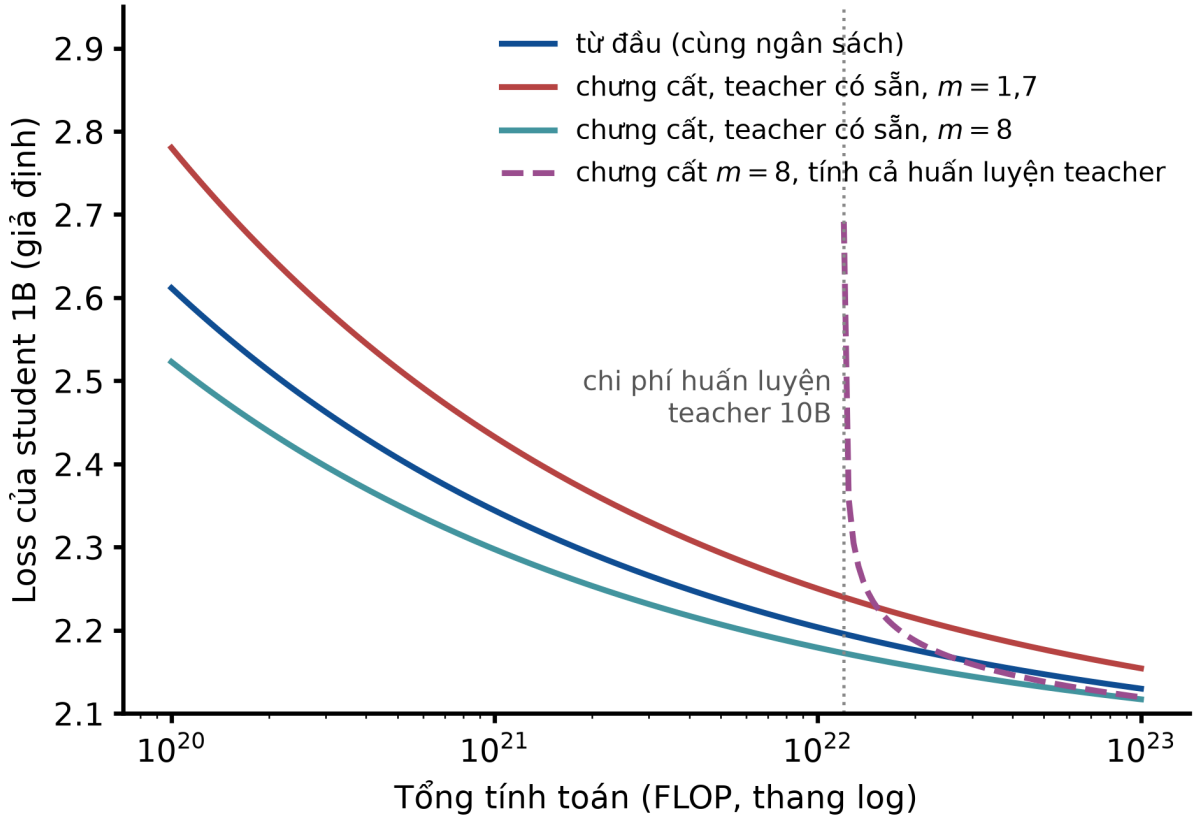
với E là phần nhiễu không thể giảm, hai số hạng sau là phần "thiếu mô hình" và "thiếu dữ liệu". Theo ghi chú distillation 42, tỉ lệ compute-optimal ở dạng này khoảng 20 token cho mỗi tham số.

Trong chương này ta dùng dạng hàm trên làm **mô hình đồ chơi** với tham số giả định $E = 1{,}7$, $A = 400$, $B = 410$, $\alpha = 0{,}34$, $\beta = 0{,}28$. Đây là số giả định để minh hoạ, không phải kết quả fit từ dữ liệu thật; mọi con số loss trong hình và ví dụ tay là từ mô hình đồ chơi này (mô phỏng).

2.3 Hệ số dữ liệu hiệu dụng

Để mô hình hoá chưng cất, ta dùng một giả định đơn giản (suy luận của chương): một token chưng cất có giá trị bằng m token huấn luyện từ đầu, nghĩa là học trò chưng cất với D token có loss $L(N,\, mD)$. Hệ số $m > 1$ nói lên "tín hiệu mỗi token giàu hơn". Đây là một **giả định**, vì ở thực tế m phụ thuộc vào ngân sách, cỡ học trò và chất lượng thầy; ghi chú distillation 42 dùng một dạng hàm phức tạp hơn nhiều. Mục đích của m là để đọc ra điều kiện hoà vốn một cách rõ ràng.

3. Trực giác bằng hình: cùng ngân sách, đường nào thấp hơn



Hình 22: Mô phỏng: loss của student 1B theo tổng tính toán, so sánh từ đầu và chưng cất (hệ số m giả định, có hoặc không tính huấn luyện teacher) (minh hoạ, tự tính)

Mô phỏng: loss của student 1B theo tổng tính toán, so sánh từ đầu và chưng cất (hệ số m giả định, có hoặc

không tính huấn luyện teacher) (minh hoạ, tự tính)

Cách đọc hình: trục hoành là **tổng tính toán** (FLOP, thang log), trục tung là loss của học trò 1B (mô hình đồ chơi, giả định). Đường xanh dương là học trò từ đầu. Đường đỏ và đường xanh ngọc là học trò chưng cất từ thầy 10B **đã có sẵn** (chỉ trả chi phí suy luận của thầy), với $m = 1{,}7$ và $m = 8$. Đường tím nét đứt là chưng cất $m = 8$ nhưng **tính cả chi phí huấn luyện thầy**, nên chỉ bắt đầu ở bên phải đường chấm (chi phí huấn luyện thầy, khoảng $1{,}2 \times 10^{22}$). Chỗ đường chưng cất nằm **dưới** đường xanh dương là chỗ chưng cất đáng.

Có ba điều cần đọc ra. Một, với $m = 1{,}7$ đường đỏ nằm trên đường xanh dương ở mọi ngân sách: chưng cất thua, vì mỗi token của thầy tốn thêm, trong khi giá trị mỗi token chỉ tăng 1,7 lần. Hai, với $m = 8$ đường xanh ngọc nằm dưới: chưng cất thắng ở mọi ngân sách trong hình. Ba, khi tính cả huấn luyện thầy, đường tím chỉ vượt đường từ đầu sau khoảng $2{,}6 \times 10^{22}$ FLOP trong mô hình này; ở chỗ thấp hơn, tiền thầy ăn hết lợi ích. Đó là cách đọc đúng của câu "chưng cất tốt hơn", cũng là lý do ghi chú distillation 40 nhấn rằng "cùng số token không có nghĩa cùng compute".

4. Định nghĩa chính thức

Chi phí chưng cất (thầy có sẵn). Với học trò N_S tham số huấn luyện D_S token, thầy N_T tham số chạy suy luận trên cùng D_S token:

$$C_{\text{dist}} = 6N_S D_S + 2N_T D_S, \qquad C_{\text{sup}} = 6N_S D_S.$$

Tỉ lệ chi phí mỗi token (từ đầu so với chưng cất):

$$\frac{C_{\text{dist}}}{C_{\text{sup}}} = 1 + \frac{N_T}{3N_S}.$$

Chi phí khi phải huấn luyện thầy. Nếu thầy N_T phải được huấn luyện (ví dụ $D_T = 20N_T$ token) chỉ cho học trò này:

$$C_{\text{total}} = 6N_S D_S + 2N_T D_S + 6N_T D_T.$$

Nếu thầy được dùng cho K học trò và chi phí huấn luyện thầy chia đều, mỗi học trò chịu thêm $6N_T D_T/K$.

Điều kiện hoà vốn (suy luận của chương, trong mô hình hệ số m). Chưng cất với D_S token có loss $L(N_S, mD_S)$. Từ đầu ở **cùng tổng chi phí** huấn luyện được $D_{\text{sup}} = C_{\text{dist}}/(6N_S) = D_S\,(1 + N_T/(3N_S))$ token. Vì L giảm theo D, chưng cất thắng khi và chỉ khi

$$m > 1 + \frac{N_T}{3N_S}.$$

Nếu tính cả huấn luyện thầy, ngưỡng còn cao hơn: $m > 1 + \frac{N_T}{3N_S} + \frac{6N_T D_T}{6N_S D_S K}$.

Thuật ngữ. *Biên Pareto* của tập các cấu hình (chi phí, loss) là tập các cấu hình mà không có cấu hình nào khác vừa rẻ hơn vừa tốt hơn. Một cấu hình nằm trên biên khi ta không thể cải thiện loss mà không tốn thêm chi phí.

5. Ví dụ tay: ngân sách tính toán tổng, so hai phương án

Đặt bài toán (tham số giả định): học trò $N_S = 1$ tỉ tham số, $D_S = 100$ tỉ token; thầy $N_T = 10$ tỉ tham số.

Chi phí.

- Phương án A, từ đầu: $6 \times 10^9 \times 10^{11} = 6 \times 10^{20}$ FLOP (tự tính).
- Chi phí suy luận của thầy trên 100 tỉ token: $2 \times 10^{10} \times 10^{11} = 2 \times 10^{21}$ FLOP (tự tính).

- Phương án B, chưng cất với thầy có sẵn: $6 \times 10^{20} + 2 \times 10^{21} = 2{,}6 \times 10^{21}$ FLOP, tức gấp $2{,}6 \times 10^{21}/6 \times 10^{20} = 4{,}33$ lần phương án A, trùng với công thức $1 + N_T/(3N_S) = 1 + 10/3 = 4{,}33$ (tự tính).
- Phương án C, chưng cất và phải huấn luyện thầy ($D_T = 20N_T = 2 \times 10^{11}$ token): huấn luyện thầy tốn $6 \times 10^{10} \times 2 \times 10^{11} = 1{,}2 \times 10^{22}$ FLOP (tự tính), tổng $2{,}6 \times 10^{21} + 1{,}2 \times 10^{22} = 1{,}46 \times 10^{22}$ FLOP, gấp $24{,}3$ lần phương án A (tự tính).

So sánh công bằng. Thay vì so "cùng 100 tỉ token", hãy cho phương án A **cùng ngân sách** $2{,}6 \times 10^{21}$ FLOP với B: A huấn luyện được $2{,}6 \times 10^{21}/(6 \times 10^9) = 4{,}33 \times 10^{11}$ token, tức 433 tỉ token (tự tính). Nếu chấp nhận ngân sách của C, A huấn luyện được $1{,}46 \times 10^{22}/(6 \times 10^9) \approx 2433$ tỉ token (tự tính).

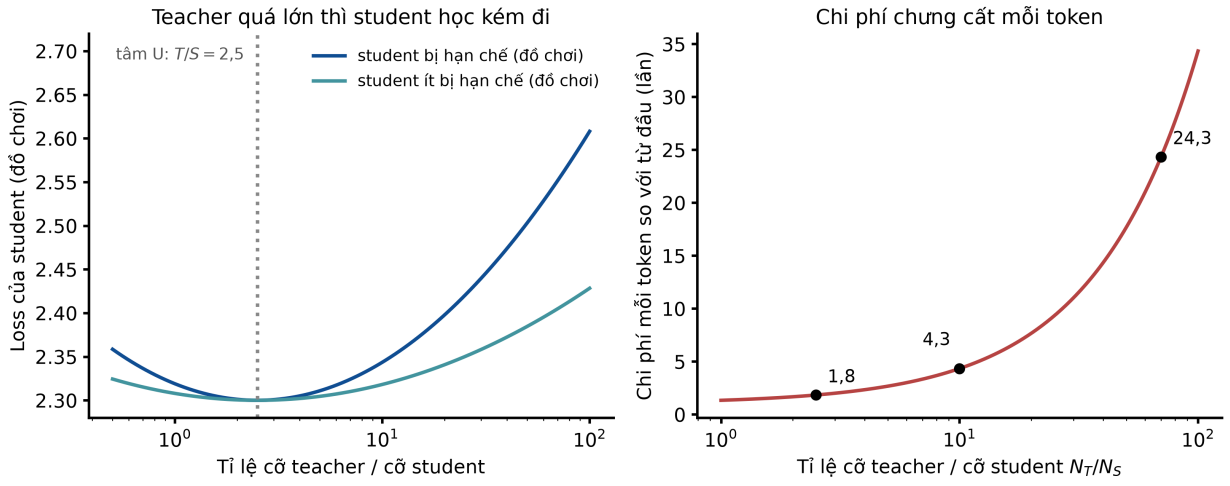
Loss theo mô hình đồ chơi (mô phỏng, tham số giả định ở 2.2).

Cấu hình	Token (tương đương)	Loss (mô phỏng)
A, từ đầu, 100 tỉ token	100 tỉ	2,389
B, chưng cất, $m = 1{,}7$	170 tỉ tương đương	2,342
A', từ đầu, cùng tổng chi phí của B	433 tỉ	2,275
B, chưng cất, $m = 8$	800 tỉ tương đương	2,239

Đọc bảng: cùng "100 tỉ token", B với $m = 1{,}7$ tốt hơn A (2,342 so với 2,389), và đây chính là kiểu so sánh trong nhiều bài báo. Nhưng ở cùng tổng chi phí (A'), từ đầu thắng (2,275). Chỉ khi $m = 8$ (lớn hơn ngưỡng $4{,}33$) thì B thắng A' (2,239 so với 2,275).

Chi phí thầy khấu hao. Nếu thầy được huấn luyện một lần và dùng cho K học trò, mỗi học trò chịu $1{,}2 \times 10^{22}/K$ thêm. Tỉ lệ so với từ đầu là $4{,}33 + 20/K$ (tự tính): $K = 1$ cho $24{,}3$; $K = 5$ cho $8{,}33$; $K = 30$ cho $5{,}0$ (tự tính). Khấu hao có hiệu quả nhưng không đưa tỉ lệ xuống dưới $4{,}33$, vì phần suy luận của thầy trên D_S token vẫn phải trả cho từng học trò (trừ khi tính logit một lần rồi lưu lại và dùng lại, xem phần 7).

6. Tỉ lệ chi phí theo cỡ thầy, và khoảng cách năng lực



Hình 23: Trái: dạng chữ U của khoảng cách năng lực (đồ chơi, tâm đặt ở tỉ lệ 2,5 theo ghi chú distillation 41). Phải: tỉ lệ chi phí mỗi token 1 + NT/(3NS) theo tỉ lệ cỡ thầy trên trò (minh hoạ, tự tính)

Trái: dạng chữ U của khoảng cách năng lực (đồ chơi, tâm đặt ở tỉ lệ 2,5 theo ghi chú distillation 41). Phải: tỉ lệ chi phí mỗi token $1 + N_T/(3N_S)$ *theo tỉ lệ cỡ thầy trên trò (minh hoạ, tự tính)*

Cách đọc hình: **panel phải** là công thức thuần tuý (tự tính): thầy gấp $2{,}5$ lần trò thì mỗi token chưng cất đắt gấp $1{,}83$ lần từ đầu; thầy gấp 10 lần thì $4{,}33$ lần; thầy gấp 70 lần thì $24{,}3$ lần. Nghĩa là càng chọn thầy lớn, ngưỡng m để hoà vốn càng cao. **Panel trái** là dạng chữ U giả định, không phải số đo: đường cong có cực tiểu quanh tỉ lệ $2{,}5$ rồi tăng lại, nghĩa là thầy quá lớn thì học trò học tệ đi. Hai đường thể hiện hai mức độ nhạy khác nhau; hình chỉ để thấy dạng U, tâm U đặt theo ghi chú distillation 41.

Khoảng cách năng lực (capacity gap). Theo ghi chú distillation 41, nghiên cứu sơ bộ với GPT-2 và Pythia cắt tỉa ra các học trò, thấy teacher tối ưu gần như tuyến tính theo cỡ học trò, $T^\star \approx 2{,}5\,S$ (hệ số xác định $R^2 = 0{,}9957$); với học trò 3B (MiniMA-3B) tỉ lệ 0,4 cho teacher khoảng 7,5B nên chọn LLaMA2-7B. Theo cùng ghi chú, ở thiết lập 126 tỉ token, MMLU của MiniMA-3B với teacher 7B là 28,51, cao hơn teacher 13B (26,82) và 70B (26,91): teacher lớn hơn lại thua. Ghi chú cũng lưu ý đây chỉ là một điểm ngoại suy và chênh lệch nhỏ, không có nhiều seed; teacher lớn hơn thua cũng có thể vì học trò chưa được huấn luyện đủ lâu.

Theo ghi chú distillation 42, bức tranh tinh hơn: điều quyết định ảnh hưởng của thầy không phải cỡ hay số token riêng lẻ mà là **cross-entropy L_T của thầy**; học trò giỏi lên khi L_T giảm, cho tới một điểm, sau đó thầy quá mạnh làm học trò khó mô phỏng và loss tăng lại (broken power law theo L_T). Luật tuyến tính của ghi chú 41 chỉ là trường hợp đặc biệt khi thầy là mô hình compute-optimal. Ghi chú distillation 43 cho thấy một cơ chế tương tự: với học trò 200M, thầy 500M tốt nhất, thầy 300M quá yếu, thầy 4B làm thang giá trị log-xác suất lệch quá xa so với mô hình tham chiếu nên bước chọn mẫu mất tác dụng.

7. Cái giá của thầy: sinh logit và sinh mẫu

Chi phí chưng cất có hai phần: huấn luyện thầy (nếu chưa có) và chạy thầy trên dữ liệu của trò. Phần thứ hai có ba cách hạ giá:

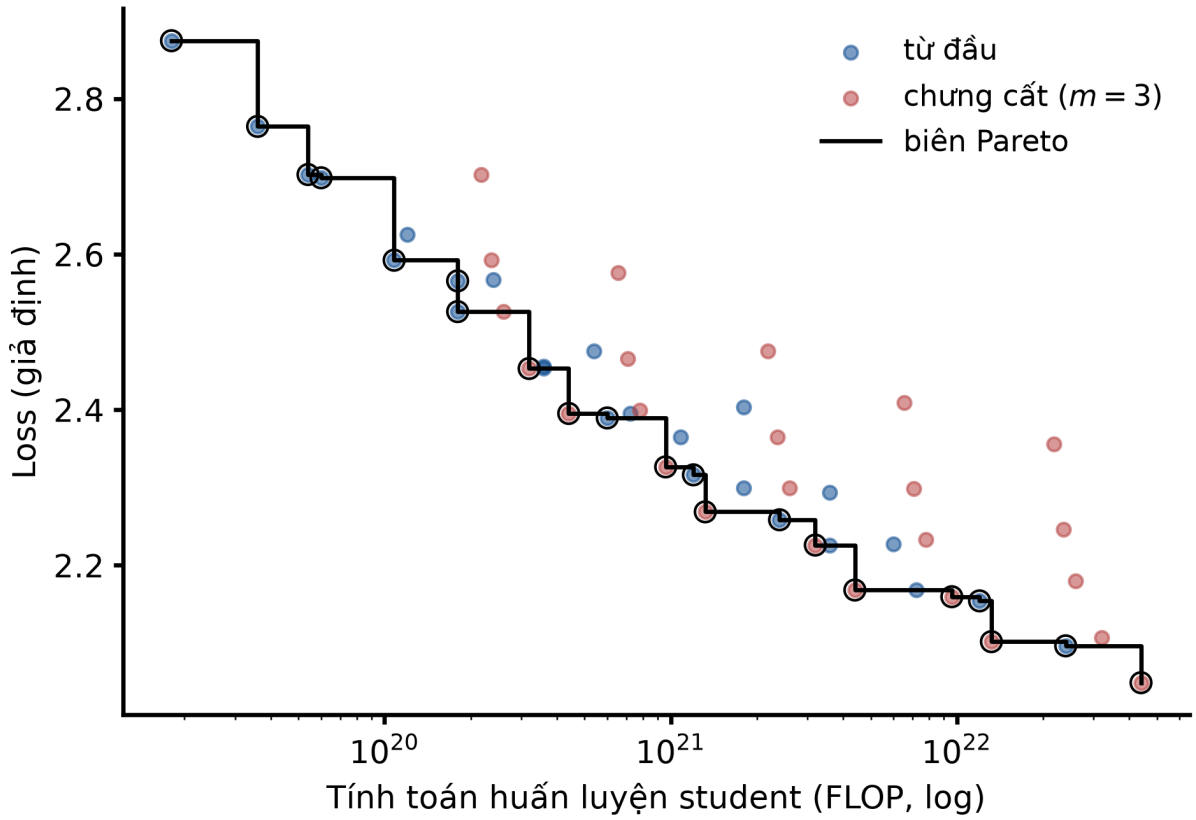
1. **Chạy thầy trực tuyến** (online): mỗi bước huấn luyện chạy thầy một lần. Chi phí $2N_T D_S$ như trên, trả lại cho từng học trò.
2. **Tính một lần, lưu logit** (offline): chia $2N_T D_S$ cho nhiều học trò hoặc nhiều lần huấn luyện, nhưng lưu trữ rất lớn: theo ghi chú distillation 42, khoảng 260 PB cho khoảng 2 nghìn tỉ token C4 với từ vựng 32.168 (logit float32); theo ghi chú distillation 43, khoảng 30 PB cho 50 tỉ token với từ vựng 150 nghìn. Ghi chú distillation 45 cho thấy lấy mẫu thưa vài chục token mỗi vị trí có thể thay cho lưu đủ: trong thiết lập thử nghiệm, khoảng 12 token mỗi vị trí đạt mức chưng cất đầy đủ, chiếm khoảng 3,6 TB cho 100 tỉ token.
3. **Chỉ lưu một số mỗi chuỗi.** Theo ghi chú distillation 43, MiniPLM tính một điểm số mỗi chuỗi 1024 token, nên 50 tỉ token chỉ cần khoảng 200 MB, và ghi chú nhận xét rằng khi so với các phương pháp chưng cất trực tuyến ở cùng tổng FLOP thì các phương pháp đó không hơn (hoặc tệ hơn) huấn luyện thường (ví dụ MiniLLM 39,0 so với 39,9 ở 200M). Đây là số của paper ở quy mô nhỏ (học trò tối đa 1,2B và 50 tỉ token), nên không được khái quát.

Với chưng cất dữ liệu (nen06), chi phí thầy là $2N_T$ mỗi token **thầy sinh**, và ta có thể tính y như trên. Ví dụ tay: thầy 70B sinh $1{,}6 \times 10^9$ token (đại khái 800K mẫu, mỗi mẫu 2000 token), tốn $2 \times 70 \times 10^9 \times 1{,}6 \times 10^9 = 2{,}24 \times 10^{20}$ FLOP; học trò 7B học một epoch trên tập đó tốn $6 \times 7 \times 10^9 \times 1{,}6 \times 10^9 = 6{,}72 \times 10^{19}$ FLOP; tỉ lệ sinh trên học là $3{,}33$, tổng gấp $4{,}33$ lần (tự tính), đúng công thức $1 + 70/(3 \cdot 7)$. Ghi chú distillation 38 nhấn rằng chi phí sinh 800K mẫu từ thầy lớn không được tính vào so sánh "chưng cất thắng RL".

8. Biên Pareto: nhìn cả tập cấu hình

Mô phỏng: biên Pareto giữa chi phí huấn luyện và loss cho các cấu hình từ đầu và chưng cất (teacher 10B có sẵn, hệ số m=3 giả định) (minh hoạ, tự tính)

Cách đọc hình: mỗi chấm là một cấu hình (cỡ học trò trong $\{0{,}3;\ 0{,}6;\ 1;\ 2;\ 4\}$ tỉ, số token trong $\{10;\ 30;\ 100;\ 300;\ 1000\}$ tỉ), chấm xanh dương là huấn luyện từ đầu và chấm đỏ là chưng cất từ thầy 10B có sẵn với $m = 3$ (giả định). Trục hoành là chi phí huấn luyện (bao gồm suy luận của thầy), trục tung là loss



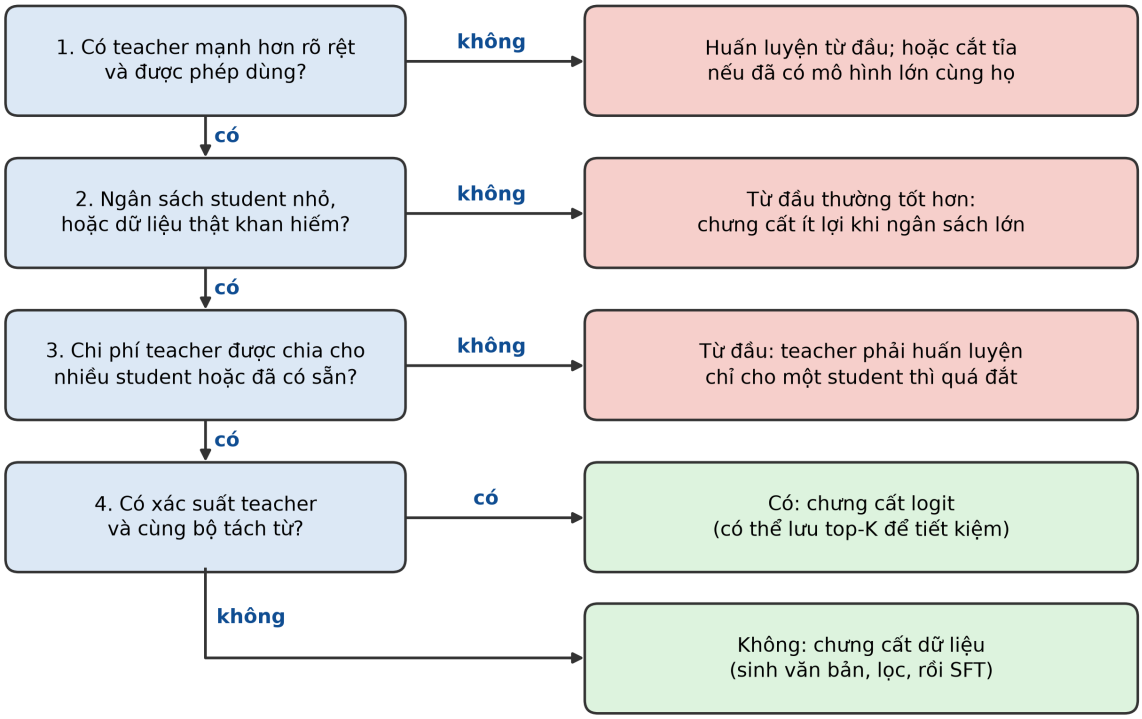
Hình 24: Mô phỏng: biên Pareto giữa chi phí huấn luyện và loss cho các cấu hình từ đầu và chưng cất (teacher 10B có sẵn, hệ số m=3 giả định) (minh hoạ, tự tính)

(đồ chơi). Đường đen nét bậc thang nối các cấu hình **trên biên Pareto**: không cấu hình nào vừa rẻ hơn vừa tốt hơn. Tìm các chấm đỏ nằm trên biên.

Trong mô phỏng này, biên có 21 cấu hình, 12 từ đầu và 9 chưng cất (tự tính). Các cấu hình chưng cất trên biên đều thuộc học trò 2B và 4B. Điều này khớp với điều kiện hoà vốn: tỉ lệ chi phí mỗi token là $1 + 10/(3 \cdot 2) = 2{,}67 < 3$ và $1 + 10/(3 \cdot 4) = 1{,}83 < 3$, còn học trò 1B có tỉ lệ $4{,}33 > 3$ nên không bao giờ lên biên. Điều rút ra (suy luận, vì phụ thuộc vào m giả định): **chưng cất hữu ích khi thầy không quá lớn so với học trò**, và nó xuất hiện ở phần giữa và phải của biên, không phải ở phần ngân sách rất nhỏ.

Ý nghĩa thực hành của biên Pareto: đừng chỉ so hai điểm (một từ đầu, một chưng cất). Hãy vẽ cả hai họ cấu hình theo cùng trục chi phí và xem họ nào nằm dưới.

9. Khi nào nên chưng cất, và khi nào không



Hình 25: Cây quyết định: khi nào nên chưng cất (suy luận của chương dựa trên các ghi chú distillation 42 và 43) (minh hoạ, tự tính)

Cây quyết định: khi nào nên chưng cất (suy luận của chương dựa trên các ghi chú distillation 42 và 43) (minh hoạ, tự tính)

Cách đọc hình: bắt đầu từ ô 1 ở góc trên trái, đi xuống theo "có", rẽ phải theo "không". Ô đỏ là kết luận "không chưng cất", ô xanh lục là "chưng cất" kèm loại chưng cất. Đây là khuyến nghị của chương, tổng hợp từ các kết luận trong ghi chú, không phải thuật toán chính thức.

Các điều kiện chưng cất thường đáng (theo ghi chú distillation 42, tóm tắt):

- Ngân sách huấn luyện học trò khiêm tốn, dưới một ngưỡng tăng theo cỡ học trò; và thầy đã có sẵn hoặc được dùng lại. Ghi chú nhấn rằng chưng cất không bao giờ thắng huấn luyện từ đầu nếu cả hai đều có đủ nhiều tính toán và dữ liệu.
- Nếu phải huấn luyện thầy chỉ để chưng cất một học trò, huấn luyện từ đầu nói chung tốt hơn.
- Thầy dùng cho nhiều học trò hoặc dùng làm mô hình phục vụ thì chưng cất có thể rẻ hơn.

Các điều kiện thực tế thúc đẩy chưng cất (suy luận):

- Dữ liệu thật khan hiếm hoặc đắt (nhãn, bài toán lập luận có lời giải), nhưng thầy sinh được dữ liệu tổng hợp có thể kiểm chứng: chưng cất dữ liệu và lập luận (nen06).
- Cần mô hình nhỏ để phục vụ nhanh, nhưng đã có sẵn một mô hình lớn cùng nhiệm vụ.
- Muốn phục hồi mô hình sau khi cắt tỉa: thầy chính là mô hình trước khi cắt (theo ghi chú distillation 47).

Khi huấn luyện từ đầu có thể tốt hơn:

- Ngân sách lớn, dữ liệu dồi dào: chưng cất bị chặn bởi cận dưới của huấn luyện từ đầu (ghi chú distillation 42).
- Thầy quá lớn so với học trò (khoảng cách năng lực): thầy thêm tốn mà học trò không tiến.
- Thầy không đáng tin trong lĩnh vực đích (sai nhiều, phong cách riêng): dữ liệu thật tốt hơn.

Khi cắt tỉa tốt hơn (suy luận): khi đã có mô hình lớn đã huấn luyện, cắt tỉa cấu trúc rồi huấn luyện lại thường rẻ hơn huấn luyện học trò từ đầu, và có thể kết hợp với chưng cất như ghi chú distillation 41 (cắt tỉa tạo học trò, rồi chưng cất) và 47 (chưng cất để phục hồi). Chọn giữa chúng phụ thuộc vào việc có kiến trúc đích cố định hay không: nếu kiến trúc đích tuỳ ý, huấn luyện từ đầu hoặc chưng cất cho tự do; nếu cần "một biến thể của mô hình lớn", cắt tỉa tận dụng trọng số có sẵn.

10. Nối sang các bài paper của cuốn

- **distillation 40 (Gemma 2).** Theo ghi chú, ablation với học trò 2B và 500 tỉ token, thầy 7B cho điểm trung bình tăng từ 60,3 lên 67,7. Ghi chú cũng nhấn mạnh so sánh ở "cùng số token" và không tính chi phí thầy; hãy tự đặt câu hỏi: ở cùng tổng FLOP thì sao (phần 5 của chương này)? Đây là ví dụ cổ điển của bài toán so sánh công bằng.
- **distillation 41 (khoảng cách năng lực).** Luật $T^\star \approx 2{,}5S$ và kết quả teacher 7B hơn 13B và 70B (theo ghi chú). Dùng phần 6 để đọc: đây là dữ liệu của một vùng (dải 70M đến 3B, một điểm ngoại suy lên 7B), không phải luật phổ quát.
- **distillation 42 (scaling law của chưng cất).** Cho khung định lượng: loss học trò phụ thuộc L_T của thầy, có điểm gãy, và ba kết luận về khi chưng cất thắng. Ghi chú cũng nhắc các hệ số fit đặc thù cho kiến trúc và dữ liệu C4, không phổ quát, và dải ngoại suy (học trò lớn hơn 12,6B, hơn 512 tỉ token).
- **distillation 43 (MiniPLM).** Tránh chi phí chạy thầy trực tuyến bằng cách dùng thầy một lần để chọn dữ liệu; điểm đáng học là ngân sách FLOP cố định và lưu trữ 200 MB so với 30 PB. Theo ghi chú, nó giảm nhu cầu dữ liệu 2,4 lần trong thiết lập thiếu dữ liệu, ở quy mô nhỏ.
- **distillation 44 (Qwen3).** Theo ghi chú, chưng cất off-policy rồi on-policy cho mô hình nhỏ; ở cỡ 8B, on-policy dùng khoảng một phần mười giờ GPU của học tăng cường (chưa tính chi phí thầy). Đọc cùng phần 7.
- **distillation 45 (lấy mẫu logit thưa).** Là lời giải cho bài toán lưu trữ ở phần 7.
- **distillation 46 (Distil-Whisper).** Theo ghi chú, mô hình nhỏ hơn 51%, nhanh hơn 5,8 lần, pseudo-label được lọc theo tỉ lệ lỗi từ 10%: một trường hợp chưng cất vì tốc độ phục vụ.
- **distillation 47 (ShortOPD).** Phục hồi sau cắt tỉa bằng chưng cất on-policy từ mô hình trước khi cắt; theo ghi chú tiết kiệm 71% token trong thí nghiệm của họ.

11. Danh sách kiểm báo cáo trung thực

Khi đọc hoặc viết một bài chưng cất, hãy hỏi:

1. **Có học trò từ đầu cùng kiến trúc, cùng dữ liệu, cùng số bước không?** Nếu không có baseline này, "chưng cất tốt" không có nghĩa.
2. **So ở cùng token hay cùng FLOP?** Nếu cùng token, tính chi phí thầy riêng (phần 4 và 5).

3. **Chi phí thầy có được tính (huấn luyện + suy luận hoặc sinh) không, hay chỉ là "thầy có sẵn"?** Nếu thầy có sẵn, nói rõ ai trả tiền.
4. **Nhiều học trò hay một?** Khấu hao chỉ hợp lệ khi thầy dùng lại thật.
5. **Cỡ thầy có thay đổi không?** Chỉ một thầy thì không biết khoảng cách năng lực.
6. **Hyperparameter (nhiệt độ, trọng số loss, learning rate) có được dò công bằng cho cả hai nhánh không?** Baseline bị dò kém sẽ làm chưng cất trông tốt.
7. **Có nhiều seed và khoảng tin cậy không?** Nhiều chênh lệch trong bảng nhỏ hơn 1 điểm (như nhiều ghi chú của cuốn đã chỉ ra, ví dụ distillation 41 và 43).
8. **Đánh giá trên nhiệm vụ nào?** Perplexity có thể không dịch sang benchmark; ghi chú distillation 42 cũng lưu ý nó không đánh giá lập luận.
9. **Dữ liệu kiểm có sạch khỏi dữ liệu của thầy không?** Rò rỉ làm số đẹp giả.
10. **Giới hạn nội suy và ngoại suy được nói rõ không?** Luật fit ở 70M đến 3B không tự động đúng cho 70B.

12. Sai lầm hay gặp

- So sánh "cùng số token" rồi kết luận "chưng cất rẻ hơn", bỏ qua $2N_T$ mỗi token của thầy.
- Quên huấn luyện thầy khi thầy được tạo riêng cho học trò này.
- Chọn thầy lớn nhất có thể, cho rằng "thầy càng giỏi càng tốt", trong khi có khoảng cách năng lực.
- Coi các đường cong mô phỏng (như trong chương này) là số đo thực; hệ số m phụ thuộc vào ngân sách và cỡ học trò.
- Chỉ so hai điểm thay vì nhìn biên chi phí và chất lượng.
- Trộn chưng cất với các thay đổi khác (kiến trúc, dữ liệu, độ dài huấn luyện) rồi gán toàn bộ lợi ích cho chưng cất.

13. Tóm tắt công thức

Ý	Công thức hoặc kết luận
Chi phí huấn luyện, suy luận mỗi token	$6N$ và $2N$ (xấp xỉ)
Chưng cất, thầy có sẵn	$C_{\text{dist}} = 6N_S D_S + 2N_T D_S$
Tỉ lệ chi phí mỗi token	$1 + \frac{N_T}{3N_S}$
Huấn luyện thầy dùng chung K học trò	thêm $\frac{6N_T D_T}{K}$ mỗi học trò
Hoà vốn (hệ số m, suy luận)	$m > 1 + \frac{N_T}{3N_S}$
Teacher tối ưu (ghi chú 41)	$T^\star \approx 2{,}5\, S$
Quyết định chính (ghi chú 42)	thầy có sẵn và ngân sách nhỏ: chưng cất; thầy phải huấn luyện cho một học trò: từ đầu

14. Bài tập tự tính

Bài 1. Học trò $N_S = 2 \times 10^9$, $D_S = 4 \times 10^{10}$ token; thầy $N_T = 3 \times 10^{10}$ có sẵn. Tính C_{sup}, C_{dist} và tỉ lệ.

Đáp án: $C_{\text{sup}} = 6 \times 2 \times 10^9 \times 4 \times 10^{10} = 4{,}8 \times 10^{20}$; suy luận thầy $2 \times 3 \times 10^{10} \times 4 \times 10^{10} = 2{,}4 \times 10^{21}$; $C_{\text{dist}} = 2{,}88 \times 10^{21}$; tỉ lệ 6,0, trùng $1 + 30/(3 \cdot 2) = 6$ (tự tính).

Bài 2. Thầy 10 tỉ tham số được huấn luyện 20 token mỗi tham số, dùng cho $K = 8$ học trò 1B, mỗi học trò 100 tỉ token. Tính tỉ lệ chi phí mỗi học trò so với từ đầu.

Đáp án: huấn luyện thầy $6 \times 10^{10} \times 2 \times 10^{11} = 1{,}2 \times 10^{22}$, chia 8 là $1{,}5 \times 10^{21}$; mỗi học trò $2{,}6 \times 10^{21} + 1{,}5 \times 10^{21} = 4{,}1 \times 10^{21}$; tỉ lệ $4{,}1 \times 10^{21} / 6 \times 10^{20} = 6{,}83$, trùng $4{,}33 + 20/8 = 6{,}83$ (tự tính).

Bài 3. Thầy 7B, học trò 1B, $D_S = 100$ tỉ token; thầy được huấn luyện 20 token mỗi tham số chỉ cho học trò này. Tính tỉ lệ chi phí tổng so với từ đầu và ngưỡng m để hoà vốn.

Đáp án: huấn luyện thầy $6 \times 7 \times 10^9 \times 1{,}4 \times 10^{11} = 5{,}88 \times 10^{21}$; tỉ lệ $1 + 7/3 + 5{,}88 \times 10^{21}/6 \times 10^{20} = 3{,}33 + 9{,}8 = 13{,}13$; ngưỡng $m > 13{,}13$ (tự tính).

Bài 4. Cho ngân sách $C = 10^{22}$ FLOP với học trò 1B. Hỏi từ đầu huấn luyện được bao nhiêu token và chưng cất (thầy 10B có sẵn) huấn luyện được bao nhiêu?

Đáp án: từ đầu $10^{22}/(6 \times 10^9) = 1{,}667 \times 10^{12}$, tức khoảng 1667 tỉ token; chưng cất $10^{22}/(6 \times 10^9 + 2 \times 10^{10}) = 3{,}85 \times 10^{11}$, tức khoảng 385 tỉ token (tự tính). Tỉ lệ $4{,}33$.

Bài 5. Năm cấu hình (chi phí tương đối, loss): A(1; 2,6), B(2; 2,4), C(3; 2,5), D(4; 2,2), E(5; 2,3). Cấu hình nào nằm trên biên Pareto?

Đáp án: C bị B lấn át (B rẻ hơn và loss thấp hơn); E bị D lấn át. Biên Pareto gồm A, B, D (tự tính).

Tài liệu tham khảo nền: Hoffmann và cộng sự 2022 (Chinchilla) và Kaplan và cộng sự 2020 về scaling law; các ghi chú trong repo: distillation 40, distillation 41, distillation 42, distillation 43, distillation 45; distillation 27 cho tổng quan.

Bài 00

Preface: Toán nền tảng và hướng dẫn đọc về Knowledge Distillation

Tác giả: ghi chú tổng hợp từ kiến thức nền của người biên soạn (không phải ghi chú một paper cụ thể)
Venue / arXiv: không áp dụng
Nguồn: các bài 01–47 trong thư mục này; bản đồ phân loại lấy từ bài 05 (Gou et al., arXiv:2006.05525)
Chủ đề: KD từ con số 0: ý tưởng, phân loại, toán (nhiệt độ, gradient, T^2, forward/reverse KL, feature loss, exposure bias, ước lượng top-k), cạm bẫy thực hành, thuật ngữ, cách đọc paper, lịch đọc.
Vị trí trong lộ trình: Bài số 00, đọc trước tất cả. Sau đó đi theo 01 → 05 để có nền tảng, rồi 06–14 (KD cổ điển), 15–20 (NLP/Transformer), 21–26 (thị giác và VLM), 27–47 (LLM).

> Tài liệu học tập bằng tiếng Việt, viết từ kiến thức chung của người biên soạn, **không** là ghi chú của một paper và chưa được đối chiếu từng câu với nguồn. Các con số minh hoạ tự tính (có kiểm tra bằng tay). Khi một paper trong thư mục nói khác, hãy tin paper và kiểm tra lại.

1. KD là gì và tại sao hoạt động

1.1 Định nghĩa

Knowledge distillation (KD, chưng cất tri thức): huấn luyện một mô hình nhỏ (student) để bắt chước một mô hình lớn hoặc mạnh hơn (teacher), thay vì (hoặc ngoài việc) học trực tiếp từ nhãn gốc. Mục tiêu thường là: giữ phần lớn chất lượng của teacher với chi phí suy luận nhỏ hơn nhiều.

Khác biệt cần nhớ:

- **Pruning** (cắt tỉa): sửa trực tiếp một mô hình có sẵn bằng cách bỏ phần của nó. Sau khi cắt, mô hình thường kém đi và cần "phục hồi".
- **KD:** là cách *huấn luyện* (một hàm mất mát và nguồn tín hiệu), không phải một kiến trúc. Student có thể là mô hình khởi tạo ngẫu nhiên, một mô hình nhỏ có sẵn, hoặc **một mô hình đã bị cắt tỉa**.
- Hai thứ thường đi cùng nhau: cắt tỉa tạo student, KD phục hồi chất lượng.

Bài 01 (Ba & Caruana 2014) cho thấy mạng nông bắt chước mạng sâu tốt hơn mạng nông học từ nhãn; bài 02 (Hinton et al. 2015) đặt tên "distillation" và chuẩn hoá công thức. Nguồn gốc xa hơn là "Model Compression" của Buciluǎ, Caruana, Niculescu-Mizil (2006), theo các bài 01 và 02 trích dẫn.

1.2 Vì sao nhãn của teacher giàu thông tin hơn nhãn gốc

Giả sử bài toán 3 lớp: mèo, chó, xe tải. Ảnh một con chó, nhãn gốc là one-hot $[0, 1, 0]$. Teacher có thể cho $[0.09, 0.90, 0.01]$. Thông tin thêm: "con chó này hơi giống mèo, gần như không giống xe tải". Nhãn one-hot không nói điều đó. Hinton gọi cấu trúc tương đồng này (ẩn trong xác suất nhỏ của các lớp sai) là **dark knowledge** (tri thức tối). Bài 02 dùng ví dụ: xe BMW có khả năng bị nhầm với xe tải rác cao hơn rất nhiều so với bị nhầm với củ cà rốt.

Bốn cách nhìn bổ sung, đều xuất hiện trong các bài đầu của lộ trình:

1. **Thông tin mỗi mẫu cao hơn.** Một nhãn one-hot có $\log_2 K$ bit tối đa, còn phân phối mềm mang nhiều bit hơn. Bài 02: soft target entropy cao cho nhiều thông tin mỗi mẫu và phương sai gradient thấp hơn, nên student học được với ít dữ liệu hơn (ví dụ trong bài 02: 3% dữ liệu, soft target cho 57.0% so với 44.5% nếu dùng nhãn cứng).
2. **Regularizer.** Soft target ngăn student quá tự tin và overfit; bài 05 liên hệ với label smoothing. Bài 01 xem đây là regularization: mạng nông học nhãn gốc overfit sớm, học từ teacher thì không.

3. **Làm sạch nhãn / làm mượt bài toán.** Teacher có thể "kiểm duyệt" nhãn sai và làm phẳng vùng khó học (bài 01, mục 5.1).
4. **Privileged information (thông tin đặc quyền).** Teacher có thể được huấn luyện với thông tin mà student không có lúc suy luận (đầu vào lớn hơn, nhiều modality, ensemble, thời gian tính toán không giới hạn). Student học "đường tắt" về cách tổng quát hoá. Bài 05 nhắc đến khung "learning using privileged information" (Vapnik, Lopez-Paz).

1.3 Hai nguồn lợi khác nhau cần tách bạch

- **Lợi ích tối ưu hoá/regularization:** ngay cả teacher *cùng kích thước* với student cũng có thể giúp (Born-again networks, bài 06).
- **Lợi ích chuyển giao năng lực:** teacher mạnh hơn có thêm kiến thức mà student không thể tự học từ dữ liệu có hạn.

Khi đọc kết quả, hãy hỏi: lợi ích đến từ nguồn nào? Bài 10 (Does KD really work?) và 09 (Efficacy of KD) đặt ra đúng câu hỏi này.

1.4 Khi nào KD không giúp

Khi student quá nhỏ so với teacher (capacity gap, bài 08, 41), khi teacher quá tự tin nên soft target gần như one-hot (không còn dark knowledge), khi teacher sai có hệ thống, hoặc khi student có thể đạt chất lượng tương đương bằng cách huấn luyện lâu hơn trên dữ liệu thật.

---

2. Phân loại KD

Nhiều trục độc lập. Một bài báo thường chọn một giá trị trên mỗi trục.

2.1 Theo loại tri thức (bài 05, §2)

Loại	Student bắt chước gì	Ví dụ bài
Response / logit-based	đầu ra cuối (logit hoặc xác suất)	01, 02, 14, 15, 28–34
Feature / hidden-state	kích hoạt trung gian, có thể qua projector	03 FitNets, 04 AT, 17, 18, 19, 23, 26
Relation-based	quan hệ giữa mẫu hoặc giữa lớp (khoảng cách, góc, Gram matrix)	12 RKD, 13 CRD

2.2 Theo sơ đồ huấn luyện (bài 05, §3)

- **Offline:** teacher huấn luyện xong, đóng băng, rồi dạy student.
- **Online:** teacher và student (hoặc nhiều mạng) cập nhật cùng lúc, ví dụ deep mutual learning (bài 07).
- **Self-distillation:** teacher và student là cùng kiến trúc hoặc cùng một mạng (bài 06 Born-again; bài 22 DINO là một biến thể khác trong học tự giám sát).

2.3 Theo đơn vị tín hiệu trong mô hình sinh (LLM)

- **Token-level (word-level):** ở mỗi vị trí chuỗi, so khớp phân phối token tiếp theo của teacher và student trên toàn từ vựng. Tín hiệu dày (nhiều thông tin mỗi vị trí).
- **Sequence-level:** teacher sinh cả chuỗi (ví dụ beam search hoặc lấy mẫu), student huấn luyện bằng cross-entropy trên chuỗi đó như dữ liệu thật (bài 15, Kim & Rush; về sau là nền của 35, 36, 37, 38).

2.4 Theo nguồn của chuỗi đầu vào: off-policy vs on-policy

Ngôn ngữ mượn từ học tăng cường/imitation learning.

- **Off-policy (hay "supervised KD"):** student học trên dữ liệu có sẵn (chuỗi từ tập huấn luyện hoặc do teacher sinh). Student không bao giờ thấy chuỗi do chính nó sinh ra.
- **On-policy:** student tự sinh chuỗi, teacher chấm điểm (cho phân phối token) trên chính các tiền tố của student. Giảm lệch phân phối giữa lúc huấn luyện và lúc suy luận. Đại diện: MiniLLM (bài 28), GKD (bài 29); bài 32 (Speculative KD) và 30 (DistiLLM) đề xuất cách pha trộn hiệu quả; bài 47 ShortOPD cũng thuộc hướng này.

2.5 Data-level distillation (chưng cất ở mức dữ liệu)

Teacher sinh dữ liệu huấn luyện (câu trả lời, giải thích, chuỗi suy luận), student fine-tune như bình thường (SFT). Không cần truy cập logit của teacher, nên dùng được với API đóng. Đại diện: Self-Instruct (35), Distilling step-by-step (36), Orca (37), distill từ DeepSeek-R1 (38), Qwen3 (44, phần strong-to-weak). Hạn chế: tín hiệu thưa hơn (chỉ thấy token được chọn, không thấy phân phối).

2.6 Các trục khác (ngắn gọn)

Multi-teacher, cross-modal (CLIP-KD bài 24, LLaVA-KD bài 26), data-free, quantized, lifelong (bài 05, §5).

---

3. Toán cốt lõi

Ký hiệu: K là số lớp (hoặc cỡ từ vựng V), $z \in \mathbb{R}^K$ là logit student, $v \in \mathbb{R}^K$ là logit teacher, q và p lần lượt là phân phối student và teacher.

3.1 Softmax có nhiệt độ

$$\boxed{q_i^{(T)} = \frac{\exp(z_i/T)}{\sum_j \exp(z_j/T)}}$$

- $T = 1$: softmax bình thường. $T \to \infty$: phân phối đều. $T \to 0^+$: gần one-hot (argmax).
- Softmax bất biến khi cộng hằng số vào mọi logit: $z \to z + c\mathbf{1}$ cho cùng xác suất. Vì vậy logit thường được "zero-mean" khi phân tích (bài 02 làm vậy).

Ví dụ. Logit teacher $v = [10, 5, 0]$.

T	v/T	xác suất p
1	$[10, 5, 0]$	$[0.9933,\ 0.0066,\ 0.00005]$
5	$[2, 1, 0]$	$[0.665,\ 0.245,\ 0.090]$

Ở $T = 1$ student gần như chỉ thấy "lớp 1"; ở $T = 5$ student thấy thêm "lớp 2 gần hơn lớp 3". Nhiệt độ là cách *phóng đại* dark knowledge.

3.2 Hàm mất mát KD kinh điển

$$\boxed{\mathcal{L} = \alpha T^2 \,\mathrm{KL}\big(p^{(T)} \,\|\, q^{(T)}\big) + (1-\alpha)\,\mathrm{CE}\big(y,\, q^{(1)}\big)}$$

Số hạng thứ hai là loss nhãn thật thông thường (ở $T = 1$). Bài 02 dùng cross-entropy với soft target thay cho KL; chúng khác nhau một hằng số không phụ thuộc student nên gradient như nhau (xem 3.5). Khi suy luận dùng $T = 1$.

3.3 Gradient và hệ số T^2

Đặt $C = \text{CE}(p^{(T)}, q^{(T)}) = -\sum_i p_i \log q_i$ với $q = \text{softmax}(z/T)$. Dùng đạo hàm softmax chuẩn:

$$\frac{\partial C}{\partial z_i} = \frac{1}{T}(q_i - p_i).$$

Hai điều cần nhớ:

1. **Hệ số $1/T$ từ quy tắc chuỗi** (do chia z cho T).
2. Khi T lớn so với độ lớn logit, khai triển bậc nhất $e^x \approx 1 + x$ và giả sử logit zero-mean ($\sum_j z_j = \sum_j v_j = 0$):

$$\boxed{\frac{\partial C}{\partial z_i} \approx \frac{1}{N\,T^2}(z_i - v_i)}$$

với N là số lớp (đây là Eq. 4 của bài 02). Nghĩa là ở T cao, KD tương đương hồi quy logit với L2 (đúng cách của bài 01) và độ lớn gradient co theo $1/T^2$. Nhân loss với T^2 bù lại để tỉ lệ giữa số hạng soft và hard không đổi khi đổi T. Đây là lý do có T^2.

Ở T thấp hơn, các logit rất âm so với trung bình ít được quan tâm hơn. Bài 02 nói điều này có thể có lợi (logit thấp bị nhiễu) hoặc có hại, và là câu hỏi thực nghiệm.

Kiểm tra bằng số nhỏ (tự tính). $v = [2, 0, -2]$, $T = 10$, student $z = [0, 0, 0]$ nên $q = [1/3, 1/3, 1/3]$. Teacher: $p = \text{softmax}([0.2, 0, -0.2]) \approx [0.4018,\ 0.3289,\ 0.2693]$. Gradient chính xác tại $i = 1$: $\frac{1}{10}(0.3333 - 0.4018) \approx -0.00685$. Xấp xỉ $\frac{1}{NT^2}(z_1 - v_1) = \frac{1}{3 \cdot 100}(0 - 2) \approx -0.00667$. Hai số rất gần nhau (khi $T \gg |v|$ xấp xỉ tốt; nếu giảm T xuống 2 thì sai lệch tăng rõ).

3.4 Cross-entropy trên nhãn teacher so với KL trên soft label

- Với phân phối teacher cố định p: $\text{CE}(p, q) = H(p) + \text{KL}(p\|q)$. $H(p)$ không phụ thuộc student nên gradient của CE và KL **giống hệt**.
- **Soft label** (token-level KD): dùng toàn bộ p ở mỗi vị trí, tín hiệu dày.
- **Hard label từ teacher** (sequence-level KD, bài 15): lấy chuỗi y^* do teacher sinh, tối thiểu $-\log q(y^*)$, tức CE với phân phối one-hot $\hat{p} = \delta_{y^*}$.
- Nếu teacher gần xác định ($p \approx \delta_{y^*}$) thì $\text{KL}(p\|q) \approx -\log q(y^*)$: hai cách **trùng nhau**. Phần soft label chỉ thêm giá trị khi teacher không chắc chắn (phân phối có entropy đáng kể), ví dụ nhiều đáp án hợp lệ hoặc chỗ mơ hồ.
- Khi lấy mẫu $y \sim p$: $\mathbb{E}_{y\sim p}[-\log q(y)] = \text{CE}(p, q)$. Tức hard-label SFT trên dữ liệu teacher lấy mẫu là ước lượng *không chệch nhưng phương sai cao* của gradient KD (token-level) trên mỗi vị trí.

Hệ quả cho OCR (suy luận): văn bản OCR thường có đáp án gần như duy nhất, teacher rất tự tin, nên KD toàn phân phối có thể không hơn nhiều so với SFT trên đầu ra teacher cho phần lớn token; phần khác biệt tập trung ở các ký tự mơ hồ. Cần đo entropy phân phối teacher trước khi đầu tư.

3.5 Forward KL và reverse KL

Định nghĩa với teacher p và student q:

$$\text{KL}(p\|q) = \sum_i p_i \log \frac{p_i}{q_i} \quad \text{(forward KL)}, \qquad \text{KL}(q\|p) = \sum_i q_i \log \frac{q_i}{p_i} \quad \text{(reverse KL)}.$$

- **Forward KL** $= \mathbb{E}_p[\log p - \log q]$: phạt nặng nếu $q_i \approx 0$ ở nơi $p_i > 0$. Student bị ép *phủ hết* mọi chế độ của teacher: **mode-covering** (phủ mode). Khi student quá nhỏ để khớp chính xác, nó "dàn mỏng" xác suất, có thể đặt khối lượng ở vùng mà teacher cho xác suất rất thấp (sinh ra văn bản kỳ lạ).
- **Reverse KL** $= \mathbb{E}_q[\log q - \log p]$: phạt nặng nếu $q_i > 0$ ở nơi $p_i \approx 0$. Student bị ép *chọn* một vài mode mà teacher tin: **mode-seeking** (bám mode). Có thể bỏ sót các mode khác của teacher (mất đa dạng).

Ví dụ số (đơn vị nats, tự tính). Teacher hai đỉnh trên 4 lớp: $p = [0.49,\ 0.01,\ 0.01,\ 0.49]$. Hai student:

- $q_A = [0.25, 0.25, 0.25, 0.25]$ (đều, phủ cả hai đỉnh nhưng dàn mỏng),
- $q_B = [0.97, 0.01, 0.01, 0.01]$ (chỉ một đỉnh).

Student	$\mathrm{KL}(p\|q)$ forward	$\mathrm{KL}(q\|p)$ reverse
q_A (đều)	0.595	1.273
q_B (một đỉnh)	1.572	0.623

Forward KL thích q_A (0.595 nhỏ hơn 1.572): không được bỏ rơi đỉnh nào. Reverse KL thích q_B (0.623 nhỏ hơn 1.273): không được đặt khối lượng vào vùng teacher cho xác suất ~0. Hai cách đo "giống nhau" cho ra student tối ưu khác nhau.

Trong thực tế LLM: token-level forward KL dễ cho student "mơ hồ" (đặt xác suất lên các token teacher cho xác suất thấp), MiniLLM (bài 28) dùng reverse KL với tối ưu policy gradient; GKD (bài 29) tổng quát hoá và dùng được phân kỳ khác kèm on-policy; bài 31 (Rethinking KL) xem lại giả thiết "forward = phủ, reverse = bám" và nêu rằng khác biệt thực tế phức tạp hơn. Hãy đọc kỹ các bài đó thay vì tin một quy tắc đơn giản.

3.6 JS, TV, skew KL

- **Jensen-Shannon (JS):** với $m = \frac{1}{2}(p+q)$, $\mathrm{JS}(p,q) = \frac{1}{2}\mathrm{KL}(p\|m) + \frac{1}{2}\mathrm{KL}(q\|m)$. Đối xứng, bị chặn trên (bởi $\log 2$ nats), không bùng nổ khi $q_i = 0$ vì $m_i > 0$. GKD dùng dạng JS tổng quát có trọng số β nội suy giữa forward và reverse (theo mô tả của bài 29; hãy xem định nghĩa chính xác trong bài đó).
- **Total variation (TV):** $\mathrm{TV}(p,q) = \frac{1}{2}\sum_i |p_i - q_i| \in [0,1]$. Gradient không bùng nổ, nhưng ít trơn nên ít dùng làm loss; xuất hiện trong phân tích lý thuyết và trong speculative decoding (xác suất chấp nhận $= 1 - \mathrm{TV}$, liên quan bài 32).
- **Skew KL:** $\mathrm{SKL}_\lambda(p\|q) = \mathrm{KL}\big(p \,\|\, \lambda p + (1-\lambda)q\big)$ (và dạng ngược). Trộn một phần p vào mẫu số để tránh $q_i \approx 0$ gây gradient nổ, ổn định huấn luyện. Là ý tưởng của DistiLLM (bài 30) theo tên "skew"; kiểm tra định nghĩa chính xác ở bài đó.

Nguyên tắc: các phân kỳ này chỉ khác nhau ở hành vi khi q và p lệch nhau nhiều (đúng trường hợp student nhỏ). Khi student đã khớp tốt, chúng gần như giống nhau.

3.7 Feature matching

Cho hidden state teacher $h_T \in \mathbb{R}^{d_T}$, student $h_S \in \mathbb{R}^{d_S}$.

MSE với projector (FitNets, bài 03):

$$\mathcal{L}_{\text{feat}} = \big\| h_T - W\, h_S \big\|_2^2, \quad W \in \mathbb{R}^{d_T \times d_S}$$

W ("projector", "regressor", "adapter") là ánh xạ nhỏ học được để đưa hai không gian về cùng chiều; sau huấn luyện thường bỏ đi. Nếu $d_S = d_T$ và cấu trúc tương ứng có thể không cần.

Cosine:

$$\mathcal{L}_{\cos} = 1 - \frac{\langle h_T,\, W h_S \rangle}{\|h_T\|\,\|W h_S\|}$$

chỉ quan tâm hướng, bỏ qua độ lớn. Hữu ích khi các kênh có biên độ rất lệch (ví dụ vài kênh "massive activations" chiếm hết MSE). **Chuẩn hoá** trước khi so (L2-norm như bài 04, LayerNorm, hoặc chuẩn hoá theo từng chiều) cũng cùng mục đích.

Với h_S cố định, tối ưu W là tìm phép khớp tuyến tính theo loss trên. Loss thấp cho biết các biểu diễn khớp tốt sau phép chiếu trên dữ liệu đang xét; chỉ riêng kết quả đó chưa đủ để kết luận student giữ được năng lực của teacher khi suy luận. Cũng không thể mặc nhiên xem phần dư là sai lệch thuần phi tuyến.

Attention-map transfer (bài 04) và **relational** (bài 12) là hai cách tránh khớp trực tiếp feature thô: lần lượt khớp bản đồ tóm tắt đã chuẩn hoá, và khớp quan hệ giữa các mẫu.

3.8 Exposure bias và lỗi tích luỹ trong mô hình tự hồi quy

Huấn luyện tự hồi quy thường dùng **teacher forcing**: ở bước t, mô hình nhận tiền tố đúng $y_{<t}$. Khi suy luận, nó nhận tiền tố do chính nó sinh. Sự khác nhau giữa phân phối tiền tố lúc huấn luyện và lúc suy luận gọi là **exposure bias**.

Hệ quả (kết quả quen thuộc từ imitation learning, Ross et al. 2011, nhắc theo kiến thức chung): nếu mỗi bước student sai với xác suất ϵ trên phân phối tiền tố đúng, thì lỗi tích luỹ trên chuỗi dài L có thể tăng cỡ $O(\epsilon L^2)$ thay vì $O(\epsilon L)$, vì một lỗi đưa student vào trạng thái chưa từng thấy khi huấn luyện, nơi nó còn dễ sai hơn. Với chuỗi OCR dài (một trang), điều này đáng lo.

KD làm dịu hoặc làm nặng thêm vấn đề này tuỳ cách làm:

- **Off-policy token-level KD** vẫn bị exposure bias.
- **On-policy KD** (bài 28–30): student tự sinh tiền tố, teacher chấm trên đó, nên student học cách sửa lỗi ở chính trạng thái nó thực sự gặp.
- **Sequence-level KD** (bài 15): dùng chuỗi do teacher sinh ra, ít đa dạng hơn dữ liệu thật và dễ học hơn (teacher có đầu ra "sạch" hơn), nhưng vẫn là dữ liệu do teacher chọn, không phải do student.

3.9 Ước lượng thưa (top-k) của logit và độ chệch

Lưu hoặc truyền toàn bộ phân phối $p \in \mathbb{R}^V$ với $V \sim 10^5$ cho mỗi token rất tốn kém. Ví dụ: $V = 150{,}000$ ở fp16 là 300 KB mỗi vị trí; 10^9 vị trí là 300 TB. Nên người ta cắt:

Top-k + chuẩn hoá lại: giữ k token có p lớn nhất, đặt $\tilde{p}_i = p_i / \sum_{j \in \text{top-}k} p_j$ cho $i \in$ top-k và 0 cho phần còn lại.

Độ chệch: gọi $\rho = \sum_{j \in \text{top-}k} p_j < 1$ là khối lượng giữ lại, phần đuôi $1 - \rho$ bị bỏ. Hệ quả:

- Student được dạy rằng token ngoài top-k có xác suất đúng bằng 0, và các token trong top-k lớn hơn thực tế $1/\rho$ lần: **hiệu chuẩn sai** (student quá tự tin), mất tri thức ở đuôi.
- Gradient kỳ vọng không còn bằng gradient của KL đầy đủ, tức là ước lượng **chệch** (biased). Chệch lớn khi teacher có entropy cao (đuôi dày), ví dụ chuỗi sáng tạo; nhỏ khi teacher gần xác định (OCR).
- Có thể lưu thêm phần "khối lượng đuôi" vào một lớp "other", làm giảm chệch nhưng mất cấu trúc bên trong đuôi.

Lấy mẫu có trọng số quan trọng (importance sampling): lấy m token từ một phân phối đề xuất r rồi ước lượng $\sum_i p_i f_i \approx \frac{1}{m} \sum_s \frac{p_{i_s}}{r_{i_s}} f_{i_s}$; không chệch nhưng phương sai phụ thuộc r. Bài 45 (Sparse logit sampling) phân tích chi tiết so sánh top-k chệch và lấy mẫu không chệch; hãy đọc bài đó để biết con số, bài này chỉ cung cấp khung.

3.10 Tóm tắt các lỗi trực giác hay gặp

- "KD = bắt student giống teacher": đúng cho **hàm** (đầu vào đến đầu ra), không đúng cho tham số.
- "Loss thấp hơn thì tốt hơn": KL trên tập huấn luyện thấp chưa chắc cho chất lượng nhiệm vụ cao hơn; luôn đánh giá bằng metric nhiệm vụ (CER/WER).
- "Nhiệt độ cao luôn tốt": nhiệt độ cao làm phẳng, gradient yếu đi, cần T^2 bù; student nhỏ có thể muốn T vừa (bài 02: 2.5–4 cho student 30 unit).

---

4. Cạm bẫy thực hành

4.1 Capacity gap (khoảng cách dung lượng)

Student quá nhỏ so với teacher thì không bắt chước được, và teacher "tốt hơn" đôi khi cho student tệ hơn. Giải pháp trong lộ trình: Teacher Assistant (bài 08), teacher "kiên nhẫn và nhất quán" (bài 11), quy luật capacity gap (bài 41), và thang đo teacher/student trong scaling laws (bài 42).

4.2 Tokenizer mismatch

KD *ở mức logit* đòi hỏi student và teacher chia sẻ **cùng từ vựng và cách tách token**, vì phép KL cộng theo chỉ số token. Nếu khác tokenizer, phải căn chỉnh (alignment) hoặc dùng phương pháp không phụ thuộc từ vựng, hoặc quay về data-level distillation.

4.3 Hiệu chuẩn (calibration) của teacher

KD dạy student theo *xác suất* của teacher. Nếu teacher quá tự tin hoặc kém hiệu chuẩn (probabilities không khớp tần suất đúng thực tế), student kế thừa điều đó. Cách xử lý: nhiệt độ $T > 1$, label smoothing cho teacher, ensemble, hoặc chọn lọc mẫu (chỉ distill khi teacher đúng, như vài bài về reasoning). Đo **entropy** và **ECE** (expected calibration error) của teacher trên tập dev là bước đơn giản.

4.4 Kế toán chi phí tính toán (compute accounting)

So sánh công bằng cần tính đủ:

- Chi phí **forward của teacher** trên mỗi token huấn luyện (thường lớn hơn forward+backward của student nhỏ).
- Chi phí **lưu trữ logit** nếu cache offline (mục 3.9) so với chạy teacher trực tuyến.
- Với on-policy KD: thêm chi phí **sinh chuỗi** của student.
- Chi phí huấn luyện **teacher** (nếu phải train riêng) có được tính không?
- Phép so sánh thích hợp là "cùng ngân sách FLOPs hoặc cùng số token" giữa KD và huấn luyện student trực tiếp (bài 42 làm việc này); một số bài chỉ so cùng số bước, khiến KD trông tốt hơn thực tế.

4.5 Các cạm bẫy khác

- **Quên** T^2 khi trộn soft và hard loss: tỉ lệ đóng góp thay đổi theo T mà ta không để ý.
- **Teacher ở chế độ train** (dropout bật, batch-norm cập nhật) khi sinh soft target: phải `eval()` và `no_grad`.
- **Rò rỉ đánh giá:** teacher đã thấy tập test (hoặc dữ liệu tương tự) làm student hưởng lợi gián tiếp. Với benchmark công khai, kiểm tra nguồn dữ liệu của teacher.
- **Nhầm KD với tinh chỉnh nhẹ:** khi student là bản cắt, một phần lợi ích chỉ là "huấn luyện thêm sau cắt", không phải KD. Luôn có baseline "cùng dữ liệu và bước, chỉ dùng nhãn thật".

---

5. Bảng thuật ngữ

Thuật ngữ	Tiếng Việt / giải nghĩa ngắn
Teacher / Student	mô hình dạy / mô hình học
Soft label / soft target	phân phối xác suất mềm do teacher cho
Hard label	nhãn one-hot hoặc token cụ thể
Dark knowledge	thông tin trong xác suất nhỏ của các lớp sai
Temperature T	tham số làm phẳng softmax
Logit	giá trị trước softmax
KL divergence	độ đo lệch giữa hai phân phối, không đối xứng
Forward KL	$\mathrm{KL}(p_{teacher} \| q_{student})$, mode-covering
Reverse KL	$\mathrm{KL}(q_{student} \| p_{teacher})$, mode-seeking
JS / TV / skew KL	các phân kỳ thay thế, ổn định hơn
Response-based	KD dựa trên output
Feature-based	KD dựa trên hidden state / feature
Relation-based	KD dựa trên quan hệ giữa mẫu
Hint / guided layer	lớp teacher dùng làm gợi ý / lớp student được dẫn dắt
Projector / regressor	ánh xạ khớp chiều giữa hai không gian feature
Offline / online / self-distillation	teacher cố định / học cùng / chính nó
Token-level / sequence-level KD	khớp phân phối từng token / học từ chuỗi teacher sinh
Off-policy / on-policy	chuỗi từ dữ liệu có sẵn / chuỗi do student sinh
Exposure bias	lệch giữa tiền tố lúc huấn luyện và lúc suy luận
Teacher forcing	huấn luyện với tiền tố đúng
Capacity gap	khoảng cách dung lượng giữa teacher và student
Teacher assistant	mô hình trung gian cầu nối
Data-level distillation	teacher sinh dữ liệu, student SFT
SFT	supervised fine-tuning, tinh chỉnh có giám sát
Top-k logits	chỉ giữ k xác suất lớn nhất
Calibration (ECE)	độ khớp giữa xác suất dự đoán và tần suất đúng
Self-distillation	teacher và student cùng kiến trúc / cùng mạng
Born-again network	student cùng kiến trúc, lặp lại nhiều thế hệ
Dustbin class	lớp gộp phần còn lại (bài 02, specialist)
Label smoothing	trộn nhãn one-hot với phân phối đều, regularizer
Mode-covering / mode-seeking	phủ mọi đỉnh / bám một vài đỉnh

---

6. Cách đọc paper KD một cách phản biện

Một danh sách câu hỏi dùng cho mọi bài trong thư mục:

1. **Trục phân loại:** tri thức nào (response/feature/relation)? offline/online/self? token-level hay sequence-level? on/off-policy? Đặt bài vào bản đồ của bài 05 trước khi đọc chi tiết.
2. **Student và teacher là gì?** Tỉ lệ kích thước? Cùng họ kiến trúc? Cùng tokenizer? (Capacity gap, mục 4.1.)
3. **Baseline có công bằng không?** Student có được huấn luyện với *cùng dữ liệu, cùng số bước, cùng tối ưu hoá* nhưng chỉ dùng nhãn thật không? Siêu tham số baseline có được tinh chỉnh bằng công sức tương đương? Teacher có được tính vào ngân sách?

4. **Kích thước cải thiện so với nhiễu:** một số bài báo cáo +0.3 điểm trên một seed. Có bao nhiêu seed? Trung vị hay tốt nhất? (Bài 04 dùng trung vị của 5 lần chạy cho CIFAR, nhưng ImageNet chỉ một lần.)
5. **Có ablation tách đóng góp không?** Ví dụ khi một bài kết hợp feature + response + curriculum, mỗi phần đóng góp bao nhiêu?
6. **Metric nào?** Loss KD thấp hay metric nhiệm vụ? Có đánh giá trên phân phối khác (out-of-distribution) không?
7. **Cuộc chơi thích nghi:** ai chọn siêu tham số trên tập test? Có early stopping trên test không?
8. **Chi phí:** thời gian, bộ nhớ, forward của teacher, lưu trữ logit (mục 4.4).
9. **Phạm vi khái quát:** thí nghiệm ở CNN ảnh nhỏ (bài 01–04) có chuyển sang Transformer/LLM không? Tác giả có nói không, hay ta tự suy ra?
10. **Phân biệt ba loại câu:** điều bài *chứng minh/đo*, điều bài *giả thuyết* (thường sau từ "we conjecture", "we believe"), và điều *ta suy ra*. Ghi chú trong thư mục này cố gắng tách chúng; hãy làm thế khi ghi chú riêng.
11. **Mâu thuẫn giữa các bài:** nhiều kết luận sớm bị phản biện (ví dụ bài 10 hỏi liệu student có thực sự khớp teacher, bài 31 xem lại hành vi forward/reverse KL, bài 39 cho thấy mô hình nhỏ khó học từ chuỗi suy luận mạnh). Khi hai bài mâu thuẫn, hỏi điều kiện thí nghiệm nào khác nhau.

---

7. Lịch đọc gợi ý

Mỗi giai đoạn khoảng một tuần, đọc ghi chú trước, paper gốc sau nếu cần.

Giai đoạn	Bài	Mục tiêu
0. Nền	00 (bài này), 01, 02	Hiểu soft target, nhiệt độ, T^2, KL; tự cài một vòng KD trên MNIST/CIFAR.
1. Feature và bản đồ	03, 04, 05	Hint, projector, attention map; dùng survey để định vị.
2. KD cổ điển, tại sao và khi nào	06, 07, 08, 09, 10, 11	Self/online KD, capacity gap, kiểm chứng thực nghiệm.
3. Quan hệ và đối nghịch/tách thành phần	12, 13, 14	Relational, contrastive, decoupled.
4. Chuỗi và NLP	15, 16, 17, 18, 19, 20	Sequence-level, DistilBERT, TinyBERT, MiniLM, causal distillation.
5. Thị giác và VLM	21, 22, 23, 24, 25, 26	DeiT, DINO, AM-RADIO, CLIP-KD, LLaVA-MoD, LLaVA-KD.
6. LLM: nền và phân kỳ	27, 28, 29, 30, 31, 32, 33, 34	Khảo sát, MiniLLM, GKD, DistiLLM, Rethinking KL, Speculative KD, TAID, DistiLLM-2.
7. LLM: dữ liệu và suy luận	35, 36, 37, 38, 39	Self-Instruct, step-by-step, Orca, R1 distillation, small models struggle.
8. Quy mô và hệ thống	40, 41, 42, 43, 44, 45, 46, 47	Gemma 2, capacity gap law, scaling laws, MiniPLM, Qwen3, sparse logit sampling, Distil-Whisper, ShortOPD.

Đây chỉ là gợi ý của người biên soạn; điều chỉnh theo kết quả thí nghiệm.

---

8. Năm câu tự kiểm tra

Câu 1. Vì sao khi dùng loss $\alpha T^2 \mathrm{KL}(p^{(T)} \| q^{(T)}) + (1-\alpha)\mathrm{CE}(y, q^{(1)})$ phải nhân T^2? Điều gì xảy ra nếu quên?

Đáp án.

Gradient của soft-loss theo logit student là $\frac{1}{T}(q_i - p_i)$, và ở T cao xấp xỉ $\frac{1}{NT^2}(z_i - v_i)$; độ lớn co theo $1/T^2$. Nhân T^2 giữ độ lớn gradient của số hạng soft gần như không đổi khi thay T, nên tỉ lệ với số hạng hard không đổi. Nếu quên, tăng T sẽ vô tình giảm tầm quan trọng của số hạng soft, và siêu tham số α tối ưu sẽ phụ thuộc vào T (khó tinh chỉnh, kết luận về "nhiệt độ tốt nhất" bị nhiễu).

Câu 2. Teacher gần như xác định ($p \approx \delta_{y^*}$ ở mọi vị trí). KD token-level (KL) khác gì so với SFT trên chuỗi do teacher sinh? Khi nào KD token-level vẫn đáng giá?

Đáp án.

Khi $p \approx \delta_{y^*}$ thì $\mathrm{KL}(p\|q) \approx -\log q(y^*)$, tức hai loss trùng nhau (và gradient giống nhau); KD toàn phân phối không thêm thông tin. Nó vẫn đáng giá ở những vị trí teacher *không* chắc (entropy cao): nhiều đáp án hợp lệ, vùng mơ hồ, nơi hard-label SFT chỉ thấy một mẫu ngẫu nhiên (ước lượng không chệch nhưng phương sai cao của gradient KL). Với OCR, điều này gợi ý đo entropy của teacher trước để biết KD toàn phân phối có lợi không.

Câu 3. Cho $p = [0.49, 0.01, 0.01, 0.49]$. Student nào tối ưu hơn theo forward KL, student nào theo reverse KL: $q_A = [0.25, 0.25, 0.25, 0.25]$ hay $q_B = [0.97, 0.01, 0.01, 0.01]$? Giải thích bằng "phủ mode" và "bám mode".

Đáp án.

Forward KL: $\mathrm{KL}(p\|q_A) \approx 0.595$ so với $\mathrm{KL}(p\|q_B) \approx 1.572$, nên q_A tốt hơn: forward KL phạt nặng nếu $q \approx 0$ ở nơi $p > 0$ (q_B gần bỏ hẳn đỉnh thứ hai), tức mode-covering. Reverse KL: $\mathrm{KL}(q_A\|p) \approx 1.273$ so với $\mathrm{KL}(q_B\|p) \approx 0.623$, nên q_B tốt hơn: reverse KL phạt nặng nếu $q > 0$ ở nơi $p \approx 0$ (q_A đặt 0.5 khối lượng vào hai lớp giữa mà p cho 0.01), tức mode-seeking. Sau đó cần nhớ bài 31 cảnh báo rằng thực tế phức tạp hơn mô hình đơn giản này.

Câu 4. Bạn lưu top-k xác suất của teacher (đã chuẩn hoá lại) để tiết kiệm bộ nhớ. Điều này gây độ chệch gì? Khi nào độ chệch nhỏ?

Đáp án.

Gọi ρ là tổng xác suất của top-k. Chuẩn hoá lại nhân các xác suất giữ lại với $1/\rho > 1$ và đặt đuôi bằng 0, nên student được dạy một phân phối quá tự tin, không có đuôi; gradient kỳ vọng khác gradient KL đầy đủ (ước lượng chệch). Độ chệch nhỏ khi $\rho \approx 1$, tức teacher có entropy thấp hoặc k đủ lớn. Lớn khi phân phối đuôi dày. Giải pháp: lấy mẫu có trọng số quan trọng cho ước lượng không chệch (bài 45) hoặc lưu thêm khối lượng đuôi.

Câu 5. Vì sao feature loss với projector tuyến tính có thể giảm thấp mà chất lượng student vẫn không tốt?

Đáp án.

Projector tuyến tính W có thể bù một phần sai khác giữa hai không gian biểu diễn. Khi W không được dùng lúc suy luận, loss $\|h_T - W h_S\|^2$ thấp không đảm bảo các lớp sau của student tự khai thác được biểu diễn đó. Vì vậy cần đánh giá cả chất lượng trên tác vụ.

Tài liệu này không có đường link gốc vì không ghi chú một paper cụ thể; hãy dùng các bài 01–47 trong thư mục và bản đồ ở bài 05 (https://arxiv.org/abs/2006.05525).

Bài 01

Do Deep Nets Really Need to be Deep? (2014)

Tác giả: Lei Jimmy Ba, Rich Caruana
Venue / arXiv: arXiv:1312.6184 (v7, 11/10/2014); bản trong file ghi “Draft for NIPS 2014” (NIPS 2014, bản draft chưa phải camera-ready)
Nguồn: https://arxiv.org/abs/1312.6184
Chủ đề: Model compression / mimic learning: huấn luyện mạng nông (shallow) bắt chước mạng sâu bằng cách hồi quy logit với L2 loss; câu hỏi “độ sâu có thực sự cần thiết?”.
Vị trí trong lộ trình: Đọc ngay sau bài 00 (preface). Đây là bài “tiền thân” của Hinton KD (02): nó cho thấy student học từ output của teacher có thể tốt hơn học từ nhãn gốc. Bài này đóng góp: (i) dùng **logit** làm target (không phải xác suất), (ii) bằng chứng thực nghiệm rằng khoảng cách shallow vs deep phần lớn là vấn đề *học/regularization* chứ không phải vấn đề *biểu diễn*. Tiếp theo: 02 (soft target + nhiệt độ T, tổng quát hoá việc “regress logit”), 03 (FitNets, bắt chước cả hidden layer).

> Tài liệu học tập bằng tiếng Việt, là ghi chú diễn giải, không phải bản dịch paper.

1. Bối cảnh

Câu hỏi của bài báo: Mạng sâu (deep) cho độ chính xác cao hơn mạng nông (một hidden layer) với cùng dữ liệu. Nguyên nhân là gì? Paper liệt kê các giả thuyết: (a) sâu có nhiều tham số hơn; (b) sâu học hàm phức tạp hơn với cùng số tham số; (c) sâu có “bias” tốt hơn, học biểu diễn phân cấp; (d) mạng không có convolution không học được cái mạng có convolution học được; (e) thuật toán học và regularization hiện tại hợp với mạng sâu hơn.

Nguồn gốc ý tưởng (từ thông tin nền, chưa đối chiếu với bài gốc): Buciluǎ, Caruana, Niculescu-Mizil, “Model Compression” (KDD 2006). Bài này được Ba & Caruana trích dẫn là [3]. Ý tưởng: một ensemble lớn rất chính xác nhưng chậm; dùng nó để gán nhãn cho một lượng lớn dữ liệu *không nhãn* (nhiều khi là dữ liệu sinh tổng hợp), rồi huấn luyện một mạng nhỏ trên các nhãn đó. Theo mô tả của Ba & Caruana, mạng nhỏ có thể có số tham số ít hơn tới 1000 lần mà vẫn gần bằng độ chính xác ensemble. *(Ghi chú: chi tiết ngoài những gì Ba & Caruana tóm tắt là thông tin nền, chưa được đối chiếu với bài 2006.)*

Thuật ngữ cần biết:

- **Teacher / student (mimic model):** teacher là mô hình lớn đã huấn luyện; student/mimic là mô hình nhỏ học bắt chước teacher.
- **Logit:** vector giá trị trước softmax, z; xác suất $p_k = e^{z_k} / \sum_j e^{z_j}$.
- **Shallow net (SNN):** mạng chỉ một hidden layer (rất rộng), có thể có thêm một lớp tuyến tính bottleneck.
- **Ensemble (ECNN):** trung bình nhiều mô hình; chính xác hơn nhưng tốn tính toán.

2. Ý tưởng chính

Trước hết, tác giả huấn luyện một mạng sâu hoặc một ensemble. Mạng nông sau đó học hồi quy các logit do teacher tạo ra, thay vì chỉ học nhãn gốc. Trong các thí nghiệm, student học theo teacher tiến gần độ chính xác của mạng sâu, trong khi cùng kiến trúc học trực tiếp từ nhãn cho kết quả kém hơn. Điều này gợi ý cần phân biệt khả năng biểu diễn một hàm với khả năng học được hàm đó bằng quy trình tối ưu đã chọn.

3. Phương pháp chi tiết

3.1 Quy trình (model compression)

1. Huấn luyện teacher bình thường (softmax + cross-entropy trên nhãn gốc).
2. Cho dữ liệu (có thể bỏ nhãn gốc, hoặc dữ liệu không nhãn thêm) đi qua teacher, thu logit z.
3. Huấn luyện student trên cặp (x, z); student không nhìn nhãn gốc.

Nếu student bắt chước hoàn hảo thì nó tạo đúng dự đoán và đúng lỗi của teacher.

3.2 Vì sao hồi quy logit thay vì xác suất

Paper đưa ví dụ: teacher dự đoán xác suất $[2 \cdot 10^{-9},\ 4 \cdot 10^{-5},\ 0.9999]$. Nếu dùng cross-entropy lên xác suất, student chỉ chú ý lớp thứ ba, bỏ qua hai lớp còn lại. Logit tương ứng là $[10, 20, 30]$ (paper lấy làm ví dụ minh hoạ), student phải học cả hiệu giữa lớp 1 và lớp 2. Thêm ví dụ: logit $[-10, 0, 10]$ cho cùng xác suất sau softmax như $[10, 20, 30]$ (softmax bất biến khi cộng một hằng số), nhưng teacher "nội bộ" mô hình hoá hai trường hợp rất khác nhau. Hồi quy logit giữ lại thông tin này, không bị mất ở bước qua softmax.

Hàm mất mát SNN-MIMIC, với dữ liệu $\{(x^{(t)}, z^{(t)})\}_{t=1}^{T}$:

$$\boxed{L(W, \beta) = \frac{1}{2T} \sum_{t} \left\| g(x^{(t)}; W, \beta) - z^{(t)} \right\|_2^2}, \qquad g(x) = \beta f(Wx)$$

trong đó W là trọng số input-hidden, β là trọng số hidden-output, f là phi tuyến.

Paper nói đã thử các loss khác như $\mathrm{KL}(p_{teacher} \,\|\, p_{student})$ và L2 trên xác suất; **hồi quy logit với L2 tốt hơn tất cả các loss khác** và là một trong các kỹ thuật chính (đây là kết quả báo cáo trong text, không có bảng so sánh riêng trong bản này). Ngoài ra chuẩn hoá logit của teacher (trừ mean, chia std theo từng target trên tập train) cải thiện L2 loss một chút khi huấn luyện, nhưng "không quan trọng".

3.3 Lớp tuyến tính bottleneck để tăng tốc

Mạng nông muốn có nhiều tham số phải có rất nhiều hidden unit, ma trận $W \in \mathbb{R}^{H \times D}$ rất lớn, hội tụ chậm (nhiều tuần dù có GPU). Giải pháp: phân rã $W = UV$ với $U \in \mathbb{R}^{H \times k}$, $V \in \mathbb{R}^{k \times D}$, $k \ll D, H$:

$$L(U, V, \beta) = \frac{1}{2T} \sum_{t} \left\| \beta f(UVx^{(t)}) - z^{(t)} \right\|_2^2$$

Bộ nhớ giảm từ $O(HD)$ xuống $O(k(H + D))$. Paper nhấn mạnh lớp tuyến tính này chỉ giảm (không tăng) năng lực biểu diễn và sau khi học có thể gộp lại thành một W. Đây là low-rank factorization giúp *tối ưu hoá* nhanh hơn, không phải giúp biểu diễn.

3.4 Pseudo-code

```
# Model compression (mimic learning), theo mô tả paper
teacher = train_deep_model(D_train)                 # CE + softmax; có thể là ensemble
Z = { logit(teacher, x) for x in D_transfer }        # logit, không phải softmax
if ensemble: z = mean over members of logit_m(x)     # paper: trung bình logit
(optional) z = (z - mean_k) / std_k                  # chuẩn hoá theo từng target
student = Shallow(x -> linear(k) -> ReLU(H) -> out)  # lớp linear k chỉ để tăng tốc
for (x, z) in minibatches(D_transfer):
    loss = 0.5 * || student(x) - z ||^2
    SGD+momentum step
# Student KHÔNG dùng nhãn gốc
```

4. Thí nghiệm và kết quả chính

4.1 TIMIT (nhận dạng âm vị)

Dữ liệu: 462 speaker train, 50 speaker dev, 24 speaker test; đầu vào 1845 chiều (filter-bank 40 hệ số + energy, cùng đạo hàm bậc 1, 2, cửa sổ ±7 frame); 61 âm vị, mỗi âm vị 3 trạng thái, tức 183 target (khi chấm điểm map về 39 lớp). Vì không có dữ liệu không nhãn từ phân phối TIMIT, nhóm dùng lại chính 1.1M điểm train (bỏ nhãn) làm tập chuyển. Paper nhận xét điều này làm giảm độ chính xác student; để bù, student bắt chước ECNN (ensemble 9 CNN).

Phone error rate (PER, càng thấp càng tốt) trên test core, trích Table 1:

Mô hình	Tham số	Hidden unit	PER
SNN-8k (train trên nhãn gốc, dropout)	~12M	~8k	23.1%
SNN-50k (nhãn gốc, dropout)	~100M	~50k	23.0%
SNN-400k (nhãn gốc, dropout)	~180M	~400k	23.6%
DNN (2k-2k-2k, dropout)	~12M	~6k	21.9%
CNN (c-p-2k-2k-2k, dropout)	~13M	~10k	19.5%
ECNN (ensemble 9 CNN)	~125M	~90k	18.5%
SNN-MIMIC-8k (250L-8k, không conv/pool)	~12M	~8k	21.6%
SNN-MIMIC-400k (250L-400k, không conv/pool)	~180M	~400k	20.0%

Đọc kết quả: SNN-MIMIC-8k (21.6%) đạt ngang/hơn DNN cùng số tham số (21.9%); SNN-MIMIC-400k (20.0%) gần CNN (19.5%) dù không có convolution. Các SNN huấn luyện trên nhãn gốc kém DNN từ 1.4% đến 2%, kém CNN 3.5–4.1%, kém ECNN 4.5–5.1% (paper nêu trong text).

Từ Figure 1 (mô tả của bài báo, không có số cụ thể trong text): ở ~1M tham số, SNN, DNN và SNN-MIMIC tương đương; SNN-MIMIC khớp DNN tới ~4M tham số rồi DNN tụt lại; DNN bão hoà quanh ~10M, mimic tiếp tục tăng và bão hoà ở ~100M tham số, độ chính xác tương đương CNN. Mimic không bao giờ đạt ECNN (do thiếu dữ liệu không nhãn).

4.2 CIFAR-10

Tập chuyển: 50,000 ảnh CIFAR-10 + 1 triệu ảnh đầu của 80 million tiny images = 1.05M. Student có **một lớp conv + max-pool** rồi 1200 unit tuyến tính và 30k unit phi tuyến ("64c-p-1200L-30k" hoặc "128c-p-1200L-30k"); paper lưu ý mạng không convolution hoạt động kém trên CIFAR-10 bất kể độ sâu nên cho phép một lớp conv làm bộ trích đặc trưng. Lỗi phân loại (Table 2):

Mô hình	Tham số	Lỗi
DNN 2000-2000 + dropout	~10M	57.8%
SNN-30k (128c-p-1200L-30k, dropout)	~70M	21.8%
CNN [11] 64c-p-64c-p-64c-p-16lc	~10k	15.6%
CNN [21] (stochastic pooling)	~56k	15.13%
teacher CNN 128c-p-128c-p-128c-p-1000fc	~35k	12.0%
ECNN (ensemble 4 CNN)	~140k	11.0%
SNN-CNN-MIMIC-30k (64c-p-1200L-30k)	~54M	15.4%
SNN-CNN-MIMIC-30k (128c-p-1200L-30k)	~70M	15.1%
SNN-ECNN-MIMIC-30k (bắt chước ensemble)	~70M	14.2%

(Cột "Tham số" và "Hidden units" trong text trích ra có đơn vị hơi khó đọc cho các CNN nhỏ; chỉ dùng như tham khảo.) Điểm đáng nói: mimic từ 1 CNN đạt 15.1%, từ ensemble đạt 14.2% (cải thiện thêm 0.9%).

5. Phân tích / ablation

Vì sao mimic có thể tốt hơn học từ nhãn gốc (mục 5.1 paper):

1. Teacher có thể "kiểm duyệt" nhãn sai (TIMIT có frame bị gán sai bởi HMM forced alignment).
2. Ở vùng $p(y|x)$ khó học, teacher đưa nhãn mềm đơn giản hơn, làm phẳng độ phức tạp của dữ liệu.
3. Nhãn 0/1 chỉ có 1 trong 183 output khác 0; teacher cho target khác 0 ở hầu hết output, dàn trải độ bất định khi không chắc và tập trung khi dễ. Lợi ích này được tăng cường khi dùng logit.

Paper xem các cơ chế này như **regularization**: SNN huấn luyện trên nhãn gốc overfit sớm hơn mạng sâu (kể cả với dropout; Figure 2), còn mimic thì không. Gợi ý của bài báo: nếu có regularizer tốt hơn cho mạng nông thì khoảng cách có thể biến mất.

Capacity (mục 5.2): hai mimic (8k và 160k hidden unit) trên các teacher có độ chính xác khác nhau cho thấy độ chính xác student tăng đều khi teacher tốt hơn (các đường song song đường chéo), SNN-MIMIC-160k luôn hơn 8k nhưng 8k có thể bằng 160k nếu học từ teacher tốt hơn. Kết luận paper: "little evidence that shallow models have limited capacity"; nút cổ chai là thủ tục học và regularization.

Song song vs tuần tự (5.3): mạng sâu mất 8–12 giờ trên GTX 580, mimic nông chỉ 1–2 giờ dù nhiều tham số hơn; khi suy luận với tài nguyên song song, mạng nông chỉ cần 2–3 "chu kỳ" thay vì tỷ lệ với độ sâu, lợi cho real-time.

Cái paper KHÔNG ablation chi tiết: không có bảng so sánh định lượng cho các loss (KL, L2 prob, L2 logit) trong bản này, chỉ có khẳng định trong text.

6. Giới hạn

Do tác giả nêu:

- Thuật toán "awkward": phụ thuộc vào một tập dữ liệu không nhãn lớn hoặc teacher rất chính xác, hoặc cả hai.
- Với TIMIT phải tái sử dụng tập train làm tập chuyển nên student không bao giờ đạt teacher (ECNN).
- Mục tiêu thực sự (huấn luyện mạng nông chính xác trực tiếp từ dữ liệu gốc) chưa đạt.

Nhận xét:

- Chỉ thử trên TIMIT và CIFAR-10, các mạng feed-forward cỡ ~10M–180M tham số; không có Transformer, không có chuỗi tự hồi quy.
- Student "nông" ở đây nhiều tham số hơn (hàng chục đến hàng trăm triệu) so với teacher CNN trên CIFAR-10; "cùng số tham số" chỉ đúng ở TIMIT (SNN-MIMIC-8k ~12M so với DNN ~12M). Về chi phí suy luận, mạng nông rộng 400k unit không rẻ.
- Chưa có kiểm soát kiểu "cùng teacher + cùng dữ liệu nhưng student sâu": không tách được hiệu ứng "độ sâu" khỏi hiệu ứng "học từ teacher" một cách sạch.
- Kết luận "độ sâu không cần thiết" mạnh hơn bằng chứng; về sau (ví dụ bài 41 và 10) người ta cho thấy khoảng cách capacity vẫn quan trọng khi student quá nhỏ.

7. Liên hệ với các bài khác

- **02 Hinton KD (2015):** tổng quát hoá ý tưởng. Hinton chỉ ra rằng regress logit là *trường hợp giới hạn* của KD với nhiệt độ cao (khi T lớn so với độ lớn logit, gradient xấp xỉ $\frac{1}{NT^2}(z_i - v_i)$ sau khi trung tâm hoá logit). Tức L2 trên logit (bài này) và KL trên soft target ở nhiệt độ cao (bài 02) cùng họ.

- **03 FitNets:** bắt chước thêm hidden layer, trả lời câu hỏi "student *sâu và mảnh* thì sao" ngược với kết luận "nông" ở bài này.
- **05 KD survey, 09 Efficacy of KD, 10 Does KD really work:** các bài sau phân tích kỹ khi nào bắt chước logit/xác suất giúp được và khi nào không.
- **15 Sequence-level KD, 36 Distilling step-by-step, 37 Orca:** dùng teacher gán nhãn cho dữ liệu chưa nhãn (data-level distillation) là hậu duệ trực tiếp của "pass unlabeled data through the large model" ở bài này.
- **41 Capacity gap law:** kiểm chứng định lượng rằng student không thể nhỏ vô hạn.

Gợi ý học tập

So sánh một student học nhãn gốc với cùng kiến trúc học logit teacher, giữ nguyên dữ liệu và ngân sách. Sau đó thay đổi lượng dữ liệu chưa gán nhãn để khảo sát vai trò của tập chuyển giao. Đây là gợi ý thực hành, không phải thí nghiệm bổ sung được báo cáo trong bài.

8. Tóm tắt công thức / khái niệm

Khái niệm	Công thức / ý nghĩa
Softmax	$p_k = e^{z_k} / \sum_j e^{z_j}$
Loss mimic (hồi quy logit)	$\frac{1}{2T} \sum_t \|g(x^{(t)}) - z^{(t)}\|_2^2$
Mạng nông	$g(x) = \beta f(Wx)$
Bottleneck tuyến tính	$W = UV, U \in \mathbb{R}^{H \times k}, V \in \mathbb{R}^{k \times D}$; bộ nhớ $O(HD) \to O(k(H+D))$
Chuẩn hoá logit	trừ mean, chia std theo từng target trên tập train
Ensemble teacher	trung bình logit của các thành viên
Model compression	gán nhãn dữ liệu không nhãn bằng teacher, huấn luyện student trên đó

9. Kết luận

Trên TIMIT, mạng một lớp ẩn học từ logit teacher đạt 21.6% PER so với 21.9% của mạng sâu cùng số tham số; kết quả so với CNN là 20.0% và 19.5%. Các thí nghiệm này cho thấy tín hiệu teacher và dữ liệu chuyển giao có thể giúp huấn luyện mạng nông, dù chưa chứng minh kết luận cho mọi kiến trúc.

Link gốc: https://arxiv.org/abs/1312.6184

Bài 02

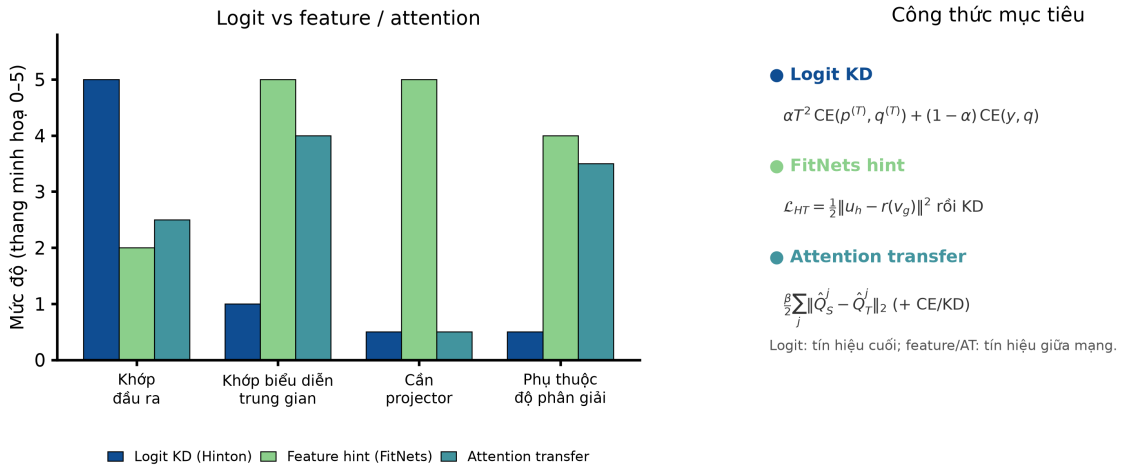
Distilling the Knowledge in a Neural Network (2015)

Tác giả: Geoffrey Hinton, Oriol Vinyals, Jeff Dean (Google)
Venue / arXiv: arXiv:1503.02531 (v1, 9/3/2015); bản văn không ghi nơi công bố.
Nguồn: https://arxiv.org/abs/1503.02531
Chủ đề: Knowledge distillation (KD) bằng soft target có nhiệt độ; chứng minh "khớp logit" là trường hợp đặc biệt; ensemble chuyên gia (specialists); soft target như regularizer.
Vị trí trong lộ trình: Đọc sau 00 (preface) và 01 (Ba & Caruana). Bài này đặt tên "distillation", đưa ra công thức chuẩn T + T^2 mà gần như mọi bài sau dùng lại. Tiếp theo: 03 (FitNets: thêm hint ở hidden layer), 04 (attention transfer), 05 (survey làm bản đồ), 06–07 (self/online distillation).

> Tài liệu học tập bằng tiếng Việt, là ghi chú diễn giải, không phải bản dịch paper.

1. Bối cảnh

Trung bình dự đoán của nhiều mô hình (ensemble) có thể cải thiện độ chính xác, nhưng khiến suy luận tốn kém. Bài báo đặt vấn đề tách hai giai đoạn: dùng mô hình lớn khi huấn luyện, sau đó chuyển những gì nó học được sang một mô hình nhẹ hơn để triển khai. Tác giả gọi quá trình chuyển giao này là distillation.



So sánh logit KD, feature hint (FitNets) và attention transfer theo tín hiệu khớp, projector và độ phụ thuộc không gian (minh hoạ, tự tính)

Bài báo xem tri thức là ánh xạ đã học từ đầu vào sang đầu ra, thay vì đồng nhất nó với một tập tham số cụ thể. Cách nhìn này cho phép chuyển hành vi dự đoán sang một kiến trúc khác.

Dark knowledge (thuật ngữ sau này, bài báo không dùng từ này trong bản này): khi huấn luyện, xác suất mô hình gán cho các lớp *sai* tuy nhỏ nhưng tỷ lệ giữa chúng mang thông tin. Ví dụ paper: ảnh xe BMW có xác suất nhỏ bị nhầm với xe tải rác, nhưng vẫn lớn gấp nhiều lần xác suất nhầm với củ cà rốt.

Công trình trước: Buciluǎ et al. 2006 (xem bài 01) dùng logit và L2 để khớp. Hinton nói cách tổng quát hơn: tăng nhiệt độ softmax.

Thuật ngữ:

- **Soft target:** phân phối xác suất (mềm) của teacher dùng làm nhãn.

- **Transfer set:** tập dữ liệu dùng ở giai đoạn truyền kiến thức (có thể là tập train gốc hoặc dữ liệu không nhãn).
- **Temperature T:** tham số làm phẳng phân phối softmax.

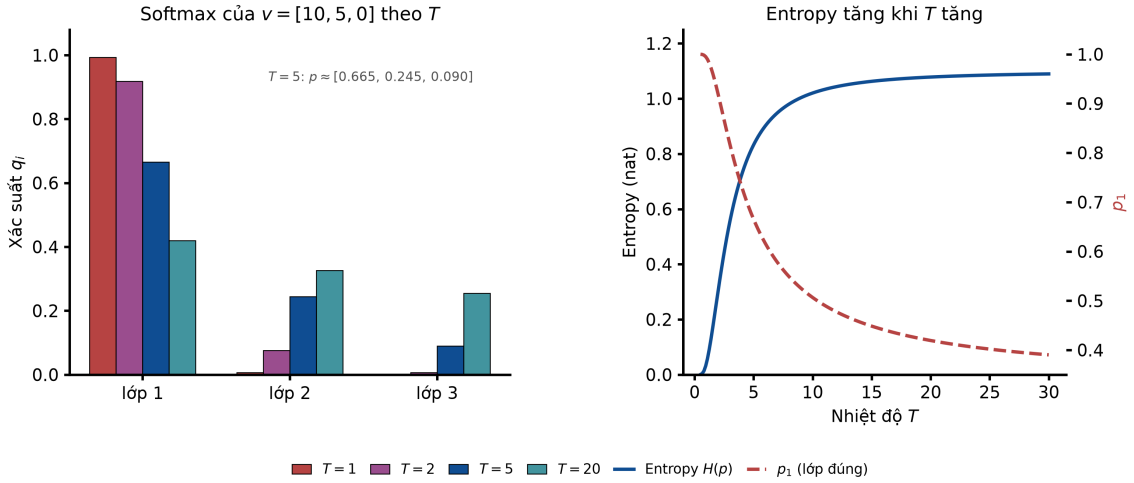
2. Ý tưởng chính

Teacher và student cùng dùng softmax ở nhiệt độ cao trong quá trình chưng cất. Phân phối mềm làm rõ quan hệ giữa các lớp; student học phân phối ấy, đồng thời có thể nhận thêm tín hiệu từ nhãn thật. Khi suy luận, nhiệt độ trở về $T = 1$. Ở giới hạn nhiệt độ cao, mục tiêu này liên hệ với phép khớp logit của các công trình trước. Trong thí nghiệm, một student cùng cỡ với từng mô hình thành viên thu được phần lớn lợi ích của ensemble gồm mười mô hình.

3. Phương pháp chi tiết

3.1 Softmax có nhiệt độ

$$q_i = \frac{\exp(z_i/T)}{\sum_j \exp(z_j/T)}$$



Softmax của logit $v = [10, 5, 0]$ theo nhiệt độ T: T lớn làm phẳng phân phối và tăng entropy, lộ quan hệ giữa các lớp sai (dark knowledge) (minh hoạ, tự tính)

$T = 1$ là softmax thường; T lớn làm phân phối phẳng hơn (entropy cao), $T \to \infty$ tiến đến đồng đều.

3.2 Loss kết hợp

Paper dùng trung bình có trọng số của hai cross-entropy:

1. CE với soft target: student dùng softmax ở nhiệt độ T, teacher dùng cùng nhiệt độ T để sinh p.
2. CE với nhãn thật: dùng cùng logit của student nhưng ở $T = 1$.

Paper nói kết quả tốt nhất thường đến từ **trọng số thấp hơn đáng kể cho loss thứ hai**. Vì độ lớn gradient của soft target co theo $1/T^2$, cần nhân loss thứ nhất với T^2 để tỷ lệ đóng góp giữa hai loss không đổi khi thay T.

Dạng viết thường gặp (là cách viết minh họa, tương thích mô tả paper):

$$\mathcal{L} = \alpha\, T^2\, \mathrm{CE}(p^{(T)}_{teacher},\, q^{(T)}_{student}) + (1-\alpha)\, \mathrm{CE}(y,\, q^{(1)}_{student})$$

(Với phân phối teacher cố định, CE khác KL chỉ một hằng số entropy của teacher, nên gradient như nhau.)

3.3 Gradient và vì sao có T^2 (Eq. 2–4)

Gọi z_i là logit student, v_i logit teacher, q_i và p_i là softmax ở nhiệt độ T tương ứng. Gradient CE theo logit:

$$\frac{\partial C}{\partial z_i} = \frac{1}{T}(q_i - p_i) = \frac{1}{T}\left(\frac{e^{z_i/T}}{\sum_j e^{z_j/T}} - \frac{e^{v_i/T}}{\sum_j e^{v_j/T}}\right)$$

Nếu T lớn so với độ lớn logit, khai triển bậc nhất $e^x \approx 1 + x$ với N lớp:

$$\frac{\partial C}{\partial z_i} \approx \frac{1}{T}\left(\frac{1 + z_i/T}{N + \sum_j z_j/T} - \frac{1 + v_i/T}{N + \sum_j v_j/T}\right)$$

Nếu logit đã được trung tâm hoá cho từng mẫu ($\sum_j z_j = \sum_j v_j = 0$):

$$\boxed{\frac{\partial C}{\partial z_i} \approx \frac{1}{NT^2}(z_i - v_i)}$$

Hai điều rút ra:

- Ở nhiệt độ cao, distillation tương đương cực tiểu hoá $\frac{1}{2}(z_i - v_i)^2$, tức **khớp logit là trường hợp đặc biệt** của distillation (đúng với điều kiện logit trung tâm hoá).
- Gradient co theo $1/T^2$, vì vậy nhân T^2 để bù.

Ở nhiệt độ thấp hơn, distillation ít chú ý các logit rất âm so với trung bình. Điều này có thể có lợi (logit đó gần như không bị ràng buộc bởi loss huấn luyện teacher nên có thể nhiễu) hoặc có hại (có thể chứa thông tin hữu ích); paper nói đây là câu hỏi thực nghiệm, và khi student quá nhỏ thì nhiệt độ trung bình tốt nhất, gợi ý rằng bỏ qua logit rất âm là có ích.

3.4 Ví dụ số nhỏ (minh họa)

Logit teacher $v = [10, 5, 0]$. Với $T = 1$: $p \approx [0.9933, 0.0066, 0.00005]$ (gần one-hot). Với $T = 5$: $v/T = [2, 1, 0]$, $p \approx [0.665, 0.245, 0.090]$. Ở $T = 5$ student thấy rõ lớp 2 gần lớp 1 hơn lớp 3; ở $T = 1$ thông tin này hầu như biến mất khỏi loss.

3.5 Ensemble làm teacher

Khi teacher là ensemble, soft target là trung bình số học hoặc hình học của các phân phối thành viên.

3.6 Pseudo-code

```
# Hinton distillation
for x, y in loader:                       # transfer set (có thể có hoặc không có y)
    with no_grad: v = teacher(x)          # logit teacher
    z = student(x)
    p = softmax(v / T)
    q = softmax(z / T)
    L_soft = T**2 * cross_entropy(p, q)  # = T^2 * (KL + const)
    L_hard = cross_entropy(y, softmax(z))# T = 1, nếu có nhãn
    L = alpha * L_soft + (1 - alpha) * L_hard    # alpha lớn (L_hard trọng số thấp)
    step(L)
inference: softmax(student(x))            # T = 1
```

4. Thí nghiệm và kết quả chính

4.1 MNIST (thử nghiệm sơ bộ)

- Teacher: 2 hidden layer × 1200 ReLU, dropout + weight constraints, jitter ảnh tới 2 pixel: **67 lỗi test**.
- Student 800-800 không regularization (hard label): **146 lỗi**.
- Student cùng cỡ học thêm khớp soft target của teacher ở $T = 20$: **74 lỗi**. Paper nhấn mạnh student nhận được cả kiến thức về tổng quát hoá từ dữ liệu dịch chuyển dù transfer set không chứa ảnh dịch chuyển.
- Nếu hidden layer ≥ 300 unit, mọi nhiệt độ > 8 cho kết quả gần nhau; nếu giảm còn 30 unit/lớp, nhiệt độ 2.5–4 tốt hơn rõ rệt.
- **Bỏ hết chữ số 3 khỏi transfer set:** student chỉ mắc 206 lỗi test, trong đó 133 lỗi nằm trên 1010 chữ số 3 của tập test; nguyên nhân chính là bias của lớp 3 quá thấp. Tăng bias lớp 3 thêm 3.5 thì còn 109 lỗi, trong đó 14 lỗi trên chữ 3 (đúng 98.6% các chữ 3 dù chưa từng thấy 3).
- Nếu transfer set chỉ chứa chữ 7 và 8: 47.3% lỗi; giảm bias của 7 và 8 đi 7.6 thì còn 13.2% lỗi.

4.2 Nhận dạng giọng nói (acoustic model)

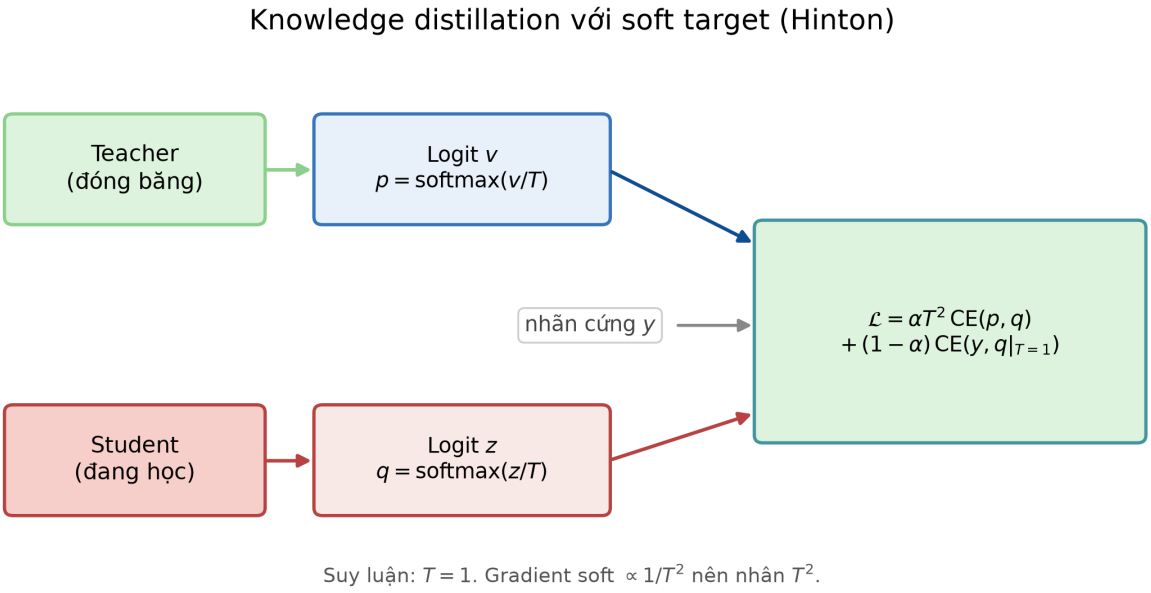
Mô hình: 8 hidden layer × 2560 ReLU, softmax 14,000 nhãn (trạng thái HMM), đầu vào 26 frame × 40 filter-bank, ~85M tham số, ~2000 giờ dữ liệu (~700M mẫu). Ensemble 10 mô hình khởi tạo ngẫu nhiên khác nhau. Nhiệt độ thử: [1, 2, 5, 10]; trọng số 0.5 cho CE hard target.

Hệ thống	Frame accuracy (test)	WER
Baseline	58.9%	10.9%
Ensemble 10 mô hình	61.1%	10.7%
Distilled single model	60.8%	10.7%

Hơn 80% mức cải thiện frame accuracy của ensemble được chuyển sang student cùng cỡ baseline. Paper so sánh với Li et al. (2014): họ distill ở $T = 1$ trên dữ liệu không nhãn lớn, chỉ thu hẹp 28% khoảng cách lỗi.

4.3 Soft target như regularizer (mục 6)

Huấn luyện mô hình 85M tham số chỉ trên 3% dữ liệu (~20M mẫu):



Sơ đồ knowledge distillation: teacher đóng băng sinh soft target ở nhiệt độ T, student cực tiểu hoá CE mềm (nhân T^2) kết hợp CE với nhãn cứng; suy luận dùng $T = 1$ (minh hoạ, tự tính)

Hệ thống & tập train	Train frame acc	Test frame acc
Baseline (100% data)	63.4%	58.9%
Baseline (3% data)	67.3%	44.5%
Soft targets (3% data)	65.4%	57.0%

Baseline 3% overfit nặng (phải early stopping, accuracy giảm mạnh sau khi đạt 44.5%); soft target không cần early stopping, hội tụ ở 57%, chỉ kém khoảng 2% so với dùng toàn bộ dữ liệu.

4.4 Specialists trên JFT

JFT: tập nội bộ Google, 100M ảnh có nhãn, 15,000 nhãn; mô hình nền huấn luyện khoảng 6 tháng. Ý tưởng: một generalist + nhiều specialist, mỗi specialist chỉ phân biệt một cụm lớp dễ nhầm (các lớp khác gộp thành "dustbin class"). Specialist khởi tạo từ trọng số generalist, huấn luyện với 50% mẫu từ tập con đặc biệt và 50% lấy ngẫu nhiên; hiệu chỉnh dustbin bằng cách cộng log của tỉ lệ oversampling vào logit dustbin. Cụm lớp lấy bằng online K-means trên cột ma trận hiệp phương sai của dự đoán generalist. Suy luận: tìm phân phối q cực tiểu hoá $\mathrm{KL}(p_g, q) + \sum_{m \in A_k} \mathrm{KL}(p_m, q)$ bằng gradient descent theo logit cho từng ảnh.

Hệ thống	Conditional test acc	Test acc
Baseline	43.1%	25.0%
+ 61 specialist (mỗi cái 300 lớp + dustbin)	45.9%	26.1%

Cải thiện tương đối 4.4% test accuracy; càng nhiều specialist phủ lớp đúng thì mức tăng càng lớn (Table 4: từ +3.4% khi 1 specialist phủ đến +14.1% khi 10 trở lên; nhóm 9 specialist +16.6%). Specialist huấn luyện trong vài ngày thay vì nhiều tuần. Paper nói rõ **chưa** chứng minh có thể distill specialist trở lại một mô hình đơn.

5. Phân tích / ablation

- **Nhiệt độ:** với student đủ lớn, mọi $T > 8$ gần như nhau; với student rất nhỏ (30 unit), T trung bình (2.5–4) tốt hơn.
- **Hard label phụ:** thêm một loss nhỏ với nhãn thật hữu ích vì student thường không khớp hoàn toàn soft target và "lệch về hướng đúng" giúp ích.
- **Soft target giàu thông tin:** entropy cao nghĩa là nhiều thông tin hơn mỗi mẫu và phương sai gradient giữa các mẫu thấp hơn, nên student có thể huấn luyện với ít dữ liệu hơn và learning rate cao hơn (theo paper).
- **Transfer set:** có thể là dữ liệu không nhãn hoặc tập train gốc.
- Bảng 5 là bằng chứng mạnh nhất cho vai trò regularizer.

6. Giới hạn

Do tác giả nêu:

- Chưa distill specialist ngược về một mô hình đơn.
- Đề xuất dùng soft target để chống overfit cho specialist với full softmax nhưng "đang khám phá", chưa có kết quả.
- Câu hỏi nhiệt độ thấp bỏ qua logit rất âm là lợi hay hại là vấn đề thực nghiệm.

Nhận xét:

- Chỉ thử bài toán phân loại (MNIST, ASR theo frame, JFT); không có chuỗi sinh tự hồi quy, nên không đề cập exposure bias hay vấn đề tokenizer.
- Student cùng kích thước với teacher đơn trong thí nghiệm giọng nói; chưa kiểm tra student nhỏ hơn nhiều về capacity gap.
- JFT là dữ liệu nội bộ, không tái lập được. Kết quả MNIST với "chữ số 3 chưa thấy" cần chỉnh bias thủ công bằng tập test, nên không phải kết quả sạch.
- Hiệu quả với bảng đầy đủ ablation α, T không được trình bày ngoài mô tả văn bản (chỉ nêu tập T thử và trọng số 0.5).

7. Liên hệ với các bài khác

- **01 Ba & Caruana:** logit regression là giới hạn $T \to$ lớn của bài này (Eq. 4); Hinton cũng nói student cần transfer set không nhãn hoặc tập train.
- **03 FitNets, 04 Attention transfer:** thêm tín hiệu ở hidden layer, vì soft target ở output có thể không đủ khi student rất sâu và mảnh.
- **05 KD survey:** bản đồ phân loại; bài này là response-based KD cơ bản.
- **06 Born-again, 07 Deep mutual learning:** cùng loss nhưng teacher cùng kích thước hoặc student học lẫn nhau.
- **08 Teacher assistant, 09, 10, 41:** khi student quá nhỏ, nhiệt độ trung bình/teacher trung gian; bài này mới chỉ gợi ý ở thí nghiệm 30 unit.
- **14 Decoupled KD:** tách KL thành phần target-class và non-target-class, liên quan trực tiếp đến "dark knowledge ở lớp sai".
- **15 Sequence-level KD, 28–34:** KL token-level trên từ vựng là bản mở rộng của loss này cho LLM.
- **45 Sparse logit sampling:** khi từ vựng lớn, không thể lưu toàn bộ logit; ước lượng thưa và độ chệch.

Gợi ý học tập

Trên cùng cặp teacher–student và tập dữ liệu, thử $T \in \{1, 2, 4\}$ và $\alpha \in \{0.5, 0.9, 1.0\}$; khi thay nhiệt độ, giữ rõ quy ước nhân T^2. Có thể so sánh soft target với nhãn cứng khi giảm lượng dữ liệu có nhãn xuống 3%. Đây là gợi ý học tập, không phải kết quả mới của bài báo.

8. Tóm tắt công thức / khái niệm

Khái niệm	Công thức / ý nghĩa
Softmax nhiệt độ	$q_i = \exp(z_i/T)/\sum_j \exp(z_j/T)$
Loss kết hợp	$\alpha T^2\,\mathrm{CE}(p^{(T)}, q^{(T)}) + (1-\alpha)\,\mathrm{CE}(y, q^{(1)})$ (trọng số hard thấp)
Gradient theo logit	$\partial C/\partial z_i = \frac{1}{T}(q_i - p_i)$
Giới hạn T cao	$\partial C/\partial z_i \approx \frac{1}{NT^2}(z_i - v_i)$ khi logit zero-mean
Hệ quả	khớp logit (Ba & Caruana) là trường hợp đặc biệt; nhân T^2 để giữ tỉ lệ
Specialist	chỉ phân biệt cụm lớp dễ nhầm + dustbin class
Suy luận ensemble chuyên gia	$\min_q\ \mathrm{KL}(p_g, q) + \sum_{m\in A_k} \mathrm{KL}(p_m, q)$
Soft target as regularizer	3% data: 44.5% (hard) vs 57.0% (soft) test frame acc

9. Kết luận

Bài báo trình bày KD dựa trên phân phối đầu ra: softmax có nhiệt độ, loss kết hợp nhãn mềm và nhãn cứng, cùng hệ số T^2. Ở giới hạn nhiệt độ cao, cách này liên hệ với khớp logit. Trong thí nghiệm chỉ dùng 3% dữ

liệu, soft target đạt 57.0% so với 44.5% của nhãn cứng; thí nghiệm ensemble cũng giữ được hơn 80% mức cải thiện frame accuracy.

Link gốc: https://arxiv.org/abs/1503.02531

Bài 03

FitNets: Hints for Thin Deep Nets (2015)

Tác giả: Adriana Romero, Nicolas Ballas, Samira Ebrahimi Kahou, Antoine Chassang, Carlo Gatta, Yoshua Bengio
Venue / arXiv: arXiv:1412.6550 (v4, 27/3/2015); ICLR 2015 (ghi trên file)
Nguồn: https://arxiv.org/abs/1412.6550
Chủ đề: Feature-based / hint-based distillation: dùng đầu ra hidden layer của teacher làm "gợi ý" để huấn luyện student sâu hơn và mảnh hơn; huấn luyện hai giai đoạn.
Vị trí trong lộ trình: Đọc sau 02 (Hinton KD). Bài này là bài đầu tiên của nhóm "feature distillation": thêm tín hiệu ở hidden layer, dùng regressor (projector) để khớp chiều. Tiếp theo: 04 (Attention transfer: khớp bản đồ attention thay vì feature thô), 12 (Relational KD), 13 (CRD).

> Tài liệu học tập bằng tiếng Việt, là ghi chú diễn giải, không phải bản dịch paper.

1. Bối cảnh

Các phương pháp nén trước đó (bài 01, 02) cho student *nông và rộng* hoặc *cùng độ sâu* với teacher. Tác giả lập luận độ sâu là yếu tố quan trọng của representation learning (tái sử dụng feature, biểu diễn trừu tượng hơn), và thực nghiệm cho thấy mạng rất sâu (19, 22 lớp) thắng ImageNet. Nhưng mạng sâu và mảnh (thin) rất **khó tối ưu**: nhiều phi tuyến nối tiếp tạo hàm không lồi cao.

Các cách hỗ trợ tối ưu đã biết mà paper nhắc: pretraining theo từng lớp (greedy layer-wise), thêm giám sát vào lớp trung gian (deeply-supervised nets), curriculum learning (học từ dễ đến khó).

Ý tưởng của bài: **dùng teacher để cho hint ở lớp giữa**, nhằm (i) cho phép student sâu hơn teacher, (ii) mảnh hơn nên ít tham số và nhanh hơn.

Thuật ngữ:

- **Hint layer:** hidden layer của teacher dùng làm mục tiêu.
- **Guided layer:** hidden layer của student được dẫn dắt để dự đoán hint.
- **Regressor:** ánh xạ nhỏ gắn trên guided layer để đưa về cùng chiều với hint (ngày nay gọi là projector/adapter).
- **Thin / wide:** ít / nhiều kênh mỗi lớp.

2. Ý tưởng chính

FitNets chọn một lớp trung gian của teacher làm *hint* và một lớp của student để nhận hướng dẫn. Trước hết, phần student đến lớp được chọn cùng một regressor học dự đoán hint bằng loss L2. Sau đó, toàn bộ student được khởi tạo từ kết quả này và tiếp tục học bằng KD, với λ giảm dần. Cách khởi tạo ấy giúp tối ưu các mạng sâu và mảnh trong những thiết lập được khảo sát.

3. Phương pháp chi tiết

3.1 Tóm tắt KD dùng trong bài báo

Với $P_T = \text{softmax}(a_T)$, $P_S = \text{softmax}(a_S)$ và nới nhiệt độ $\tau > 1$:

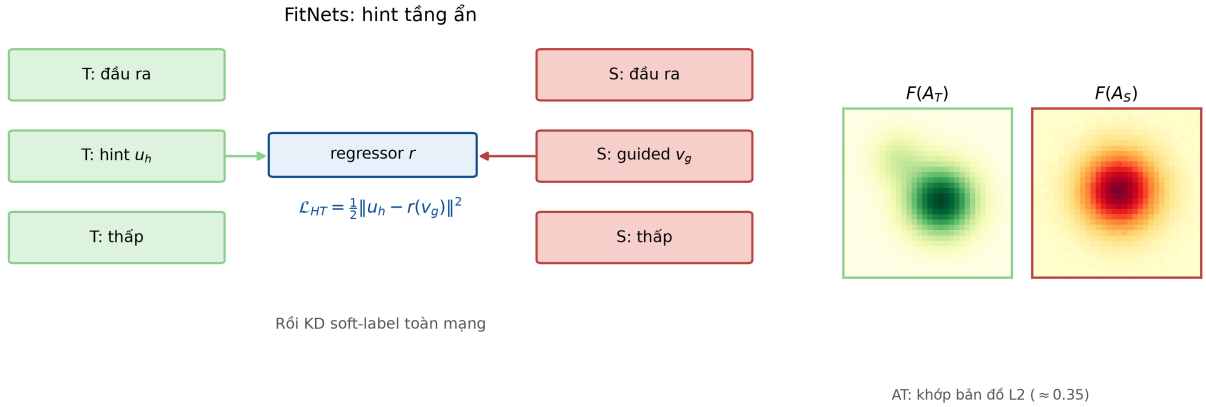
$$P_T^\tau = \text{softmax}(a_T/\tau), \quad P_S^\tau = \text{softmax}(a_S/\tau)$$

$$\boxed{\mathcal{L}_{KD}(W_S) = H(y_{true}, P_S) + \lambda\, H(P_T^{\tau}, P_S^{\tau})}$$

với H là cross-entropy, λ cân bằng hai số hạng. (Khác Hinton ở chỗ không thấy nhân τ^2 trong công thức này; paper đặt $\tau = 3$.)

3.2 Hint-based training

Hint $u_h(x; W_{Hint})$ là đầu ra của teacher đến lớp h; student có $v_g(x; W_{Guided})$ đến lớp g; regressor $r(\cdot; W_r)$. Loss:



FitNets khớp hint qua regressor ở lớp giữa; attention transfer khớp bản đồ kích hoạt đã chuẩn hoá L2 giữa teacher và student (minh hoạ, tự tính)

$$\boxed{\mathcal{L}_{HT}(W_{Guided}, W_r) = \tfrac{1}{2}\left\| u_h(x; W_{Hint}) - r\big(v_g(x; W_{Guided}); W_r\big)\right\|^2}$$

Lưu ý của bài báo: u_h và r phải "so sánh được", cùng loại phi tuyến.

Chọn lớp: hint càng sâu thì student càng ít tự do, dễ bị over-regularize. Paper chọn **lớp giữa của teacher** làm hint và **lớp giữa của student** làm guided (tuy trong thí nghiệm cụ thể, ví dụ guided là lớp 11 mimic lớp 2 của teacher, nên "giữa" là xấp xỉ; các cặp được liệt kê ở bảng kiến trúc trong phụ lục).

Regressor tích chập: regressor fully-connected tốn $N_{h,1}N_{h,2}O_hN_{g,1}N_{g,2}O_g$ tham số. Paper dùng regressor *tích chập* với kernel $k_1 \times k_2$ sao cho $N_{g,i} - k_i + 1 = N_{h,i}$, tức đầu ra cùng kích thước không gian với hint, chỉ tốn $k_1k_2O_hO_g$ tham số.

3.3 Huấn luyện hai giai đoạn (Algorithm 1)

```
Input: W_S (student, khởi tạo ngẫu nhiên), W_T (teacher đã học), g, h
W_Hint   = teacher params lớp 1..h
W_Guided = student params lớp 1..g
W_r      = khởi tạo ngẫu nhiên nhỏ
# Stage 1: pretrain tới guided layer
W_Guided* = argmin_{W_Guided, W_r}  L_HT(W_Guided, W_r)
gán W_S[1..g] = W_Guided*
# Stage 2: KD toàn mạng
W_S* = argmin_{W_S}  L_KD(W_S)      # lambda giảm tuyến tính
```

Regressor chỉ dùng ở giai đoạn 1 và bị bỏ sau đó.

3.4 Liên hệ curriculum learning (mục 2.4)

Paper giải thích: số hạng CE với teacher trong $\mathcal{L}_{KD}$ làm các mẫu mà teacher tự tin (gần one-hot) có ảnh hưởng mạnh hơn (mẫu teacher kém tự tin cho phân phối gần đều nên "đẩy" tham số yếu hơn), tức là đo "độ dễ" của mẫu qua độ tự tin của teacher. Sau đó giảm λ tuyến tính để dần chuyển sang mẫu khó. Curriculum có hai giai đoạn: học khái niệm trung gian (hint), rồi học toàn mạng. Đây là lập luận của tác giả (chưa phải thí nghiệm kiểm chứng riêng).

3.5 Siêu tham số (phụ lục)

$\tau = 3$; λ khởi tạo 4 và giảm tuyến tính trong 500 epoch tới 1 (CIFAR); RMSProp, lr 0.005, batch 128; khởi tạo trọng số $U(-0.005, 0.005)$; dừng sau 100 epoch không cải thiện validation, tối đa 500 epoch.

4. Thí nghiệm và kết quả chính

Teacher đều là mạng maxout conv của Goodfellow et al. (2013). FitNet là chuỗi conv 3×3 + maxout, nhiều lớp hơn và ít kênh hơn.

4.1 Các benchmark

Dataset	Teacher	FitNet	Ghi chú
CIFAR-10	~9M tham số, 90.18%	~2.5M, **91.61%** (19 lớp, FitNet 4)	tốt hơn teacher; so với mimic ensemble (Ba & Caruana) 85.8% (~70M), ít hơn khoảng 28 lần tham số
CIFAR-100	~9M, 63.54%	~2.5M, **64.96%**	state-of-the-art tham chiếu trong bài báo: DSN 65.43%
SVHN	~4.9M, 2.38% lỗi	~1.5M, 2.42% lỗi	"ngang teacher với 32% capacity"
MNIST	~361K, 0.55% lỗi	~30K, **0.51%**	backprop thường 1.9%, KD 0.65%
AFLW (16×16 mặt/không mặt)	teacher 4.21% lỗi	FitNet 1 (ít hơn 15x phép nhân): KD 4.58%, HT 2.55%; FitNet 2 (ít hơn 2.5x): KD 1.95%, HT 1.85%	mô tả trong text

Teacher CIFAR-10/100 có 3 conv maxout (96-192-192) + FC maxout 500. Student CIFAR-10 trong Bảng 1 có 17 conv maxout + 1 FC maxout + softmax; lớp 11 của student mimic lớp 2 của teacher. Các số trong bảng trên chép từ Table 1–4 và text mục 3.4.

Các hệ thống mạnh nhất khác trong Table 1 (CIFAR-10): Maxout 90.65%, Network in Network 91.2%, Deeply-Supervised Networks 91.78%; "mimic" của Ba & Caruana: 84.6% (~54M), 84.9% (~70M), 85.8% ensemble (~70M).

4.2 Độ sâu dưới ngân sách tính toán cố định (mục 4.1)

Thí nghiệm CIFAR-10 với hai ngân sách khoảng 30M và 107M phép nhân, mạng 3/5/7/9 conv (thực tế đồ thị trục x là số lớp 5, 7, 9, 11):

- Backprop thường không huấn luyện được mạng > 5 lớp ở ngân sách 30M.
- KD huấn luyện được tới 7 lớp, rồi chạm giới hạn.
- Hint-based training (HT) huấn luyện được mạng 13 lớp ở 30M phép nhân.
- Ở 107M, mạng 7 lớp với HT tăng +0.7% (từ 89.45% lên 90.1%) so với không hint.
- Mạng 11 lớp (HT) vượt mạng 5 lớp 4.11% tuyệt đối ở 107M và 3.4% ở 30M.

4.3 Đánh đổi tốc độ / độ chính xác (Table 5, CIFAR-10)

Mạng	Lớp	Tham số	Phép nhân	Acc	Tăng tốc	Tỉ lệ nén
Teacher	5	~9M	~725M	90.18%	1	1
FitNet 1	11	~250K	~30M	89.01%	13.36	36
FitNet 2	11	~862K	~108M	91.06%	4.64	10.44
FitNet 3	13	~1.6M	~392M	91.10%	1.37	5.62
FitNet 4	19	~2.5M	~382M	91.61%	1.52	3.60

FitNet 1 nhanh gấp một bậc độ lớn, chỉ giảm 1.3% (so 90.18% và 89.01%). FitNet 2 vượt teacher 0.9% với tốc độ 4.64x. (Tăng tốc đo bằng tổng thời gian suy luận trên GPU.)

5. Phân tích / ablation

- **HT vs KD thuần:** khác biệt duy nhất là điểm khởi đầu (ngẫu nhiên hay lấy từ hint). Paper cho rằng hint đưa student vào vùng khởi tạo tốt hơn (lợi ích tối ưu hoá) và còn là regularizer mạnh hơn KD (test tốt hơn khi cố định dung lượng).
- **Hint từ hidden layer teacher vs hint từ nhãn (mục 3.1, các lựa chọn 1–4):** (1) stage-wise: stage 1 tối ưu nửa mạng đầu theo nhãn, stage 2 tối ưu toàn mạng theo nhãn: stage 2 không học được. (2) tương tự nhưng stage 2 theo Eq. (2): không học được. (3) tối ưu chung hint có giám sát + nhãn ở output: thử nhiều khởi tạo và learning rate RMSprop, không có mạng nào học được. (4) tương tự với Eq. (2): không học được. Thêm hint giám sát ở mọi lớp theo kiểu Deeply-Supervised Nets (DSN) với kiến trúc 19 lớp đạt **88.2%**, kém xa 91.61% của FitNet. Paper diễn giải: hint rất phân biệt theo lớp có thể quá "gắt"; hint mềm hơn từ teacher cho tổng quát hoá tốt hơn.
- **Độ sâu quan trọng:** với cùng ngân sách phép nhân, mạng sâu hơn (có hint) tốt hơn mạng nông hơn.
- **MNIST (kiểm chứng đóng góp của hint):** cùng kiến trúc student: backprop 1.9%, KD 0.65%, HT 0.51%.
- Paper **không** có ablation riêng cho vị trí hint/guided layer (chỉ chọn "giữa" theo trực giác), cũng không quét λ hay τ.

6. Giới hạn

Do tác giả nêu:

- Hint sâu hơn dễ over-regularize; cần chọn cặp hint/guided để student không bị khoá quá.
- Phân tích mức dư thừa trong filter của FitNet, kết hợp với matrix factorization và quantization được để làm future work; các phương pháp đó được coi là bổ trợ.

Nhận xét:

- Chỉ có một cặp hint/guided, chọn bằng heuristic; không có phân tích độ nhạy. Hint lấy từ lớp 2 của teacher chỉ 3 lớp conv (rất nông), khác xa teacher kiểu Transformer.
- Quy trình hai giai đoạn thêm bước huấn luyện và nhiều siêu tham số (λ lịch giảm, τ, lr riêng).
- Regressor/feature L2 giả định hai không gian đặc trưng có thể dự đoán nhau; không có cơ sở lý thuyết cho "khớp tuyệt đối" feature, và tín hiệu L2 trên feature có thể bị chi phối bởi các kênh có biên độ lớn (nhận xét bổ sung, bài 04 chỉ ra cách chuẩn hoá).
- Thí nghiệm là CNN ảnh nhỏ (32×32, 16×16), không có chuỗi hay Transformer.
- "Student vượt teacher" xảy ra trên teacher tương đối nông (3 conv) và có thể do kiến trúc student tốt hơn chứ không do KD thuần; paper cố gắng kiểm soát bằng các baseline backprop/KD trên cùng kiến trúc ở MNIST và hình 2.

7. Liên hệ với các bài khác

- **02 Hinton KD:** stage 2 của FitNets chính là KD; FitNets bổ sung khởi tạo bằng hint.
- **04 Attention transfer:** thay vì khớp feature thô qua regressor, khớp bản đồ attention (tổng bình phương/lũy thừa theo kênh, rồi chuẩn hoá), không cần regressor vì kích thước kênh bị gộp.
- **12 Relational KD, 13 CRD:** khớp quan hệ giữa các mẫu thay vì từng feature; bước tiến tiếp.
- **19 MiniLM, 18 TinyBERT, 17 Patient KD:** feature/hidden-state KD cho Transformer; TinyBERT dùng projection tương tự regressor.
- **23 AM-RADIO, 24 CLIP-KD, 26 LLaVA-KD:** feature distillation giữa các mô hình thị giác.
- **20 Causal distillation:** khớp hidden state theo can thiệp nhân quả.
- **10 Does KD really work:** nghiên cứu mức độ student khớp teacher; nhắc rằng khớp feature không tự động đồng nghĩa với khớp hàm.

Gợi ý học tập

Để khảo sát vai trò của hint, so sánh student có và không có giai đoạn khởi tạo bằng đặc trưng trung gian, giữ các điều kiện huấn luyện khác nhất quán. Thử các cặp lớp và cách ánh xạ chiều; hiệu quả có thể phụ thuộc cả vị trí lớp lẫn regressor.

8. Tóm tắt công thức / khái niệm

Khái niệm	Công thức / ý nghĩa
Loss KD	$H(y, P_S) + \lambda H(P_T^\tau, P_S^\tau), \tau = 3$
Hint loss	$\frac{1}{2}\|u_h(x) - r(v_g(x))\|^2$
Regressor conv	kernel $k_i = N_{g,i} - N_{h,i} + 1$, tham số $k_1 k_2 O_h O_g$
Stage 1	huấn luyện student tới guided layer + regressor theo hint
Stage 2	KD toàn mạng với λ giảm tuyến tính từ 4 về 1
Kết quả chính	CIFAR-10: 91.61% (~2.5M) vs teacher 90.18% (~9M); MNIST 0.51% vs 0.55%
Hint hay DSN?	hint từ teacher 91.61% vs DSN theo nhãn 88.2% (cùng kiến trúc 19 lớp)

9. Kết luận

FitNets dùng đặc trưng lớp giữa của teacher để khởi tạo student sâu và mảnh trước khi chưng cất toàn mạng. Trên CIFAR-10, các cấu hình student ít tham số hơn khoảng 3.6 đến 36 lần; một kết quả đạt 91.61% so với 90.18% của teacher, còn DSN dùng hint cứng đạt 88.2%.

Link gốc: https://arxiv.org/abs/1412.6550

Bài 04

Paying More Attention to Attention: Improving the Performance of Convolutional Neural Networks via Attention Transfer (2017)

Tác giả: Sergey Zagoruyko, Nikos Komodakis
Venue / arXiv: arXiv:1612.03928 (v3, 12/2/2017); ICLR 2017 (ghi trên file)
Nguồn: https://arxiv.org/abs/1612.03928
Chủ đề: Feature-based distillation với "attention map" (activation-based và gradient-based), chuẩn hoá L2, kết hợp với KD, kết quả trên CIFAR-10, Scenes, CUB, ImageNet.
Vị trí trong lộ trình: Đọc sau 03 (FitNets). FitNets khớp feature thô qua regressor; bài này nói: khớp một bản tóm tắt không gian của feature (attention map) tốt hơn và không cần tham số mới. Tiếp theo: 05 (survey, đặt AT vào bản đồ), 12 (Relational KD), 13 (CRD).

> Tài liệu học tập bằng tiếng Việt, là ghi chú diễn giải, không phải bản dịch paper.

1. Bối cảnh

Trước bài này, các cách chuyển giao tri thức (Buciluǎ 2006, Ba & Caruana, Hinton, FitNets) chủ yếu dựa trên output hoặc feature thô. FitNets chọn student mảnh và sâu. Tác giả lập luận ngược lại: theo "Wide Residual Networks" (Zagoruyko & Komodakis 2016), sau một độ sâu nhất định lợi ích đến chủ yếu từ dung lượng (số tham số), và mạng mảnh-sâu khó song song hoá hơn mạng rộng. Vì vậy ông chọn hướng **student nông hơn** teacher.

Câu hỏi của bài báo: teacher có thể giúp student bằng cách cho biết "nó nhìn vào đâu" không? Cần định nghĩa chính xác "attention" cho CNN.

Thuật ngữ:

- **Attention map (ở đây):** bản đồ không gian 2D $H \times W$ tóm tắt tensor kích hoạt $C \times H \times W$, cho biết vùng nào của ảnh mạng kích hoạt mạnh. Không phải self-attention của Transformer.
- **Activation-based vs gradient-based:** attention định nghĩa theo giá trị kích hoạt, hoặc theo gradient của loss theo đầu vào.
- **Double backpropagation:** lan truyền ngược lần hai qua gradient (đạo hàm bậc hai hỗn hợp).

2. Ý tưởng chính

Từ tensor kích hoạt $C \times H \times W$ của CNN, phương pháp gộp theo kênh để tạo bản đồ không gian $H \times W$, rồi chuẩn hóa L2. Student được huấn luyện để khớp các bản đồ của teacher tại nhiều vị trí; loss này bổ sung cho cross-entropy và có thể kết hợp với KD. Trong các thí nghiệm, cách khớp bản đồ cải thiện nhiều kiến trúc student và tốt hơn cách khớp toàn tensor theo kiểu FitNets.

3. Phương pháp chi tiết

3.1 Activation-based attention

Với $A \in \mathbb{R}^{C\times H\times W}$, hàm ánh xạ $F : \mathbb{R}^{C\times H\times W} \to \mathbb{R}^{H\times W}$. Giả định ngầm: **giá trị tuyệt đối của kích hoạt neuron cho biết mức quan trọng của neuron đó với đầu vào**. Ba lựa chọn paper xét ($A_i = A(i, :, :)$):

- $F_{sum}(A) = \sum_{i=1}^{C} |A_i|$
- $F^p_{sum}(A) = \sum_{i=1}^{C} |A_i|^p, p > 1$

- $F^p_{max}(A) = \max_{i=1,\dots,C} |A_i|^p$, $p > 1$

Tính chất theo paper: p lớn thì tập trung hơn vào vị trí kích hoạt cao nhất (vùng phân biệt nhất); F_{max} chỉ xét một kênh trên mỗi vị trí, còn F_{sum} ưu tiên vị trí có nhiều neuron kích hoạt mạnh. Quan sát trực quan: mạng chính xác hơn có attention tương quan không gian cao hơn với đối tượng; lớp thấp kích hoạt ở điểm có gradient thấp-cấp, lớp giữa ở vùng phân biệt (mắt, bánh xe), lớp cao ở toàn bộ đối tượng.

3.2 Hàm mất mát attention transfer

Gọi I là tập chỉ số cặp lớp (student, teacher) cùng độ phân giải không gian; $Q^j_S = \text{vec}(F(A^j_S))$, $Q^j_T = \text{vec}(F(A^j_T))$. Tổng loss:

$$\boxed{\mathcal{L}_{AT} = \mathcal{L}(W_S, x) + \frac{\beta}{2} \sum_{j \in I} \left\| \frac{Q^j_S}{\|Q^j_S\|_2} - \frac{Q^j_T}{\|Q^j_T\|_2} \right\|_p}$$

với $\mathcal{L}$ là cross-entropy, p là loại chuẩn (thí nghiệm dùng $p = 2$). **Chuẩn hoá L2 từng bản đồ được paper nhấn mạnh là quan trọng cho thành công**: nó loại bỏ khác biệt về thang giá trị giữa teacher và student, chỉ giữ *hình dạng* (vùng nào nổi bật). Có thể dùng chuẩn hoá L1 thay thế.

Vị trí đặt loss với ResNet: nếu cùng độ sâu, đặt sau mỗi residual block; nếu khác độ sâu, đặt ở đầu ra của mỗi *nhóm* residual block (Figure 5). Nếu độ phân giải không khớp, có thể nội suy.

Kết hợp KD: chỉ cần cộng thêm số hạng cross-entropy giữa phân phối nới nhiệt độ của teacher và student; AT thêm rất ít chi phí vì teacher đã chạy forward cho KD.

3.3 Gradient-based attention

Định nghĩa attention là gradient của loss theo đầu vào (độ nhạy đầu ra với thay đổi tại mỗi vị trí đầu vào):

$$J_S = \frac{\partial}{\partial x}\mathcal{L}(W_S, x), \quad J_T = \frac{\partial}{\partial x}\mathcal{L}(W_T, x), \quad \mathcal{L}_{AT} = \mathcal{L}(W_S, x) + \frac{\beta}{2}\|J_S - J_T\|_2$$

Đạo hàm theo W_S chứa số hạng đạo hàm bậc hai hỗn hợp $\partial^2\mathcal{L}/\partial W_S \partial x$ nên cần lan truyền ngược lần hai (như double backpropagation của Drucker & LeCun 1992; chi phí lần hai xấp xỉ lần đầu, bỏ forward).

Paper cũng đề xuất ràng buộc bất biến lật ngang cho gradient attention:

$$\mathcal{L}_{sym}(W, x) = \mathcal{L}(W, x) + \frac{\beta}{2} \left\| \frac{\partial}{\partial x}\mathcal{L}(W, x) - \text{flip}\left(\frac{\partial}{\partial x}\mathcal{L}(W, \text{flip}(x))\right) \right\|_2$$

Đây là ràng buộc mềm, có tác dụng regularization theo quan sát của tác giả.

3.4 Pseudo-code (activation-based AT)

```
for x, y in loader:
    with no_grad: acts_T = teacher.forward_collect(x, groups)     # tensor C x H x W
    ↪  theo nhóm
    logits_S, acts_S = student.forward_collect(x, groups)
    loss = cross_entropy(logits_S, y)
    if use_KD: loss += alpha_term(KD(logits_S, teacher_logits, T=4))   # alpha=0.9
    ↪  (CIFAR)
    for j in pairs:                                  # cặp nhóm cùng độ phân giải
        QS = normalize( flatten( sum_c (acts_S[j]**2) ) )   # F_sum^2 + L2 norm
        QT = normalize( flatten( sum_c (acts_T[j]**2) ) )
```

```
        loss += (beta_j / 2) * norm_p(QS - QT)       # p = 2
    step(loss)
```

Ví dụ nhỏ (minh họa): tensor 2 kênh × 1×2: kênh 1 = $[3, 0]$, kênh 2 = $[4, 0]$ → $F^2_{sum} = [25, 0]$ → chuẩn hoá L2: $[1, 0]$. Nếu student có $[0.6, 0.8]$ bản đồ chuẩn hoá thì khoảng cách $\approx \sqrt{0.4^2 + 0.8^2} = 0.89$. Chỉ hình dạng (vị trí nổi bật) được so, độ lớn thì không.

4. Thí nghiệm và kết quả chính

4.1 CIFAR-10 (activation-based, Table 1)

Cài đặt: augmentation lật ngang + random crop, batch norm; KD với temperature 4, $\alpha = 0.9$; sai số là **trung vị 5 lần chạy**. Lỗi test (%):

Student	Teacher	Student	AT	F-ActT	KD	AT+KD	Teacher
NIN-thin (0.2M)	NIN-wide (1M)	9.38	8.93	9.05	8.55	8.33	7.28
WRN-16-1 (0.2M)	WRN-16-2 (0.7M)	8.77	7.93	8.51	7.41	7.51	6.31
WRN-16-1 (0.2M)	WRN-40-1 (0.6M)	8.77	8.25	8.62	8.39	8.01	6.58
WRN-16-2 (0.7M)	WRN-40-2 (2.2M)	6.31	5.85	6.24	6.08	5.71	5.23

F-ActT = chuyển giao toàn bộ activation kiểu FitNets (L2 trên activation đầy đủ, 1×1 conv để khớp hình dạng). Quan sát: AT luôn tốt hơn student baseline; AT+KD tốt nhất trong ba hàng (hàng WRN-16-2 → WRN-16-1 thì KD thuần 7.41 và AT+KD 7.51 tương đương, KD nhỉnh hơn một chút).

4.2 Gradient-based và so sánh (Table 3)

Baseline là NIN mảnh 0.2M tham số, không batch norm, chỉ lật ngang, một lần chạy. Teacher (cho AT và KD) là wide NIN 1M. Lỗi CIFAR-10 (%):

Phương pháp	Lỗi
Baseline	13.5
min-ℓ_2 (Drucker & LeCun)	12.5
gradient-based AT	12.1
KD	12.1
symmetry norm	11.8
activation-based AT	11.2

Activation-based AT tốt nhất trong thiết lập này. Paper lưu ý phải huấn luyện một teacher mới (có batch norm, không ReLU trước pooling) để áp dụng AT trong cài đặt này.

4.3 Transfer learning (Table 4) và ImageNet (Table 5)

Finetune ResNet-18 (student) với teacher ResNet-34 đã finetune; loss F^2_{sum} ở 2 nhóm cuối. Lỗi (%) :

	ImageNet → CUB	ImageNet → Scenes
student ResNet-18	28.5	28.2
KD	27 (-1.5)	28.1 (-0.1)

	ImageNet → CUB	ImageNet → Scenes
AT	27 (-1.5)	27.1 (-1.1)
teacher ResNet-34	26.5	26

ImageNet (single crop validation error, top-1 / top-5 %): ResNet-18: 30.4 / 10.8; ResNet-18 + AT: 29.3 / 10.0; ResNet-34: 26.1 / 8.3. Tức AT cải thiện 1.1% top-1 và 0.8% top-5. Chỉ dùng 2 loss ở 2 nhóm cuối, không chỉnh siêu tham số; hai loss AT được thêm vào ở epoch 60/100 (theo chú thích bảng). Paper nói KD không cho kết quả tích cực trên ImageNet với cặp này (làm chậm hội tụ) và đoán AT là phương pháp chuyển giao tri thức đầu tiên chạy thành công trên ImageNet.

5. Phân tích / ablation

- **Số điểm đặt loss:** WRN-16-2 → WRN-16-1: một loss ở group1/group2/group3 riêng cho lỗi 8.11 / 7.96 / 7.97; cả ba loss: 7.93. Mỗi loss đóng góp thêm một phần.
- **Hàm ánh xạ attention (Table 2, lỗi CIFAR-10, trung vị 5 lần chạy, không AT là 8.77):** F_{sum} 7.99; F^2_{sum} 7.93; F^4_{sum} 8.09; F^2_{max} 8.08. (Trong text trích xuất, ký hiệu mũ khó đọc, thứ tự theo bảng gốc: F_{sum}, F^2_{sum}, F^4_{sum}, F^1_{max} hoặc F^2_{max}, nên chỉ nên tin xu hướng: các hàm dựa trên tổng cho kết quả gần nhau và tốt hơn hàm dựa trên max.) Paper chọn F^2_{sum}.
- **Hệ số β:** thường quanh 0.1; đặt bằng 10^3 chia cho (số phần tử bản đồ × batch size) mỗi lớp. Khi kết hợp với KD thì giảm β dần trong quá trình huấn luyện. (Số mũ của 10 trong text bị mất định dạng; suy từ ngữ cảnh "usually varies about 0.1", đừng coi là chính xác.)
- **Attention so với full activation:** F-ActT (FitNets-style với conv 1×1) chỉ cải thiện nhẹ (ví dụ 8.77 → 8.51) trong khi AT cho 7.93. Cả hai đều tăng tốc hội tụ (Fig. 7b), nhưng AT cho độ chính xác cuối tốt hơn nhiều. Tác giả nhận xét thú vị: attention map mang thông tin quan trọng hơn cho việc chuyển giao so với toàn bộ activation.
- **Chuẩn hoá L2** và giảm dần β cải thiện kết quả (theo tác giả).
- **Fine-grained (CUB):** AT tốt hơn KD (27 so 28.5 baseline... cả hai đều -1.5 theo bảng; còn trên Scenes AT -1.1 so KD -0.1). Tác giả đoán attention trung gian quan trọng với nhận dạng chi tiết. Đây là giả thuyết, không được kiểm chứng riêng.

6. Giới hạn

Do tác giả nêu:

- Gradient-based AT chỉ thử với NIN không batch norm trên CIFAR vì cần vi phân tự động phức tạp; chưa rõ batch norm nên hoạt động ra sao ở lần lan truyền ngược thứ hai.
- Trên ImageNet mới đặt loss ở 2 trong 4 nhóm và chưa tinh chỉnh siêu tham số (dự định cập nhật).
- Chưa thử detection hoặc weakly-supervised localization nơi thông tin không gian quan trọng hơn.
- CIFAR có độ phân giải sau downsampling rất nhỏ nên ít chỗ cho attention.

Nhận xét:

- Chỉ CNN phân loại ảnh; không có Transformer, không có chuỗi.
- Giả định "|kích hoạt| ≈ độ quan trọng" thô; với kích hoạt có dấu hoặc khối lượng "massive activations" (tồn tại ở LLM/ViT lớn) giả định này có thể vỡ (suy đoán minh họa).
- Hiệu ứng cải thiện nhỏ (khoảng 0.4 đến 1.1 điểm %), tương đương nhiễu giữa các lần chạy ở một số cặp (CIFAR dùng trung vị 5 lần, ImageNet chỉ một lần).
- Gradient-based AT đắt và chỉ có kết quả 1 lần chạy, nên so sánh trong Table 3 yếu thống kê.

7. Liên hệ với các bài khác

- **03 FitNets:** bài này trực tiếp so với FitNets-style hint (F-ActT) và thắng; bỏ regressor.
- **02 Hinton KD:** AT+KD cộng gộp; trên ImageNet KD không giúp.
- **05 KD survey:** xếp AT vào feature-based/attention-based KD.
- **12 Relational KD, 13 CRD:** chuyển từ khớp từng điểm sang khớp quan hệ/đối nghịch tương phản.
- **18 TinyBERT, 19 MiniLM:** "attention transfer" cho Transformer thực sự là khớp ma trận attention/quan hệ self-attention; cùng tinh thần nhưng khác định nghĩa. (Chú ý nhầm tên: AT ở đây không phải self-attention.)
- **23 AM-RADIO, 24 CLIP-KD, 26 LLaVA-KD:** feature distillation trên ViT, trong đó chuẩn hoá feature trước khi khớp là chi tiết quan trọng, như chuẩn hoá L2 ở đây.

Gợi ý học tập

So sánh khớp bản đồ kích hoạt đã chuẩn hóa với khớp tensor đặc trưng trực tiếp, đồng thời giữ cùng teacher, student và ngân sách. Cần kiểm tra kích thước không gian và vị trí các lớp được ghép trước khi diễn giải chênh lệch.

8. Tóm tắt công thức / khái niệm

Khái niệm	Công thức / ý nghĩa
Attention map	$F(A) = \sum_i \lvert A_i \rvert^p$ (chọn $p = 2$), $F : \mathbb{R}^{C\times H\times W} \to \mathbb{R}^{H\times W}$
Loss AT	$\mathcal{L} + \frac{\beta}{2} \sum_j \lVert Q_S^j / \lVert Q_S^j \rVert_2 - Q_T^j / \lVert Q_T^j \rVert_2 \rVert_p$
Gradient attention	$J = \partial \mathcal{L} / \partial x$; cần double backprop
Symmetry loss	ép $J(x)$ khớp lật của $J(\text{flip}\, x)$
Kết hợp KD	thêm CE nới nhiệt độ ($T = 4$, $\alpha = 0.9$ trên CIFAR)
ImageNet	ResNet-18 30.4/10.8 → 29.3/10.0 với AT (ResNet-34: 26.1/8.3)
CIFAR-10	WRN-16-1: 8.77 → 7.93 (AT), 7.41 (KD), 7.51 (AT+KD)

9. Kết luận

Attention transfer cho student khớp bản đồ kích hoạt không gian đã chuẩn hóa của teacher. Trong các thí nghiệm, lỗi CIFAR-10 của WRN-16-1 giảm từ 8.77% xuống 7.93%; ResNet-18 trên ImageNet cải thiện 1.1% top-1. Phương pháp cũng tốt hơn cách khớp toàn bộ tensor kiểu FitNets trong các so sánh được báo cáo.

Link gốc: https://arxiv.org/abs/1612.03928

Bài 05

Knowledge Distillation: A Survey (2021)

Tác giả: Jianping Gou, Baosheng Yu, Stephen J. Maybank, Dacheng Tao
Venue / arXiv: arXiv:2006.05525 (v7, 20/5/2021). Trong file text chỉ có tiêu đề "Noname manuscript"; theo thông tin khác bài được đăng ở International Journal of Computer Vision (2021), không kiểm chứng từ file.
Nguồn: https://arxiv.org/abs/2006.05525
Chủ đề: Khảo sát KD (chủ yếu giai đoạn 2014–2021, thiên về thị giác máy tính): phân loại theo loại tri thức (response / feature / relation), sơ đồ huấn luyện (offline / online / self), kiến trúc teacher-student, các thuật toán (adversarial, multi-teacher, cross-modal, graph, attention, data-free, quantized, lifelong, NAS), so sánh hiệu năng, ứng dụng.
Vị trí trong lộ trình: Đọc sau 01–04 (đã biết logit KD, hint, attention transfer). Đây là **bản đồ**: dùng để định vị từng bài sau (06 Born-again, 07 DML, 08 TA, 12 RKD, 13 CRD...). Survey này kết thúc ở 2021 nên **không có** LLM distillation (xem 27 LLM KD survey và 28–47 cho thời kỳ sau).

> Tài liệu học tập bằng tiếng Việt, là ghi chú diễn giải, không phải bản dịch paper. Đây là survey nên ghi chú tập trung vào phân loại và cách dùng như bản đồ, không đi sâu từng phương pháp con.

1. Bối cảnh

Mô hình học sâu lớn khó triển khai trên thiết bị hạn chế tài nguyên. Survey xếp KD vào họ nén và tăng tốc mô hình, cùng với bốn nhóm (theo Cheng et al., 2018, mà survey trích dẫn): (1) tỉa và chia sẻ tham số (gồm lượng tử hoá, nhị phân hoá, ma trận có cấu trúc, chia sẻ tham số), (2) phân rã hạng thấp (low-rank factorization), (3) bộ lọc tích chập nén chuyển giao, (4) KD. Survey chỉ tập trung nhóm thứ tư và không khảo sát đầy đủ nén mô hình nói chung.

Lịch sử theo survey: Buciluǎ et al. (2006) đề xuất model compression; Urner et al. (2011) dùng chuyển giao từ teacher sang student trên dữ liệu không nhãn cho học bán giám sát (và chứng minh học được theo PAC); Hinton et al. (2015) phổ biến hoá thành "knowledge distillation".

Hiểu biết lý thuyết còn ít (theo survey): Phuong & Lampert (2019) cho giới hạn tổng quát hoá với tốc độ hội tụ nhanh cho bộ phân loại tuyến tính sâu, và nêu ba yếu tố: hình học dữ liệu, thiên lệch tối ưu hoá của mục tiêu distillation, và tính đơn điệu mạnh của student; Cheng et al. (2020) lượng hoá khái niệm thị giác trích từ lớp giữa; Ji & Zhu (2020) phân tích mạng rộng theo giới hạn rủi ro, hiệu quả dữ liệu, teacher không hoàn hảo; Cho & Hariharan (2019) phân tích thực nghiệm hiệu quả của KD; Mirzadeh et al. (2020) cho thấy teacher lớn hơn không chắc là teacher tốt hơn do khoảng cách capacity; Tang et al. (2020) xem KD như label smoothing, đánh giá độ chính xác teacher, và prior hình học lớp đầu ra.

Ba thành phần của một hệ KD (theo survey): (i) tri thức, (ii) thuật toán distillation, (iii) kiến trúc teacher-student.

2. Ý tưởng chính và cách dùng survey như bản đồ

Survey không có một ý tưởng kỹ thuật mới. Đóng góp là **hệ thống hoá** KD theo các trục độc lập:

Trục	Câu hỏi	Giá trị (mục trong survey)
Tri thức lấy từ đâu	Teacher cho student cái gì?	response (§2.1), feature (§2.2), relation (§2.3)
Sơ đồ huấn luyện	Teacher có được cập nhật cùng student không?	offline (§3.1), online (§3.2), self (§3.3)

Trục	Câu hỏi	Giá trị (mục trong survey)
Kiến trúc	Student trông như thế nào so với teacher?	§4
Thuật toán	Cơ chế đặc biệt nào?	adversarial, multi-teacher, cross-modal, graph, attention, data-free, quantized, lifelong, NAS (§5.1–5.9)
Ứng dụng	Dùng ở đâu?	thị giác, NLP, giọng nói, khác (§7)

Cách dùng: khi gặp một bài KD mới, hãy tự hỏi lần lượt (1) tri thức thuộc loại nào, (2) sơ đồ huấn luyện nào, (3) student/teacher quan hệ ra sao, (4) có cơ chế phụ nào. Việc này giúp so sánh các bài mà không bị chìm trong tên riêng. Cẩn thận: survey không bàn về **token-level vs sequence-level** theo nghĩa LLM, hay **on-policy vs off-policy**; những trục đó có trong preface 00 và các bài 15, 27–34.

3. Phương pháp chi tiết: phân loại của survey

3.1 Response-based knowledge (§2.1)

Tri thức là phản hồi của lớp đầu ra teacher. Với logit z:

$$\mathcal{L}_{ResD}(z_t, z_s) = \mathcal{L}_R(z_t, z_s)$$

Dạng soft target:

$$\boxed{p(z_i, T) = \frac{\exp(z_i/T)}{\sum_j \exp(z_j/T)}}, \qquad \mathcal{L}_{ResD} = \mathcal{L}_R(p(z_t, T), p(z_s, T))$$

trong đó $\mathcal{L}_R$ thường là KL. Student loss luôn là CE giữa nhãn thật và $p(z_s, T{=}1)$. Survey nhận xét: hiệu quả của soft target tương tự **label smoothing / regularizer**; hạn chế: chỉ dùng lớp cuối nên bỏ qua giám sát trung gian và gắn với học có giám sát (vì là phân phối lớp). Mở rộng cho các tác vụ khác: object detection (logit + offset hộp), pose estimation (heatmap mỗi landmark).

3.2 Feature-based knowledge (§2.2)

$$\boxed{\mathcal{L}_{FeaD}(f_t, f_s) = \mathcal{L}_F(\Phi_t(f_t(x)), \Phi_s(f_s(x)))}$$

Φ_t, Φ_s là hàm biến đổi (khi hình dạng khác nhau), $\mathcal{L}_F$ là hàm tương đồng. Đại diện (Table 1 của survey): FitNet (feature, lớp hint, L2), NST (mẫu chọn lọc neuron, MMD), AT (bản đồ attention, nhóm nhiều lớp, L2), FT (paraphraser, L1), AB (ranh giới kích hoạt, trước ReLU, L2), Margin ReLU của Heo et al., AdaIN (thống kê feature), ALP-KD (chiếu lớp có attention), SemCKD (cross-layer, gán lớp teacher cho từng lớp student bằng attention allocation), v.v. Khó khăn survey nêu: chọn lớp hint/guided; khác biệt lớn kích thước hint và guided.

3.3 Relation-based knowledge (§2.3)

Quan hệ giữa các lớp hoặc giữa các mẫu.

$$\mathcal{L}_{RelD}(f_t, f_s) = \mathcal{L}_{R_1}(\Psi_t(\hat{f}_t, \check{f}_t), \Psi_s(\hat{f}_s, \check{f}_s))$$
$$\mathcal{L}_{RelD}(F_t, F_s) = \mathcal{L}_{R_2}(\psi_t(t_i, t_j), \psi_s(s_i, s_j))$$

Công thức đầu: quan hệ giữa cặp feature map (ví dụ **FSP matrix** của Yim et al. 2017, Gram matrix giữa hai lớp). Công thức sau: quan hệ giữa cặp mẫu (**instance relations**): RKD (khoảng cách và góc), SP (ma trận tương đồng, chuẩn Frobenius), CCKD (tương quan giữa các mẫu), PKT (phân phối xác suất tương đồng), IRG (đồ thị quan hệ mẫu), DarkRank, v.v. (Table 2). Survey còn liên hệ với **privileged information** (Vapnik & Izmailov; Lopez-Paz et al.).

3.4 Sơ đồ huấn luyện (§3)

- **Offline:** teacher huấn luyện trước, cố định; hai giai đoạn. Ưu: đơn giản. Nhược: teacher lớn tốn thời gian huấn luyện; **khoảng cách capacity** giữa teacher và student luôn tồn tại, student phụ thuộc nhiều vào teacher.
- **Online:** teacher và student cập nhật đồng thời, end-to-end, huấn luyện một pha, song song hiệu quả. Ví dụ: deep mutual learning (Zhang et al. 2018b; các mạng học lẫn nhau), co-distillation (Anil et al. 2018), kiến trúc nhiều nhánh dùng chung backbone (Zhu & Gong 2018, ONE), feature fusion cho teacher (Kim et al. 2019b), online adversarial (Chung et al. 2020). Hạn chế: khó có teacher dung lượng lớn trong thiết lập online.
- **Self-distillation:** cùng một mạng làm teacher và student, coi là trường hợp đặc biệt của online. Ví dụ: chưng cất từ phần sâu xuống phần nông của mạng (Zhang et al. 2019b), snapshot distillation (epoch sớm làm teacher cho epoch muộn), early-exit mimic, teacher-free KD (Yuan et al. 2020, dựa trên phân tích label smoothing), class-wise self-KD, và **Born-Again Networks** (Furlanello et al. 2018: student ở bước t làm teacher ở bước $t+1$; survey xếp vào nhóm này và cũng nhắc trong multi-teacher).
- Survey ví với việc học của con người: offline = thầy dạy trò; online = cùng học; self = tự học; ba loại có thể kết hợp (Sun et al. 2021).

3.5 Kiến trúc teacher-student (§4)

Student thường là: (1) bản đơn giản hoá của teacher (ít lớp, ít kênh); (2) bản lượng tử hoá (giữ cấu trúc); (3) mạng nhỏ với phép toán hiệu quả (depthwise separable: MobileNet, ShuffleNet); (4) mạng nhỏ có cấu trúc toàn cục tối ưu (NAS); (5) cùng mạng với teacher. Khoảng cách capacity làm giảm chuyển giao: **Teacher Assistant** (Mirzadeh et al. 2020) chèn mô hình trung gian; Gao et al. (2021) dùng học phần dư (assistant học residual error). Các hướng khác: nén cấu trúc nhiều lớp thành một lớp (Nowak & Corso), chuyển giao block-wise giữ receptive field (Wang et al. 2018a), RL tự bỏ lớp dư (N2N learning), tìm student tối ưu bằng NAS (oracle-based KD, architecture-aware KD). Survey nhận xét: phần lớn công trình tập trung loại tri thức/loss, **thiết kế kiến trúc teacher/student còn ít được nghiên cứu**.

3.6 Thuật toán distillation (§5)

Mục	Ý tưởng	Công thức/ghi chú của survey
5.1 Adversarial	GAN sinh dữ liệu cho KD; discriminator phân biệt đầu ra student/teacher; hoặc online	Ví dụ: $\mathcal{L}_{KD} = \mathcal{L}_G(F_t(G(z)), F_s(G(z)))$; $\mathcal{L}_{GAN\text{-}KD} = \mathcal{L}_{CE} + \alpha\mathcal{L}_{KL} + \beta\mathcal{L}_{GAN}$; KD còn dùng để nén GAN
5.2 Multi-teacher	Nhiều teacher: trung bình logit, hoặc teacher riêng cho response và feature, chọn ngẫu nhiên mỗi bước, mô phỏng nhiều teacher bằng nhiễu/stochastic block	Table 3 tóm tắt loại tri thức và sơ đồ
5.3 Cross-modal	Teacher ở modality này, student ở modality khác (RGB → depth, video → skeleton, text → vision...)	Cần dữ liệu cặp; khó khi lệch modality
5.4 Graph-based	Đồ thị làm vật mang tri thức hoặc điều khiển truyền tin	Cấu trúc đồ thị khó xây
5.5 Attention-based	Attention map (AT), neuron selectivity...	xem bài 04

Mục	Ý tưởng	Công thức/ghi chú của survey
5.6 Data-free	Không có dữ liệu gốc: sinh dữ liệu từ teacher (GAN, tái tạo từ kích hoạt, DeepInversion, zero-shot KD)	Thách thức: sinh dữ liệu đa dạng chất lượng cao. "Data distillation" (nhãn do teacher gán cho dữ liệu không nhãn) được nhắc là loại tương tự
5.7 Quantized	Teacher chính xác cao, student lượng tử hoá thấp bit	Polino et al. 2018; apprentice (Mishra & Marr)
5.8 Lifelong	KD để tránh quên thảm hoạ (LwF, global distillation...)	Dùng ngoài nén
5.9 NAS-based	Chọn kiến trúc student phù hợp teacher; KD để tăng hiệu quả NAS	TGSA, one-shot NAS...

4. Thí nghiệm và kết quả chính

Survey **không tự chạy thí nghiệm**: Table 5 (CIFAR-10) và Table 6 (CIFAR-100) chép lại độ chính xác từ các bài gốc. Các con số trong ngoặc là độ chính xác baseline huấn luyện riêng, mũi tên là mức cải thiện của student. Vì nguồn khác nhau (cấu hình khác nhau), **không nên so sánh chéo các hàng**; chỉ đọc "student baseline → student sau KD" trong cùng hàng. Dưới đây là vài hàng minh hoạ chép đúng như bảng (bảng trích xuất từ PDF có thể sai vài chỗ, đã ghi chú).

Một số hàng CIFAR-10 (Table 5)

Phương pháp	Tri thức	Teacher (baseline)	Student (baseline)	Sau KD
FSP (Yim 2017), offline	RelK	ResNet26 (91.91)	ResNet8 (87.91)	88.70 (+0.79)
FT (Kim 2018), offline	FeaK	ResNet56 (93.61)	ResNet20 (92.22)	93.15 (+0.93)
IRG (Liu 2019g), offline	RelK	ResNet20 (91.45)	ResNet20-x0.5 (88.36)	90.69 (+2.33)
SP (Tung & Mori 2019), offline	RelK	WRN-40-1 (93.49)	WRN-16-1 (91.26)	91.87 (+0.61)
DML (Zhang 2018b), online	ResK	ResNet32 (92.47)	ResNet32 (92.47)	92.68 / 92.80 (+0.33)
ONE (Zhu & Gong 2018), online	ResK	ResNet32+ONE	ResNet32 (93.07)	94.01 (+0.84)
Xu & Liu (2019), self	ResK, FeaK	không có	ResNet32 (92.78)	93.68 (+0.90)

Ghi chú về độ tin cậy: hàng "TOFD, teacher ResNet152, student ResNeXt50-4: 94.49 → 97.09 (+2.60)" được in như vậy trong bảng; con số này trông bất thường so với các hàng khác nên không dùng nó để rút ra kết luận.

Một số hàng CIFAR-100 (Table 6)

Phương pháp	Tri thức	Teacher (baseline)	Student (baseline)	Sau KD
FSP, offline	RelK	ResNet32 (64.06)	ResNet14 (58.65)	63.33 (+4.68)
FT, offline	FeaK	ResNet110 (73.09)	ResNet56 (71.96)	74.48 (+2.52)
RKD (Park 2019), offline	RelK, FeaK	ResNet50 (77.76)	VGG11 (71.26)	74.66 (+3.40)

Phương pháp	Tri thức	Teacher (baseline)	Student (baseline)	Sau KD
SSKD (Xu 2020a), offline	RelK, ResK	ResNet50 (79.10)	MobileNetV2 (65.79)	72.57 (+6.78)
SemCKD (Chen 2021), offline	FeaK	ResNet-32x4 (79.42)	VGG13 (74.82)	79.43 (+4.61)
DML (Zhang 2018b), online	ResK	MobileNet (73.65)	ResNet32 (68.99)	76.13 / 71.10 (+8.11)
ONE (Zhu & Gong 2018), online	ResK	ResNet32+ONE	ResNet32 (68.82)	73.39 (+4.57)
CS-KD (Yun 2020), self	ResK	không có	ResNet18 (75.29)	78.01 (+2.72)

(Với các hàng online như DML, "teacher" và "student" thực chất là hai mạng học lẫn nhau; hai số sau KD là độ chính xác của cả hai mạng, mũi tên tính cho student. Không nên dùng các hàng online để so sánh tuyệt đối với offline.)

Kết luận của survey từ hai bảng (trích ý): KD áp dụng được cho nhiều mô hình khác nhau; nén được các loại mạng; online KD qua học cộng tác cải thiện đáng kể; self-KD cũng cải thiện; offline thường chuyển tri thức feature còn online thường chuyển tri thức response; student nhẹ được lợi từ teacher dung lượng lớn.

5. Phân tích / ablation (những nhận định của survey)

Survey không có ablation riêng. Những nhận định đáng nhớ:

- **Response-based ≈ label smoothing/regularizer** (dẫn Kim & Kim; Müller et al. 2019; Ding et al. 2019).
- **Teacher lớn hơn không nhất thiết tốt hơn** do capacity gap (Mirzadeh et al. 2020; Gao et al. 2021). Mặt khác, từ lý thuyết sớm, mạng nông có thể học cùng biểu diễn với mạng sâu (Ba & Caruana, bài 01). Nên thiết kế student/teacher còn là bài toán mở.
- **Hint/guided layer sâu có thể over-regularize** (từ FitNets, bài 03).
- **Tri thức khác nhau bổ sung cho nhau** nhưng chưa có khung thống nhất để mô hình hoá kết hợp response, feature, relation.
- **Sequence-level KD** trong NLP (Kim & Rush 2016) và giọng nói (CTC): survey nói frame-level KD thường dùng response-based, còn sequence-level thường chuyển feature-based từ lớp hint (nhận xét ở §7.3). Gordon & Duh (2019) giải thích hiệu quả của sequence-level KD như tăng cường dữ liệu và regularization (§7.2); Zhou et al. (2019a) cho rằng hiệu năng của NAT mô hình KD phụ thuộc capacity và dữ liệu distill.
- **Hybrid compression** (KD + pruning/quantization) được nêu là hướng tương lai vì hầu hết kỹ thuật nén cần retrain/finetune; thứ tự áp dụng là câu hỏi mở.

6. Giới hạn

Do tác giả nêu (§8.1, 8.2):

- Chất lượng tri thức: chưa biết ảnh hưởng riêng và bổ sung của từng loại tri thức; chưa có khung hợp nhất.
- Kiến trúc teacher-student còn ít được nghiên cứu.
- Hiểu biết lý thuyết và đánh giá thực nghiệm chưa đủ; đo chất lượng tri thức hay chất lượng cặp teacher-student là rất khó.
- Chưa nhiều nghiên cứu kết hợp KD với nén khác; thứ tự áp dụng là câu hỏi mở.

Nhận xét:

- Phạm vi thiên về CNN và phân loại ảnh; NLP chỉ có NMT và BERT nén (BERT-PKD, TinyBERT, DistilBERT); không có LLM sinh văn bản lớn, không có on-policy/RL, không có ước lượng top-k hay vấn đề tokenizer.
- Hai bảng hiệu năng chép từ nhiều bài với cấu hình khác nhau (baseline khác, augmentation khác), nên chỉ dùng để thấy xu hướng; không phải benchmark có kiểm soát. Khi trích xuất từ PDF có một số số liệu in lệch cột.
- Nhiều trích dẫn và mô tả ngắn gọn, ít phân tích cơ chế; không nêu phương trình gradient hay phân tích T^2 (xem bài 02 và preface 00).
- Nhiều nghiên cứu sau 2021 (KD cho LLM, reasoning distillation, scaling laws) nằm ngoài phạm vi; cần bài 27 và các bài 28–47.

7. Liên hệ với các bài khác

Bản đồ từ survey sang lộ trình của thư mục này:

Mục trong survey	Bài trong lộ trình
§2.1 Response-based	01 Ba & Caruana, 02 Hinton; nâng cao: 14 Decoupled KD
§2.2 Feature-based	03 FitNets, 04 Attention transfer; Transformer: 18 TinyBERT, 19 MiniLM, 17 Patient KD
§2.3 Relation-based	12 Relational KD, 13 CRD (đối nghịch/tương phản, liên quan §5.3 contrastive của Tian et al.)
§3.2 Online	07 Deep mutual learning
§3.3 Self-distillation	06 Born-again networks; 22 DINO (self-distillation dạng khác)
§4 Capacity gap, TA	08 Teacher assistant, 09 Efficacy of KD, 10 Does KD really work, 11 Patient and consistent teacher, 41 Capacity gap law
§5.6 Data-free / data distillation	15 Sequence-level KD, 35 Self-Instruct, 36 Distilling step-by-step, 37 Orca, 38 DeepSeek-R1 distillation
§5.3 Cross-modal	24 CLIP-KD, 25 LLaVA-MoD, 26 LLaVA-KD
§7.2 NLP	15, 16 DistilBERT, 17, 18, 19
§7.3 Speech	46 Distil-Whisper
§8 Hybrid compression	kết hợp cắt tỉa và chưng cất
Sau 2021	27 LLM KD survey, 28–34 (MiniLLM, GKD, DistiLLM, Rethinking KL, Speculative KD, TAID, DistiLLM-2), 40 Gemma 2, 42 Distillation scaling laws, 44 Qwen3, 45 Sparse logit sampling, 47 ShortOPD

8. Tóm tắt công thức / khái niệm

Khái niệm	Ý nghĩa
Response-based	khớp output (logit/soft target) của teacher; $p(z_i, T) = \exp(z_i/T)/\sum_j \exp(z_j/T)$
Feature-based	khớp feature trung gian, $\mathcal{L}_F(\Phi_t(f_t), \Phi_s(f_s))$
Relation-based	khớp quan hệ giữa lớp (FSP) hoặc giữa mẫu (RKD, SP, CCKD...)
Offline / Online / Self	teacher cố định / cùng huấn luyện / cùng một mạng
Capacity gap	student quá nhỏ so với teacher thì chuyển giao kém; TA là một giải pháp
Data-free KD	sinh dữ liệu từ teacher khi không có dữ liệu gốc
Quantized KD	student lượng tử hoá, teacher chính xác cao
Hybrid compression	kết hợp KD và pruning/quantization; thứ tự là câu hỏi mở

Khái niệm	Ý nghĩa
Các bảng 5–6	chép từ bài gốc, chỉ đọc theo từng hàng, không so chéo

9. Kết luận

Survey phân loại KD theo loại tri thức, sơ đồ huấn luyện, kiến trúc và thuật toán. Những câu hỏi mở gồm cách kết hợp các dạng tri thức, chọn cặp teacher–student và phối hợp KD với phương pháp nén khác. Hai bảng hiệu năng nên được đọc theo xu hướng; survey không bao quát các công trình LLM ở các bài 27–47.

Link gốc: https://arxiv.org/abs/2006.05525

Bài 06

Born Again Neural Networks (2018)

Tác giả: Tommaso Furlanello, Zachary C. Lipton, Michael Tschannen, Laurent Itti, Anima Anandkumar.
Venue / arXiv: arXiv:1805.04770 (v2, 29/06/2018); Proceedings of ICML 2018 (PMLR 80), Stockholm.
Nguồn: https://arxiv.org/abs/1805.04770
Chủ đề: Self-distillation (student cùng kiến trúc với teacher), phân rã gradient của KD thành "dark knowledge" và "importance weighting", chuỗi nhiều thế hệ (BAN), ensemble các thế hệ.
Vị trí trong lộ trình: Đọc trước: 02 (Hinton KD, định nghĩa soft target và dark knowledge), 01 (Ba & Caruana, logit matching). Bài này thêm: KD không cần nén, teacher và student có thể cùng cỡ, thậm chí student mạnh hơn teacher; đồng thời đặt câu hỏi "cái gì trong teacher output thực sự có ích". Đọc tiếp: 07 (Deep mutual learning, bỏ luôn teacher cố định), 08 (Teacher assistant, trường hợp ngược lại: student quá nhỏ so với teacher), 10 (Does KD really work).

> Đây là ghi chú học tập bằng tiếng Việt, diễn giải và tóm lược, không phải bản dịch paper.

1. Bối cảnh

Knowledge distillation (KD) = huấn luyện một mô hình *student* bắt chước phân phối đầu ra (softmax) của một mô hình *teacher* đã train xong, thay vì (hoặc cùng với) học từ nhãn cứng one-hot. Trước bài này, hầu như mọi công trình (Bucilua 2006, Ba & Caruana 2014, Hinton 2015, FitNets, Attention Transfer) đều coi KD là **công cụ nén mô hình**: teacher lớn, student nhỏ, student không bao giờ qua mặt teacher, chỉ "gần hơn" so với train thẳng từ nhãn.

Hinton giải thích thành công của KD bằng **dark knowledge**: xác suất của các lớp sai (ví dụ "xe tải" có xác suất cao hơn "con mèo" khi ảnh là "ô tô") chứa thông tin về độ tương tự giữa các lớp. Bài này đặt hai câu hỏi mới:

1. Nếu bỏ yếu tố nén (student cùng kiến trúc, cùng số tham số với teacher), KD còn có ích không?
2. Có đúng là "dark knowledge" là nguồn lợi ích chính không, hay còn cơ chế khác (ví dụ KD chỉ là một dạng đánh trọng số mẫu)?

Tiền đề cần nhớ: **cross-entropy** giữa phân phối mục tiêu p và dự đoán q là $-\sum_i p_i \log q_i$; đạo hàm theo logit z_i của softmax là $q_i - p_i$ (rất đẹp và dùng ở dưới).

2. Ý tưởng chính

Một teacher được huấn luyện từ nhãn trước. Student có cùng kiến trúc nhưng khởi tạo ngẫu nhiên khác, sau đó học phân phối đầu ra của teacher, có thể kết hợp nhãn thật. Student thu được gọi là Born-Again Network (BAN) và thường vượt teacher trong các thí nghiệm. Quy trình có thể lặp qua nhiều thế hệ và lấy trung bình dự đoán thành ensemble BANE. Hai phép phân tích CWTM và DKPP khảo sát đóng góp của độ tin cậy ở lớp đúng và cấu trúc xác suất của các lớp còn lại.

3. Phương pháp chi tiết

3.1 Quy trình BAN

ERM chuẩn: $\theta_1^* = \arg\min_{\theta_1} L(y, f(x, \theta_1))$. BAN thế hệ 2 học từ output của thế hệ 1:

$$\theta_2^* = \arg\min_{\theta_2} L(f(x, \theta_1^*),\ f(x, \theta_2))$$

với L là cross-entropy giữa hai output. Thế hệ k học từ thế hệ $k-1$:

$$\theta_k^* = \arg\min_{\theta_k} L(f(x, \theta_{k-1}^*),\ f(x, \theta_k))$$

Ensemble BANE sau k thế hệ:

$$\boxed{\hat{f}_k(x) = \frac{1}{k}\sum_{i=1}^{k} f(x, \theta_i)}$$

Chi tiết quan trọng (paper nêu): lịch train của student giữ nguyên lịch của teacher; **không dùng temperature** (paper nói họ lần đầu cho thấy dark knowledge trong self-distillation hiệu quả "ngay cả khi không làm mềm logit"); hai biến thể BAN: "BAN" chỉ dùng loss theo teacher, "BAN+L" dùng cả loss nhãn và loss teacher. Các biến thể ép student giống teacher hơn (chia sẻ lớp đầu/cuối, thêm loss L2 giữa activation) được thử nhưng "systematically perform slightly worse" so với KD cross-entropy đơn giản.

Pseudo-code:

```
T1 = train(arch, data, labels, seed=1)            # teacher thường
for k in 2..K:
    Tk = init(arch, seed=k)                        # cùng kiến trúc, seed khác
    for batch (x, y) in data:
        p = softmax(T_{k-1}(x))                    # đóng băng, không temperature
        q = softmax(Tk(x))
        loss = CE(p, q)  [+ CE(y, q) nếu BAN+L]
        update Tk
ensemble(x) = mean_k softmax(Tk(x))                # BANE
```

3.2 Gradient của KD: "dark knowledge" hay "importance weighting"?

Với logit z_i của student, $q_i = \text{softmax}(z)_i$ và teacher p_i, gradient cross-entropy theo logit thứ i:

$$\frac{\partial L_i}{\partial z_i} = q_i - p_i$$

Khi mục tiêu là nhãn one-hot y thì gradient là $q_* - y_*$ (chỉ lớp đúng $*$ là khác). Với KD, trên một minibatch b mẫu (kể cả $1/b$), gradient tách thành hai tổng:

$$\sum_{s=1}^{b}\sum_{i=1}^{n}\frac{\partial L_{i,s}}{\partial z_{i,s}} = \underbrace{\sum_{s=1}^{b}(q_{*,s} - p_{*,s})}_{\text{lớp đúng}} + \underbrace{\sum_{s=1}^{b}\sum_{i=1}^{n-1}(q_{i,s} - p_{i,s})}_{\text{lớp sai: dark knowledge}}$$

Số hạng đầu có thể viết lại thành $\frac{1}{b}\sum_s (q_{*,s} - p_{*,s}\, y_{*,s})$ (paper viết dưới dạng này): tức gradient của nhãn thật **nhân với** $p_{*,s}$, độ tự tin của teacher trên lớp đúng. Khi teacher đúng và rất tự tin ($p_* \approx 1$) thì nó trở về gradient nhãn thật; mẫu mà teacher không chắc thì bị giảm trọng số. Đó là **importance weighting** (đánh trọng số mẫu): mẫu "dễ" (teacher chắc) được nhấn mạnh, mẫu "khó/nhiễu" bị hạ.

3.3 Hai thí nghiệm tách thành phần

- **CWTM (Confidence-Weighted by Teacher Max):** loại bỏ toàn bộ thông tin trên lớp sai. Train student bằng cross-entropy với nhãn thật, nhưng nhân loss mỗi mẫu với trọng số tỉ lệ max $p_{\cdot,s}$ (xác suất lớn nhất của teacher, kể cả khi teacher sai), chuẩn hóa theo batch:

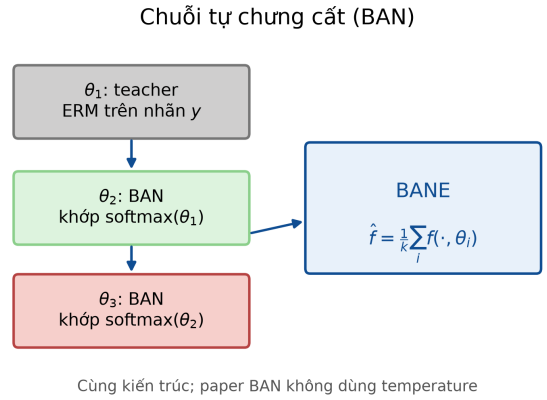
$$\sum_{s=1}^{b} \frac{\max p_{\cdot,s}}{\sum_{u=1}^{b} \max p_{\cdot,u}} (q_{*,s} - y_{*,s})$$

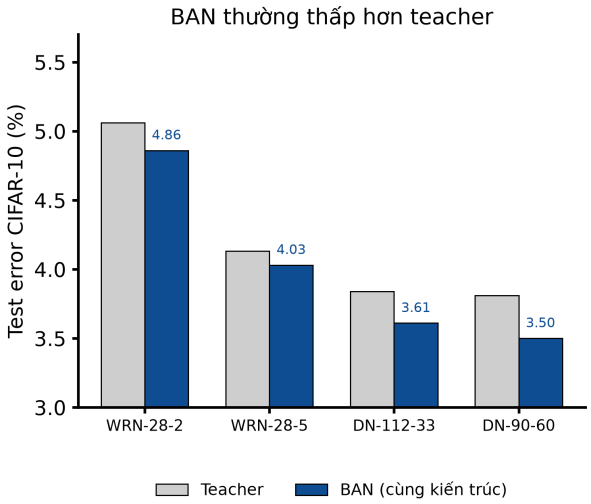
- **DKPP (Dark Knowledge with Permuted Predictions):** giữ nguyên giá trị argmax của teacher nhưng **xáo ngẫu nhiên** các xác suất của các lớp còn lại (ϕ là phép hoán vị). Việc này phá quan hệ "lớp nào giống lớp nào" (ma trận hiệp phương sai output), nhưng giữ nguyên các moment bậc cao bất biến với hoán vị (độ "nhọn", entropy...).

Logic: nếu lợi ích của KD chỉ đến từ việc lớp sai tương đồng với lớp đúng thì DKPP phải mất hết tác dụng; nếu chỉ do weighting thì CWTM phải đạt gần bằng BAN.

3.4 Đổi kiến trúc student

Ngoài cùng kiến trúc, tác giả: (a) thay đổi depth/growth/compression rate của DenseNet student cho cùng kích thước hidden state tại mỗi chỗ giảm độ phân giải; (b) "DenseNet to ResNet": student là Wide-ResNet hay bottleneck Pre-ResNet khớp shape từng block của DenseNet-90-60 (dùng chung lớp đầu conv1 và lớp cuối fc, bị đóng băng); (c) "ResNet to DenseNet": teacher là Wide-ResNet, student DenseNet-90-60.





Self-distillation BAN: nhiều thế hệ cùng kiến trúc học từ softmax thế hệ trước, ensemble BANE; cột lỗi CIFAR-10 lấy từ bảng trong note (minh hoạ, tự tính)

4. Thí nghiệm và kết quả chính

Dữ liệu và mô hình: CIFAR-10, CIFAR-100 (DenseNet các cỡ, Wide-ResNet-28-k), Penn Tree Bank (LSTM 1500 đơn vị và CNN-LSTM của Kim et al.). Preprocessing và hyperparameter theo Wide-ResNet (Zagoruyko & Komodakis), ngoại trừ chuẩn hóa mean-std.

CIFAR-10 test error (%), Bảng 1 (Teacher / BAN):

Mạng	Tham số	Teacher	BAN
Wide-ResNet-28-1	0.38 M	6.69	6.64
Wide-ResNet-28-2	1.48 M	5.06	4.86
Wide-ResNet-28-5	9.16 M	4.13	4.03
Wide-ResNet-28-10	36 M	3.77	3.86
DenseNet-112-33	6.3 M	3.84	3.61
DenseNet-90-60	16.1 M	3.81	3.5

Mạng	Tham số	Teacher	BAN
DenseNet-80-80	22.4 M	3.48	3.49
DenseNet-80-120	50.4 M	3.37	3.54

Lưu ý: ở CIFAR-10 BAN không thắng ở mọi cấu hình (Wide-ResNet-28-10, DenseNet-80-120 lại tệ hơn; DenseNet-80-80 gần như hòa). Số 3.5% trong abstract tương ứng DenseNet-90-60.

CIFAR-100 test error (%), Bảng 2 (trích; BAN chỉ dùng teacher loss, BAN+L thêm nhãn):

Mạng	Teacher	BAN	BAN+L	CWTM	DKPP	BAN-2	BAN-3	Ens*2	Ens*3
DenseNet-112-33	18.25	16.95	17.68	17.84	17.84	17.22	16.59	15.77	15.68
DenseNet-90-60	17.69	16.69	16.93	17.42	17.43	16.44	16.72	15.39	15.74
DenseNet-80-80	17.16	16.36	16.5	17.16	16.84	16.30	15.5	15.46	15.14
DenseNet-80-120	16.87	16.00	16.41	17.12	16.34	16.13	16.13	15.13	14.9

(Cột BAN-1 bị bỏ khỏi bảng này cho gọn; BAN-1 là một BAN khác seed. Với DenseNet-80-120 không có BAN-3 nên Ens*3 bao gồm teacher.)

Các con số nổi bật được paper nêu: BAN-3-DenseNet-80-80 (22M tham số) đạt **15.5%** trên CIFAR-100, mô hình đơn tốt nhất mà họ biết khi không dùng shake-shake; BAN-ensemble lớn nhất (BAN-3-DenseNet-BC-80-120, 150M tham số) đạt **14.9%**; DenseNet-112-33 BAN (6.5M tham số theo văn bản) 16.95% xấp xỉ teacher DenseNet-80-120 (16.87%) đắt hơn ~8 lần.

Student kiến trúc khác, CIFAR-100:

Bảng 3 (Teacher / BAN-WRN / Dense-90-60 student từ WRN teacher): WRN-28-1: 30.05 / 29.43 / 24.93; WRN-28-2: 25.32 / 24.38 / 18.49; WRN-28-5: 20.88 / 20.93 / 17.52; WRN-28-10: 19.08 / 18.25 / 16.79. Tức là một teacher **yếu hơn** (WRN-28-1, 30.05%) vẫn đủ để DenseNet student đạt 24.93%, tốt hơn nhiều so với WRN-28-1 train thẳng; chỉ riêng WRN-28-5 là BAN-WRN không cải thiện.

Bảng 4 (student DenseNet biến đổi từ teacher DenseNet-90-60, lỗi 17.69): 0.5×Depth 16.95 (21.2M), 2×Depth 16.43 (13.7M), 3×Depth 16.64 (12.9M), 4×Depth 16.64, 0.5×Compression 19.83 (5.1M), 0.75×Compression 17.3 (10.1M), 1.5×Compression 18.89 (80.5M). (Cột tham số trong bảng trích xuất từ PDF hơi lộn xộn; nên kiểm tra lại bản gốc nếu cần số chính xác.)

Bảng 5 (DenseNet teacher sang ResNet student, baseline / BAN): Pre-ResNet-14-0.5: 20.28/18.8; Pre-ResNet-14-1: 18.84/17.39; WRN-1-1: 20.4/19.12; Match-WRN-2-1: 18.83/17.42; WRN-4-0.5: 19.63/17.13; WRN-4-1: 18.77/17.18. Pre-activation ResNet-1001 (10.2M) baseline 22.71. Student ResNet đạt 17.13–17.39% so với teacher DenseNet-90-60 17.69%.

Penn Tree Bank, perplexity (Bảng 6):

Mạng	Tham số	Teacher Val	BAN+L Val	Teacher Test	BAN+L Test
ConvLSTM	19M	83.69	80.27	80.05	76.97
LSTM	52M	75.11	71.19	71.87	68.56

Ở LSTM, chỉ dùng teacher output thuần không hiệu quả; cần BAN+L (cả nhãn). Giải thích tác giả đưa ra (phỏng đoán): teacher CIFAR đạt 100% train accuracy, còn PTB train perplexity còn xa mức tối ưu.

5. Phân tích / ablation

- **Bỏ dark knowledge vẫn có lợi một phần (CWTM):** trên CIFAR-100, CWTM cải thiện nhẹ hơn teacher ở 3/4 mô hình (ví dụ 112-33: 18.25 -> 17.84; 80-80 hòa 17.16; 80-120 lại tệ hơn: 17.12 so với 16.87). Giải thích paper: thông tin từ teacher có thể dùng để cân lại tập train, giảm trọng số mẫu mà phân phối teacher không tập trung ở argmax.
- **Phá cấu trúc lớp sai vẫn có lợi (DKPP):** cải thiện "comparable but systematic". Paper kết luận: dark knowledge không chỉ là thông tin của từng lớp sai cụ thể; các moment bậc cao bất biến với hoán vị cũng đóng góp.
- **Cả hai đều kém BAN đầy đủ** trên hầu hết cấu hình, nên hai thành phần bổ sung nhau (đây là cách đọc minh họa từ Bảng 2; paper nói "KD does not simply contribute information on each specific non-correct output").
- **Nhãn thêm có đáng không?** Với ảnh, BAN thuần (chỉ teacher loss) thường tốt hơn BAN+L (Bảng 2), ngược với LSTM.
- **Nhiều thế hệ:** cải thiện "inconsistent but positive", bão hòa sau vài thế hệ; ensemble các thế hệ cho lợi ích lớn hơn.
- **Robust với đổi kiến trúc:** student DenseNet nửa số lớp vẫn thắng teacher (16.95 so với 17.69); giảm compression rate xuống 0.5 là bước làm tệ nhất (19.83). Student ResNet khớp shape thắng teacher DenseNet.

6. Giới hạn

Do tác giả nêu hoặc hàm ý:

- Cải thiện không đều: CIFAR-10 có vài cấu hình không cải thiện; thế hệ BAN-3 đôi khi tệ hơn BAN-2 (DenseNet-90-60: 16.44 sang 16.72); LSTM cần nhãn.
- Phân rã gradient chỉ giải thích một phần, bài báo không đưa ra lý thuyết chặt chẽ.

Nhận xét:

- Chỉ thử trên CIFAR và PTB, mô hình nhỏ; chưa có ImageNet hay dữ liệu lớn. Mức cải thiện thường nhỏ (0.1 đến 1 điểm) và chỉ báo cáo một lần chạy, không có độ lệch chuẩn; phần cải thiện vài phần trăm có thể nằm trong nhiễu seed ở CIFAR-10.
- Số "tốt nhất" (15.5%, 14.9%) lấy theo validation/test của các lần chọn thế hệ, có nguy cơ chọn lọc; ensemble 150M tham số thì không còn là "cùng cỡ" nữa.
- Chi phí train gấp K lần với K thế hệ, trong khi bài không so với ensemble các model train độc lập có cùng ngân sách đầy đủ (chỉ so với một số coupled ensemble).

7. Liên hệ với các bài khác

- **02 Hinton KD:** BAN dùng đúng loss đó nhưng bỏ nén và bỏ temperature; phần phân rã gradient bổ sung cách hiểu "soft target = label smoothing/importance weight có điều kiện" đối chọi với giải thích dark knowledge thuần túy.
- **07 Deep mutual learning:** cùng ý "student không cần nhỏ hơn", nhưng DML train các mạng đồng thời, không có teacher cố định.
- **08 Teacher assistant, 09 Efficacy of KD, 41 Capacity gap law:** BAN là cực "không có khoảng cách dung lượng"; các bài này nghiên cứu hậu quả khi khoảng cách lớn.
- **10 Does KD really work:** đặt lại câu hỏi "student có thực sự khớp teacher không?"; BAN chỉ báo cáo lỗi top-1 chứ không đo agreement/độ khớp.

- **22 DINO, 21 DeiT:** self-distillation theo kiểu khác (EMA teacher), cùng tinh thần teacher và student cùng kiến trúc.

8. Tóm tắt công thức / khái niệm

Khái niệm	Ý nghĩa
BAN	Student cùng kiến trúc, seed khác, học từ output teacher; thường thắng teacher
BAN+L	BAN dùng thêm loss nhãn thật
BANE	Ensemble trung bình dự đoán các thế hệ BAN
$\partial L/\partial z_i = q_i - p_i$	Gradient CE theo logit; nền tảng của phân rã
Tách gradient	Số hạng lớp đúng (importance weighting theo p_*) + số hạng lớp sai (dark knowledge)
CWTM	Chỉ giữ trọng số max p của teacher, bỏ thông tin lớp sai
DKPP	Hoán vị xác suất các lớp không phải argmax để phá cấu trúc tương đồng
Dark knowledge	Thông tin trong xác suất của các lớp sai
Self-distillation	Teacher và student cùng kiến trúc

9. Kết luận

Trong CIFAR-100 và PTB, student cùng cỡ học từ teacher thường vượt teacher; chuỗi nhiều thế hệ và ensemble đem lại thêm lợi ích. Các phân tích CWTM và DKPP gợi ý cả độ tin cậy theo mẫu lẫn cấu trúc phân phối đều có thể góp phần vào kết quả.

Nguồn gốc: https://arxiv.org/abs/1805.04770

Bài 07

Deep Mutual Learning (2017)

Tác giả: Ying Zhang, Tao Xiang, Timothy M. Hospedales, Huchuan Lu.
Venue / arXiv: arXiv:1706.00384 (v1, 01/06/2017). Bản text đã đọc không ghi tên hội nghị; tiêu đề ghi năm 2017 theo ngày arXiv, còn tên file lộ trình dùng 2018 (năm xuất bản chính thức, thông tin ngoài paper).
Nguồn: https://arxiv.org/abs/1706.00384
Chủ đề: Học tương hỗ giữa nhiều mạng học viên (student cohort), không cần teacher huấn luyện trước; mimicry loss bằng KL divergence giữa các peer; giải thích bằng entropy hậu nghiệm cao và cực tiểu phẳng.
Vị trí trong lộ trình: Đọc trước: 02 (Hinton KD), 06 (Born-again networks: student cùng cỡ vẫn học được từ teacher cùng cỡ). Bài này thêm: bỏ luôn khái niệm teacher huấn luyện trước; các mạng cùng train, cùng dạy lẫn nhau; và cho thấy dạy lẫn nhau còn thắng KD từ teacher mạnh cố định trong các thí nghiệm của họ. Đọc tiếp: 08 (Teacher assistant), 10 (Does KD really work).

> Đây là ghi chú học tập bằng tiếng Việt, diễn giải và tóm lược, không phải bản dịch paper.

1. Bối cảnh

Distillation chuẩn là truyền một chiều: teacher lớn đã train xong, student nhỏ chưa train, student học theo xác suất lớp (class posterior) của teacher. Lập luận động cơ trong bài báo: mạng nhỏ thường có đủ dung lượng biểu diễn hàm cần học, khó khăn nằm ở **tối ưu hóa** (khó tìm được tham số tốt), không phải ở kích thước. Teacher giúp vì tín hiệu mimic dễ tối ưu hơn tín hiệu nhãn cứng.

Khái niệm cần có:

- **Posterior / class probability:** vector softmax $p^m(x)$ trên M lớp.
- **KL divergence** $D_{KL}(p\|q) = \sum_m p^m \log(p^m/q^m)$: đo độ lệch giữa hai phân phối, không đối xứng; phạt nặng khi q cho xác suất gần 0 ở chỗ p có khối lượng.
- **Posterior entropy cao / cực tiểu phẳng (wide minima):** nhiều nghiên cứu cùng thời (Entropy-SGD, Pereyra et al.) cho rằng nghiệm nằm trong "thung lũng rộng" và có output ít quá tự tin thì tổng quát hóa tốt hơn.

Câu hỏi của bài: nếu cả nhóm đều chưa được huấn luyện, thì tri thức "bổ sung" đến từ đâu? Có bị "người mù dắt người mù" (groupthink) không?

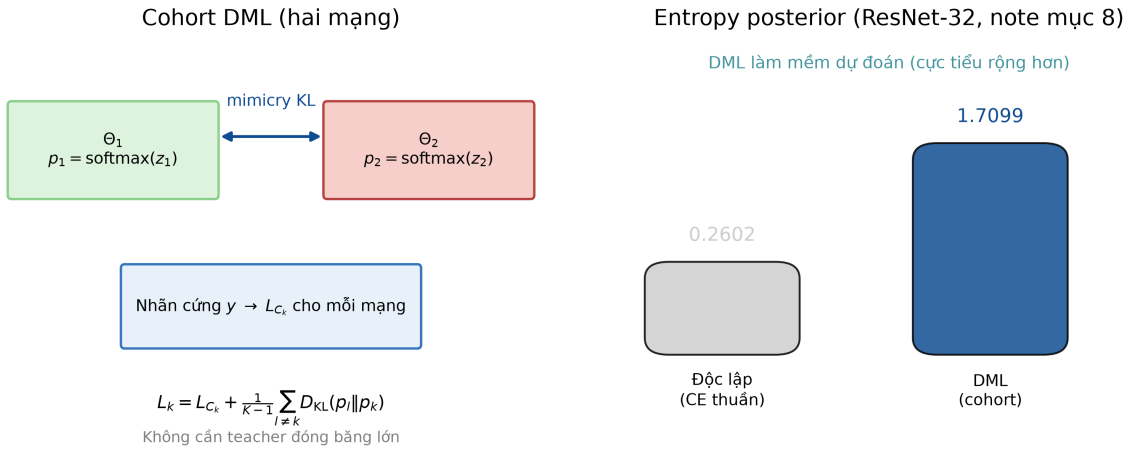
2. Ý tưởng chính

Nhiều mạng được khởi tạo độc lập và huấn luyện đồng thời. Mỗi mạng vừa học nhãn thật bằng cross-entropy, vừa dùng KL để khớp phân phối dự đoán của các mạng còn lại. Khởi tạo khác nhau tạo ra những dự đoán khác nhau để các mạng trao đổi trong quá trình học. Trên các bài toán được khảo sát, mỗi mạng tốt hơn khi học độc lập và cũng vượt phương án chưng cất từ teacher cố định.

3. Phương pháp chi tiết

3.1 Hai mạng

Với N mẫu, M lớp, xác suất lớp m của mạng Θ_1 cho mẫu x_i: $p_1^m(x_i) = \dfrac{\exp(z_1^m)}{\sum_{m'} \exp(z_1^{m'})}$ (không có temperature trong công thức của bài báo). Loss giám sát:



DML: cohort mạng ngang hàng, mỗi mạng CE nhãn + KL tới peer; entropy posterior ResNet-32 cao hơn CE thuần (số note mục 8) (minh hoạ, tự tính)

$$L_{C_1} = -\sum_{i=1}^{N}\sum_{m=1}^{M} I(y_i, m)\log p_1^m(x_i)$$

Mimicry loss từ peer Θ_2 và loss tổng:

$$\boxed{L_{\Theta_1} = L_{C_1} + D_{KL}(p_2 \,\|\, p_1)}, \qquad L_{\Theta_2} = L_{C_2} + D_{KL}(p_1 \,\|\, p_2)$$

Chú ý hướng KL trong bài báo: $D_{KL}(p_2\|p_1) = \sum p_2 \log(p_2/p_1)$, trong đó peer p_2 đóng vai "mục tiêu". Khi peer cho xác suất khác 0 mà mạng này cho 0 thì bị phạt rất nặng (đây là lý do paper dùng để giải thích cơ chế, mục 5).

3.2 Tối ưu hóa (Algorithm 1)

Ở **mỗi minibatch**, cập nhật lần lượt: tính p_1, p_2, cập nhật Θ_1 theo L_{Θ_1}; rồi tính lại p_1, p_2 (với Θ_1 mới), cập nhật Θ_2 theo L_{Θ_2}. Lặp đến hội tụ.

```
init Theta_1..Theta_K khác nhau (random init khác)
repeat:
    sample minibatch x
    for k in 1..K:
        p = [softmax(f_{Theta_j}(x)) for j in 1..K]     # tính lại sau mỗi lần update
        L_k = CE(y, p_k) + 1/(K-1) * sum_{l != k} KL(p_l || p_k)
        Theta_k <- Theta_k - lr * grad(L_k)             # p_l coi là hằng số khi tính
        ↪  grad của k
until converge
```

(Paper viết gradient cho mạng đang cập nhật; việc coi p_l là hằng số khi đạo hàm theo Θ_k là cách hiểu hợp lý minh họa, vì mỗi mạng chỉ cập nhật tham số của chính nó.)

3.3 Nhiều mạng

$$\boxed{L_{\Theta_k} = L_{C_k} + \frac{1}{K-1}\sum_{l=1, l\neq k}^{K} D_{KL}(p_l \,\|\, p_k)}$$

Hệ số $\frac{1}{K-1}$ giữ cho tín hiệu giám sát nhãn thật vẫn là thành phần chính. Khi $K = 2$ trở về công thức ở 3.1. Có thể phân tán: mỗi mạng một thiết bị, chỉ truyền các vector xác suất nhỏ.

Phương án thay thế **DML_e**: coi trung bình các peer làm một teacher:

$$L_{\Theta_k} = L_{C_k} + D_{KL}(p_{avg} \,\|\, p_k), \qquad p_{avg} = \frac{1}{K-1}\sum_{l \neq k} p_l$$

Paper thấy DML_e **tệ hơn** DML (mục 5).

4. Thí nghiệm và kết quả chính

Dữ liệu: CIFAR-100 (50.000 train, 10.000 test, 100 lớp, top-1 accuracy). Market-1501 (person re-identification, 32.668 ảnh, 1.501 identity; 751 identity để train, 750 để test; train bằng phân loại 751 lớp, test bằng nearest-neighbour trên feature lớp pooling cuối; đo mAP và CMC Rank-k).

Mạng và số tham số trên CIFAR-100: ResNet-32 0.5M, MobileNet 3.3M, InceptionV1 7.8M, WRN-28-10 36.5M.

Thiết lập: CIFAR-100 theo Zagoruyko & Komodakis: SGD Nesterov, lr 0.1, momentum 0.9, batch 64, lr nhân 0.1 mỗi 60 epoch, 200 epoch, augmentation lật ngang và random crop. Market-1501: Adam, lr 0.0002, $\beta_1 = 0.5$, $\beta_2 = 0.999$, batch 16, 100.000 iteration, có và không pretrain ImageNet. TensorFlow, một GTX 1080.

CIFAR-100 top-1 (%), cohort 2 mạng (Bảng 2):

Net 1	Net 2	Độc lập 1 / 2	DML 1 / 2	Chênh 1 / 2
ResNet-32	ResNet-32	68.99 / 68.99	71.19 / 70.75	+1.20 / +1.76
WRN-28-10	ResNet-32	78.69 / 68.99	78.96 / 70.73	+0.27 / +1.74
MobileNet	ResNet-32	73.65 / 68.99	76.13 / 71.10	+2.48 / +2.11
MobileNet	MobileNet	73.65 / 73.65	76.21 / 76.10	+2.56 / +2.45
WRN-28-10	MobileNet	78.69 / 73.65	80.28 / 77.39	+1.59 / +3.74
WRN-28-10	WRN-28-10	78.69 / 78.69	80.28 / 80.08	+1.59 / +1.39

Mọi tổ hợp đều cải thiện; mạng nhỏ hưởng lợi nhiều hơn; kể cả mạng lớn WRN-28-10 học cùng mạng nhỏ vẫn tốt lên.

Market-1501 (Bảng 3), mỗi MobileNet trong cohort 2 mạng, báo cáo trung bình hai mạng:

Phương pháp	Pretrain ImageNet	Single-query mAP / Rank-1	Multi-query mAP / Rank-1
MobileNet	không	46.07 / 72.18	54.31 / 79.81
MobileNet + DML	không	52.15 / 76.90	60.97 / 83.48
MobileNet	có	60.68 / 83.94	68.25 / 87.89
MobileNet + DML	có	68.83 / 87.73	77.14 / 91.66

Paper cũng liệt kê các phương pháp trước đó (ví dụ k-reciprocal Re-ranking: 63.63 mAP / 77.11 Rank-1) và nói DML với hai MobileNet vượt state of the art học sâu lúc đó.

So sánh với distillation (Bảng 4) (CIFAR-100 top-1; Market-1501 mAP). Cột: Independent Net1 / Net2; "1 distills 2" = Net 2 sau khi chưng cất từ Net 1 đã train; DML Net1 / Net2:

Tập	Net 1	Net 2	Độc lập 1 / 2	1 distills 2	DML 1 / 2
CIFAR-100	WRN-28-10	ResNet-32	78.69 / 68.99	69.48	78.96 / 70.73
CIFAR-100	MobileNet	ResNet-32	73.65 / 68.99	69.12	76.13 / 71.10
Market-1501	InceptionV1	MobileNet	65.26 / 46.07	49.11	65.34 / 52.87
Market-1501	MobileNet	MobileNet	46.07 / 46.07	45.16	52.95 / 51.26

Kết luận paper: KD từ teacher mạnh cũng giúp student (so với độc lập), nhưng DML giúp nhiều hơn; và dùng cùng một mạng cùng cỡ làm teacher cố định thậm chí làm student *tệ hơn* học một mình (45.16 so với 46.07).

Cohort lớn hơn (Hình 2, Market-1501, MobileNet): mAP trung bình mỗi mạng tăng theo số mạng trong cohort (từ 1 đến 5), độ lệch chuẩn giảm; ensemble của cohort DML cũng tốt hơn ensemble của các mạng độc lập. Con số cụ thể chỉ có trong hình (không đọc được chính xác từ text trích xuất), nên không ghi ở đây.

5. Phân tích / ablation

Tại sao không bị "mù dắt mù": (lập luận của tác giả) loss nhãn thật khiến mọi mạng đoán đúng nhãn; khác biệt giữa các mạng nằm ở xác suất các lớp phụ; mạng tập hợp ước lượng tập thể về chúng.

Cực tiểu rộng hơn (Hình 3, MobileNet trên Market-1501):

- Training loss của DML và độc lập đều về mức nhỏ như nhau (cả hai khớp hoàn toàn dữ liệu train, train accuracy 100%), nên DML không tìm cực tiểu *sâu* hơn.
- Thêm nhiễu Gauss với độ lệch chuẩn σ vào từng tham số: loss của mô hình độc lập tăng vọt, của DML tăng ít hơn nhiều, tức DML nằm ở vùng phẳng hơn (theo lập luận [Chaudhari, Keskar] làm tổng quát hóa tốt hơn; đây là bằng chứng gián tiếp).

Cơ chế tăng entropy: vì KL phạt nặng chỗ mạng này gán 0 mà peer gán khác 0, các mạng đều dồn thêm khối lượng xác suất vào các lớp phụ và phủ nhiều lớp phụ khác nhau hơn. Với ResNet-32 trên CIFAR-100, entropy hậu nghiệm trung bình trên tập train là **1.7099** (DML) so với **0.2602** (độc lập). Tác giả liên hệ với "entropy regularisation", nhưng của họ là "mutual probability matching on reasonable alternatives" thay vì thiên vị entropy mù quáng.

DML_e (ensemble teacher) kém hơn DML (Hình 4, 5 MobileNet trên Market-1501): ensemble tạo mục tiêu nhọn hơn tại lớp đúng, entropy dự đoán thấp hơn, mất đi chính "tín hiệu lớp phụ" mà DML cần. Ensemble tốt cho dự đoán nhưng không tốt cho tín hiệu dạy.

6. Giới hạn

Tác giả nêu hoặc hàm ý:

- DML làm các mạng *giống nhau hơn* (giảm đa dạng) nên không nhằm tạo ensemble đa dạng; mục tiêu là nghiệm bền vững.
- Dùng thêm nhiều mạng trong cohort tốn bộ nhớ và tính toán khi huấn luyện (họ lập luận mạng nhỏ thì nhét được nhiều mạng vào một GPU).

Nhận xét:

- Chỉ ảnh phân loại/re-ID, mạng nhỏ đến vừa, không có ngôn ngữ, không có chuỗi, không có mô hình cực lớn.
- Chi phí train gấp K lần (mỗi bước forward K mạng). So sánh công bằng nên tính theo compute: "1 distills 2" chỉ tốn compute của một student cộng teacher đã có, DML tốn nhiều hơn.
- Không có độ lệch chuẩn nhiều seed trong các bảng chính, và kết quả "DML thắng KD" so với một teacher cụ thể, không tune temperature hay trọng số (formulation không có temperature, α bằng 1).

- Một paper sau (10 Does KD really work) cho thấy nhiều câu chuyện "student bắt chước teacher" thật ra chưa đạt khớp function; DML không đo agreement.
- Không có phân tích lý thuyết hội tụ; giải thích "cực tiểu rộng" chỉ là bằng chứng gián tiếp.

7. Liên hệ với các bài khác

- **02 Hinton KD:** DML dùng cùng ý "lớp phụ chứa thông tin" nhưng nguồn tín hiệu là peer cùng train.
- **06 Born-again networks:** BAN là nối tiếp theo thế hệ (tuần tự), DML là song song, đồng thời. Cả hai đều chứng minh teacher không cần mạnh hơn student.
- **08 Teacher assistant, 09 Efficacy of KD, 41 Capacity gap law:** các bài về "teacher quá lớn so với student"; DML là giải pháp lách (peer cùng cỡ).
- **10 Does KD really work:** nhắc rằng KD (kể cả cohort/ensemble) thường không bắt chước đủ trung thực; hữu ích khi đánh giá DML.
- **22 DINO:** teacher/student cùng kiến trúc, nhưng teacher là EMA của student (một dạng self-distillation khác).

8. Tóm tắt công thức / khái niệm

Khái niệm	Ý nghĩa
DML	Nhóm mạng cùng train, mỗi mạng học nhãn thật + khớp xác suất peer
Loss mỗi mạng	$L_{C_k} + \frac{1}{K-1}\sum_{l\neq k} D_{KL}(p_l \| p_k)$
Mimicry loss	Thành phần KL kéo về xác suất peer
DML_e	Dùng trung bình peer làm một teacher; kém hơn DML
Posterior entropy	Entropy của softmax; DML cao hơn (1.7099 so với 0.2602 ở ResNet-32)
Cực tiểu rộng	Loss ít tăng khi nhiễu tham số; gắn với tổng quát hóa tốt
Cohort	Nhóm các mạng học cùng nhau

9. Kết luận

Deep Mutual Learning huấn luyện đồng thời nhiều mạng, mỗi mạng học cả nhãn thật lẫn dự đoán của các mạng khác. Trên CIFAR-100 và Market-1501, cách này vượt học độc lập và KD từ teacher cố định trong các thiết lập được thử. Tác giả liên hệ kết quả với entropy hậu nghiệm và cực tiểu phẳng.

Nguồn gốc: https://arxiv.org/abs/1706.00384

Bài 08

Improved Knowledge Distillation via Teacher Assistant (2020)

Tác giả: Seyed Iman Mirzadeh, Mehrdad Farajtabar, Ang Li, Nir Levine, Akihiro Matsukawa, Hassan Ghasemzadeh.
Venue / arXiv: arXiv:1902.03393 (v2, 17/12/2019); bản trong file mang thông báo bản quyền AAAI 2020 (AAAI Conference on Artificial Intelligence).
Nguồn: https://arxiv.org/abs/1902.03393
Chủ đề: Khoảng cách dung lượng (capacity gap) giữa teacher và student; chưng cất nhiều bước qua mạng trung gian "teacher assistant" (TA); phân tích lý thuyết dựa trên VC và cảnh quan loss.
Vị trí trong lộ trình: Đọc trước: 02 (Hinton KD), 06–07 (BAN, DML: teacher không cần mạnh hơn student). Bài này thêm: ngược lại với "teacher càng lớn càng tốt", chứng minh có điểm quá lớn thì student học tệ đi, và đề xuất bắc cầu bằng các mạng cỡ trung. Đọc tiếp: 09 (Efficacy of KD: giải thích vì sao teacher quá tốt có thể hại), 10 (Does KD really work), 41 (Capacity gap law cho LLM).

> Đây là ghi chú học tập bằng tiếng Việt, diễn giải và tóm lược, không phải bản dịch paper. Số liệu lấy từ paper; chỗ nào bảng bị lộn trong file trích xuất thì ghi rõ.

1. Bối cảnh

KD chuẩn (Hinton 2015) coi như: có teacher lớn đã train, student nhỏ cố định, student học nhãn thật cộng soft target của teacher. Ngầm hiểu: teacher càng mạnh càng tốt. Bài này kiểm tra điều đó và thấy sai khi **khoảng cách kích thước** quá lớn. Điều này có ý nghĩa thực tế: kích thước student thường bị giới hạn bởi thiết bị và teacher có thể đã được chọn sẵn.

Khái niệm cần có:

- **Baseline KD (BLKD):** loss $= (1-\lambda)\,\mathrm{CE}(\text{nhãn}) + \lambda\,\tau^2\,\mathrm{KL}$ giữa softmax làm mềm của student và teacher với **temperature** τ (chia logit cho τ để phân phối bớt nhọn, lộ rõ các lớp phụ).
- **NOKD:** huấn luyện student từ đầu, không distillation.
- **Capacity / size** (proxy trong bài): số lớp conv với CNN thường, số block với ResNet.

2. Ý tưởng chính

Tác giả cố định student CNN hai lớp tích chập rồi thay đổi teacher từ 4 đến 10 lớp. Độ chính xác của teacher tăng theo kích thước, nhưng kết quả student tăng rồi giảm. Phương pháp TAKD đưa một hoặc nhiều mô hình trung gian vào chuỗi chưng cất: assistant học từ teacher, sau đó student học từ assistant. Bài báo bổ sung phân tích cận lỗi VC và minh họa về cảnh quan loss để giải thích quan sát này.

3. Phương pháp chi tiết

3.1 Loss

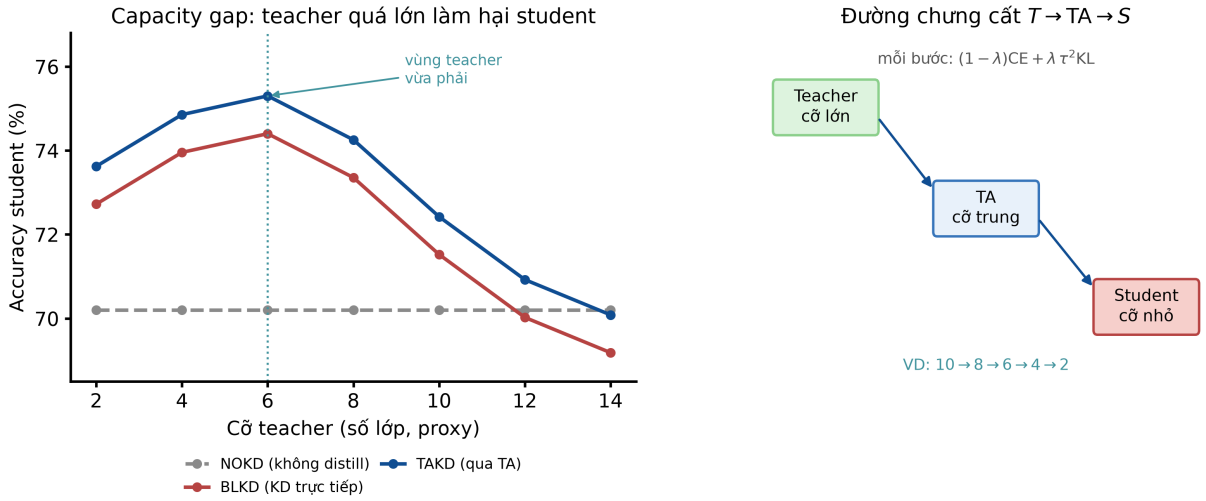
$$L_{SL} = H(\text{softmax}(a_s), y_r), \qquad L_{KD} = \tau^2\,\mathrm{KL}(\text{softmax}(a_s/\tau),\ \text{softmax}(a_t/\tau))$$

$$\boxed{L_{student} = (1-\lambda)L_{SL} + \lambda L_{KD}}$$

(Ký hiệu a_s, a_t là logit của student và teacher; λ cân bằng hai loss; paper viết KL theo thứ tự đối số như trên.)

3.2 Ba yếu tố cạnh tranh khi teacher lớn lên

1. Teacher dự đoán tốt hơn nên giám sát tốt hơn (có lợi).
2. Teacher quá phức tạp, student **không đủ dung lượng hay cơ chế** để bắt chước (có hại).
3. Teacher **tự tin hơn** nên logit/soft target bớt mềm, tín hiệu lớp phụ yếu đi (có hại).



Capacity gap (đường cong định tính tự tính theo mô tả note): accuracy student tăng rồi giảm khi teacher quá lớn; TAKD bắc cầu $T \to \text{TA} \to S$ với loss BLKD (minh hoạ, tự tính)

Ban đầu yếu tố 1 thắng, sau đó 2 và 3 thắng. Cũng đúng chiều đối ngẫu: cố định teacher 10 lớp và thu nhỏ student, lợi ích của KD so với NOKD cũng tăng rồi giảm (Hình 3).

3.3 TAKD

Bước 1: distill teacher T sang TA. Bước 2: dùng TA như teacher để distill sang student S. Mở rộng: chuỗi nhiều TA, gọi là **đường chưng cất (distillation path)**, ví dụ $10 \to 8 \to 6 \to 4 \to 2$ (số là cỡ mạng).

```
Input: teacher T (đã train), student S (cố định), danh sách cỡ TA c_1 > c_2 > ... > c_m
↪  (giữa cỡ S và cỡ T)
prev = T
for TA_i in [TA_1 .. TA_m]:                  # cỡ giảm dần
    TA_i = train(arch(c_i), loss=BLKD, teacher=prev)   # (1-λ)CE + λ τ^2 KL
    prev = TA_i
S = train(arch(S), loss=BLKD, teacher=prev)
```

Mọi bước dùng cùng công thức BLKD; hyperparameter (λ, τ) được tìm bằng công cụ tối ưu hyperparameter (Neural Network Intelligence, thuật toán TPE), 120 lần thử cho mỗi thí nghiệm, và con số báo cáo là top-1 test accuracy tốt nhất tìm được (lưu ý ở mục Giới hạn).

3.4 Phân tích lý thuyết (tóm tắt)

Dựa trên VC theory: lỗi của một hàm f_s so với hàm thật f_r bị chặn bởi sai số ước lượng $O\left(|F_s|_C / n^{\alpha_{sr}}\right)$ cộng sai số xấp xỉ ϵ_{sr}; $|F|_C$ là độ đo dung lượng lớp hàm, n số mẫu, $\frac{1}{2} \le \alpha \le 1$ là **tốc độ học** (gần $\frac{1}{2}$ với bài khó, gần 1 với bài dễ). Với student học từ nhãn thật (NOKD), student học từ teacher (BLKD), và student qua TA (TAKD), cận trên lần lượt là:

$$\text{NOKD: } O\left(\frac{|F_s|_C}{n^{\alpha_{sr}}}\right) + \epsilon_{sr}$$

$$\text{BLKD: } O\left(\frac{|F_t|_C}{n^{\alpha_{tr}}} + \frac{|F_s|_C}{n^{\alpha_{st}}}\right) + \epsilon_{tr} + \epsilon_{st}$$

$$\text{TAKD: } O\Big(\frac{|F_t|_C}{n^{\alpha_{tr}}} + \frac{|F_a|_C}{n^{\alpha_{at}}} + \frac{|F_s|_C}{n^{\alpha_{sa}}}\Big) + \epsilon_{tr} + \epsilon_{at} + \epsilon_{sa}$$

Lập luận: khoảng cách càng lớn thì α càng nhỏ, tức học khó hơn: $\alpha_{st} \leq \alpha_{sa}$ và $\alpha_{st} \leq \alpha_{at}$. Chia khoảng cách $T \to S$ thành hai đoạn ngắn hơn thì hai số mũ lớn hơn và tổng cận trên nhỏ hơn BLKD (theo tác giả, với giả định của Hinton là $\epsilon_{at} + \epsilon_{sa} \leq \epsilon_{st}$). Họ thừa nhận đây là **cận trên, tiệm cận** ($n \to \infty$), kế thừa Lopez-Paz et al. (2015); ở mẫu hữu hạn, BLKD có thể hỏng khi $|F_t|_C$ quá lớn hoặc khoảng cách quá lớn (khi α_{st} gần α_{sr}) và đó chính là lúc TAKD quan trọng.

4. Thí nghiệm và kết quả chính

Dữ liệu: CIFAR-10, CIFAR-100, và một thí nghiệm ImageNet. **Kiến trúc:** CNN kiểu VGG (cỡ = số lớp conv) và ResNet (cỡ = số block). PyTorch, SGD Nesterov momentum 0.9, lr 0.1, 150 epoch (ResNet hạ lr 0.01 ở epoch 80 và 0.001 ở epoch 120, weight decay 0.0001).

Bảng 1: BLKD, NOKD, TAKD (top-1 accuracy %; CNN: S=2, TA=4, T=10; ResNet CIFAR: S=8, TA=20, T=110; ImageNet ResNet: S=14, TA=20, T=50):

Mô hình	Dữ liệu	NOKD	BLKD	TAKD
CNN	CIFAR-10	70.16	72.57	73.51
CNN	CIFAR-100	41.09	44.57	44.92
ResNet	CIFAR-10	88.52	88.65	88.98
ResNet	CIFAR-100	61.37	61.41	61.82
ResNet	ImageNet	65.20	66.60	67.36

Kích cỡ TA tốt nhất (Bảng 2, CNN S=2, T=10; bảng trích xuất hơi lộn xộn nhưng đọc được):

Dữ liệu	TA=8	TA=6	TA=4
CIFAR-10	72.75	73.15	73.51
CIFAR-100	44.28	44.57	44.92

Bảng 3 (ResNet S=8, T=110): CIFAR-10: TA=56: 88.70, TA=32: 88.73, TA=20: 88.90, TA=14: 88.98. CIFAR-100: TA=56: 61.47, TA=32: 61.55, TA=20: 61.82, TA=14: 61.5.

Hình 2 và 3 (accuracy của student theo cỡ teacher, lợi ích theo cỡ student) chỉ có dưới dạng đồ thị, nên không có con số chính xác trong text. Hình 5 (tất cả đường chưng cất cho CNN trên CIFAR-100, T=10) cũng ở dạng đồ thị.

So sánh với các phương pháp khác (Hình 6 phải): TAKD một TA so với FitNet, AT, FSP, BSS (con số lấy từ Heo et al. 2018) và Mutual (DML do họ tự cài), teacher ResNet26, student ResNet8 và ResNet14, TA là ResNet20 (cho S=ResNet14) và ResNet14 (cho S=ResNet8). Họ báo cáo TAKD thắng tất cả, nhưng bảng số trong hình không đọc được từ text, nên không ghi số ở đây.

5. Phân tích / ablation

- **TA kích thước nào?** Mọi kích thước TA trung gian đều tốt hơn BLKD và NOKD. Nhưng TA tối ưu không nằm đúng ở giữa theo số lớp: với S=2, T=10, TA=4 tốt hơn TA=6 hay 8. Quan sát của tác giả: TA tối ưu gần điểm giữa theo **accuracy trung bình khi huấn luyện từ đầu** của student và teacher (Hình 4: cỡ 4 gần mức trung bình giữa S=2 và T=10 hơn cỡ 6 hay 8; với ResNet CIFAR-10 là TA=14,

CIFAR-100 là TA=20). Heuristic đề xuất: chọn TA có accuracy (từ đầu) gần trung bình của student và teacher, nhưng tác giả nói còn cần nghiên cứu hệ thống hơn.

- **Một bước hay nhiều bước?** Thử mọi đường chưng cất trên các cỡ chẵn (với n mạng trung gian có 2^n đường). Mọi đường có TA đều tốt hơn BLKD và NOKD; với S=2 và S=4, đường đi qua **tất cả** TA là tốt nhất; với S=2, các biến thể nhiều bước đều "comparably good". Không bị ràng buộc thì đường đầy đủ là tối ưu, và trong thực tế chọn đường theo ngân sách thời gian/tính toán. Tìm tự động đường tốt nhất (khi giới hạn số TA) được để cho nghiên cứu tương lai (có thảo luận trong appendix, không có trong text này).
- **Cảnh quan loss (Hình 7, kỹ thuật của Li et al. 2018):** student train bằng TAKD có bề mặt loss **phẳng hơn** quanh cực tiểu so với NOKD và BLKD (CNN S=2 trên CIFAR-100; ResNet S=8). Tác giả thừa nhận mối liên hệ giữa độ phẳng và tổng quát hóa còn tranh luận, và coi đây là bằng chứng gián tiếp (cùng hướng với DML).
- **Không phụ thuộc phép biến thể khác:** tác giả nói TAKD bổ trợ, có thể kết hợp với FitNet, AT, v.v.

6. Giới hạn

Do tác giả nêu:

- Phân tích lý thuyết dùng cận trên và tiệm cận, không phải hiệu năng thực.
- Chọn kích cỡ và số lượng TA là chi phí thêm; chưa có phương pháp tự động hoàn toàn; chọn đường chưng cất khi giới hạn số TA còn để ngỏ.

Nhận xét:

- Mức cải thiện nhỏ ở ResNet (ví dụ CIFAR-10: BLKD 88.65 sang TAKD 88.98; CIFAR-100: 61.41 sang 61.82), chỉ báo cáo "accuracy tốt nhất qua 120 lần thử hyperparameter" mà không có độ lệch chuẩn theo seed; chênh vài phần mười có thể nằm trong nhiễu và phần chọn-theo-test của tuner.
- Cỡ mạng được đo bằng số lớp/block (proxy), không kiểm soát tách biệt chiều rộng và chiều sâu; "khoảng cách" chưa được định nghĩa bằng số tham số hay FLOP.
- Kiến trúc cùng họ (CNN-CNN, ResNet-ResNet); chưa kiểm tra transformer, VLM hay LLM.
- Các tác giả giải thích hiện tượng bằng ba yếu tố, nhưng không tách riêng và đo từng yếu tố (đây là giả thuyết giải thích hợp lý chứ không được chứng minh cô lập). Bài 09 sẽ thử làm rõ nguyên nhân này.
- Chi phí huấn luyện nhiều hơn: mỗi TA là một lần train đầy đủ.

7. Liên hệ với các bài khác

- **02 Hinton KD:** TAKD dùng đúng loss BLKD ở mỗi bước, chỉ thay đổi cấu trúc chuỗi.
- **06 BAN, 07 DML:** cả hai hạ yêu cầu về teacher; TAKD nói teacher quá lớn cũng là vấn đề.
- **09 Efficacy of KD:** phân tích sâu hơn vì sao teacher lớn hơn không tốt hơn và đề xuất dừng sớm teacher/regularize teacher. Đọc bài này trước để có động cơ.
- **10 Does KD really work:** hỏi student có khớp teacher thật không; có thể giải thích rằng capacity gap làm khớp tệ hơn.
- **39 Small models struggle with strong reasoners, 41 Capacity gap law, 42 Distillation scaling laws:** phiên bản hiện đại của cùng vấn đề cho LLM; 40 Gemma 2 và 44 Qwen3 xử lý bằng teacher vừa phải hoặc chuỗi.
- **33 TAID** (nếu đọc sau): nội suy teacher-student theo thời gian, cùng tinh thần "bắc cầu qua khoảng cách".

Gợi ý học tập

Cố định student, dữ liệu và ngân sách rồi so sánh chưng cất trực tiếp từ các teacher khác nhau với một chuỗi có assistant. Cũng nên báo cáo chi phí huấn luyện assistant: cải thiện chất lượng không tự nó chứng minh tiết

kiệm tính toán.

8. Tóm tắt công thức / khái niệm

Khái niệm	Ý nghĩa
BLKD	$(1-\lambda)$CE $+ \lambda\tau^2$KL chưng cất trực tiếp T $\rightarrow$ S
NOKD	Student huấn luyện từ đầu
Capacity gap	Chênh lệch cỡ giữa T và S; quá lớn thì KD kém hiệu quả
TA	Mạng cỡ trung gian, học từ T rồi dạy S
Distillation path	Chuỗi T $\rightarrow$ TA$_1$ $\rightarrow \cdots \rightarrow$ S
3 yếu tố	Teacher giỏi hơn (+), student không đủ sức bắt chước (−), teacher quá tự tin làm target kém mềm (−)
Cận VC	Lỗi $\leq O(\lvert F\rvert_C/n^\alpha) + \epsilon$; α nhỏ khi khoảng cách lớn
Heuristic cỡ TA	Chọn TA có accuracy gần trung bình của S và T

9. Kết luận

Trong các thí nghiệm, teacher lớn hơn không phải lúc nào cũng giúp student tốt hơn. TAKD thêm mô hình trung gian và cải thiện kết quả so với KD trực tiếp trên CIFAR-10/100 và ImageNet; mức cải thiện không lớn. Bài báo bổ sung cận lý thuyết và quan sát về cảnh quan loss.

Nguồn gốc: https://arxiv.org/abs/1902.03393

Bài 09

On the Efficacy of Knowledge Distillation (2019)

Tác giả: Jang Hyun Cho, Bharath Hariharan (Cornell University).
Venue / arXiv: arXiv:1910.01348 (v1, 03/10/2019). Bản trong file không ghi venue (đây được biết là bài ICCV 2019, nhưng thông tin này không có trong text đã đọc).
Nguồn: https://arxiv.org/abs/1910.01348
Chủ đề: Đánh giá thực nghiệm KD theo cặp teacher/student; "teacher lớn không dạy tốt hơn"; capacity mismatch; sequential KD không giúp; early-stopped KD (ESKD) và teacher dừng sớm.
Vị trí trong lộ trình: Đọc trước: 02 (Hinton KD), 06 (BAN), 08 (Teacher assistant). Bài này thêm: kiểm chứng rộng hơn trên CIFAR10 và ImageNet cho hiện tượng "teacher mạnh hơn nhưng dạy tệ hơn", chứng minh nguyên nhân là student không bắt chước nổi (capacity mismatch) và bác bỏ giả thuyết "teacher quá nhọn"; đồng thời cho thấy cách chữa của 08 (chuỗi qua trung gian) **không hiệu quả** trong thiết lập của họ, thay vào đó chữa bằng dừng sớm. Đọc tiếp: 10 (Does KD really work), 41 (Capacity gap law).

> Đây là ghi chú học tập bằng tiếng Việt, diễn giải và tóm lược, không phải bản dịch paper. Số liệu lấy từ paper; bảng trong file trích xuất bị lộn ở vài chỗ, nơi nào đọc không chắc thì được ghi rõ.

1. Bối cảnh

KD hấp dẫn vì "tổng quát": bất kỳ student nào học được từ bất kỳ teacher nào. Nhưng các bài trước thường chỉ thử vài cặp được chọn kỹ rồi ngầm giả định kết luận đúng cho mọi cặp. Cũng có vài báo cáo thất bại lẻ tẻ (ví dụ Zagoruyko & Komodakis không thành công với KD trên ImageNet). Paper này hỏi: KD thực sự hiệu quả khi nào, cặp teacher-student nào tốt, và làm gì cho các cặp xấu?

Khái niệm cần có:

- **KD cổ điển (Hinton):** với **temperature** $\tau > 1$, softmax làm mềm $\tilde{p}_k = \dfrac{e^{s_k/\tau}}{\sum_j e^{s_j/\tau}}$, loss student:

$$L = \alpha L_{cls} + (1-\alpha) L_{KD}, \qquad L_{KD} = -\tau^2 \sum_k \tilde{p}^t_k \log \tilde{p}^s_k$$

  Lưu ý: ở đây α là trọng số của **nhãn thật** và $(1-\alpha)$ cho KD, ngược với ký hiệu của bài 08 (nơi λ là trọng số KD). Giá trị phổ biến: $\tau \in \{3, 4, 5\}$, $\alpha = 0.9$ (theo paper; chính xác là họ dùng $\alpha = 0.9$ cho cả hai thí nghiệm CIFAR10 và ImageNet).
- **Attention transfer (AT, bài 04):** thêm loss khiến student khớp bản đồ attention của teacher; ở đây trọng số $\beta = 1000$.
- **Sequential KD:** lặp KD, mô hình đời i học từ đời $i-1$ (đã xuất hiện ở BAN, bài 06).

2. Ý tưởng chính

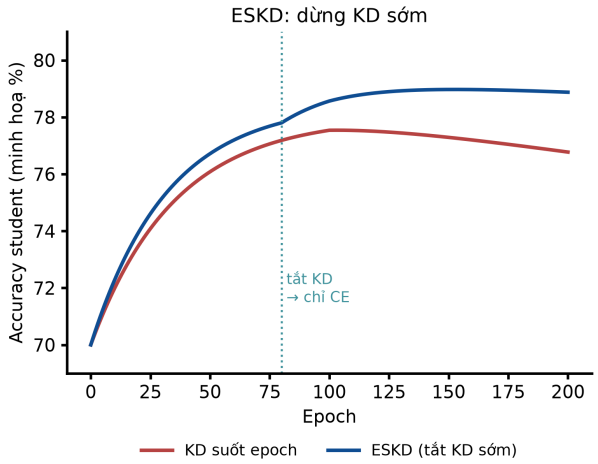
Điểm xuất phát: teacher chính xác hơn thường **không** dạy tốt hơn. Sau khi vượt qua một cỡ nhất định, student chính xác kém đi dù teacher tốt hơn. Cơ chế đề xuất: **capacity mismatch**, student nhỏ không tìm được nghiệm trong không gian của nó khớp với teacher lớn (KL tới teacher cao ngay cả trên tập train). Có thêm vấn đề thứ hai: với bài khó như ImageNet, student ở chế độ underfit và phải đánh đổi giữa hai loss, KD loss làm hại cross-entropy ở cuối huấn luyện. Cách chữa đơn giản: **dừng KD sớm** (chỉ dùng CE cho phần còn lại) cho student, và **dừng sớm quá trình huấn luyện teacher** (teacher "regularized", hành xử như mạng nhỏ hơn). Chuỗi KD nhiều bước (như TA) không cứu được.

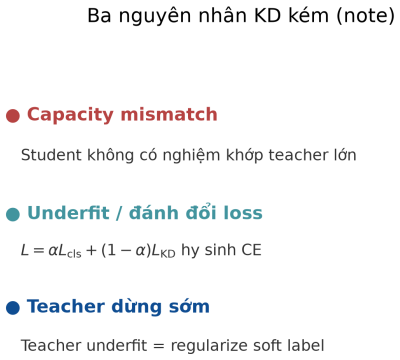
3. Phương pháp chi tiết

Paper chủ yếu là nghiên cứu thực nghiệm; hai kỹ thuật chính:

3.1 ESKD (Early-Stopped Knowledge Distillation) cho student

Chỉ dùng $L = \alpha L_{cls} + (1-\alpha) L_{KD}$ ở giai đoạn đầu; sau đó chuyển sang chỉ L_{cls} đến hết. Động cơ: ở cuối huấn luyện, trong chế độ underfit, KD loss kéo tham số ra khỏi nghiệm cross-entropy tốt. Bằng chứng (Bảng 3): ESKD có CE train thấp hơn nhưng KD loss cao hơn so với Full KD, tức Full KD đang đánh đổi hai loss.





ESKD tắt KD sớm rồi chỉ CE; ba nguyên nhân KD kém: capacity mismatch, đánh đổi loss, teacher underfit (minh hoạ, tự tính)

3.2 Teacher dừng sớm

Thay vì tìm kiếm kiến trúc teacher tối ưu bằng lưới (đắt), huấn luyện teacher lớn **ít epoch hơn** rồi dùng làm teacher. Ý tưởng trích dẫn: mạng lớn trained ngắn hành xử như mạng nhỏ nhưng vẫn có không gian tìm kiếm lớn. Cài đặt (CIFAR10): tổng số epoch $n \in \{35, 50, 65, 80\}$, lr giảm 0.2 mỗi $\lfloor (n-5)/3 \rfloor$ epoch. ImageNet: $n \in \{35, 50\}$, lịch hạ lr (15,25,30) cho 35 và (20,35,45) cho 50. Chi phí chỉ khoảng một phần ba đến một phần tư số epoch gốc.

```
# Quy trình khuyến nghị của paper (ESKD + early-stopped teacher)
teacher = train(big_arch, epochs=n_small)          # n_small << n_full, ví dụ 35 hay 50
↪  epoch (ImageNet)
student = train(small_arch, loss=α CE + (1-α) KD(teacher, τ), epochs=E_kd)   # giai đoạn
↪  KD
student = continue_train(student, loss=CE, epochs=E_rest)                    # giai đoạn
↪  chỉ CE
```

(Số epoch KD/CE cụ thể cho ESKD trên ImageNet không rõ từ text trích xuất; paper chỉ nói dừng KD "close to convergence".)

3.3 Cấu hình thực nghiệm

CIFAR10: 200 epoch, SGD momentum 0.9, lr 0.1 giảm 0.2 mỗi 60 epoch, $\tau = 4$, $\alpha = 0.9$, $\beta = 1000$; lặp 5 lần, báo median, mean, std; kiến trúc ResNet, WideResNet, DenseNet. ImageNet: theo Zagoruyko & Komodakis, SGD Nesterov momentum 0.9, lr 0.1, weight decay 10^{-4}, hạ lr 0.1 mỗi 30 epoch, ResNet.

4. Thí nghiệm và kết quả chính

ImageNet, student ResNet18 (Bảng 1, top-1 error %):

Teacher	Lỗi teacher	Lỗi student
(không, huấn luyện từ đầu)	-	30.24
ResNet18	30.24	30.57
ResNet34	26.70	30.79
ResNet50	23.85	30.95

Teacher càng mạnh, student càng kém, và **mọi** teacher đều làm student tệ hơn student huấn luyện từ đầu (30.24). Trên CIFAR10 (Hình 2, dạng đồ thị): với student WRN16-1, WRN28-1, DenseNet40-12 thay đổi độ sâu hoặc độ rộng teacher, accuracy student tăng rồi giảm; số liệu cụ thể chỉ có trong hình.

KD error trên CIFAR10 (Bảng 2, tỉ lệ mẫu mà student và teacher dự đoán khác nhau, %, cột đọc được là train / test):

Student	Teacher	KD error train	KD error test
WRN28-1	WRN28-3	0.23	4.05
WRN28-1	WRN28-4	0.25	4.53
WRN28-1	WRN28-6	0.23	4.54
WRN28-1	WRN28-8	0.31	4.81
WRN16-1	WRN16-3	1.70	6.32
WRN16-1	WRN16-4	1.69	6.52
WRN16-1	WRN16-6	1.94	6.91
WRN16-1	WRN16-8	1.69	7.01

(Cách ghép cột train/test dựa trên thứ tự trong bảng; sự tăng dần của cột test theo cỡ teacher là điểm chính.) Student càng không khớp teacher khi teacher lớn hơn.

ESKD trên ImageNet, ResNet18 student (Bảng 3, top-1 error test % / CE train / KD train / KD test):

Teacher	Lỗi	CE	KD (train)	KD (test)
ResNet18	30.57	0.146	2.916	3.358
ResNet18 (ES KD)	29.01	0.123	2.234	2.491
ResNet34	30.79	0.145	1.357	1.503
ResNet34 (ES KD)	29.16	0.123	2.359	2.582
ResNet50	30.95	0.146	1.553	1.721
ResNet50 (ES KD)	29.35	0.124	2.659	2.940

Dòng “(ES KD)” là student dùng KD dừng sớm với teacher đó. Cả ba đều thắng student huấn luyện từ đầu (30.24). Dù vậy, KD loss test của student vẫn tăng theo cỡ teacher, tức vẫn là vấn đề capacity.

Tổng hợp ImageNet (Bảng 8, top-1 error % của student ResNet18):

Phương pháp	Teacher	Lỗi
Scratch	-	30.24
Full KD	ResNet18 / 34 / 50	30.57 / 30.79 / 30.95
Seq. Full KD (số trong bài báo gốc)	3 Gen. / 6 Gen.	30.12* / 29.6*

Phương pháp	Teacher	Lỗi
KD+ONE (số trong bài báo gốc)	3 Branches	29.45 ± 0.23*
Full KD + AT	ResNet34	30.94 (cài lại) / 29.3* (paper gốc)
ESKD	ResNet18 / 34 / 50	29.01 / 29.16 / 29.35
ESKD	ResNet152	29.45
ESKD với teacher dừng sớm	ResNet34 (50 epoch) / ResNet50 (35) / ResNet152 (35)	29.02 / 29.05 / 29.26
ESKD + AT	ResNet34 / ResNet34 (50)	28.84 / 28.61

(Dấu * là số lấy từ paper khác.) Kết quả tốt nhất: ESKD + AT với teacher ResNet34 dừng sớm 50 epoch, lỗi **28.61%**, tức cải thiện khoảng 1.6 điểm so với huấn luyện từ đầu (30.24) và 0.7 điểm so với số tốt nhất của attention transfer gốc (theo paper).

Sequential KD không giúp (CIFAR10):

- Bảng 4 (5 thế hệ, lỗi % - cột: Last Gen / ensemble tất cả thế hệ / Scratch / ensemble scratch): ResNet8 AT+KD: 13.469 / 12.786 / 12.569* / 10.176; ResNet14 AT+KD: 9.226 / 8.653 / 9.078* / 6.675; WRN16-2 KD: 6.101 / 5.181 / 6.428 / 4.865; WRN16-2 AT+KD: 5.696 / 5.310 / 6.418 / 5.003. (* nghĩa là lặp KD thậm chí làm giảm hiệu năng.) Ensemble các thế hệ KD **không** thắng ensemble cùng số model huấn luyện từ đầu.
- Bảng 5 (student WRN16-1, scratch 8.759 ± 0.129; teacher WRN16-3): teacher huấn luyện từ đầu (lỗi 5.34) cho student 7.68 ± 0.259; teacher đã qua 5 vòng KD (lỗi 4.89, tốt hơn) cho student 7.67 ± 0.19. Teacher chính xác hơn nhưng dạy không tốt hơn.
- Bảng 6 (Large WRN16-8 -> Medium WRN16-3 -> Small WRN16-1): Large->Med->Small cho 7.99 ± 0.24; Med->Small 7.68 ± 0.26; Large->Small 8.03 ± 0.14. Chuỗi qua trung gian gần như bằng KD trực tiếp từ Large, và **Med->Small trực tiếp** tốt nhất, dù Med kém chính xác hơn Large. (Đọc bảng này có hơi mơ hồ do mangle; con số 7.614 xuất hiện gần dòng Med->Small trong text trích xuất, có thể là một lần chạy riêng lẻ.)
- ImageNet (Bảng 7, ESKD tuần tự): Large->Small->Small: teacher thứ nhất 21.69, teacher thứ hai 29.45, student 29.41; Med->Small->Small: 23.85, 29.35, 29.35; Small->Small->Small: 30.24, 29.01, 29.15. Kết quả tốt nhất vẫn là KD một lần từ "small" sang "small".

Teacher dừng sớm (CIFAR10, Hình 4): với student WRN16-1 và WRN28-1 có "sweet spot" cho số epoch của teacher; mọi teacher dừng sớm đều cho student tốt hơn teacher đầy đủ tối ưu (WRN16-3 và WRN28-3).

Chuyển giao sang Places365 (Bảng 9, top-1 / top-5 error %): Scratch 47.38 / 18.51; ESKD ResNet18 47.09 / 18.13; Full KD ResNet34 47.86 / 18.61; ESKD ResNet34 (50) 47.14 / 18.32; Full KD ResNet50 47.92 / 18.72; ESKD ResNet50 (35) 47.02 / 18.14; ESKD ResNet152 (35) 47.25 / 18.25.

$\tau = 20$ **trên ImageNet (Bảng 10, top-1 / top-5):** Scratch 30.24 / 10.92; Full KD với ResNet18 / 50 / 152: 30.75 / 11.11, 30.98 / 10.20, 31.27 / 11.59; ESKD ResNet18 / ResNet50: 29.00 / 9.91, 29.00 / 9.76; ESKD ResNet50 (35): 28.89 / 9.76.

5. Phân tích / ablation

- **Giả thuyết "teacher quá nhọn" bị bác:** làm mềm bằng temperature cao không đổi kết quả cho Full KD (Bảng 10 và Hình 6). Khác với giả thuyết yếu tố 3 của bài 08.
- **Giả thuyết capacity mismatch được ủng hộ:** KD error/loss cao nhất với teacher lớn nhất, ngay cả trên tập train (Bảng 2, 3).

- **Underfit và đánh đổi hai loss:** trên ImageNet, KD giúp lúc đầu rồi làm hại ở cuối (Hình 3). ESKD đạt CE train thấp hơn nhưng KD loss cao hơn, nhất quán với giả thuyết đánh đổi.
- **ESKD tương thích với AT**, và dừng sớm teacher cũng tương thích (cộng dồn: 28.61%).
- **Nhiều họ kiến trúc:** kết luận (teacher lớn không tốt hơn; teacher dừng sớm tốt hơn) lặp lại với các tổ hợp DenseNet và WideResNet (Hình 5), và với α, τ khác nhau (Hình 6); teacher dừng sớm cũng cho kết quả tốt hơn khi chuyển sang Places365.
- **Thế hệ lặp:** hiệu quả phụ thuộc mạnh vào kiến trúc (ResNet8 và ResNet14 thế hệ cuối thậm chí tệ hơn scratch); ensemble các thế hệ tương quan nhau nên yếu hơn ensemble scratch.

6. Giới hạn

Do tác giả nêu hoặc hàm ý:

- Đề xuất teacher dừng sớm là heuristic (“some evidence” từ tài liệu khác), chưa có lý thuyết; chọn số epoch tối ưu cần tuning (“sweet spot”).
- Tác giả coi tìm kiếm lưới kiến trúc teacher là quá đắt, nên tối ưu hóa kiến trúc teacher không được giải.

Nhận xét:

- Chỉ phân loại ảnh (CIFAR10, ImageNet, Places365), mạng tích chập cùng họ; không có chuỗi, ngôn ngữ, VLM.
- Chênh lệch trên ImageNet khá nhỏ (khoảng 1.2 điểm) và nhiều bảng ImageNet chỉ một lần chạy; chỉ CIFAR10 có 5 lần lặp và std.
- Kết luận “sequential KD vô dụng” dựa trên một vài cặp WRN và ResNet và không tinh chỉnh tách biệt từng bước; nó mâu thuẫn bề ngoài với bài 08 (TAKD có lợi) chứ không bác bỏ trực tiếp, vì thiết lập hyperparameter và cách chọn TA khác nhau (TAKD tune 120 lần thử và chọn TA theo heuristic).
- Cách dừng sớm teacher giảm chất lượng teacher (lỗi teacher cao hơn), nên thực chất là trao đổi giữa chất lượng và “dễ bắt chước”, không phải miễn phí.
- Dùng “KD error” (disagreement top-1) trên CIFAR vì thang đo của loss; nhưng KD error top-1 không đo khớp toàn phân phối (xem bài 10 để thấy mức khớp thực sự có thể kém hơn nhiều).

7. Liên hệ với các bài khác

- **08 Teacher assistant:** cùng quan sát (teacher lớn hại student) nhưng đưa ra chẩn đoán khác (08: ba yếu tố gồm “teacher quá tự tin”; 09 bác “quá nhọn” bằng temperature và nhấn mạnh capacity mismatch + underfit) và giải pháp khác (08: chuỗi qua TA; 09: không hiệu quả, dùng dừng sớm). Nên đọc cặp này cùng nhau và coi kết quả mâu thuẫn là cảnh báo rằng giải pháp phụ thuộc thiết lập.
- **06 BAN:** 09 kiểm tra lại sequential KD và thấy hiệu quả không ổn định, ensemble các thế hệ không hơn ensemble scratch.
- **07 DML:** được nhắc ở phần liên quan.
- **04 Attention transfer:** ESKD + AT cộng dồn; AT là phụ gia thường dùng.
- **10 Does KD really work:** đào sâu “student có thực sự khớp teacher” và bổ sung kết luận bi quan hơn: ngay cả khi capacity dư, khớp vẫn kém do tối ưu.
- **41 Capacity gap law, 39 Small models struggle with strong reasoners, 42 Distillation scaling laws:** phiên bản hiện đại cho LLM; 09 là bằng chứng thực nghiệm CNN sớm nhất trong lộ trình về hiện tượng này.
- **11 Patient and consistent teacher:** ngược hẳn với “dừng sớm”: train rất lâu (nhiều giờ/epoch) với cùng ảnh cho cả teacher và student, kết quả lại tốt (cần xem vì sao hai kết luận có thể cùng đúng: 11 xử lý bằng cách làm cho student “khớp” teacher bằng tăng cường dữ liệu mạnh và rất nhiều epoch).

Gợi ý học tập

So sánh loss và độ chính xác trên cả tập huấn luyện lẫn tập giữ lại khi thay teacher. Một đối chứng dừng KD sớm giúp phân biệt lợi ích của lịch chưng cất với lợi ích do lựa chọn teacher.

8. Tóm tắt công thức / khái niệm

Khái niệm	Ý nghĩa
$L = \alpha L_{cls} + (1-\alpha) L_{KD}$	Loss KD ở bài này; $\alpha = 0.9$ trọng số nhãn thật
$L_{KD} = -\tau^2 \sum_k \tilde{p}_k^t \log \tilde{p}_k^s$	Cross-entropy giữa softmax làm mềm của teacher và student
KD error	Tỉ lệ mẫu mà dự đoán student khác teacher
Capacity mismatch	Student không có nghiệm khớp teacher lớn trong không gian của nó
Underfit/đánh đổi hai loss	Student không tối thiểu hóa cả CE và KD nên hy sinh CE
ESKD	Dừng KD sớm, phần còn lại chỉ CE
Teacher dừng sớm	Train teacher ít epoch hơn như một dạng regularization
Sequential KD	Student đời i học từ đời $i-1$ (BAN) hoặc chuỗi qua trung gian (TA)

9. Kết luận

Các thí nghiệm cho thấy teacher chính xác hơn có thể khó được student bắt chước hơn; trên ImageNet, KD đầy đủ thậm chí làm kết quả kém hơn huấn luyện từ đầu. Chuỗi trung gian không giúp trong thiết lập này, còn dừng KD sớm và dùng teacher huấn luyện ngắn hơn đem lại cải thiện: lỗi ResNet-18 giảm từ 30.24% xuống 28.61% khi kết hợp AT.

Nguồn gốc: https://arxiv.org/abs/1910.01348

Bài 10

Does Knowledge Distillation Really Work? (2021)

Tác giả: Samuel Stanton, Pavel Izmailov, Polina Kirichenko, Alexander A. Alemi, Andrew Gordon Wilson.
Venue / arXiv: arXiv:2106.05945 (v2, 06/12/2021); NeurIPS 2021 (35th Conference on Neural Information Processing Systems, Sydney).
Nguồn: https://arxiv.org/abs/2106.05945
Chủ đề: Tách "fidelity" (student khớp teacher) khỏi "generalization" (student đúng trên dữ liệu mới); KD thường có fidelity thấp; hai giả thuyết nguyên nhân: identifiability (dữ liệu distillation) và optimization (tối ưu hóa).
Vị trí trong lộ trình: Đọc trước: 02 (Hinton KD), 06 (BAN: student vượt teacher), 08–09 (capacity gap). Bài này thêm: ngay cả khi student **đủ dung lượng** (self-distillation, cùng kiến trúc), student vẫn khớp teacher rất kém, nguyên nhân chủ yếu là **tối ưu hóa**, không phải dung lượng hay dữ liệu. Đọc tiếp: 11 (Patient and consistent teacher: công thức thực tế để đạt fidelity cao nhờ huấn luyện rất lâu), rồi các bài về KD cho LLM.

> Đây là ghi chú học tập bằng tiếng Việt, diễn giải và tóm lược, không phải bản dịch paper. Số liệu lấy từ paper; nhiều kết quả chỉ có dưới dạng đồ thị (và phụ lục không có trong bản text đã đọc) nên chỉ ghi những số có trong văn bản.

1. Bối cảnh

Câu chuyện quen thuộc về KD: student học một biểu diễn "trung thực" của teacher (lớn, hoặc ensemble) nhờ soft label. Các bài trước (BAN, TA, Efficacy) đã nghi ngờ phần "tốt hơn teacher" hoặc "teacher lớn không dạy tốt", nhưng đa số đo bằng **accuracy** của student. Bài này nói: accuracy trộn lẫn hai thứ khác nhau.

- **Fidelity:** student dự đoán giống teacher đến mức nào (khớp phân phối dự đoán).
- **Generalization:** student dự đoán đúng trên dữ liệu mới.

Hai thứ này có thể ngược chiều: self-distillation (student cùng kiến trúc) làm student tốt hơn teacher **vì không khớp** (regularization nhờ thất bại) chứ không phải vì khớp tốt. Nếu mục tiêu là nén một ensemble hay mô hình đáng tin để triển khai trên thiết bị nhỏ, thì fidelity mới là thứ cần.

Khái niệm cần có:

- **Ensemble teacher:** trung bình xác suất của m mạng; logit teacher $z_t = \log\left(\sum_i \sigma(z_i)/m\right)$.
- **Calibration / ECE:** mức độ khớp giữa độ tự tin dự đoán và độ đúng thực tế; **NLL** là negative log-likelihood.
- **Identifiability** (khả năng nhận dạng): dữ liệu distillation có đủ để phân biệt student "đúng" với student "sai" không.
- **Basin (lòng chảo) của loss:** vùng quanh một cực tiểu cục bộ.

2. Ý tưởng chính

Tác giả đo **agreement** top-1 và **KL** giữa teacher và student trên tập test. Họ thấy fidelity thấp đến khó tin (ResNet-56 trên CIFAR-100: chỉ đạt khoảng 80 đến 86%, xa mức 99% của ví dụ đơn giản LeNet-5/MNIST), kể cả khi student đủ dung lượng, kiến trúc khác (VGG), tập dữ liệu lớn hơn (ImageNet) hoặc miền khác (văn bản IMDB). Hai giả thuyết: (1) dữ liệu distillation chưa đủ/đúng (identifiability), (2) student không giải nổi bài toán tối ưu. Họ bác (1) tương đối và ủng hộ (2): student không khớp teacher ngay cả **trên tập distillation (train)**, tăng thêm augmentation hay dữ liệu GAN còn làm bài toán tối ưu khó hơn. Thí nghiệm khởi tạo student gần teacher cho thấy bề mặt loss có nhiều lòng chảo và student bị kẹt ở lòng chảo xấu.

3. Phương pháp chi tiết

3.1 Loss KD

$$L_s = \alpha L_{NLL} + (1-\alpha)L_{KD}, \qquad L_{KD}(z_s, z_t) = -\tau^2 \sum_{j=1}^{c} \sigma_j\Big(\tfrac{z_t}{\tau}\Big) \log \sigma_j\Big(\tfrac{z_s}{\tau}\Big)$$

Để tách fidelity khỏi tác động của nhãn thật, **trong thí nghiệm chính họ đặt** $\alpha = 0$ (chỉ loss KD; ablation về α nằm ở phụ lục). Khi $\tau = 1$, L_{KD} tương đương KL(teacher ‖ student) (sai khác hằng số). Khi $\tau \to \infty$, gradient theo logit student xấp xỉ $z_t - z_s$, tức tương đương hồi quy ℓ_2 trên logit, coi mọi lớp quan trọng như nhau. Ý của tác giả: τ quyết định "mềm" đến đâu và phân bổ dung lượng student: student rất nhỏ có thể dồn sức khớp top-k thay vì khớp toàn bộ phân phối.

3.2 Đo fidelity

$$\text{Agreement} = \frac{1}{n}\sum_i \mathbb{1}\{\arg\max_j \sigma_j(z_{t,i}) = \arg\max_j \sigma_j(z_{s,i})\}, \qquad \text{Predictive KL} = \frac{1}{n}\sum_i \mathrm{KL}(\hat{p}_t(y|x_i) \,\|\, \hat{p}_s(y|x_i))$$

Agreement chỉ xét top-1; KL nhạy với toàn bộ phân phối. Tác giả cũng báo accuracy, NLL, ECE.

Cẩn thận với đo agreement: ba mô hình độc lập có accuracy 55%, 75%, 95%: f_1 và f_3 chỉ có thể trùng tối đa 60%, trong khi f_2 và f_3 trùng ít nhất 70%, nhưng điều đó không có nghĩa f_2 là bản distill tốt hơn của f_3. Cách kiểm soát: cho thêm một student s' chưng cất bằng đúng quy trình đó từ một **teacher độc lập** t'; so fidelity (t, s) với (t, s'). Nếu hai con số xấp xỉ nhau thì sự trùng khớp chỉ do cùng tổng quát hóa tốt chứ không phải do mimic.

3.3 Sơ đồ chẩn đoán trong bài báo

```
Fidelity thấp -> loại các nguyên nhân dễ:
   dung lượng student (self-distillation vẫn thấp; tăng student ít tác dụng)
   kiến trúc (VGG cũng vậy), quy mô dữ liệu (ImageNet cũng vậy), miền (IMDB cũng vậy)
-> hai giả thuyết còn lại:
   (A) Identifiability: tăng/đa dạng hóa dữ liệu distillation (augmentation, GAN, dữ
   ↪  liệu mới D1) có nâng fidelity?
   (B) Optimization: student có khớp teacher ngay trên dữ liệu distillation (train
   ↪  agreement)?
        -> kéo dài huấn luyện, đổi optimizer, khởi tạo gần teacher (θ_s = λθ_t +
        ↪  (1-λ)θ_r)
```

4. Thí nghiệm và kết quả chính

Thiết lập chung: CIFAR-100 với ResNet-56, ensemble $m \in \{1, 3, 5\}$ (đến 12 ở phụ lục), MNIST/EMNIST với LeNet-5, nhiều kết quả trung bình 3 lần chạy. Mã: github.com/samuelstanton/gnosis.

Khi nào KD thành công (MNIST): teacher LeNet-5 train trên tập con 200 mẫu của MNIST (test accuracy 84% đến 86%), distill trên toàn bộ MNIST (60.000), và 25%, 50%, 100% của EMNIST (697.932 ảnh): student đạt trên 99% agreement trên test. Ở đây self-distillation không cải thiện generalization.

CIFAR-100 self-distillation (Hình 1a): teacher và student đều một ResNet-56; thêm ảnh GAN (SN-GAN) 12.5k, 25k, 37.5k, 50k vào dữ liệu distillation: fidelity trên test tăng nhưng chỉ lên dưới 80% (theo văn bản), còn test accuracy của student **giảm** theo hướng tiệm cận teacher. Nghĩa là student vượt teacher khi dữ liệu nhỏ, nhờ không khớp.

Distill ensemble (Hình 1b): teacher là ensemble 3 ResNet-56: fidelity tăng thì accuracy cũng nhích lên, tức fidelity và generalization đồng hướng, nhưng vẫn còn khoảng cách fidelity lớn. Khoảng cách accuracy giữa teacher và student tăng theo số thành phần ensemble.

Augmentation (Hình 3, ensemble 5 ResNet-56): các chính sách: baseline (crop+flip), baseline với $\tau = 4$, Rotation, Vertical Flip, ColorJitter, CombAug, MixUp, MixUp ($\tau = 4$), GAN ($\tau = 4$), OOD, Noise.

- MixUp và GAN cho generalization tốt nhất; MixUp ($\tau = 4$) cho fidelity tốt nhất nhưng chỉ đạt **86%** test agreement.
- Baseline ($\tau = 4$) đã là baseline mạnh: **84.5%** agreement.
- Augmentation dạng nhiễu (Noise) và ngoài phân phối (OOD) làm tệ trên mọi chỉ số.

Tái sử dụng dữ liệu (Hình 4, CIFAR-100 chia đôi D_0, D_1): teacher train trên D_0. Student distill trên D_0 (tái dùng) có accuracy test cao hơn nhưng fidelity thấp hơn và ECE tệ hơn so với student distill trên D_1 (dữ liệu chưa từng thấy); distill trên $D_0 \cup D_1$ cho kết quả tốt nhất. Agreement tốt nhất khoảng 85%, không hơn kết quả từ augmentation.

Train agreement (Hình 5): thêm dữ liệu ngoài tập train của teacher làm student **khó khớp hơn** trên chính tập distillation. Với 50k ảnh GAN, ngay cả student cùng dung lượng chỉ đạt 95% train agreement. Với augmentation nặng: Rotation, Vertical Flip, ColorJitter cho train agreement 80% đến 90% với mọi cỡ teacher; Combined Augs (kết hợp ba loại) chỉ còn khoảng **60%** ở self-distillation. Nhưng Combined Augs lại có test agreement tốt hơn các chính sách có train agreement cao hơn: đánh đổi giữa "nhiều nhãn teacher" (dễ nhận dạng) và "dữ liệu khó khớp" (khó tối ưu).

Tối ưu hóa (ResNet-20 trên CIFAR-100, self-distillation, Hình 6): để đơn giản hóa, họ thay BatchNorm bằng LayerNorm, vì với BatchNorm, hai mạng có trọng số giống nhau vẫn có thể dự đoán khác nhau do thống kê activation tích lũy khác nhau. Kết quả:

- Huấn luyện lâu hơn giúp chút ít: SGD 300 epoch cho 78.95% train agreement, 5k epoch cho 83.3%; từ 1k lên 5k chỉ thêm khoảng 2%. Adam không tốt hơn SGD.
- Khởi tạo $\theta_s = \lambda\theta_t + (1-\lambda)\theta_r$ (θ_t trọng số teacher, θ_r khởi tạo ngẫu nhiên): với $\lambda \leq 0.25$ student hội tụ tới giá trị loss kém, agreement thấp; tại $\lambda = 0.375$ có bước nhảy đột ngột: loss về mức tối ưu, agreement tăng vọt gần 100%. Hình chiếu 2D của bề mặt loss cho thấy $\lambda \in \{0, 0.25\}$ rơi vào một lòng chảo khác lòng chảo của teacher.
- Chia sẻ khởi tạo ban đầu với teacher (Bảng 1, 10 lần, teacher accuracy trung bình 70.522 (0.412)): Agreement: ngẫu nhiên 77.174 (0.352) so với giống teacher 77.098 (0.238); KL: 0.836 (0.016) so với 0.838 (0.020); CKA (độ tương đồng activation) tăng nhẹ ở 3 stage (0.939/0.925/0.885 so với 0.951/0.937/0.890). Chức năng không khác biệt.

Loại trừ nguyên nhân khác (nêu ở Mục 4.4, chi tiết ở phụ lục không có trong bản text đã đọc): tăng dung lượng student ít tác dụng; VGG cho kết quả tương tự; ImageNet và IMDB (phân loại cảm xúc) cho fidelity thấp tương tự.

5. Phân tích / ablation

- **Accuracy tốt không có nghĩa fidelity tốt:** ngay cả ngoài self-distillation, mô hình tốt nhất về generalization không phải lúc nào cũng là tốt nhất về fidelity.
- **Fidelity tương quan với calibration khi distill ensemble:** student có fidelity cao nhất luôn là student calibrate tốt nhất (dù chưa chắc chính xác nhất).
- **Trade-off dữ liệu và tối ưu:** mở rộng dữ liệu distillation ngoài tập train của teacher giúp xác định nghiệm đúng (identifiability), nhưng làm bài tối ưu vốn đã khó trở nên khó hơn.
- **Temperature là đòn bẩy rẻ nhất:** $\tau = 4$ trên baseline augmentation đã cho 84.5% agreement, gần bằng cách phức tạp hơn; tác giả lập luận hiệu quả nhỏ của việc mở rộng support so với tempering cho thấy thiếu nhãn teacher không phải nút thắt chính.
- **Nguyên nhân gốc:** tối ưu hóa. Student không khớp teacher trên tập train, không phải vì thiếu dung lượng (self-distillation có nghiệm biết trước: chính trọng số teacher) mà vì SGD/Adam hội tụ tới lòng chảo xấu. Ngay cả khởi tạo giống teacher cũng không đủ; phải "gần" mới thoát.

- **Tương phản với học thường:** trong huấn luyện thông thường ta không cần giải bài tối ưu thật tốt (nghiệm phẳng, thiên kiến ngầm của SGD giúp tổng quát hóa), còn ở KD fidelity gắn trực tiếp với việc giải bài tối ưu đó.

6. Giới hạn

Do tác giả nêu hoặc hàm ý:

- "Hợp lý nhưng không chắc" rằng huấn luyện cực lâu (hàng chục nghìn epoch) có thể đạt 100% train agreement; họ không thử.
- Họ chỉ xét classification ảnh/văn bản vừa phải; phần lớn kết luận tổng quát (ImageNet, IMDB, VGG) nằm ở phụ lục.

Nhận xét:

- Phần phân tích tối ưu hóa dùng ResNet-20 + LayerNorm (bỏ BatchNorm) và $\alpha = 0$ để đơn giản hóa; hệ quả sang kiến trúc transformer lớn, dữ liệu rất lớn và loss có nhãn ($\alpha > 0$) chưa rõ.
- Không đề xuất phương pháp mới để cải thiện fidelity; bài chủ yếu là chẩn đoán. Cách chữa thực tế xuất hiện ở bài 11 (đồng thời công bố).
- Thí nghiệm khởi tạo $\lambda\theta_t$ yêu cầu biết trọng số teacher cùng kiến trúc, nên chỉ chẩn đoán được trong self-distillation, không áp dụng khi student là mạng nhỏ hơn.
- Fidelity được đo bằng top-1 agreement và KL trung bình; không đo fidelity ở các đuôi (ví dụ lỗi hiếm), mà trong OCR có thể quan trọng.
- Một số số trong bản text bị mất do hình vẽ, nên nhiều kết luận (như xu hướng theo kích thước ensemble) chỉ xác nhận qua mô tả văn bản.

7. Liên hệ với các bài khác

- **06 BAN:** bài này giải thích lại "student vượt teacher" như một hệ quả của việc **không** khớp (regularization), không phải của việc khớp tốt.
- **08 Teacher assistant, 09 Efficacy of KD:** ở đó "capacity mismatch" là nguyên nhân; bài này chỉ ra rằng ngay cả khi không mismatch, khớp vẫn kém. Hai kết luận không loại trừ nhau.
- **07 DML:** nhận định về entropy/calibration có liên hệ (fidelity cao tương quan với calibration tốt).
- **11 Patient and consistent teacher:** bài song sinh (công bố gần như cùng lúc; được trích trong Related Work của bài này). 11 đưa ra công thức thực tế để thu hẹp khoảng cách fidelity: cùng ảnh đầu vào (kể cả augmentation) cho teacher và student, mixup (tạo "function matching"), và train rất lâu (bài này gọi tương ứng: "careful treatment of data augmentation, MixUp, extended training duration").
- **13 CRD, 14 Decoupled KD, 12 Relational KD:** các cải tiến loss; bài 10 gợi ý rằng điểm nghẽn thật nằm ở tối ưu hóa chứ không (chỉ) ở dạng loss.
- **Bài LLM (28 MiniLLM, 29 GKD, 31 Rethinking KL):** quan hệ với khái niệm "khớp phân phối" và lựa chọn hướng KL; bài 10 đóng khung tầm quan trọng của việc đo khớp trực tiếp.

8. Tóm tắt công thức / khái niệm

Khái niệm	Ý nghĩa
Fidelity	Student khớp dự đoán của teacher
Generalization	Student đúng trên dữ liệu chưa thấy
Agreement	Tỉ lệ mẫu mà argmax của teacher và student trùng nhau
Predictive KL	KL trung bình từ phân phối teacher sang student
Student độc lập s'	Student chưng cất từ teacher độc lập, dùng làm đối chứng
Identifiability	Dữ liệu distillation có đủ để xác định nghiệm đúng không

Khái niệm	Ý nghĩa
Optimization	Student không khớp ngay cả trên dữ liệu distillation
$\theta_s = \lambda\theta_t + (1-\lambda)\theta_r$	Khởi tạo nội suy; ngưỡng $\lambda = 0.375$ cho bước nhảy fidelity
Self-distillation	Teacher và student cùng kiến trúc

9. Kết luận

KD cải thiện khả năng tổng quát hóa trong các thí nghiệm nhưng student thường có fidelity thấp với teacher, kể cả khi đủ dung lượng. Tác giả quy phần khó khăn cho tối ưu: student không khớp teacher ngay cả trên dữ liệu chưng cất, trừ khi được khởi tạo trong vùng nghiệm thích hợp. Mở rộng dữ liệu giúp nhận dạng nhưng cũng làm bài toán khớp khó hơn.

Nguồn gốc: https://arxiv.org/abs/2106.05945

Bài 11

Knowledge distillation: A good teacher is patient and consistent (2022)

Tác giả: Lucas Beyer, Xiaohua Zhai, Amélie Royer, Larisa Markeeva, Rohan Anil, Alexander Kolesnikov (Google Research, Brain Team).
Venue / arXiv: arXiv:2106.05237 (bản v2 ngày 21/06/2022; bản v1 xuất hiện 09/06/2021). Bản text đã đọc không ghi tên hội nghị. Tên file lộ trình dùng năm 2022 theo bản v2.
Nguồn: https://arxiv.org/abs/2106.05237
Chủ đề: Công thức thực nghiệm để chưng cất mô hình thị giác lớn (BiT-ResNet-152x2) xuống ResNet-50/MobileNetV3; "function matching" (khớp hàm); nhất quán đầu vào giữa teacher và student, mixup mạnh, huấn luyện cực lâu; Shampoo; chưng cất trên dữ liệu ngoài miền.
Vị trí trong lộ trình: Đọc trước: 02 (Hinton KD), 09 (Efficacy of KD: khuyên dừng sớm), 10 (Does KD really work: fidelity thấp do tối ưu hóa). Bài này thêm: đáp án thực dụng cho những vấn đề đó, ngược hướng với 09: **train rất lâu**, cùng input cho teacher và student, và không precompute logit. Đọc tiếp: 12–14 (Relational KD, CRD, Decoupled KD: cải tiến loss), 21 (DeiT), 22 (DINO), 23 (AM-RADIO).

> Đây là ghi chú học tập bằng tiếng Việt, diễn giải và tóm lược, không phải bản dịch paper. Số liệu lấy từ paper; nhiều đồ thị không có số trong text nên chỉ ghi số có trong văn bản và bảng.

1. Bối cảnh

Mô hình thị giác tốt nhất rất lớn, còn thứ mọi người thực sự triển khai là ResNet-50 hay MobileNet (tác giả dẫn số lượt tải của các model BiT từ TensorFlow Hub: bản ResNet-50 nhỏ nhất được tải nhiều hơn hẳn các bản lớn). Bài hỏi: cho một ứng dụng và một mô hình lớn rất tốt, nén sang kiến trúc nhỏ mà không mất hiệu năng bằng cách nào?

Hai họ phương pháp nén chính: **pruning** (cắt bộ phận của mô hình; hạn chế: không đổi được họ mô hình, ví dụ ResNet sang MobileNet; và có thể gặp khó khăn phụ thuộc kiến trúc, ví dụ với group normalization cắt kênh làm phải cân lại nhóm kênh) và **knowledge distillation**. Bài chọn KD vì nó cho phép đổi kiến trúc, và không đề xuất phương pháp mới mà tìm ra cách làm KD **đúng**.

Các vấn đề thực tế khiến người ta làm KD chưa tốt (theo tác giả):

1. Hay **precompute** (tính sẵn) logit teacher offline cho mỗi ảnh để tiết kiệm compute ("fixed teacher").
2. KD cũng dùng ở nhiều ngữ cảnh khác (bán giám sát, tự giám sát) với khuyến nghị thiết kế ngược nhau.
3. KD cần số epoch lớn bất thường so với supervised.
4. Lựa chọn tốt cho lịch ngắn thường xấu cho lịch dài và ngược lại.

Khái niệm cần có: **mixup** (trộn tuyến tính hai ảnh $x = \lambda x_1 + (1-\lambda)x_2$ — ở đây KHÔNG trộn nhãn, vì nhãn là output của teacher cho ảnh đã trộn); **inception-style crop** (cắt ngẫu nhiên vùng kích thước và tỉ lệ ngẫu nhiên rồi resize); **Shampoo** (bộ tối ưu bậc hai dùng preconditioner theo tensor, đắt hơn Adam nhưng hội tụ nhanh hơn theo bước).

2. Ý tưởng chính

Nhìn KD như **function matching**: ép hàm do student biểu diễn khớp hàm của teacher trên **nhiều điểm hỗ trợ** (support points), chứ không phải "teacher cho nhãn mềm tốt hơn". Hệ quả ba nguyên tắc: (i) teacher và student phải thấy **chính xác cùng một view của ảnh** (cùng crop, cùng augmentation, tính teacher online ở mỗi bước), (ii) mở rộng đa tạp ảnh bằng augmentation mạnh, cụ thể mixup với hệ số lấy đều từ $[0, 1]$, để có

nhiều điểm hỗ trợ cả ngoài đa tạp ảnh gốc, và (iii) huấn luyện **rất lâu** (kiên nhẫn, hàng nghìn đến hàng vạn epoch) vì bài toán khớp hàm là bài tối ưu khó. Kết quả: ResNet-50 đạt 82.8% top-1 trên ImageNet.

3. Phương pháp chi tiết

3.1 Loss

Chỉ dùng KL giữa xác suất teacher và student, **không có** loss nhãn cứng:

$$\boxed{\mathrm{KL}(p_t \| p_s) = \sum_{i \in C} [\, -p_{t,i} \log p_{s,i} + p_{t,i} \log p_{t,i}]}$$

với **temperature** T: $p_s \propto \exp(\log p_s / T)$, $p_t \propto \exp(\log p_t / T)$ (tức softmax làm mềm cả hai phía; T nằm trong dải quét {1, 2, 5, 10}).

3.2 Bốn kiểu cấu hình (Hình 2)

Kiểu	Mô tả	Biến thể trong thí nghiệm
Fixed teacher	Dự đoán teacher cố định cho mỗi ảnh (precompute)	fix/rs (chỉ resize 224 cho cả hai), fix/cc (teacher dùng central crop cố định, student crop ngẫu nhiên nhẹ), fix/ic_ens (teacher = trung bình dự đoán trên 1k inception crop, student dùng inception crop ngẫu nhiên)
Independent noise	Teacher và student nhận augmentation độc lập	ind/rc (random crop nhẹ), ind/ic (inception crop)
Consistent teaching	Cắt/augment **một lần**, đưa cùng view cho cả hai	same/rc, same/ic
Function matching	Consistent + mixup (cùng ảnh trộn cho cả hai)	same/rc,mix, same/ic,mix (“FunMatch”)

3.3 Thiết lập huấn luyện

Adam mặc định (chỉ lr được quét), cosine decay không restart, weight decay “decoupled” được quét, gradient clipping 1.0 theo L2-norm toàn cục. Batch 512 (ImageNet: 4096). Ảnh 128x128 để phân tích rộng (ImageNet: 224x224). Mỗi cấu hình quét toàn bộ tổ hợp lr {0.0003, 0.001, 0.003, 0.01} x weight decay {1e-5, 3e-5, 1e-4, 3e-4, 1e-3} x T {1, 2, 5, 10}; trong hình mọi run được vẽ mờ, và nhấn mạnh run có validation accuracy cuối cùng tốt nhất; chọn siêu tham số trên validation split, báo test. Chạy trên Google Cloud TPU, “dozens of thousands of models” được huấn luyện.

```
for step in range(N):                              # N rất lớn: hàng nghìn đến chục nghìn
↪  epoch
    x = batch(images)
    x, _ = random_inception_crop(x) ; x = mixup(x, lam ~ U[0,1])   # MỘT view, lấy MỘT
    ↪  lần
    with no_grad: p_t = softmax(teacher(x) / T)  # teacher chạy ONLINE trên đúng view này
    p_s = softmax(student(x) / T)
    loss = KL(p_t || p_s)                         # không dùng nhãn cứng
    student <- Adam / Shampoo(loss)
```

3.4 Các mở rộng

- **Khác độ phân giải:** crop trên ảnh độ phân giải gốc, rồi resize khác nhau cho teacher và student: vẫn nhất quán nội dung. Có thể student 160 px (nhanh gấp 2) với teacher 224 px, hoặc teacher fine-tune ở 384 px với student 224 px.

- **Teacher ensemble:** trung bình logit của hai teacher (224 px và 384 px).
- **Khác họ kiến trúc:** ResNet sang MobileNetV3 (Large, thay BatchNorm bằng GroupNorm 8 nhóm).

4. Thí nghiệm và kết quả chính

Dữ liệu: Flowers102 (1020 ảnh train), Pets (37 lớp, 3312 ảnh), Food101 (khoảng 68k), SUN397 (khoảng 76k), ILSVRC-2012/ImageNet (1000 lớp, 1.281.167 ảnh). **Teacher chính:** BiT-M-R152x2 (BiT-ResNet-152x2, pretrain trên ImageNet-21k, GroupNorm và weight standardization; cỡ tính toán khoảng 10x ResNet-50 chuẩn). **Student:** ResNet-50 (biến thể BiT, GroupNorm). Teacher đạt 83.0% top-1 trên ImageNet.

Tính nhất quán là then chốt (Hình 3, Flowers102, 10.000 epoch, định tính):

- Mọi cấu hình "fixed teacher" đạt distillation loss train rất thấp nhưng student tổng quát hóa tệ (overfit mạnh trên dữ liệu nhỏ); mọi cấu hình "inconsistent" dừng ở điểm thấp.
- Consistent teaching tăng accuracy đáng kể; **function matching tốt nhất**, và nó không bao giờ đạt loss train thấp như fixed teacher nhưng tổng quát hóa tốt hơn.

Kiên nhẫn (Hình 4): accuracy student tiệm cận teacher khi tăng số epoch (trục log, đến 1M epoch cho Flowers102) trên cả bốn tập, không thấy overfitting kể cả ở 1M epoch. Ở 100 epoch (thời lượng bình thường), student kém hơn baseline transfer (ResNet-50 pretrain ImageNet-21k) nhiều. Student cuối cùng vượt cả baseline huấn luyện từ đầu với nhãn gốc lẫn baseline transfer BiT-M ResNet-50 (hai baseline đều được tune lr và weight decay kỹ).

ImageNet (Hình 5 trái, Bảng 1): với fixed teacher, student overfit sau khoảng 600 epoch; consistent teaching tăng liên tục theo thời lượng. Function matching hơi kém ở lịch ngắn (underfit) nhưng với 1200 epoch đã bằng consistent teaching 4800 epoch (tiết kiệm 75% compute). Bảng 1 (top-1 test accuracy %):

Thí nghiệm	300 epoch	1200	4800	9600
T224 -> S224	80.30	81.54	82.18	82.31
T224 -> S160	78.17	79.61	N/A	80.49
T384 -> S224	80.46	81.82	82.33	82.64

(T = độ phân giải đầu vào của teacher, S = của student; teacher 384 px đạt 83.7% top-1.)

So sánh với tài liệu (Bảng 2, top-1 ImageNet, kiến trúc, độ phân giải):

Mô hình	Kiến trúc	Độ phân giải	Accuracy
Revisiting ResNet	R50	160	78.8%
FunMatch (T224)	R50	160	80.5%
Original ResNet	R50	224	77.2%
BiT-M-R50	R50	224	78.4%
Meal-v2	R50	224	80.7%
FunMatch (T384+224)	R50	224	**82.8%**
Revisiting ResNet	R152	224	82.8%

Tác giả nói đây là ResNet-50 SOTA mới (khoảng 4.4% hơn ResNet-50 gốc của BiT[23] và 2.2% hơn kết quả tốt nhất trong tài liệu, vốn dùng thiết lập phức tạp hơn); tổng cộng 9600 epoch chưng cất.

Ensemble teacher: trung bình logit teacher 224 px và 384 px; student tốt hơn student của teacher mặc định ở mọi độ dài thử, sau 9600 epoch đạt **82.82%** top-1.

MobileNetV3 (Large, GroupNorm): chỉ dùng function matching, không dùng thủ thuật huấn luyện của bài MobileNet gốc: **74.60%** sau 300 epoch và **76.31%** sau 1200 epoch, mô hình MobileNetV3 tốt nhất đã công bố (theo tác giả).

Shampoo (Hình 5 giữa): đạt cùng accuracy mà Adam đạt ở 4800 epoch chỉ sau 1200 epoch, với chi phí mỗi bước tăng ít; cải thiện đều ở các thiết lập (chi tiết trong appendix D).

Khởi tạo bằng pretrain (Hình 5 phải): student khởi tạo từ BiT-M-ResNet50 cải thiện hơn 2% ở lịch ngắn, nhưng khoảng cách khép lại từ khoảng 1200 epoch; từ scratch ngang bằng ở 1200 và vượt nhẹ ở 4800 epoch.

Dữ liệu ngoài miền (Hình 6, Pets và SUN397): chưng cất trên ảnh hoàn toàn không liên quan vẫn "work to some extent": ví dụ student học phân loại Pets với khoảng **30% accuracy** chỉ nhờ xem ảnh Food được gán nhãn mềm bằng giống thú cưng. Dữ liệu trong miền tốt nhất; dữ liệu liên quan/chồng lấn (Pets với ImageNet, SUN397 với ImageNet) gần bằng nhưng cần lịch dài hơn.

Baseline supervised dùng chung công thức (Hình 7): train ResNet-50 bằng nhãn cứng với cùng thiết lập (mixup, lịch dài, thử cả SGD momentum) tệ hơn nhiều và overfit ở lịch dài, tức KD là bắt buộc để công thức hoạt động.

5. Phân tích / ablation

- **Consistency vs fixed teacher:** precompute logit nhìn rẻ nhưng hại: student "overfit" vào bộ nhãn cố định; nhất quán + online cho tín hiệu đa dạng vô hạn.
- **Function matching vs consistent teaching:** mixup tạo điểm hỗ trợ ngoài đa tạp ảnh; lợi ích rõ khi lịch dài, hơi hại ở lịch ngắn vì underfit.
- **Tối ưu hóa là nút cổ chai:** tác giả suy đoán khó tối ưu vì khớp một hàm có đầu ra nhiều chiều khó hơn khớp nhãn ảnh cố định. Shampoo rút ngắn 4x.
- **Khởi tạo và lịch:** khởi tạo tốt giúp lịch ngắn, mất lợi thế ở lịch dài (phù hợp kết luận He et al. "Rethinking ImageNet pre-training").
- **Không phụ thuộc cặp kiến trúc:** hoạt động khi đổi độ phân giải, họ kiến trúc, dùng ensemble teacher.
- **Out-of-domain:** hàm cần khớp trên đa dạng input; không cần ảnh trong miền, nhưng trong miền vẫn tốt nhất.

6. Giới hạn

Do tác giả nêu hoặc hàm ý:

- Chi phí huấn luyện rất lớn (9600 epoch ImageNet; hàng chục nghìn mô hình trong nghiên cứu); tác giả dùng độ phân giải thấp 128 px để phân tích. Họ thừa nhận chi phí tính toán và dùng Shampoo như bước đầu giảm.
- Phạm vi chỉ phân loại ảnh; không so sánh cạnh tranh với pruning (chỉ bàn lý do chọn KD).
- Không tối ưu loss; tập trung "recipe", không đề xuất phương pháp mới.

Nhận xét:

- Teacher mỗi bước phải chạy forward (online) nên chi phí mỗi bước cao hơn precompute; với teacher lớn (BiT-R152x2 hay 4B) đây là chi phí lớn; bài so compute theo epoch của student chứ không công bố tổng FLOPs đầy đủ.
- Mixup và inception crop là augmentation ảnh; tương đương cho văn bản/token/OCR không rõ ràng.
- Cạnh tranh với hướng "dừng sớm" của 09: không mâu thuẫn trực tiếp, 09 xét student nhỏ trên ImageNet với cấu hình KD đơn giản (precompute-like, không mixup), còn 11 xử lý bằng chuẩn nhất quán và thời gian cực dài; nhưng cho thấy kết luận "KD hại" trong 09 có thể một phần là sản phẩm của cấu hình huấn luyện.

- Teacher ở đây tốt hơn student chỉ khoảng 0.2 đến 0.7 điểm sau cùng, tức student gần như khớp teacher: bài này (chưa rõ trong text) gợi ý fidelity cao, ngược với bi quan của bài 10; hai bài ra đời cùng lúc (bài 10 trích bài này) và kết luận bổ sung nhau.

7. Liên hệ với các bài khác

- **10 Does KD really work:** bài đồng hành. 10 chẩn đoán (fidelity thấp, lý do là tối ưu hóa); 11 là công thức chữa: nhất quán đầu vào, mixup, lịch dài, Shampoo (bài 10 nhắc đến chính các khuyến nghị này như "careful treatment of data augmentation, MixUp, extended training duration" và nhận xét rằng chúng tăng accuracy nhưng vẫn còn khác biệt dự đoán).
- **09 Efficacy of KD:** bài 09 khuyên teacher dừng sớm và KD dừng sớm; 11 nói train cực lâu và dùng cả teacher đầy đủ. Hai kết luận đến từ thiết lập khác nhau (09: loss kết hợp nhãn cứng, không mixup; 11: không nhãn, mixup, online teacher).
- **08 Teacher assistant:** 11 không dùng TA mà vẫn ghép được ResNet-152x2 với ResNet-50 nhờ lịch rất dài.
- **06 BAN, 07 DML:** 11 cũng loại bỏ nhãn cứng ("không dùng thêm loss nhãn cứng"), gần với BAN thuần; ý "khớp hàm" bổ sung cho giải thích dark knowledge.
- **12 Relational KD, 13 CRD, 14 Decoupled KD:** các bài sau cải tiến dạng loss; 11 cho thấy cách huấn luyện (nhất quán + lâu) có thể quan trọng hơn dạng loss.
- **21 DeiT:** dùng KD với convnet teacher, token distillation; cùng tinh thần teacher online cùng view.
- **22 DINO, 23 AM-RADIO, 24 CLIP-KD:** KD đa-teacher và tự giám sát hiện đại dựa vào cùng nguyên tắc: student/teacher thấy cùng (hoặc consistent) view.
- **Các bài LLM (28 MiniLLM, 29 GKD, 31 Rethinking KL, 32 Speculative KD):** on-policy/online teacher trong GKD tương ứng với nguyên tắc "không precompute, teacher đánh giá đúng input mà student thấy".

8. Tóm tắt công thức / khái niệm

Khái niệm	Ý nghĩa
Function matching	Coi KD là khớp hàm trên nhiều điểm hỗ trợ
Consistent teaching	Teacher và student nhận cùng một view (cùng crop và augmentation)
Fixed teacher	Dự đoán teacher tính sẵn cho một view cố định; kém
Patient	Huấn luyện rất lâu (hàng nghìn đến chục nghìn epoch)
Loss	$\mathrm{KL}(p_t \| p_s)$ với temperature T, không có nhãn cứng
Mixup mạnh	Hệ số trộn lấy đều từ $[0, 1]$ để sinh điểm ngoài đa tạp ảnh
Shampoo	Optimizer bậc hai; đạt kết quả Adam 4800 epoch ở 1200 epoch
Out-of-domain distillation	Chưng cất bằng ảnh không thuộc miền tác vụ; vẫn học được ít nhiều
Kết quả chính	ResNet-50 82.8% (224 px), 80.5% (160 px); MobileNetV3 76.31%

9. Kết luận

Nghiên cứu nhấn mạnh tính nhất quán của dữ liệu đầu vào cho teacher và student, augmentation mạnh và lịch huấn luyện dài. Với thiết lập của tác giả, ResNet-50 đạt 82.8% top-1 ImageNet và MobileNetV3 đạt 76.31%. Các kết quả này cho thấy quy trình huấn luyện có thể quan trọng ngay cả khi không thêm loss mới.

Nguồn gốc: https://arxiv.org/abs/2106.05237

Bài 12

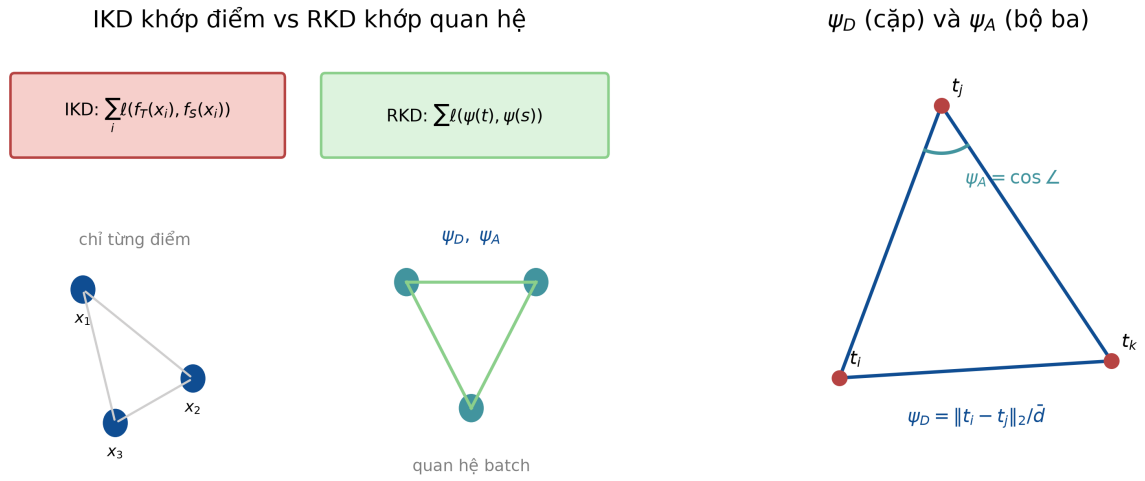
Relational Knowledge Distillation (2019)

Tác giả: Wonpyo Park, Dongju Kim, Yan Lu, Minsu Cho (POSTECH, Microsoft Research).
Venue / arXiv: arXiv:1904.05068 [cs.CV], v2 (1/5/2019); CVPR 2019 (theo ghi chú trên trang arXiv, bản text không ghi).
Nguồn: https://arxiv.org/abs/1904.05068
Chủ đề: Knowledge distillation dựa trên *quan hệ giữa các mẫu* (khoảng cách, góc) thay vì khớp từng đầu ra riêng lẻ; metric learning, phân loại, few-shot.
Vị trí trong lộ trình: Đọc sau 02 (Hinton KD), 03 (FitNets), 04 (Attention transfer), 05 (survey). Bài này đổi câu hỏi "tri thức là gì" từ *đầu ra của từng mẫu* sang *cấu trúc quan hệ giữa các mẫu*. Bài kế tiếp: 13 (CRD) cũng dùng thông tin giữa các mẫu nhưng bằng contrastive loss.

> Tài liệu học tập bằng tiếng Việt, là ghi chú diễn giải, không phải bản dịch paper.

1. Bối cảnh

Knowledge distillation (KD) = huấn luyện một mô hình nhỏ (*student*) bắt chước mô hình lớn (*teacher*). Trước bài này, hầu hết phương pháp KD có chung một dạng mà paper gọi là **Individual KD (IKD)**:



IKD khớp từng điểm; RKD khớp quan hệ batch qua ψ_D (khoảng cách chuẩn hoá) và ψ_A (cosine góc bộ ba) (minh hoạ, tự tính)

$$L_{IKD} = \sum_{x_i \in X} l(f_T(x_i),\ f_S(x_i)) \tag{1}$$

tức là với *từng* mẫu x_i, ép đầu ra của student $f_S(x_i)$ gần đầu ra của teacher $f_T(x_i)$. Ba ví dụ paper nêu:

- **Hinton KD** (bài 02): f là logit, l = KL giữa softmax có nhiệt độ τ.
- **FitNet** (bài 03): f là activation tầng giữa, l = bình phương khoảng cách Euclid, thêm ánh xạ tuyến tính β để khớp số chiều khi student hẹp hơn.
- **Attention transfer** (bài 04): f là bản đồ attention.

Vấn đề: IKD buộc student phải tái tạo *giá trị* đầu ra của teacher. Nếu student có số chiều embedding nhỏ hơn (16, 32, 64 chiều so với 512 chiều của teacher), việc khớp từng vector là phi thực tế và không cần thiết. Paper

lập luận (mượn từ ngôn ngữ học cấu trúc của Saussure) rằng ý nghĩa của một mẫu nằm ở *quan hệ của nó với các mẫu khác*, không phải ở tọa độ tuyệt đối.

2. Ý tưởng chính

Thay vì bảo student "mẫu này phải ra đúng vector này", RKD bảo student "hãy giữ *hình học* giữa các mẫu giống teacher". Cụ thể: với một nhóm 2 mẫu thì giữ tỉ lệ khoảng cách; với nhóm 3 mẫu thì giữ góc tạo bởi ba điểm embedding. Vì khoảng cách (đã chuẩn hóa) và góc không phụ thuộc số chiều hay hệ tọa độ, student ít chiều hơn vẫn có thể bắt chước được cấu trúc đó. IKD là trường hợp đặc biệt (quan hệ bậc 1, hàm thế là hàm đồng nhất), nên RKD là một tổng quát hóa. RKD cũng bổ trợ IKD: kết hợp hai loại cho kết quả tốt hơn.

3. Phương pháp chi tiết

3.1 Khung tổng quát

Ký hiệu $t_i = f_T(x_i)$, $s_i = f_S(x_i)$. Chọn một **relational potential function** ψ (hàm "thế quan hệ": nhận một bộ n embedding, trả về một đại lượng đo quan hệ của bộ đó):

$$\boxed{L_{RKD} = \sum_{(x_1,\ldots,x_n)\in X^N} l(\psi(t_1,\ldots,t_n),\ \psi(s_1,\ldots,s_n))} \tag{4}$$

Khi $n = 1$ và ψ = identity thì (4) quay về (1). Paper cho biết thế bậc cao hơn mạnh hơn nhưng tốn kém hơn, và đề xuất hai thế đơn giản.

3.2 Distance-wise loss (bậc 2)

$$\psi_D(t_i, t_j) = \frac{1}{\mu}\|t_i - t_j\|_2, \qquad \mu = \frac{1}{|X^2|}\sum_{(x_i,x_j)\in X^2}\|t_i - t_j\|_2 \tag{5,6}$$

μ là khoảng cách trung bình trong *mini-batch*, để chỉ quan tâm khoảng cách *tương đối*, bất kể teacher và student khác scale (ví dụ khác số chiều). Paper nói chuẩn hóa này cho hội tụ ổn định và nhanh hơn.

$$\boxed{L_{RKD\text{-}D} = \sum_{(x_i,x_j)\in X^2} l_\delta(\psi_D(t_i,t_j), \psi_D(s_i,s_j))} \tag{7}$$

với l_δ là **Huber loss** (bình phương khi sai lệch ≤ 1, tuyến tính khi lớn hơn; ít nhạy với outlier hơn MSE):

$$l_\delta(x,y) = \begin{cases} \frac{1}{2}(x-y)^2 & |x-y| \le 1 \\ |x-y| - \frac{1}{2} & \text{otherwise} \end{cases}$$

3.3 Angle-wise loss (bậc 3)

$$\psi_A(t_i,t_j,t_k) = \cos\angle t_i t_j t_k = \langle e^{ij}, e^{kj}\rangle, \qquad e^{ij} = \frac{t_i - t_j}{\|t_i - t_j\|_2} \tag{9}$$

$$\boxed{L_{RKD\text{-}A} = \sum_{(x_i,x_j,x_k)\in X^3} l_\delta(\psi_A(t_i,t_j,t_k), \psi_A(s_i,s_j,s_k))} \tag{10}$$

Góc là thuộc tính bậc cao hơn khoảng cách, nên cho student nhiều tự do hơn; paper quan sát angle-wise thường hội tụ nhanh hơn và tốt hơn.

3.4 Huấn luyện

$$L = L_{task} + \lambda_{KD} \cdot L_{KD} \tag{11}$$

Lấy *tất cả* cặp/bộ ba trong mini-batch làm tuple (không lấy mẫu phức tạp). Tên biến thể: RKD-D, RKD-A, RKD-DA (cả hai). Lưu ý của bài báo: vì RKD không truyền giá trị từng đầu ra, không nên dùng *riêng* RKD ở nơi giá trị đầu ra là quan trọng (như lớp softmax của phân loại); khi đó phải dùng kèm IKD hoặc loss của task.

Pseudo-code

```
def rkd_loss(t, s, w_d=1.0, w_a=2.0):
    # t, s: [B, D_T], [B, D_S] embeddings của teacher (no_grad) và student
    # --- distance-wise ---
    dt = pdist(t)                    # [B, B] khoảng cách Euclid
    ds = pdist(s)
    dt = dt / dt[dt > 0].mean()      # chia cho mu (mean khoảng cách cặp trong batch)
    ds = ds / ds[ds > 0].mean()
    L_d = huber(ds, dt).mean()
    # --- angle-wise ---
    et = normalize(t[None, :, :] - t[:, None, :], dim=2)   # e^{ij}, [B, B, D]
    es = normalize(s[None, :, :] - s[:, None, :], dim=2)
    at = bmm(et, et.transpose(1, 2))                       # cos góc, [B, B, B]
    as_ = bmm(es, es.transpose(1, 2))
    L_a = huber(as_, at).mean()
    return w_d * L_d + w_a * L_a

loss = task_loss(student(x), y) + lam * rkd_loss(teacher(x).detach(), student_emb(x))
```

(Pseudo-code minh họa công thức 7 và 10; code chính thức do tác giả công bố tại http://cvlab.postech.ac.kr/research/RKD/.)

Ví dụ nhỏ

Batch 3 mẫu. Teacher (2D): $t_1 = (0,0), t_2 = (2,0), t_3 = (2,2)$. Khoảng cách: 2, $2\sqrt{2}$, 2; $\mu = (2 + 2.83 + 2)/3 \approx 2.28$, nên ψ_D = 0.88, 1.24, 0.88. Student 1D (không thể giống toạ độ teacher): $s = (0, 2, 4)$ cho khoảng cách 2, 4, 2, $\mu = 2.67$, ψ_D = 0.75, 1.5, 0.75. Loss khác 0 nhưng nhỏ, và student 1D chỉ cần tái tạo *tỉ lệ* (cặp 1-3 xa gấp ~1.4 lần cặp kề) chứ không phải tọa độ. Với góc: $\psi_A(t_1, t_2, t_3) = \cos 90^\circ = 0$, trong khi student 1D chỉ có góc 0 hoặc 180 độ, nên góc là ràng buộc *cứng hơn* khi số chiều chênh nhau nhiều (nhận xét bổ sung).

4. Thí nghiệm và kết quả chính

Ba bài toán: metric learning (image retrieval), phân loại, few-shot. Độ đo retrieval: recall@K.

4.1 Metric learning (CUB-200-2011, Cars 196, Stanford Online Products)

Teacher: ResNet50 (pretrain ImageNet) + FC 512 chiều + l2 norm, huấn luyện bằng triplet loss. Student: ResNet18 với embedding 16/32/64/128 chiều. Với RKD *không dùng* task loss (triplet) hay nhãn gốc, chỉ học từ teacher. $\lambda_{RKD-D} = 1, \lambda_{RKD-A} = 2$. Batch 128, Adam, 5 ảnh dương mỗi lớp.

Recall@1, Cars 196 (trích Table 1b; ô RKD là "có l2 norm / không l2 norm"):

Student	Triplet	FitNet	Attention	DarkRank	RKD-D	RKD-A	RKD-DA
ResNet18-16	45.39	57.46	46.44	64.00	63.23 / 66.02	61.39 / 66.25	61.78 / 66.04
ResNet18-32	56.01	65.81	59.40	72.41	73.50 / 76.15	73.23 / 75.89	73.12 / 74.80
ResNet18-64	64.53	70.67	67.24	76.20	78.64 / 80.57	77.92 / 80.32	78.48 / 80.17
ResNet18-128	68.79	73.10	71.95	77.00	79.72 / 81.70	80.54 / 82.27	80.00 / 82.50

Teacher ResNet50-512 đạt 77.17 trên Cars 196 và 61.24 trên CUB. Student ResNet18-128 không l2 norm đạt 82.50, vượt teacher (paper nhấn mạnh điểm này). Trên CUB, ResNet18-128 RKD-DA đạt 59.69 / 60.67, thấp hơn teacher 61.24 (nên "student vượt teacher" trên CUB chỉ xảy ra ở một số cấu hình; ở bảng 1a các ô RKD ResNet18-128 là 58-60.9, nhỏ hơn 61.24). Paper nói gain tương đối của recall@1 từ RKD-DA tăng khi embedding nhỏ đi: 12.5, 13.8, 23.0, 27.7 trên CUB, và 20.0, 24.2, 33.5, 45.5 trên Cars.

Self-distillation (Table 2, student cùng kiến trúc, không l2 norm):

	CUB	Cars	SOP
ResNet50-512-Triplet (teacher)	61.24	77.17	76.58
Gen1	65.68	85.65	77.61
Gen2	65.11	85.61	77.36
Gen3	64.26	85.23	76.96

Mọi thế hệ đều vượt teacher ban đầu nhưng không cải thiện thêm từ thế hệ 2.

So với state-of-the-art (Table 3): RKD-DA-128 (backbone ResNet50) đạt Recall@1 là 64.9 (CUB), 84.9 (Cars), 77.5 (SOP); đứng đầu nhóm ResNet50 (so với Margin-128: 63.6, 79.6, 72.7). Ở nhóm GoogLeNet, RKD-DA-512 đạt 61.4 / 82.3 / 75.1, nhưng ABE8-512 vẫn cao hơn trên Cars (85.2) và SOP (76.3).

4.2 Phân loại (Table 4)

Teacher ResNet50 / student VGG11-BN trên CIFAR-100; ResNet101 / ResNet18 trên Tiny ImageNet. $\lambda_{RKD-D} = 25, \lambda_{RKD-A} = 50$, thêm cross-entropy.

Phương pháp	CIFAR-100	Tiny ImageNet
Baseline (student tự train)	71.26	54.45
RKD-D	72.27	54.97
RKD-DA	72.97	56.36
HKD (Hinton)	74.26	57.65
HKD + RKD-DA	74.66	58.15
FitNet	70.81	55.59
FitNet + RKD-DA	72.98	55.54
Attention	72.68	55.51
Attention + RKD-DA	73.53	56.55
Teacher	77.76	61.55

Đáng chú ý: ở phân loại, RKD *một mình* kém HKD; lợi ích nằm ở chỗ **kết hợp** với IKD.

4.3 Few-shot (Omniglot, miniImageNet; prototypical network, self-distillation)

Omniglot: RKD-DA 98.64 / 99.64 / 95.52 / 98.67 (5-way 1-shot, 5-shot, 20-way 1-shot, 5-shot) so với teacher 98.55 / 99.56 / 95.11 / 98.68. miniImageNet (5-way): RKD-DA 50.02 và 68.16 so với teacher 49.1 và 66.87; FitNet 50.38 / 68.08; Attention rất kém (34.67 / 46.21). Cải thiện nhỏ nhưng nhất quán so với teacher.

5. Phân tích / ablation

- **Không dùng l2 normalization tốt hơn** cho RKD (nhóm ô "X" trong Table 1 hầu như luôn cao hơn "O"), vì embedding được dùng cả không gian thay vì mặt cầu đơn vị. Ngược lại các phương pháp khác (kể cả DarkRank) tụt mạnh khi bỏ l2 norm (ví dụ DarkRank ResNet18-128 trên Cars: 52.92 so với 77.00).
- **D vs A vs DA:** ba biến thể gần nhau; không biến thể nào thắng áp đảo. Paper nói angle thường hội tụ nhanh/tốt hơn nhưng số liệu Table 1 không phân biệt rõ.
- **RKD là chuyên biệt hóa domain:** student huấn luyện bằng RKD trên Cars 196 có recall@1 trên miền khác (CUB, Stanford Dog, CIFAR-100) thấp hơn nhiều, trong khi teacher (triplet) gần với đặc trưng pretrain. Tức RKD thích nghi mạnh với miền huấn luyện đổi lại giảm khả năng khái quát (Figure 3, không có số liệu chi tiết trong text).
- **Vì sao student vượt teacher** (giả thuyết của tác giả): nhãn liên tục (khoảng cách, góc) mang thông tin mà nhãn nhị phân positive/negative của triplet không có, giống cách soft label của Hinton mang quan hệ giữa các lớp.
- **Bổ trợ với IKD:** Table 4 cho thấy cộng RKD vào HKD/FitNet/Attention hầu hết đều cải thiện.

6. Giới hạn

Do tác giả nêu:

- RKD không phù hợp để dùng một mình ở tầng mà giá trị đầu ra quan trọng (softmax).
- Thế bậc cao tốn kém hơn; paper chỉ thử bậc 2 và 3.
- Đánh đổi khả năng khái quát sang miền khác (Section 4.1.4).

Nhận xét:

- Chi phí angle-wise là $O(B^3)$ theo batch size (mọi bộ ba), điều này làm giảm khả năng mở rộng khi batch lớn hoặc khi mỗi "mẫu" là một chuỗi dài.
- Toàn bộ thí nghiệm là vision cỡ nhỏ (ResNet, CIFAR, CUB); không có thí nghiệm với chuỗi/ngôn ngữ/mô hình sinh.
- Cải thiện trên phân loại khá nhỏ (khoảng 1 đến 2 điểm khi dùng một mình), và hyperparameter (λ) phải chỉnh theo từng task (1/2, 25/50, 50/100).
- Paper không thử so sánh với các batch kích thước khác nhau; chất lượng ước lượng μ phụ thuộc batch.

7. Liên hệ với các bài khác

- **02 Hinton KD / 03 FitNets / 04 Attention transfer:** đều là IKD theo cách phân loại của bài này; RKD được thiết kế để cộng thêm vào chúng (Table 4).
- **05 KD survey:** RKD thuộc nhóm "relation-based knowledge" trong cách phân loại thường gặp (suy luận: nhãn này là của survey, không phải của bài này).
- **06 Born-again / 07 Deep mutual learning:** self-distillation của RKD (Table 2) cùng tinh thần với born-again, nhưng ở đây dùng quan hệ thay vì xác suất.
- **13 CRD:** cũng khai thác thông tin giữa các mẫu (dùng negative samples và contrastive objective) thay vì khớp từng mẫu; có thể đọc 12 như bước nối giữa IKD và CRD.
- **14 Decoupled KD:** nhìn lại KD dựa trên logit; cho thấy IKD cổ điển vẫn mạnh nếu tách đúng thành phần.

- **18 TinyBERT / 19 MiniLM:** MiniLM dùng quan hệ giữa các vị trí (Q-K, V-V) làm mục tiêu distill, là một dạng "relational" khác (suy luận, đọc ở bài 19).

8. Tóm tắt công thức / khái niệm

Khái niệm	Ý nghĩa
IKD	KD khớp từng đầu ra riêng lẻ: $\sum_i l(f_T(x_i), f_S(x_i))$
RKD	KD khớp quan hệ: $\sum l(\psi(t_1..t_n), \psi(s_1..s_n))$
Relational potential ψ	Hàm đo quan hệ của một bộ n mẫu
ψ_D	Khoảng cách Euclid chia cho trung bình khoảng cách trong batch
ψ_A	Cosine góc tại đỉnh t_j của bộ ba (t_i, t_j, t_k)
Huber loss l_δ	Bình phương gần 0, tuyến tính xa 0
Tổng loss	$L_{task} + \lambda_{KD} L_{KD}$
Triplet loss	Ép $d(a,p) + m < d(a,n)$ cho anchor, positive, negative
Recall@K	Tỉ lệ truy vấn có ảnh cùng lớp trong top-K lân cận
Self-distillation	Student cùng kiến trúc với teacher

9. Kết luận

RKD chuyển quan hệ khoảng cách và góc giữa các mẫu thay vì khớp từng giá trị đặc trưng. Kết quả nổi bật nhất thuộc metric learning: trên Cars 196, student đạt 82.50 so với 77.17 của teacher. Khi kết hợp với KD cổ điển, cải thiện trên phân loại khiêm tốn hơn.

Link gốc: https://arxiv.org/abs/1904.05068

Bài 13

Contrastive Representation Distillation (2020)

Tác giả: Yonglong Tian, Dilip Krishnan, Phillip Isola (MIT CSAIL, Google Research).
Venue / arXiv: ICLR 2020; arXiv:1910.10699 (v3, 24/1/2022).
Nguồn: https://arxiv.org/abs/1910.10699
Chủ đề: Distill biểu diễn (representation) bằng contrastive learning: tối đa hóa cận dưới của mutual information giữa embedding teacher và student; nén mô hình, chuyển giao đa phương thức, distill ensemble.
Vị trí trong lộ trình: Đọc sau 02 (Hinton KD), 03 (FitNets), 04 (Attention transfer), 12 (Relational KD). Bài này thay "khớp từng chiều" bằng "khớp thông tin tương hỗ" ở tầng embedding gần cuối. Tiếp theo: 14 (Decoupled KD) quay lại phân tích logit KD.

> Tài liệu học tập bằng tiếng Việt, là ghi chú diễn giải, không phải bản dịch paper.

1. Bối cảnh

Hinton KD (bài 02) cực tiểu hóa KL giữa xác suất đầu ra của teacher và student. Paper chỉ ra hai điểm yếu:

1. **Giả định độc lập giữa các chiều.** Mục tiêu KD có dạng phân rã $\psi(y^S, y^T) = \sum_i \phi_i(y_i^S, y_i^T)$ (mỗi chiều tính riêng, tổng lại). Nó không trực tiếp bắt các *phụ thuộc* giữa các chiều. Tác giả so sánh với việc loss L2 khi sinh ảnh cho ảnh mờ do coi các pixel độc lập.
2. **Chỉ áp dụng được khi đầu ra là phân phối.** Khi cần chuyển giao *biểu diễn* (ví dụ từ mạng RGB sang mạng depth, hoặc sang âm thanh), KL không xác định được.

Các phương pháp tầng giữa (FitNet, Attention transfer; bài 03, 04) làm việc trên biểu diễn nhưng cũng dùng loss kiểu hồi quy từng chiều và thường đòi hỏi student/teacher có kiến trúc hoặc độ phân giải feature map tương tự.

Khái niệm cần có: **contrastive learning** = học biểu diễn sao cho các cặp "dương" (cùng nguồn) gần nhau, các cặp "âm" (khác nguồn) xa nhau. **Mutual information (MI)** $I(T; S)$ = lượng thông tin mà biến này chứa về biến kia; $I = 0$ khi độc lập. **Critic** = một hàm chấm điểm học được để phân biệt cặp dương với cặp âm.

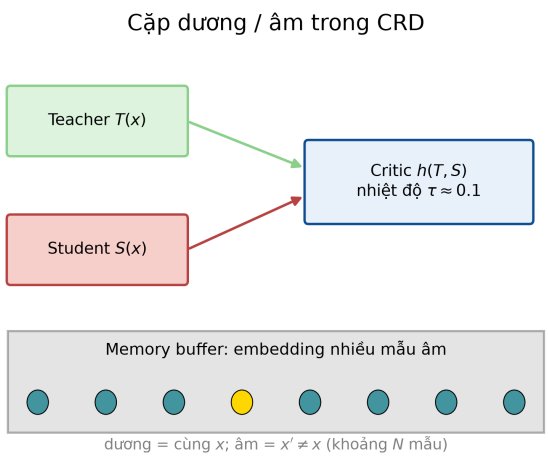
2. Ý tưởng chính

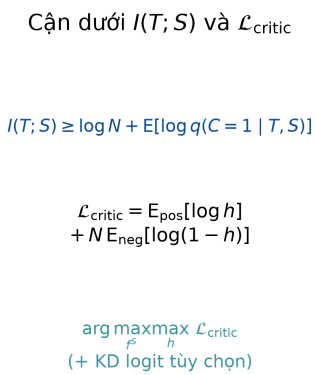
Với cùng một ảnh x_i, embedding của teacher và student phải "khớp nhau" (cặp dương); với hai ảnh khác nhau x_i, x_j thì không (cặp âm). Huấn luyện student để một bộ phân loại nhị phân phân biệt được hai trường hợp này chính là tối đa hóa một cận dưới của mutual information giữa biểu diễn của teacher và của student. Vì thế student không chỉ sao chép giá trị mà phải giữ toàn bộ cấu trúc thông tin teacher mã hóa trong embedding, kể cả các tương quan giữa chiều. Phương pháp chạy trên embedding ở tầng penultimate (ngay trước logit), nên hoạt động với số chiều teacher và student khác nhau nhờ hai phép chiếu tuyến tính.

3. Phương pháp chi tiết

3.1 Loss contrastive

Gọi $T = f^T(x)$ và $S = f^S(x)$ là embedding penultimate. Xét hai phân phối: phân phối đồng thời $p(S, T)$ (cùng một đầu vào cho hai mạng) và tích các biên $p(S)p(T)$ (hai đầu vào độc lập). Đặt biến ẩn C: $C = 1$ nếu cặp đến từ phân phối đồng thời, $C = 0$ nếu từ tích biên; lấy 1 cặp dương cho mỗi N cặp âm. Bayes cho posterior:





CRD cực đại hoá MI teacher–student bằng critic dương/âm và memory buffer; cận dưới InfoNCE với nhiệt độ τ (minh hoạ, tự tính)

$$q(C=1 \mid T,S) = \frac{p(T,S)}{p(T,S) + N\,p(T)p(S)}$$

Lấy log rồi kỳ vọng dưới cặp dương, ta có cận dưới MI (công thức 9):

$$\boxed{I(T;S) \;\geq\; \log N + \mathbb{E}_{q(T,S|C=1)}[\log q(C=1 \mid T,S)]}$$

Vì không biết posterior thật, ta khớp một critic $h : \{\mathcal{T}, \mathcal{S}\} \to [0,1]$ bằng cực đại hóa log-likelihood nhị phân:

$$\mathcal{L}_{critic}(h) = \mathbb{E}_{q(T,S|C=1)}[\log h(T,S)] + N\,\mathbb{E}_{q(T,S|C=0)}[\log(1-h(T,S))] \tag{10}$$

Paper chứng minh (Gibbs inequality, Mục 6.2.1) critic tối ưu $h^* = q(C=1|T,S)$, và làm yếu cận dưới để tối ưu *đồng thời* student và critic:

$$\boxed{f^{S*} = \arg\max_{f^S} \max_{h}\; \mathcal{L}_{critic}(h)} \tag{17,18}$$

Do $I(T;S) \geq \log N + \mathcal{L}_{critic}(h)$ với mọi h, nên không cần critic hoàn hảo. Thực tế chọn critic:

$$h(T,S) = \frac{e^{g^T(T)' g^S(S)/\tau}}{e^{g^T(T)' g^S(S)/\tau} + \dfrac{N}{M}} \tag{19}$$

với M là kích thước tập dữ liệu, τ là temperature, g^T, g^S là phép chiếu tuyến tính đưa hai embedding về cùng chiều (paper cho biết memory bank lưu embedding 128-d ở ImageNet) rồi chuẩn hóa L2 trước khi nhân vô hướng. Dạng này lấy cảm hứng từ NCE (noise-contrastive estimation, Gutmann & Hyvärinen 2010; Wu et al. 2018).

Memory buffer: theo lý thuyết N càng lớn thì cận dưới càng chặt; để khỏi dùng batch rất lớn, tác giả (theo Wu et al. 2018) giữ một bộ nhớ lưu embedding của mọi mẫu từ các batch trước và lấy mẫu âm từ đó.

3.2 Hợp với KD và các biến thể

- Loss KD chuẩn: $L_{KD} = (1-\alpha)H(y, y^S) + \alpha\rho^2 H(\sigma(z^T/\rho), \sigma(z^S/\rho))$.
- **CRD+KD** = cộng hai loss (kết quả tốt nhất trong nhiều bảng).
- **Cross-modal** (teacher và student ở hai modality, dữ liệu ghép nhưng không nhãn): bỏ số hạng cross-entropy nhãn, dùng loss contrastive (10).
- **Ensemble:** $\mathcal{L}_{CRD\text{-}EN} = H(y, y^S) - \beta \sum_i \mathcal{L}_{critic}(T_i, S)$, tức một loss contrastive cho mỗi teacher.

Pseudo-code

```
# Khởi tạo: g_T, g_S = Linear(d_T,128), Linear(d_S,128); memory_T, memory_S: [M,128]
for x, y, idx in loader:                    # idx: chỉ số mẫu trong dataset
    with no_grad(): fT = teacher.penultimate(x)
    fS, logits = student.penultimate_and_logits(x)
    vT = normalize(g_T(fT)); vS = normalize(g_S(fS))
    # N mẫu âm: chọn ngẫu nhiên từ memory (không trùng idx; phiên bản supervised: khác lớp)
    negT = memory_T[sample_neg(idx, N)]  # [B,N,128]
    negS = memory_S[sample_neg(idx, N)]
    # critic dạng (19), cặp dương (vS,vT), cặp âm (vS,negT) và (vT,negS)
    pos = exp((vS*vT).sum(-1)/tau)
    neg = exp(bmm(negT, vS)/tau)
    h_pos = pos / (pos + N/M);  h_neg = neg / (neg + N/M)
    L_crd = -(log(h_pos).mean() + N*log(1-h_neg).mean())   # dấu: cực đại L_critic
    loss = CE(logits, y) + beta*L_crd (+ alpha*KD tùy chọn)
    update_memory(idx, vT, vS)
```

(Pseudo-code được tóm tắt từ công thức, bỏ qua chi tiết của code chính thức RepDistiller; code của tác giả: github.com/HobbitLong/RepDistiller.)

Ví dụ nhỏ

Batch có 3 ảnh A, B, C. Với anchor A, cặp dương là (student(A), teacher(A)), cặp âm là (student(A), teacher(B)) và (student(A), teacher(C)). Nếu cosine của cặp dương là 0.8 và của các cặp âm là 0.1, critic (19) với $\tau = 0.1$ cho điểm mũ $e^8 \approx 2981$ so với $e^1 \approx 2.7$: cặp dương áp đảo, loss thấp. Nếu student chưa học gì thì cả ba cosine gần nhau, critic không phân biệt được và gradient đẩy cosine dương lên, cosine âm xuống. (Ví dụ minh họa, bỏ qua hằng số N/M và chi tiết chuẩn hóa trong code gốc.)

4. Thí nghiệm và kết quả chính

Dữ liệu: CIFAR-100, ImageNet, STL-10, TinyImageNet, NYU-Depth V2. Paper so sánh với 12 phương pháp distill (KD, FitNet, AT, SP, CC, VID, RKD, PKT, AB, FT, FSP, NST).

4.1 Nén mô hình trên CIFAR-100 (trung bình 5 lần chạy)

Cùng phong cách kiến trúc (Table 1), top-1 accuracy (%):

Teacher → Student	Teacher	Student	KD	CRD	CRD+KD
WRN-40-2 → WRN-16-2	75.61	73.26	74.92	75.48	75.64
WRN-40-2 → WRN-40-1	75.61	71.98	73.54	74.14	74.38
resnet56 → resnet20	72.34	69.06	70.66	71.16	71.63
resnet110 → resnet20	74.31	69.06	70.67	71.46	71.56
resnet110 → resnet32	74.31	71.14	73.08	73.48	73.75
resnet32x4 → resnet8x4	79.42	72.50	73.33	75.51	75.46
vgg13 → vgg8	74.64	70.36	72.98	73.94	74.29

Kiến trúc khác phong cách (Table 2, trung bình 3 lần):

Teacher → Student	Student	KD	CRD	CRD+KD
vgg13 → MobileNetV2	64.6	67.37	69.73	69.94
ResNet50 → MobileNetV2	64.6	67.35	69.11	69.54
ResNet50 → vgg8	70.36	73.81	74.30	74.58
resnet32x4 → ShuffleNetV1	70.5	74.07	75.11	75.12
resnet32x4 → ShuffleNetV2	71.82	74.45	75.65	76.05
WRN-40-2 → ShuffleNetV1	70.5	74.83	76.05	76.27

Quan sát của bài báo: CRD là phương pháp duy nhất luôn vượt KD trong mọi cặp thử, và đạt "57% average relative improvement" so với KD (công thức ở chú thích trang 2: trung bình của $(Acc_{crd} - Acc_{kd})/(Acc_{kd} - Acc_{van})$, tức so với phần lợi mà KD đã mang lại, không phải 57 điểm phần trăm tuyệt đối). Các phương pháp tầng giữa (FitNet, AT) *thua cả student huấn luyện thường* trong nhiều cặp khác kiến trúc (ví dụ AT vgg13 → MobileNetV2 chỉ 59.40 so với student thường 64.6). Trong Table 1, RKD (bài 12, do các tác giả CRD chạy lại) thấp hơn KD ở cả bảy cặp, ví dụ WRN-40-2 → WRN-16-2: 73.35 so với KD 74.92. Lưu ý: ở cả hai bảng, CRD hầu như không vượt teacher (ví dụ resnet32x4 → resnet8x4: 75.51 so với teacher 79.42); chỉ có WRN-40-2 → WRN-16-2 là sát teacher (75.48/75.64 so với 75.61).

4.2 ImageNet (Table 3, error %, ResNet-34 → ResNet-18)

	Teacher	Student	AT	KD	CRD	CRD+KD
Top-1 err	26.69	30.25	29.30	29.34	28.83	28.62
Top-5 err	8.58	10.93	10.00	10.12	9.87	9.51

Khoảng cách top-1 teacher-student là 3.56; AT thu hẹp 0.95, CRD thu hẹp 1.42 (paper nói tương đương cải thiện tương đối 50%).

4.3 Transferability (Table 4)

Student WRN-16-2 (từ teacher WRN-40-2) đóng băng, huấn luyện linear classifier trên STL-10 / TinyImageNet: Student 69.7 / 33.7; KD 70.9 / 33.9; CRD 71.6 / 35.6; CRD+KD 72.2 / 35.5; Teacher 68.6 / 31.5. Tức student distill còn *chuyển giao tốt hơn teacher* (+3.6 STL-10, +4.1 TinyImageNet cho CRD+KD).

4.4 Cross-modal

- Luminance → chrominance (Figure 3): CRD hiệu quả hơn các loss khác (số liệu chỉ ở hình, text không liệt kê).
- RGB → depth (Table 5, segmentation ngữ nghĩa): Random init 56.4 pixel acc / 35.8 mIoU; KD 58.9 / 38.0; KD+AT 60.1 / 39.5; FitNet 60.8 / 40.7; CRD 61.6 / 41.8.

4.5 Ensemble

CIFAR-100: single WRN-16-2 lỗi 26.7%, ResNet20 lỗi 30.9%. CRD với 8 teacher giảm xuống 23.7% và 28.3%; CRD tốt hơn KD ở mọi số teacher thử (Figure 4).

5. Phân tích / ablation

- **InfoNCE so với loss của bài báo (Table 6):** cùng số mẫu âm, loss của bài báo (18) thắng InfoNCE ở 4 trên 5 cặp ; với mẫu âm khác lớp ($y_i \neq y_j$), loss của bài báo đạt 75.48 / 71.46 / 73.48 / 75.51 / 73.94 còn InfoNCE 75.15 / 71.39 / 73.53 / 75.22 / 73.74 (cặp thứ ba InfoNCE nhỉnh hơn). Chênh lệch nhỏ.

- **Chiến lược mẫu âm:** chọn âm khác nhãn ($y_i \neq y_j$) cao hơn chọn âm bất kỳ ($i \neq j$) 0.81% cho loss của bài báo và 0.62% cho InfoNCE, vì cách đầu tránh đẩy xa các mẫu cùng lớp.
- **Số mẫu âm N:** thử 16 đến 16384; càng nhiều càng tốt, nhưng $N = 4096$ so với $N = 16384$ chênh dưới 0.1%; paper báo cáo với 16384 và nói 4096 là đủ.
- **Temperature τ:** thử 0.02 đến 0.3; quá thấp hoặc quá cao đều xấu; dải 0.05-0.2 ổn; dùng 0.1 (0.07 cho ImageNet).
- **Chi phí:** trên ResNet-18/ImageNet thêm 260 MFLOPs (khoảng 12% của 2 GFLOPs); tốc độ 1.75 so với 1.67 epoch/giờ trên hai Titan-V; memory bank 128-d cho ImageNet khoảng 600MB.
- **Lý do tầng giữa tệ khi khác kiến trúc (giả thuyết của tác giả):** mỗi kiến trúc có "đường giải" riêng từ đầu vào đến đầu ra, nên ép bắt chước feature trung gian xung đột với inductive bias.
- **Tương quan giữa class logits (Figure 2):** CRD làm khoảng cách giữa ma trận tương quan của teacher và student nhỏ nhất so với AT và KD (chỉ định tính qua hình).

6. Giới hạn

Do tác giả nêu: nhiệt độ tối ưu thay đổi theo dataset và cần chỉnh lại; cận dưới MI chỉ chặt khi N lớn (nên cần memory buffer); với cross-modal trên depth phải dùng thêm thủ thuật contrast local/global (Hjelm et al. 2018) vì dữ liệu ít.

Nhận xét:

- Toàn bộ là phân loại ảnh cỡ nhỏ; không có chuỗi, không có mô hình sinh. Embedding "penultimate" không rõ nghĩa ở decoder token-level.
- Cần memory bank chứa embedding **mọi mẫu** (kích thước M). Với dữ liệu OCR lớn (hàng triệu trang, mỗi trang nhiều token) thì phải đổi thiết kế (mỗi mẫu = một trang hay một token?).
- Cặp dương/âm được định nghĩa ở mức *mẫu* với nhãn lớp; với chuỗi sinh, "mẫu âm khác lớp" không có ngữ nghĩa tương đương.
- Lợi thế so với KD tuyệt đối thường chỉ 0.5-2 điểm; riêng ở cặp khác kiến trúc lên đến khoảng 2.4 điểm (vgg13 → MobileNetV2: 69.73 so với 67.37).
- Kết quả của RKD trong bảng là bản chạy lại của các tác giả CRD; kết luận "RKD kém KD" ở đây có thể khác với bài 12 vì setup khác.

7. Liên hệ với các bài khác

- **02 Hinton KD:** CRD thừa nhận KD là baseline mạnh thứ hai; "KD hoạt động khá tốt, không phương pháp nào khác luôn thắng KD khi đứng một mình".
- **03 FitNets / 04 Attention transfer:** thất bại khi khác kiến trúc (Table 2) nhắc lại cảnh báo về khớp feature trung gian.
- **10 Does KD really work:** câu hỏi liệu student có thật sự khớp teacher; Figure 2 của CRD (tương quan logit) là một cách đo liên quan (suy luận về mối nối).
- **12 Relational KD:** cũng truyền thông tin giữa các mẫu; RKD khớp hình học (khoảng cách, góc) trong batch, CRD tối đa hóa MI với nhiều mẫu âm từ memory bank.
- **14 Decoupled KD:** sẽ phân rã KD thành hai phần (TCKD và NCKD) để giải thích vì sao logit KD vẫn đủ mạnh; đọc tiếp để so sánh hướng "sửa loss tầng logit" với "đổi sang biểu diễn".
- **22 DINO và 24 CLIP-KD:** các bài tự giám sát và CLIP dùng contrastive/self-distillation; CRD là điểm nối giữa distillation và representation learning (suy luận).

8. Tóm tắt công thức / khái niệm

Khái niệm	Ý nghĩa
Mutual information $I(T;S)$	Thông tin chung giữa embedding teacher và student
Cận dưới MI	$I(T;S) \geq \log N + \mathbb{E}[\log q(C=1\|T,S)]$
Critic h	Hàm phân biệt cặp dương/âm, dạng (19)
$\mathcal{L}_{critic}$	$\mathbb{E}_{pos}[\log h] + N\,\mathbb{E}_{neg}[\log(1-h)]$
Mục tiêu CRD	$\arg\max_{f^S} \max_h \mathcal{L}_{critic}(h)$
Memory buffer	Lưu embedding mọi mẫu để lấy rất nhiều mẫu âm
τ	Temperature của critic; 0.1 (0.07 cho ImageNet)
CRD+KD	Cộng loss contrastive và loss KD
InfoNCE	Contrastive loss dạng softmax chọn 1 dương trong tập nhiễu
Linear probing	Đóng băng mạng, huấn luyện lớp tuyến tính để đo chất lượng biểu diễn

9. Kết luận

CRD dùng mục tiêu tương phản để phân biệt biểu diễn teacher của cùng mẫu với biểu diễn của mẫu khác. Trong các so sánh CIFAR-100 và ImageNet, loss biểu diễn này vượt KD; student cũng chuyển giao sang tác vụ khác tốt hơn teacher trong thí nghiệm được báo cáo.

Link gốc: https://arxiv.org/abs/1910.10699

Bài 14

Decoupled Knowledge Distillation (2022)

Tác giả: Borui Zhao, Quan Cui, Renjie Song, Yiyu Qiu, Jiajun Liang (MEGVII Technology, Waseda University, Tsinghua University).
Venue / arXiv: arXiv:2203.08679 (v2, 12/7/2022); CVPR 2022 (theo ghi chú trên trang arXiv, bản text không ghi).
Nguồn: https://arxiv.org/abs/2203.08679
Chủ đề: Phân rã loss KD của Hinton thành TCKD (lớp đích) và NCKD (các lớp không phải đích); chỉ ra KD cổ điển "ghép cặp" (coupled) làm NCKD bị nén; đề xuất DKD với hai trọng số α, β độc lập.
Vị trí trong lộ trình: Đọc sau 02 (Hinton KD), 12 (RKD), 13 (CRD), trước 15 (sequence-level KD). Bài này trả lời vì sao KD trên logit đôi khi "kém" feature-distillation và sửa đúng điểm yếu đó chỉ bằng đổi trọng số. Về sau 31 (Rethinking KL) và 30 (DistiLLM) tiếp tục phân tích dạng loss.

> Tài liệu học tập bằng tiếng Việt, là ghi chú diễn giải, không phải bản dịch paper.

1. Bối cảnh

Logit distillation = student bắt chước *logit* (đầu ra trước softmax) của teacher, thường qua KL divergence trên softmax có nhiệt độ (bài 02). **Feature distillation** = bắt chước activation tầng giữa (FitNet, AT, CRD, ReviewKD... bài 03, 04, 13).

Sau FitNet (2015), phần lớn nghiên cứu chuyển sang feature distillation vì hiệu quả cao hơn trên nhiều task; logit distillation ít được quan tâm. Nhược điểm của feature distillation: chi phí huấn luyện cao hơn (module phụ, tham số thêm, bộ nhớ). Logit distillation gần như không tốn thêm gì, nhưng kết quả thấp hơn. Tác giả nghi ngờ rằng tiềm năng của logit distillation bị hạn chế bởi "những lý do chưa biết", và họ đi mổ xẻ chính công thức KD.

2. Ý tưởng chính

Một dự đoán phân loại có thể nhìn ở hai mức: (1) nhị phân: "lớp đích" so với "tất cả lớp còn lại"; (2) đa lớp *bên trong* các lớp không phải đích (ví dụ trong ảnh con mèo, xe tải giống mèo bao nhiêu so với xe buýt). KD cổ điển, về mặt toán học, bằng đúng KL ở mức (1) cộng với KL ở mức (2) nhân với hệ số $(1 - p_t^T)$, với p_t^T là độ tự tin của teacher trên lớp đích. Hệ số này nghĩa là: *teacher càng tự tin thì thông tin "dark knowledge" giữa các lớp sai càng bị nhân với số nhỏ*, trong khi đó chính là những mẫu mà thông tin này đáng tin nhất. DKD đơn giản là bỏ cái hệ số ràng buộc đó và cho hai thành phần hai trọng số độc lập α và β.

3. Phương pháp chi tiết

3.1 Ký hiệu

Mẫu thuộc lớp t; xác suất $p_i = \exp(z_i) / \sum_j \exp(z_j)$. Định nghĩa:

- $\mathbf{b} = [p_t,\ p_{\setminus t}]$: xác suất nhị phân của lớp đích và tổng các lớp còn lại, với $p_{\setminus t} = \sum_{k \neq t} p_k$.
- $\hat{\mathbf{p}}$: phân phối *chỉ trên các lớp không phải đích*, $\hat{p}_i = \exp(z_i) / \sum_{j \neq t} \exp(z_j) = p_i / p_{\setminus t}$.

3.2 Phân rã KD

KD gốc (bỏ nhiệt độ để gọn):

$$\mathrm{KD} = \mathrm{KL}(\mathbf{p}^T \| \mathbf{p}^S) = p_t^T \log \frac{p_t^T}{p_t^S} + \sum_{i \neq t} p_i^T \log \frac{p_i^T}{p_i^S}$$

Thay $p_i = p_{\setminus t}\hat{p}_i$ ở phần tổng, tách $\log \frac{p_i^T}{p_i^S} = \log \frac{\hat{p}_i^T}{\hat{p}_i^S} + \log \frac{p_{\setminus t}^T}{p_{\setminus t}^S}$, ta được:

$$\boxed{\mathrm{KD} = \underbrace{\mathrm{KL}(\mathbf{b}^T \| \mathbf{b}^S)}_{\mathrm{TCKD}} + (1 - p_t^T)\, \underbrace{\mathrm{KL}(\hat{\mathbf{p}}^T \| \hat{\mathbf{p}}^S)}_{\mathrm{NCKD}}} \tag{5,6}$$

vì $p_{\setminus t}^T = 1 - p_t^T$. Nghĩa: **TCKD** = KL nhị phân trên "đích hay không"; **NCKD** = KL trên phân phối giữa các lớp không phải đích.

Hai "ghép cặp" (coupling) mà paper chỉ ra:

1. NCKD bị nhân với $(1 - p_t^T)$: mẫu teacher dự đoán tốt thì NCKD gần 0.
2. TCKD và NCKD không có trọng số độc lập: không thể cân bằng chúng.

3.3 DKD

$$\boxed{\mathrm{DKD} = \alpha\, \mathrm{TCKD} + \beta\, \mathrm{NCKD}} \tag{7}$$

Tức thay $(1 - p_t^T)$ bằng hằng β và thêm α cho TCKD. Loss tổng huấn luyện: cross-entropy nhãn thật (trọng số 1.0) cộng DKD (nhân T^2 như KD chuẩn).

Pseudo-code (từ Algorithm 1 của bài báo, viết lại gọn)

```
def dkd_loss(l_stu, l_tea, t_onehot, alpha, beta, T):
    p_stu, p_tea = softmax(l_stu / T), softmax(l_tea / T)
    # nhị phân: [p_target, p_non_target]
    b_stu = stack([p_stu[t_onehot], p_stu[~t_onehot].sum(-1)], -1)
    b_tea = stack([p_tea[t_onehot], p_tea[~t_onehot].sum(-1)], -1)
    tckd = kl_div(log(b_stu), b_tea)
    # phân phối trên các lớp không phải đích (loại logit đích ra, rồi softmax lại)
    pn_stu = softmax(l_stu[~t_onehot] / T)       # [N, C-1]
    pn_tea = softmax(l_tea[~t_onehot] / T)
    nckd = kl_div(log(pn_stu), pn_tea)
    # KD cổ điển tương đương: (tckd + (1 - p_tea_t) * nckd) * T**2
    return (alpha * tckd + beta * nckd) * T**2
```

Chi tiết hiện thực trong Phụ lục: CIFAR-100 dùng $T = 4$, $\alpha = 1.0$, **warm-up tuyến tính 20 epoch** (vì β lớn khiến loss ban đầu to); ImageNet dùng $T = 1$, $\alpha = 0.5$; COCO dùng $\alpha = 1.0$, $\beta = 0.25$, $T = 1$.

Ví dụ số

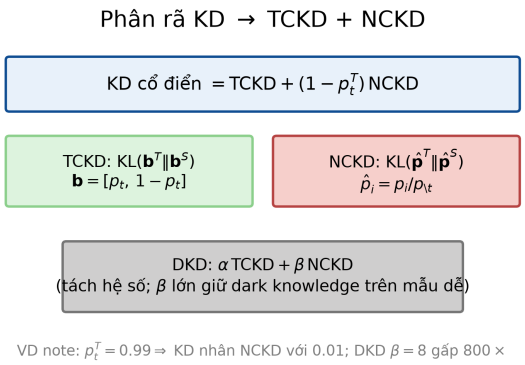
Teacher trả $p_t^T = 0.99$ cho một mẫu dễ, còn lại 0.01 chia cho hai lớp sai theo tỉ lệ 0.7/0.3 (tức $\hat{\mathbf{p}}^T = [0.7, 0.3]$). KD cổ điển có NCKD nhân với $1 - 0.99 = 0.01$: thông tin "lớp sai nào giống đích hơn" gần như bị bỏ qua. Với DKD và $\beta = 8$, NCKD nhân với 8, gấp 800 lần. Với mẫu khó ($p_t^T = 0.5$) hệ số KD là 0.5, còn DKD vẫn 8: DKD không còn "ưu tiên" mẫu khó hơn mẫu dễ ở phần NCKD. (Ví dụ minh họa.)

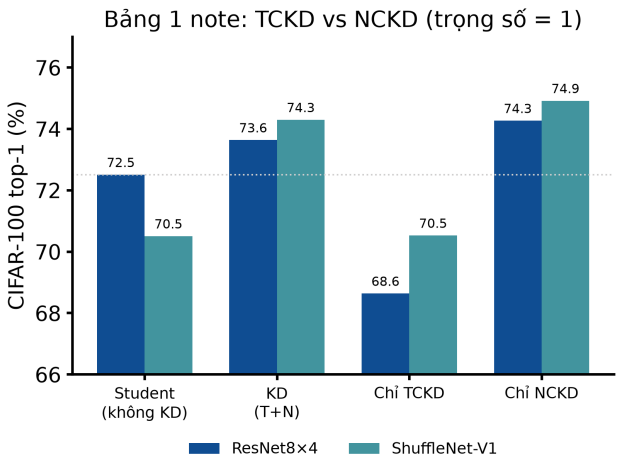
4. Thí nghiệm và kết quả chính

Dữ liệu: CIFAR-100 (ResNet, WRN, ShuffleNet, VGG, MobileNet-V2), ImageNet, MS-COCO (Faster R-CNN + FPN).

4.1 Hiệu quả riêng của TCKD và NCKD (Table 1, CIFAR-100, top-1 %, mọi trọng số = 1.0)

Bảng gốc bị mangle khi trích từ PDF; các số dưới đây được đọc theo từng khối teacher/student và khớp với phần văn bản.





DKD tách KL thành TCKD và NCKD với hệ số độc lập; Bảng 1 CIFAR-100 (note): chỉ NCKD vượt KD đầy đủ, chỉ TCKD có hại (minh hoạ, tự tính)

Teacher → Student	Student	TCKD+NCKD (=KD)	Chỉ TCKD	Chỉ NCKD
ResNet32×4 → ResNet8×4	72.50	73.63 (+1.13)	68.63 (−3.87)	74.26 (+1.76)
ResNet32×4 → ShuffleNet-V1	70.50	74.29 (+3.79)	70.52 (+0.02)	74.91 (+4.41)
WRN-40-2 → WRN-16-2	73.26	74.96 (+1.70)	70.96 (−2.30)	74.76 (+1.50)
WRN-40-2 → ShuffleNet-V1	70.50	74.92 (+4.42)	70.62 (+0.12)	75.12 (+4.62)

Kết luận của tác giả: TCKD một mình vô ích hoặc có hại; NCKD một mình bằng hoặc tốt hơn KD đầy đủ, nên NCKD (kiến thức giữa các lớp sai) mới là "dark knowledge" thực sự.

4.2 TCKD giúp khi dữ liệu "khó"

(ResNet32×4 → ResNet8×4 hoặc ShuffleNet-V1; mọi so sánh là có/không thêm TCKD lên NCKD)

- Augmentation mạnh (AutoAugment, teacher 81.29%): ResNet8×4 từ 73.82 lên 75.33 (+1.51); ShuffleNet-V1 từ 77.13 lên 77.98 (+0.85).
- Nhiễu nhãn đối xứng: 0.1: 70.99 → 70.96 (−0.03); 0.2: 67.55 → 68.03 (+0.48); 0.3: 64.62 → 65.26 (+0.64).
- ImageNet (ResNet-34 → ResNet-18): 70.71 → 71.03 (+0.32).

Diễn giải của tác giả: TCKD chuyển thông tin về độ khó của mẫu (mẫu có $p_t^T = 0.99$ dễ hơn mẫu 0.75); khi dữ liệu dễ học (CIFAR-100 có thể fit 100% trên train) thì thông tin đó không hữu ích.

4.3 NCKD bị nén trên mẫu mà teacher dự đoán tốt (Table 5)

Chia mẫu thành hai nửa theo p_t^T và chỉ áp NCKD lên một nửa (CE vẫn trên toàn bộ): áp lên nửa teacher tự tin nhất (top 50%) cho 74.23 (đọc từ bảng đã mangle, khớp mô tả trong văn bản), nửa còn lại 73.96, cả hai nửa 74.26. Tức mẫu được dự đoán tốt mang tri thức phong phú hơn, nhưng KD cổ điển lại giảm trọng số của chúng.

4.4 Ablation α, β (ResNet32×4 → ResNet8×4)

β	$1-p_t^T$ (KD)	1.0	2.0	4.0	8.0	10.0
top-1 (α=1)	73.63	74.79	75.44	75.94	76.32	76.18

α (β=8)	0.0	0.2	0.5	1.0	2.0	4.0
top-1	75.30	75.64	76.12	76.32	76.11	75.42

Tách $(1-p_t^T)$ khỏi NCKD cho +1.16 (73.63 → 74.79); tách trọng số TCKD và NCKD cho thêm +1.53 (74.79 → 76.32). TCKD cần thiết (α=0 thấp hơn 1.02) và ổn định quanh α≈1.

4.5 CIFAR-100 chính (Table 6, teacher/student cùng họ; trung bình 5 lần)

Teacher → Student	Student	KD	CRD	ReviewKD	DKD	Δ so với KD
ResNet56 → ResNet20	69.06	70.66	71.16	71.89	71.97	+1.31
ResNet110 → ResNet32	71.14	73.08	73.48	73.89	74.11	+1.03
ResNet32×4 → ResNet8×4	72.50	73.33	75.51	75.63	76.32	+2.99
WRN-40-2 → WRN-16-2	73.26	74.92	75.48	76.12	76.24	+1.32
WRN-40-2 → WRN-40-1	71.98	73.54	74.14	75.09	74.81	+1.27
VGG13 → VGG8	70.36	72.98	73.94	74.84	74.68	+1.70

Kiến trúc khác nhau (Table 7): ResNet32×4 → ShuffleNet-V1: KD 74.07, ReviewKD 77.45, DKD 76.45 (+2.38); WRN-40-2 → ShuffleNet-V1: KD 74.83, DKD 76.70 (+1.87); VGG13 → MobileNet-V2: KD 67.37, DKD 69.71 (+2.34); ResNet50 → MobileNet-V2: KD 67.35, ReviewKD 69.89, DKD 70.35 (+3.00); ResNet32×4 → ShuffleNet-V2: KD 74.45, ReviewKD 77.78, DKD 77.07 (+2.62).

4.6 ImageNet

	Teacher	Student	AT	OFD	CRD	ReviewKD	KD	KD*	DKD
R34 → R18 top-1	73.31	69.75	70.69	70.81	71.17	71.61	70.66	71.03	71.70
R34 → R18 top-5	91.42	89.07	90.01	89.98	90.13	90.51	89.88	90.05	90.41
R50 → MobileNet-V1 top-1	76.16	68.87	69.56	71.25	71.37	72.56	68.58	70.50	72.05
R50 → MobileNet-V1 top-5	92.86	88.76	89.33	90.34	90.41	91.00	88.98	89.80	91.05

(KD* là bản chạy lại của các tác giả.)

4.7 COCO (Table 10, Faster-RCNN-FPN, AP)

Pair	Student	KD	ReviewKD	DKD	DKD+ReviewKD
R101 → R18	33.26	33.97	36.75	35.05	37.01
R101 → R50	37.93	38.35	40.36	39.25	40.65
R50 → MV2	29.47	30.13	33.71	32.34	34.35

DKD vượt KD nhưng thua ReviewKD (feature); kết hợp hai cho kết quả tốt nhất.

5. Phân tích / ablation

- **Teacher lớn không hẳn là teacher tốt:** Tác giả đề xuất lời giải thích mới: khi teacher to hơn, p_t^T gần 1 hơn, hệ số $(1 - p_t^T)$ nhỏ hơn, NCKD bị nén mạnh hơn. Table 11 (student WRN-16-2, teacher WRN-28-2/40-2/16-4/28-4 với accuracy 75.45/75.61/77.51/78.60): KD cho 75.37/74.92/75.79/75.04 (không tăng theo teacher), DKD cho 75.92/76.24/76.00/76.45. Table 12 (VGG13/WRN-16-4/ResNet50): KD 74.93/75.79/75.36, DKD 75.45/76.00/76.60. Đây là giả thuyết được ủng hộ bằng thí nghiệm, chưa phải chứng minh.
- **Hiệu quả huấn luyện (Figure 2):** DKD có chi phí gần như bằng KD, không thêm tham số, trong khi feature-based thêm thời gian và bộ nhớ; DKD đạt trade-off tốt nhất (đọc từ hình).
- **Transferability (Table 13):** WRN-16-2 distill từ WRN-40-2, linear probe: baseline/KD/FitNet/CRD/ReviewKD/DKD = STL-10 69.7/70.9/70.3/71.6/72.4/72.9; Tiny-ImageNet 33.7/33.9/33.5/35.6/36.6/37.1.
- **Chỉnh β (Phụ lục A.3):** β tốt nhất tỉ lệ thuận với khoảng cách logit $z_t - z_{\text{max}}$ trung bình của teacher. Với ResNet56/WRN-40-2/ResNet32×4 gap là 5.36/7.24/8.40 và β chọn là 2.0/6.0/8.0 (Table A.2: ResNet110 6.73 → 2.0, VGG13 8.25 → 6.0, ResNet50 8.53 → 8.0). Đây là heuristic, tác giả nói quan hệ chặt chưa được khảo sát.
- **Vì sao TCKD một mình có thể hại (A.5):** với $T = 4$, gradient của TCKD kéo mạnh các logit lớp sai lên, hại độ đúng của student khi không có thông tin tương đồng giữa lớp; khi chỉ dùng TCKD và $T = 1$ kết quả gần baseline (73.24 so với 73.26), $T = 4$ cho 70.96.
- **Phân tích định tính:** t-SNE của DKD tách lớp rõ hơn KD; hiệu ma trận tương quan logit student-teacher nhỏ hơn (Figure 3, 4).

6. Giới hạn

Do tác giả nêu:

- Không vượt feature-based SOTA (ReviewKD) trên detection vì logit không mang thông tin định vị.
- Quan hệ chặt chẽ giữa β và hiệu năng chưa được nghiên cứu hết; hướng dẫn chỉnh β chỉ là heuristic.

Nhận xét:

- Toàn bộ trên phân loại ảnh có C nhỏ (100 hoặc 1000 lớp) và một nhãn mỗi mẫu; chưa có chuỗi/ngôn ngữ, nơi "lớp đích" thay đổi theo vị trí và C là vocab 30k-150k.
- β lớn (tới 8) trên CIFAR phụ thuộc cặp teacher/student và cần warm-up; độ nhạy cao nếu áp sang bài toán khác.
- Lý thuyết "bigger teacher bị nén NCKD" là giải thích hợp lý nhưng không loại trừ giải thích capacity gap (bài 08, 41).
- Cải thiện của DKD so với ReviewKD không nhất quán (ví dụ ShuffleNet: ReviewKD 77.45 > DKD 76.45).

7. Liên hệ với các bài khác

- **02 Hinton KD:** DKD là phân tích đại số trên chính loss của Hinton; "dark knowledge" = NCKD.

- **12 RKD, 13 CRD:** hai đối thủ cần vượt; trong Table 6 CRD cho 71.16/73.48/75.51... còn DKD cho 71.97/74.11/76.32...
- **08 Teacher assistant, 09 Efficacy of KD, 41 Capacity gap law:** DKD đưa ra một giải thích khác cho "teacher quá lớn không giúp ích" (NCKD bị nén chứ không chỉ capacity gap).
- **10 Does KD really work:** cùng quan tâm đến việc student có thực sự bắt chước phân phối của teacher hay không.
- **31 Rethinking KL, 30 DistiLLM, 28 MiniLLM, 29 GKD:** các bài LLM cũng xem lại trọng số/hướng KL; DKD là ví dụ sớm cho việc tách và gán lại trọng số theo từng thành phần của KL (suy luận).

8. Tóm tắt công thức / khái niệm

Khái niệm	Ý nghĩa
$\mathbf{b}$	Xác suất nhị phân [đích, không-đích]
$\hat{\mathbf{p}}$	Phân phối trên các lớp không-đích, $\hat{p}_i = p_i / p_{\setminus t}$
TCKD	$\mathrm{KL}(\mathbf{b}^T \| \mathbf{b}^S)$: truyền "độ khó" của mẫu
NCKD	$\mathrm{KL}(\hat{\mathbf{p}}^T \| \hat{\mathbf{p}}^S)$: truyền tương đồng giữa lớp sai
KD cổ điển	TCKD $+ (1 - p_t^T)$ NCKD
DKD	α TCKD $+ \beta$ NCKD
Warm-up	Tăng tuyến tính trọng số loss KD trong 20 epoch đầu (CIFAR-100)
Hướng dẫn β	Tỉ lệ với $z_t - z_{\text{max}}$ trung bình của teacher
Dark knowledge	Thông tin ngầm trong xác suất của các lớp sai

9. Kết luận

DKD tách loss KD thành thành phần cho lớp đích và thành phần quan hệ giữa các lớp còn lại. Phần sau vốn được nhân với $1 - p_t^T$, nên yếu đi khi teacher rất tự tin. Đặt trọng số độc lập cải thiện 1–3 điểm trên CIFAR-100 và đạt mức tương đương feature distillation trên ImageNet trong các thiết lập so sánh.

Link gốc: https://arxiv.org/abs/2203.08679

Bài 15

Sequence-Level Knowledge Distillation (2016)

Tác giả: Yoon Kim, Alexander M. Rush (Harvard SEAS).
Venue / arXiv: arXiv:1606.07947 (v4, 22/9/2016); EMNLP 2016 (theo ghi chú trên trang arXiv, bản text không ghi).
Nguồn: https://arxiv.org/abs/1606.07947
Chủ đề: Distillation cho mô hình sinh chuỗi (neural machine translation): word-level KD, sequence-level KD (huấn luyện student trên đầu ra beam search của teacher), sequence-level interpolation; kết hợp với weight pruning.
Vị trí trong lộ trình: Đọc sau 02 (Hinton KD) và 14 (Decoupled KD, KD cho phân loại). Bài phân biệt chưng cất chuỗi với phân loại. Nối tiếp: 28 (MiniLLM), 29 (GKD), 32 (Speculative KD), 38 (DeepSeek-R1 distillation, thực chất là Seq-KD ở quy mô lớn), 46 (Distil-Whisper dùng pseudo-label từ teacher).

> Tài liệu học tập bằng tiếng Việt, là ghi chú diễn giải, không phải bản dịch paper.

1. Bối cảnh

Neural machine translation (NMT) = dịch máy bằng mạng nơ-ron: một *encoder* đọc câu nguồn s, một *decoder* sinh câu đích $t = [t_1, \ldots, t_J]$ từng từ một, mỗi bước cho phân phối trên từ vựng V có điều kiện vào s và các từ đã sinh $t_{<j}$. Xác suất cả câu là tích:

$$p(t \mid s) = \prod_{j=1}^{J} p(t_j \mid s, t_{<j}).$$

Để đạt chất lượng tốt, NMT năm 2016 cần mô hình khổng lồ (LSTM 4 tầng × 1000 đơn vị, hoặc 16 tầng × 512), nặng với phần cứng thông thường. Các cách nén có hai nhóm: **pruning** (cắt trọng số/neuron theo độ quan trọng) và **knowledge distillation** (student bắt chước teacher; Bucila 2006, Ba & Caruana, Hinton 2015; bài 01, 02).

KD "chuẩn" (Hinton) nhìn dự đoán như một bài **phân loại nhiều lớp** độc lập trên một mẫu (x, y): student cực tiểu cross-entropy với phân phối xác suất của teacher q thay vì nhãn one-hot:

$$\mathcal{L}_{KD}(\theta; \theta_T) = -\sum_{k=1}^{|V|} q(y = k \mid x; \theta_T) \log p(y = k \mid x; \theta), \qquad \mathcal{L} = (1 - \alpha)\mathcal{L}_{NLL} + \alpha \mathcal{L}_{KD}.$$

Điểm khác của NMT: mô hình được *huấn luyện* như phân loại ở mức từ (word-level), nhưng được *dùng* để dự đoán cả chuỗi, trong đó mỗi quyết định dựa trên các quyết định trước đó (lỗi lan truyền khi test). Câu hỏi của bài báo: có nên distill ở mức chuỗi không?

2. Ý tưởng chính

Bài báo phân biệt chưng cất theo từ và theo chuỗi. Word-KD khớp phân phối token của teacher tại từng bước; Seq-KD dùng cả chuỗi do teacher sinh làm mục tiêu huấn luyện student. Seq-Inter chọn một chuỗi trong tập ứng viên của teacher dựa trên độ gần với bản dịch tham chiếu. Phân phối chuỗi $q(t \mid s)$ có không gian khả năng rất lớn; bài báo xem việc thay phân phối bằng mode là một phép xấp xỉ. Student học theo mode có phân phối nhọn hơn, nên greedy decoding gần đủ tốt trong thí nghiệm và nhanh hơn beam search. Các cách này được đánh giá trên dịch máy và khi kết hợp với pruning.

3. Phương pháp chi tiết

3.1 Word-level KD (cơ sở so sánh)

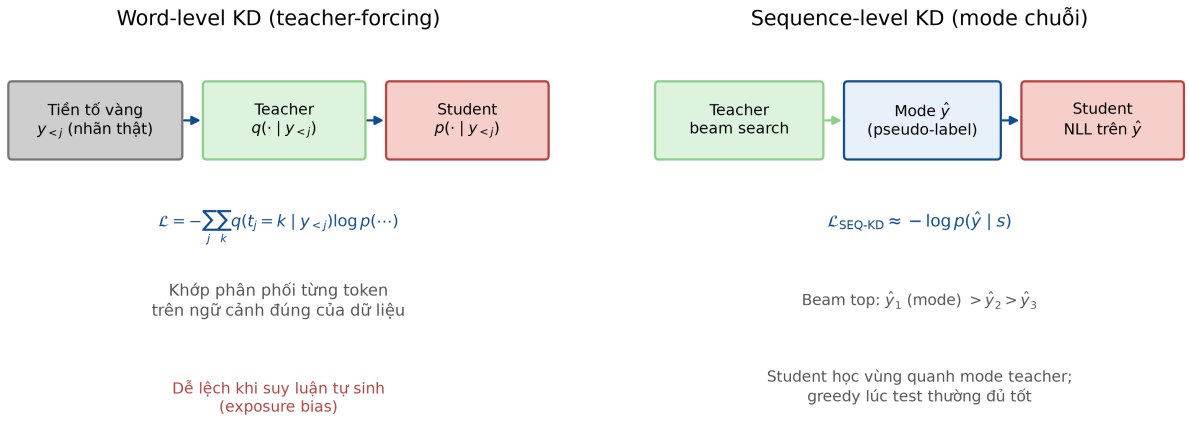
Áp Hinton KD tại mỗi vị trí j của câu đích thật:

$$\mathcal{L}_{WORD\text{-}KD} = -\sum_{j=1}^{J}\sum_{k=1}^{|V|} q(t_j = k \mid s, t_{<j}) \log p(t_j = k \mid s, t_{<j}).$$

Ngữ cảnh $t_{<j}$ là tiền tố của câu *thật* (teacher-forcing), teacher cho phân phối từ đó. Có thể trộn với NLL nhãn thật (tối ưu $\alpha = 0.5$; thử $\{0.5, 0.9\}$). Nhiệt độ $\tau \in \{1, 1.5, 2\}$ đã thử và $\tau = 1$ tốt nhất.

3.2 Sequence-level KD (Seq-KD)

NLL ở mức chuỗi với nhãn one-hot bằng đúng NLL ở mức từ ($\mathcal{L}_{SEQ\text{-}NLL} = \mathcal{L}_{WORD\text{-}NLL}$), nên với dữ liệu thật hai cách là như nhau. Nhưng nếu thay phân phối dữ liệu bằng phân phối chuỗi của teacher:



Word-level KD khớp phân phối token trên tiền tố vàng; Seq-KD thay bằng NLL trên mode chuỗi $\hat{y}$ từ beam search của teacher (minh hoạ, tự tính)

$$\mathcal{L}_{SEQ\text{-}KD} = -\sum_{t \in \mathcal{T}} q(t \mid s) \log p(t \mid s),$$

tổng này có số mũ số hạng nên không tính được. Xấp xỉ đơn giản nhất: thay q bằng mode của nó, $q(t \mid s) \approx \mathbb{1}\{t = \arg\max_t q(t \mid s)\}$, và tìm mode xấp xỉ bằng beam search:

$$\boxed{\mathcal{L}_{SEQ\text{-}KD} \approx -\log p(t = \hat{y} \mid s)} \qquad \hat{y} = \text{beam search của teacher.}$$

Số liệu hỗ trợ giả định "phần lớn khối lượng nằm trong một chuỗi": với beam 1, $q(\hat{y} \mid s)$ trung bình chiếm 1.3% (De→En) và 2.3% (Th→En); với beam 5 là 2.8% và 3.8% (chú thích 5). Khá nhỏ, nhưng tác giả vẫn thấy mode hiệu quả.

3.3 Sequence-level interpolation (Seq-Inter)

Muốn đưa dữ liệu thật y vào lại: $\mathcal{L} = (1-\alpha)\mathcal{L}_{SEQ\text{-}NLL} + \alpha\mathcal{L}_{SEQ\text{-}KD}$. Huấn luyện trên cả y lẫn $\hat{y}$ nhân đôi dữ liệu và hai chuỗi thường khác nhau nhiều. Thay vào đó (lấy cảm hứng từ *local updating* của Liang 2006): chọn một chuỗi trong K-best của teacher vừa có xác suất cao vừa gần y:

$$\tilde{y} = \arg\max_{t \in \mathcal{T}} \ \mathrm{sim}(t, y)\, q(t \mid s) \ \approx \ \arg\max_{t \in \mathcal{T}_K} \mathrm{sim}(t, y)$$

với sim = smoothed sentence-level BLEU (Chen & Cherry 2014); thực nghiệm dùng beam $K = 35$. Cơ sở sinh dữ liệu (generative story): có chuỗi thật $t \sim D$ không quan sát được, ta chỉ thấy phiên bản nhiễu $y \sim \epsilon(t)$ (bản dịch của một người dịch cụ thể). Khi đó student nên khớp hỗn hợp $D_{Seq\text{-}Inter} \sim (1-\alpha)D + \alpha\, q(t \mid s)$, và $\tilde{y}$ là xấp xỉ của argmax hỗn hợp đó.

Các phương pháp này **bổ sung cho nhau**: Seq-KD + Word-KD, fine-tune thêm trên dữ liệu Seq-Inter (học rate 0.1 từ mô hình pretrained), v.v.

Pseudo-code

```
# Seq-KD (3 bước, như tóm tắt trong paper)
teacher = train(teacher_arch, D_gold)                    # (1) teacher đã huấn luyện
D_kd = []
for (s, y) in D_gold:                                    # (2) beam search trên tập train
    hyps = teacher.beam_search(s, K=5)                   #     K=5, không thấy lợi khi lớn
↪  hơn
    D_kd.append((s, hyps[0]))                            #     lấy chuỗi điểm cao nhất
student = train(student_arch, D_kd, loss=NLL)            # (3) NLL thường trên dữ liệu mới
# tuỳ chọn: thêm Word-KD (cross-entropy tới phân phối từng từ của teacher)

# Seq-Inter
D_inter = []
for (s, y) in D_gold:
    hyps = teacher.beam_search(s, K=35)
    y_tilde = max(hyps, key=lambda h: smoothed_bleu(h, y))
    D_inter.append((s, y_tilde))
student = finetune(student, D_inter, lr=0.1)             # bắt đầu từ mô hình đã
↪  pretrained
```

Ví dụ nhỏ

Một nguồn s có teacher beam top-3: h_1 (xác suất 0.2), h_2 (0.15), h_3 (0.1), trong khi bản dịch gốc y giống h_3 nhất (BLEU cao nhất). Seq-KD huấn luyện trên h_1 (mode). Seq-Inter huấn luyện trên h_3 (trong beam, gần y nhất). Word-KD thì không dùng h_i nào mà khớp phân phối từ ở mỗi bước dọc theo y. (Ví dụ minh họa.)

4. Thí nghiệm và kết quả chính

Dữ liệu. English → German (WMT 2014, 4m câu, top-50k từ, test newstest2014) và Thai → English (IWSLT 2015, 90k câu, vocab 25k). Teacher En → De là LSTM 4×1000 (221m tham số); student 2×500 (84m) và 2×300 (49m). Teacher Th → En 2×500 (47m), student 2×100 (8m). Kiến trúc: attention kiểu Luong 2015. Đo BLEU (tokenized).

Table 1 (rút gọn; BLEU với beam $K = 1$ (greedy) và $K = 5$; Δ là gia tăng so với student baseline; $p(t = \hat{y})$ là xác suất mà mô hình gán cho đầu ra greedy)

English → German:

Mô hình	BLEU K=1	Δ K=1	BLEU K=5	Δ K=5	PPL	$p(t = \hat{y})$
Teacher 4×1000	17.7		19.5		6.7	1.3%
Teacher + Seq-Inter	19.6	+1.9	19.8	+0.3	10.4	8.2%

Mô hình	BLEU K=1	Δ K=1	BLEU K=5	Δ K=5	PPL	$p(t = \hat{y})$
Student 2×500 baseline	14.7		17.6		8.2	0.9%
Word-KD	15.4	+0.7	17.7	+0.1	8.0	1.0%
Seq-KD	18.9	+4.2	19.0	+1.4	22.7	16.9%
Seq-KD + Seq-Inter	18.9	+4.2	19.3	+1.7	15.8	7.6%
Seq-KD + Word-KD	18.7	+4.0	18.9	+1.3	10.9	4.1%
Student 2×300 baseline	14.1		16.9		10.3	0.6%
Word-KD	14.9	+0.8	17.6	+0.7	10.9	0.7%
Seq-KD	18.1	+4.0	18.1	+1.2	64.4	14.8%
Seq-KD + Seq-Inter + Word-KD	18.5	+4.4	18.9	+2.0	97.1	5.9%

Thai → English: Teacher 2×500: 14.3 (K=1) / 15.7 (K=5). Student 2×100 baseline: 10.6 / 12.7; Word-KD 11.8 / 13.6; Seq-KD 12.8 / 13.4; Seq-KD + Word-KD 13.7 / 14.2; Seq-KD + Seq-Inter + Word-KD 14.2 (+3.6) / 14.4 (+1.7).

(Bảng đầy đủ còn các dòng Baseline + Seq-Inter, Word-KD + Seq-Inter; xem paper.)

Tốc độ (Table 2, số từ nguồn dịch mỗi giây, En → De)

	GPU (Titan X)	CPU	Android (Galaxy 6)
Greedy, 4×1000 teacher	425.5	15.0	không chạy được
Greedy, 2×500	1051.3	63.6	8.8
Greedy, 2×300	1267.8	104.3	15.8
Beam 5, 4×1000	101.9	7.9	không chạy được
Beam 5, 2×500	181.9	22.1	1.9
Beam 5, 2×300	189.1	38.4	3.4

Student 2×500 greedy chạy nhanh hơn teacher beam search 10 lần trên GPU (1051.3 so với 101.9) với chất lượng tương đương (18.9 so với 19.5 BLEU).

Pruning kết hợp (Table 3, student 2×500 Seq-KD + Seq-Inter, cắt theo độ lớn trọng số rồi retrain)

Prune %	Tham số	BLEU (beam 5)	Tỉ lệ nén so với teacher
0% (teacher 4×1000)	221m	19.5	1×
0%	84m	19.3	3×
50%	42m	19.3	5×
80%	17m	19.1	13×
85%	13m	18.8	18×
90%	8m	18.5	26×

5. Phân tích / ablation

- **Seq-KD đáng kể hơn Word-KD** trên En → De (+4.2 so với +0.7 BLEU greedy), và bằng nhau ở Th → En; kết hợp hai cho thêm lợi ích trên hai trong ba student, nên chúng "trực giao": Word-KD truyền kiến thức cục bộ, Seq-KD truyền kiến thức toàn cục.

- **Khối lượng xác suất dồn về mode:** $p(t = \hat{y})$ của student Seq-KD 2×500 là 16.9% so với 0.9% ở baseline. Phân phối nhọn hơn nên argmax dễ tìm hơn, giải thích vì sao greedy đủ. Seq-Inter là điểm giữa (7.6%).
- **Perplexity không dự đoán BLEU:** baseline 2×500 có PPL 8.2, Seq-KD có PPL 22.7 (tệ hơn), nhưng BLEU Seq-KD cao hơn +4.2 (greedy). Lý do: Seq-KD mô hình hóa phân phối đã "co" quanh mode của teacher nên PPL trên tập test thật cao hơn.
- **Giả thuyết của tác giả** vì sao Seq-KD hiệu quả: student không phải "lãng phí" tham số cho toàn bộ không gian bản dịch, chỉ mô hình hóa vùng liên quan quanh mode của teacher. Đây là giả thuyết được ủng hộ bởi số liệu $p(\hat{y})$, chưa phải chứng minh.
- **Seq-Inter cải thiện cả teacher:** fine-tune teacher 4×1000 trên dữ liệu Seq-Inter: greedy 17.7 → 19.6 (+1.9), tương đương beam search của teacher gốc (19.5), cho phép giải mã nhanh hơn với cùng kích thước mô hình.
- **Beam lớn hơn 5 cho Seq-KD không giúp** (tác giả không thấy cải thiện).
- **Pruning và KD trực giao:** cắt 80% trọng số student chỉ mất thêm 0.4 BLEU so với teacher (19.5 → 19.1, nén 13×); retrain sau khi cắt là *rất quan trọng*. Embedding từ chiếm khoảng 63% tham số của 2×500 (50m trên 84m), nên pruning tập trung vào phần đó.
- **Thử và bỏ:** hồi quy hidden state trên cùng của student theo teacher như pretraining (kiểu FitNet) cho kết quả tương đương Word-KD nên không theo đuổi.

6. Giới hạn

Do tác giả nêu: nhận ra mode là xấp xỉ "kém" của một phân phối cỡ mũ (mode chỉ chiếm 1.3% khối lượng với beam 1) nhưng vẫn hiệu quả nhờ beam search đã tốt; chưa kết hợp với giảm kích thước embedding (character-level, v.v.); thí nghiệm chỉ trên NMT, dự đoán có thể áp cho các tác vụ seq2seq khác.

Nhận xét:

- Toàn bộ là LSTM + attention, tập dữ liệu dịch cỡ 90k đến 4m câu; không có mô hình Transformer, không có tác vụ nhìn-sang-chữ.
- Chỉ dùng 1 chuỗi (mode) từ teacher nên bỏ phí thông tin xác suất mềm; student học "đúng như teacher dịch" kể cả lỗi của teacher (lỗi của teacher bị khóa cứng vào dữ liệu huấn luyện), trừ khi dùng Seq-Inter.
- Phải chạy beam search của teacher trên toàn bộ tập train; đây là chi phí tính toán đáng kể khi teacher và tập dữ liệu lớn (chi phí một lần, nhưng không nhỏ).
- Seq-KD làm student quá "tự tin" (PPL tệ, $p(\hat{y})$ cao): xác suất của nó kém hiệu chuẩn, không phù hợp nếu cần xác suất để rerank hoặc ước lượng độ tin cậy (suy luận).
- Exposure bias (lệch giữa khi huấn luyện có tiền tố đúng và lúc suy luận tự sinh): bài báo không giải quyết vấn đề train/test mismatch của teacher-forcing; các bài sau (29 GKD, 28 MiniLLM) trực tiếp xử lý bằng dữ liệu on-policy.
- Đánh giá bằng BLEU, không có đánh giá của người.

7. Liên hệ với các bài khác

- **01 Ba & Caruana, 02 Hinton KD:** word-level KD ở đây là Hinton KD áp theo từng vị trí; Seq-KD là biến thể "dữ liệu giả từ teacher" (pseudo-labels).
- **14 Decoupled KD:** DKD phân tích KL ở mức từng mẫu phân loại; trong chuỗi mỗi vị trí token là một mẫu như vậy.
- **06 Born-again networks:** student được huấn luyện trên đầu ra của teacher; Seq-KD cùng tinh thần nhưng ở không gian chuỗi.
- **28 MiniLLM, 29 GKD:** hai bài này (suy luận từ tên bài, kiểm tra ở bài đó) xử lý khoảng cách giữa dữ liệu huấn luyện và dữ liệu student tự sinh, bằng KL ngược hoặc huấn luyện on-policy.

- **32 Speculative KD:** (suy luận, kiểm tra ở bài đó) liên quan đến việc chọn dữ liệu do student hoặc teacher sinh; Seq-KD làm student nhọn hơn nên là điểm tham chiếu.
- **36 Distilling step-by-step, 37 Orca, 38 DeepSeek-R1 distillation:** tất cả thực chất là Seq-KD ở quy mô lớn, student SFT trên chuỗi do teacher sinh.
- **46 Distil-Whisper:** huấn luyện student ASR trên pseudo-label sinh bởi teacher, rất gần với Seq-KD (suy luận từ tên bài; kiểm tra ở bài 46).
- **03 FitNets:** paper này thử hồi quy hidden state kiểu FitNet và thấy không lợi hơn Word-KD.

Gợi ý học tập

So sánh Word-KD, Seq-KD và kết hợp hai tín hiệu trên cùng một bài toán sinh chuỗi. Khi dùng chuỗi do teacher tạo, kiểm tra riêng chất lượng nhãn sinh và chi phí tìm kiếm; lỗi có hệ thống của teacher có thể được truyền sang student.

8. Tóm tắt công thức / khái niệm

Khái niệm	Ý nghĩa
NMT	Mô hình sinh chuỗi đích từng từ, $p(t\|s) = \prod_j p(t_j\|s, t_{<j})$
Word-KD	$-\sum_j \sum_k q(t_j = k\|\cdot) \log p(t_j = k\|\cdot)$, KD Hinton theo từng vị trí
Seq-KD	Huấn luyện student bằng NLL trên $\hat{y}$ = beam search top-1 của teacher
Mode approximation	$q(t\|s) \approx \mathbb{1}\{t = \arg\max q\}$
Seq-Inter	Dùng $\tilde{y} = \arg\max_{t \in \mathcal{T}_K} \text{sim}(t, y)$, $K = 35$, sim = smoothed BLEU
$p(t = \hat{y})$	Xác suất student gán cho đầu ra greedy; Seq-KD làm nó tăng mạnh
Greedy decoding	Chọn argmax từng bước (beam = 1)
Beam search	Giữ K giả thuyết tốt nhất mỗi bước; chi phí tăng tuyến tính theo K
Pruning theo độ lớn	Xóa trọng số có
Orthogonal	Nén bằng KD và pruning cộng được với nhau

9. Kết luận

Sequence-level KD huấn luyện student trên chuỗi do teacher sinh; phương pháp này được so sánh với khớp phân phối từng token trong dịch máy. Student nhỏ hơn ba lần, giải mã greedy nhanh hơn teacher dùng beam mười lần với mức giảm khoảng 0.6 BLEU. Khi thêm pruning 80%, mức nén là 13 lần với giảm 0.4 BLEU theo kết quả báo cáo.

Link gốc: https://arxiv.org/abs/1606.07947

Bài 16

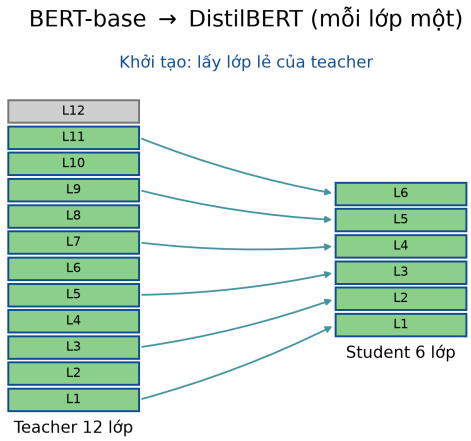
DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter (2019)

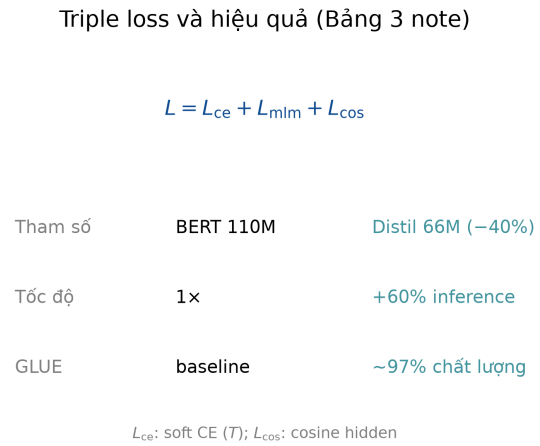
Tác giả: Victor Sanh, Lysandre Debut, Julien Chaumond, Thomas Wolf (Hugging Face).
Venue / arXiv: arXiv:1910.01108 (v4, 1/3/2020); bản text ghi "EMC^2: 5th Edition, co-located with NeurIPS'19" (workshop), không phải hội nghị chính.
Nguồn: https://arxiv.org/abs/1910.01108
Chủ đề: Distillation lúc *pre-training* cho mô hình ngôn ngữ (BERT): loss ba thành phần, khởi tạo student bằng cách lấy một layer trong hai của teacher, giảm số layer.
Vị trí trong lộ trình: Đọc sau 02 (Hinton KD) và 15 (Seq-KD). Đây là công thức "giữ một nửa số layer, khởi tạo từ teacher, thêm distill" đơn giản nhất cho Transformer; 17 (Patient KD) nói về chọn layer teacher để khớp, 18 (TinyBERT) và 19 (MiniLM) thêm distill attention và hidden state.

> Tài liệu học tập bằng tiếng Việt, là ghi chú diễn giải, không phải bản dịch paper.

1. Bối cảnh

Năm 2018-2019, NLP chuyển sang **transfer learning**: pre-train một mô hình lớn (BERT, GPT-2, RoBERTa) trên văn bản không nhãn, rồi *fine-tune* (huấn luyện tiếp) cho từng tác vụ. Mô hình có hàng trăm triệu tham số và xu hướng còn tăng, kéo theo chi phí môi trường/tính toán và khó chạy on-device.





DistilBERT khởi tạo cách lớp từ BERT-12 và triple loss $L_{ce} + L_{mlm} + L_{cos}$; −40% tham số, ~97% GLUE (Bảng 3 note) (minh hoạ, tự tính)

Trước đó, distillation trong NLP chủ yếu là **task-specific**: lấy BERT đã fine-tune cho một tác vụ, distill sang một mô hình nhỏ chỉ cho tác vụ đó (ví dụ BERT → LSTM của Tang et al. 2019). Bài này distill ở giai đoạn **pre-training** để thu được một mô hình nhỏ đa dụng, vẫn fine-tune được cho nhiều tác vụ như BERT.

Khái niệm: **masked language modeling (MLM)** là tác vụ pre-train của BERT: che một số token trong câu và dự đoán lại chúng. Lớp output trên mỗi token bị che là phân phối trên cả vocab; phần đuôi xác suất ("day" và "life" cao, "future, story, world" thấp mà vẫn hợp lý, ví dụ của bài báo) chứa "dark knowledge".

2. Ý tưởng chính

DistilBERT giảm số lớp của BERT, khởi tạo student từ các lớp được chọn của teacher, rồi chưng cất trong giai đoạn tiền huấn luyện. Mục tiêu kết hợp tín hiệu dự đoán mềm, masked language modeling và cosine giữa biểu diễn. Kết quả là một mô hình nhỏ hơn vẫn có thể được fine-tune cho nhiều tác vụ, thay vì chỉ một tác vụ cụ thể.

3. Phương pháp chi tiết

Kiến trúc student. Cùng khung BERT, số layer giảm một nửa (hidden size giữ nguyên). Lý do chọn giảm layer thay vì hidden size: tác giả thấy biến thể chiều cuối của tensor (hidden size) ảnh hưởng ít đến hiệu quả tính toán (với ngân sách tham số cố định) hơn so với số layer, vì các phép toán nền (linear, LayerNorm) đã được tối ưu cao.

Khởi tạo. Vì teacher và student cùng số chiều, student lấy trọng số từ teacher bằng cách lấy một layer trong hai (ví dụ layer 0, 2, 4, ...).

Loss. Với nhiệt độ T (cùng T cho student và teacher khi huấn luyện; khi suy luận $T = 1$):

$$p_i = \frac{\exp(z_i/T)}{\sum_j \exp(z_j/T)}, \qquad L_{ce} = \sum_i t_i \log s_i \ \text{(lấy dấu âm khi cực tiểu hóa)}$$

trong đó t_i là xác suất của teacher, s_i của student (công thức trong bài báo viết $\sum_i t_i \log(s_i)$, ngầm hiểu là cross-entropy). Loss cuối cùng là tổ hợp tuyến tính

$$\boxed{L = L_{ce} + L_{mlm} + L_{cos}}$$

(paper gọi là "triple loss", nói là "linear combination" nhưng *không nêu giá trị trọng số* trong bản text này). L_{mlm} là loss MLM trên nhãn thật; L_{cos} là **cosine embedding loss** kéo hướng của vector hidden state của student gần hướng của teacher (cosine giữa hai hidden state càng gần 1 càng tốt).

Huấn luyện. Làm theo thực hành tốt của RoBERTa (Liu et al. 2019): batch rất lớn (tối đa 4K mẫu, dùng gradient accumulation), dynamic masking, bỏ next-sentence prediction. Dữ liệu giống BERT gốc: English Wikipedia và Toronto Book Corpus. Compute: 8 GPU V100 16GB trong khoảng 90 giờ (so với RoBERTa cần 1 ngày trên 1024 GPU V100 32GB, theo paper).

Pseudo-code

```
student = copy_layers(teacher, layers=[0, 2, 4, 6, 8, 10])   # 12 -> 6 layer; bỏ token-type emb + pooler
for batch in wiki_bookcorpus_loader(batch_size=4096, dynamic_masking=True):  # không có NSP
    with no_grad(): t_logits, t_hidden = teacher(batch.masked)
    s_logits, s_hidden = student(batch.masked)
    L_ce  = soft_cross_entropy(s_logits[mask] / T, t_logits[mask] / T)        # trên token bị che
    L_mlm = cross_entropy(s_logits[mask], batch.labels[mask])
    L_cos = cosine_embedding_loss(s_hidden, t_hidden)                         # kéo hướng gần nhau
    loss = L_ce + L_mlm + L_cos                                               # paper không nêu hệ số
```

(Phạm vi áp dụng L_{cos} — hidden state cuối hay mọi layer — không nêu rõ trong bản text; pseudo-code trên là nhận xét bổ sung.)

Ví dụ nhỏ

Câu "I think this is the beginning of a beautiful [MASK]". Teacher: "day" 0.4, "life" 0.3, "future" 0.05, ... CE one-hot với nhãn thật "day" chỉ cho tín hiệu 1 token. L_{ce} với $T > 1$ cho student biết "life" cũng hợp lý và "banana" thì không. (Con số minh họa; ví dụ câu lấy từ chú thích 3 của bài báo.)

4. Thí nghiệm và kết quả chính

GLUE (dev set, trung vị 5 lần chạy, Table 1)

Model	Score	CoLA	MNLI	MRPC	QNLI	QQP	RTE	SST-2	STS-B	WNLI
ELMo	68.7	44.1	68.6	76.6	71.1	86.2	53.4	91.5	70.4	56.3
BERT-base	79.5	56.3	86.7	88.6	91.8	89.6	69.3	92.7	89.0	53.5
DistilBERT	77.0	51.3	82.2	87.5	89.2	88.5	59.9	91.3	86.9	56.3

DistilBERT luôn ngang hoặc hơn baseline ELMo (tới 19 điểm trên STS-B) và giữ 97% điểm của BERT với ít hơn 40% tham số. Các tác vụ nhỏ như RTE (69.3 → 59.9) và CoLA (56.3 → 51.3) mất nhiều nhất.

Downstream (Table 2)

Model	IMDb (acc)	SQuAD 1.1 (EM/F1, dev)
BERT-base	93.46	81.2 / 88.5
DistilBERT	92.82	77.7 / 85.8
DistilBERT (D), thêm bước distill lúc fine-tune	không có	79.1 / 86.9

IMDb kém BERT 0.6 điểm; SQuAD kém trong 3.9 điểm (nói trong text); thêm bước distill thứ hai (fine-tune trên SQuAD với teacher là BERT đã fine-tune SQuAD) thu hẹp khoảng cách còn trong vòng 3 điểm. Lưu ý: văn bản nói "79.8 F1 và 70.4 EM" cho trường hợp này, không khớp với bảng (79.1 EM / 86.9 F1); đây là bất nhất nội bộ trong bài báo, chưa rõ số nào đúng.

Kích thước và tốc độ (Table 3)

Model	Tham số (triệu)	Thời gian suy luận STS-B dev, CPU, batch 1 (giây)
ELMo	180	895
BERT-base	110	668
DistilBERT	66	410

Nhỏ hơn 40%, nhanh hơn 60% (theo paper; 410/668 = 0.61 thời gian, tức nhanh hơn khoảng 1.6 lần). Trên iPhone 7 Plus (không tính tokenization) cho QA, DistilBERT nhanh hơn 71% so với BERT-base, mô hình nặng 207 MB (có thể giảm thêm bằng quantization).

5. Phân tích / ablation

Table 4, thay đổi GLUE macro-score so với mô hình đầy đủ (triple loss + khởi tạo từ teacher):

Biến thể	Δ macro-score
Bỏ L_{ce}	−2.96
Bỏ L_{cos}	−1.46
Bỏ L_{mlm}	−0.31
Giữ triple loss nhưng khởi tạo ngẫu nhiên	−3.69

(Bảng gốc dùng ký hiệu $\emptyset$ đánh dấu thành phần bị bỏ; thứ tự dòng và giá trị khớp với câu "removing MLM loss has little impact while the two distillation losses account for a large portion".)

Kết luận của bài báo: (1) khởi tạo từ teacher là thành phần quan trọng nhất (−3.69 khi bỏ); (2) cả hai loss distillation đều quan trọng, L_{ce} nhiều hơn L_{cos}; (3) loss MLM gần như không cần khi đã có hai loss kia.

So với công trình khác (Related work): Turc et al. 2019 tiền huấn luyện student bằng mục tiêu gốc rồi mới distill lúc fine-tune; Tsai et al. 2019 pre-train đa ngôn ngữ chỉ bằng distillation; DistilBERT khẳng định rằng tận dụng teacher qua khởi tạo và loss phụ cho lợi ích rõ rệt. Pruning head attention (Michel et al. 2019) và quantization được xem là trực giao.

6. Giới hạn

Do tác giả nêu: paper là báo cáo workshop ngắn; ghi nhận pruning/quantization chưa kết hợp (nói trực giao); chưa so với nhiều biến thể khác.

Nhận xét:

- Không nêu trọng số của ba loss, nhiệt độ T cụ thể, hay phạm vi áp L_{cos} trong bản text; không tái lập được chỉ từ paper.
- Chỉ thử một cấu hình (6 layer, hidden size cố định); không quét độ sâu, không so với "bỏ layer rồi fine-tune không distill" (baseline đó chỉ gián tiếp qua ablation khởi tạo ngẫu nhiên).
- Khoảng cách trên tác vụ nhỏ (RTE −9.4, CoLA −5.0) cho thấy "97%" là trung bình bị che bởi các tác vụ dễ.
- Khởi tạo "một layer trong hai" chỉ khả thi khi student có cùng hidden size; student khác chiều không thể làm vậy.
- Tốc độ đo trên CPU batch 1 cho STS-B, chỉ là một tác vụ ngắn; không có đo độ trễ cho chuỗi dài.

7. Liên hệ với các bài khác

- **02 Hinton KD:** L_{ce} chính là loss Hinton với nhiệt độ.
- **03 FitNets, 04 Attention transfer:** L_{cos} là loss khớp hidden state đơn giản (chỉ khớp hướng).
- **17 Patient KD BERT:** chọn tập layer teacher cụ thể để khớp, thay cho "layer chẵn" cố định.
- **18 TinyBERT, 19 MiniLM:** distill thêm attention map/value-relation và hidden state, thường giảm cả chiều.
- **08 Teacher assistant, 41 Capacity gap law:** DistilBERT chỉ giảm một nửa nên khoảng cách nhỏ; với giảm lớn hơn cần các bài này.

Gợi ý học tập

Khi đánh giá một student giảm độ sâu, so sánh khởi tạo từ các lớp teacher với khởi tạo ngẫu nhiên và giữ lịch huấn luyện nhất quán. Sau đó mới đánh giá riêng đóng góp của từng thành phần loss.

8. Tóm tắt công thức / khái niệm

Khái niệm	Ý nghĩa
Pre-training distillation	Distill lúc pre-train, student đa dụng
Task-specific distillation	Distill lúc fine-tune cho một tác vụ
Student	BERT 6 layer (một nửa), bỏ token-type embedding và pooler
Khởi tạo	Lấy một layer trong hai của teacher
L_{ce}	Cross-entropy tới xác suất mềm của teacher, nhiệt độ T
L_{mlm}	Loss masked language modeling trên nhãn thật
L_{cos}	Cosine embedding loss giữa hidden state student và teacher
Triple loss	$L_{ce} + L_{mlm} + L_{cos}$ (hệ số không nêu trong bản text)
Kết quả chính	−40% tham số, +60% tốc độ, 97% GLUE

9. Kết luận

DistilBERT kết hợp giảm một nửa số lớp, khởi tạo từ teacher và ba thành phần loss: dự đoán mềm, cosine và MLM. Mô hình giữ 97% điểm GLUE của BERT và nhanh hơn 60% theo đánh giá được báo cáo.

Link gốc: https://arxiv.org/abs/1910.01108

Bài 17

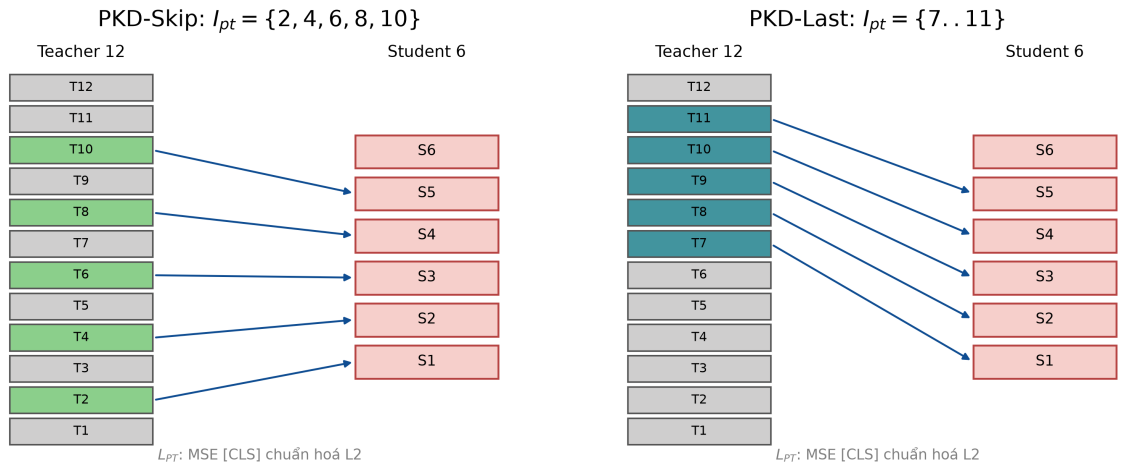
Patient Knowledge Distillation for BERT Model Compression (2019)

Tác giả: Siqi Sun, Yu Cheng, Zhe Gan, Jingjing Liu (Microsoft Dynamics 365 AI Research).
Venue / arXiv: arXiv:1908.09355 (v1, 25/8/2019); EMNLP 2019 (theo ghi chú trên trang arXiv, bản text không ghi).
Nguồn: https://arxiv.org/abs/1908.09355
Chủ đề: Distill đa tầng cho BERT: student khớp trạng thái ẩn của token `[CLS]` ở nhiều layer trung gian của teacher (PKD-Skip và PKD-Last), kèm KD ở lớp cuối.
Vị trí trong lộ trình: Đọc sau 03 (FitNets: khớp tầng giữa), 04 (Attention transfer), 16 (DistilBERT). Bài này đặt câu hỏi "student nên bắt chước những layer nào của teacher" và trả lời bằng hai kiểu ánh xạ layer. Tiếp theo: 18 (TinyBERT) làm tổng quát hóa ánh xạ layer cộng thêm khớp attention; 19 (MiniLM) chỉ khớp layer cuối.

> Tài liệu học tập bằng tiếng Việt, là ghi chú diễn giải, không phải bản dịch paper.

1. Bối cảnh

BERT-Base có 12 layer Transformer, 110 triệu tham số; BERT-Large 24 layer, 330 triệu. Huấn luyện cần nhiều ngày trên TPU, và fine-tune một epoch có thể mất hàng giờ. Paper này tự nhận là nỗ lực đầu tiên nén BERT (ghi chú: DistilBERT, bài 16, ra sau khoảng hai tháng và đi theo hướng pre-training).



Patient-KD khớp [CLS] nhiều tầng: PKD-Skip $\{2, 4, 6, 8, 10\}$ vs PKD-Last $\{7..11\}$ cho student 6 lớp (minh hoạ, tự tính)

Vanilla KD (Hinton, bài 02) trên BERT chỉ dùng **đầu ra lớp cuối** của teacher (logit/xác suất mềm) cộng nhãn thật:

$$L_{KD} = (1 - \alpha) L^s_{CE} + \alpha L_{DS}$$

với L_{DS} là cross-entropy tới xác suất mềm của teacher (có nhiệt độ T). Vấn đề tác giả quan sát: student học bằng vanilla KD **bão hòa nhanh trên dev set** (Figure 2: accuracy train tiếp tục tăng, dev ngừng), có thể do

overfit sớm trong quá trình distill. Nói cách khác, signal ở lớp cuối quá "hẹp" để student tiếp tục học tốt khi huấn luyện lâu hơn.

Khái niệm cần có: `[CLS]` là token đặc biệt đứng đầu mỗi input của BERT; hidden state của nó ở lớp cuối được dùng cho phân loại câu. **Layer mapping** = quy tắc nói "layer j của student bắt chước layer nào của teacher".

2. Ý tưởng chính

Dạy student "kiên nhẫn" (*patient*): thay vì chỉ học từ lớp cuối của teacher, student cũng khớp hidden state của token `[CLS]` ở một số layer trung gian. Hai cách chọn layer của teacher: **PKD-Last** (học từ k layer cuối, giả định layer trên chứa nhiều thông tin nhất) và **PKD-Skip** (học từ mỗi k layer, giả định cả layer thấp cũng quan trọng). Thêm một loss mới (MSE giữa hidden state đã chuẩn hóa) cạnh loss KD cũ. Kết quả: student 6 layer đạt gần teacher 12 layer ở nhiều tác vụ.

3. Phương pháp chi tiết

Teacher. BERT-Base (12 layer) fine-tune cho từng tác vụ; xác suất mềm với nhiệt độ T:

$$\hat{y}_i = P^t(y_i|x_i) = \text{softmax}\left(\frac{W \cdot \text{BERT}_{12}(x_i;\hat{\theta}_t)}{T}\right)$$

Student. BERT_k ($k \in \{3, 6\}$) khởi tạo bằng k layer đầu tiên của BERT-Base pre-trained (khác DistilBERT là lấy xen kẽ).

Chỉ dùng `[CLS]`. Khớp hidden state của mọi token đắt và nhiễu; trong BERT gốc, dự đoán chỉ dùng `[CLS]` ở lớp cuối, nên student chỉ cần bắt chước `[CLS]` ở các layer trung gian:

$$h_i = [h_{i,1}, \ldots, h_{i,k}] = \text{BERT}_k(x_i) \in \mathbb{R}^{k\times d}$$

Tập layer teacher được khớp I_{pt} (với student 6 layer, $k = 5$ vì layer cuối đã được cover bởi L_{DS}): với student n layer thì luôn $k = n - 1$.

- **PKD-Skip:** $I_{pt} = \{2, 4, 6, 8, 10\}$.
- **PKD-Last:** $I_{pt} = \{7, 8, 9, 10, 11\}$.

(Student layer j ghép với teacher layer $I_{pt}(j)$, $j = 1..M$ với M = số layer student được khớp.)

Loss khớp tầng giữa: MSE giữa hidden state đã chuẩn hóa L2 (tức khoảng cách hướng, tương tự cosine):

$$\boxed{L_{PT} = \sum_{i=1}^{N}\sum_{j=1}^{M}\left\|\frac{h^s_{i,j}}{\|h^s_{i,j}\|_2} - \frac{h^t_{i,I_{pt}(j)}}{\|h^t_{i,I_{pt}(j)}\|_2}\right\|_2^2} \tag{7}$$

Loss tổng:

$$\boxed{L_{PKD} = (1-\alpha)L^s_{CE} + \alpha L_{DS} + \beta L_{PT}} \tag{8}$$

α cân bằng CE nhãn thật và KD logit; β cân bằng loss khớp tầng giữa.

Pseudo-code

```
# teacher: BERT-12 đã fine-tune; student: BERT-6 khởi tạo từ 6 layer đầu của BERT-Base
I_pt = [2, 4, 6, 8, 10]                    # PKD-Skip (PKD-Last: [7, 8, 9, 10, 11])
```

```
for x, y in task_loader:
    with no_grad(): t_logits, t_cls = teacher(x, return_cls_per_layer=True)   # t_cls[l]:
    ↪ [B, d]
    s_logits, s_cls = student(x, return_cls_per_layer=True)                   # s_cls[j],
↪ j=1..5
    L_ce = cross_entropy(s_logits, y)
    L_ds = soft_cross_entropy(s_logits / T, t_logits / T)
    L_pt = sum(((normalize(s_cls[j]) - normalize(t_cls[I_pt[j-1]]))**2).sum(-1).mean()
               for j in range(1, 6))
    loss = (1 - alpha) * L_ce + alpha * L_ds + beta * L_pt
```

(Pseudo-code được viết từ công thức 7 và 8; chuẩn hóa mean/sum có thể khác code gốc.)

Ví dụ nhỏ

Student 6 layer, teacher 12 layer. PKD-Skip ghép student layer 1 → teacher layer 2, 2 → 4, 3 → 6, 4 → 8, 5 → 10; layer 6 của student ghép với lớp cuối của teacher thông qua L_{DS}. PKD-Last ghép 1 → 7, 2 → 8, 3 → 9, 4 → 10, 5 → 11. Với layer student j có `[CLS]` $= (3, 4)$ và teacher có `[CLS]` $= (6, 8)$: sau chuẩn hóa cả hai thành $(0.6, 0.8)$ nên $L_{PT} = 0$: loss chỉ phạt sai hướng, không phạt sai độ lớn. (Ví dụ minh họa.)

Huấn luyện: batch 32, 4 epoch, lr $\in \{5e\text{-}5, 2e\text{-}5, 1e\text{-}5\}$; vanilla KD quét $T \in \{5, 10, 20\}$, $\alpha \in \{0.2, 0.5, 0.7\}$; PKD cố định α, T theo vanilla KD tốt nhất và chỉ quét $\beta \in \{10, 100, 500, 1000\}$ cùng lr.

4. Thí nghiệm và kết quả chính

Bốn loại tác vụ, bảy tập: SST-2 (cảm xúc), MRPC và QQP (paraphrase), MNLI-m/mm, QNLI, RTE (suy luận), và RACE (đọc hiểu). Teacher là BERT-Base tự fine-tune. So sánh: **FT** (fine-tune trực tiếp student, không KD), **KD** (vanilla), **PKD** (Skip).

GLUE test server (Table 1)

Model	SST-2	MRPC (F1/Acc)	QQP (F1/Acc)	MNLI-m	MNLI-mm	QNLI	RTE
BERT12 (Google)	93.5	88.9/84.8	71.2/89.2	84.6	83.4	90.5	66.4
BERT12 (Teacher, tự làm)	94.3	89.2/85.2	70.9/89.0	83.7	82.8	90.4	69.1
BERT6-FT	90.7	85.9/80.2	69.2/88.2	80.4	79.7	86.7	63.6
BERT6-KD	91.5	86.2/80.6	70.1/88.8	80.2	79.8	88.3	64.7
BERT6-PKD	92.0	85.0/79.9	70.7/88.9	81.5	81.0	89.0	65.5
BERT3-FT	86.4	80.5/72.6	65.8/86.9	74.8	74.3	84.3	55.2
BERT3-KD	86.9	79.5/71.1	67.3/87.6	75.4	74.8	84.0	56.2
BERT3-PKD	87.5	80.7/72.5	68.1/87.8	76.7	76.3	84.7	58.2

(Số mẫu train: SST-2 67k, MRPC 3.7k, QQP 364k, MNLI 393k, QNLI 105k, RTE 2.5k.)

Điểm chính của tác giả: PKD tốt nhất trên hầu hết tác vụ, trừ MRPC. Student 6 layer gần teacher ở 5 trên 7 tác vụ (chênh so với BERT-Base: SST-2 −2.3%, QQP −0.1%, MNLI-m −2.2%, MNLI-mm −1.8%, QNLI −1.4%); cả năm tác vụ này có hơn 60k mẫu train, nên tác giả nhận xét phương pháp tốt hơn khi có nhiều dữ liệu. Với QQP, student 3 layer vẫn gần teacher.

RACE (đọc hiểu, test, Table 3)

Model	RACE	RACE-M	RACE-H
BERT12 (Leaderboard)	65.00	71.70	62.30
BERT12 (Teacher)	65.30	71.17	62.89
BERT6-FT	54.32	61.07	51.54
BERT6-KD	58.74	64.69	56.29
BERT6-PKD-Skip	60.34	66.57	57.78

Vanilla KD hơn FT 4.42%, PKD thêm 1.6%. Kết quả BERT3 không báo cáo vì khoảng cách teacher quá lớn.

PKD-Last so với PKD-Skip (Table 2, BERT6)

	SST-2	MRPC	QQP	MNLI-m	MNLI-mm	QNLI	RTE
PKD-Last	91.9	85.1/79.5	70.5/88.9	80.9	81.0	88.2	65.0
PKD-Skip	92.0	85.0/79.9	70.7/88.9	81.5	81.0	89.0	65.5

Hiệu quả (Table 4, QNLI train 105k, Titan RTX, batch 128, độ dài 128, FP16)

Layer	Tham số (Emb)	Tham số (Trm)	Tổng	Thời gian suy luận
3	23.8M	21.3M	45.7M (2.40×)	27.35s (3.73×)
6	23.8M	42.5M	67.0M (1.64×)	52.51s (1.94×)
12	23.8M	85.1M	109M (1×)	101.89s (1×)

Tốc độ gần tuyến tính theo số layer; embedding (24M, vocab 30k × 768) chia sẻ nên tỉ lệ nén tổng thể nhỏ hơn tỉ lệ số layer.

5. Phân tích / ablation

- **Bão hòa trên dev (Figure 2):** student vanilla KD ngừng cải thiện sớm trên QNLI và MNLI; PKD tiếp tục tăng trên dev rồi mới bão hòa muộn hơn. Đây là bằng chứng (đọc từ biểu đồ) cho giả thuyết của tác giả rằng tín hiệu từ tầng giữa giúp tổng quát hóa; giả thuyết "overfitting khi distill" chưa được kiểm tra riêng.
- **Skip so với Last:** Skip nhỉnh hơn (ví dụ MNLI-m 81.5 so với 80.9, QNLI 89.0 so với 88.2) nhưng cả hai đều thắng vanilla KD. Giải thích của tác giả (không kiểm chứng): Skip bắt được biểu diễn đa dạng từ mức thấp tới cao; Last bắt thông tin khá đồng nhất.
- **MRPC:** PKD kém KD trên test (85.0/79.9 so với 86.2/80.6), tác giả giải thích bằng overfit dev trên tập nhỏ; chạy lại ba lần, mean dev accuracy: FT 82.23, KD 82.84, PKD 83.46. RTE (2.5k mẫu) cũng cho khoảng cách lớn so với teacher.
- **Teacher tốt hơn có giúp không? (Table 5):** teacher BERT-Large (BERT24) so với BERT-Base: student BERT6[Base]-KD từ teacher BERT12 và BERT24 không khác nhiều (#1 so với #2: SST-2 91.5 so với 91.2; MNLI-m 79.7 so với 80.2; RTE 64.7 so với 65.7). Student BERT6[Large] (layer cỡ BERT-Large) kém hơn nhiều (#3: SST-2 89.6, MRPC 79.0/70.0, MNLI-m 75.3, RTE 53.7), dù tham số nhiều hơn 1.6 lần. Giải thích của tác giả: tỉ lệ nén 4:1 (24:6) cao hơn 2:1, và student chỉ khởi tạo từ 6 layer đầu của BERT24 (chưa tiền huấn luyện từ đầu nên nhận layer thấp chưa phản ánh đặc trưng cao). PKD vẫn tốt hơn vanilla KD trong cấu hình này (#4 so với #3 trên hầu hết tác vụ, ví dụ MNLI-m 77.2 so với 75.3), nên tác giả kết luận PKD không phụ thuộc vào chọn teacher.
- **Chỉ `[CLS]` hay mọi token:** tác giả chọn `[CLS]` vì rẻ và ít nhiễu, nhưng bài báo không có ablation so sánh với khớp mọi token.

6. Giới hạn

Do tác giả nêu: khởi tạo student chỉ từ layer đầu của teacher có thể không khớp (mismatch), cần tiền huấn luyện student từ đầu (hướng tương lai); chưa thử loss khoảng cách phức tạp hơn; chưa thử multi-task hay meta-learning; MRPC và RTE (dữ liệu nhỏ) cho kết quả kém hơn.

Nhận xét:

- Distill sau khi teacher đã fine-tune cho từng tác vụ (task-specific); mỗi tác vụ phải chạy lại teacher và student. Không phải distill pre-training như bài 16.
- Chỉ ánh xạ layer theo quy tắc cố định (Skip hoặc Last); không có cách tự động chọn ánh xạ, và không có ablation về β (cố định α, T từ KD thường rồi chỉ quét β).
- Cải thiện của PKD so với KD thường nhỏ (khoảng 0.5 đến 1.5 điểm), lại phụ thuộc quét siêu tham số rộng (β từ 10 đến 1000) nên chưa chắc ổn định.
- Hidden state phải cùng chiều; chỉ khớp hướng (chuẩn hóa L2) nên bỏ qua độ lớn (norm), vốn có thể mang thông tin trong Transformer (suy luận).
- Khởi tạo student từ các layer đầu của BERT-Base (DistilBERT, bài 16, lấy xen kẽ) là một lựa chọn chưa được so với phương án khác; hai bài báo không so trực tiếp.
- Chỉ đánh giá trên phân loại câu; `[CLS]` không có ý nghĩa với tác vụ sinh chuỗi.

7. Liên hệ với các bài khác

- **03 FitNets, 04 Attention transfer:** cùng ý tưởng "hint" tầng giữa; PKD là phiên bản cho Transformer, chỉ khớp một vector mỗi layer (`[CLS]`), không dùng bộ chuyển đổi chiều (cùng chiều).
- **16 DistilBERT:** cùng thời gian; DistilBERT distill lúc pre-train, khởi tạo xen kẽ, cosine loss ở layer cuối. PKD distill lúc fine-tune, khởi tạo k layer đầu, khớp nhiều layer.
- **18 TinyBERT:** tổng quát hóa ý tưởng này: ánh xạ layer uniform cộng khớp attention và embedding; PKD là tiền thân.
- **19 MiniLM:** đối lập: chỉ khớp layer cuối (self-attention distribution và value relation), không cần ánh xạ layer.
- **08 Teacher assistant, 09 Efficacy of KD, 41 Capacity gap law:** thí nghiệm BERT-Large teacher (Table 5) cho thấy teacher lớn hơn không tự động giúp; student càng nhỏ so với teacher (4:1) càng khó.
- **11 Patient and consistent teacher:** chỉ trùng từ "patient"; ý nghĩa khác (theo tên bài, nói về teacher kiên nhẫn và nhất quán trong cách huấn luyện), đừng nhầm; kiểm tra ở bài 11.
- **12 RKD, 13 CRD:** các cách khớp biểu diễn tầng giữa thay thế.

8. Tóm tắt công thức / khái niệm

Khái niệm	Ý nghĩa
Patient-KD	Student học từ nhiều layer trung gian của teacher, không chỉ lớp cuối
PKD-Skip	$I_{pt} = \{2, 4, 6, 8, 10\}$ cho 12 → 6 layer
PKD-Last	$I_{pt} = \{7, 8, 9, 10, 11\}$ cho 12 → 6 layer
k	Số layer được khớp; luôn $n - 1$ cho student n layer
`[CLS]`	Token tóm tắt của BERT; PKD chỉ khớp token này
L_{PT}	MSE giữa hidden state chuẩn hóa L2 của student và teacher
L_{PKD}	$(1 - \alpha)L_{CE} + \alpha L_{DS} + \beta L_{PT}$
β	Trọng số khớp tầng giữa, quét $\{10, 100, 500, 1000\}$
Khởi tạo	k layer đầu của BERT-Base pre-trained
Kết quả	BERT6: 1.64× nhỏ hơn, 1.94× nhanh hơn; gần teacher trên 5/7 tác vụ GLUE

9. Kết luận

Patient KD khớp biểu diễn `[CLS]` tại các lớp trung gian thay vì chỉ học từ lớp cuối. Với các tác vụ có nhiều dữ liệu, PKD-Skip tiếp tục cải thiện student và đưa BERT6 đến gần BERT12 trong thiết lập của bài báo.

Link gốc: https://arxiv.org/abs/1908.09355

Bài 18

TinyBERT: Distilling BERT for Natural Language Understanding (2020)

Tác giả: Xiaoqi Jiao, Yichun Yin, Lifeng Shang, Xin Jiang, Xiao Chen, Linlin Li, Fang Wang, Qun Liu (Huawei Noah's Ark Lab, HUST). **Venue / arXiv:** arXiv:1909.10351 (bản v5, tháng 10/2020; paper được biết đến là Findings of EMNLP 2020 nhưng bản text không ghi venue). **Nguồn:** https://arxiv.org/abs/1909.10351 **Chủ đề:** Distillation theo từng tầng cho Transformer: loss trên embedding, attention, hidden state, logit; khung hai giai đoạn (general + task-specific) và data augmentation. **Vị trí trong lộ trình:** Đọc trước: 02 (Hinton KD), 03 (FitNets, "hint" ở tầng giữa), 16 (DistilBERT), 17 (Patient KD). Bài này thêm: chuẩn hóa việc distill **mọi** loại biểu diễn bên trong Transformer, kèm bảng ablation cho biết loại nào quan trọng nhất. Bài kế tiếp: 19 (MiniLM) làm ngược lại, chỉ distill tầng cuối.

> Ghi chú học tập bằng tiếng Việt, không phải bản dịch nguyên văn. Con số lấy từ paper; bảng nào bị lỗi khi trích PDF sẽ được ghi rõ.

1. Bối cảnh

BERT là mô hình ngôn ngữ **pre-train rồi fine-tune**: pre-train trên kho văn bản lớn (không nhãn), sau đó fine-tune cho từng tác vụ (phân loại, hỏi đáp...). BERT-base có 109M tham số, quá nặng cho thiết bị biên. Trước TinyBERT, các cách distill BERT đã có:

- **BERT-PKD** (17): distill tầng giữa nhưng chỉ ở giai đoạn fine-tune.
- **DistilBERT** (16): distill ở giai đoạn pre-train, nhưng student phải khởi tạo bằng cách lấy lại một nửa số tầng của teacher, nên bị khóa cùng hidden size.
- Cả hai chủ yếu bắt chước đầu ra (logit) hoặc vài hidden state; chưa tận dụng **attention matrix**, vốn được các nghiên cứu (Clark et al. 2019) cho là chứa nhiều tri thức ngôn ngữ (cú pháp, đồng tham chiếu).

Vấn đề thứ hai: vì pipeline BERT có hai pha (pre-train, fine-tune), distill chỉ ở một pha thì bỏ lỡ tri thức của pha kia.

Khái niệm cần nhớ:

- **Logit**: vector điểm số trước softmax. **Hidden state**: đầu ra của một tầng Transformer (ma trận $l \times d$, l là độ dài chuỗi). **Attention matrix** $A = QK^\top/\sqrt{d_k}$ (chưa softmax): ma trận $l \times l$ cho mỗi head, cho biết token nào nhìn token nào.
- **Layer mapping** $g(m)$: quy tắc nói tầng m của student bắt chước tầng nào của teacher.

2. Ý tưởng chính

Student (TinyBERT) có ít tầng và hidden size nhỏ hơn teacher. Ta ép student khớp teacher ở **mọi chỗ**: embedding, attention từng head, hidden state từng tầng được ánh xạ, và logit cuối. Vì hidden size khác nhau, dùng một ma trận tuyến tính học được để chiếu hidden của student lên chiều của teacher. Ngoài ra, distill hai lần: (1) pre-train "general" với BERT gốc làm teacher trên Wikipedia, (2) task-specific với BERT đã fine-tune làm teacher trên dữ liệu đã được **data augmentation** (thay từ bằng BERT/GloVe). Kết quả: student 4 tầng đạt ≥96.8% điểm GLUE của BERT-base, nhỏ hơn 7.5x và nhanh hơn 9.4x.

3. Phương pháp chi tiết

3.1 Bài toán và layer mapping

Student có M tầng, teacher có N tầng. Đặt chỉ số 0 là embedding, $M+1$ là prediction layer, và ép $g(0)=0$, $g(M+1)=N+1$. Hàm mục tiêu tổng:

$$\mathcal{L}_{model} = \sum_{x\in\mathcal{X}} \sum_{m=0}^{M+1} \lambda_m \, \mathcal{L}_{layer}(f_m^S(x),\, f_{g(m)}^T(x))$$

Trong thí nghiệm mọi $\lambda_m = 1$ và $g(m) = 3m$ ("uniform": student 4 tầng học từ tầng 3, 6, 9, 12 của BERT-base).

3.2 Bốn loại loss

$$\mathcal{L}_{attn} = \frac{1}{h}\sum_{i=1}^{h} \text{MSE}(A_i^S, A_i^T), \qquad \mathcal{L}_{hidn} = \text{MSE}(H^S W_h,\, H^T)$$

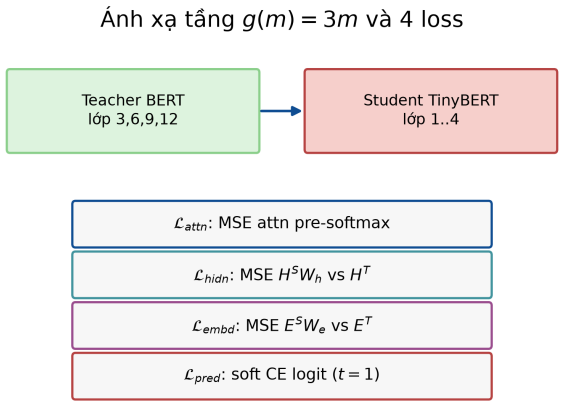
$$\mathcal{L}_{embd} = \text{MSE}(E^S W_e,\, E^T), \qquad \mathcal{L}_{pred} = \text{CE}(z^T/t,\, z^S/t) \quad (t=1)$$

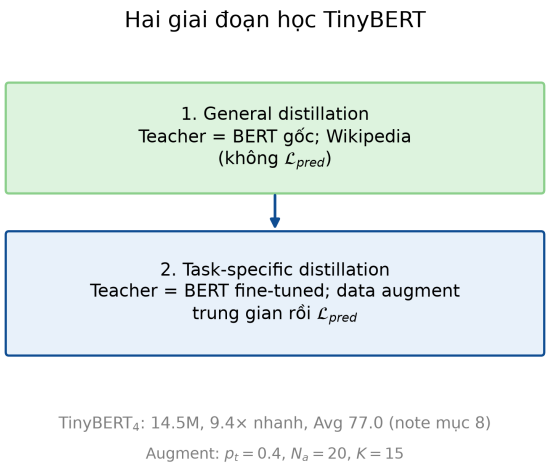
- $A_i \in \mathbb{R}^{l\times l}$ là attention của head i; **dùng bản chưa softmax** vì paper thấy hội tụ nhanh hơn và kết quả tốt hơn.
- $W_h \in \mathbb{R}^{d'\times d}$, W_e: ma trận chiếu học được, đưa hidden/embedding của student ($d'=312$) lên chiều teacher ($d=768$). Student và teacher có cùng số head $h=12$ nên attention so được trực tiếp.
- Loss mỗi tầng:

$$\mathcal{L}_{layer} = \begin{cases} \mathcal{L}_{embd} & m=0 \\ \mathcal{L}_{hidn} + \mathcal{L}_{attn} & 1\le m\le M \\ \mathcal{L}_{pred} & m=M+1 \end{cases}$$

3.3 Hai giai đoạn học

1. **General distillation**: teacher = BERT-base gốc (chưa fine-tune), dữ liệu = English Wikipedia (2.500M từ), 3 epoch, **không có** prediction loss (tác giả thử và không thêm lợi ích). Cho ra "general TinyBERT".
2. **Task-specific distillation**: teacher = BERT đã fine-tune cho tác vụ. Bước 1: distill tầng trung gian (embedding + Transformer layer) trên dữ liệu đã augment, 20 epoch (10 epoch cho MNLI/QQP/QNLI, 50 cho CoLA), batch 32, lr 5e-5. Bước 2: distill prediction layer 3 epoch.





TinyBERT: ánh xạ $g(m) = 3m$, bốn loss attn/hidn/embd/pred và hai giai đoạn general rồi task-specific (minh hoạ, tự tính)

3.4 Data augmentation (Algorithm 1)

```
Input: câu x; p_t=0.4, N_a=20, K=15
D' = []
repeat N_a lần:
    x_m = x
    for mỗi từ x[i]:
        if x[i] là từ 1 word-piece:
            thay x_m[i] bằng [MASK]; C = K từ có xác suất cao nhất theo BERT(x_m)[i]
        else:
            C = K từ gần nhất theo GloVe
        p ~ U(0,1)
        if p <= p_t: x_m[i] = một từ ngẫu nhiên trong C
    D'.append(x_m)
return D'
```

3.5 Ví dụ nhỏ

Student 4 tầng, teacher 12 tầng. Cặp (student, teacher): embedding-embedding; tầng 1-3, 2-6, 3-9, 4-12; prediction-prediction. Mỗi cặp tầng có loss MSE attention (12 head) cộng MSE hidden (sau chiếu 312 sang 768).

4. Thí nghiệm và kết quả chính

Teacher BERT-base ($N = 12$, $d = 768$, FFN 3072, $h = 12$, 109M). TinyBERT$_4$: $M = 4$, $d' = 312$, FFN 1200, $h = 12$, **14.5M** tham số. TinyBERT$_6$: $M = 6$, $d' = 768$, FFN 3072, 67.0M. Đánh giá trên tập test GLUE chính thức (Table 1, trích gọn, số "Avg" như trong bài báo):

Mô hình	#Param	FLOPs	Tăng tốc	Avg GLUE
BERT-base (teacher)	109M	22.5B	1.0x	79.5
BERT$_{TINY}$ (pre-train trực tiếp)	14.5M	1.2B	9.4x	70.2
BERT$_4$-PKD	52.2M	7.6B	3.0x	72.6
DistilBERT$_4$	52.2M	7.6B	3.0x	71.9
MobileBERT$_{TINY}$ (24 tầng)	15.1M	3.1B	-	77.0
TinyBERT$_4$	14.5M	1.2B	9.4x	**77.0**
DistilBERT$_6$	67.0M	11.3B	2.0x	76.8
TinyBERT$_6$	67.0M	11.3B	2.0x	**79.4**

Một vài con số khác: TinyBERT$_4$ hơn BERT$_{TINY}$ 6.8% (trung bình), hơn BERT$_4$-PKD và DistilBERT$_4$ ít nhất 4.4% với ~28% tham số và 3.1x suy luận nhanh hơn. TinyBERT$_6$ hơn các baseline cùng kiến trúc 2.6% trung bình. Tốc độ đo trên một GPU K80.

SQuAD (dev, phụ lục B): BERT-base teacher EM/F1 80.7/88.4 (v1.1), 74.5/77.7 (v2.0). TinyBERT$_4$: 72.7/82.1 (v1.1), 68.2/71.8 (v2.0). TinyBERT$_6$: 79.7/87.5 (v1.1), 74.7/77.7 (v2.0).

5. Phân tích / ablation

Bảng 2: các thủ tục học (dev, TinyBERT$_4$, Avg trên MNLI-m, MNLI-mm, MRPC, CoLA):

Biến thể	Avg
Đầy đủ	75.6
bỏ General Distillation (GD)	72.5
bỏ Task-specific Distillation (TD)	68.5
bỏ Data Augmentation (DA)	68.4

Cả ba đều quan trọng; TD và DA ảnh hưởng ngang nhau và lớn hơn GD. GD giúp CoLA nhiều hơn MNLI/MRPC (tác giả phỏng đoán do khả năng khái quát ngôn ngữ).

Bảng 3: các loss (Avg như trên):

Biến thể	Avg
Đầy đủ	75.6
bỏ Embd	74.1
bỏ Pred (dùng CE với nhãn thật)	73.5
bỏ Trm (cả Attn+Hidn)	56.3
bỏ Attn	71.0
bỏ Hidn	72.9

Attention quan trọng hơn hidden state, hai loại bổ trợ nhau. "bỏ Trm" sập mạnh vì ở giai đoạn pre-train, các tầng Transformer của student không nhận tín hiệu nào từ tầng trên nên khởi tạo rất tệ (giải thích của tác giả).

Bảng 4: layer mapping (Avg): Uniform 75.6; Top ($g(m) = m + N - M$) 70.9; Bottom ($g(m) = m$) 71.3. Top tốt hơn Bottom trên MNLI, ngược lại trên MRPC và CoLA, nghĩa là tác vụ khác nhau dựa vào tầng khác nhau; uniform phủ cả dải nên thắng ở mọi tác vụ. Tác giả để ngỏ việc chọn tầng thích nghi theo tác vụ.

6. Giới hạn

Do tác giả nêu: chỉ thử teacher BERT-base (hướng tương lai: teacher rộng/sâu hơn như BERT-large); chưa kết hợp với quantization/pruning; chọn mapping thích nghi theo tác vụ còn bỏ ngỏ.

Nhận xét:

- Chi phí huấn luyện cao: 20 epoch trên dữ liệu augment ($N_a = 20$ mẫu/câu) cho mỗi tác vụ, tức là phải distill lại cho từng task. MiniLM (19) so sánh trực tiếp ở dạng task-agnostic và cho thấy cách này nhẹ hơn.
- Ràng buộc: số head student = số head teacher, và cần ma trận chiếu W_h, W_e (tham số thêm chỉ dùng khi huấn luyện).
- Toàn bộ thực nghiệm là NLU trên văn bản; không có chứng cứ cho sinh chuỗi, thị giác hoặc multimodal.
- Dùng chuỗi ngắn (độ dài 64/128): loss attention $l \times l$ chưa được kiểm tra với chuỗi dài.

7. Liên hệ với các bài khác

- **03 FitNets**: TinyBERT là FitNets (hint ở tầng giữa, có regressor tuyến tính) áp dụng có hệ thống cho Transformer, cộng thêm loss attention.
- **04 Attention transfer**: cùng triết lý "khớp bản đồ attention", ở đây áp dụng cho từng head của self-attention.
- **16 DistilBERT, 17 Patient KD**: baseline trực tiếp; TinyBERT hơn nhờ khớp nhiều loại biểu diễn và hai giai đoạn.

- **19 MiniLM**: đối lập: chỉ distill tầng cuối, tránh bài toán layer mapping, hidden size student tùy ý. MiniLM tự so sánh với TinyBERT và cho thấy "last layer" thắng "layer-to-layer" trong thiết lập của họ.
- **20 Causal distillation**: tranh luận cùng chủ đề: có nên ép student khớp *mọi* khía cạnh trạng thái trong của teacher hay chỉ phần có vai trò nhân quả?
- **08 Teacher assistant**: MiniLM thêm TA; TinyBERT không dùng.

8. Tóm tắt công thức / khái niệm

Khái niệm	Nội dung
$g(m)$	Ánh xạ tầng student m sang tầng teacher; paper dùng $g(m) = 3m$
$\mathcal{L}_{attn}$	MSE giữa attention chưa softmax, trung bình theo head
$\mathcal{L}_{hidn}$	MSE giữa $H^S W_h$ và H^T
$\mathcal{L}_{embd}$	MSE giữa $E^S W_e$ và E^T
$\mathcal{L}_{pred}$	Cross-entropy mềm giữa logit, $t = 1$
General distillation	Teacher = BERT gốc, dữ liệu Wikipedia, không dùng $\mathcal{L}_{pred}$
Task-specific distillation	Teacher = BERT fine-tuned, dữ liệu augment, trung gian rồi prediction
Data augmentation	Thay từ bằng BERT (từ 1 piece) hoặc GloVe (nhiều piece), $p_t = 0.4$, $N_a = 20$, $K = 15$
Kết quả	TinyBERT$_4$: 14.5M, 9.4x nhanh, Avg 77.0; TinyBERT$_6$: Avg 79.4 vs teacher 79.5

9. Kết luận

TinyBERT khớp attention và hidden state ở các lớp được ánh xạ, qua cả giai đoạn tiền huấn luyện và giai đoạn tác vụ. Ablation cho thấy đóng góp lớn của attention loss; ánh xạ lớp đều và phép chiếu khi chiều ẩn khác nhau cũng là những lựa chọn được khảo sát. Đổi lại, quy trình cần chưng cất riêng cho từng tác vụ và dữ liệu augmentation.

Link gốc: https://arxiv.org/abs/1909.10351

Bài 19

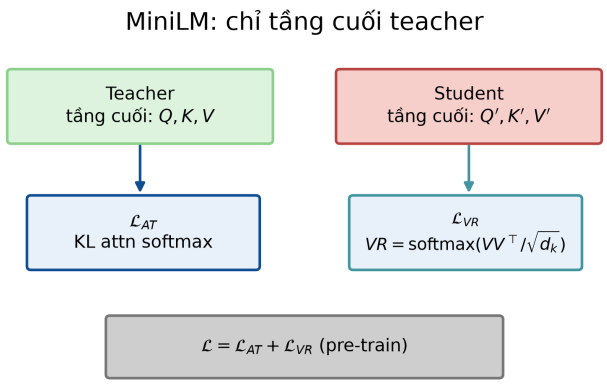
MiniLM: Deep Self-Attention Distillation for Task-Agnostic Compression of Pre-Trained Transformers (2020)

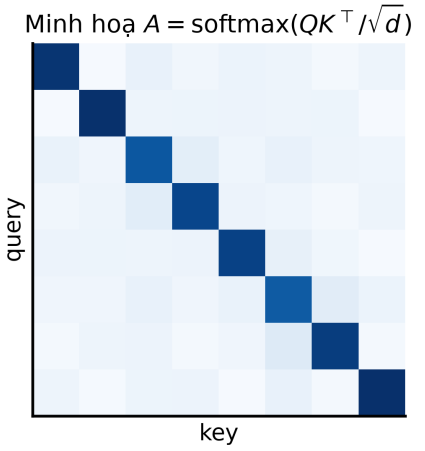
Tác giả: Wenhui Wang, Furu Wei, Li Dong, Hangbo Bao, Nan Yang, Ming Zhou (Microsoft Research). **Venue / arXiv:** arXiv:2002.10957 (venue không được nêu trong bản text; paper được biết tới là NeurIPS 2020). **Nguồn:** https://arxiv.org/abs/2002.10957 **Chủ đề:** Distillation task-agnostic cho Transformer; chỉ distill **self-attention của tầng cuối** (phân phối attention + quan hệ giữa các value), không cần layer mapping, hidden size student tùy ý. **Vị trí trong lộ trình:** Đọc trước: 16 (DistilBERT), 18 (TinyBERT), 08 (Teacher assistant). Bài này thêm: bỏ hẳn bài toán "tầng nào bắt chước tầng nào" bằng cách chỉ nhìn tầng cuối, và thay MSE hidden bằng so sánh *quan hệ* (ma trận $|x| \times |x|$) nên không cần ma trận chiếu. Bài kế: 20 (Causal distillation) hỏi lại "nên ép student khớp cái gì".

Ghi chú học tập bằng tiếng Việt, không phải bản dịch nguyên văn. Bảng nào lỗi khi trích PDF sẽ được ghi rõ.

1. Bối cảnh

Hai kiểu distill mô hình ngôn ngữ đã pre-train:





VR tương tự nhưng từ V (cùng kích thước $|x|\times|x|$)

MiniLM distill tầng cuối: KL attention $\mathcal{L}_{AT}$ và value-relation $VR = \text{softmax}(VV^\top/\sqrt{d_k})$ (minh hoạ, tự tính)

- **Task-specific**: fine-tune teacher lớn cho từng tác vụ, rồi distill. Hiệu quả nhưng phải fine-tune teacher cho mỗi tác vụ, tốn kém trên dữ liệu lớn.
- **Task-agnostic**: student bắt chước teacher *khi pre-train*, sau đó student được fine-tune bình thường cho từng tác vụ (như BERT gốc). Đây là cái ta muốn, vì chỉ distill một lần.

Các phương pháp task-agnostic trước đó mắc ràng buộc:

Phương pháp	Ràng buộc
DistilBERT	student phải cùng hidden size với teacher (khởi tạo bằng cách lấy lại 1 trong 2 tầng)
TinyBERT	cần layer-to-layer mapping (uniform) và ma trận chiếu hidden
MobileBERT	student phải cùng số tầng, và cần teacher thiết kế riêng (IB-BERT-large) cùng bottleneck

Khái niệm: **task-agnostic** = không phụ thuộc tác vụ. **Value** V trong attention là vector được "trộn" theo trọng số attention; **KL-divergence** $D_{KL}(P\|Q)$ đo độ khác nhau của hai phân phối xác suất.

2. Ý tưởng chính

Ba ý, theo chính tác giả: (1) chỉ distill **module self-attention của tầng Transformer cuối cùng** của teacher, không phải mọi tầng. (2) Ngoài phân phối attention (tích vô hướng query-key), distill thêm **quan hệ giữa các value** (tích vô hướng value-value, qua softmax). Vì cả hai đều là ma trận $|x| \times |x|$ cho mỗi head, kích thước không phụ thuộc hidden size, nên student chọn hidden size bất kỳ mà không cần tham số chiếu. (3) Khi student quá nhỏ so với teacher, thêm một **teacher assistant** (student cỡ trung gian) để làm cầu nối.

3. Phương pháp chi tiết

Với tầng cuối của teacher (L tầng) và student (M tầng), A_h head, $|x|$ độ dài chuỗi, d_k chiều mỗi head của teacher, d'_k của student.

Self-attention distribution transfer (attention = softmax của $QK^\top/\sqrt{d_k}$):

$$\boxed{\mathcal{L}_{AT} = \frac{1}{A_h|x|}\sum_{a=1}^{A_h}\sum_{t=1}^{|x|} D_{KL}(A^T_{L,a,t} \,\|\, A^S_{M,a,t})}$$

Value-relation transfer:

$$VR^T_{L,a} = \text{softmax}\Big(\frac{V^T_{L,a}V^{T\top}_{L,a}}{\sqrt{d_k}}\Big), \quad VR^S_{M,a} = \text{softmax}\Big(\frac{V^S_{M,a}V^{S\top}_{M,a}}{\sqrt{d'_k}}\Big)$$

$$\boxed{\mathcal{L}_{VR} = \frac{1}{A_h|x|}\sum_{a=1}^{A_h}\sum_{t=1}^{|x|} D_{KL}(VR^T_{L,a,t} \,\|\, VR^S_{M,a,t})}, \qquad \boxed{\mathcal{L} = \mathcal{L}_{AT} + \mathcal{L}_{VR}}$$

Hiểu trực giác: $VR_a[i,j]$ cho biết "value của token i giống value của token j cỡ nào". Đó là một dạng **relational knowledge** (xem 12 Relational KD): không so vector với vector mà so *cấu trúc quan hệ* giữa các token. Số head của student đặt bằng 12 (bằng teacher) để so từng head.

Dạng tương đương dùng cho các student khác nhau: nếu hidden size student $d'_h = 384$, mỗi head vẫn có $d'_k = d'_h/12 = 32$ chiều thay vì 64, nhưng ma trận VR vẫn là $|x| \times |x|$.

Teacher assistant (TA): với student nhỏ ($M \le L/2$, $d'_h \le d_h/2$), trước hết distill teacher (L tầng, d_h) vào TA có L tầng, hidden d'_h; sau đó dùng TA làm teacher cho student cuối.

Pseudo-code:

```
# task-agnostic, dữ liệu pre-train (Wikipedia + BookCorpus)
teacher = BERT-base (đóng băng)
student = Transformer M tầng, hidden d', 12 head   # khởi tạo ngẫu nhiên
for batch x in corpus:
    (Q_T, K_T, V_T) = teacher.last_layer_QKV(x)
    (Q_S, K_S, V_S) = student.last_layer_QKV(x)
    A_T,  A_S  = softmax(Q K^T / sqrt(dk)) cho từng head
    VR_T, VR_S = softmax(V V^T / sqrt(dk)) cho từng head
    loss = KL(A_T || A_S) + KL(VR_T || VR_S)     # trung bình theo head và vị trí
    loss.backward(); step()
# sau đó fine-tune student trên tác vụ như BERT bình thường
```

Siêu tham số: batch 1024, lr đỉnh 5e-4, 400.000 bước cho student 6x768; batch 256, lr 3e-4 cho các kiến trúc khác; warmup 4.000 bước; 8 GPU V100; dữ liệu English Wikipedia + BookCorpus; teacher BERT-base uncased (109M).

4. Thí nghiệm và kết quả chính

So sánh cùng kiến trúc 6x768 (66M tham số) distill từ BERT-base, không có task-specific distillation hay data augmentation (Table 2, trung bình 4 lần chạy):

Mô hình	#Param	SQuAD2	MNLI-m	SST-2	QNLI	CoLA	RTE	MRPC	QQP	Avg
BERT-base	109M	76.8	84.5	93.2	91.7	58.9	68.6	87.3	91.3	81.5
DistilBERT	66M	70.7	79.0	90.7	85.3	43.6	59.9	87.5	84.9	75.2
TinyBERT	66M	73.1	83.5	91.6	90.5	42.8	72.2	88.4	90.6	79.1
MiniLM	66M	76.4	84.0	92.0	91.0	49.2	71.5	88.4	91.0	80.4

MiniLM 6x768 nhanh 2.0x (46.9s so với 93.1s) và giữ >99% độ chính xác của BERT-base trên SQuAD 2.0 và một số tác vụ GLUE, theo paper.

Student nhỏ hơn (Table 3; trung bình 4 lần; cột SQuAD2 / MNLI-m / SST-2 / Avg):

Kiến trúc	Mô hình	SQuAD2	MNLI-m	SST-2	Avg
M=6, d’=384 (22M)	MLM-KD (soft-label)	67.9	79.6	89.8	79.1
	TinyBERT (tự cài)	71.6	81.4	90.2	81.1
	MiniLM	72.4	82.2	91.0	81.9
	MiniLM + TA	72.7	82.4	91.2	82.1
M=4, d’=384 (19M)	MiniLM	69.4	80.3	90.2	80.0
	MiniLM + TA	69.7	80.6	90.6	80.3
M=3, d’=384 (17M)	MiniLM	66.2	78.8	89.3	78.1
	MiniLM + TA	66.9	79.1	89.7	78.6

Tốc độ (Table 4, P100): 12x768 là 1.0x; 6x768 2.0x; 12x384 2.7x; 6x384 5.3x; 4x384 7.8x; 3x384 10.1x.

Teacher tốt hơn thì student tốt hơn (Table 8): distill từ teacher nội bộ cỡ BERT-base (huấn luyện tương tự RoBERTa-base, 160GB dữ liệu): MiniLM 12x384 (33M) đạt SQuAD2 81.7, MNLI-m 85.7, Avg 83.1, vượt BERT-base (Avg 81.5) ở 2.7x tốc độ; MiniLM 6x384 + TA (22M) đạt Avg 79.6. Phần sinh văn bản (question generation, tóm tắt CNN/DailyMail và XSum) và mô hình đa ngôn ngữ distill từ XLM-R-base (XNLI, MLQA) đều được báo cáo cạnh tranh với số tham số Transformer ít hơn; chi tiết xem Table 9 đến 13 trong bài báo (không chép lại ở đây).

5. Phân tích / ablation

- **Value-relation có ích** (Table 5): bỏ Value-Rel thì giảm ở mọi cỡ student. Ví dụ M=6,d’=384: SQuAD2 72.4 xuống 71.0, MNLI-m 82.2 xuống 80.9, SST-2 91.0 xuống 89.9; M=3: SQuAD2 66.2 xuống 64.2.
- **KL trên quan hệ value hơn MSE trên value** (Table 6): với M=6, MSE đạt 71.4 / 82.0 / 90.8 so với 72.4 / 82.2 / 91.0; M=4: 68.3 / 80.1 / 89.9 so với 69.4 / 80.3 / 90.2; M=3: 65.5 / 78.4 / 89.3 so với 66.2 / 78.8 / 89.3. MSE trên value còn cần ma trận chiếu thêm.

- **Chỉ tầng cuối hơn layer-to-layer** (Table 7, cùng kiến thức, mapping uniform như TinyBERT): Avg MiniLM 81.9 vs 81.3 (M=6), 80.0 vs 79.0 (M=4), 78.1 vs 77.0 (M=3). Ngoài ra huấn luyện nhanh hơn.
- **Teacher assistant** giúp, nhất là student nhỏ (Table 3: +0.2 đến +0.5 Avg).
- Tác giả cũng thử transfer quan hệ giữa hidden states và thấy kết quả **không ổn định** giữa các teacher khác nhau (không có bảng).

6. Giới hạn

Do tác giả nêu hoặc suy ra từ paper:

- Tăng ích nhỏ khi student đã đủ lớn; TA chỉ rõ rệt cho student nhỏ.
- Cách làm gắn với cấu trúc self-attention; quan hệ hidden state thì không ổn định.

Nhận xét:

- So sánh với TinyBERT trong Table 2 fine-tune bản public (phiên bản 2) của TinyBERT, và trong Table 3 dùng bản tự cài lại, không phải đầy đủ pipeline hai giai đoạn + data augmentation của TinyBERT. Vì thế "MiniLM thắng TinyBERT" chỉ đúng trong thiết lập task-agnostic không augmentation.
- Chỉ ép tầng cuối: các tầng giữa của student không có giám sát trực tiếp, chỉ gián tiếp qua gradient. Với student rất sâu/hẹp điều này có thể khó huấn luyện; bài báo không thử student rất khác teacher về độ sâu ngoài 3 đến 12 tầng.
- Dữ liệu pre-train lớn (Wikipedia + BookCorpus, 400.000 bước batch 1024): không phải là distill "rẻ".
- Mọi thí nghiệm là encoder hai chiều; việc áp dụng cho decoder nhân quả không được kiểm tra (trừ fine-tune MiniLM với attention mask seq2seq cho tác vụ sinh).

7. Liên hệ với các bài khác

- **18 TinyBERT**: đối chứng trực tiếp; layer-to-layer vs last-layer.
- **16 DistilBERT**: baseline soft-label; MiniLM bỏ ràng buộc hidden size.
- **08 Teacher assistant**: nguồn gốc ý tưởng TA (Mirzadeh); MiniLM xác nhận hữu ích với Transformer pre-train. Xem thêm 41 (Capacity gap law) cho phân tích gap lớn giữa teacher và student.
- **12 Relational KD**: value-relation là cùng họ "distill quan hệ giữa các mẫu/token" thay vì giá trị tuyệt đối.
- **04 Attention transfer**: cùng ý tưởng khớp bản đồ attention.
- **43 MiniPLM, 29 GKD** (hướng LLM): task-agnostic distillation ở quy mô LLM thường dùng logit thay vì attention.

8. Tóm tắt công thức / khái niệm

Khái niệm	Nội dung
$\mathcal{L}_{AT}$	KL giữa phân phối attention tầng cuối, trung bình theo head và vị trí
VR	softmax$(VV^\top/\sqrt{d_k})$, quan hệ value-value $\lvert x\rvert \times \lvert x\rvert$
$\mathcal{L}_{VR}$	KL giữa VR^T và VR^S
$\mathcal{L}$	$\mathcal{L}_{AT} + \mathcal{L}_{VR}$
Teacher assistant	Student trung gian cùng số tầng teacher, hidden bằng student cuối
Task-agnostic	Distill khi pre-train; fine-tune student như bình thường
Kết quả chính	6x768: Avg 80.4 vs DistilBERT 75.2, TinyBERT 79.1; 12x384 từ teacher mạnh: Avg 83.1 > BERT-base 81.5

9. Kết luận

MiniLM dùng KL để khớp attention và quan hệ value ở lớp cuối, không cần ánh xạ nhiều lớp hay ma trận chiếu chiều ẩn. Trong thiết lập được thử, kết quả không kém và còn tốt hơn cách khớp từng lớp. Phương pháp giả định lớp được chọn có softmax attention.

Link gốc: https://arxiv.org/abs/2002.10957

Bài 20

Causal Distillation for Language Models (2022)

Tác giả: Zhengxuan Wu, Atticus Geiger, Joshua Rozner, Elisa Kreiss, Hanson Lu, Thomas Icard, Christopher Potts, Noah D. Goodman (Stanford). **Venue / arXiv:** arXiv:2112.02505 (bản v2, 3/6/2022; bản text không nêu venue, paper được biết tới là NAACL 2022). **Nguồn:** https://arxiv.org/abs/2112.02505 **Chủ đề:** Thêm vào distillation chuẩn một loss thứ ba, **DIITO** (distillation interchange intervention training objective), buộc student bắt chước *cấu trúc nhân quả* của teacher, nối distillation với interpretability (causal abstraction, interchange intervention). **Vị trí trong lộ trình:** Đọc trước: 16 (DistilBERT, baseline của bài này), 18 (TinyBERT), 19 (MiniLM). Bài này thêm: không chỉ hỏi "student có cùng đầu ra / cùng hidden state không" mà hỏi "student phản ứng thế nào khi **can thiệp** vào trạng thái trong, có giống teacher không". Liên quan phần cuối lộ trình về hiểu biểu diễn bên trong.

> Ghi chú học tập bằng tiếng Việt, không phải bản dịch nguyên văn. Một số số trong Table 1 không khớp với con số tóm tắt ở abstract (xem mục 4); cả hai được ghi lại mà không tự sửa.

1. Bối cảnh

Distillation chuẩn cho mô hình ngôn ngữ (DistilBERT, 16) huấn luyện student với ba loss: loss tác vụ (MLM), loss bắt chước đầu ra teacher (KL/CE mềm), và loss cosine giữa hidden state cuối. Các biến thể (17, 18) thêm loss khớp hidden state/attention tầng giữa. Tất cả đều ép student **giống teacher ở trạng thái quan sát được**, bất kể trạng thái đó có vai trò nhân quả trong phép tính hay không. Tác giả cho rằng điều này chưa tối ưu.

Ba khái niệm nền (viết cho người mới):

- **Causal abstraction**: mô hình đơn giản (cấp cao) là "trừu tượng nhân quả" của mô hình phức tạp (cấp thấp) nếu có một ánh xạ giữa biến cấp cao và nhóm neuron cấp thấp sao cho **can thiệp** vào hai bên cho kết quả tương ứng. Nói cách khác student "giải thích" teacher một cách trung thực.
- **Can thiệp (intervention)**: ghi đè giá trị của một nhóm neuron bằng giá trị khác rồi chạy tiếp (như thí nghiệm ablation/activation patching trong interpretability, ví dụ bài Transformer Circuits trong thư mục bên cạnh).
- **Interchange intervention**: lấy hai đầu vào x_1 (nguồn) và x_2 (nền); chạy x_2 nhưng thay giá trị của nhóm neuron N bằng giá trị chúng có khi chạy x_1. Đây là một dạng phản thực: "đầu ra sẽ ra sao nếu tại vị trí này mô hình thấy thông tin từ câu khác?".

2. Ý tưởng chính

Với mỗi bước huấn luyện, lấy hai câu x_1, x_2. Chọn một tầng của student và các tầng tương ứng của teacher (ví dụ tầng 2 của student 3 tầng ứng với tầng 3-4 của teacher 6 tầng), chọn một số token, và **thay hidden state tại đó bằng hidden state của** x_1 trong cả hai mô hình (cho chạy tiếp trên x_2). Rồi bắt phân phối đầu ra của student *sau can thiệp* khớp phân phối đầu ra của teacher *sau can thiệp tương ứng*. Nếu student khớp teacher dưới mọi can thiệp như vậy, nó đã nắm cách teacher "dùng" thông tin tại tầng đó. Loss mới này được **cộng thêm** vào ba loss của DistilBERT, và tất cả đều khả vi.

3. Phương pháp chi tiết

3.1 Toán tử

- GetVals(M, x, N): giá trị các neuron N khi mô hình M xử lý x.
- SetVals(M, N, v): mô hình mới trong đó neuron N bị đặt bằng hằng v (ghi đè tại chỗ nên gradient vẫn chảy qua v).

- $M_{x_1}^N = \text{SetVals}(M, N, \text{GetVals}(M, x_1, N))$.

$$\text{IntInv}(M, N, x_1, x_2) = \text{GetVals}(M_{x_1}^N,\ x_2,\ N_y)$$

với N_y là neuron đầu ra. Tức là đầu ra khi chạy nền x_2 nhưng neuron N lấy từ nguồn x_1.

3.2 Loss DIITO

Cho alignment Π (ánh xạ nhóm neuron student sang nhóm neuron teacher):

$$\boxed{\mathcal{L}_{CE}^{DIITO} = \sum_{x_1, x_2 \in \mathcal{D}} CE^S\Big(\text{IntInv}(S, N^S, x_1, x_2),\ \text{IntInv}(T, \Pi(N^S), x_1, x_2)\Big)}$$

CE^S là cross-entropy mềm (smoothed) như trong Hinton. Loss tổng của student là tổ hợp tuyến tính, **trọng số bằng nhau**:

$$\mathcal{L} = \mathcal{L}_{MLM} + \mathcal{L}_{CE} + \mathcal{L}_{Cos} + \mathcal{L}_{CE}^{DIITO}\ \ (+\ \mathcal{L}_{Cos}^{DIITO})$$

$\mathcal{L}_{Cos}$: cosine embedding loss giữa hidden state tầng cuối của student và teacher; $\mathcal{L}_{Cos}^{DIITO}$ là phiên bản tương tự nhưng cả hai mô hình đều đang chịu can thiệp (mục A.2 của bài báo).

3.3 Alignment Π (ba loại)

Gọi L_T, L_S là số tầng teacher/student, S_i^j là biểu diễn ở hàng (tầng) i, cột (token) j của student.

- **FULL**: mọi tầng student: $\Pi(S_i^j) = \{T_{4i+k}^j :\ 0 \le k < L_T/L_S\}$ (công thức trong bài báo dùng hệ số 4 cho thiết lập 12 và 3 tầng; hiểu như "khối tầng teacher tương ứng").
- **MIDDLE**: chỉ tầng giữa $L_S/2$ của student $\leftrightarrow$ tầng giữa $L_T/2$ của teacher.
- **LATE**: hai tầng đầu của student $\leftrightarrow$ hai tầng áp cuối của teacher ($L_T - 2$, $L_T - 1$).

Mỗi vòng lặp chọn **ngẫu nhiên một** tầng student được alignment để can thiệp, và ngẫu nhiên **30%** vị trí token trong chuỗi. Với FULL thử ba cách chọn token: liên tiếp (mặc định), ngẫu nhiên (+Random), token bị mask (+Masked).

3.4 Pseudo-code (Algorithm 1, rút gọn)

```
S.train(); T.eval()
D' = shuffle(D)
for ({x1,y1}, {x2,y2}) in zip(D, D'):
    N_S = sample_student_neurons()          # 1 tầng alignment, 30% token
    N_T = Pi(N_S)
    with no_grad:
        T_a = SetVals(T, N_T, GetVals(T, x1, N_T))   # teacher với trạng thái từ x1
        o_T = GetVals(T_a, x2, N_y^T)                 # logit teacher dưới can thiệp
    S_a = SetVals(S, N_S, GetVals(S, x1, N_S))        # student: gradient chảy qua
    o_S = GetVals(S_a, x2, N_y^S)
    L_DIITO = soft_CE(o_T, o_S)
    L = L_MLM + L_CE + L_Cos + L_DIITO
    L.backward(); optimizer.step()
```

3.5 Ví dụ (Figure 1)

Câu nền x_2 = "I ate some [MASK]." Ta thay biểu diễn của token [MASK] ở tầng giữa bằng biểu diễn token [MASK] của câu nguồn "The water [MASK] solid." (từ có nghĩa kiểu *froze*). Student (chưa luyện) dự đoán

"salad" trong tình huống can thiệp, teacher dự đoán "pizza"; DIITO kéo phân phối của student về của teacher. (Hai từ này chỉ là minh họa của bài báo về cơ chế, không phải kết quả đo.)

4. Thí nghiệm và kết quả chính

Thiết lập: teacher BERT-base 12 tầng; student 6 tầng hoặc 3 tầng (12 head, hidden 768), khởi tạo từ các tầng bị bỏ qua của teacher (kiểu DistilBERT). Distill trên **WikiText-103M** (khoảng 0.1B token) để mô phỏng ngân sách nhỏ; 3 epoch, effective batch 240, mọi loss trọng số 1; khoảng 9 giờ trên 4 GPU A100. Fine-tune đánh giá: GLUE (trừ WNLI), CoNLL-2003 NER, SQuAD v1.1. Kết quả là trung bình 15 lần chạy (3 seed distill x 5 seed fine-tune; perplexity trung bình 3 lần).

Table 1 (dev, trích; trong ngoặc là độ lệch chuẩn; chỉ các cột PPL = perplexity WikiText, GLUE, SQuAD EM/F1):

Mô hình	Tầng	PPL	GLUE	SQuAD EM	SQuAD F1
BERT-base (Wikipedia+BookCorpus, 3.3B token)	12	10.27	82.75	80.80	88.50
DistilBERT chính thức (3.3B token)	6	17.48	79.59	77.70	85.80
DistilBERT (WikiText)	6	15.69	75.80	70.23	79.99
DIITO-MIDDLE	6	14.32	76.71	71.93	81.32
DIITO-LATE	6	14.93	76.80	71.47	81.01
DIITO-FULL	6	13.59	76.67	71.96	81.33
DIITO-FULL + $\mathcal{L}_{Cos}^{DIITO}$	6	**13.45**	**77.14**	71.94	81.35
DistilBERT (WikiText)	3	29.51	67.42	(xem lưu ý)	68.38
DIITO-FULL	3	24.85	69.36	(xem lưu ý)	70.50

Lưu ý bảng: cột SQuAD EM của hàng 3 tầng trong bản text trích có giá trị 26.04 cho DistilBERT và 58.7x cho các DIITO; 26.04 trùng với một giá trị perplexity ở hàng bên cạnh nên có khả năng bảng bị lẫn khi trích PDF. Không dùng cột này.

Các con số tóm tắt trong abstract/văn bản: -2.24 perplexity (FULL + $\mathcal{L}_{Cos}^{DIITO}$, 13.45 so với 15.69 khớp với bảng), **+1.77%** GLUE, **+0.38%** F1 trên CoNLL-2003 (DIITO-MIDDLE; vượt cả teacher BERT-base +0.05 F1), **+2.46%** EM trên SQuAD v1.1. Không tái lập được hai con số cuối bằng phép trừ trực tiếp từ cột Table 1 trong bản trích (ví dụ GLUE 77.14 - 75.80 = 1.34; EM tốt nhất 71.96 - 70.23 = 1.73), có thể do cách tính (ví dụ lấy tối ưu theo từng cấu hình hoặc bảng khác); cần đối chiếu với PDF gốc nếu trích dẫn.

Nhận xét tổng thể của bài báo: DIITO với 0.1B token đạt khoảng 97% điểm GLUE của DistilBERT chính thức (dùng 3.3B token), tức dùng ít hơn 97% dữ liệu; gain lớn nhất ở student nhỏ hơn (3 tầng) và alignment phong phú nhất (FULL).

5. Phân tích / ablation

- **Alignment càng giàu càng tốt:** FULL thường cho perplexity tốt nhất (13.59 vs 14.32 MIDDLE vs 14.93 LATE ở 6 tầng; 3 tầng: 24.85 / 26.04 / 25.97). Trên GLUE/SQuAD/NER chênh lệch giữa các alignment nhỏ.
- **Chọn token:** liên tiếp, ngẫu nhiên, hay token bị mask cho PPL gần nhau (13.59 / 13.95 / 13.99) và GLUE gần nhau (76.67 / 76.84 / 76.80); không rõ rệt.
- **Thêm** $\mathcal{L}_{Cos}^{DIITO}$ cải thiện nhẹ PPL (13.45) và GLUE (77.14).
- **Theo từng task GLUE** (Table 2 trong phụ lục): DIITO thắng DistilBERT cùng thiết lập trên đa số tác vụ, lớn nhất ở CoLA (40.43 lên 45.17 với FULL + Cos, 6 tầng) và STS-B ở student 3 tầng (56.33 lên 66.93 MIDDLE); RTE thay đổi không rõ ràng.

- Phân phối điểm qua 15 lần chạy (Figure 2, 3) cho thấy gain nhất quán, student nhỏ nhận gain lớn hơn.

6. Giới hạn

Do tác giả nêu: chỉ thử trên WikiText nhỏ để mô phỏng ngân sách hạn chế; chưa thử dữ liệu lớn hơn ("leave for future research"). Gain trên NER nhỏ (tác giả tự nhận "small").

Nhận xét:

- Không có thí nghiệm kiểm tra *vì sao* DIITO giúp. Việc student "trở thành causal abstraction" là một lập luận khái niệm; bài báo không đo mức độ student có thực sự trừu tượng hóa teacher (ví dụ không đo độ trung thành của can thiệp trên tập đánh giá).
- Alignment dựa trên giả định tầng-theo-tầng (khối tầng đều); chưa có cách chọn alignment tự động.
- Chi phí mỗi bước tăng: thêm một lần forward có can thiệp cho cả teacher và student.
- Kích thước hiệu ứng khiêm tốn và thang so sánh nhỏ (6 tầng/3 tầng BERT, encoder, MLM); các con số tóm tắt trong abstract không khớp hoàn toàn với bảng chính (xem mục 4).
- Không so với TinyBERT/MiniLM; baseline duy nhất là DistilBERT cùng thiết lập.

7. Liên hệ với các bài khác

- **16 DistilBERT**: baseline, DIITO chỉ *cộng thêm* một loss.
- **18 TinyBERT / 19 MiniLM**: cùng muốn truyền thông tin tầng giữa; DIITO không so giá trị hay quan hệ mà so *phản ứng với can thiệp*.
- **12 Relational KD**: khác nhưng cùng tinh thần "dạy cấu trúc hơn là giá trị".
- **10 Does KD really work**: đặt câu hỏi student có thực sự khớp teacher; DIITO là một cách làm "khớp" mạnh hơn nhưng bằng chứng của bài này chưa kiểm tra độ khớp.
- **Transformer Circuits (2021)** trong thư mục liên quan: activation patching/path expansion là công cụ phân tích; DIITO biến patching thành *tín hiệu huấn luyện*.
- **38 DeepSeek-R1 distillation, 28 MiniLLM**: ở hướng LLM, không dùng can thiệp; chủ yếu bắt chước đầu ra.

8. Tóm tắt công thức / khái niệm

Khái niệm	Nội dung
Causal abstraction	Mô hình đơn giản là mô tả trung thực về hiệu ứng nhân quả của mô hình phức tạp
Interchange intervention	Chạy x_2 nhưng neuron N lấy giá trị từ x_1
Alignment II	FULL / MIDDLE / LATE: neuron student $\leftrightarrow$ neuron teacher
$\mathcal{L}_{CE}^{DIITO}$	CE mềm giữa đầu ra student và teacher dưới can thiệp tương ứng
Tổng loss	$\mathcal{L}_{MLM} + \mathcal{L}_{CE} + \mathcal{L}_{Cos} + \mathcal{L}_{CE}^{DIITO} (+\mathcal{L}_{Cos}^{DIITO})$, trọng số 1
Lấy mẫu	1 tầng alignment ngẫu nhiên/bước, 30% token
Kết quả	6 tầng: PPL 13.45 vs 15.69; GLUE 77.14 vs 75.80 (Table 1)

9. Kết luận

Causal Distillation bổ sung loss dựa trên interchange intervention để so sánh phản ứng của teacher và student dưới can thiệp. Kết quả tốt hơn DistilBERT cùng ngân sách nhỏ một cách nhất quán nhưng khiêm tốn. Bằng chứng chưa bao gồm OCR hay VLM.

Link gốc: https://arxiv.org/abs/2112.02505

Bài 21

Training data-efficient image transformers & distillation through attention (2021)

Tác giả: Hugo Touvron, Matthieu Cord, Matthijs Douze, Francisco Massa, Alexandre Sablayrolles, Hervé Jégou (Facebook AI, Sorbonne University). **Venue / arXiv:** arXiv:2012.12877 (bản v2, 15/1/2021; paper được biết tới là ICML 2021, bản text không nêu venue). Mô hình có tên **DeiT** (Data-efficient image Transformers). **Nguồn:** https://arxiv.org/abs/2012.12877 **Chủ đề:** (1) huấn luyện ViT chỉ trên ImageNet-1k với recipe augment/regularize mạnh; (2) **distillation token**: thêm một token vào ViT để học nhãn của teacher; so sánh distillation **cứng** và **mềm**; teacher convnet dạy transformer tốt hơn teacher transformer. **Vị trí trong lộ trình:** Đọc trước: 02 (Hinton KD: soft label, temperature), 03 (FitNets), 06 (Born-again networks). Bài này thêm: distillation đặc thù cho ViT, và chứng cứ rằng distillation truyền **inductive bias** của convnet sang transformer. Bài kế: 22 (DINO, cùng nhóm tác giả, distillation không nhãn), 23 (AM-RADIO, nén ViT từ nhiều teacher).

> Ghi chú học tập bằng tiếng Việt, không phải bản dịch nguyên văn. Một số bảng trích từ PDF bị lẫn ký hiệu (biểu tượng alembic "⚗" là dấu của DeiT có distillation); chỉ ghi lại những số đọc được rõ.

1. Bối cảnh

ViT (Dosovitskiy et al.) cắt ảnh thành các patch 16x16, chiếu mỗi patch thành một token và đưa vào Transformer nguyên bản của NLP, kèm một **class token** (vector học được, thêm vào đầu chuỗi, dùng để dự đoán nhãn). ViT tốt nhưng bài gốc kết luận nó "không khái quát tốt khi huấn luyện trên lượng dữ liệu không đủ" và dùng JFT-300M (300 triệu ảnh nội bộ) cùng hạ tầng lớn. Vấn đề: không ai có sẵn dữ liệu và máy như vậy. Convnet có **inductive bias** (giả định cấu trúc: cục bộ, bất biến tịnh tiến) nên học hiệu quả với ít dữ liệu hơn.

Knowledge distillation (Hinton): student học từ **soft label** (vector softmax của teacher) thay vì chỉ nhãn cứng. Hai tác dụng được paper nhắc: giống label smoothing, và teacher phản ánh được tác động của data augmentation (một ảnh gán nhãn "mèo" nhưng crop mất con mèo thì nhãn gốc sai, teacher thì thấy crop thực tế). Distillation cũng có thể truyền inductive bias "một cách mềm" từ một teacher có bias cứng.

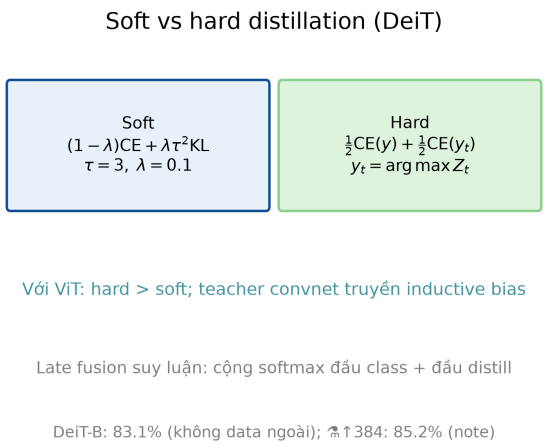
2. Ý tưởng chính

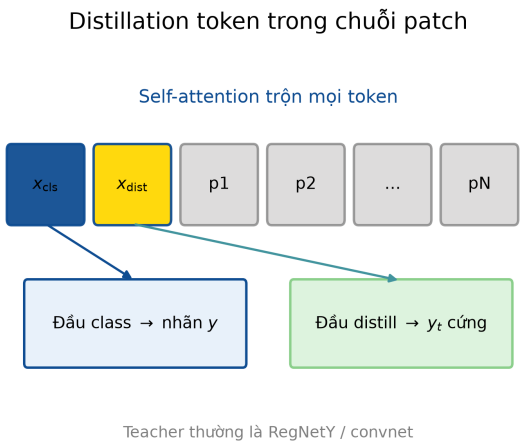
Hai đóng góp. (1) Dùng recipe huấn luyện mạnh (augmentation nặng, Mixup/CutMix, repeated augmentation, stochastic depth, AdamW, ...) để ViT-B (86M) đạt 83.1% top-1 trên ImageNet-1k thuần (không dữ liệu ngoài), huấn luyện chưa tới 3 ngày trên một máy. (2) Thêm **distillation token**: một token học được, đặt cạnh class token và patch tokens, đi qua mọi tầng self-attention, đầu ra của nó bị ép dự đoán **nhãn cứng do teacher (một convnet) gán**, trong khi class token vẫn học nhãn thật. Hai token tương tác qua attention nhưng nhận hai mục tiêu khác nhau. Lúc suy luận cộng softmax của hai đầu phân loại. Kết quả: DeiT chưng cất từ convnet vượt chính teacher về đánh đổi độ chính xác/thông lượng.

3. Phương pháp chi tiết

3.1 Soft vs hard distillation

Gọi Z_t, Z_s là logit teacher/student, ψ là softmax, τ nhiệt độ, λ hệ số cân bằng, y nhãn thật.





DeiT soft vs hard distillation và distillation token x_{dist} song song class token; late fusion lúc suy luận (minh hoạ, tự tính)

Soft distillation (Hinton):

$$\mathcal{L}_{global} = (1-\lambda)\,\mathcal{L}_{CE}(\psi(Z_s), y) + \lambda\tau^2\,\mathrm{KL}(\psi(Z_s/\tau), \psi(Z_t/\tau))$$

Paper dùng $\tau = 3.0$, $\lambda = 0.1$ cho kiểu mềm (theo khuyến nghị của Cho et al.).

Hard-label distillation: lấy quyết định cứng của teacher $y_t = \arg\max_c Z_t(c)$ làm nhãn thứ hai:

$$\boxed{\mathcal{L}_{global}^{hardDistill} = \tfrac{1}{2}\mathcal{L}_{CE}(\psi(Z_s), y) + \tfrac{1}{2}\mathcal{L}_{CE}(\psi(Z_s), y_t)}$$

Không có tham số nào để chỉnh, và y_t thay đổi theo từng phép augmentation (vì teacher xem ảnh đã crop). Khi dùng nhãn thật, paper dùng label smoothing $\epsilon = 0.1$.

3.2 Distillation token

Chuỗi đầu vào gồm $N+1$ token của ViT được mở rộng thành $N+2$: [class] + [distillation] + N patch (với 224x224, patch 16: $N = 14 \times 14 = 196$). Distillation token được khởi tạo và học bằng back-prop như class token; đầu ra cuối của nó đi qua một bộ phân loại tuyến tính riêng, và loss của nó là $\mathcal{L}_{CE}(\psi(Z_s^{dist}), y_t)$ (kiểu cứng). Class token tiếp tục dự đoán y.

Quan sát của tác giả: hai token hội tụ về hai vector khác nhau (cosine trung bình 0.06 ở đầu vào), qua các tầng dần giống nhau (0.93 ở tầng cuối nhưng vẫn <1). Ngược lại hai *class token* cùng nhãn thì hội tụ thành một vector (cosine 0.999) và không đem lại gì, cho thấy phần cải thiện thực sự đến từ mục tiêu khác biệt, không phải từ việc thêm token.

Lúc kiểm tra: **late fusion**, cộng softmax của hai bộ phân loại (class, distillation).

Fine-tune độ phân giải cao (384): dùng cả nhãn thật lẫn dự đoán teacher (teacher cùng độ phân giải); chỉ nhãn thật làm giảm lợi ích. Nội suy bicubic cho positional embedding (bilinear làm giảm chuẩn ℓ_2 vector, gây sụt chính xác).

3.3 Pseudo-code

```
# teacher: RegNetY-16GF đã huấn luyện (đóng băng), student: DeiT-x
for (img, y) in loader:
    img = augment(img)                    # RandAug, Mixup, CutMix, Erasing, repeated aug
```

```
    y_t = argmax(teacher(img))            # nhãn cứng của teacher cho chính ảnh đã
    ↪  augment
    tokens = [CLS, DIST, patch_1..patch_N] + pos_emb
    cls_out, dist_out = student(tokens)   # hai đầu tuyến tính
    loss = 0.5*CE(cls_out, y) + 0.5*CE(dist_out, y_t)
    loss.backward(); adamw_step()
# inference
pred = argmax( softmax(cls_out) + softmax(dist_out) )
```

3.4 Các kiến trúc (Table 1)

Mô hình	embedding	head	tầng	tham số	throughput (ảnh/s)
DeiT-Ti	192	3	12	5M	2536
DeiT-S	384	6	12	22M	940
DeiT-B (= ViT-B)	768	12	12	86M	292

Chiều mỗi head cố định 64. Throughput đo trên một V100 16GB, ảnh 224^2.

4. Thí nghiệm và kết quả chính

Dữ liệu và huấn luyện: ImageNet-1k (1.281.167 ảnh train, 50.000 val). 300 epoch (hoặc 1000), pre-train 224^2, fine-tune 384^2 (25 epoch). DeiT-B 300 epoch mất khoảng 37 giờ với 2 máy hoặc 53 giờ với 1 máy; fine-tune 384 thêm 20 giờ. Teacher mặc định: RegNetY-16GF (84M tham số, 82.9% top-1) huấn luyện cùng augmentation.

Loại teacher (Table 2, student DeiT-B⚗, top-1):

Teacher	acc teacher	student pretrain	student ↑384
DeiT-B	81.8	81.9	83.1
RegNetY-4GF	80.0	82.7	83.6
RegNetY-8GF	81.7	82.7	83.8
RegNetY-12GF	82.4	83.1	84.1
RegNetY-16GF	82.9	83.1	84.2

Convnet dạy tốt hơn transformer; teacher mạnh hơn cho student mạnh hơn.

So sánh phương pháp distillation (Table 3, 300 epoch, top-1 ImageNet):

Phương pháp	nhãn	teacher	Ti 224	S 224	B 224	B↑384
không distill	có	không	72.2	79.8	81.8	83.1
distill thường (mềm)	không	mềm	72.2	79.8	81.8	83.2
distill cứng	không	cứng	74.3	80.9	83.0	84.0
DeiT⚗ chỉ class	có	cứng	73.9	80.9	83.0	84.2
DeiT⚗ chỉ distil.	có	cứng	74.6	81.1	83.1	84.4
DeiT⚗ class+distil.	có	cứng	74.5	81.2	83.4	84.5

Đánh đổi độ chính xác/tốc độ (Table 5, trích):

Mô hình	tham số	ảnh	ảnh/s	top-1
EfficientNet-B0	5M	224^2	2694.3	77.1
ResNet-50	25M	224^2	1226.1	76.2
ViT-B/16 [Dosovitskiy]	86M	384^2	85.9	77.9
DeiT-Ti	5M	224^2	2536.5	72.2
DeiT-S	22M	224^2	940.4	79.8
DeiT-B	86M	224^2	292.3	81.8
DeiT-Ti⚗	6M	224^2	2529.5	74.5
DeiT-S⚗	22M	224^2	936.2	81.2
DeiT-B⚗	87M	224^2	290.9	83.4
DeiT-B⚗ 1000 epoch	87M	224^2	290.9	84.2
DeiT-B⚗ ↑384	87M	384^2	85.8	84.5
DeiT-B⚗ ↑384, 1000 epoch	87M	384^2	85.8	**85.2**

(Dòng ViT-B/16 là con số của bài ViT gốc trong cùng bảng; bản ViT-B pre-train JFT-300M ở 384^2 là 84.15 theo văn bản. DeiT-B↑384 không distill: 83.1.) Văn bản nói +6.3% top-1 so với ViT huấn luyện trên ImageNet-1k thuần trong thiết lập tương đương, và DeiT-B⚗ 85.2% hơn ViT-B pre-train JFT-300M (84.15%) khoảng 1%.

Transfer (Table 7, trích): DeiT-B⚗↑384: CIFAR-10 99.2, CIFAR-100 91.4, Flowers 98.9, Cars 93.9, iNat-18 80.1, iNat-19 83.0; ngang các convnet tốt nhất. Huấn luyện từ đầu trên CIFAR-10: RegNetY-16GF 98.0, DeiT-B 97.5, DeiT-B⚗ 98.5.

5. Phân tích / ablation

- **Cứng thắng mềm cho transformer:** ngay cả khi chỉ dùng class token, distill cứng đạt 83.0% (224^2) so với distill mềm 81.8%, tức mềm không hơn không distill (81.8). Paper không giải thích rõ lý do; lập luận cạnh đó là nhãn cứng phản ánh trực tiếp tác động augmentation, và hard label không có siêu tham số.
- **Distillation token thêm lợi ích:** DeiT⚗ class+distil. (83.4) cao hơn distill cứng thường (83.0). Hai token cho thông tin bổ sung; bộ phân loại chung tốt hơn từng bộ riêng.
- **Phân tích đồng thuận (Table 4):** tỉ lệ ảnh phân loại khác nhau: ground truth vs convnet 0.171, vs DeiT (không distill) 0.182; DeiT⚗ class+distil. với convnet 0.102 so với DeiT thường 0.133; đầu distillation gần convnet hơn đầu class (0.100 so với 0.112). Student distill thực sự "nhập" hành vi của convnet.
- **Số epoch:** distill hưởng lợi từ huấn luyện dài (300 → 1000 epoch: DeiT-B⚗ 83.4 → 84.2); không distill bão hòa khoảng 400 epoch.
- **Thành phần của recipe (Table 8):** bảng ablation trong bản text bị lẫn cột (ký hiệu có/không của từng thành phần không thể gán chắc chắn cho từng hàng), nên chỉ ghi lại những gì đọc được rõ: mặc định (DeiT-B) 81.8 / 83.1 (224^2 / sau fine-tune 384^2); đổi optimizer pre-train sang SGD cho 74.5 / 77.3; một số cấu hình bỏ thành phần (stochastic depth hoặc random erasing, theo thứ tự trong bảng) không hội tụ (4.3 và 3.4, tác giả nghi do siêu tham số chưa thích nghi). Lời văn của bài báo: gần như mọi phép augmentation đều có ích, **ngoại trừ dropout (bị loại)**; repeated augmentation là "một trong những thành phần then chốt"; AdamW tốt nhất với weight decay nhỏ hơn nhiều so với ViT gốc (0.05 so với 0.3).
- **EMA:** lợi ích nhỏ (khoảng 0.1) và biến mất sau fine-tune.
- **Độ phân giải fine-tune (Table 10):** 160^2 79.9, 224^2 81.8, 320^2 82.7, 384^2 83.1.

6. Giới hạn

Do tác giả nêu/hàm ý: transformer cần augmentation mạnh; hyper-parameter chỉ cross-validate vài giá trị; "có thể chưa tối ưu". Chọn teacher là RegNetY do tác giả huấn luyện với cùng recipe (đã có tối ưu mạnh, dấu ? trong bảng), kết quả phụ thuộc điều này.

Nhận xét:

- Chưa giải thích được tại sao hard > soft; chỉ là giả thuyết (inductive bias, tương tác với augmentation). Với ViT nhỏ hơn và teacher mạnh hơn rất nhiều, hard label có thể mất thông tin.
- Mọi kết quả là phân loại ImageNet; chưa có dự đoán cấu trúc/dense output. Distill nhãn lớp (1000 chiều) khác xa distill chuỗi token của decoder.
- Distillation token đòi hỏi chỉnh kiến trúc (thêm token + head), nên không áp dụng "nguyên xi" cho mô hình đã huấn luyện sẵn mà không fine-tune.
- Teacher và student có cùng input resolution; chưa xét teacher lớn hơn nhiều (ví dụ 4B) so với student.

7. Liên hệ với các bài khác

- **02 Hinton KD**: DeiT thử soft distillation đúng công thức Hinton ($\tau = 3$, $\lambda = 0.1$) và cho thấy nó không giúp ViT trong thiết lập này.
- **06 Born-again networks**: student cùng kiến trúc hơn teacher; DeiT-B⚗ từ DeiT-B cho 81.9 so với 81.8 (rất nhỏ), nhưng từ convnet cho 83.1+; tức khác kiến trúc/inductive bias mới là điều quan trọng.
- **10 Does KD really work, 09 Efficacy of KD**: bàn về việc student có khớp teacher; Table 4 của DeiT là một phép đo "agreement" tương tự.
- **22 DINO**: cùng nhóm tác giả/ViT, không nhãn; **23 AM-RADIO**: nén/gộp nhiều teacher vào ViT; **24 CLIP-KD**: distill cho encoder thị giác-ngôn ngữ.
- **21 DeiT với 14 Decoupled KD / 31 Rethinking KL**: các bài sau phân tích loại loss (NCKD/TCKD, forward/reverse KL) lý giải một phần vì sao hard label có thể hiệu quả; DeiT chỉ báo cáo hiện tượng.

8. Tóm tắt công thức / khái niệm

Khái niệm	Nội dung
Soft distillation	$(1-\lambda)CE(\psi(Z_s), y) + \lambda\tau^2 KL(\psi(Z_s/\tau), \psi(Z_t/\tau)), \tau = 3, \lambda = 0.1$
Hard distillation	$\frac{1}{2}CE(\psi(Z_s), y) + \frac{1}{2}CE(\psi(Z_s), y_t), y_t = \arg\max Z_t$
Distillation token	Token học được, mục tiêu = nhãn cứng teacher; tương tác qua self-attention
Late fusion	Cộng softmax của đầu class và đầu distillation khi suy luận
Inductive bias	Giả định cấu trúc của kiến trúc (convnet: cục bộ); distill truyền "mềm"
Kết quả chính	DeiT-B 83.1 (không ngoài); DeiT-B⚗↑384 1000 epoch 85.2
Phát hiện chính	Hard > soft cho transformer; teacher convnet > teacher transformer

9. Kết luận

DeiT kết hợp nhãn cứng từ convnet teacher với distillation token để huấn luyện ViT. Các thí nghiệm phân loại ảnh cho thấy ViT có thể đạt đánh đổi độ chính xác và tốc độ ngang hoặc hơn convnet trong thiết lập được khảo sát.

Link gốc: https://arxiv.org/abs/2012.12877

Bài 22

Emerging Properties in Self-Supervised Vision Transformers (2021)

Tác giả: Mathilde Caron, Hugo Touvron, Ishan Misra, Hervé Jégou, Julien Mairal, Piotr Bojanowski, Armand Joulin (Facebook AI Research, Inria, Grenoble). **Venue / arXiv:** arXiv:2104.14294 (paper được biết tới là ICCV 2021; bản text không nêu venue). Phương pháp có tên **DINO** (self-**di**stillation with **no** labels). **Nguồn:** https://arxiv.org/abs/2104.14294 **Chủ đề:** Học biểu diễn tự giám sát cho Vision Transformer bằng "self-distillation không nhãn": student học khớp đầu ra của một teacher là **trung bình động (EMA)** của chính student; centering + sharpening chống sụp đổ. **Vị trí trong lộ trình:** Đọc trước: 02 (Hinton KD, nhất là soft target và temperature), 06 (Born-again networks, student cùng kiến trúc với teacher), 21 (DeiT, ViT + distillation). Bài này thêm: distillation **không cần teacher có sẵn và không cần nhãn**; teacher được dựng từ chính student. Bài kế: 23 (AM-RADIO) dùng DINOv2 (hậu duệ của DINO) như một trong các teacher cho vision encoder.

> Ghi chú học tập bằng tiếng Việt, không phải bản dịch nguyên văn. Đây là bài về self-supervised learning (SSL), được xếp vào lộ trình distillation vì tác giả diễn giải DINO là "knowledge distillation không nhãn".

1. Bối cảnh

Trong NLP, Transformer thành công nhờ pre-train **tự giám sát** (BERT, GPT) trên dữ liệu không nhãn. Trong thị giác, ViT thường được huấn luyện có nhãn (như DeiT, 21), và nhãn ảnh thường rút gọn cả bức ảnh phong phú thành một khái niệm trong vài nghìn lớp. Tác giả hỏi: ViT có thu được tính chất mới nếu pre-train tự giám sát?

Các phương pháp SSL trước đó (SimCLR, MoCo, BYOL, SwAV) đều phải tránh **sụp đổ (collapse)**: mô hình cho đầu ra hằng số với mọi ảnh thì loss "tốt" nhưng vô dụng. Mỗi phương pháp có một cách: so sánh tương phản (contrastive) với nhiều mẫu âm, ràng buộc clustering (Sinkhorn-Knopp), predictor + batch norm (BYOL).

Khái niệm cần nhớ:

- **Self-supervised**: tự tạo nhiệm vụ từ dữ liệu, không dùng nhãn người gán.
- **Multi-crop**: từ một ảnh cắt vài *global view* (phủ vùng lớn, độ phân giải 224) và nhiều *local view* (vùng nhỏ, 96).
- **EMA / momentum encoder**: tham số teacher là trung bình trượt mũ của tham số student, $\theta_t \leftarrow \lambda\theta_t + (1-\lambda)\theta_s$.
- **Stop-gradient**: không lan truyền gradient qua teacher.

2. Ý tưởng chính

DINO nhìn bài toán như distillation: hai mạng **cùng kiến trúc**, student và teacher. Cả hai nhận các góc nhìn (crop) khác nhau của cùng một ảnh. Teacher chỉ nhận global view, student nhận mọi view; student phải dự đoán phân phối đầu ra của teacher (loss cross-entropy) nên học quan hệ "cục bộ tới toàn cục". Khác distillation thường, **không có teacher huấn luyện sẵn**: teacher là EMA của student, và theo quan sát của tác giả teacher này tốt hơn student suốt quá trình huấn luyện (giống ensemble Polyak-Ruppert). Để tránh sụp đổ chỉ cần hai phép trên đầu ra teacher: trừ một *tâm* (centering) và dùng softmax nhiệt độ thấp (sharpening). Kết quả bất ngờ: đặc trưng ViT chứa bản đồ phân đoạn đối tượng trong attention của tầng cuối, và k-NN thẳng trên đặc trưng đóng băng đã đạt rất cao.

3. Phương pháp chi tiết

3.1 Loss

Mạng $g = h \circ f$ gồm backbone f (ViT hoặc ResNet) và projection head h (MLP 3 tầng, ẩn 2048, rồi chuẩn hóa ℓ_2 và một tầng FC có weight-normalization ra K chiều). Đặc trưng dùng cho tác vụ sau là đầu ra của f. Phân phối đầu ra:

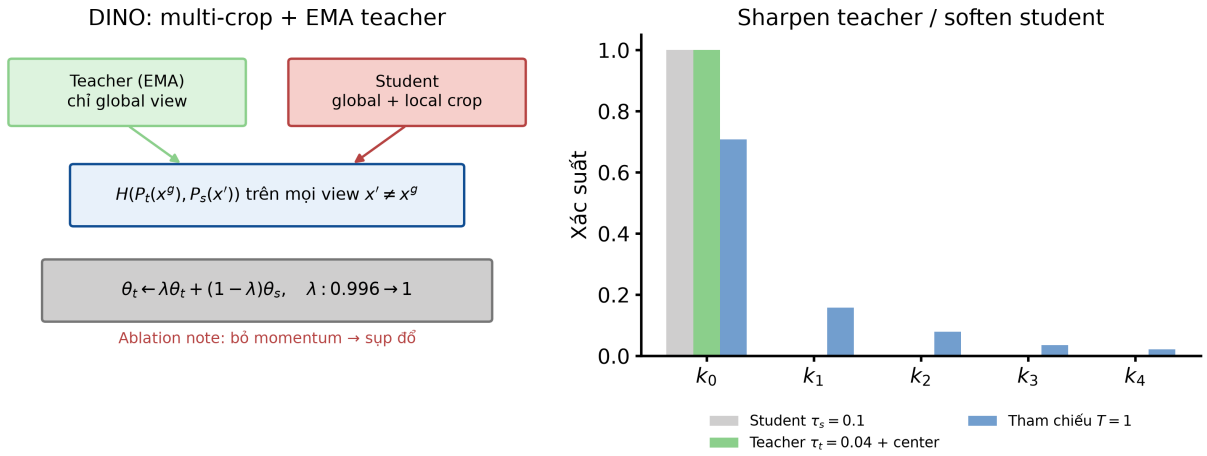
$$P_s(x)^{(i)} = \frac{\exp(g_{\theta_s}(x)^{(i)}/\tau_s)}{\sum_{k=1}^{K} \exp(g_{\theta_s}(x)^{(k)}/\tau_s)}$$

và tương tự P_t với τ_t. Với tập view V (2 global x_1^g, x_2^g cộng các local):

$$\boxed{\min_{\theta_s} \sum_{x \in \{x_1^g, x_2^g\}} \sum_{\substack{x' \in V \\ x' \neq x}} H(P_t(x), P_s(x'))}, \qquad H(a,b) = -a \log b$$

3.2 Cập nhật teacher, centering, sharpening

$$\theta_t \leftarrow \lambda \theta_t + (1-\lambda)\theta_s, \;\; \lambda : 0.996 \to 1 \text{ (cosine)}$$



DINO: teacher EMA của student, multi-crop; centering + sharpening (τ_t nhỏ) chống sụp đổ (minh hoạ, tự tính)

$$g_t(x) \leftarrow g_t(x) + c, \qquad c \leftarrow mc + (1-m)\frac{1}{B}\sum_{i=1}^{B} g_{\theta_t}(x_i)$$

Centering trừ tâm trung bình theo batch (EMA) khỏi đầu ra teacher; sharpening dùng τ_t nhỏ. Trực giác: centering ngăn *một chiều thống trị* nhưng đẩy tới phân phối đều; sharpening làm ngược lại; dùng cả hai thì cân bằng.

3.3 Pseudo-code (Algorithm 1 của bài báo, bỏ multi-crop)

```
gt.params = gs.params
for x in loader:
    x1, x2 = augment(x), augment(x)
    s1, s2 = gs(x1), gs(x2)          # student
    t1, t2 = gt(x1), gt(x2)          # teacher (stop-grad)
```

```
    loss = H(t1, s2)/2 + H(t2, s1)/2
    loss.backward(); update(gs)       # SGD/AdamW
    gt.params = l*gt.params + (1-l)*gs.params
    C = m*C + (1-m)*cat([t1,t2]).mean(dim=0)

def H(t, s):
    t = t.detach()
    s = softmax(s / tps, dim=1)
    t = softmax((t - C) / tpt, dim=1)   # center + sharpen
    return -(t * log(s)).sum(dim=1).mean()
```

3.4 Chi tiết huấn luyện

AdamW, batch 1024 (16 GPU với ViT-S/16), lr $= 0.0005 \times$ batch$/256$, warmup 10 epoch rồi cosine; weight decay cosine 0.04 đến 0.4; $\tau_s = 0.1$, τ_t tăng tuyến tính từ 0.04 lên 0.07 trong 30 epoch đầu; augmentation kiểu BYOL (color jitter, Gaussian blur, solarization). Với ViT **không dùng BatchNorm** ở cả backbone lẫn head (hệ thống hoàn toàn BN-free). Cấu hình mạng (Table 1): ViT-S/16 có 12 block, dim 384, 6 head, 21M tham số, 1007 ảnh/s; ViT-B/16 85M, 312 ảnh/s; ViT-S/8 785 token, 180 ảnh/s; ViT-B/8 63 ảnh/s; ResNet-50 23M, 1237 ảnh/s.

Đánh giá: linear probe (bộ phân loại tuyến tính trên đặc trưng đóng băng) hoặc **k-NN có trọng số** (20 láng giềng, không cần siêu tham số).

4. Thí nghiệm và kết quả chính

ImageNet (Table 2, top-1, linear / k-NN):

Phương pháp	Kiến trúc	Tham số	Linear	k-NN
Supervised	RN50	23M	79.3	79.3
SwAV	RN50	23M	75.3	65.7
DINO	RN50	23M	75.3	67.5
Supervised	ViT-S	21M	79.8	79.8
BYOL*	ViT-S	21M	71.4	66.6
MoCov2*	ViT-S	21M	72.7	64.4
SwAV*	ViT-S	21M	73.5	66.3
DINO	ViT-S	21M	**77.0**	**74.5**
DINO	ViT-B/16	85M	78.2	76.1
DINO	ViT-S/8	21M	79.7	78.3
DINO	ViT-B/8	85M	**80.1**	77.4

(* do tác giả tự chạy.) Với ViT, DINO hơn BYOL/MoCov2/SwAV +3.5% (linear) và +7.9% (k-NN). Hiện tượng k-NN gần bằng linear (74.5 so với 77.0) chỉ xuất hiện với DINO + ViT, không với ResNet-50. ViT-B/8 80.1% linear với 10x ít tham số và nhanh hơn 1.4x so với SOTA trước đó (SimCLRv2, theo paper). Huấn luyện ViT-S/16 bằng 2 máy 8 GPU trong 3 ngày đạt 76.1% linear.

Các tác vụ khác: truy hồi ảnh (Oxford/Paris revisited, mAP, ViT-S/16 ImageNet: M 41.8 / H 13.7 trên ROx, tốt hơn supervised 33.5 / 8.9; pre-train trên GLDv2 lên 51.5 / 24.3); copy detection Copydays ("strong" subset, mAP): supervised ViT-B/16 76.4, DINO ViT-B/16 81.7, DINO ViT-B/8 85.5; phân đoạn đối tượng video DAVIS-2017 (không fine-tune): DINO ViT-S/8 $(J\&F)_m$ = 69.9, ViT-B/8 = 71.4, supervised ViT-S/8 = 66.0; transfer bằng fine-tune (Table 6): ví dụ INat18 ViT-S/16 supervised 70.7 vs DINO 72.0, ImageNet 79.9 vs 81.5.

Phân đoạn từ attention: thresholding attention của token [CLS] ở tầng cuối (giữ 60% khối lượng) cho mask đối tượng; Jaccard trên PASCAL VOC12: ngẫu nhiên 22.0, supervised ViT-S/16 27.3, DINO ViT-S/16 45.9 (ViT-S/8: 21.8, 23.7, 44.7).

5. Phân tích / ablation

Thành phần quan trọng (Table 7, ViT-S/16, 300 epoch, k-NN / linear):

Biến thể	k-NN	Linear
DINO mặc định (momentum, multi-crop, CE, không predictor)	72.8	76.1
Bỏ momentum	0.1	0.1 (sụp đổ)
Thêm Sinkhorn-Knopp	72.2	76.0
Bỏ multi-crop	67.9	72.5
Loss MSE thay cross-entropy	52.6	62.4
Thêm predictor	71.8	75.6

Kết luận: **momentum teacher bắt buộc**, multi-crop và cross-entropy quan trọng, predictor và Sinkhorn-Knopp gần như không giúp.

Patch nhỏ hơn tốt hơn mà không thêm tham số: ViT-S k-NN tăng đáng kể khi patch giảm 16 thành 8 thành 5, nhưng throughput giảm (8x8: 180 ảnh/s; 5x5: 44 ảnh/s).

Loại teacher (k-NN): momentum 72.8; teacher là student của epoch trước 66.6; teacher là bản sao student hay student của bước trước **không hội tụ** (0.1). Teacher momentum **luôn vượt student** trong quá trình train (Fig. 6); tác giả diễn giải là Polyak-Ruppert averaging.

Chống sụp đổ: phân rã $H(P_t, P_s) = h(P_t) + D_{KL}(P_t \| P_s)$. Thiếu một trong hai phép thì KL về 0 (sụp đổ), nhưng entropy h về 0 nếu thiếu centering (một chiều chiếm hết) và về $-\log(1/K)$ nếu thiếu sharpening (đầu ra đều). Dùng cả hai thì cân bằng. Centering chỉ dùng thống kê bậc một của batch nên ít phụ thuộc kích thước batch.

Chi phí (Table 8, ViT-S/16): 2x224² (không multi-crop): 300 epoch đạt 72.5% trong 45.9h, bộ nhớ 9.3G/GPU; 2x224² + 10x96²: 76.1% trong 72.6h, 15.4G; 100 epoch với 2x224²+10x96² đạt 74.6% trong 24.2h. Multi-crop cải thiện đánh đổi độ chính xác/thời gian, bù bằng bộ nhớ.

6. Giới hạn

Do tác giả nêu: hướng tương lai là pre-train ViT lớn trên ảnh ngẫu nhiên không biên tập; patch nhỏ làm tăng chi phí/bộ nhớ; còn "không gian khám phá các teacher thay thế" (teacher epoch trước chạy được dù kém hơn).

Nhận xét:

- Không phải distillation "để nén": student và teacher cùng kích thước; mục tiêu là học biểu diễn, không phải chuyển tri thức từ mô hình lớn sang nhỏ.
- Thực nghiệm chủ yếu là phân loại/truy hồi ảnh tự nhiên (ImageNet); không có chứng cứ trên ảnh tài liệu/văn bản.
- Nhạy với siêu tham số (nhiệt độ, momentum, số crop); sụp đổ là rủi ro nếu cấu hình sai.
- k-NN và phân đoạn qua attention là chỉ số đặc trưng; mức độ chúng dự báo chất lượng cho một tác vụ cụ thể như OCR chưa được kiểm chứng.

7. Liên hệ với các bài khác

- **02 Hinton KD**: DINO dùng chính ý tưởng soft target + temperature; khác là "teacher" học cùng lúc.
- **06 Born-again networks, 07 Deep mutual learning**: student cùng kiến trúc với teacher; DINO khác ở chỗ teacher là EMA của student và không có nhãn.
- **21 DeiT**: cùng ViT/cùng nhóm tác giả; DeiT distill từ convnet có nhãn qua distillation token, DINO không nhãn. Cả hai nhắc cùng triển khai ViT của DeiT.
- **23 AM-RADIO**: dùng DINOv2 (kế thừa DINO) làm một trong ba teacher; DINO-style features là "teacher" cho nén tuần tự.
- **24 CLIP-KD**: distillation cho mô hình ngôn ngữ-hình ảnh, tập trung vào mô hình ngôn ngữ–hình ảnh.
- Kỹ thuật EMA-teacher cũng liên quan 11 (Patient and consistent teacher): nhất quán giữa các view giống nhất quán giữa student và teacher.

8. Tóm tắt công thức / khái niệm

Khái niệm	Nội dung
Student / Teacher	Cùng kiến trúc; teacher = EMA của student, chỉ nhận global view
Loss	$\sum_{x\in\{x_1^g,x_2^g\}}\sum_{x'\neq x} H(P_t(x),P_s(x'))$
EMA	$\theta_t \leftarrow \lambda\theta_t + (1-\lambda)\theta_s$, $\lambda : 0.996 \rightarrow 1$
Centering	$g_t \leftarrow g_t + c$, c là EMA của trung bình batch (momentum m)
Sharpening	τ_t nhỏ (0.04 lên 0.07), $\tau_s = 0.1$
Multi-crop	2 global 224^2 + nhiều local 96^2
Kết quả chính	ViT-B/8: 80.1% linear; ViT-S/16: k-NN 74.5 vs linear 77.0
Ablation chính	Bỏ momentum: sụp đổ; bỏ multi-crop: 72.8 xuống 67.9 k-NN

9. Kết luận

DINO dùng teacher cập nhật bằng EMA để self-distillation không nhãn, cùng centering và sharpening nhằm tránh sụp đổ biểu diễn. Với ViT, bài báo ghi nhận attention mang thông tin phân đoạn và kết quả k-NN mạnh. Đây cũng là nền tảng để đọc các teacher thị giác như DINOv2 trong bài AM-RADIO.

Link gốc: https://arxiv.org/abs/2104.14294

Bài 23

AM-RADIO: Agglomerative Vision Foundation Model -- Reduce All Domains Into One (2024)

Tác giả: Mike Ranzinger, Greg Heinrich, Jan Kautz, Pavlo Molchanov (NVIDIA). **Venue / arXiv:** arXiv:2312.06709 (bản v5, 30/4/2024); CVPR 2024 (ghi ở đầu bản text). Mô hình student tên **RADIO**, biến thể hiệu quả **E-RADIO**. **Nguồn:** https://arxiv.org/abs/2312.06709 **Chủ đề:** Distillation **nhiều teacher** ở mức **feature** (không nhãn, không logit) để gộp CLIP, DINOv2 và SAM vào một vision encoder; so sánh nhiều kiến trúc nhanh làm student; kiến trúc lai CNN-Transformer E-RADIO. **Vị trí trong lộ trình:** Đọc trước: 03 (FitNets: bắt chước feature trung gian), 21 (DeiT), 22 (DINO, tiền thân của teacher DINOv2). Bài này thêm: khung feature distillation cho **vision encoder**, xử lý teacher bất đồng nhất (độ phân giải, chiều feature, tác vụ), và một thí nghiệm hiếm hoi so sánh nhiều kiến trúc student dưới cùng recipe distill. Bài kế: 24 (CLIP-KD), 25 (LLaVA-MoD), 26 (LLaVA-KD) cho nén VLM.

> Ghi chú học tập bằng tiếng Việt, không phải bản dịch nguyên văn. Đây là bài **dài, nhiều bảng**; tập trung vào phần distillation, ablation và kiến trúc student, bỏ qua chi tiết phụ trong phụ lục. Một số bảng ablation (Table 2, 3, 5 một phần) bị lỗi khi trích PDF nên chỉ ghi lại những số đọc rõ, và nói rõ khi bỏ qua.

1. Bối cảnh

Vision foundation model (VFM) là bộ mã hóa ảnh lớn, đa dụng, dùng làm backbone cho nhiều tác vụ. Ba mô hình tiêu biểu có thế mạnh khác nhau:

- **CLIP** (huấn luyện trên cặp ảnh-chú thích): mạnh về zero-shot và ngữ nghĩa mức ảnh, tương thích với một mô hình văn bản.
- **DINOv2** (tự giám sát, kế thừa DINO, 22): mạnh ở tác vụ dày (dense) như phân đoạn ngữ nghĩa với feature tốt theo không gian.
- **SAM** (Segment Anything): mạnh ở phân đoạn instance, biên đối tượng; nhưng như tác giả nhận xét, "out of the box" không phù hợp tác vụ downstream khác.

Mỗi mô hình có "lỗ hổng". Distillation truyền thống thường nén một teacher thành student nhỏ hơn, có cả logit lẫn nhãn. Ở đây teacher không đồng nhất (khác tác vụ, độ phân giải đầu vào, patch size, chiều feature) nên không thể dùng logit hay nhãn chung. Cách làm: **bắt chước feature** (intermediate activations) và để mỗi teacher có "đầu" riêng.

Khái niệm: **summary feature** = vector toàn ảnh (class token); **spatial/dense feature** = lưới vector theo patch; **adaptor head** = mạng nhỏ chiếu feature student sang không gian feature của từng teacher; **linear probe** = huấn luyện một bộ phân loại tuyến tính trên feature đóng băng để đo chất lượng biểu diễn; **throughput** = số ảnh/giây.

2. Ý tưởng chính

Huấn luyện một student **từ đầu** (không khởi tạo từ teacher) trên ảnh không nhãn (DataComp-1B). Mỗi ảnh đi qua student một lần và qua từng teacher; student có, cho mỗi teacher, hai "đầu" MLP nhỏ (một cho summary, một cho feature theo patch), và loss là khoảng cách cosine (kèm smooth-L1 cho feature patch) giữa đầu ra của đầu và feature thật của teacher. Mọi teacher được cân bằng đơn giản (trọng số 1). Khi student đủ dung lượng, nó vượt từng teacher trên nhiều chỉ số, đồng thời "thừa hưởng" khả năng của teacher: dùng lại mô hình văn bản của CLIP (zero-shot) và dùng lại decoder mặt nạ của SAM (thay encoder). Phần hiệu quả: so sánh hơn 10 kiến trúc nhanh với cùng recipe; kết luận CNN thuần nhanh nhưng khó bắt chước ViT, nên đề xuất E-RADIO (stem conv + C2f + transformer cửa sổ).

3. Phương pháp chi tiết

3.1 Teacher, dữ liệu, student

Teacher dùng cho RADIO cuối (dấu * ở Table 1): DFN CLIP ViT-H/14 (378px), OpenAI CLIP ViT-L/14 (336px), DINOv2 ViT-g/14-reg, SAM ViT-H/16 (1024px). Student chuẩn: **ViT-H/16**; không phụ thuộc kiến trúc teacher. Dữ liệu: DataComp-1B (chỉ lấy ảnh; ImageNet-1k cho metric cao nhất nhưng tác giả cho rằng làm "zero-shot" mất ý nghĩa vì student học feature teacher trên chính miền đánh giá).

3.2 Adaptor heads

Mỗi teacher i có hai đầu: MLP 2 tầng với LayerNorm và GELU ở giữa; chiều vào = embedding student, chiều giữa = embedding lớn nhất của các teacher, chiều ra = chiều feature của teacher đó. Ký hiệu $x' = f(x|\Theta_0)$ là feature student, $y_i = h_i(x'|\Theta_i)$ đầu ra đầu i, $z_i = t_i(x|\Phi_i)$ feature teacher.

3.3 Loss

Summary (dùng class token của CLIP và DINOv2; **không** khớp summary cho SAM, $\lambda_{SAM} = 0$):

$$\boxed{L_{summary}(x) = \sum_i \lambda_i \, L_{cos}(y_i^{(s)}, z_i^{(s)})}$$

Spatial (feature theo patch của mọi teacher):

$$L_{match}(x, y) = \alpha L_{cos}(x, y) + \beta L_{smooth\text{-}\ell_1}(x, y), \qquad \boxed{L_{features}(x) = \sum_i \gamma_i \, L_{match}(h_i^{(v)}(x'),\, t_i^{(v)}(x))}$$

với $\alpha = 0.9$, $\beta = 0.1$; $\gamma_i = 1$ (cân bằng "naive"), $\lambda_{CLIP} = \lambda_{DINO} = 1$. Tổng loss là $L_{summary} + L_{features}$. Smooth-L1 được thêm để **khớp độ lớn vector**, cần thiết để student thay thế được teacher (drop-in) trong các framework gốc; paper cũng **bỏ chuẩn hóa feature** trong loss vì lý do này. Cosine tốt hơn L1, MSE, smooth-L1 đơn lẻ theo thực nghiệm của tác giả (không có bảng riêng).

3.4 Chi tiết kỹ thuật cho teacher bất đồng nhất

- **Multi-scale / độ phân giải:** student ăn 432^2 (tương đương 378^2 với patch 14 của teacher CLIP/DINOv2); DINOv2 chạy ở 224px rồi upsample đầu ra; SAM cần 1024px. Khi tỷ lệ downsample khác nhau (ví dụ teacher ViT-H/14, student ViT-H/16), feature map khác kích thước; loss nội suy song tuyến tính lên độ phân giải lớn hơn.
- **Rank/teacher partitioning:** các nhóm teacher được đặt lên các GPU khác nhau để mỗi GPU có batch và độ phân giải nhất quán; có SAM thì 64 GPU, nửa số GPU xử lý CLIP+DINOv2 (batch 32/GPU, 432px), nửa còn lại xử lý SAM (batch 2/GPU, 1024px); batch hiệu dụng 1.152. CLIP+DINOv2 một mình: 32 GPU, batch 1024.
- **Huấn luyện chung:** AdamW, batch 1024, cosine schedule, lr cơ sở 0.001, 600k bước (614M mẫu). Có huấn luyện hai giai đoạn (đoạn đầu với CLIP+DINOv2 trong 300k bước; phần mô tả tiếp theo bị cắt trong bản text nên không trích số).
- **CPE (Cropped Position Embedding):** student có bảng position embedding cỡ 128×128 vị trí, cắt ngẫu nhiên và nội suy cho khớp số patch; cho phép độ phân giải/aspect ratio tùy ý và giảm artifact tần số cao.
- **Cửa sổ attention thành augmentation:** để huấn luyện với SAM tiết kiệm bộ nhớ, thay global attention bằng windowed attention với kích thước cửa sổ lấy mẫu ngẫu nhiên khi huấn luyện; ở inference chọn được cửa sổ. Thêm spectral reparametrization và weight decay 0.02 để tránh attention entropy collapse khi huấn luyện độ phân giải cao.

3.5 Pseudo-code

```
teachers = [CLIP, DINOv2, SAM]   # đóng băng, mỗi teacher có (res, patch, dim) riêng
student  = ViT-H/16 + CPE + CLS token   # từ đầu
heads[i] = {summary: MLP2(LN,GELU), spatial: MLP2(LN,GELU)}  for each teacher i
for img in DataComp-1B:
    x' = student(resize(img, student_res))
    loss = 0
    for i in teachers:
        z = teacher_i(resize(img, teacher_res_i))        # summary + spatial
        if lambda_i > 0:   loss += lambda_i * cos(heads[i].summary(x'.cls), z.summary)
        y = heads[i].spatial(x'.patches)
        y, z = match_resolution(y, z)                     # bilinear
        loss += gamma_i * (0.9*cos(y, z.spatial) + 0.1*smooth_l1(y, z.spatial))
    loss.backward(); adamw_step()
```

3.6 E-RADIO

Kiến trúc lai: stem conv stride làm giảm ảnh 4x; 2 tầng khối conv YOLOv8 C2f; 2 tầng transformer dùng windowed attention (như Swin) xen kẽ với attention "toàn cục" kiểu EdgeViT (downsample 2x bằng conv stride 2 3x3, windowed attention, upsample lại). Cuối cùng upsample feature 2x bằng deconv và cộng với feature stage 3, nên độ giảm không gian tổng chỉ còn 16x (cải thiện tác vụ dày, chỉ chậm khoảng 10%).

4. Thí nghiệm và kết quả chính

So sánh với teacher (Table 1, trích; k-NN và zero-shot trên ImageNet-1k; ADE20k/VOC: mIoU linear probe; GQA, POPE, TextVQA, VQAv2: thay encoder trong LLaVA-1.5; COCO: instance segmentation kiểu SAM; throughput trên A100 với TensorRT):

Mô hình	Tham số	Độ phân giải	ảnh/s	Zero-shot	k-NN	ADE20k	VOC	GQA	POPE	TextVQA	VQAv2	COCO
DFN CLIP-H/14*	633M	378	170	83.90	85.27	39.00	70.29	61.73	85.91	56.78	78.78	-
OpenAI CLIP-L/14*	305M	336	414	75.54	79.80	36.51	67.04	62.20	86.09	57.92	78.49	-
DINOv2-g/14-reg*	1,137M	224	294	-	83.41	48.68	82.78	61.88	85.62	47.18	76.23	-
SAM-H/16*	637M	1024	12	-	22.12	28.08	34.34	49.92	81.76	43.91	57.65	77.18
E-RADIO-L	391M	512	468	80.73	83.89	48.22	81.64	61.70	85.07	51.47	76.73	76.31
RADIO-ViT-H/16	653M	432	158	82.93	**86.06**	**51.34**	**84.71**	63.01	86.20	56.32	79.28	76.23

Nhận xét của bài báo: RADIO-ViT-H/16 nhanh gần bằng teacher nhưng vượt trên 6 trong 9 tác vụ; E-RADIO ít nhất nhanh hơn teacher 6x ở độ phân giải khớp. Ở bản text, hàng Intern-ViT-6B bị lẫn nên không trích số.

Thay SAM trực tiếp (Table 8, mIoU COCO / throughput): SAM Base 75.78 / 50.94, Large 77.02 / 20.62, Huge 77.18 / 11.83; E-RADIO Large 76.31 / 121.74; RADIO với ViTDet-H/16 cửa sổ 8: 76.09 / 29.09, cửa sổ 16: 76.23 / 27.91.

VQA (LLaVA-1.5 + Vicuna 7B): thay encoder bằng RADIO; nhờ nhận độ phân giải bất kỳ, resize cạnh dài còn 432px, trung bình 462 token so với 576 token của encoder patch-14 336px (ít hơn 20%).

Kiểm tra nhận thức 3D (Table 9, NAVI, độ chính xác trung bình qua ngưỡng; depth / surface normals / multi-view corr.): DFN CLIP-H/14 52.5 / 23.0 / 20.3; OpenAI CLIP-L/14 53.7 / 25.3 / 20.7; DINOv2-g/14-reg 83.2 / 59.6 / 59.9; SAM-H/16 68.2 / 50.3 / 45.3; RADIO-ViT-H/16 81.0 / 58.5 / 62.1.

5. Phân tích / ablation

Kiến trúc student (Table 7; student huấn luyện trên ImageNet-21k bằng OpenCLIP ViT-H/14 + DINOv2 g/14; throughput TensorRT, A100, batch 128, 224^2; FD loss = feature distillation loss với DINOv2 teacher, tương quan cao với ADE20k mIoU):

Backbone	Tham số	ảnh/s	Zero-shot	k-NN	ADE20k	FD loss
(teacher) DINOv2 G/14	1.14B	313	n/a	83.41	47.53	
(teacher) OpenCLIP H/14	632M	556	77.19	81.10	40.04	
EfficientNetV2-S	21M	9017	65.37	70.72	27.75	0.415
ResNetv2-101	44M	7283	69.58	75.32	29.61	0.405
RegNetY-064	30M	6573	69.84	74.59	28.9	0.394
EfficientViT-L1	38M	6048	71.73	79.90	33.12	0.376
ConvNext-B	88M	1805	75.43	81.73	38.95	0.358
MaxViT-B	119M	1486	77.49	79.34	38.46	0.340
MViTV2-B	51M	975	75.92	81.39	41.39	0.345
E-RADIO-B	118M	6422	75.19	82.21	44.03	0.319
E-RADIO-B không upsample	113M	7040	75.45	82.05	41.26	0.353
E-RADIO-L	265M	3472	77.87	83.73	45.5	0.265

Kết luận của tác giả: nhiều backbone "hiệu quả" tụt xa teacher, CNN thuần nhanh hơn ViT nhưng kém chính xác, và hiệu suất kém trên ADE20k vì giảm không gian 32x (feature map 7×7 với ảnh 224^2). Họ cho rằng nhiều thiết kế bị "over-tuned" cho huấn luyện có giám sát ImageNet-1k nên không scale tốt sang bài toán VFM. FD loss tương quan cao với ADE20k, nghĩa là có thể dùng FD loss làm chỉ báo rẻ cho chất lượng dense.

Pooling của student (Table 6, ViT, CLS token / average pooling; zero-shot, k-NN, ADE20k, VOC, VQAv2): CLS: 78.55 / 83.91 / 49.01 / 83.51 / 77.66; average pooling: 80.12 / 83.83 / 38.36 / 77.04 / 78.28. Avgpool tốt hơn cho summary loss nhưng hại mạnh feature theo patch; tác giả chọn CLS riêng.

CPE (Table 5, ViT-L/14, k-NN / ADE20k): không CPE 82.96 / 47.30; có CPE 82.84 / 48.52. Dù chỉ train ở 224^2, CPE khái quát tới 518^2 và còn vượt DINOv2-g.

Cân bằng loss (Table 4, ViT-B/14, CLIP+DINOv2; zero-shot / k-NN / ADE20k): Naive 70.63 / 79.50 / 44.71; Uncertainty 70.92 / 79.37 / 44.57; AdaLoss 71.31 / 79.77 / 44.36. Khác biệt nhỏ nên chọn naive. AMTML-KD (học trọng số teacher) luôn dồn toàn bộ trọng số vào CLIP và tệ hơn cân bằng thủ công.

Dữ liệu và teacher cho spatial feature (Table 2, 3): bản text bị lỗi cột nên không trích số số. Theo mô tả: ImageNet-1k cho metric ImageNet cao nhất nhưng không dùng vì làm sai lệch "zero-shot"; thêm loss feature

cho một teacher luôn có lợi; DINOv2 cho feature không gian tốt hơn CLIP, và khớp cả hai cho kết quả tốt nhất.

Tóm tắt "Key insights" của bài báo: feature distillation đầy đủ (kể cả spatial) giúp tác vụ dày, được mô tả là cải thiện 18% tương đối trên ADE20K so với teacher; teacher tốt hơn thì student tốt hơn; SAM thô khá kém ở tác vụ cấp cao (ADE20k linear probe: DINOv2 hơn SAM 1.7x, k-NN ImageNet hơn 4x).

6. Giới hạn

Do tác giả nêu: RADIO có hai "chế độ" độ phân giải thấp/cao (Figure 5): ở khoảng 720px sai số MSE giữa đầu DINOv2 của RADIO và DINOv2 thật nhảy vọt kèm đổi hẳn không gian màu, nhiều khả năng do huấn luyện phân vùng giữa CLIP+DINOv2 và SAM; tác giả hứa sửa trong tương lai. Thiết kế adaptor head đơn giản ("leave alternative architectures as future work"). Các cách cân bằng loss phức tạp không giúp rõ.

Nhận xét:

- Student là mô hình lớn (653M cho RADIO-ViT-H/16; E-RADIO-L 391M, 265M ở Table 7). Đây không phải nén xuống cỡ nhỏ (<100M); E-RADIO-B còn 118M. Chưa có kết quả cho student rất nhỏ.
- Chi phí huấn luyện cao: 600k bước, 614M ảnh, 32 đến 64 GPU.
- So sánh nhiều kiến trúc ở Table 7 dùng ImageNet-21k và 2 teacher, khác với setup RADIO cuối (DataComp-1B, 3-4 teacher); kết luận về kiến trúc có thể không chuyển nguyên sang cấu hình cuối.
- Kết quả "vượt teacher" tính trên 6/9 tác vụ ở Table 1 chứ không phải mọi tác vụ; ví dụ TextVQA 56.32 thấp hơn OpenAI CLIP-L/14 57.92 và DFN CLIP 56.78; COCO 76.23 thấp hơn SAM-H 77.18.
- Không có đánh giá trên OCR/ảnh tài liệu; TextVQA là thước đo gần nhất và RADIO không dẫn đầu ở đó.

7. Liên hệ với các bài khác

- **03 FitNets, 04 Attention transfer:** gốc của feature distillation; AM-RADIO mở rộng sang nhiều teacher và bỏ chuẩn hóa để giữ được độ lớn.
- **12 Relational KD, 13 CRD:** các cách khác để khớp feature; AM-RADIO chỉ dùng cosine + smooth-L1 trực tiếp.
- **21 DeiT, 22 DINO:** DINOv2 (kế thừa DINO) là teacher; DeiT là ví dụ trước về chuyển bias từ convnet sang ViT. Kết quả "CNN-like khó bắt chước ViT" ở đây trái chiều thú vị với DeiT (ViT học từ convnet teacher).
- **24 CLIP-KD, 25 LLaVA-MoD, 26 LLaVA-KD:** nén mô hình thị giác-ngôn ngữ; AM-RADIO là "bên encoder" của bài toán đó.
- **08 Teacher assistant, 41 Capacity gap law:** gap dung lượng teacher-student; ở đây không dùng TA, student đủ lớn.
- **Pruning:** AM-RADIO không dùng pruning; chủ đề liên quan là tìm kiến trúc student hiệu quả bằng thực nghiệm thay vì cắt từ teacher.

8. Tóm tắt công thức / khái niệm

Khái niệm	Nội dung
Agglomerative	Gộp nhiều teacher khác loại vào một student
Adaptor head	MLP 2 tầng (LN, GELU), mỗi teacher 2 đầu (summary, spatial)
$L_{summary}$	$\sum_i \lambda_i L_{cos}$; CLIP và DINOv2 $\lambda = 1$, SAM $\lambda = 0$
$L_{features}$	$\sum_i \gamma_i(\alpha L_{cos} + \beta L_{smooth\text{-}\ell_1})$, $\alpha = 0.9, \beta = 0.1, \gamma_i = 1$
Naive balancing	Trọng số bằng nhau; tốt tương đương AdaLoss/Uncertainty
CPE	Position embedding 128^2, cắt ngẫu nhiên + nội suy

Khái niệm	Nội dung
E-RADIO	Stem conv + C2f + transformer cửa sổ + upsample cuối; giảm không gian 16x
Kết quả chính	RADIO-ViT-H/16: k-NN 86.06, ADE20k 51.34, VOC 84.71; E-RADIO-L 468 ảnh/s
Phát hiện	CNN thuần nhanh nhưng khó bắt chước ViT VFM; FD loss tương quan với ADE20k

9. Kết luận

AM-RADIO chưng cất nhiều teacher vào một encoder, kết hợp cosine và smooth-L1 với adaptor head. Các thí nghiệm đánh giá những đặc tính như zero-shot, dự đoán dày và khả năng thay thế encoder của SAM. Student có từ hàng trăm triệu đến hơn nửa tỷ tham số; bài báo chưa thử OCR.

Link gốc: https://arxiv.org/abs/2312.06709

Bài 24

CLIP-KD: An Empirical Study of CLIP Model Distillation (2024)

Tác giả: Chuanguang Yang, Zhulin An, Libo Huang, Junyu Bi, Xinqiang Yu, Han Yang, Boyu Diao, Yongjun Xu (Institute of Computing Technology, CAS).
Venue / arXiv: arXiv:2307.12732 (v1 tháng 7/2023; bản v2 7/5/2024 dùng để ghi chú này). Paper không nêu venue trong phần text đã trích (bản cs.CV).
Nguồn: https://arxiv.org/abs/2307.12732
Chủ đề: Chưng cất (distillation) mô hình CLIP lớn sang CLIP nhỏ; so sánh 6 loại loss: relation, feature (MSE), masked feature, gradient, interactive contrastive, augmented feature.
Vị trí trong lộ trình: Đọc sau 02 (Hinton KD), 03 (FitNets: bắt chước feature) và 12/13 (Relational KD, CRD: bắt chước cấu trúc quan hệ), cùng 23 (AM-RADIO: distill đa teacher cho vision). Bài này là bài đầu tiên trong nhóm "multimodal" (24 → 25 LLaVA-MoD → 26 LLaVA-KD). Nó cho thấy trong không gian embedding chung của CLIP, một loss MSE đơn giản thắng các loss "thông minh" hơn. Bài tiếp theo (25, 26) chuyển sang distill cả mô hình ngôn ngữ-thị giác sinh văn bản.

> Tài liệu học tập bằng tiếng Việt, là ghi chú diễn giải, không phải bản dịch nguyên văn paper.

1. Bối cảnh

CLIP (Contrastive Language-Image Pre-training) huấn luyện hai encoder: một encoder ảnh f_i và một encoder văn bản f_t, sao cho embedding của cặp (ảnh, caption) đúng gần nhau và cặp sai xa nhau. Sau huấn luyện, ta có thể phân loại ảnh "zero-shot" (không cần fine-tune) bằng cách so embedding ảnh với embedding của câu mô tả các lớp.

Vấn đề: CLIP tốt thường lớn (ViT-L/14 có 304M tham số ở encoder ảnh). Rất ít công trình nghiên cứu cách làm CLIP **nhỏ** tốt hơn cho môi trường hạn chế tài nguyên. Công trình gần nhất là TinyCLIP, dựa trên **weight inheritance** (thừa kế trọng số: copy một phần trọng số teacher sang student). Cách đó bắt buộc teacher và student cùng "họ" kiến trúc (ViT-B/32 sang ViT-61M/32, ResNet-101 sang ResNet-30M), nên không thể distill từ ViT sang MobileNet.

Mục tiêu của CLIP-KD: một bộ công thức distill **không phụ thuộc kiến trúc**, vì chỉ dùng embedding đầu ra cuối (final output), không dùng feature trung gian.

Khái niệm nền (gloss ngắn):

- **InfoNCE / contrastive loss**: cross-entropy trên ma trận tương đồng trong một batch, coi cặp đúng là lớp "dương", các cặp còn lại là "âm".
- **Zero-shot**: đánh giá mà không huấn luyện lại trên tập đích.
- **CKA** (Centered Kernel Alignment): thước đo độ giống nhau giữa hai không gian biểu diễn.
- **Recall@K (R@K)**: tỉ lệ truy vấn có đáp án đúng nằm trong top K.

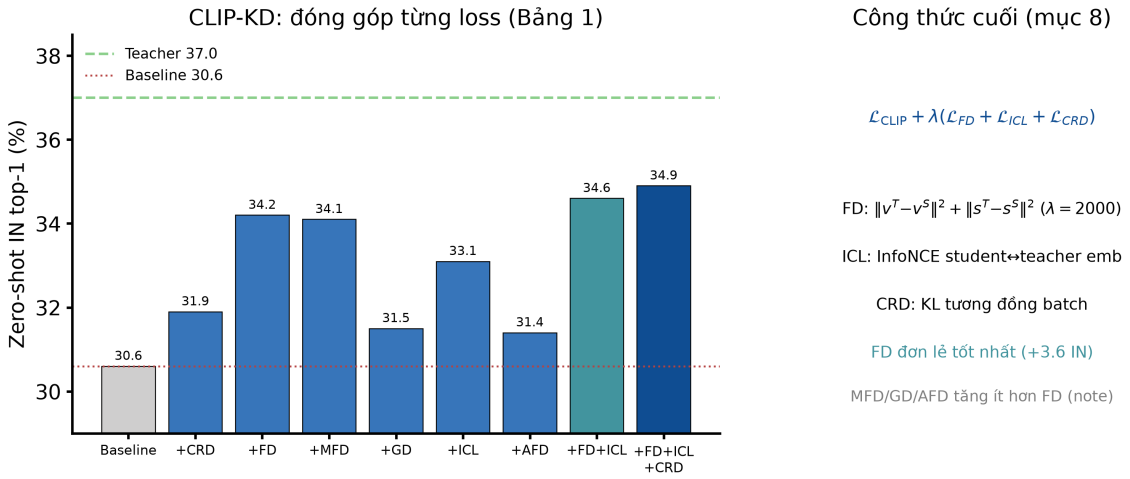
2. Ý tưởng chính

CLIP-KD khảo sát cách chuyển tri thức từ CLIP teacher sang student, bao gồm việc khớp trực tiếp biểu diễn và các mục tiêu dựa trên quan hệ. Các thí nghiệm so sánh kiến trúc, kích thước teacher và thiết kế loss. Cần đọc các kết quả theo từng thiết lập, vì teacher lớn hơn không phải lúc nào cũng cho student tốt hơn.

3. Phương pháp chi tiết

3.1 CLIP loss gốc

Với batch B, embedding đã chuẩn hóa ℓ_2: $v_k = f_i(I_k)$, $s_k = f_t(T_k)$.



CLIP-KD: so sánh sáu loss trên zero-shot IN (Bảng 1 note); FD tốt nhất đơn lẻ, tổ hợp FD+ICL+CRD đạt 34.9 (minh hoạ, tự tính)

$$\mathcal{L}_{I\to T} = -\log \frac{\exp(v_k \cdot s_k/\tau)}{\sum_{b=1}^{|B|}\exp(v_k \cdot s_b/\tau)}, \qquad \mathcal{L}_{CLIP} = \tfrac{1}{2}(\mathcal{L}_{I\to T} + \mathcal{L}_{T\to I})$$

τ là temperature học được (khởi tạo 0.07).

3.2 Sáu loại distillation loss

Ký hiệu: v^T, s^T là embedding ảnh/văn bản của teacher; v^S, s^S của student.

(a) CRD – Contrastive Relational Distillation. Với mỗi ảnh k, tạo phân phối tương đồng ảnh→văn bản trong batch cho teacher và student:

$$p_k^T[j] = \frac{\exp(v_k^T \cdot s_j^T/\tau)}{\sum_b \exp(v_k^T \cdot s_b^T/\tau)}, \quad p_k^S[j] = \frac{\exp(v_k^S \cdot s_j^S/\tau)}{\sum_b \exp(v_k^S \cdot s_b^S/\tau)}$$

Tương tự q_k^T, q_k^S cho văn bản→ảnh. Loss là KL giữa hai phân phối:

$$\mathcal{L}_{CRD}^{I\to T} = \frac{1}{|B|}\sum_k \sum_j p_k^T[j] \log \frac{p_k^T[j]}{p_k^S[j]}, \qquad \mathcal{L}_{CRD} = \mathcal{L}_{CRD}^{I\to T} + \mathcal{L}_{CRD}^{T\to I}$$

Đây là Hinton KD (02) áp dụng lên "logit" là ma trận tương đồng ảnh-văn bản trong batch. Chú ý tên "CRD" ở đây không phải CRD của bài 13 (Contrastive Representation Distillation); đây là Contrastive *Relational* Distillation.

(b) FD – Feature Distillation (MSE trực tiếp lên embedding):

$$\boxed{\mathcal{L}_{FD} = \frac{1}{|B|}\sum_k \left(\|v_k^T - v_k^S\|_2^2 + \|s_k^T - s_k^S\|_2^2\right)}$$

Nếu chiều embedding khác nhau thì thêm một linear projection head cho student.

(c) MFD – Masked Feature Distillation. Như FD nhưng student nhìn ảnh bị che patch (theo MAE) còn teacher nhìn ảnh đầy đủ: $\|v_k^T - \tilde{v}_k^S\|^2 + \|s_k^T - s_k^S\|^2$.

(d) GD – Gradient Distillation. Ép gradient của CLIP loss theo embedding của student giống teacher (MSE):

$$\mathcal{L}_{GD} = \frac{1}{|B|}\sum_k \left(\left\| \frac{\partial \mathcal{L}_{CLIP}^T}{\partial v_k^T} - \frac{\partial \mathcal{L}_{CLIP}^S}{\partial v_k^S} \right\|^2 + \left\| \frac{\partial \mathcal{L}_{CLIP}^T}{\partial s_k^T} - \frac{\partial \mathcal{L}_{CLIP}^S}{\partial s_k^S} \right\|^2 \right)$$

Ý nghĩa: student học "đầu ra nên thay đổi thế nào khi đầu vào đổi", không chỉ giá trị đầu ra.

(e) ICL – Interactive Contrastive Learning. Dùng student làm "anchor", đối chiếu với embedding **của teacher**:

$$\mathcal{L}_{I\to T}^{ICL} = -\log \frac{\exp(v_k^S \cdot s_k^T/\tau)}{\sum_b \exp(v_k^S \cdot s_b^T/\tau)}, \qquad \mathcal{L}_{T\to I}^{ICL} = -\log \frac{\exp(s_k^S \cdot v_k^T/\tau)}{\sum_b \exp(s_k^S \cdot v_b^T/\tau)}$$

$$\mathcal{L}_{ICL} = \tfrac{1}{2}(\mathcal{L}_{I\to T}^{ICL} + \mathcal{L}_{T\to I}^{ICL})$$

Trực giác: ảnh của student phải tìm được caption đúng trong không gian **văn bản của teacher** (và ngược lại). Phụ lục B chứng minh $\mathcal{L}_{ICL}$ cực tiểu hóa thì tối đa hóa cận dưới của thông tin tương hỗ (mutual information) $I(v^S, s^T)$: $I(v^S, s^T) \geq \log N - \mathbb{E}\,\mathcal{L}_{I\to T}^{ICL}$ với $N = |B| - 1$ (cùng cách chứng minh của InfoNCE).

(f) AFD – Augmented Feature Distillation. Nối embedding student và teacher qua một lớp fusion tuyến tính: $v^A = \phi_i(v^S \| v^T)$, $s^A = \phi_t(s^S \| s^T)$, rồi áp loss CLIP thường lên (v^A, s^A).

3.3 Loss tổng

$$\boxed{\mathcal{L}_{CLIP\text{-}KD} = \mathcal{L}_{CLIP} + \lambda\,\mathcal{L}_{KD}}$$

Bản "unified" mặc định: $\mathcal{L}_{KD} = \mathcal{L}_{FD} + \mathcal{L}_{ICL} + \mathcal{L}_{CRD}$ với $\lambda_{FD} = 2000$, $\lambda_{CRD} = 1$, $\lambda_{ICL} = 1$ (và $\lambda_{MFD} = 2000$, $\lambda_{GD} = 10^8$ khi thử các loss đó). Hệ số FD rất lớn vì MSE trên embedding đã chuẩn hóa có độ lớn rất nhỏ (Bảng 6 trong bài báo: ở $\lambda_{FD} = 2000$ loss đi từ 15.880 xuống 0.902).

3.4 Pseudo-code

```
# teacher T (frozen), student S (trainable); batch of (image, text)
for I, Txt in loader:
    vT, sT = T.img(I), T.txt(Txt)            # no_grad, l2-normalized
    vS, sS = S.img(I), S.txt(Txt)            # l2-normalized (+ projection if dims
    ↪  differ)
    L_clip = 0.5*(CE(vS@sS.T/tau, diag) + CE(sS@vS.T/tau, diag))
    L_fd   = mse(vS, vT) + mse(sS, sT)       # sum of squared L2, mean over batch
    L_icl  = 0.5*(CE(vS@sT.T/tau, diag) + CE(sS@vT.T/tau, diag))
    pT_i2t = softmax(vT@sT.T/tau); pS_i2t = softmax(vS@sS.T/tau)  # + text->image
    L_crd  = KL(pT_i2t || pS_i2t) + KL(pT_t2i || pS_t2i)
    loss   = L_clip + 2000*L_fd + 1*L_icl + 1*L_crd
    loss.backward(); opt.step()
```

3.5 Ví dụ nhỏ

Batch 3 cặp. Nếu embedding student của ảnh 1 bằng đúng embedding teacher thì $\mathcal{L}_{FD} = 0$ cho ảnh đó, và student tự động kế thừa mọi quan hệ teacher có với caption khác. Đó là lý do tác giả nói "nếu feature khớp

hoàn hảo thì khoảng cách hiệu năng biến mất". Ngược lại CRD chỉ khớp *phân phối softmax* nên có vô số embedding khác nhau cùng cho một phân phối, tức tín hiệu yếu hơn.

4. Thí nghiệm và kết quả chính

Dữ liệu: pretrain trên Conceptual Captions 3M + 12M (CC3M+12M). Đánh giá: zero-shot ImageNet (IN) và các biến thể IN-V2, IN-R, IN-S; retrieval trên CC3M val (13K cặp), MSCOCO, Flickr. **Huấn luyện:** AdamW, lr 0.001, weight decay 0.1, cosine, warm-up 10K iter, 32 epoch, 8 GPU A800, batch 1024.

Teacher/Student (Bảng 2): teacher ViT-B/16 (86.2M) hoặc ResNet-101 (56.3M) hoặc ViT-L/14 (304M, pretrain Laion-400M); student ViT-T/16 (5.6M), MobileViT-S (5.3M), Swin-T (27.9M), ResNet-18 (11.4M), MobileNetV3 (2.0M), EfficientNet-B0 (4.7M). Text encoder student: 12 layer, width 384 (21.3M) hoặc 512 (37.8M).

Bảng 1 (trích): teacher ViT-B/16, student ViT-T/16, zero-shot IN top-1

Phương pháp	IN Acc
Teacher ViT-B/16	37.0
Student baseline ViT-T/16	30.6
+CRD	31.9
+FD	34.2
+MFD	34.1
+GD	31.5
+ICL	33.1
+AFD	31.4
+FD+ICL	34.6
+FD+ICL+CRD	34.9
+FD+ICL+CRD+GD	34.8
+FD+ICL+CRD+AFD	34.8

Với FD: tăng 3.6 IN, 3.7 R@1 CC3M, 1.9 MSCOCO, 4.2 Flickr (I2T, theo văn bản paper). FD+ICL+CRD tốt nhất trong 6/10 chỉ số.

Bảng 4 (trích): IN-1K, teacher ViT-B/16

Student	Baseline	+CLIP-KD
MobileViT-S	32.6	36.0
Swin-T	36.4	40.2 (teacher 37.0)
MobileNetV3	25.1	27.0
EfficientNet-B0	32.6	35.4
ResNet-18	28.6	31.4

Teacher ResNet-101 (36.8): MobileViT-S 35.0, Swin-T 39.5, MobileNetV3 26.2, EfficientNet-B0 34.6, ResNet-18 30.9. Swin-T sau distill vượt cả teacher; tác giả giải thích do Swin-T vốn mạnh và khoảng cách nhỏ.

Retrieval (Bảng 3, ViT-B/16 teacher, ví dụ Swin-T)

CC3M I2T/T2I: 39.8/39.2 lên 43.7/42.5; MSCOCO 24.7/25.3 lên 28.5/28.6; Flickr 53.4/54.4 lên 62.2/60.9. Văn bản paper: I2T tăng 2.0%~3.9% R@1 trên CC3M (teacher ViT-B/16).

Chuyển từ teacher pretrain Laion-400M (Bảng 5)

Mô hình	IN-1K	Ghi chú
T1 ViT-L/14	72.8	
T2 ViT-B/16	67.1	
S: ViT-T/16 baseline	30.6	
+TinyCLIP (từ T1 / T2)	39.3 / 40.8	
+CLIP-KD (từ T1 / T2)	40.9 / 42.6	
S: ViT-B/16 baseline	37.0	
+TinyCLIP (từ T1)	55.4	
+CLIP-KD (từ T1)	57.5	abstract: hơn baseline 20.5 điểm
S: ResNet-50 baseline	35.3	
+CLIP-KD	55.4	abstract: hơn baseline 20.1 điểm; bảng ghi nhãn (from T2), bảng bị mangled nên không chắc teacher, abstract nói ViT-L/14

Teacher Laion-400M so với teacher CC3M+12M cho ViT-T/16: +7.7 IN và +6.8 retrieval trung bình (văn bản paper). CLIP-KD hơn TinyCLIP trung bình 1.8 IN và 1.7 retrieval.

Downstream (phụ lục)

ResNet-50 pretrain CC3M+12M, Mask-RCNN, linear evaluation trên MS-COCO: AP^{bb} 32.6 lên 34.0; AP^{seg} 29.9 lên 31.1.

5. Phân tích / ablation

- **Mọi loss đều giúp**, nhưng thứ hạng: FD ≈ MFD > ICL > CRD > GD ≈ AFD. MFD (che ảnh) không hơn FD, nên tác giả bỏ nó (Bảng 10: mask ratio 0.25/0.5/0.75 không vượt 0).
- **FD và ICL bổ sung cho nhau**: FD+ICL (34.6) hơn từng loại; thêm CRD lên 34.9; thêm GD hoặc AFD không cải thiện.
- **Vì sao FD thắng:** Hình 3 đo cosine và CKA giữa feature student/teacher sau distill; độ chính xác student "đi cùng" mức độ giống. FD ép giống trực tiếp nên cao nhất; ICL tăng thông tin tương hỗ nên cao thứ hai; CRD, GD, AFD yếu hơn. Đây là tương quan, không phải bằng chứng nhân quả chặt.
- **Trọng số loss:** λ_{FD} tăng thì tốt hơn và bão hòa quanh 2000 (IN: 31.1 ở 10, 33.7 ở 1000, 34.2 ở 2000, 34.1 ở 3000). λ_{CRD} và λ_{ICL} khá bền, chọn 1 vì cùng độ lớn với loss task. λ_{GD} bão hòa ở 10^8.
- **Teacher lớn hơn không tốt hơn:** ViT-L/14 cho student ViT-T/16 kém hơn ViT-B/16 (40.9 so với 42.6 IN). Tác giả đoán là capacity gap và để mở cho nghiên cứu sau.
- **Distill feature trung gian nhạy với kiến trúc:** cặp đồng kiểu (ViT-T/16 từ ViT-B/16) tăng 2.17%, còn cặp khác kiểu (từ ResNet-101) chỉ 0.56% (Hình 5). Vì thế họ chỉ dùng embedding cuối.

6. Giới hạn

Tác giả nêu: teacher lớn hơn có thể kém hơn (capacity gap, chưa giải thích); phân tích chủ yếu thực nghiệm ("empirical study").

Người ghi chú nhận xét (suy luận):

- Chỉ nghiên cứu **encoder** (embedding), không có thành phần sinh văn bản/decoder; không áp dụng trực tiếp cho VLM sinh chuỗi.
- Student nhỏ (2M–56M) và dữ liệu nhỏ (CC3M+12M, 32 epoch); chưa rõ kết quả khi scale tới student cỡ trăm triệu tham số hoặc dữ liệu hàng trăm triệu cặp.

- Hệ số $\lambda_{FD} = 2000$ phụ thuộc cách chuẩn hóa embedding và kích thước chiều; cần chỉnh lại khi đổi encoder.
- Lập luận "tương quan giống feature, tăng hiệu năng" không loại trừ nhân tố gây nhiễu (ví dụ độ lớn gradient của từng loss).
- Chỉ so với TinyCLIP, không so với các baseline distill khác dưới cùng ngân sách.

7. Liên hệ với các bài khác

- **02 Hinton KD:** CRD ở đây là KD softmax nhưng trên ma trận tương đồng batch thay vì logit lớp.
- **03 FitNets, 04 Attention transfer:** FD là dạng đơn giản nhất của "hint" (MSE feature), nhưng chỉ lên đầu ra cuối, tránh khớp tầng trung gian.
- **12 Relational KD, 13 CRD:** ICL và CRD là họ hàng của distill quan hệ/contrastive.
- **41 Capacity gap law, 42 Distillation scaling laws:** quan sát teacher lớn không luôn tốt hơn là điểm khởi đầu cho các bài về capacity gap.
- **23 AM-RADIO:** cũng distill encoder thị giác bằng loss feature (có cả CLIP-type teacher).
- **25 LLaVA-MoD, 26 LLaVA-KD:** mở rộng từ CLIP embedding sang MLLM sinh văn bản; LLaVA-KD cũng có bước align quan hệ giữa visual token.

8. Tóm tắt công thức / khái niệm

Khái niệm	Công thức / ý	Ghi chú
CLIP loss	$\frac{1}{2}(\mathcal{L}_{I\to T} + \mathcal{L}_{T\to I})$	InfoNCE đối xứng
CRD	KL giữa phân phối tương đồng batch teacher/student	$\lambda = 1$
FD	$\|v^T - v^S\|^2 + \|s^T - s^S\|^2$	$\lambda = 2000$, loss đơn lẻ tốt nhất
MFD	FD với ảnh student bị che	không hơn FD
GD	MSE gradient loss theo embedding	$\lambda = 10^8$, tăng ít
ICL	InfoNCE giữa student anchor và embedding teacher	$\lambda = 1$, tốt thứ 2
AFD	fusion student+teacher rồi CLIP loss	tăng ít
Loss cuối	$\mathcal{L}_{CLIP} + \lambda(\mathcal{L}_{FD} + \mathcal{L}_{ICL} + \mathcal{L}_{CRD})$	

9. Kết luận

Trong CLIP-KD, khớp embedding bằng MSE và contrastive giữa teacher–student cải thiện zero-shot ImageNet khoảng 3–4 điểm cho student nhỏ, và gần 20 điểm khi teacher được tiền huấn luyện trên dữ liệu lớn hơn. Nghiên cứu tập trung vào encoder và chưa giải thích được hiện tượng teacher lớn hơn có thể dạy kém hơn.

Link gốc: https://arxiv.org/abs/2307.12732

Bài 25

LLaVA-MoD: Making LLaVA Tiny via MoE Knowledge Distillation (2024)

Tác giả: Fangxun Shu, Yue Liao, Le Zhuo, Chenning Xu, Lei Zhang, Guanghao Zhang, Haonan Shi, Long Chen, Tao Zhong, Wanggui He, Siming Fu, Haoyuan Li, Bolin Li, Zhelun Yu, Si Liu, Hongsheng Li, Hao Jiang (Alibaba, CUHK, Zhejiang Univ., Beihang Univ.).
Venue / arXiv: arXiv:2408.15881 (v3, 23/10/2024), ghi là "Technical Report"; bài báo không nêu venue hội nghị.
Nguồn: https://arxiv.org/abs/2408.15881
Chủ đề: Distill một MLLM (multimodal LLM, mô hình ngôn ngữ-thị giác) 7B xuống 0.5B/1.8B có LLM dạng Mixture-of-Experts; distill 2 giai đoạn: mimic (KL) rồi preference (KTO/DPO với teacher làm reference).
Vị trí trong lộ trình: Đọc sau 24 (CLIP-KD, distill encoder), 02 (Hinton KD), 15 (sequence-level KD), và nên biết 28 (MiniLLM)/29 (GKD) vì paper trích các bài đó. Bài này đưa KD sang MLLM sinh văn bản và thêm "preference distillation". Bài kế tiếp 26 (LLaVA-KD) cạnh tranh trực tiếp với hướng này nhưng giữ student dense, không dùng MoE.

> Tài liệu học tập bằng tiếng Việt, là ghi chú diễn giải, không phải bản dịch nguyên văn paper. Số liệu lấy từ paper (một số bảng PDF bị mangled; chỗ không chắc đã ghi rõ).

1. Bối cảnh

MLLM kiểu LLaVA = vision encoder (ở đây CLIP-ViT-L/14) + adapter (VL adaptor, MLP) + LLM. Mô hình lớn tốn kém để huấn luyện và triển khai. Các hướng trước cho MLLM nhỏ (MobileVLM, Imp, Bunny, MiniCPM-V...) chủ yếu tập trung **dữ liệu chất lượng cao**, và bị giới hạn bởi dung lượng mô hình. KD từ MLLM lớn sang nhỏ "hứa hẹn nhưng chưa được khám phá" (lời tác giả). Hai thách thức tác giả nêu:

1. Thiết kế kiến trúc student nhẹ nhưng vẫn đủ khả năng hấp thụ tri thức phức tạp.
2. Truyền tri thức đa nhiệm một cách hiệu quả.

Khái niệm nền:

- **MoE (Mixture of Experts):** thay một FFN bằng N FFN ("expert") cộng một router chọn top-k expert cho từng token. Tổng tham số lớn nhưng chỉ k expert chạy mỗi token (sparse).
- **Sparse upcycling:** khởi tạo các expert bằng cách copy FFN của mô hình dense đã train.
- **Preference optimization (PO), DPO, KTO:** huấn luyện trực tiếp từ dữ liệu "câu trả lời tốt/xấu" mà không cần reward model; DPO dùng cặp (tốt, xấu); KTO chỉ cần nhãn tốt/xấu từng mẫu.
- **Hallucination (ảo giác):** mô hình mô tả vật không có trong ảnh.

2. Ý tưởng chính

Student là MLLM có LLM nhỏ (Qwen-1.5 1.8B hoặc 0.5B), nhưng FFN được "upcycle" thành MoE để tăng khả năng biểu diễn mà vẫn tính toán ít. Huấn luyện theo 3 bước: (0) khởi tạo adapter bằng caption; (1) **mimic distillation** tối thiểu KL với teacher 7B, chia hai chặng "dense-to-dense" (kiến thức chung) rồi "dense-to-sparse" (kiến thức chuyên biệt, chuyển sang MoE); (2) **preference distillation**: dùng teacher làm mô hình reference trong loss kiểu DPO/KTO, để student gán xác suất cao hơn teacher cho câu trả lời tốt và thấp hơn cho câu xấu, nhờ đó vượt teacher về ảo giác. Kết quả: 2B vượt Qwen-VL-Chat-7B trung bình 8.8% chỉ với 5M mẫu.

3. Phương pháp chi tiết

3.1 Kiến trúc

$$y = \text{LLM}_\phi(\text{Proj}_\omega(\text{ViT}_\chi(x_v)),\, x_i)$$

Ảnh 336x336, patch 14x14 cho 576 visual token. ViT và adaptor giữ nguyên; chỉ LLM bị thu nhỏ rồi sparsify.

MoE router:

$$r = \text{Softmax}(x\, W_r), \qquad \tilde{r} = \text{Top-k}(r), \qquad y = \sum_{i=1}^{N} \tilde{r}_i\, E_i(x)$$

Router là một lớp tuyến tính; trọng số các expert không chọn bị đặt bằng 0. Cấu hình ký hiệu E4T2 = 4 expert, top-2 hoạt động; E4T1 = top-1 (số tham số hoạt động bằng dense).

3.2 Ba giai đoạn

Initialization. Đóng băng ViT và LLM, chỉ train adaptor bằng cross-entropy:

$$\mathcal{L}_{Init} = -\mathbb{E} \log \pi_S(y_k \mid y_{<k}, x)$$

Dữ liệu: 0.6M caption (LLaVA-1.5-Pretrain).

Mimic distillation (a): Dense-to-Dense (D2D). ViT đóng băng; train LLM + adaptor. Dữ liệu: 2.4M caption/hội thoại tổng quát. Loss KL giữa phân phối token (sau softmax) của student và teacher; công thức (6) trong bài báo viết là $-\mathbb{E}_{\pi_T} \log \frac{\pi_T}{\pi_S}$ (đã biểu diễn dưới dạng forward KL, kỳ vọng theo teacher):

$$\mathcal{L}_{D2D} = \text{KL}(\pi_T \| \pi_S) \text{ tại từng vị trí token}$$

(Ghi chú: dấu trừ trong công thức gốc có vẻ thừa, đây là cách người ghi chú hiểu theo ngữ cảnh "minimize KLD". Điều này trái với MiniLLM (28) dùng reverse KL.)

Mimic distillation (b): Dense-to-Sparse (D2S). Biến FFN của LLM thành MoE (sparse upcycling), dữ liệu đa nhiệm 1.4M (VQA, document, science, OCR). Chỉ train experts và adaptor. Loss = CE (nhãn thật) + KL:

$$\mathcal{L}_{D2S} = -\mathbb{E} \log \pi_S(y_k | y_{<k}, x) + \text{KL}(\pi_T \| \pi_S)$$

CE được thêm để "giảm bias vốn có của teacher".

Preference distillation. Với dữ liệu (x, y^+, y^-), 80K mẫu RLAIF-V, loss kiểu DPO với teacher làm reference:

$$\boxed{\mathcal{L}_{PD} = -\mathbb{E}\, \log \sigma\Big(\beta \log \frac{\pi_S(y^+|x)}{\pi_T(y^+|x)} - \beta \log \frac{\pi_S(y^-|x)}{\pi_T(y^-|x)}\Big)}$$

Điểm mấu chốt là π_T (teacher) đóng vai reference model thay vì SFT model của chính student. Ý nghĩa: (i) student học phân biệt tốt/xấu như teacher; (ii) student cố gán xác suất cao hơn teacher cho y^+ và thấp hơn cho y^-, tức vượt teacher. Triển khai thực tế dùng **KTO** (Phụ lục A.1: "KTO loss"); công thức (8) viết ở dạng DPO. Hai điều này không khớp tuyệt đối, ghi chú để tránh nhầm khi reproduce.

3.3 Pseudo-code

```
# Stage 0: init
freeze ViT, LLM; train adaptor with CE on 0.6M captions
# Stage 1a: D2D (dense student, 2.4M general data)
for batch: logits_T = teacher(x) (no_grad); logits_S = student(x)
    loss = KL(softmax(logits_T) || softmax(logits_S)) on RESPONSE tokens only
    update LLM + adaptor
# Stage 1b: D2S
student.FFN -> MoE: copy FFN N times, add router (sparse upcycling)
for batch (1.4M multi-task):
    loss = CE(student, y) + KL(teacher || student)
    update experts + adaptor (top-k routing)
# Stage 2: preference
for (x, y+, y-) in 80K:
    lp_S(y), lp_T(y) = sequence log-probs
    loss = KTO/DPO(beta; reference = teacher)
    update experts + adaptor
```

4. Thí nghiệm và kết quả chính

Thiết lập: CLIP-ViT-L/14 (336), adaptor MLP 2 lớp, LLM Qwen-1.5/2. Teacher 7B (Qwen-1.5-7B hoặc Qwen-2-7B); student Qwen-1.5-1.8B, Qwen-1.5-0.5B (và Qwen-2 tương ứng). Teacher và student cùng họ LLM để chung vocabulary. 16 GPU A100, 1 epoch mỗi chặng, tổng khoảng 960 GPU giờ. Tổng dữ liệu 5M mẫu.

Teacher (Bảng 1)

LLM teacher	GQA	VisWiz	SQA	VQAT	MME	MMB	MMBCN	AVG
Qwen-1.5-7B	62.3	40.7	70.9	55.3	72.1	68.5	64.9	62.1
Qwen-2-7B	62.5	37.9	73.2	57.2	78.0	71.6	71.1	64.5

Kết quả hiểu (Bảng 2, trích AVG)

Mô hình	#Sample	AVG
LLaVA-MoD-2B (Qwen-1.5-1.8B)	5M	59.9
LLaVA-MoD-2B (Qwen-2-1.5B)	5M	61.6
Qwen-VL-Chat-7B	1450M	56.7
MiniCPM-V-2 (2.4B)	570M	66.9
Mini-Gemini-2B	2.7M	57.1
DeepSeek-VL-1.3B	2000M	58.5
LLaVA-MoD-1B (Qwen-1.5-0.5B)	5M	54.1
LLaVA-MoD-1B (Qwen-2-0.5B)	5M	55.7
SPHINX-Tiny (1.1B)	15M	49.2

Tác giả: bản 2B “vượt Qwen-VL-Chat-7B 8.8%” (nói về lợi thế trung bình trong Fig. 1; bảng ghi 59.9 so với 56.7 cho bản Qwen-1.5, nên con số 8.8% phụ thuộc cách tính, ghi chú: không tái tạo được từ cột AVG này). Bản 1B hơn SPHINX-Tiny 13.2% (văn bản paper). Đáng chú ý MiniCPM-V-2 (66.9) vẫn cao hơn LLaVA-MoD-2B; paper nói “matches VILA-3B và MiniCPM-V” (bản đầu), không so mạnh với MiniCPM-V-2.

Ảo giác (Bảng 3, trích)

Mô hình	Object HalBench Resp ↓	Ment ↓	POPE F1 ↑	MMHal Score ↑	Hall ↓
Teacher Qwen-1.5-7B	50.1	24.8	84.9	2.60	20.7
Teacher Qwen-2-7B	29.7	23.4	85.7	2.88	14.5
RLHF-V (13B)	12.2	7.5	86.2	2.45	51.0
LLaVA-MoD-2B (Qwen-1.5-1.8B)	11.4	7.2	87.0	2.76	17.2
LLaVA-MoD-2B (Qwen-2-1.5B)	11.2	5.9	87.2	2.91	13.8

Student vượt teacher trên các chỉ số ảo giác (ví dụ Resp 11.4 so với 50.1).

5. Phân tích / ablation

Tất cả ablation mimic dùng student Qwen-1.5-1.8B, teacher Qwen-1.5-7B, AVG trên 7 benchmark.

Thí nghiệm	Kết quả AVG
Mimic vs Mimic+Preference (Bảng 4)	60.2 vs 59.9 (hiểu giảm nhẹ)
Hallucination: Mimic vs +Preference (Bảng 5)	Resp 39.1 xuống 11.4; Ment 22.6 xuống 7.2
SFT vs KD, MoE E4T2 (Bảng 6)	55.7 vs 60.2 (KD hơn 8.1%; MME 63.2 lên 68.4, MMB 59.2 lên 65.1)
D2S vs D2D+D2S (Bảng 7)	57.9 (1536 GPU giờ) vs 60.2 (960 GPU giờ)
Distill Response+Instruction vs Response (Bảng 8)	58.4 vs 60.2
Dense vs Sparse E4T1 (Bảng 9)	57.1 vs 59.2
DPO vs KTO (Bảng 15)	59.6 vs 59.9; Resp halluc 20.0 vs 11.4 (Bảng 16)
Số expert 4 vs 8, top-2 (Bảng 13)	AVG 64.0 vs 63.6 (5 benchmark; không cải thiện)
Top-1 vs Top-2, 4 expert (Bảng 14)	62.6 vs 64.0

Những điểm chính:

- **Preference distillation giảm ảo giác mạnh nhưng không tăng khả năng hiểu** (tác giả liên hệ hiện tượng tương tự ở LLM).
- **KD cung cấp tín hiệu tối ưu tốt hơn SFT cho MoE** (có CE ground-truth chỉ ở D2S).
- **General-to-specialized** vừa tốt hơn vừa rẻ hơn (62.5% GPU giờ).
- **Chỉ distill token response**, không distill token instruction, tốt hơn (tác giả đoán: tránh overfit mẫu instruction).
- **Teacher capacity:** với student 1.8B, teacher 7B hơn teacher 4B 2.2% (60.2 vs 58.9); với student 0.5B hầu như không khác (53.8 vs 53.7). Tác giả đề xuất "middle teacher" để bắc cầu khoảng cách dung lượng (đề xuất, không thử nghiệm đầy đủ trong bài báo).
- **KTO ổn định hơn DPO**, tác giả lý giải KTO dùng nhãn tốt/xấu từng mẫu (là giả thuyết).

6. Giới hạn

Tác giả nêu (Phụ lục C): student và teacher phải cùng họ LLM để chung vocabulary; cần nạp cả hai mô hình khi huấn luyện nên tốn bộ nhớ (gợi ý pre-extract logit của teacher).

Người ghi chú nhận xét:

- Không tách bạch tác dụng của MoE và của KD/dữ liệu so với dense có cùng tham số tổng (chỉ có so sánh E4T1 vs dense ở cùng số tham số hoạt động).
- Student cuối là MoE: tổng tham số lớn hơn số hoạt động, nhưng bộ nhớ GPU phải chứa tất cả expert.
- Công thức (6)–(8) có dấu/ký hiệu không nhất quán với triển khai (xem mục 3.2); số hyperparameter (batch size) trong văn bản (512/256) và trong Bảng 11 (256/128) khác nhau.
- Preference distillation làm AVG hiểu giảm nhẹ; chưa rõ trên tác vụ cần độ chính xác từng ký tự như OCR.
- Không so sánh với phương pháp distill MLLM dense khác trong cùng điều kiện (bài 26 sau đó làm).

7. Liên hệ với các bài khác

- **02 Hinton KD, 15 Sequence-level KD:** mimic distillation = forward KL token-level (word-level KD).
- **28 MiniLLM, 29 GKD:** paper nhắc cả hai; nhưng chính LLaVA-MoD dùng KL thường, không dùng reverse KL.
- **26 LLaVA-KD:** cùng mục tiêu MLLM nhỏ; LLaVA-KD không dùng MoE và thêm distill visual token và quan hệ.
- **24 CLIP-KD:** ở đây ViT được đóng băng, nên vision encoder không được distill.
- **41 Capacity gap law, 39 Small models struggle...:** ablation teacher 7B vs 4B cho student 0.5B/1.8B là bằng chứng nhỏ về capacity gap.
- **36 Distilling step-by-step, 37 Orca:** các bài khác dùng dữ liệu/giải thích từ teacher; ở đây là logit + preference.

8. Tóm tắt công thức / khái niệm

Khái niệm	Công thức / ý	Ghi chú
Router MoE	$r = \text{softmax}(xW_r)$, top-k, $y = \sum \tilde{r}_i E_i(x)$	E4T2 mặc định
Sparse upcycling	copy FFN thành N expert	khởi tạo từ dense
D2D	KL(teacher‖student) trên response	dữ liệu chung 2.4M
D2S	CE + KL, train experts + adaptor	dữ liệu đa nhiệm 1.4M
Preference distill	DPO/KTO với reference = teacher	80K RLAIF-V
Thứ tự	init → D2D → D2S → preference	tổng ~960 GPU giờ

9. Kết luận

LLaVA-MoD kết hợp KD token-level qua hai giai đoạn với preference distillation. Trong các thí nghiệm, MLLM 1–2B tiến gần các mô hình 7B với 5 triệu mẫu và giảm ảo giác so với teacher.

Link gốc: https://arxiv.org/abs/2408.15881

Bài 26

LLaVA-KD: A Framework of Distilling Multimodal Large Language Models (2024)

Tác giả: Yuxuan Cai, Jiangning Zhang, Haoyang He, Xinwei He, Ao Tong, Zhenye Gan, Chengjie Wang, Zhucun Xue, Yong Liu, Xiang Bai (HUST, Zhejiang Univ., Tencent Youtu Lab, HZAU).
Venue / arXiv: arXiv:2410.16236 (v3, 3/7/2025). Paper không nêu venue trong phần text đã trích.
Nguồn: https://arxiv.org/abs/2410.16236
Chủ đề: Distill MLLM (mô hình ngôn ngữ-thị giác) lớn sang nhỏ mà không đổi kiến trúc: MDist (KL trên cả token response lẫn token visual), RDist (khớp ma trận tự tương quan giữa các visual token), và lịch huấn luyện 3 giai đoạn DPT → SFT → DFT.
Vị trí trong lộ trình: Đọc sau 25 (LLaVA-MoD, đối thủ trực tiếp), 24 (CLIP-KD), 02 (Hinton KD), 12 (Relational KD) và 28 (MiniLLM, reverse KL). Bài này đưa ra hai thông điệp: (1) khi distill VLM, **thứ tự giai đoạn** và **distill cả token thị giác** quan trọng; (2) forward KL đơn giản không thua reverse KL/JSD trong bối cảnh này. Tiếp theo là nhóm KD cho LLM (28 trở đi).

> Tài liệu học tập bằng tiếng Việt, là ghi chú diễn giải, không phải bản dịch nguyên văn paper. Số liệu lấy từ paper; bản trích không có phần Phụ lục (hyperparameter chi tiết) nên không ghi các số đó.

1. Bối cảnh

Một MLLM kiểu LLaVA-1.5 gồm: **visual encoder** (đóng băng) → **projector** (MLP 2 lớp) → **LLM**. Huấn luyện chuẩn có hai giai đoạn: **Pre-Training (PT)** chỉ train projector trên cặp ảnh-caption để căn chỉnh không gian thị giác với không gian embedding văn bản; **Supervised Fine-Tuning (SFT)** train projector + LLM trên dữ liệu instruction. Với MLLM nhỏ, quy trình này cho kết quả kém: ví dụ TinyLLaVA 4B đạt 65.0% nhưng giảm xuống 54.7% ở 0.5B (paper).

Các cách làm MLLM nhỏ trước đó: thay LLM bằng backbone nhỏ (LLaVA-Phi), thêm MoE (MoE-LLaVA), lọc dữ liệu (Bunny). LLaVA-MoD (25) là công trình KD đầu tiên cho s-MLLM, dùng MoE + preference distillation. Các phương pháp KD cho LLM (MiniLLM, DistiLLM...) chỉ chưng cất trong miền **văn bản**, bỏ qua tín hiệu từ token thị giác. LLaVA-KD tập trung vào **cách huấn luyện** chứ không đổi kiến trúc hay dữ liệu.

Khái niệm nền:

- **s-MLLM / l-MLLM:** MLLM nhỏ / lớn (student / teacher).
- **Visual token:** vector đầu ra của projector cho mỗi patch ảnh, đi vào LLM như token. Ở LLM, các vị trí này cũng có logit đầu ra trên từ vựng (dù không phải token "thật" để sinh).
- **Ma trận tự tương quan (self-correlation):** $R = YY^\top$, ma trận $N_p \times N_p$ đo quan hệ giữa mọi cặp token.

2. Ý tưởng chính

Hai quan sát: (1) căn chỉnh thị giác-ngôn ngữ là nền tảng, nên cho phép distill ngay trong giai đoạn pre-training (DPT); (2) nếu chỉ chen distill vào SFT thì lợi ích hạn chế, tốt hơn là SFT thường trước để student có năng lực cơ bản, rồi một giai đoạn **Distilled Fine-Tuning (DFT)** riêng để tinh chỉnh. Về loss, ngoài KL trên token response (như KD cho LLM), tác giả thêm KL trên **token visual** (MDist) và một loss **quan hệ**: ép ma trận tự tương quan của visual token student giống teacher (RDist). Student giữ nguyên kiến trúc dense.

3. Phương pháp chi tiết

3.1 Kiến trúc (teacher và student cùng thiết kế)

SigLIP-B/14@384 đóng băng (dùng chung), projector 2 lớp MLP+GELU, LLM Qwen1.5 hoặc Qwen2.5. Đầu vào LLM: $H = [H_v, H_t]$; chuỗi dự đoán $y = [y_p, y_v, y_r]$ (prompt, visual, response). Teacher và student **phải cùng họ LLM** để chung vocabulary.

3.2 Các loss

Autoregressive (baseline):

$$\mathcal{L}_{reg} = -\sum_{m=1}^{M} \log \phi_l(y_m \mid y_{<m})$$

(paper dùng ký hiệu ϕ cho cả teacher/student; trong Eq. 1 nói chung cho mô hình đang huấn luyện.)

MDist – response:

$$\mathcal{L}_{res} = \sum_{m=1}^{M}\sum_{j=1}^{V} \phi_l(Y_j \mid y_{<m}) \log \frac{\phi_l(Y_j \mid y_{<m})}{\phi_s(Y_j \mid y_{<m})}$$

Đây là **forward KL** (teacher ở vị trí "p"), token-level, trên mọi vị trí response.

MDist – visual:

$$\mathcal{L}_{vis} = \sum_{k=1}^{K}\sum_{j=1}^{V} \phi_l(Y_j \mid y_{<k}) \log \frac{\phi_l(Y_j \mid y_{<k})}{\phi_s(Y_j \mid y_{<k})}$$

Cùng dạng KL nhưng lấy tại K vị trí của visual token (logit LLM tại vị trí ảnh). Hiểu đơn giản: student bắt chước cách teacher "dự đoán" trên chuỗi visual token.

RDist – quan hệ giữa visual token: lấy y_v^s, y_v^t (đầu ra LLM tại vị trí visual của student/teacher), tính

$$R_v^s = y_v^s \otimes y_v^s \in \mathbb{R}^{N_p \times N_p}, \quad R_v^t = y_v^t \otimes y_v^t$$

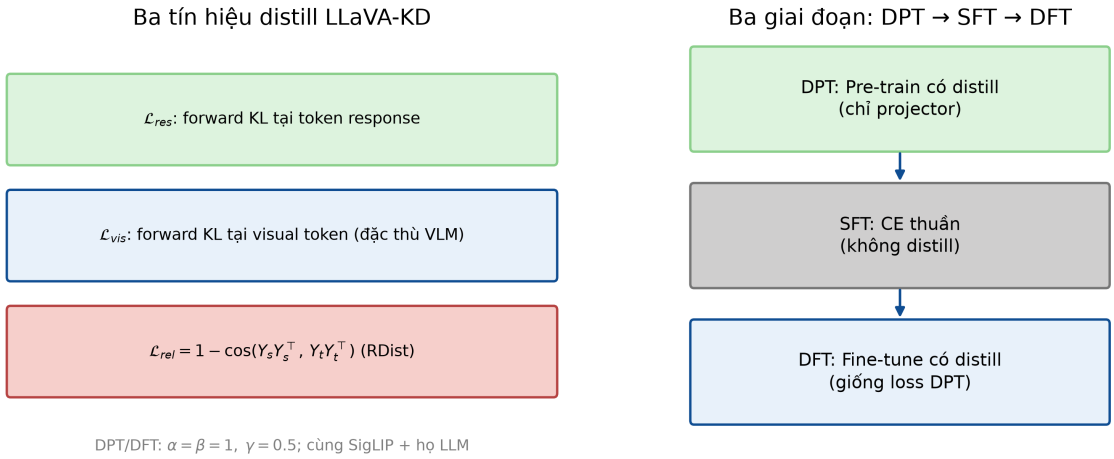
$$\boxed{\mathcal{L}_{rel} = 1 - \cos(R_v^s, R_v^t) = 1 - \frac{R_v^s \cdot R_v^t}{\|R_v^s\| \, \|R_v^t\|}}$$

Cosine được tính trên cả ma trận (dạng phẳng). Vì cần $N_p \times N_p$ cùng kích thước nên chỉ cần số visual token bằng nhau giữa hai mô hình (cùng visual encoder), không cần cùng chiều ẩn. Vì R là Gram matrix nên không phụ thuộc chiều embedding; đây là điểm chung với Relational KD (12).

3.3 Ba giai đoạn

Giai đoạn	Train gì	Dữ liệu	Loss
1. DPT (Distilled Pre-Training)	chỉ projector (ViT, LLM student đóng băng)	LLaVA-1.5-558k caption	$\mathcal{L}_{DPT} = \mathcal{L}_{reg} + \alpha\mathcal{L}_{res} + \beta\mathcal{L}_{vis} + \gamma\mathcal{L}_{rel}$
2. SFT	projector + LLM	LLaVA-mix-665k	$\mathcal{L}'_{SFT} = \mathcal{L}_{reg}$

Giai đoạn	Train gì	Dữ liệu	Loss
3. DFT (Distilled Fine-Tuning)	projector + LLM	LLaVA-mix-665k	$\mathcal{L}_{DFT} = \mathcal{L}_{reg} + \alpha'\mathcal{L}_{res} + \beta'\mathcal{L}_{vis} + \gamma'\mathcal{L}_{rel}$



LLaVA-KD: $\mathcal{L}_{res}$, $\mathcal{L}_{vis}$, $\mathcal{L}_{rel}$ và ba giai đoạn DPT → SFT → DFT (minh hoạ, tự tính)

Hệ số: α, β, γ (và dấu phẩy) = 1.0, 1.0, 0.5. Teacher được huấn luyện bằng PT+SFT thường (Mục 3.2 paper), vision encoder chung.

3.4 Pseudo-code

```
teacher = train_standard(PT, SFT)            # l-MLLM, frozen afterwards
student = init(small LLM, projector random)
# Stage 1: DPT  (only projector trainable)
for (img, caption) in caption_data:
    out_s, out_t = student(img, caption), teacher(img, caption)      # logits at
    ↪  visual+response positions
    L = CE(out_s.resp) + 1.0*KL(out_t.resp || out_s.resp) \
        + 1.0*KL(out_t.vis || out_s.vis) + 0.5*(1 - cos(gram(out_s.vis_hidden),
        ↪  gram(out_t.vis_hidden)))
# Stage 2: SFT   (projector + LLM): L = CE only, on instruction data
# Stage 3: DFT   (projector + LLM): same loss as stage 1, on instruction data
```

(Ghi chú: bài báo không nói rõ y_v trong RDist là hidden state hay logit; “visual tokens generated by LLM” — người ghi chú chọn cách hiểu chung chung là đầu ra LLM tại vị trí visual.)

4. Thí nghiệm và kết quả chính

Thiết lập: SigLIP-B/14@384 + 2-layer MLP + Qwen1.5/Qwen2.5. Dữ liệu 1.2M (558k + 665k). 10 benchmark: VQAv2, GQA, VizWiz, SciQA, TextVQA, MME, MMB, MMB-CN, POPE, MMMU; Avg7 (bỏ VQAv2, POPE, MMMU để so với LLaVA-MoD) và Avg10. Baseline TinyLLaVA† là bản tự chạy lại với PT-SFT.

Kết quả chính (Bảng 1, trích Avg10 / Avg7)

Student	Baseline PT-SFT Avg10	LLaVA-KD Avg10	Avg7 (LLaVA-KD)
Qwen1.5-0.5B	54.7	57.9	55.2

Student	Baseline PT-SFT Avg10	LLaVA-KD Avg10	Avg7 (LLaVA-KD)
Qwen2.5-0.5B	56.5	58.9	56.7
Qwen1.5-1.8B	56.8	62.1	60.3
Qwen2.5-1.5B	64.0	65.0	63.9

So sánh trực tiếp: LLaVA-KD Qwen1.5-0.5B hơn LLaVA-MoD cùng LLM 1.1% Avg7 (55.2 vs 54.1) với 1.2M mẫu so với 5M; hơn SPHINX-Tiny (15M mẫu) 4.0% Avg7 (paper). Với 1.8B: hơn LLaVA-MoD 0.4% và Imp-2B 1.4% Avg7 (60.3 so với 59.9 và 58.9). Cải thiện so với TinyLLaVA: +3.2% và +2.4% Avg10 ở 0.5B (Qwen1.5/Qwen2.5), +5.3% và +1.0% ở 1.8B/1.5B (text paper). Lưu ý Bảng 1 có vài dòng LLaVA-MoD Avg10 thiếu giá trị; cột Avg7 của bảng 1 lấy từ cột "Avg7" trong bài báo.

Xác minh trên khung MobileVLM (Bảng 7, Avg10)

Teacher MobileLLaMA 2.7B: 55.2; student 1.7B: PT-SFT 48.8, DPT-SFT 50.5, DPT-SFT-DFT 53.4 (+4.6 so với baseline).

5. Phân tích / ablation

Tất cả ablation dùng student Qwen1.5-0.5B, Avg10, teacher Qwen1.5-4B.

Chiến lược giai đoạn (Bảng 2):

Lịch	Avg10
PT-SFT (baseline)	54.7
DPT-SFT	55.6
PT-DFT	55.8
DPT-DFT	55.9
PT-SFT-DFT	56.6
DPT-SFT-DFT	**57.9**
DPT-DFT-DFT	58.0 (tốn thêm 120 GPU giờ)

Bỏ SFT (DPT-DFT) thì kém hẳn (55.9); DPT cộng DFT cộng SFT cho +1.3 so với PT-SFT-DFT. DPT-SFT-DFT có đánh đổi hiệu năng/tính toán tốt nhất.

MDist vs RDist (Bảng 3, 6 hàng; thứ tự cột bị mangled, gán nhãn theo văn bản): ở DPT riêng lẻ, RDist (55.5) hơn MDist (55.1), kết hợp 55.6; ở DFT, MDist (57.7) hơn RDist (57.0), kết hợp 57.9. Tác giả: MDist hiệu quả hơn ở DFT vì khớp trực tiếp phân phối đầu ra đa phương thức.

Mục tiêu distill (Bảng 4): thêm token visual có lợi nhất: DPT: response 54.9, +prompt 55.0, +visual 55.1, cả ba 54.6; DFT: response 57.2, +prompt 56.9, **+visual 57.7**, cả ba 57.1. Khác LLaVA-MoD (chỉ response).

Loại divergence (Bảng 5, PT-DFT): FKL 55.5, RKL 55.0, JSD 55.5, phương pháp của bài báo 55.8. Chênh lệch nhỏ (0.3–0.8).

So với pipeline LLaVA-MoD (Bảng 6): mimic distillation (MD) 56.9; MD+preference distillation (PD) 55.1; LLaVA-KD 57.9. Phù hợp quan sát của LLaVA-MoD rằng PD giảm ảo giác nhưng hạ khả năng hiểu.

Kích thước teacher (Bảng 8, Avg10, student 0.5B): Qwen1.5-4B teacher cho 57.9 > Qwen1.5-7B 57.4; Qwen2.5-3B cho 58.9 > Qwen2.5-7B 58.3. Teacher lớn hơn (7B) lại kém hơn, nhất quán với CLIP-KD (24) và khái niệm capacity gap (tác giả viện dẫn "diminishing returns" và "alignment bottleneck", không chứng minh).

6. Giới hạn

Tác giả nêu: teacher và student phải cùng họ LLM (vocabulary chung); hướng tương lai là kết hợp MLLM dị chủng và xử lý capacity mismatch.

Người ghi chú nhận xét:

- Một số khác biệt rất nhỏ (0.3–1.0 điểm Avg), trên một seed; không có khoảng tin cậy.
- Ablation chủ yếu ở student 0.5B; hiệu lực khi student lớn hơn ít được kiểm tra (gain 1.0 ở Qwen2.5-1.5B).
- Chi phí huấn luyện (cần chạy teacher 4B/7B forward trong hai giai đoạn) không được định lượng ngoài một con số 120 GPU giờ.
- Nhiều chữ số trong bảng PDF bị mangled; Bảng 3 không đọc chắc ánh xạ hàng-cột.
- Không phân tích chất lượng/hành vi OCR chi tiết ngoài TextVQA.
- Không tách riêng ảnh hưởng của dữ liệu (1.2M) so với phương pháp trong so sánh với các mô hình dùng dữ liệu khác.

7. Liên hệ với các bài khác

- **25 LLaVA-MoD:** đối thủ trực tiếp; LLaVA-KD cho rằng dense + 3 giai đoạn + distill visual tốt hơn MoE + preference trong benchmark hiểu.
- **24 CLIP-KD:** cùng quan sát teacher lớn không luôn tốt hơn; LLaVA-KD cites CLIP-KD cho điều này.
- **12 Relational KD:** RDist là phiên bản "relational" cho visual token (khớp Gram matrix thay vì khoảng cách cặp).
- **28 MiniLLM, 30 DistiLLM, 31 Rethinking KL:** Bảng 5 so FKL/RKL/JSD; kết quả nói RKL không vượt FKL ở đây.
- **41 Capacity gap law, 39 Small models struggle...:** Bảng 8.
- **08 Teacher assistant:** gợi ý dùng teacher trung gian.
- **23 AM-RADIO:** distill encoder thị giác; ở LLaVA-KD encoder bị đóng băng nên không distill phần này.

8. Tóm tắt công thức / khái niệm

Khái niệm	Công thức / ý	Ghi chú
$\mathcal{L}_{res}$	forward KL(teacher‖student) tại vị trí response	token-level
$\mathcal{L}_{vis}$	forward KL tại vị trí visual token	khác biệt so với KD cho LLM
$\mathcal{L}_{rel}$	$1 - \cos(Y_s Y_s^\top, Y_t Y_t^\top)$	RDist, $N_p \times N_p$
DPT	pre-train có distill (chỉ projector)	$\alpha = \beta = 1, \gamma = 0.5$
SFT	CE thuần	giữa DPT và DFT
DFT	fine-tune có distill	giống loss DPT
Cặp teacher–student	cùng họ LLM, cùng SigLIP	vocabulary chung

9. Kết luận

LLaVA-KD kết hợp các giai đoạn DPT–SFT–DFT với tín hiệu trên token thị giác và quan hệ giữa chúng. Student 0.5B tăng Avg10 từ 54.7 lên 57.9 với 1.2 triệu mẫu. Chênh lệch còn nhỏ; thiết lập cần teacher cùng họ và teacher lớn hơn không luôn cho kết quả tốt hơn.

Link gốc: https://arxiv.org/abs/2410.16236

Bài 27

A Survey on Knowledge Distillation of Large Language Models (2024)

Tác giả: Xiaohan Xu, Ming Li, Chongyang Tao, Tao Shen, Reynold Cheng, Jinyang Li, Can Xu, Dacheng Tao, Tianyi Zhou (HKU, Univ. of Maryland, Microsoft, UTS, Peking Univ., Univ. of Sydney).
Venue / arXiv: arXiv:2402.13116 (v4, 21/10/2024). Paper không nêu venue trong phần text đã trích.
Nguồn: https://arxiv.org/abs/2402.13116
Chủ đề: Survey KD cho LLM theo ba trụ: **algorithm** (cách rút tri thức từ teacher và cách nạp vào student), **skill** (kỹ năng được chưng cất), **verticalization** (ứng dụng theo miền). Nhấn mạnh data augmentation (DA) là cơ chế trung tâm của KD thời LLM. Ghi chú này tập trung vào phần taxonomy và thuật toán (Mục 2 và 3); các mục 4 (skills) và 5 (vertical domains) chỉ lướt, dùng như danh mục tra cứu.
Vị trí trong lộ trình: Đọc sau 05 (KD survey cổ điển) và 15 (SeqKD), trước nhóm 28–40. Nó đóng vai bản đồ: chỉ ra nhóm "black-box KD bằng dữ liệu sinh" (35 Self-Instruct, 36 Distilling step-by-step, 37 Orca, 38 DeepSeek-R1 distillation) và nhóm "white-box KD bằng phân phối/feature" (28 MiniLLM, 29 GKD, 30 DistiLLM...).

> Tài liệu học tập bằng tiếng Việt, là ghi chú diễn giải, không phải bản dịch nguyên văn paper. Survey không có thí nghiệm riêng, nên mục "Thí nghiệm và kết quả" chỉ tóm các luận điểm có trích dẫn trong survey, không có số liệu đo mới.

1. Bối cảnh

Trước LLM, KD (Hinton, bài 02; DistilBERT, bài 16; TinyBERT, bài 18...) chủ yếu là nén mô hình: teacher lớn → student nhỏ trên cùng tác vụ, để triển khai trên thiết bị yếu. Khi có GPT-4, Gemini, Claude (đóng, API) và LLaMA, Mistral (mở), KD có thêm hai vai trò (Hình 1 của survey):

1. **Advance (nâng khả năng):** chuyển năng lực từ LLM "proprietary" (đóng, hoặc mở nhưng rất lớn như LLaMA-2-70B) sang LLM mở nhỏ hơn. Vai trò chính, thường dựa vào dữ liệu do teacher sinh.
2. **Compress (nén):** vai trò truyền thống, hiệu quả triển khai.
3. **Self-improve (tự cải thiện):** dùng chính mô hình mở làm teacher cho nó.

Khái niệm nền:

- **Black-box KD:** chỉ có văn bản teacher sinh (qua API). **White-box KD:** có logit/phân phối/hidden state.
- **Data augmentation (DA)** trong ngữ cảnh LLM: không chỉ paraphrase/back-translation, mà là *sinh dữ liệu mới theo kỹ năng/miền* từ một ít hạt giống (seed). Survey coi DA là cách thực thi chính của KD thời LLM.
- **Instruction tuning / SFT:** tinh chỉnh có giám sát trên cặp (instruction, response).
- **RLAIF, DPO:** học từ phản hồi/preference do AI (teacher) tạo, thay vì từ người; DPO tối ưu preference trực tiếp không cần reward model.

2. Ý tưởng chính

Survey tách KD của LLM thành hai bước độc lập có thể ghép tự do:

(i) **Knowledge elicitation** – dùng teacher để tạo dữ liệu tri thức $D^{(kd)}$ (nhãn, mở rộng, curation, feature, feedback, self-knowledge);
(ii) **Distillation algorithm** – cách học từ dữ liệu đó (SFT, divergence/similarity, RL, ranking optimization). Rồi kết quả của hai bước này phục vụ *skill distillation* (instruction following, alignment, agent, NLP

task, multimodal) và *vertical distillation* (luật, y tế, tài chính, khoa học). Bản đồ này giúp trả lời nhanh: "một bài mới thuộc ô nào, khác bài cũ ở đâu".

3. Phương pháp chi tiết: taxonomy và các thuật toán

3.1 Pipeline chung (Mục 2.4, bốn bước)

1. **Steering:** dùng instruction/template I để hướng teacher vào kỹ năng/miền.
2. **Seed knowledge:** đưa dữ liệu hạt giống s (một ít ví dụ, chủ đề, tài liệu...).
3. **Generation:** teacher sinh các ví dụ tri thức (thường là QA/giải thích; đôi khi logit/hidden).
4. **Training:** huấn luyện student với hàm mục tiêu.

Hai công thức trừu tượng (Eq. 1 và 2 của survey):

$$\mathcal{D}_I^{(kd)} = \{\text{Parse}(o, s) \mid o \sim p_T(o \mid I \oplus s),\ \forall s \sim \mathcal{S}\}$$

$$\mathcal{L} = \sum_I \mathcal{L}_I(\mathcal{D}_I^{(kd)}; \theta_S)$$

($\oplus$ là ghép văn bản; có thể nhiều kỹ năng I cùng chưng cất vào một student.)

3.2 Sáu kiểu "knowledge elicitation" (Mục 3.1)

Kiểu	Ý chính	Công thức (survey)	Ví dụ
Labeling	teacher gắn nhãn y cho input x có sẵn	$\{x, y \mid x \sim \mathcal{X},\ y \sim p_T(y \mid I \oplus c \oplus x)\}$ (Eq. 3)	Orca (FLAN-v2 + ChatGPT/GPT-4), CoT-Distill, Vicuna
Expansion	teacher sinh cả x và y từ vài demo (in-context)	$\{(x, y) \mid x \sim p_T(x \mid I \oplus c),\ y \sim p_T(y \mid I \oplus x)\}$ (Eq. 4)	Self-Instruct, Alpaca, WizardLM (Evol-Instruct)
Data curation	sinh từ đầu theo *meta-information* (chủ đề, kiến thức) để kiểm soát chất lượng/đa dạng	$\{(x, y) \mid x \sim p_T(x \mid I \oplus m),\ y \sim p_T(y \mid I \oplus x)\}$ (Eq. 5)	UltraChat, phi-1 ("textbook quality"), Magicoder
Feature	lấy phân phối/hidden state của teacher (white-box)	$\{(x, y, \phi_{feat}(x, y; \theta_T))\}$ (Eq. 6)	MiniLLM, GKD, TED, FuseLLM, quantization-aware KD
Feedback	teacher đánh giá/sửa/ranking *đầu ra của student*	$\{(x, y, \phi_{fb}(x, y; \theta_T)) \mid y \sim p_S(y\|x)\}$ (Eq. 7)	RLAIF, UltraFeedback, Lion, PERsD, SelFee
Self-knowledge	student tự làm teacher: sinh, lọc, chấm điểm lại	$\{(x, y, \phi_{sk}(x, y)) \mid y \sim p_S(y \mid I \oplus x)\}$ (Eq. 8)	Self-Instruct, Self-Align, ReST, Self-Rewarding

Điểm cốt lõi về trade-off (trích ý của survey): Labeling bị giới hạn bởi dữ liệu đầu vào và có thể nhiễu; Expansion dễ đồng nhất và khuếch đại bias của seed; Curation kiểm soát tốt nhưng phụ thuộc thiết kế meta-information; Feature chỉ dùng được với teacher mở, và student từ teacher mở có thể yếu hơn student từ teacher đóng mạnh hơn (như GPT-4) (phát biểu của survey, không phải kết quả đo).

3.3 Bốn họ thuật toán "distillation" (Mục 3.2)

(a) SFT / sequence-level KD (Eq. 9): student cực đại hóa log-likelihood trên chuỗi do teacher sinh:

$$\mathcal{L}_{SFT} = \mathbb{E}_{x\sim\mathcal{X},\, y\sim p_T(y|x)}\big[-\log p_S(y|x)\big]$$

Đơn giản nhưng là nền của Alpaca, Vicuna, WizardLM, Orca... (liên hệ bài 15).

(b) Divergence và Similarity (white-box):

$$\mathcal{L}_{Div} = \mathbb{E}_{x\sim\mathcal{X},y\sim\mathcal{Y}}\, D\big(p_T(y|x),\, p_S(y|x)\big)$$

với Bảng 1 của survey: forward KL $\sum p \log \frac{p}{q}$ (mode-covering, dùng trong KD chuẩn như DistilBERT), reverse KL $\sum q \log \frac{q}{p}$ (mode-seeking, MiniLLM), JS divergence. Survey nhắc lại lập luận của MiniLLM và GKD: forward KL buộc student phủ mọi mode nên khi student quá yếu có thể gán xác suất cho token thấp xác suất dưới teacher, gây ảo giác/chất lượng thấp; reverse KL tránh điều đó nhưng giảm đa dạng; theo GKD và Sason & Verdú, divergence tối ưu **phụ thuộc tác vụ**: forward KL hợp với dịch máy (ít mode), reverse KL hợp với hội thoại/instruction tuning (nhiều mode). Phía **Similarity** (Eq. 11):

$$\mathcal{L}_{Sim} = \mathbb{E}\,\mathcal{L}_F\big(\Phi_T(f_T(x,y)),\, \Phi_S(f_S(x,y))\big)$$

với f là feature map, Φ là hàm biến đổi cho cùng kích thước, $\mathcal{L}_F$ là L2/L1/cross-entropy/MMD (Bảng 2). Survey nói rất ít công trình dùng similarity cho LLM (ví dụ TED: task-aware layer-wise distillation), dù phổ biến ở encoder nhỏ (liên hệ 03 FitNets, 18 TinyBERT, 19 MiniLM).

(c) Reinforcement learning: hai bước (1) huấn luyện reward model student từ preference của teacher:

$$\mathcal{L}_{RM} = -\mathbb{E}_{(x,y_w,y_l)} \log \sigma\big(r_\phi(x,y_w) - r_\phi(x,y_l)\big)$$

(2) tối ưu policy student với ràng buộc KL tới policy tham chiếu:

$$\max_{\pi_\theta}\ \mathbb{E}_{x,\, y\sim\pi_\theta}[r_\phi(x,y)] - \beta\, \mathrm{D}_{\mathrm{KL}}[\pi_\theta(y|x)\,\|\,\pi_{ref}(y|x)]$$

(thường PPO). Có thể dùng chính teacher làm reward (đắt hơn).

(d) Ranking optimization: thay RL bằng học từ preference cố định: **DPO** (Zephyr dùng DPO để chưng cất preference của GPT-4):

$$\mathbb{E}\Big[\log\sigma\Big(\beta\log\frac{\pi_\theta(y_w|x)}{\pi_{ref}(y_w|x)} - \beta\log\frac{\pi_\theta(y_l|x)}{\pi_{ref}(y_l|x)}\Big)\Big]$$

RRHF: $\sum_{r_i<r_j}\max(0,p_i-p_j)$; **PRO** (xếp hạng nhiều ứng viên): $-\sum_k \log\frac{\exp(p_k)}{\sum_{i\geq k}\exp(p_i)}$ (p_k: log-prob có điều kiện của y_k dưới student).

3.4 Skill và vertical (lướt)

Bảng 3 của survey tóm các công trình theo: skill, seed, teacher, student, phương pháp elicitation, objective. Nhóm skill: **context following** (instruction following — Self-Instruct 175 tác vụ mở rộng 52K; Alpaca; WizardLM Evol-Instruct; UltraChat; Orca/Orca 2 với system message và explanation trace; multi-turn; RAG), **alignment** (thinking pattern, preference, value — Zephyr, UltraFeedback, Constitutional AI), **agent** (tool use: Toolformer, Gorilla, ToolLLM; planning: FireAct, AgentTuning), **NLP task specialization** (NLU, NLG, IR, recommendation, đánh giá text, toán, code — WizardMath, WizardCoder, Magicoder), **multimodal** (LLaVA, SVIT, LVIS-Instruct4V, LLaVAR, MIMIC-IT; tất cả là Labeling + SFT từ GPT-4/ChatGPT). **Vertical:** luật,

y tế, tài chính, khoa học, mục "miscellaneous". Hai mục này chủ yếu là danh mục công trình, không có phát hiện mới nên không mô tả sâu ở đây.

3.5 Pseudo-code (bản đồ ra quyết định, người ghi chú tóm lại)

```
Có logit của teacher (white-box)?
  yes -> feature knowledge:  KL token-level (forward / reverse / skew)  [28,29,30,31]
         + (tuỳ chọn) similarity trên hidden state                       [03,18,19]
  no  -> chỉ có văn bản (black-box):
         labeling/expansion/curation -> SFT                              [35,36,37,38]
         + feedback/preference -> RL hoặc DPO/ranking                    [37, survey
         ↪  3.2.3-3.2.4]
Student có thể là teacher của chính nó?  -> self-knowledge (lọc, tự chấm)  [38]
Dung lượng student << teacher? -> cân nhắc reverse KL / on-policy / teacher trung gian
↪  [28,29,08,41]
```

4. Những gì survey báo cáo (thay cho thí nghiệm)

Survey không chạy thí nghiệm. Các "kết quả" là tóm tắt từ công trình khác (chưa kiểm chứng ở đây):

- **Self-Instruct:** GPT-3 tự mở rộng 175 tác vụ hạt giống lên 52K instruction; sau SFT, đạt gần InstructGPT trong zero-shot (theo mô tả survey). Alpaca: 52K dữ liệu từ text-davinci-003, student LLaMA-7B.
- **WizardLM:** 4 vòng Evol-Instruct bằng ChatGPT, 250K instruction phức tạp; trên tập instruction khó, win rate cao hơn ChatGPT 7.9% (theo survey).
- **phi-1:** 1B token "textbook" Python + 180M token bài tập; phát biểu của survey: vượt gần mọi mô hình mã nguồn mở trên HumanEval/MBPP dù nhỏ 10x về mô hình và 100x về dữ liệu. Phi-2 (2.7B) vượt Mistral 7B và Llama-2 13B trên nhiều benchmark (theo survey).
- **UltraChat:** 1.5M mẫu từ 30 meta-topic; UltraLLaMA vượt các chat model mở.
- **LIMA** (Zhou et al., nhắc ở phần open problems): 1000 mẫu chất lượng cao có thể đủ cho alignment.
- **Cảnh báo** (Gudibande et al.): student huấn luyện trên dữ liệu ChatGPT dễ bắt chước *phong cách* nhưng không sao chép được độ đúng thực tế; cần teacher mạnh hơn và dữ liệu đa dạng/chất lượng (như Orca).

5. Phân tích / các luận điểm đáng chú ý

- **Chất lượng và đa dạng dữ liệu quan trọng hơn lượng** (LIMA, phi, UltraChat, Orca): survey nhấn mạnh data curation/selection.
- **Học cách suy nghĩ, không chỉ trả lời:** Orca/Orca 2 thêm system message ("think step-by-step") và explanation trace; SelFee/Reflection-Tuning huấn luyện cả quá trình sửa. Phần này liên hệ với bài 36 và 37.
- **Từ imitation sang feedback/feature:** survey chỉ ra đa số công trình dùng teacher đóng với hard label + SFT; hướng "richer knowledge" (feature, feedback) còn ít khai thác.
- **On-policy:** MiniLLM và GKD cho student tự sinh chuỗi, teacher cho phân phối làm phản hồi ("feedback + feature" cùng lúc, Bảng 3 xếp GKD là "Feature + Feedback", objective D&S + RL).
- **Self-improvement:** ReST (Grow/Improve), Self-Play, Self-Rewarding (LLM-as-a-judge tự chấm) cho phép vượt "trần teacher".
- **Open problems (Mục 6):** (1) chọn dữ liệu (bao nhiêu, lọc thế nào); (2) khai thác tri thức giàu hơn từ teacher (feedback, feature, kết hợp nhiều kiểu); (3) quên thảm họa (catastrophic forgetting) khi chưng cất kỹ năng mới; (4) KD đáng tin cậy (trustworthiness: truthfulness, safety, fairness, privacy); (5) weak-to-strong distillation; (6) self-alignment.

6. Giới hạn

Tác giả nêu: (Declaration) không thể bao quát mọi công trình vì lĩnh vực tăng quá nhanh; ngoài ra, lưu ý tuân thủ điều khoản dịch vụ khi chưng cất từ ChatGPT/Llama (restrictions on developing competitive products).

Người ghi chú nhận xét:

- Survey nghiêng mạnh về **black-box, dữ liệu sinh, instruction tuning**; phần white-box (logit, hidden state) ngắn, chỉ vài đoạn trong 3.1.4 và 3.2.2. Không có phần về **nén có cấu trúc + KD** (pruning rồi phục hồi), không có phần về **vision-language distillation theo nghĩa nén** (Mục 4.5 chỉ bàn về dữ liệu instruction đa phương thức từ GPT-4).
- Hầu hết "kết quả" được trích lại từ bài gốc, không có chuẩn so sánh chung giữa các phương pháp, nên khó rút ra "phương pháp nào tốt hơn".
- Một số kết luận (như divergence phụ thuộc tác vụ) dựa trên một vài bài (GKD, MiniLLM); các bài sau (30, 31) tranh luận thêm.
- Phạm vi thời gian: đến khoảng đầu 2024; thiếu nhóm reasoning distillation (R-1, 38), scaling laws (41, 42), speculative KD (32).
- Bản trích văn bản dùng ở đây là PDF hai cột, một số bảng bị lẫn dòng; đã tránh trích số từ bảng mangled.

7. Liên hệ với các bài khác

Phần của survey	Bài trong lộ trình	Ghi chú
Nền tảng KD	02 Hinton KD, 05 KD survey, 03 FitNets	survey LLM bỏ qua phần cổ điển
SFT / SeqKD (3.2.1)	15 Sequence-level KD, 35 Self-Instruct	
Labeling/Expansion/Curation	35 Self-Instruct, 36 Distilling step-by-step, 37 Orca, 38 DeepSeek-R1 distillation	dữ liệu sinh từ teacher
Feature + Divergence (3.1.4, 3.2.2)	28 MiniLLM, 29 GKD, 30 DistiLLM, 31 Rethinking KL, 33 TAID, 34 DistiLLM-2	phân phối và biểu diễn
Similarity (hidden state)	03 FitNets, 18 TinyBERT, 19 MiniLM, 20 Causal distillation	trong survey LLM chỉ nhắc TED
Feedback / RL / Ranking (3.2.3–3.2.4)	25 LLaVA-MoD (preference distillation), Zephyr/DPO	
Self-knowledge	38 DeepSeek-R1, 06 Born-again networks	
Multimodal (4.5)	24 CLIP-KD, 25 LLaVA-MoD, 26 LLaVA-KD	survey chủ yếu nói về dữ liệu
Teacher–student gap	08 Teacher assistant, 09 Efficacy of KD, 41 Capacity gap law, 39 Small models struggle...	không thuộc survey
Scaling, tối ưu mẫu	42 Distillation scaling laws, 45 Sparse logit sampling	không thuộc survey

Cách dùng làm bản đồ: gặp một bài mới, hỏi ba câu: (1) tri thức lấy từ đâu (6 kiểu elicitation), (2) học bằng thuật toán nào (4 họ), (3) nhắm skill/miền nào.

8. Tóm tắt công thức / khái niệm

Khái niệm	Công thức / ý	Ghi chú
Pipeline KD LLM	steer → seed → generate → train	4 bước
Labeling	$y \sim p_T(y \mid I \oplus c \oplus x)$	x có sẵn
Expansion	x, y đều do teacher sinh	Self-Instruct

Khái niệm	Công thức / ý	Ghi chú
Curation	sinh từ meta-information m	UltraChat, phi
Feature	logit/hidden của teacher	white-box
Feedback	teacher chấm/sửa đầu ra student	RLAIF
Self-knowledge	student tự sinh và lọc	ReST
SFT / SeqKD	$-\log p_S(y\|x)$, $y \sim p_T$	baseline mạnh
Divergence	forward KL, reverse KL, JS	task-dependent
Similarity	$L_F(\Phi_T f_T, \Phi_S f_S)$	L1/L2/CE/MMD
RL distillation	reward model + PPO có KL tới π_{ref}	đắt
Ranking	DPO, RRHF, PRO	ổn định hơn RL

9. Kết luận

Survey phân loại KD cho LLM theo thuật toán, kỹ năng và lĩnh vực, đồng thời phân biệt bước khơi gợi năng lực với bước chưng cất. Bài tổng quan nhấn mạnh mở rộng dữ liệu và các hướng học suy luận, phản hồi, tự cải thiện bên cạnh bắt chước đầu ra.

Link gốc: https://arxiv.org/abs/2402.13116

Bài 28

MiniLLM: On-Policy Distillation of Large Language Models (2023)

Tác giả: Yuxian Gu, Li Dong, Furu Wei, Minlie Huang (Tsinghua CoAI; Microsoft Research).
Venue / arXiv: arXiv:2306.08543 (v1 tháng 6/2023; bản v6 ngày 31/1/2026 dùng để ghi chú này). Paper trong text đã trích không nêu venue.
Nguồn: https://arxiv.org/abs/2306.08543
Chủ đề: Distill LLM sinh văn bản (white-box) bằng **reverse KL** thay vì forward KL; tối ưu bằng policy gradient on-policy với ba mẹo ổn định (single-step decomposition, teacher-mixed sampling, length normalization).
Vị trí trong lộ trình: Đọc sau 02 (Hinton KD, forward KL), 15 (Sequence-level KD, SeqKD là baseline chính) và 06 (Born-again networks, student vượt teacher). Bài này mở nhóm KD cho LLM sinh văn bản (28–34). Nó đặt ra câu hỏi "dùng divergence nào, lấy mẫu từ đâu"; các bài sau trả lời tiếp: 29 GKD (on-policy với divergence tổng quát), 30 DistiLLM (skew KL, off-policy hiệu quả), 31 Rethinking KL (xem lại tranh luận forward/reverse), 33 TAID.

> Tài liệu học tập bằng tiếng Việt, là ghi chú diễn giải, không phải bản dịch nguyên văn paper. Các phép suy dẫn trình bày lại theo hướng dễ hiểu; số liệu lấy từ paper.

1. Bối cảnh

Hai loại KD cho LLM:

- **Black-box KD:** chỉ có văn bản do teacher sinh (ví dụ gọi API ChatGPT), student fine-tune trên cặp (prompt, response) đó.
- **White-box KD:** có cả phân phối xác suất đầu ra (và có thể hidden state) của teacher. Quan trọng hơn khi các LLM mã nguồn mở nở rộ. Trước bài này, white-box KD chủ yếu dùng cho mô hình hiểu ngôn ngữ nhỏ (<1B) kiểu DistilBERT/TinyBERT (16, 18).

Các KD chuẩn (word-level KD của Hinton, và sequence-level KD / SeqKD của Kim & Rush, bài 15) đều xấp xỉ giảm **forward KL** $\mathrm{KL}[p\|q_\theta]$ với p là teacher, q_θ là student. Nhắc nhanh:

$$\mathrm{KL}[p\|q] = \mathbb{E}_{y\sim p}\log\frac{p(y)}{q(y)}, \qquad \mathrm{KL}[q\|p] = \mathbb{E}_{y\sim q}\log\frac{q(y)}{p(y)}$$

- **Forward KL** $\mathrm{KL}[p\|q]$: phạt nặng nếu student gán xác suất ~0 ở nơi teacher có xác suất. Do đó student phải **phủ mọi mode** (mass-covering). Nếu student quá nhỏ để biểu diễn mọi mode, nó "phết" xác suất lên cả vùng trống giữa các mode (void regions), và khi sinh tự do sẽ ra những mẫu rất khó xảy ra dưới teacher.
- **Reverse KL** $\mathrm{KL}[q\|p]$: phạt nặng nếu student gán xác suất ở nơi teacher gần 0. Student **chọn vài mode chính** của teacher và bỏ phần đuôi (mode-seeking). Hình 2 trong bài báo là thí nghiệm đồ chơi: khớp hỗn hợp Gauss bằng một Gauss đơn.

Với phân loại văn bản, đầu ra ít lớp nên forward KL ổn. Với sinh văn bản mở, đầu ra của teacher có rất nhiều mode mà student nhỏ không biểu diễn hết, nên tác giả cho rằng forward KL là chưa tối ưu.

Khái niệm phụ:

- **On-policy:** lấy mẫu từ chính mô hình đang được huấn luyện (student), thay vì từ dữ liệu cố định hoặc teacher.

- **Exposure bias:** khi huấn luyện teacher forcing, mô hình luôn thấy tiền tố đúng; khi sinh tự do thì thấy tiền tố do chính nó tạo, tạo mismatch tích lũy.
- **Policy gradient (REINFORCE):** ước lượng gradient của kỳ vọng phần thưởng dưới một policy bằng $\mathbb{E}[\text{reward} \cdot \nabla \log \pi]$.
- **Reward hacking:** mô hình khai thác kẽ hở của hàm phần thưởng (ví dụ lặp từ để điểm cao).
- **ECE (Expected Calibration Error):** sai số hiệu chỉnh xác suất (thấp là tốt).

2. Ý tưởng chính

Đổi mục tiêu KD từ forward KL sang **reverse KL** $\mathrm{KL}[q_\theta \| p]$, để student tập trung vào các mode chính của teacher thay vì cố bắt chước cả đuôi dài. Vì kỳ vọng nằm trên mẫu của *student*, ta không thể dùng dữ liệu có sẵn, mà phải sinh mẫu từ student rồi chấm điểm bằng teacher: đó chính là bài toán policy gradient (giống RLHF, nhưng phần thưởng là $\log p/q$). Paper rút ra gradient, rồi thêm ba mẹo giảm phương sai và chặn reward hacking. Kết quả: student sinh câu trả lời chính xác hơn, ít exposure bias hơn, hiệu chỉnh tốt hơn so với KD và SeqKD, từ 120M đến 13B tham số.

3. Phương pháp chi tiết

3.1 Mục tiêu

Điều kiện: prompt $x \sim p_x$, response $y = \{y_t\}_{t=1}^{T}$.

$$\boxed{\theta^* = \arg\min_\theta \mathcal{L}(\theta) = \arg\min_\theta \mathrm{KL}[q_\theta \| p] = -\mathbb{E}_{x\sim p_x,\, y\sim q_\theta} \log \frac{p(y|x)}{q_\theta(y|x)}}$$

Góc nhìn thứ hai (Phụ lục A.1): coi sinh token là MDP, KD chuẩn là *behavior cloning*, còn MiniLLM là Inverse RL kiểu max-entropy. Lấy logit f của teacher làm Q-function, phần thưởng là $r_t = f(y_t) - \log \sum_{y'} e^{f(y')} = \log p(y_t|y_{<t}, x)$. Tối đa hóa $\sum_t r_t + H[q_\theta]$ cho ra (xấp xỉ, bỏ một số hạng biên) đúng $-\mathrm{KL}[q_\theta \| p]$.

3.2 Gradient bằng policy gradient (Phụ lục A.2, trình bày lại)

Bắt đầu từ $\nabla\mathcal{L} = -\nabla \int q_\theta(y) \log \frac{p(y)}{q_\theta(y)}\, dy$ (bỏ x cho gọn). Dùng quy tắc tích và $\nabla q_\theta = q_\theta \nabla \log q_\theta$:

$$\nabla\mathcal{L} = -\int q_\theta \nabla \log q_\theta \, \log \frac{p}{q_\theta}\, dy \; - \; \int q_\theta \nabla \log \frac{p}{q_\theta}\, dy$$

Vì $\nabla \log \frac{p}{q_\theta} = -\nabla \log q_\theta$, số hạng thứ hai bằng $+\int q_\theta \nabla \log q_\theta \, dy = \mathbb{E}_q[\nabla \log q_\theta]$, tức là một số hạng có kỳ vọng bằng 0 (nhưng paper giữ nó lại để có dạng "-1"):

$$\nabla\mathcal{L}(\theta) = -\mathbb{E}_{y\sim q_\theta}\Big[\Big(\log \frac{p(y|x)}{q_\theta(y|x)} - 1\Big)\nabla \log q_\theta(y|x)\Big]$$

Hằng số -1 chỉ đóng vai trò baseline, không đổi giá trị kỳ vọng. Phân rã theo token, vì $\log q_\theta(y_t|y_{<t})$ chỉ ảnh hưởng các token từ vị trí t trở đi:

$$\boxed{\nabla\mathcal{L}(\theta) = -\mathbb{E}_{x,\, y\sim q_\theta} \sum_{t=1}^{T} (R_t - 1)\, \nabla \log q_\theta(y_t|y_{<t}, x), \quad R_t = \sum_{t'=t}^{T} \log \frac{p(y_{t'}|y_{<t'}, x)}{q_\theta(y_{t'}|y_{<t'}, x)}}$$

Trực giác: $r_{t'} = \log p - \log q_\theta$ là "phần thưởng" từng bước: cao nếu teacher thấy token này hợp lý (p lớn), và bị trừ nếu student quá tự tin (q lớn), giữ cho student đa dạng. R_t là phần thưởng tích lũy từ bước t (reward-to-go).

Hai vấn đề (paper): phương sai cao và reward hacking; và R_t thiên về câu **ngắn** (tổng các r âm càng ít càng tốt), dẫn tới trả lời rỗng.

3.3 Ba mẹo

(1) Single-step decomposition. Lỗi ở token đầu cộng dồn qua cả câu nên rất ồn. Tách phần bước hiện tại r_t ra khỏi $R_t = r_t + R_{t+1}$. Phần kỳ vọng theo $y_t \sim q_\theta(\cdot|y_{<t})$ của r_t tính **trực tiếp bằng cách cộng trên toàn từ vựng** thay vì lấy mẫu:

$$\nabla\mathcal{L} = \underbrace{-\mathbb{E}\nabla\sum_t \mathbb{E}_{y_t\sim q_\theta(t)}[r_t]}_{(\nabla\mathcal{L})_{Single}} \underbrace{-\mathbb{E}\sum_t R_{t+1}\nabla\log q_\theta(y_t|y_{<t},x)}_{(\nabla\mathcal{L})_{Long}}$$

Chứng minh (A.3): từ $(R_t - 1) = R_{t+1} + (r_t - 1)$; số hạng $(r_t - 1)\nabla\log q_\theta$ có kỳ vọng theo y_t bằng $\nabla\mathbb{E}_{y_t}[-r_t]$... (dùng quy tắc tích và $\mathbb{E}\nabla\log q = 0$), chính là $\nabla\mathbb{E}_{y_t\sim q}[-r_t] = \nabla\,\mathrm{KL}[q_\theta(t)\|p(t)]$ ở bước t. Nghĩa là phần "một bước" chính là **KL token-level đúng nghĩa** (tính chính xác trên từ vựng), chỉ phần "dài hạn" mới cần REINFORCE.

(2) Teacher-mixed sampling. Student nhỏ hay sinh câu thoái hóa (lặp cụm) mà teacher lại chấm cao. Lấy mẫu từ hỗn hợp:

$$\tilde{p}(y_t|y_{<t},x) = \alpha\, p(y_t|y_{<t},x) + (1-\alpha)\, q_\theta(y_t|y_{<t},x)$$

rồi sửa lệch bằng importance weight $w_t = \prod_{t'\le t}\frac{q_\theta(y_{t'}|\cdot)}{\tilde{p}(y_{t'}|\cdot)}$. Tích này phương sai rất cao nên xấp xỉ $w_t \approx \frac{q_\theta(y_t|y_{<t},x)}{\tilde{p}(y_t|y_{<t},x)}$ (chỉ giữ thừa số bước hiện tại).

(3) Length normalization. Chia reward-to-go cho độ dài còn lại:

$$R_{t+1}^{Norm} = \frac{1}{T-t-1}\sum_{t'=t+1}^{T}\log\frac{p(y_{t'}|y_{<t'},x)}{q_\theta(y_{t'}|y_{<t'},x)}$$

3.4 Gradient cuối và thuật toán

$$\nabla\mathcal{L}(\theta) = -\mathbb{E}_{y\sim\tilde{p}}\sum_{t=1}^{T} w_t\Big[\nabla\sum_{y'\in V} q_\theta(y'|y_{<t},x)\log\frac{p(y'|y_{<t},x)}{q_\theta(y'|y_{<t},x)} + R_{t+1}^{Norm}\frac{\nabla q_\theta(y_t|y_{<t},x)}{q_\theta(y_t|y_{<t},x)}\Big]$$

Thêm **clipping** kiểu PPO cho phần long-term (tỉ số $\rho_t = q_\theta/\tilde{p}$, cắt ở $1\pm\epsilon$, $\epsilon = 0.2$) và **loss pre-training** $\mathcal{L}_{PT} = -\mathbb{E}_{d\sim D_{PT}}\log q_\theta(d)$ để giữ năng lực NLP chuẩn (kiểu PPO-ptx của InstructGPT). Pipeline giống RLHF.

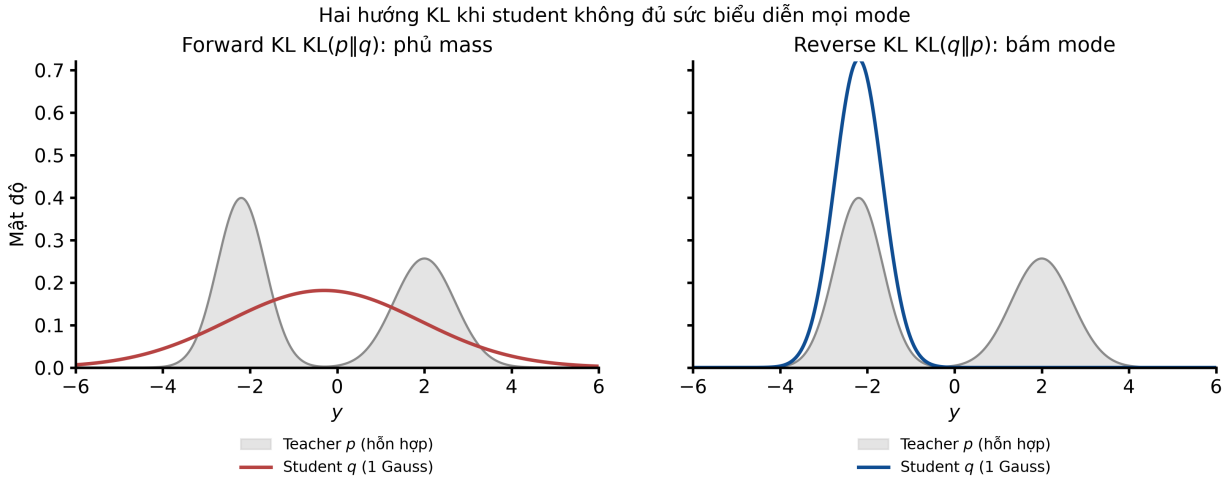
```
Input: dataset D (prompt, gt response), pretrain corpus D_PT, teacher p, student q_theta0
1. SFT student on D (ground truth) -> pick checkpoint with lowest val loss  (init)
2. repeat:
     sample prompts x from D
     roll out y ~ p_tilde = alpha*p + (1-alpha)*q_theta   (token by token)
     for each step t:
         single_t = grad_theta  sum_{v in V} q(v|.) * log( p(v|.) / q(v|.) )     # exact
         ↪  over vocab
         long_t   = R_norm_{t+1} * grad min[ rho_t, clip(rho_t, 1-eps, 1+eps) ]    #
         ↪  rho_t = q(y_t|.)/p_tilde(y_t|.)
         weight   = w_t ~= q(y_t|.)/p_tilde(y_t|.)
```

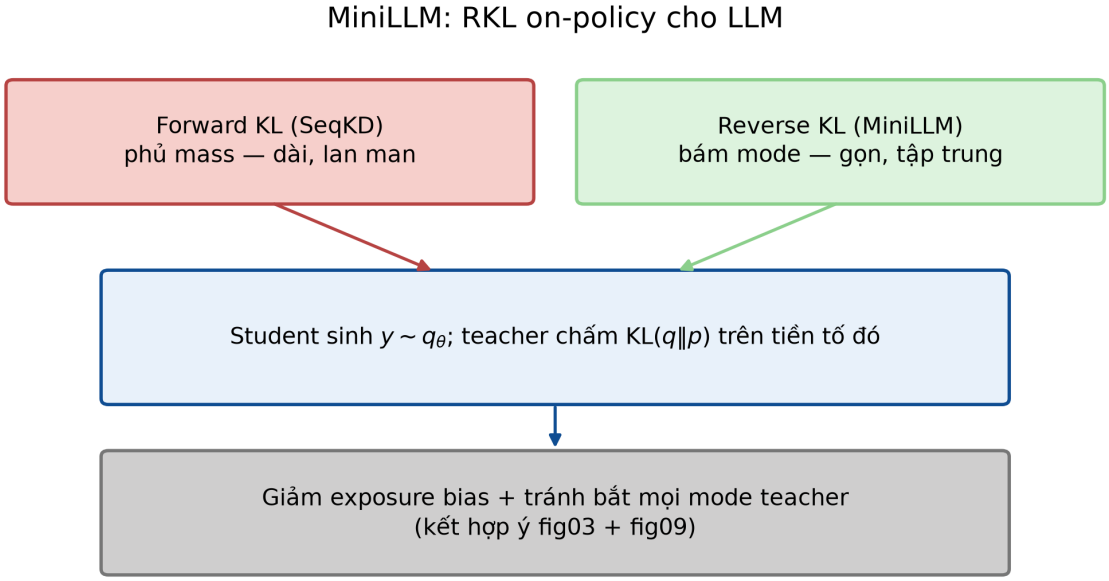
```
    L_PT = LM loss on batch from D_PT
    theta <- theta - lr * ( -(sum_t weight*single_t) - (sum_t long_t) + grad L_PT )
```

(Dấu của hai hạng tử "single" và "long" theo công thức (7) và Algorithm 1 của bài báo; người ghi chú tóm tắt dạng gradient, không thay đổi nội dung.)

3.5 Ví dụ nhỏ để thấy khác biệt forward/reverse

Teacher có hai đỉnh xác suất 50/50 tại token "A" và "B", và student chỉ biểu diễn được một đỉnh (ví dụ phân phối "chỉ một lựa chọn"). Forward KL tối thiểu sẽ ép student chia xác suất, dẫn đến một phân phối "ở giữa" gán xác suất cao cho các token vô nghĩa nếu cấu trúc hạn chế. Reverse KL sẽ chọn "A" (hoặc "B") với xác suất ~1: câu trả lời đúng nhưng mất đa dạng. Đây là mode-seeking vs mass-covering (Hình 2 paper dùng Gauss).





MiniLLM chọn reverse KL on-policy: student tự sinh, teacher chấm $\mathrm{KL}(q\|p)$ để bám mode và giảm exposure bias (minh hoạ, tự tính)

Khi khớp hỗn hợp bằng một Gauss, forward $\mathrm{KL}(p\|q)$ phủ mass (trải mỏng), reverse $\mathrm{KL}(q\|p)$ bám một mode

— trực giác chọn divergence cho KD sinh văn bản (minh hoạ, tự tính)

4. Thí nghiệm và kết quả chính

Tác vụ: instruction-following. Dữ liệu train: databricks-dolly-15K (15K cặp, tách 1K validation, 0.5K test, khoảng 12.5K train). D_{PT}: OpenWebText (GPT-2) hoặc RoBERTa corpus (còn lại). $\alpha = 0.2$.

Mô hình: teacher fine-tune sẵn trên D: GPT-2-1.5B (student GPT-2 120M/340M/760M), OPT-13B (student OPT 1.3B/2.7B/6.7B), LLaMA-13B (student LLaMA-7B); phụ lục: GPT-J-6B teacher.

Đánh giá: DollyEval (500), SelfInst (252), VicunaEval (80), S-NI (nhóm response ≥11 token), UnNI (10K mẫu); chỉ số Rouge-L, điểm GPT-4 (tỉ lệ so với ground truth), đánh giá người. Sinh ở temperature 1, trung bình 5 seed. Baseline: SFT w/o KD, KD (word-level), SeqKD.

Bảng 1 (trích, R-L và GPT4)

Teacher → student	Phương pháp	DollyEval GPT4 / R-L	SelfInst GPT4 / R-L	VicunaEval GPT4 / R-L	S-NI R-L	UnNI R-L
GPT-2 1.5B (teacher)		58.4 / 27.6	42.9 / 14.3	48.6 / 16.3	27.6	31.8
GPT-2 120M	SFT w/o KD	38.6 / 23.3	26.3 / 10.0	32.8 / 14.7	16.3	18.5
	KD	40.3 / 22.8	27.8 / 10.8	31.9 / 13.4	19.7	22.0
	SeqKD	41.2 / 22.7	26.2 / 10.1	31.0 / 14.3	16.4	18.8
	MiniLLM	44.7 / 24.6	29.2 / 13.2	34.1 / 16.9*	25.3	26.6
OPT 13B (teacher)		70.3 / 29.2	56.1 / 18.4	58.0 / 17.8	30.4	36.1
OPT 1.3B	SFT w/o KD	52.6 / 26.0	37.7 / 11.4	40.5 / 15.6	23.1	28.4
	KD	52.7 / 25.4	36.0 / 12.2	40.8 / 14.9	21.9	27.0
	SeqKD	51.0 / 26.1	36.6 / 12.7	42.6 / 16.6	21.4	28.2
	MiniLLM	60.7 / 26.7	47.0 / 14.8	50.6 / 17.9*	28.6	33.4
LLaMA 13B (teacher)		79.0 / 29.7	75.5 / 23.4	65.1 / 19.4	35.8	38.5
LLaMA 7B	SFT w/o KD	73.0 / 26.3	69.2 / 20.8	61.6 / 17.5	32.4	35.8
	KD	73.7 / 27.4	70.5 / 20.2	62.7 / 18.4	33.7	37.9
	SeqKD	73.6 / 27.5	71.5 / 20.8	62.6 / 18.1	33.7	37.6
	MiniLLM	76.4 / 29.0	73.1 / 23.2	64.1 / 20.7*	35.5	40.2*

(* = student vượt teacher theo paper.) MiniLLM tốt nhất ở hầu hết ô; student đôi khi vượt teacher về Rouge-L trên Vicuna, S-NI, UnNI. Tác giả đoán do teacher bị exposure bias từ teacher-forcing fine-tuning. Cải thiện nhất quán từ 120M đến 13B (Hình 1). Đánh giá người (LLaMA-7B vs teacher 13B trên SelfInst): MiniLLM được ưa hơn mọi baseline và tương đương teacher (Hình 4).

Chi phí: distill LLaMA-7B từ LLaMA-13B dưới 10 giờ trên 16 GPU V100 32GB; Phase 2 dùng lr 5e-6, batch 64, 256 câu mỗi lần thu thập, 4 inner epoch, 5000 bước, max length 512.

5. Phân tích / ablation

- **Teacher lớn hơn thì tốt hơn (ở đây):** với student GPT-2-125M, teacher GPT-2 340M/760M/1.5B, MiniLLM luôn thắng SeqKD và điểm student tăng theo kích thước teacher (Hình 5; không thấy capacity

gap trong dải này). Kết quả OPT tương tự (Phụ lục C.2).
- **Exposure bias (Hình 6):** ExAccErr của các baseline tăng theo độ dài sinh, MiniLLM thấp hơn nhiều và ngừng tích lũy sau ~150 token.
- **Calibration (Bảng 2, LLaMA-7B, zero-shot):**

	SST2 ECE ↓ / Acc	BoolQ ECE ↓ / Acc
Teacher	0.025 / 93.0	0.356 / 74.5
KD	0.191 / 84.7	0.682 / 63.5
SeqKD	0.243 / 66.5	0.681 / 62.8
MiniLLM	0.099 / 89.7	0.502 / 67.8

- **Độ dài đáp án (Hình 7):** với đáp án ngắn (≤5 token) mọi phương pháp tương đương (không gian đầu ra nhỏ, student phủ được mọi mode); với đáp án dài MiniLLM hơn rõ.
- **Đa dạng (Bảng 3):** Dist-4 DollyEval 99.0 (SFT 99.5, teacher 99.3), SelfInst 98.6 (SFT 99.0, teacher 99.1); loss LM test 3.95 / 5.33 (SFT 3.89 / 5.28), tức mất ít đa dạng. Tác giả lập luận một đáp án đúng thường là đủ.
- **Ablation ba mẹo (Bảng 4, GPT-2-125M ← 1.5B; Rouge-L validation / Dolly):** đầy đủ 27.4 / 24.6; bỏ length norm 17.4 / 14.7; bỏ teacher-mixed 22.3 / 20.4; bỏ single-step 27.0 / 23.7. Bỏ length norm hoặc teacher-mixed khiến reverse KL vẫn giảm nhưng student học sinh chuỗi lặp/ngắn/vô nghĩa (reward hacking; ví dụ ở Bảng 8 phụ lục); single-step chủ yếu giảm phương sai (Hình 8).
- **Hệ số α:** $\alpha = 0.2$ chung tốt; mô hình lớn ít nhạy (Hình 16, không có số cụ thể trong text).
- **Loss pre-training L_{PT} (Bảng 7):** thêm vào giữ năng lực phân loại (CLS) với 1.3B: 70.2 so với 65.7 khi bỏ; Inst. 52.8 so với 53.2 (tức không đổi nhiều); 7B: CLS 78.8 so với 74.3, Inst. 71.2 so với 71.1.

6. Giới hạn

Tác giả nêu/suy ra: reverse KL có nguy cơ mất mode, nên họ đã đo đa dạng và cho là chấp nhận được (nhưng chỉ đo trên Dist-4/loss LM, và lập luận "một đáp án đúng đủ" không đúng cho mọi ứng dụng). Cần teacher có phân phối đầu ra (white-box). Cần cùng vocabulary (suy ra từ cách tính p/q trên từ vựng).

Người ghi chú nhận xét:

- Chi phí: phải rollout từ student (và từ hỗn hợp với teacher) mỗi bước, tức là chạy sinh tự hồi quy trong vòng lặp huấn luyện, chậm và phức tạp hơn nhiều so với KD offline.
- Thí nghiệm chỉ trên instruction-following với dữ liệu nhỏ (12.5K) và đánh giá một phần dùng GPT-4 làm giám khảo; không có reasoning/toán/code.
- Các cải thiện Rouge-L trên một số tập khá nhỏ (ví dụ 0.1–0.6) và không có khoảng tin cậy; ảnh hưởng α, ϵ khó tái hiện chính xác.
- Mô hình student tối đa 7B, teacher tối đa 13B; chưa kiểm tra tỉ số kích thước lớn (ví dụ 40x).
- Bài sau (31 Rethinking KL, 30 DistiLLM) cho rằng ranh giới forward/reverse nhạy cảm và có thể dùng các biến thể ổn định hơn; đây là cách đọc của người ghi chú dựa trên danh sách lộ trình, cần đọc các bài đó để xác nhận.

7. Liên hệ với các bài khác

- **02 Hinton KD, 15 Sequence-level KD:** baseline KD và SeqKD là forward KL; MiniLLM nhắm vào điểm yếu của chúng.
- **06 Born-again networks:** nhắc đến việc student vượt teacher (Rouge-L).
- **29 GKD:** cùng thời, cũng on-policy nhưng dùng divergence tổng quát và không rút gradient policy-gradient (dùng gradient trực tiếp qua log-prob); đọc để so sánh.

- **30 DistiLLM:** skew KL, giảm chi phí sinh mẫu bằng replay/adaptive off-policy.
- **31 Rethinking KL:** phân tích lại vai trò forward/reverse KL.
- **33 TAID, 34 DistiLLM-2:** các phương pháp sau cải tiến cách nội suy giữa teacher và student.
- **26 LLaVA-KD (Bảng 5):** thử RKL/FKL/JSD cho MLLM nhỏ; RKL (55.0) không hơn FKL (55.5), cho thấy lợi thế reverse KL không phổ quát.
- **41 Capacity gap law, 39 Small models struggle...:** ở đây không thấy capacity gap trong dải GPT-2, ngược lại với một số bài sau.
- **36 Distilling step-by-step, 37 Orca, 38 DeepSeek-R1:** các hướng black-box/dữ liệu sinh, bổ sung cho white-box.

8. Tóm tắt công thức / khái niệm

Khái niệm	Công thức / ý	Ghi chú
Forward KL	$\mathrm{KL}[p\|q] = \mathbb{E}_p \log \frac{p}{q}$	mass-covering; KD/SeqKD
Reverse KL	$\mathrm{KL}[q\|p] = \mathbb{E}_q \log \frac{q}{p}$	mode-seeking; MiniLLM
Gradient	$-\mathbb{E}_{y\sim q} \sum_t (R_t - 1) \nabla \log q(y_t\|y_{<t})$	policy gradient
Reward-to-go	$R_t = \sum_{t'\geq t} \log \frac{p}{q}$	thiên về câu ngắn
Single-step	KL token-level tính đúng trên từ vựng	giảm phương sai
Teacher-mixed	$\tilde{p} = \alpha p + (1-\alpha) q$, $\alpha = 0.2$	chặn reward hacking
Length norm	chia R_{t+1} cho $T - t - 1$	tránh câu rỗng/ngắn
Loss phụ	$\mathcal{L}_{PT}$ trên D_{PT}	giữ năng lực chuẩn
Pipeline	SFT student $\rightarrow$ on-policy (PPO-like clip $\epsilon = 0.2$)	giống RLHF

9. Kết luận

MiniLLM dùng reverse KL và policy gradient on-policy để huấn luyện student, cùng ba kỹ thuật ổn định. Trên các mô hình từ 120M đến 13B và năm bộ dữ liệu, bài báo báo cáo kết quả tốt hơn KD/SeqKD, cùng cải thiện về exposure bias, calibration và văn bản dài. Đổi lại, huấn luyện phức tạp hơn và có nguy cơ giảm đa dạng.

Link gốc: https://arxiv.org/abs/2306.08543

Bài 29

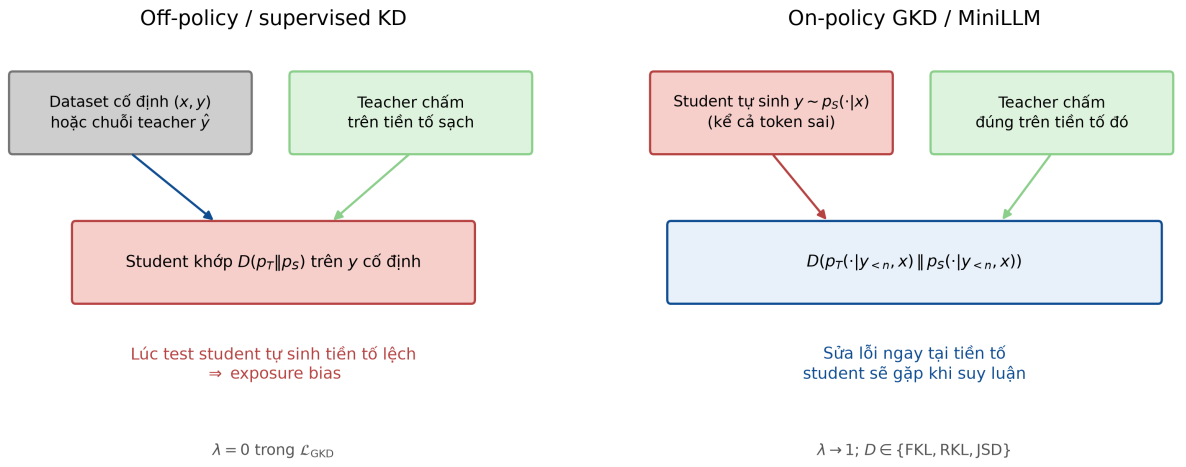
On-Policy Distillation of Language Models: Learning from Self-Generated Mistakes (2024)

Tác giả: Rishabh Agarwal, Nino Vieillard, Yongchao Zhou, Piotr Stanczyk, Sabela Ramos, Matthieu Geist, Olivier Bachem (Google DeepMind; Mila; U. Toronto).
Venue / arXiv: arXiv:2306.13649 (v3, 17/01/2024); ICLR 2024.
Nguồn: https://arxiv.org/abs/2306.13649
Chủ đề: Distillation cho mô hình sinh tự hồi quy (LLM); on-policy distillation (GKD); chọn divergence (forward KL, reverse KL, JSD); kết hợp distillation với RL fine-tuning.
Vị trí trong lộ trình: Đọc trước: 02 (Hinton KD), 15 (Sequence-level KD), 16 (DistilBERT), 27 (LLM KD survey), 28 (MiniLLM, cũng dùng reverse KL). Bài này đặt nền cho nhóm "on-policy" của LLM distillation: đưa ý tưởng imitation learning (DAgger) vào KD, và đưa ra họ loss tổng quát. Đọc tiếp: 30 (DistiLLM), 31 (Rethinking KL), 32 (Speculative KD), 33 (TAID), 34 (DistiLLM-2), 47 (ShortOPD).

> Đây là ghi chú học tập bằng tiếng Việt, diễn giải lại, không phải bản dịch paper. Con số lấy từ văn bản paper; nhiều hình trong bản trích xuất PDF bị vỡ nên chỗ nào không đọc chắc chắn được thì ghi rõ.

1. Bối cảnh

Knowledge distillation (KD) = huấn luyện mô hình nhỏ (student) bắt chước mô hình lớn (teacher). Với mô hình sinh tự hồi quy (mỗi bước sinh 1 token dựa trên các token trước), có hai cách phổ biến trước bài này:



Off-policy KD chỉ thấy tiền tố sạch; on-policy GKD/MiniLLM để student tự sinh rồi teacher chấm trên đúng tiền tố đó để giảm exposure bias (minh hoạ, tự tính)

- **Supervised KD** (kiểu Hinton/DistilBERT): lấy một tập dữ liệu cố định (x, y) (nhãn gốc hoặc dữ liệu có sẵn), teacher gán xác suất từng token, student khớp phân phối đó bằng forward KL.
- **SeqKD** (Kim & Rush, note 15): cho teacher sinh trước cả chuỗi đầu ra, rồi SFT student trên chuỗi đó. Tốn chi phí sinh bằng teacher.

Cả hai đều dùng **tập chuỗi cố định**. Vấn đề: lúc suy luận student tự sinh từng token và gặp những tiền tố $y_{<n}$ do chính nó tạo ra (có thể lệch), trong khi khi huấn luyện chỉ thấy tiền tố "sạch". Sai ở bước đầu kéo theo sai các bước sau. Đây là **exposure bias / train-inference distribution mismatch**, vấn đề kinh điển của imitation learning (Ross & Bagnell; DAgger).

Vấn đề thứ hai: forward KL ép student phủ toàn bộ support của teacher. Student nhỏ không đủ sức, nên đặt xác suất vào token mà teacher cho xác suất thấp, dẫn đến mẫu sinh kém chất lượng khi sampling theo nhiệt độ (paper minh họa ở Figure A.16).

2. Ý tưởng chính

GKD nhìn chưng cất như một bài toán imitation learning: student tự sinh chuỗi, còn teacher cung cấp phân phối token trên chính các tiền tố đó. Student nhờ vậy được huấn luyện trên những trạng thái gần hơn với trạng thái gặp lúc suy luận. Khung này cũng cho phép chọn phân kỳ, chẳng hạn forward KL, reverse KL hoặc JSD, thay vì cố định một loss.

3. Phương pháp chi tiết

3.1 Ký hiệu

Student p_S^θ, teacher p_T, input x, chuỗi đầu ra y dài L_y. Độ lệch ở mức chuỗi là trung bình theo token của divergence D giữa hai phân phối next-token:

$$D(p_T \| p_S^\theta)(y|x) := \frac{1}{L_y} \sum_{n=1}^{L_y} D(p_T(\cdot|y_{<n}, x) \,\|\, p_S^\theta(\cdot|y_{<n}, x))$$

Generalized JSD (bị chặn, nội suy giữa forward và reverse KL), với $0 < \beta < 1$:

$$D_{JSD(\beta)}(P\|Q) = \beta\, D_{KL}(P \,\|\, \beta P + (1-\beta)Q) + (1-\beta)\, D_{KL}(Q \,\|\, \beta P + (1-\beta)Q)$$

Paper nhắc (Huszár 2015) rằng $\lim_{\beta\to 0} D_{JSD(\beta)}/\beta = D_{KL}(P\|Q)$, nên β nhỏ có gradient giống forward KL, β gần 1 giống reverse KL.

Gloss: **forward KL** $D_{KL}(p_T\|p_S)$ là mean-seeking (student phải phủ mọi chỗ teacher có khối lượng); **reverse KL** $D_{KL}(p_S\|p_T)$ là mode-seeking (student dồn vào các mode của teacher, chấp nhận bỏ sót, ít đa dạng hơn).

3.2 Mục tiêu GKD

$$\boxed{\mathcal{L}_{GKD}(\theta) = (1-\lambda)\, \mathbb{E}_{(x,y)\sim(X,Y)}[D(p_T\|p_S^\theta)(y|x)] + \lambda\, \mathbb{E}_{x\sim X}\, \mathbb{E}_{y\sim p_S(\cdot|x)}[D(p_T\|p_S^\theta)(y|x)]}$$

- $\lambda \in [0,1]$ là **student data fraction** (tỉ lệ dữ liệu do student sinh).
- Không backprop qua quá trình sampling của student (ổn định, rẻ).
- $\lambda = 0$ và forward KL = Supervised KD; $\lambda = 1$ và forward KL = on-policy KD.
- ImitKD (Lin et al. 2020) và f-distill (Wen et al. 2023) được xem như biến thể "mixed" ($\lambda = 0.5$) với forward KL và total variation.
- Giả định: student đã được **SFT** trước (đủ khả năng sinh chuỗi có chất lượng để teacher còn phản hồi được), giống quy trình hai bước SFT rồi RLHF.

3.3 Pseudo-code (Algorithm 1)

```
Cho: teacher p_T, student p_S(theta), dataset (X, Y)
Siêu tham số: lambda, divergence D, learning rate eta
for step k in 1..K:
    u ~ Uniform(0,1)
    if u <= lambda:
        lấy batch input x từ X
        sinh y ~ p_S(.|x)  (nhiệt độ 1 để giữ đa dạng, không lấy gradient qua sampling)
    else:
```

```
    lấy batch (x, y) từ dữ liệu cố định (nhãn gốc hoặc do teacher sinh)
# teacher chỉ chạy forward để lấy logit trên chính y này
loss = mean_over_tokens D( p_T(.|y_<n, x) || p_S(.|y_<n, x) )
theta <- theta - eta * grad loss
```

Ví dụ tay: student sinh "Paris is the capital of Germ..." (sai ở token thứ 6). Với supervised KD, student chưa bao giờ thấy tiền tố sai này khi huấn luyện. Với GKD, teacher đưa phân phối tại vị trí "Germ" (nghiêng hẳn về "France") và student được kéo về đó ngay tại tiền tố lỗi.

3.4 Kết hợp RL fine-tuning

Vì GKD on-policy chỉ cần mẫu từ student, nó ghép tự nhiên với RLHF/RLAIF:

$$\mathbb{E}_x\Big[(1-\alpha)\,\mathbb{E}_{y\sim p_S^\theta}[r(y)] - \alpha\,\mathbb{E}_{y\sim p_S}[D(p_T\|p_S^\theta)(y|x)]\Big]$$

$\alpha = 1$ là chỉ distillation. Paper khuyến nghị reverse KL hoặc JSD(0.9) nếu muốn sửa ít workflow RL hiện có (vì RLHF vốn dùng reverse KL để giữ gần policy gốc).

4. Thí nghiệm và kết quả chính

Mô hình: teacher T5-XL (~3B) đã SFT; student T5-small (77M), T5-base (250M), T5-large (800M), nhỏ hơn teacher 38x, 12x, 3.8x. Optimizer Adafactor. Mọi baseline xuất phát từ cùng student đã SFT.

Biến thể GKD: divergence thuộc {forward KL, reverse KL, JSD(0.1), JSD(0.5), JSD(0.9)}; $\lambda \in \{0, 0.5, 1\}$.

Tác vụ	Dữ liệu / metric	Điểm đáng nhớ (theo văn bản paper)
Tóm tắt	XSum, ROUGE-2	On-policy GKD vượt SeqKD, supervised KD, ImitKD, f-distill; JSD(0.9) tốt nhất ở cả greedy và sampling (Fig. 2)
Dịch	WMT14 en-de, BLEU	Teacher BLEU 28; student small 25.58, base 26.98; thuần on-policy tốt nhất (Fig. 6)
Suy luận	GSM8K + CoT, accuracy có calculator	Teacher Flan-T5-XL 27.9; student base 10.16 (greedy); on-policy vượt mọi baseline (Fig. 9)
Task-agnostic	FLAN2021 (5.36M mẫu, 62 tác vụ); MMLU, BBH	Reverse KL on-policy tốt nhất; teacher 52.4% MMLU / 41% BBH; student large 35.6% / 31.25% (Fig. 10)

Các con số tổng hợp paper nêu: gain tương đối so với baseline, trung bình trên các cỡ student, là **2.1x** (tóm tắt), **1.7x** (dịch), **1.9x** (suy luận) (tính trên mức cải thiện so với student ban đầu). Hình 1 ghi mức tăng **+111%** (XSum), **+70%** (WMT), **+90%** (GSM8K) cho T5-small so với baseline tốt nhất (cách đọc chính xác của các phần trăm này phụ thuộc hình bị vỡ; ở đây chỉ nêu lại các con số xuất hiện trong chữ). Task-agnostic: cải thiện tuyệt đối **2% trên BBH và 1% trên MMLU** (cho T5-base). Ngoài ra, T5 nhỏ hơn PaLM (540B) 7000x vẫn vượt few-shot PaLM trên XSum.

Data efficiency (Fig. 3): distill T5-small với 1K (0.5%), 10K (5%), 50K (25%) mẫu XSum. On-policy GKD chỉ dùng 5% dữ liệu, không cần summary ground-truth, đã vượt supervised KD và ImitKD dùng toàn bộ dữ liệu có nhãn.

Cấu hình (XSum/GSM8K): 40,000 bước, batch 32, LR 3e-4 (T5-small XSum dùng 1e-3), warmup 2,000 bước, cooldown tuyến tính từ bước 30,000 đến 40,000. Nhiệt độ teacher 0.1 cho các thí nghiệm sampling trên XSum.

GKD + RL (Fig. 5): dùng điểm entailment của NLI T5-XXL làm reward. α càng lớn thì ROUGE-2 càng cao nhưng cải thiện factual consistency càng giảm. GKD+RL đạt ROUGE-2 cao hơn RLEF* và tính nhất quán sự thật cao hơn cả teacher.

Self-distillation (Phụ lục A.1): Flan-T5-large làm cả teacher (SFT GSM8K, 20.5%) và student (chưa train GSM8K, 14.4%). Student sau on-policy GKD vượt teacher.

5. Phân tích / ablation

- **Dữ liệu on-policy là yếu tố quan trọng nhất.** Trên XSum, WMT, GSM8K, biến thể on-policy và mixed luôn vượt biến thể chỉ dùng dữ liệu cố định. Trên GSM8K, hiệu năng tăng đều khi tỉ lệ dữ liệu student vượt 25% (Fig. 8).
- **Divergence tối ưu phụ thuộc tác vụ và cách decode.** Với greedy, chọn divergence ít ảnh hưởng; với sampling nhiệt độ 1, divergence mode-seeking (JSD(0.5), JSD(0.9), reverse KL) cho chất lượng cao hơn nhưng đa dạng thấp hơn (đo bằng Self-BLEU, Fig. 4). Hạ nhiệt độ làm các divergence gần nhau hơn, đồng thời đa dạng giảm.
- Trên WMT, JSD tổng quát tốt hơn forward/reverse KL, khoảng cách thu hẹp khi student lớn hơn. Trên instruction tuning, reverse KL tốt hơn nhiều so với forward KL; tác giả **giả thuyết** (không kiểm chứng) rằng tính mode-seeking giúp mô hình tập trung vào ý định chính của chỉ dẫn.
- **Chi phí:** sampling từ student làm tăng chi phí xấp xỉ 1.8x, 2x, 2.2x so với lấy mẫu từ dữ liệu cố định, ứng với tỉ lệ cỡ 38x, 12x, 3.8x (trên GSM8K). Tác giả lập luận chi phí chủ yếu nằm ở serving, nên overhead này đáng giá.

6. Giới hạn

Tác giả nêu/ngụ ý:

- Cần student đã SFT đủ tốt; không áp dụng cho student khởi tạo ngẫu nhiên.
- Chi phí sinh mẫu từ student trong vòng lặp (khoảng 2x).
- Divergence tốt nhất phụ thuộc tác vụ, phải tune (trade-off chất lượng/đa dạng).
- Mọi thí nghiệm dùng họ T5 (encoder-decoder) cỡ nhỏ.

Nhận xét:

- Không có thí nghiệm với decoder-only LLM hiện đại, chuỗi dài, hay teacher nhiều tỉ tham số khác tokenizer.
- Không backprop qua sampling nên là một ước lượng gradient thiên lệch (giống DAgger/behavior cloning on-policy chứ không phải policy gradient thật), nhưng ổn định.
- Teacher phải truy cập được logit ở mọi vị trí, nên không áp dụng cho teacher chỉ có API văn bản.
- Nếu chuỗi student sinh ra quá lệch, teacher có thể cho nhãn kém tin cậy trên tiền tố ngoài phân phối của chính nó (không được paper phân tích).

7. Liên hệ với các bài khác

- **02 Hinton KD / 16 DistilBERT:** GKD với $\lambda = 0$, forward KL chính là supervised KD token-level.
- **15 Sequence-level KD:** SeqKD là SFT trên chuỗi teacher; GKD chuyển nguồn chuỗi sang student.
- **28 MiniLLM:** cùng bắt đầu từ reverse KL cho LLM nhưng tối ưu bằng policy gradient; GKD không backprop qua sampling.
- **30 DistiLLM:** đề xuất skew KL và cơ chế chọn lọc dữ liệu student (adaptive off-policy) để giảm chi phí on-policy của GKD.
- **31 Rethinking KL:** phân tích lại vì sao forward/reverse KL thực tế hội tụ giống nhau.
- **32 Speculative KD:** thêm teacher vào vòng sinh mẫu (interleaved sampling) để teacher sửa các token student tệ.

- **33 TAID, 34 DistiLLM-2:** các hướng tiếp theo về nội suy phân phối và loss tương phản.
- **47 ShortOPD:** (theo tên) on-policy distillation cho mô hình đã cắt; GKD là tổ tiên trực tiếp.

Gợi ý học tập

So sánh dữ liệu cố định với chuỗi do student tự sinh khi giữ ngân sách tương đương. Báo cáo cả chi phí sinh mẫu và chất lượng đầu ra để đánh giá sự đánh đổi của huấn luyện on-policy.

8. Tóm tắt công thức / khái niệm

Khái niệm	Nội dung
Exposure bias	Train trên tiền tố sạch, suy luận trên tiền tố tự sinh
On-policy KD	Train trên chuỗi student sinh, nhãn là phân phối token của teacher
GKD	$\mathcal{L} = (1-\lambda)\,\mathbb{E}_{data}[D] + \lambda\,\mathbb{E}_{y\sim p_S}[D]$
λ	Tỉ lệ dữ liệu do student sinh (0 = supervised, 1 = thuần on-policy)
JSD(β)	Nội suy bị chặn giữa forward KL ($\beta \to 0$) và reverse KL ($\beta \to 1$)
Forward / reverse KL	Mean-seeking / mode-seeking
GKD + RL	$(1-\alpha)$ reward $-\ \alpha$ divergence on-policy
Chi phí	Khoảng 1.8x-2.2x so với lấy mẫu cố định (GSM8K)

9. Kết luận

GKD cho student tự sinh chuỗi và nhận phân phối đánh giá của teacher trên các tiền tố đó. Trong các thiết lập T5 được khảo sát, dữ liệu on-policy vượt dữ liệu cố định; lựa chọn phân kỳ phụ thuộc tác vụ và cách giải mã.

Nguồn gốc: https://arxiv.org/abs/2306.13649

Bài 30

DistiLLM: Towards Streamlined Distillation for Large Language Models (2024)

Tác giả: Jongwoo Ko, Sungnyun Kim, Tianyi Chen, Se-Young Yun (KAIST AI; Microsoft).
Venue / arXiv: arXiv:2402.03898 (v2, 03/07/2024); ICML 2024 (PMLR 235).
Nguồn: https://arxiv.org/abs/2402.03898
Chủ đề: Distillation cho LLM tự hồi quy: skew KL / skew reverse KL, và cách dùng dữ liệu student tự sinh (SGO) hiệu quả bằng scheduler thích nghi và replay buffer.
Vị trí trong lộ trình: Đọc trước: 02 (Hinton KD), 15 (Sequence-level KD), 28 (MiniLLM), 29 (GKD). Bài này giữ ý tưởng on-policy của GKD nhưng (a) thay loss bằng skew KL ổn định hơn, (b) giảm mạnh chi phí sinh mẫu. Đọc tiếp: 31 (Rethinking KL), 33 (TAID, cũng nội suy phân phối), 34 (DistiLLM-2, cùng nhóm tác giả).

> Ghi chú học tập bằng tiếng Việt, diễn giải lại, không phải bản dịch. Một số bảng/hình trong bản trích xuất PDF bị lẫn cột; chỉ các số đọc rõ mới được nêu lại.

1. Bối cảnh

Distillation cho LLM trước bài này có hai nhóm vấn đề.

(a) Hàm mục tiêu không chuẩn. KL thuận (forward KLD) $D_{KL}(p\|q_\theta)$ ép student phủ mọi chỗ teacher có xác suất, dẫn đến *mode averaging* (phân phối student quá "mịn"). Reverse KL (MiniLLM) hoặc generalized JSD (GKD, note 29) giảm vấn đề này nhưng thiếu phân tích lý thuyết và kết quả phụ thuộc tác vụ.

(b) Dữ liệu và chi phí. Dữ liệu cố định gây exposure bias (xem note 29). Dùng **student-generated outputs (SGO)** giúp nhưng: (i) sinh SGO ở mỗi iteration làm thời gian train tăng rất nhiều (tới 80% tổng thời gian dành cho sinh, và tới 5.5x runtime theo Fig. 7 của bài báo); (ii) teacher chưa từng thấy SGO nên có thể cho phản hồi nhiễu. Ví dụ Fig. 1: teacher GPT-2 XL cho validation loss thấp cho câu trả lời sai nhưng ngắn ("Christopher Columbus") và loss cao cho câu dài đúng, nên dùng $p(y|x)$ làm đích có thể dẫn student đi sai.

Gloss: **SGO** là chuỗi do chính student sinh; **replay buffer** (từ RL off-policy) là kho lưu mẫu cũ để dùng lại thay vì sinh mới.

2. Ý tưởng chính

DistiLLM có hai thành phần. Một: **skew KL (SKL)** và **skew reverse KL (SRKL)**: so student với một *hỗn hợp* $\alpha p + (1-\alpha)q_\theta$ thay vì so thẳng với p, nhờ đó mẫu số của tỉ số không bao giờ về 0, gradient ổn định và hội tụ nhanh. Hai: **adaptive off-policy**: lúc đầu dùng chủ yếu dữ liệu cố định, tăng dần xác suất dùng SGO theo validation loss (tránh phản hồi nhiễu), và chỉ sinh SGO thưa rồi lưu vào replay buffer để tái sử dụng (tránh sinh mỗi bước). Hai thành phần cộng hưởng vì SKL hội tụ nhanh nên chịu được dữ liệu "hơi cũ" từ buffer.

3. Phương pháp chi tiết

3.1 Skew KLD

Ký hiệu teacher p, student q_θ. Với tham số trộn $\alpha \in [0,1]$:

$$\boxed{D_{SKL}^{(\alpha)}(p, q_\theta) = D_{KL}(p,\ \alpha p + (1-\alpha)q_\theta)}, \qquad D_{SRKL}^{(\alpha)}(p, q_\theta) = D_{KL}(q_\theta,\ (1-\alpha)p + \alpha q_\theta)$$

Gradient. KL thường theo θ (cho một mẫu): $\nabla_\theta D_{KL}(p, q_\theta) = -r_{p,q_\theta} \nabla_\theta q_\theta(y|x)$, với $r = p/q_\theta$. Khi $q_\theta \to 0$, r nổ, gradient lớn và nhiễu. Với SKL:

$$\nabla_\theta D^{(\alpha)}_{SKL}(p, q_\theta) = -(1-\alpha)\, r_{p,\tilde{q}_\theta}\, \nabla_\theta q_\theta(y|x), \quad \tilde{q}_\theta = \alpha p + (1-\alpha) q_\theta$$

Vì $\tilde{q}_\theta \geq \alpha p$, mẫu số không về 0, hệ số gradient bị chặn: đây là **gradient ổn định**. Gradient của reverse KL là $-(\log r_{q_\theta,p} + 1)\nabla_\theta q_\theta$ và SRKL có dạng tương tự với hệ số $\log r_{q_\theta,\tilde{p}} + 1 - \alpha r_{q_\theta,\tilde{p}}$, $\tilde{p} = (1-\alpha)p + \alpha q_\theta$.

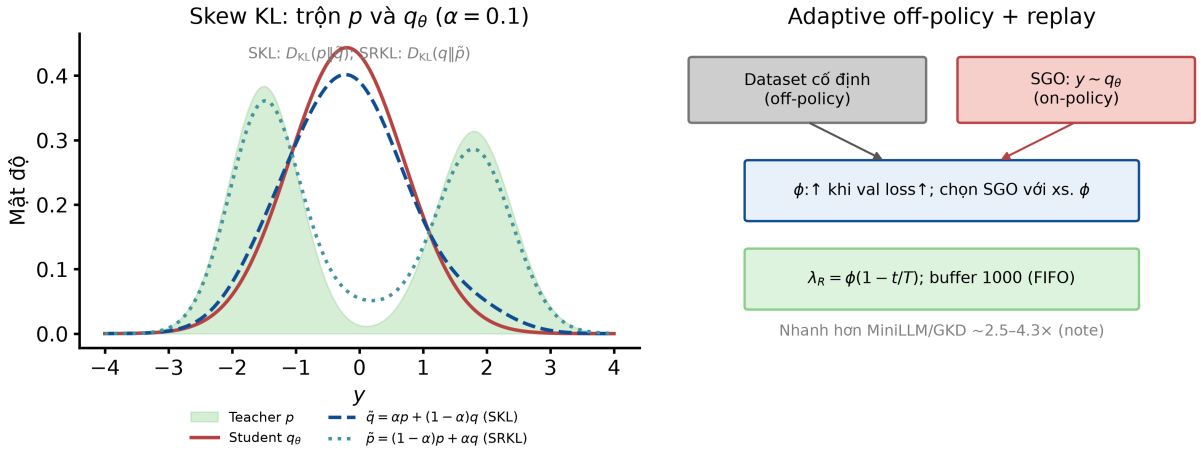
Sai số xấp xỉ nhỏ (Theorem 1). Với ước lượng thực nghiệm từ n mẫu, sai số L_2 của α-SKL bị chặn bởi $\frac{c_1(\alpha)}{n^2} + \frac{c_2 \log^2(\alpha n)}{n} + \frac{c_3 \log^2(c_4 n)}{\alpha^2 n}$ (với $c_1(\alpha) = \min\{1/\alpha^2,\ \chi^2(p_1, p_2)^2/(1-\alpha)^2\}$), tức α lớn làm sai số ước lượng nhỏ đi. Nhưng α lớn cũng làm hệ số gradient nhỏ đi, và optimizer hiện đại (Adam) sẽ bù lại nên lợi ích bị triệt tiêu. Khi xét sai số chuẩn hoá theo $1/(1-\alpha)$ (Remark 1), quan hệ giữa chặn trên và α là hàm lồi, nên cần cân bằng. Kết luận thực nghiệm: $\alpha = 0.1$ tốt nhất cho cả SKL và SRKL (Fig. 8), khớp với chỗ L2 chuẩn hoá nhỏ nhất (Fig. 3).

Khác biệt với JSD: $D^{(\beta)}_{JSD} = \beta D^{(1-\beta)}_{SKL}(p, q_\theta) + (1-\beta) D^{(\beta)}_{SKL}(q_\theta, p)$ không thể đặt mức skew nhỏ cho cả hai số hạng cùng lúc, còn S(R)KL với α vừa phải thì đạt L2 hợp lý.

Ghi chú: paper chỉ làm việc với họ KL vì nó phân rã được từ mức chuỗi xuống mức token (total variation không phân rã như vậy).

3.2 Adaptive off-policy

- **SGO scheduler thích nghi.** Xác suất dùng SGO là ϕ (còn $1-\phi$ dùng dữ liệu cố định). Bắt đầu $\phi = 0$. Mỗi lần validate, nếu validation loss tăng hơn lần trước cộng ϵ, thì $\phi \leftarrow \min(\phi + 1/N_\phi, 1)$, ngược lại giữ nguyên ($N_\phi = 10$ trong các thí nghiệm; $\phi_0 = 0$). Ý tưởng: huấn luyện trên SGO đa dạng làm giảm mismatch và giảm validation loss; khi loss có dấu hiệu xấu đi thì tăng SGO.
- **Off-policy với replay buffer.** Chỉ sinh SGO mới với xác suất $\lambda_R = \phi(1 - t/T)$, lưu vào buffer $\mathcal{D}_R$ (dung lượng 1000; đầy thì thay mẫu cũ nhất); khi cần dùng SGO thì rút ngẫu nhiên từ buffer. Hệ số $\zeta = 1 - t/T$ là **replay ratio** giảm tuyến tính: đầu train (student thay đổi nhanh) dùng nhiều SGO mới để giảm bias, cuối train (đã hội tụ) tái sử dụng buffer nhiều.



DistiLLM: Skew KL/SRKL với hỗn hợp $\alpha p + (1-\alpha) q$ và adaptive off-policy + replay SGO (minh hoạ, tự tính)

3.3 Pseudo-code (Algorithm 1, rút gọn)

```
Cho: teacher p, student q_theta, dataset D, D_val, buffer D_R = {}, phi=0, T bước
for t in 1..T:
```

```
u ~ U(0,1)
if u < phi*(1 - t/T):          # sinh SGO mới (xác suất lambda_R, thưa)
    y_tilde ~ q_theta(.|x);  D_R += {(x, y_tilde)}
if u < phi:   batch <- mẫu ngẫu nhiên từ D_R   # dùng SGO (off-policy)
else:         batch <- mẫu từ D                 # dữ liệu cố định
cập nhật theta bằng loss SKL (hoặc SRKL), alpha=0.1
if đến lượt validate:
    L = val_loss(q_theta)
    if L > L_prev + eps:  phi = min(phi + 1/N_phi, 1.0)
    else:                 L = L_prev   # giữ phi
```

Cấu hình triển khai: SRKL với $\alpha = 0.1$, phi ban đầu 0, buffer 1000 (theo ablation).

4. Thí nghiệm và kết quả chính

Instruction-following (task-agnostic). Dữ liệu databricks-dolly-15k (14K train, 500 val, 500 test). Đánh giá: Dolly eval, Self-Instruct, Vicuna eval, Super-Natural Instructions, Unnatural Instruction; metric ROUGE-L và GPT-4 feedback. Thêm loss language modeling trên OpenWebText (theo MiniLLM). Cặp teacher→student: GPT-2 XL (1.5B)→GPT-2 (0.1B); OPT-2.7B→OPT-1.3B; OpenLLaMA2-7B→3B (LoRA). Huấn luyện 20 epoch, LR trong {5e-4, 1e-4, 5e-5}, batch trong {8,16,32}, A100 40GB.

Bảng 1 (GPT-2 XL → GPT-2; ROUGE-L trung bình qua 5 seed, độ lệch chuẩn trong ngoặc):

Loss	Dolly	Self-Inst	Vicuna	Super-Nat	Unnatural
KLD	23.52	11.23	15.92	20.68	23.38
RKLD	23.82	10.90	16.11	22.47	23.03
Generalized JSD	24.34	12.01	15.21	25.08	27.54
SKL	24.80	12.86	16.20	26.26	28.06
SRKL	25.21	12.98	15.77	25.83	28.62

Bảng 2 (adaptive off-policy, SKL / SRKL): thuần on-policy cho Dolly 24.27 / 26.04; mixed 25.27 / 26.01; adaptive (ours) 25.90 / 26.37; thêm off-policy (ours) 25.79 / 26.11. Tức off-policy gần như không mất hiệu năng so với adaptive thuần on-policy (SKL: 25.79 so với 25.90 trên Dolly).

Tốc độ. Trên OpenLLaMA2-3B, DistiLLM chỉ tốn khoảng 1.6x thời gian train của KD thường, còn các phương pháp khác 3-7x (Fig. 5d). Tổng quát paper nêu tăng tốc **2.5-4.3x** so với MiniLLM/GKD (tóm tắt: tới 4.3x). Với S(R)KL, cách sinh adaptive + off-policy nhanh hơn on-policy hoặc mixed từ 2.2x đến 3.4x (Sec. 5.2).

Tóm tắt và dịch (Bảng 3, teacher T5-XL 3B; SAMSum ROUGE-L, IWSLT17 En-De BLEU):

Phương pháp	SAMSum Base	SAMSum Small	IWSLT Base	IWSLT Small
KD	46.23	39.52	29.36	21.15
SeqKD	46.89	40.24	29.07	21.42
ImitKD	48.57	41.44	29.87	21.52
GKD	48.49	41.92	30.24	22.04
DistiLLM	49.11	42.37	30.32	22.53

(Teacher cho dịch là mT5-XL; student mT5-Base/Small.) Biên độ cải thiện ở tác vụ đơn nhỏ hơn ở instruction-following. Paper có thêm kết quả XSum và CNN/DM ở phụ lục.

5. Phân tích / ablation

- **Skew divergence** (Bảng 1, Fig. 6): SKL, SRKL cho validation ROUGE-L cao ngay giai đoạn đầu, tức hội tụ nhanh và tổng quát tốt; thắng KLD, RKLD, JSD ở hầu hết tập.
- **Giá trị** α (Fig. 8): tối ưu tại 0.1 cho cả hai; SKL giảm nhẹ khi $\alpha > 0.1$, SRKL giảm rõ hơn.
- **Cộng hưởng với off-policy** (Bảng 4): thay on-policy bằng off-policy làm ImitKD và GKD tụt (ví dụ Dolly: ImitKD 21.63 → 20.62; GKD 23.75 → 22.89), nhưng DistiLLM gần như giữ nguyên (26.37 → 26.12). Paper giải thích do JSD/KLD hội tụ chậm hơn SRKL.
- **Scheduler thích nghi vs tune tay** (Bảng 5): xác suất cuối do scheduler chọn (0.4 cho SKL, 0.3 cho SRKL) gần mức tune tay tốt nhất (lệch không quá 0.2 với SKL và 0.1 với SRKL), và hiệu năng thường bằng hoặc tốt hơn, nên khỏi dò xác suất bằng tay.
- **Không cần SFT trước** (Bảng 6): khởi tạo từ GPT-2 chưa fine-tune, DistiLLM chỉ giảm rất nhẹ (ví dụ Dolly 25.79 → 25.75 với SKL) trong khi MiniLLM giảm 23.84 → 22.56 và GKD 23.75 → 23.15. Đây là điểm khác GKD (vốn giả định student đã SFT).

6. Giới hạn

Tác giả nêu (Phụ lục A, đọc được một phần): tập trung vào họ (R)KLD vì tính khả kế của nó (phân rã từ chuỗi xuống token), chưa xét các divergence khác như total variation; chỉ ghi nhận hai hạn chế, phần còn lại của đoạn bị vỡ trong bản trích xuất nên không nêu thêm.

Nhận xét:

- Thí nghiệm chủ yếu ở cỡ nhỏ-trung (tối đa 7B→3B, có dùng LoRA), dữ liệu dolly 14K; chưa kiểm chứng ở quy mô pretrain.
- Gradient bị chặn nhờ trộn p nhưng $\alpha = 0.1$ tối ưu theo thực nghiệm trên một số cấu hình; có thể cần dò lại cho miền khác.
- Tiêu chí scheduler (validation loss) giả định có tập validation cùng phân phối với dữ liệu train.
- Cần cùng tokenizer/vocab giữa teacher và student (do KL token-level).

7. Liên hệ với các bài khác

- **29 GKD:** DistiLLM thay JSD bằng skew KL và thay "mỗi bước lấy mẫu on-policy" bằng buffer; trong thực nghiệm GKD/ImitKD là baseline chính.
- **28 MiniLLM:** policy gradient cho reverse KL, cần sinh từ cả hai mô hình mỗi bước; DistiLLM tránh điều đó.
- **31 Rethinking KL:** phân tích rằng forward và reverse KL đều hội tụ về cùng đích ở huấn luyện lâu, và đề xuất loss ABKD/AKL; nên đọc để hiểu giới hạn của luận điểm "mode-seeking tốt hơn".
- **33 TAID:** nội suy teacher-student theo thời gian, ý tưởng liên quan đến mixture $\alpha p + (1-\alpha)q$ nhưng với α thay đổi.
- **34 DistiLLM-2:** cùng nhóm tác giả, thay vì cùng loss cho mọi mẫu thì dùng loss tương phản (teacher response vs student response).
- **30 vs 02 Hinton:** về mặt khái niệm skew KL là "temperature" theo kiểu khác: làm mềm đích bằng hỗn hợp.

8. Tóm tắt công thức / khái niệm

Khái niệm	Nội dung
SKL	$D_{KL}(p, \alpha p + (1-\alpha) q_\theta)$
SRKL	$D_{KL}(q_\theta, (1-\alpha) p + \alpha q_\theta)$
Gradient SKL	$-(1-\alpha)\, r_{p,\tilde{q}} \nabla q_\theta$, bị chặn nhờ trộn

Khái niệm	Nội dung
α tốt nhất	0.1 (thực nghiệm và L2 chuẩn hoá)
SGO	Chuỗi do student tự sinh
ϕ	Xác suất dùng SGO, tăng khi val loss tăng
$\lambda_R = \phi(1 - t/T)$	Xác suất sinh SGO mới (replay ratio giảm tuyến tính)
Replay buffer	Dung lượng 1000, thay mẫu cũ nhất
Tốc độ	2.5-4.3x nhanh hơn MiniLLM/GKD; khoảng 1.6x KD thường

9. Kết luận

DistiLLM kết hợp skew KL ($\alpha = 0.1$) để ổn định gradient với SGO, dùng scheduler theo validation loss và replay buffer để tận dụng dữ liệu student sinh. Các thí nghiệm liên hệ tốc độ hội tụ của loss với hiệu quả tái sử dụng dữ liệu off-policy.

Nguồn gốc: https://arxiv.org/abs/2402.03898

Bài 31

Rethinking Kullback-Leibler Divergence in Knowledge Distillation for Large Language Models (2024)

Tác giả: Taiqiang Wu, Chaofan Tao, Jiahao Wang, Runming Yang, Zhe Zhao, Ngai Wong (HKU; Tsinghua; Tencent AI Lab).
Venue / arXiv: arXiv:2404.02657 (v4, 08/12/2024). Bản trích xuất không nêu rõ venue.
Nguồn: https://arxiv.org/abs/2404.02657
Chủ đề: Forward KL vs reverse KL trong KD cho LLM; phản bác "FKL mean-seeking, RKL mode-seeking"; đề xuất Adaptive KL (AKL).
Vị trí trong lộ trình: Đọc trước: 02 (Hinton KD), 28 (MiniLLM, lập luận RKL tốt hơn), 29 (GKD), 30 (DistiLLM). Bài này là bài "phản biện" cho cả nhóm: nó nói lý do chọn reverse KL bằng trực giác mode-seeking là không đúng cho LLM rời rạc, và đưa ra cách trộn FKL/RKL theo dữ liệu. Đọc tiếp: 32 (Speculative KD), 33 (TAID), 34 (DistiLLM-2).

> Ghi chú học tập bằng tiếng Việt, diễn giải lại, không phải bản dịch.

1. Bối cảnh

Với KD, độ lệch giữa phân phối teacher p và student q_θ thường đo bằng KL:

- **Forward KL (FKL):** $FKL(p, q_\theta) = \sum_z p(z) \log \frac{p(z)}{q_\theta(z)}$ (Hinton).
- **Reverse KL (RKL):** $RKL(p, q_\theta) = \sum_z q_\theta(z) \log \frac{q_\theta(z)}{p(z)}$.

Niềm tin phổ biến (MiniLLM note 28, GKD note 29, DistiLLM note 30): FKL "mean-seeking" (student cố phủ mọi đỉnh của teacher, nên bị trải mỏng và sinh token đuôi vô nghĩa), còn RKL "mode-seeking" (student chọn một đỉnh) nên hợp với LLM hơn. Minh họa kinh điển: dùng một Gaussian để khớp hỗn hợp Gaussian, FKL cho đường ở giữa các đỉnh, RKL bám một đỉnh.

Paper chỉ ra niềm tin này dựa trên hai giả định: (1) student là Gaussian đơn đỉnh, (2) p và q liên tục. Trong LLM thì cả hai đều sai: q_θ là softmax trên từ vựng rời rạc V, không đơn đỉnh, và hoàn toàn có khả năng biểu diễn bất kỳ phân phối nào trên V (qua logit tự do).

2. Ý tưởng chính

Vì student là softmax trên không gian rời rạc, cả FKL lẫn RKL đều chỉ có **một điểm cân bằng**: $q_\theta = p$. Nên về lý thuyết hai loss "cùng mục tiêu"; chúng hội tụ về teacher nếu đủ nhiều epoch (paper dùng toy ví dụ với hơn 50 epoch; thực tế LLM chỉ train cỡ 10-20 epoch). Điều khác biệt là **quá trình tối ưu giữa chừng**: ở các epoch đầu, FKL ưu tiên khớp **phần đầu (head)** của p (các token xác suất lớn), còn RKL ưu tiên **phần đuôi (tail)** (các token xác suất nhỏ). Vì chỉ train ít epoch, ta nên trộn cả hai, với trọng số **thích nghi**: bên nào (head hay tail) đang lệch nhiều hơn thì cho loss tương ứng (FKL cho head, RKL cho tail) trọng số lớn hơn. Đó là **Adaptive KL (AKL)**.

3. Phương pháp chi tiết

3.1 Phân rã theo bước và gradient

KL cấp chuỗi phân rã thành tổng theo token: $J_{FKL} = \sum_{t=1}^{T} FKL\big(p(y_t|y_{<t}), q_\theta(y_t|y_{<t})\big)$, tương tự J_{RKL}. Gọi z^p, z^q là logit teacher/student trước softmax. Gradient theo logit student z^q_j:

$$\frac{\partial FKL}{\partial z_j^q} = q_\theta(Y_j|y_{<t}) - p(Y_j|y_{<t})$$

$$\frac{\partial RKL}{\partial z_j^q} = q_\theta(Y_j|y_{<t})\Big(\log \frac{q_\theta(Y_j|y_{<t})}{p(Y_j|y_{<t})} - RKL(p, q_\theta)\Big)$$

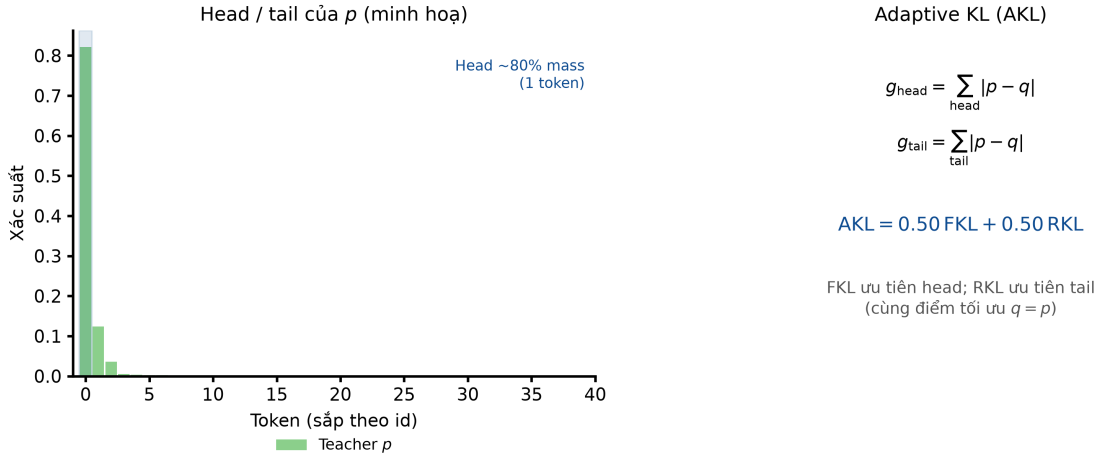
Điều kiện hội tụ cho cả hai là gradient bằng 0 với mọi j, tương đương

$$\boxed{q_\theta(Y_j|y_{<t}) = p(Y_j|y_{<t}) \quad \forall j}$$

Nghĩa là FKL và RKL **chia sẻ cùng mục tiêu tối ưu**.

3.2 Góc nhìn f-divergence

Cả hai đều là f-divergence $D_f(P\|Q) = \int f(dP/dQ)\, dQ$ với f lồi, $f(1) = 0$: FKL có $f(x) = x \log x$, RKL có $f(x) = -\log x$. Với FKL, $f(x) \to +\infty$ khi $x = p/q \to +\infty$, nên FKL tránh $q(z) \to 0$ khi $p(z)$ lớn (phạt nặng khi student bỏ sót chỗ teacher có khối lượng). Với RKL, $f \to +\infty$ khi $x \to 0^+$, nên RKL tránh $q(z)$ lớn khi $p(z) \to 0$ ("zero forcing"). Hành vi "chọn một đỉnh" chỉ xảy ra khi q là Gaussian đơn đỉnh (không đủ linh hoạt phủ nhiều đỉnh); nếu q tự do thì cả hai hội tụ về p.



AKL trộn FKL/RKL theo g_{head} và g_{tail} của $|p - q|$ trên head/tail teacher (minh hoạ, tự tính)

3.3 Khác biệt thực sự: tốc độ ở head vs tail

Trong FKL, token có $p(z)$ lớn có trọng số lớn trong tổng loss: ưu tiên khớp vùng head. Trong RKL, $\log \frac{q}{p}$ phình to khi $p(z)$ nhỏ: ưu tiên khớp vùng tail. Toy data kiểu long-tail (Fig. 3) cho thấy ở epoch 1, 5, 10 FKL khớp head tốt hơn, RKL khớp tail tốt hơn, và sau khoảng 250-300 epoch cả hai khớp teacher.

3.4 AKL

Chọn mặt nạ head $M \in \{0, 1\}^V$ nhỏ nhất bao phủ khối lượng xác suất $\geq \mu$ của teacher ($\mu = 0.5$ mặc định):

$$\min \sum_i M[i] \quad \text{s.t.} \quad \sum_j M[j]\, p(Y_j|y_{<t}) \geq \mu$$

Giải bằng cách sắp xếp p giảm dần và lấy các token đầu cho đến khi đủ μ. Khoảng cách điểm-điểm $\epsilon(p, q) = |p - q|$. Gap theo vùng:

$$g_{head} = \sum_i M[i]\,\epsilon\big(p(Y_i), q_\theta(Y_i)\big), \qquad g_{tail} = \sum_i (1 - M[i])\,\epsilon\big(p(Y_i), q_\theta(Y_i)\big)$$

$$\boxed{AKL(p, q_\theta) = \frac{g_{head}}{g_{head} + g_{tail}} FKL(p, q_\theta) + \frac{g_{tail}}{g_{head} + g_{tail}} RKL(p, q_\theta)}$$

Head lệch nhiều hơn thì FKL được trọng số lớn hơn, và ngược lại. Không thêm tham số học được (chỉ siêu tham số μ).

```
Mỗi bước token t của mỗi mẫu:
  p = softmax(teacher logits), q = softmax(student logits)
  sắp xếp p giảm dần, M[i]=1 cho tập nhỏ nhất có tổng p >= mu
  g_head = sum_i M[i]*|p_i - q_i|;  g_tail = sum_i (1-M[i])*|p_i - q_i|
  w = g_head/(g_head+g_tail)         # coi là hằng, không lấy gradient qua w (suy luận
  ↪  từ công thức)
  loss_t = w*FKL(p,q) + (1-w)*RKL(p,q)
loss = tổng/trung bình theo token
```

Ghi chú: bài báo không nói rõ có dừng gradient qua trọng số w hay không; trong pseudo-code trên đó là lựa chọn minh họa, cần kiểm tra code gốc.

Ví dụ tay: $V = 4$, $p = (0.6, 0.3, 0.08, 0.02)$, $\mu = 0.5$ thì head chỉ gồm token đầu (0.6). Giả sử $q = (0.4, 0.3, 0.2, 0.1)$: $g_{head} = 0.2$, $g_{tail} = 0 + 0.12 + 0.08 = 0.2$, nên $w = 0.5$. Nếu q gần p ở tail nhưng lệch ở head, $w \to 1$ (thiên về FKL).

4. Thí nghiệm và kết quả chính

Thiết lập. Dữ liệu instruction theo MiniLLM: databricks-dolly-15k (14K train, 500 val, 500 test). Hai cặp: GPT-2 1.5B → GPT-2 120M (batch 32, 20 epoch, LR 5e-4) và LLaMA 6.7B → TinyLLaMA 1.1B (batch 60, 10 epoch, LR 1e-5); độ dài input tối đa 512; khoảng 1 giờ trên 4 A100. Đánh giá ROUGE-L, 5 seed, trên Dolly (500), S-NI (1694), UnNI (10000). Không so với MiniLLM vì MiniLLM pretrain thêm trên OpenWebText và dữ liệu RoBERTa. SKL/SRKL dùng tỉ lệ trộn 0.1 theo DistiLLM; FKL+RKL là $0.5\,FKL + 0.5\,RKL$.

Bảng 1 (ROUGE-L, rút gọn; độ lệch chuẩn bỏ):

Phương pháp	GPT2 Dolly	S-NI	UnNI	TinyLLaMA Dolly	S-NI	UnNI
Teacher	26.98	27.25	31.61	26.73	32.75	34.61
SFT	23.01	16.48	18.43	22.05	27.79	25.96
SeqKD	23.30	16.35	18.51	22.67	26.97	27.35
FKL	23.46	16.63	19.27	22.24	28.07	26.93
RKL	22.62	17.88	19.35	23.95	28.90	27.89
SKL	23.47	16.51	18.46	23.29	29.89	29.15
SRKL	23.25	17.54	19.31	22.09	29.60	28.81
FKL+RKL	23.36	17.83	20.37	24.08	30.98	30.48
AKL	**23.88**	**19.15**	**21.97**	**24.40**	**31.37**	**31.05**

Điểm đáng chú ý theo paper: khoảng cách FKL-RKL nhỏ (ví dụ GPT-2 Dolly: FKL 23.46 cao hơn RKL 22.62); chỉ cộng FKL+RKL đã tốt hơn từng cái; AKL tốt nhất ở mọi cột, với p-value so với FKL+RKL trên UnNI là 5e-9 (GPT-2) và 9e-5 (TinyLLaMA).

Chi phí (LLaMA 6.7B→TinyLLaMA, A100 40G, Bảng 2): SFT 31.2G/60 phút; FKL và RKL 35.6G/70 phút; FKL+RKL 37.5G/76 phút; AKL 40.3G/80 phút (thêm khoảng 2.8G và 4 phút so với FKL+RKL).

Đánh giá GPT-4 (điểm /10, TinyLLaMA): AKL cao hơn các baseline cả về diversity lẫn quality (số chính xác trong hình bị vỡ nên không nêu hết). **Người đánh giá (5 chuyên gia, win/tie/loss %):** ví dụ AKL vs SFT diversity 30.6/54.2/15.2, quality 46.7/43.0/10.3; AKL vs FKL+RKL diversity 31.9/55.6/12.5, quality 27.1/57.0/15.9; AKL vs Teacher quality 15.9/35.6/48.5 (teacher vẫn thắng về chất lượng). Tác giả nói AKL thắng hoặc hòa hơn 75% trường hợp với mọi baseline. Thêm đánh giá trên Winogrande, OpenBookQA, BoolQ, ARC (chi tiết ở phụ lục, không trích số ở đây).

5. Phân tích / ablation

- **AKL-r (đảo trọng số)**: gán trọng số FKL theo g_{tail} thay vì g_{head} thì kém hơn AKL (và kém cả FKL+RKL) trên cả ba tập ở GPT-2 (Fig. 4), chứng tỏ hướng gán trọng số là đúng.
- **Theo từng subtask Dolly (TinyLLaMA):** FKL tốt ở Closed QA, Summary; RKL tốt ở Brain Storming; FKL+RKL nằm giữa (ví dụ Classification 48.00 giữa FKL 47.01 và RKL 51.39); AKL thậm chí vượt teacher ở Information Extraction và Creative Writing.
- **Ngưỡng μ:** có ablation ở Phụ lục E (không trích số).
- BERTScore cho kết luận cùng chiều (Phụ lục D).

6. Giới hạn

Tác giả nêu: chỉ thử GPT-2 và LLaMA 6.7B, chưa thử mô hình lớn hơn (như LLaMA-2 70B) do giới hạn tài nguyên.

Nhận xét:

- Cấu hình train ngắn (10-20 epoch) và chỉ một tập instruction nhỏ; lập luận "chỉ khác nhau ở giai đoạn đầu" là kết luận từ toy example và vài thí nghiệm, không phải chứng minh cho mạng sâu thật.
- Giả định khả năng biểu diễn đầy đủ của q_θ (logit tự do) chưa chắc đúng với student nhỏ bị cắt mạnh: dung lượng hạn chế vẫn có thể tạo trade-off thật giữa mean/mode (mâu thuẫn tiềm tàng với kết luận của GKD ở student nhỏ, chưa được kiểm tra).
- Là phân tích trên dữ liệu off-policy (huấn luyện trên dữ liệu cố định); tương tác với on-policy (GKD/DistiLLM) không được phân tích.
- Metric ROUGE-L khó phản ánh đúng chất lượng sinh; GPT-4/người đánh giá có số mẫu hạn chế.
- AKL cần sắp xếp p theo từ vựng ở mỗi token: chi phí đáng kể với từ vựng lớn.

7. Liên hệ với các bài khác

- **28 MiniLLM, 29 GKD:** hai bài này chọn RKL/JSD dựa trên mode-seeking; bài này phản bác trực giác đó (nhưng GKD vẫn có bằng chứng thực nghiệm riêng, ví dụ instruction tuning).
- **30 DistiLLM:** AKL dùng SKL/SRKL làm baseline (mix 0.1); hai bài cùng nhận xét FKL/RKL đơn lẻ không đủ và cần "trộn".
- **33 TAID:** bài này cho thấy "trộn" có lợi vì train ngắn; TAID giải quyết khoảng cách capacity bằng trộn theo thời gian.
- **34 DistiLLM-2:** thêm hướng tương phản; bài này vẫn nằm trong khung "một loss cho mọi mẫu".
- **02 Hinton KD:** FKL chính là loss KD gốc; phân tích gradient $q - p$ chính là gradient quen thuộc của KL với softmax.

8. Tóm tắt công thức / khái niệm

Khái niệm	Nội dung
FKL / RKL	$\sum p \log(p/q)$ / $\sum q \log(q/p)$
Gradient FKL theo logit	$q_j - p_j$
Điều kiện hội tụ	$q = p$ cho cả FKL và RKL (cùng mục tiêu)
Head / tail	Token chiếm khối lượng $\geq \mu$ của p / phần còn lại
Hành vi đầu train	FKL ưu tiên head, RKL ưu tiên tail
g_{head}, g_{tail}	Tổng $\lvert p - q \rvert$ trên head / tail
AKL	$\frac{g_{head}}{g_{head}+g_{tail}} FKL + \frac{g_{tail}}{g_{head}+g_{tail}} RKL$
Chi phí	+2.8G GPU và +4 phút so với FKL+RKL (TinyLLaMA)

9. Kết luận

Phân tích cho rằng FKL và RKL cùng hội tụ về teacher trong mô hình softmax rời rạc, nhưng khác tốc độ ở phần đầu và đuôi phân phối. AKL điều chỉnh trọng số theo sai lệch head/tail và đạt ROUGE-L tốt nhất trong các bảng GPT-2 và TinyLLaMA. So sánh này không có nghĩa một hướng KL luôn tốt nhất trên mọi tác vụ.

Nguồn gốc: https://arxiv.org/abs/2404.02657

Bài 32

Speculative Knowledge Distillation: Bridging the Teacher-Student Gap Through Interleaved Sampling (2025)

Tác giả: Wenda Xu, Rujun Han, Zifeng Wang, Long T. Le, Dhruv Madeka, Lei Li, William Yang Wang, Rishabh Agarwal, Chen-Yu Lee, Tomas Pfister (UCSB; Google Cloud AI Research; Google DeepMind; CMU).
Venue / arXiv: arXiv:2410.11325 (v3, 27/04/2025); ICLR 2025.
Nguồn: https://arxiv.org/abs/2410.11325
Chủ đề: On-policy distillation cho LLM với lấy mẫu xen kẽ (interleaved) giữa student và teacher, lấy cảm hứng từ speculative decoding.
Vị trí trong lộ trình: Đọc trước: 29 (GKD, on-policy KD), 30 (DistiLLM, cùng bàn về nhiễu của dữ liệu student). Bài này giải quyết đúng điểm yếu của on-policy KD: khi student còn kém, chuỗi nó sinh ra là OOD đối với teacher nên nhãn của teacher kém tin cậy. Cách sửa: để teacher thay các token tệ ngay trong lúc sinh. Đọc tiếp: 33 (TAID, giải quyết khoảng cách teacher-student theo cách khác), 34 (DistiLLM-2), 47 (ShortOPD).

> Ghi chú học tập bằng tiếng Việt, diễn giải lại, không phải bản dịch. Một số hình trong bản trích xuất bị vỡ nên chỉ số trong bảng đọc rõ mới được trích.

1. Bối cảnh

Hai họ phương pháp soft-label KD cho LLM:

- **Supervised KD:** huấn luyện trên dữ liệu cố định (nhãn gốc hoặc chuỗi teacher sinh). Lệch phân phối với chuỗi student tự sinh lúc suy luận (exposure bias).
- **On-policy KD** (GKD, note 29): student tự sinh, teacher chấm logit từng token. Giải quyết mismatch nhưng có điểm yếu: nếu student ban đầu yếu, chuỗi nó sinh có thể rất tệ (lặp, vô nghĩa), tức là **OOD đối với teacher**, nên logit của teacher trên những tiền tố này không đáng tin. Bài báo cho thấy on-policy KD rất nhạy với khởi tạo student: ví dụ Qwen2-0.5B-IT dịch Assamese-English, chuỗi on-policy bị lặp "in the past, in the past, ..." (COMET 36), trong khi SKD cho chuỗi hợp lý (COMET 72).

Gloss: **speculative decoding** là kỹ thuật tăng tốc suy luận: model nhỏ (draft) đề xuất vài token, model lớn kiểm tra song song và chấp nhận/loại bỏ. **Top-k sampling**: chỉ lấy mẫu trong k token xác suất cao nhất.

2. Ý tưởng chính

SKD (Speculative KD) kết hợp hai nguồn mẫu *ở mức token*: student đề xuất một đoạn token (on-policy), teacher kiểm tra song song và chấp nhận token nếu nó nằm trong **top-K** của teacher; token đầu tiên bị loại thì teacher lấy mẫu thay, các token sau bị bỏ và student tiếp tục sinh từ đó. Kết quả: chuỗi huấn luyện vừa gần phân phối suy luận của student (phần lớn token là của student), vừa không rơi vào vùng teacher lạ. Đầu train, student yếu nên bị bác nhiều, SKD giống supervised KD; về sau student tốt hơn, tỉ lệ chấp nhận cao, SKD gần như on-policy KD. Chuyển pha này tự động, không cần lịch cứng.

3. Phương pháp chi tiết

3.1 Mục tiêu huấn luyện

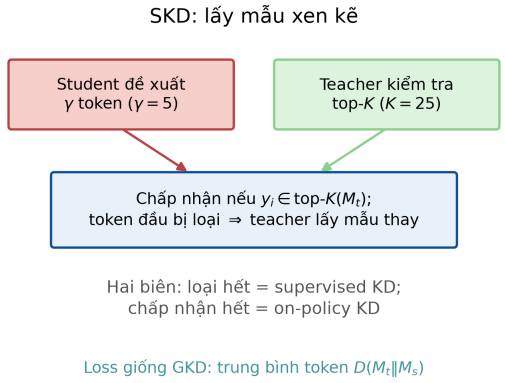
Dùng cùng dạng loss như GKD, trung bình theo token của divergence giữa phân phối teacher và student:

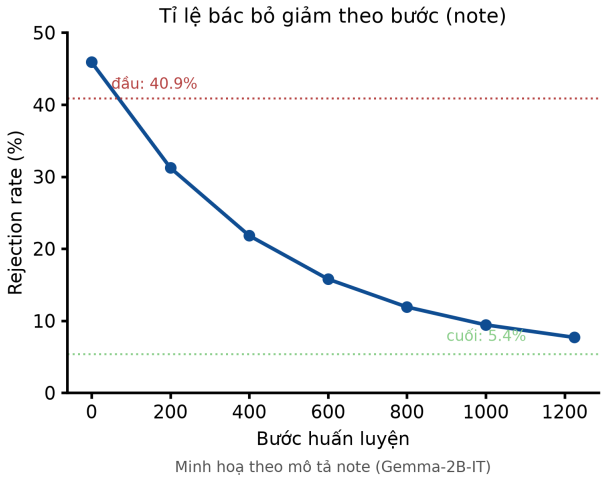
$$D(M_t \| M_s)(y|x) = \frac{1}{L_y} \sum_{i=1}^{L_y} D(M_t(\cdot|y_{<i}, x) \,\|\, M_s(\cdot|y_{<i}, x))$$

Điểm mới nằm ở cách tạo chuỗi y, không phải ở loss. (Paper dùng divergence D như một siêu tham số; phụ lục A bàn các metric.)

3.2 Lấy mẫu xen kẽ và tiêu chí chấp nhận

Cho prompt x, student sinh γ token $(y_i, \ldots, y_{i+\gamma})$ theo M_s. Teacher đánh giá song song cả đoạn. Token y_i được **chấp nhận** nếu $y_i \in \text{top-}K(M_t(\cdot|y_{<i}, x))$. Nếu bị loại: các token sau bị bỏ, y_i được lấy mẫu lại từ $M_t(\cdot|y_{<i}, x)$, rồi student tiếp tục sinh on-policy.





SKD: student đề xuất γ token, teacher chấp nhận top-K; rejection rate giảm 40.9%→5.4% (note) (minh hoạ, tự tính)

Chú ý khác speculative decoding thật: tiêu chí chấp nhận gốc của speculative decoding đảm bảo mẫu cuối đúng phân phối teacher (thoái hóa thành supervised KD); SKD cố tình dùng tiêu chí top-K để giữ mẫu *student* nhưng *có khả năng cao dưới teacher*.

$$\boxed{y_i \text{ accept} \iff y_i \in \text{top-}K(M_t(\cdot \mid y_{<i}, x))}$$

Mặc định $K = 25$ (không tune; tương tự giá trị thường dùng trong top-k sampling), $\gamma = 5$ trong cài đặt thực.

3.3 Pseudo-code

```
Cho: student M_s, teacher M_t, prompts {x}, độ dài sinh, K, divergence D
for mỗi prompt x (hoặc mini-batch):
    y = []
    while chưa đủ độ dài / chưa EOS:
        đề xuất: student sinh gamma token (y_i..y_{i+gamma}) theo M_s
        teacher chấm song song gamma+1 vị trí (một forward)
        tìm vị trí j đầu tiên có y_j không thuộc top-K của M_t
        giữ y_i..y_{j-1}; thay y_j bằng mẫu từ M_t(.|y_<j, x); bỏ phần sau
        nối vào y; tiếp tục
    loss = mean_i D( M_t(.|y_<i,x) || M_s(.|y_<i,x) )   # trên chuỗi y vừa tạo
    cập nhật student
```

Hai trường hợp suy biến: nếu mọi token đều bị loại, SKD = supervised KD (toàn token teacher); nếu mọi

token đều được chấp nhận, SKD = on-policy KD. Bản lõi của đề xuất dựa trên lý thuyết imitation learning (Ross & Bagnell; Ross et al.): chuyển dần nguồn mẫu từ expert sang learner là hiệu quả.

4. Thí nghiệm và kết quả chính

Mô hình. Teacher: Gemma-7B-IT và Qwen2-7B-IT, đã SFT theo từng tác vụ. Student: Gemma-2B-IT, Qwen2-0.5B-IT (instruction-tuned, chưa SFT tác vụ) hoặc bản SFT thêm. LR 1e-5, warmup 0.1, dropout 0.1; SFT 3 epoch chọn checkpoint val loss thấp nhất; $K = 25$.

Tác vụ và dữ liệu (dữ liệu student khoảng 1K mẫu, kịch bản data ít):

- Dịch Assamese→English: Flores-200 (997 mẫu train; test 512), metric COMET.
- Tóm tắt hội thoại: DialogSum (1K train; test 1500), ROUGE-L.
- Suy luận số học: GSM8K (1K train; test 1319), accuracy, CoT.
- Math instruction following (task-agnostic): UltraInteract (10K train / 1K dev), đánh giá trên held-out Math, GSMplus, ASDiv, SVAMP; thử 1K và 10K prompt.

Lưu ý: baseline supervised KD, SFT, ImitKD thấy (x, y) gốc; on-policy KD và SKD chỉ thấy prompt x.

Bảng 1 (student là bản instruction-tuned, chưa SFT):

Baseline	Gemma 2B: Dịch COMET	Tóm tắt ROUGE-L	GSM8K Acc	Qwen 0.5B: Dịch COMET	Tóm tắt	GSM8K
SFT	72.5	31.7	18.7	57.6	29.2	31.8
Supervised KD	73.3	34.1	22.5	57.5	31.3	35.0
On-policy KD	36.1	34.1	25.3	53.0	31.2	33.9
ImitKD	74.8	34.9	26.2	55.9	30.9	33.2
SKD	**75.3**	**35.0**	**29.1**	57.1	**31.7**	**36.6**

On-policy KD gần như sụp trong dịch (COMET 36.1), đúng hiện tượng student yếu sinh mẫu OOD. SKD thắng 5 trong 6 cột (riêng dịch với Qwen, supervised KD/SFT ngang SKD, không khác biệt có ý nghĩa thống kê).

Bảng 2 (student đã SFT):

Baseline	Gemma SFT: Dịch	Tóm tắt	GSM8K	Qwen SFT: Dịch	Tóm tắt	GSM8K
SFT	72.5	31.7	18.7	57.6	29.2	31.8
Supervised KD	73.5	34.4	22.0	59.3	32.3	31.9
On-policy KD	74.6	34.8	28.8	58.3	32.1	32.2
ImitKD	74.6	34.4	26.2	59.9	32.2	31.8
SKD	**75.0**	**35.4**	**29.2**	**60.6**	**33.1**	**33.4**

(Bản thân SFT hàng đầu của bảng 2 giống bảng 1 vì chính là checkpoint SFT dùng làm baseline.)

Tăng so với SFT (Gemma-7B→2B, Hình 1/giới thiệu): dịch low-resource +41.8%, tóm tắt +230%, số học +160% (theo cách tính tương đối của bài báo, nên không hiểu là điểm tuyệt đối). Trên held-out sau khi huấn luyện với dữ liệu instruction toán: Math +198%, GSMplus +360%.

Data cực ít (100 mẫu, Gemma-2B, Bảng 3): student IT: SFT 59.8/23.0/8.11; Supervised KD 65.8/31.7/17.0; On-policy KD 33.7/32.2/12.1; SKD 67.0/32.3/18.8 (dịch/tóm tắt/GSM8K). Student SFT: Supervised KD 60.9/32.0/14.4; On-policy 63.4/32.4/16.1; SKD 63.8/32.6/16.7. Nhận xét của tác giả: SFT trên 100 mẫu gây overfit, nên SKD chạy end-to-end từ model IT (bỏ bước SFT) tốt hơn quy trình hai bước SFT+on-policy KD.

Tăng tốc speculative decoding (Phụ lục K): draft model huấn luyện bằng SKD có tỉ lệ chấp nhận token cao hơn khoảng 71% đến 85% so với model IT gốc, tăng tốc 1.21x (dịch) và 1.17x (tóm tắt) (Gemma-2B-IT).

5. Phân tích / ablation

- **Khởi tạo student:** on-policy KD có thể làm giảm hiệu năng theo thời gian nếu khởi tạo yếu (Fig. 4); SKD tốt cả với IT lẫn SFT. ImitKD (trộn ở mức chuỗi) không thấy lợi rõ khi student tốt hơn, nên trộn ở mức token của SKD tốt hơn trộn ở mức chuỗi.
- **Huấn luyện hai giai đoạn thủ công** (supervised KD nửa đầu, on-policy nửa sau, Fig. 5): thường bị vượt bởi chỉ supervised hoặc chỉ on-policy, và bị SKD vượt rõ. Tác giả lý giải: trộn thô hai pha không có thước đo tin cậy từng token; cơ chế chấp nhận của SKD đóng vai trò đó.
- **Tỉ lệ bị loại (rejection rate, Phụ lục L):** với Gemma-2B-IT, khởi đầu 40.9% token phải teacher lấy lại, giảm xuống 9.7% sau 150 bước đầu và 5.4% sau 1,225 bước; với Gemma-2B-SFT bắt đầu 4.7% và còn 0.1% cuối. Xác nhận quá trình chuyển từ supervised-like sang on-policy-like.
- **Chi phí sinh mẫu (Phụ lục M, ước lượng lý thuyết):** với giả định tỉ lệ loại 10%, $\gamma = 5$, student 2B, teacher 7B, $D = 256$, $L = 200$: nếu phải sinh từ teacher thì supervised KD đắt khoảng 3.5x on-policy KD; SKD khoảng 1.72x on-policy KD, tức tiết kiệm khoảng 50% so với sinh trực tiếp từ teacher. Đây là ước lượng FLOP, không phải thời gian đo được.
- **Giá trị K (Phụ lục B):** nhiều giá trị K đều vượt supervised và on-policy KD; K tối ưu tuỳ tác vụ. Phụ lục C có thử nghiệm sơ bộ K thích nghi.
- Hội tụ validation: SKD có validation loss thấp nhất (Phụ lục N).

6. Giới hạn

Tác giả nêu/ngụ ý: K được cố định 25 không tune, K tối ưu phụ thuộc tác vụ và K thích nghi mới ở mức sơ bộ. Chi phí tính bằng FLOP lý thuyết. Trong bản trích xuất không thấy mục Limitations riêng.

Nhận xét:

- Cỡ nhỏ: dữ liệu chủ yếu 1K mẫu; teacher tối đa 7B; chỉ hai họ Gemma/Qwen.
- Cần teacher chấm song song (white-box) và cùng tokenizer; mỗi bước cần forward teacher trên chuỗi dài, nên bộ nhớ train cao.
- Tiêu chí top-K dựa trên *hạng* chứ không dựa trên xác suất tuyệt đối. Khi teacher rất tự tin (phân phối nhọn), K=25 gần như chấp nhận mọi thứ trong top-25 dù xác suất cực thấp, nên "lọc" yếu. Khi teacher kém tự tin thì ngược lại. Paper không xét điều này (đây là nhận xét bổ sung).
- Chưa so với DistiLLM/MiniLLM, vì đối sánh chính là supervised, on-policy, ImitKD.
- Chưa phân tích khi student bị cắt cấu trúc (pruned), nơi chất lượng khởi đầu thấp hơn nhiều.

7. Liên hệ với các bài khác

- **29 GKD:** SKD là biến thể của on-policy KD chữa điểm yếu nhiễu nhãn; hai trường hợp biên của SKD chính là GKD với $\lambda = 0$ và $\lambda = 1$.
- **30 DistiLLM:** cùng nhận ra phản hồi nhiễu khi teacher gặp SGO lạ (Fig. 1 của DistiLLM); DistiLLM xử lý ở mức lịch dữ liệu (ít SGO lúc đầu), SKD xử lý ở mức token (teacher sửa).
- **15 Sequence-level KD:** SKD là "SeqKD từng phần": teacher sinh một số token thay vì cả chuỗi.

- **33 TAID:** cùng nhắm đến khoảng cách teacher-student, TAID bằng nội suy phân phối theo thời gian thay vì trộn token.
- **34 DistiLLM-2:** dùng cặp chuỗi teacher/student theo kiểu tương phản; có thể dùng dữ liệu kiểu SKD.
- **47 ShortOPD:** hướng on-policy cho mô hình đã cắt; SKD cho một cách ổn định giai đoạn đầu khi student yếu.

8. Tóm tắt công thức / khái niệm

Khái niệm	Nội dung
Vấn đề on-policy KD	Mẫu student yếu là OOD với teacher, nhãn kém tin cậy
Interleaved sampling	Student đề xuất, teacher kiểm tra và thay token bị loại
Tiêu chí chấp nhận	$y_i \in$ top-$K(M_t)$, $K = 25$ mặc định
γ	Số token student đề xuất mỗi lượt (5 trong thực nghiệm)
Hai biên	Loại hết = supervised KD; chấp nhận hết = on-policy KD
Rejection rate	40.9% → 5.4% (Gemma-2B-IT, 1,225 bước)
Ước lượng chi phí	SKD khoảng 1.72x on-policy; supervised (sinh bằng teacher) khoảng 3.5x
Loss	Giống GKD: trung bình theo token của $D(M_t \| M_s)$

9. Kết luận

Speculative KD cho teacher thay các token student sinh lệch trong quá trình tạo chuỗi. Tỷ lệ token được thay đổi theo chất lượng student, tạo cầu nối giữa dữ liệu supervised và on-policy. Kết quả được báo cáo mạnh nhất khi student yếu hoặc dữ liệu ít.

Nguồn gốc: https://arxiv.org/abs/2410.11325

Bài 33

TAID: Temporally Adaptive Interpolated Distillation for Efficient Knowledge Transfer in Language Models (2025)

Tác giả: Makoto Shing, Kou Misaki, Han Bao, Sho Yokoi, Takuya Akiba (Sakana AI; Kyoto University; NINJAL; Tohoku University; RIKEN).
Venue / arXiv: arXiv:2501.16937 (v4, 27/02/2025); ICLR 2025.
Nguồn: https://arxiv.org/abs/2501.16937
Chủ đề: Distillation cho LM có khoảng cách capacity lớn: teacher trung gian nội suy theo thời gian giữa student và teacher; không dùng dữ liệu student-generated.
Vị trí trong lộ trình: Đọc trước: 08 (Teacher assistant, cùng vấn đề capacity gap ở vision), 09 (Efficacy of KD), 29-31 (GKD, DistiLLM, Rethinking KL). Bài này nhìn capacity gap như vấn đề "đích quá xa" và giải bằng curriculum liên tục, không bằng chọn divergence hay dữ liệu on-policy. Đọc tiếp: 34 (DistiLLM-2), 41 (Capacity gap law), 42 (Distillation scaling laws).

> Ghi chú học tập bằng tiếng Việt, diễn giải lại, không phải bản dịch. Phần lý thuyết (Theorem 4.1) chỉ trình bày ở mức ý tưởng vì paper chỉ nêu dạng không chính thức trong thân bài.

1. Bối cảnh

Hai khó khăn xuất phát từ sự khác biệt giữa teacher và student:

1. **Capacity gap (nguyền rủa khoảng cách dung lượng):** teacher quá lớn có thể làm student kém đi, vì student không đủ sức bắt chước phân phối quá phức tạp (Mirzadeh 2020 - note 08, và các tác giả khác). Càng nén mạnh thì vấn đề càng nặng.
2. **Mode averaging / mode collapse:** KL thuận khiến student cố phủ mọi mode nên phân phối bị làm mịn quá (averaging); KL ngược/JSD khiến student bám vài mode chính, mất đuôi (collapse).

Các giải pháp trước đó chủ yếu đổi loss (reverse KL, TVD, JSD, skew KL, adaptive KL) hoặc dùng dữ liệu student-generated (GKD, DistiLLM), nhưng **đích** (phân phối teacher) vẫn cố định suốt quá trình huấn luyện, và dữ liệu SGO tốn chi phí cho mô hình lớn hoặc pre-training (không có prompt để student sinh). Skew KL cũng dùng phân phối trung gian, nhưng cố định và chuyển tri thức của teacher vào đó, khác với TAID (xem Phụ lục C của bài báo).

2. Ý tưởng chính

Thay vì bắt student khớp teacher ngay, TAID cho student khớp một **teacher trung gian** p_t là nội suy (ở mức logit) giữa chính student (đã detach, coi như hằng) và teacher thật, với hệ số t tăng dần từ giá trị nhỏ lên 1. Lúc đầu t nhỏ, đích gần như là chính student (giống self-distillation, nhẹ nhàng), rồi dịch dần sang teacher. Tốc độ tăng t thích nghi: nếu loss giảm nhanh (student theo kịp) thì tăng t mạnh, nếu không thì chậm lại. Như vậy độ khó của bài toán luôn "vừa sức" student ở mọi thời điểm. Kết hợp với phân tích lý thuyết cho thấy khác với self-distillation thuần túy, cách này không dẫn đến collapse.

3. Phương pháp chi tiết

3.1 Phân phối nội suy và mục tiêu

Với tiền tố $y_{<s}$, token y_s, logit student logit_{q_θ} (detach thành $q_{\theta'}$) và logit teacher logit_p:

$$\boxed{p_t(y_s|y_{<s}) = \text{softmax}\Big((1-t)\,\text{logit}_{q_{\theta'}}(y_s|y_{<s}) + t\,\text{logit}_p(y_s|y_{<s})\Big)}$$

Nội suy ở mức **logit** (không phải mức xác suất) để giữ độ tin cậy tương đối giữa các token. Mục tiêu tại thời điểm t là KL từ p_t tới student:

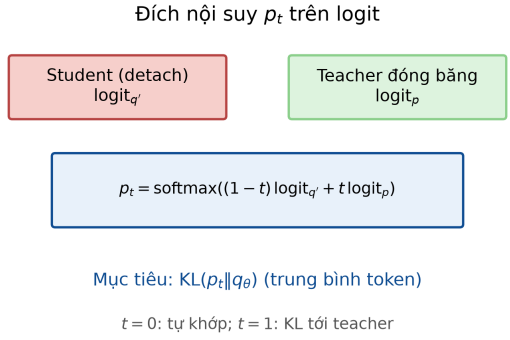
$$J^{(t)}_{TAID}(p, q_\theta) = J_{KL}(p_t, q_\theta) = \frac{1}{S}\sum_{s=1}^{S}\sum_{y_s\in\mathcal{Y}} p_t(y_s|y_{<s}) \log \frac{p_t(y_s|y_{<s})}{q_\theta(y_s|y_{<s})}$$

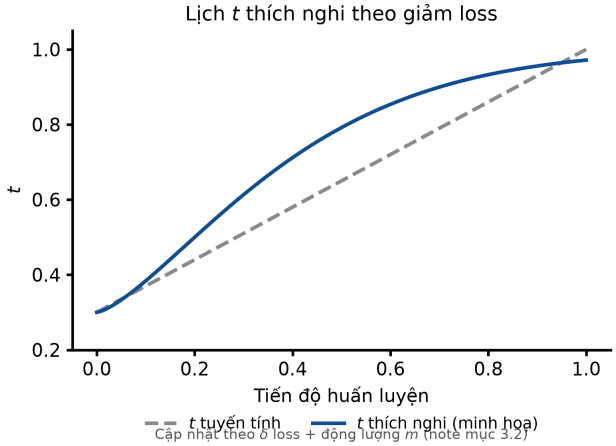
Vì student trong p_t bị detach, chỉ có q_θ ở mẫu số nhận gradient: p_t chỉ đóng vai trò đích. Dạng khác: có thể xem p_t nằm "giữa" student và teacher, nên KL tới nó luôn nhỏ hơn KL tới teacher khi $t < 1$ (ý này là suy luận bổ sung để hiểu độ khó).

Hai cực: $t = 0$ cho đích $p_0 = q_{\theta'}$ (student tự khớp chính mình, gradient bằng 0 về mặt giá trị tại điểm đó); $t = 1$ cho $p_1 = p$ (KL chuẩn tới teacher).

3.2 Cập nhật t thích nghi

Thay đổi tương đối của loss: $\delta_n = \dfrac{J^{(t_{n-1})}_{TAID} - J^{(t_n)}_{TAID}}{J^{(t_{n-1})}_{TAID} + \epsilon}$. Động lượng: $m_n = \beta m_{n-1} + (1-\beta)\delta_n$. Cập nhật:





TAID nội suy logit $(1-t)q' + tp$ làm đích KL; lịch t thích nghi so với tuyến tính (minh hoạ, tự tính)

$$t_{n+1} \leftarrow \min\Big(t_{end},\ \max\big(t_{linear},\ t_n + \alpha \cdot \text{sigmoid}(m_n)\cdot(1-t_n)\big)\Big)$$

với $t_{linear} = t_{start} + (t_{end} - t_{start})\cdot n/N$ là sàn tuyến tính (đảm bảo ít nhất tiến tuyến tính). Siêu tham số mặc định: $\beta = 0.99$, $\alpha = 5\times10^{-4}$, $t_{start} \in [0.2, 0.4]$ tùy độ giống nhau ban đầu giữa student và teacher, $t_{end} = 1.0$.

Trực giác: δ_n lớn (loss đang giảm nhanh) thì sigmoid lớn, t tăng nhanh; student đã "học xong" ở mức khó hiện tại thì $\delta_n \approx 0$, t chỉ tăng theo sàn tuyến tính hoặc chậm.

3.3 Pseudo-code (Algorithm 1)

```
Cho: eta (lr student), alpha (lr của t), beta, N, t_start, t_end
t = t_start; m = 0; J_prev = inf
for n in 1..N:
    t_lin = t_start + (t_end - t_start) * n / N
```

```
batch ~ dataset                                  # dữ liệu cố định, không cần SGO
logits_s = student(batch); logits_t = teacher(batch)
p_t = softmax((1-t)*detach(logits_s) + t*logits_t)
J = KL(p_t || q_theta)  (trung bình theo token)
cập nhật theta bằng gradient của J
delta = (J_prev - J) / (J_prev + eps)
m = beta*m + (1-beta)*delta
t = min(t_end, max(t_lin, t + alpha*sigmoid(m)*(1 - t)))
J_prev = J
```

Ví dụ tay: $t = 0.4$, logit student cho token A là 2.0 và teacher 4.0; logit nội suy $= 0.6 \cdot 2 + 0.4 \cdot 4 = 2.8$ (so với đích teacher hoàn toàn là 4.0): đích "dịch" 40% quãng đường. Khi t lên 0.8: $0.2 \cdot 2 + 0.8 \cdot 4 = 3.6$.

3.4 Phân tích lý thuyết (Theorem 4.1, không chính thức)

Mượn khung Mobahi et al. (2020) cho bài toán hồi quy bình phương tối thiểu như proxy của language modeling: mỗi bước student khớp nhãn nội suy $(1-t)y_{student} + t\,y_{teacher}$. Self-distillation lặp lại thuần túy (Mobahi) chắc chắn collapse sau đủ nhiều bước. Định lý: nếu tín hiệu teacher đủ lớn (nhãn lớn hơn $\Omega(\sqrt{T}\epsilon)$) thì student không collapse tại mọi t. Phát biểu chính thức nằm ở Phụ lục B; đây là kết quả trên mô hình hồi quy, không phải trên transformer thật.

4. Thí nghiệm và kết quả chính

Instruction tuning. Dữ liệu UltraChat 200k, đánh giá MT-Bench (GPT-4 chấm). Chỉ distillation trên dữ liệu instruction (không SFT trước, không loss CE thêm trên corpus pretrain, khác các paper trước). Ba cặp teacher → student: Phi-3-mini-4k-instruct (3.8B) → TinyLlama (1.1B); Llama-2-7b-chat → TinyLlama; StableLM Zephyr 3B → Pythia-410M. Điểm MT-Bench (Bảng 1):

Phương pháp	Phi-3-mini → TinyLlama	Llama-2 → TinyLlama	StableLM Zephyr → Pythia-0.4B
SFT	2.00	3.94	2.57
KL	2.71	3.99	2.74
RKL	3.48	3.92	2.53
TVD	3.27	3.64	2.57
Adaptive KL	3.27	3.77	2.64
GKD	2.24	3.82	2.59
DistiLLM	3.23	3.97	2.97
CTKD	1.78	2.84	1.39
DKD	2.70	4.14	2.90
TAID không adaptive	3.44	4.18	2.88
TAID	**4.05**	**4.27**	**3.05**

TAID không dùng SGO nên huấn luyện nhanh khoảng 2 lần so với DistiLLM và 10 lần so với GKD (theo paper). Cơ chế adaptive cải thiện 2.2% đến 17.7% so với phiên bản chỉ tăng t tuyến tính.

Continued pre-training. Student TinyLlama, teacher Phi-3-medium-4k-instruct, 10% đầu của SmolLM-Corpus (khoảng 20 tỉ token); các baseline chỉ dùng hàm mục tiêu (không SGO vì pre-train không có prompt). Đánh giá Open LLM Leaderboard (Bảng 2, trích một phần):

Phương pháp	ARC	HellaSwag	MMLU	TruthfulQA	Winogrande	GSM8K	Trung bình
SFT	41.38	63.66	25.89	35.64	61.25	1.21	38.17
KL	44.97	65.43	25.11	37.95	63.22	2.80	39.91
Skew KL	44.62	65.25	25.79	37.45	62.51	3.41	39.84
TAID	45.48	65.43	25.43	37.92	63.38	2.96	**40.10**

(Bản gốc có thêm TVD, Adaptive KL, GJS, Skew RKL; trung bình lần lượt 39.41, 39.11, 39.81, 39.60.) Lưu ý chênh lệch trung bình giữa TAID (40.10) và KL (39.91) rất nhỏ.

Mô hình ứng dụng. TAID-LLM-1.5B đạt LightEval 52.27 (so với SmolLM-1.7B 51.31, StableLM-2-1.6B 51.24, Phi-1.5B 50.39, Qwen2-1.5B 46.19), được tác giả nói là tốt nhất dưới 2B. TAID-VLM-2B đạt Open-VLM-LB 56.43 (InternVL2-2B 53.96, Phi-3-Vision 53.60, MiniCPM-V-2 47.93, PaliGemma 46.56), vượt cả Phi-3-Vision 4.2B.

5. Phân tích / ablation

- **Hành vi của t và độ ổn định (Fig. 2):** α lớn làm t tăng nhanh ở đầu, hợp khi gap nhỏ. Giá trị mục tiêu của TAID gần như hằng với phương sai thấp, trong khi KL chuẩn cao và dao động: độ khó học được giữ ổn định.
- **Capacity gap (6.3.2):** student cố định Pythia-70M, teacher Pythia 410M, 1B, 2.8B, 6.9B, huấn luyện 1 epoch trên 1B token SmolLM-Corpus, đánh giá LAMBADA. TAID tăng đều khi teacher lớn lên, còn KL và RKL biến động không đơn điệu (đúng "curse of capacity gap"). TAID cũng thắng Skew KL ở các cỡ teacher (Bảng 6, không trích số).
- **Cân bằng mode (Bảng 3, cặp Phi-3-mini → TinyLlama, UltraChat):** khối lượng xác suất ở head (top-10 token): KL 0.216, RKL 0.227, TAID 0.218; ở tail (phân vị 80-100): KL 40.2×10^{-7}, RKL 8.1×10^{-7}, TAID 39.0×10^{-7}. TAID giữ head tốt hơn KL và giữ tail xấp xỉ KL, trong khi RKL mất đuôi: được diễn giải là cân bằng giữa averaging và collapse.
- **Phương pháp từ phân loại ảnh (CTKD, DKD):** kém trên LM. Giả thuyết của tác giả: LM có entropy cao và xác suất lớp đúng thấp (GPT-2 so với ResNet-56, Fig. 3), nên dễ collapse và khó với lớp tần suất cực thấp (Zipf). Kiểm chứng ngược: ở ImageNet (entropy 6.67, xác suất lớp đúng 0.00130), TAID thắng CTKD và DKD, còn trên CIFAR-100 (entropy 0.485, xác suất 0.613) mức tăng khiêm tốn (Phụ lục D.3).
- **Siêu tham số:** $\beta \approx 0.99$ hoạt động ổn; t_{start} phụ thuộc độ giống nhau giữa student và teacher ban đầu; $\alpha = 5 \times 10^{-4}$ (instruction tuning), 5×10^{-5} cho TAID-LLM/VLM (các cấu hình chi tiết ở phụ lục).

6. Giới hạn

Tác giả nêu: không dùng SGO nên chưa khai thác on-policy; "tương lai có thể kết hợp TAID với on-policy"; chưa thử nội suy phi tuyến, multi-teacher, các metric khác ngoài KL. Chứng minh lý thuyết chỉ trên proxy hồi quy.

Nhận xét:

- Hiệu quả trong pre-training rất nhỏ (40.10 so với 39.91 của KL; không có độ lệch chuẩn/nhiều seed nên khó biết có ý nghĩa thống kê không).
- Kết quả instruction tuning dựa trên MT-Bench với GPT-4 chấm và student rất yếu (điểm 1.4 đến 4.3), nơi nhiễu đánh giá lớn.
- Thiết lập "chỉ distillation, không SFT" có thể làm yếu baseline on-policy (GKD 2.24 thấp hơn cả KL ở cặp Phi-3, trái với kết quả gốc của GKD), nên đối sánh với GKD/DistiLLM trong bài báo này nên đọc thận trọng.

- Thêm 3-4 siêu tham số ($\alpha, \beta, t_{start}, t_{end}$) và student phải sinh logit ở mỗi bước (không tốn thêm so với KL, nhưng cần detach).
- Phạm vi teacher chỉ đến khoảng 7B, tỉ lệ nén đến khoảng 100x chỉ ở thử nghiệm Pythia (6.9B → 70M).

7. Liên hệ với các bài khác

- **08 Teacher assistant, 09 Efficacy of KD:** cùng phát hiện teacher quá lớn có thể gây hại; TAID thay "teacher trung gian là một model" bằng "teacher trung gian là một phân phối nội suy liên tục".
- **11 Patient and consistent teacher:** cùng triết lý curriculum/quá trình dần dần, nhưng ở đó là function matching trên ảnh.
- **30 DistiLLM:** skew KL là nội suy cố định với $\alpha = 0.1$ (đích nằm giữa p và q); TAID nội suy thay đổi theo thời gian.
- **29 GKD, 31 Rethinking KL:** TAID là một lựa chọn thay thế cho nhóm "đổi loss", và Bảng 1 đưa Adaptive KL và GKD vào so sánh.
- **34 DistiLLM-2:** cùng năm; xử lý khoảng cách theo hướng loss tương phản.
- **41 Capacity gap law, 42 Distillation scaling laws:** định lượng lại hiện tượng capacity gap mà TAID giải bằng thuật toán.

8. Tóm tắt công thức / khái niệm

Khái niệm	Nội dung
Teacher trung gian	$p_t = \text{softmax}((1-t)\,\text{logit}_{q'} + t\,\text{logit}_p)$, student detach
Mục tiêu	$KL(p_t \,\|\, q_\theta)$ trung bình theo token
Lịch t	tăng từ t_{start} đến $t_{end} = 1$, thích nghi theo tốc độ giảm loss
Cập nhật	$t \leftarrow \min(t_{end}, \max(t_{lin}, t + \alpha\,\sigma(m)(1-t)))$
Mặc định	$\beta = 0.99$, $\alpha = 5 \times 10^{-4}$, $t_{start} \in [0.2, 0.4]$
Capacity gap	Teacher lớn hơn không chắc tốt hơn; TAID cải thiện đơn điệu theo cỡ teacher (Pythia-70M)
Kết quả MT-Bench	4.05 / 4.27 / 3.05 (ba cặp)
Tốc độ	Khoảng 2x nhanh hơn DistiLLM, 10x nhanh hơn GKD (không SGO)

9. Kết luận

TAID tạo phân phối mục tiêu nội suy giữa student và teacher, với hệ số thay đổi thích nghi theo tiến độ học. Bằng chứng rõ nhất nằm ở instruction tuning và các thí nghiệm thay đổi cỡ teacher; chênh lệch ở pre-training nhỏ hơn. Phương pháp không yêu cầu chuỗi do student sinh.

Nguồn gốc: https://arxiv.org/abs/2501.16937

Bài 34

DistiLLM-2: A Contrastive Approach Boosts the Distillation of LLMs (2025)

Tác giả: Jongwoo Ko, Tianyi Chen, Sungnyun Kim, Tianyu Ding, Luming Liang, Ilya Zharkov, Se-Young Yun (KAIST AI; Microsoft).
Venue / arXiv: arXiv:2503.07067 (v2, 30/05/2025); ICML 2025 (PMLR 267).
Nguồn: https://arxiv.org/abs/2503.07067
Chủ đề: Distillation cho LLM bằng loss tương phản (contrastive): dùng loss khác nhau cho phản hồi do teacher sinh (SKL) và do student sinh (SRKL); curriculum cho hệ số skew; ứng dụng DPO, VLM, lượng tử hóa, speculative decoding.
Vị trí trong lộ trình: Đọc trước: 29 (GKD), 30 (DistiLLM, nền tảng trực tiếp), 31 (Rethinking KL), 32 (Speculative KD, là baseline). Bài này là bước tiếp theo của DistiLLM: chỉ ra rằng loss và loại dữ liệu cần được "ghép đôi" thay vì dùng cùng một loss cho mọi mẫu. Đọc tiếp: 41 (Capacity gap law), 42 (Distillation scaling laws), 47 (ShortOPD).

> Ghi chú học tập bằng tiếng Việt, diễn giải lại, không phải bản dịch. Công thức cập nhật α trong Algorithm 1 bị vỡ trong bản trích xuất PDF nên chỉ mô tả bằng lời.

1. Bối cảnh

Phần lớn nghiên cứu KD cho LLM làm một trong hai việc: (a) thiết kế **loss** mới (reverse KL, JSD, skew KL), hoặc (b) chọn **dữ liệu** huấn luyện (chuỗi do teacher sinh, tức TGO; chuỗi do student sinh, tức SGO). Nhưng hầu hết đều áp **cùng một loss cho mọi loại dữ liệu**. Bài này hỏi: có nên dùng loss khác nhau cho TGO và SGO không?

Gợi ý đến từ **preference alignment**: DPO tăng likelihood của phản hồi tốt y_w và giảm likelihood của phản hồi xấu y_l (gloss: **contrastive** = học bằng cách đẩy hai thứ ra xa nhau). Ý tưởng tự nhiên: tăng $q_\theta(y_t|x)$ cho phản hồi teacher và giảm $q_\theta(y_s|x)$ cho phản hồi student. Tuy nhiên DPKD (Li et al. 2024b, thay mô hình tham chiếu của DPO bằng teacher) hay bị **reward hacking** trong thực nghiệm: vì $p(y_s|x)$ (xác suất của teacher cho chuỗi student) vốn rất nhỏ, tỉ số $q_\theta(y_s)/p(y_s)$ có thể bị đẩy xuống vô hạn, student làm giảm quá mức likelihood của chính nó (NLL của phản hồi student lên 91.25 so với 20.29 của phản hồi teacher, Hình 1b) và sinh câu thoái hóa.

Nền tảng cần nhớ từ DistiLLM (note 30): skew KL $D_{SKL}^{(\alpha)}(x, y; p\|q_\theta) = D_{KL}(p\|\alpha p + (1-\alpha)q_\theta)$ và $D_{SRKL}^{(\alpha)} = D_{KL}(q_\theta\|(1-\alpha)p + \alpha q_\theta)$; adaptive off-policy với replay buffer.

2. Ý tưởng chính

KL thuận "kéo lên" (pulling-up) likelihood q_θ ở những vùng p cao (đầu phân phối teacher, tức chuỗi teacher sinh), còn KL ngược "đẩy xuống" (pushing-down) q_θ ở những vùng p thấp (đuôi, tức chuỗi student sinh mà teacher cho xác suất bé). Vậy một cách tự nhiên để tạo hiệu ứng tương phản trong KD: **SKL trên phản hồi teacher, SRKL trên phản hồi student**, gộp trong cùng một loss (CALD, Contrastive Approach for LLM Distillation). Khác DPKD: loss tuyến tính (phân rã được theo token) và có sự trộn p với q nên tự điều chỉnh, không đẩy $q_\theta(y_s)$ xuống vô hạn. Thêm hai cải tiến: (i) α của skew chọn thích nghi theo độ khó từng mẫu (curriculum), (ii) hệ số của số hạng SRKL tăng dần trong khi huấn luyện. Gộp lại thành DistiLLM-2.

3. Phương pháp chi tiết

3.1 Quan sát về KL và RKL (Hình 1)

Trên dữ liệu long-tail đồ chơi (kiểu Wu et al. 2024, note 31) và trên LLM (Mistral-7B → Danube2-1.8B): KL tăng q_θ ở đầu phân phối teacher và trên TGO; RKL giảm q_θ ở đuôi và trên SGO. Chi tiết toán ở Phụ lục B.1 (không đọc kỹ ở đây).

3.2 Loss CALD

$$\mathcal{L}_{CALD} = \frac{1}{2|\mathcal{D}|} \sum_{(x,y_t,y_s)\sim\mathcal{D}} \left[D^{(\alpha)}_{SKL}(x, y_t) + D^{(\alpha)}_{SRKL}(x, y_s) \right]$$

với $y_t \sim p(\cdot|x)$ (teacher) và $y_s \sim q_\theta(\cdot|x)$ (student). Viết lại (Remark 1): CALD tương đương một dạng

$$-\mathbb{E}_{y_t\sim p,\ y_s\sim q_\theta}\left[\frac{1}{\lambda}\cdot\left(\lambda\log\frac{\tilde{q}_\theta(y_t|x)}{p(y_t|x)} - \lambda\log\frac{q_\theta(y_s|x)}{\tilde{p}(y_s|x)}\right)\right] \text{ (dạng gần với DPKD)}$$

với $\tilde{q}_\theta = \alpha p + (1-\alpha)q_\theta$ và $\tilde{p} = \alpha q_\theta + (1-\alpha)p$ (cách viết đúng từng ký hiệu nằm ở Eq. 6; bản trích xuất hơi vỡ nên đây là diễn giải). Hai khác biệt quan trọng so với DPKD/DPO: (1) dạng tuyến tính thay vì log-sigmoid nên phân rã token-level và có trọng số theo $p(y_t|x)$ hoặc $q_\theta(y_s|x)$; (2) $\tilde{q}_\theta$ phụ thuộc tuyến tính vào p (và $\tilde{p}$ vào q_θ) nên chặn việc $q_\theta(y_s|x)$ giảm quá mức. Nhờ đó tránh reward hacking.

Dùng SKL/SRKL làm backbone cho kết quả tốt hơn KL/RKL thuần, dù KL/RKL cũng hiệu quả trong khung CALD.

3.3 Dữ liệu tối ưu (Section 3.2)

Câu hỏi: dùng SGO và dữ liệu cố định chất lượng cao thế nào cho tốt? Thí nghiệm với speculative generation (thay y_t hoặc y_s bằng y_{spec} khi chấp nhận token nháp theo tiêu chí $q_\theta(y_{n+k}|\cdot) > \min(\epsilon^2, \epsilon\exp(-H(p(\cdot|\cdot))))$):

- Với số hạng SKL, hiệu năng cao nhất khi $y_{spec} = y_t$ ($\epsilon = 1.0$): hướng dẫn mạnh từ phản hồi teacher quan trọng hơn là giảm mismatch.
- Với số hạng SRKL, cao nhất khi $y_{spec} = y_s$ ($\epsilon = 0$), kể cả khi chất lượng thấp nhất: SGO chất lượng thấp lại hữu ích cho phía "đẩy xuống".
- Thay y_t bằng phản hồi của LLM mạnh hơn teacher (Llama-3-8B, Gemma-2-9B, Phi-3-medium thay cho Mistral-7B) **không** tốt hơn; học trên phản hồi của chính teacher vẫn tốt hơn. Giải thích của tác giả: log-likelihood cao dưới teacher quan trọng hơn chất lượng tuyệt đối, vì hiệu ứng pulling-up lớn nhất ở đầu phân phối p.

Kết luận: TGO cho SKL, SGO cho SRKL. Phản hồi được thu thập theo **batch** trước mỗi epoch (dùng vLLM được) thay vì sinh ở mỗi iteration, vừa nhanh vừa gần với tinh thần adaptive off-policy (kết quả sơ bộ ở Phụ lục D.1).

3.4 Curriculum cho α và hệ số β

- **Curriculum α:** α lớn ổn định và hội tụ nhanh nhưng làm khoảng cách p và $\alpha p + (1-\alpha)q_\theta$ nhỏ, thu được ít thông tin; α nhỏ thì ngược lại. Giá trị tối ưu thay đổi theo cặp teacher-student và theo epoch (Bảng 1: epoch sau thường dùng α bằng hoặc nhỏ hơn epoch đầu). Ý tưởng: mẫu "dễ" (p và q gần nhau) dùng α nhỏ; mẫu "khó" dùng α lớn. Dựa trên xấp xỉ bậc nhất (khai triển Mercator) $\log\frac{p(y|x)}{\tilde{q}^{(\alpha)}_\theta(y|x)} \simeq (1-\alpha)\left(p(y|x) - q_\theta(y|x)\right)$ (Eq. 7), paper tính α dạng đóng cho từng mẫu sao cho tích $(1-\alpha)(p-q_\theta)$ nhất quán trong suốt huấn luyện (công thức chính xác nằm trong Algorithm 1, bị vỡ nên không chép lại).
- **Tăng dần hệ số SRKL:** loss có dạng $(1-\beta)D_{SKL}(x, y_t) + \beta D_{SRKL}(x, y_s)$ với β tăng tuyến tính theo tiến độ huấn luyện (clip trong $[\beta_0, 1]$). Lý do: đạt $q_\theta = p$ trên mọi y_t, y_s khó vì capacity gap; tăng dần

β nghĩa là bớt chú trọng bắt chước teacher và tập trung vào phản hồi trực tiếp từ SGO, giảm mismatch train/infer.

3.5 Pseudo-code (Algorithm 1, rút gọn)

```
Cho: teacher p, student q_theta, prompts, alpha_0, beta_0, E epoch, T iter/epoch
for epoch e in 1..E:
    với mỗi prompt x: y_t ~ p(.|x), y_s ~ q_theta_{e-1}(.|x)     # sinh theo batch đầu
    ↪ epoch
    D_e = {(x, y_t, y_s)}
    for iter tau in 1..T:
        mini-batch B từ D_e
        # curriculum alpha (từng mẫu), dựa trên |p - q| trên phản hồi tương ứng
        alpha_t, alpha_s <- cập nhật từ p(.), q_theta(.) và alpha_0 (công thức Eq. 7)
        beta <- clip(e/E + tau/T ..., beta_0, 1)                  # tăng dần
        loss = mean[ (1-beta)*D_SKL^{alpha_t}(x, y_t) + beta*D_SRKL^{alpha_s}(x, y_s) ]
        cập nhật q_theta
```

4. Thí nghiệm và kết quả chính

Thiết lập chung. LoRA (rank 16, mọi lớp tuyến tính của attention và MLP), LR 5.0e-5, batch hiệu dụng 128, 2-3 epoch, A100 80GB. Instruction-following: 50K prompt UltraChat200k; đánh giá AlpacaEval, Evol-Instruct, UltraFeedback bằng LLM-as-a-judge (GPT-4o hoặc GPT-4o-mini) tính winning rate (WR) so với baseline cố định (text-davinci-003 cho AlpacaEval; gpt-3.5-turbo cho Evol-Instruct và UltraFeedback).

Bảng 2 (WR trung bình qua ba benchmark, %):

Phương pháp	Qwen2-7B → Qwen2-1.5B	Mistral-7B → Danube2-1.8B	Gemma-2-9B → Gemma-2-2B
Teacher M_T	76.12	83.01	90.15
Student M_S (trước KD)	30.33	27.02	28.62
KD	41.19	40.00	49.53
SeqKD	41.83	40.11	50.27
ImitKD	43.29	39.03	49.64
GKD	56.14	50.68	69.73
DistiLLM	56.35	52.37	70.30
Speculative KD	54.43	54.50	69.26
DistiLLM-2	**58.69**	**56.45**	**74.83**

DistiLLM-2 vượt baseline mạnh nhất kế tiếp trung bình +2.34%, +1.95%, +4.53% (Qwen2-1.5B, Danube2-1.8B, Gemma2-2B). Ngoại lệ: trên Evol-Instruct với Danube2-1.8B, Speculative KD (38.87) cao hơn DistiLLM-2 (32.84).

Suy luận toán (Bảng 3, Pass@1, 50K mẫu MetaMathQA):

	Qwen2-Math-7B → 1.5B: GSM8K	MATH	TB	Qwen2.5-Math-7B → 1.5B: GSM8K	MATH	TB
Teacher	83.93	41.28	62.61	89.31	44.82	67.07
Student	74.53	25.56	50.05	77.33	27.14	52.24
GKD	75.44	34.16	54.80	80.21	40.54	60.38
DistiLLM	75.59	34.54	55.07	81.05	41.14	61.10

	Qwen2-Math-7B→1.5B: GSM8K	MATH	TB	Qwen2.5-Math-7B→1.5B: GSM8K	MATH	TB
DistiLLM-2	**76.27**	**35.58**	**55.93**	**81.20**	**42.94**	**62.07**

Tác giả nhấn mạnh student Qwen2.5 với DistiLLM-2 vượt cả teacher Qwen2 trên MATH.

Code (Bảng 4, Pass@1; prompt từ WizardCoder):

	DS-Coder-6.7B→1.3B: HumanEval	MBPP	TB	Qwen2.5-Coder-7B→1.5B: HumanEval	MBPP	TB
Teacher	85.37	82.54	83.96	75.61	74.60	75.61
Student	50.61	72.22	61.42	30.73	60.84	45.79
GKD	54.88	74.34	64.61	40.85	61.90	51.38
DistiLLM	53.65	74.34	64.00	39.63	62.17	50.90
DistiLLM-2	**59.92**	**75.66**	**67.79**	**42.24**	**62.70**	**52.47**

(Cột "TB" của Qwen2.5-Coder cho Teacher là 75.61 trong bản trích xuất, có vẻ lỗi bảng; trung bình đúng của 75.61 và 74.60 phải khoảng 75.1. Không rút gì thêm từ ô này.)

5. Phân tích / ablation

- **Thành phần (Bảng 5, WR trung bình tại UltraFeedback/Evol-Instruct, theo ba student; "AVG"):** DistiLLM v1: 44.61 (Qwen2), 28.78 (Danube2), 51.26 (Gemma2), AVG 41.55. Thêm contrastive (1): AVG 43.61. Thêm cả tăng β (1)+(2): 44.80. Chỉ (1)+(3) curriculum α: 45.22. Cả ba (DistiLLM-2): 47.24 / 32.80 / 59.53, AVG 46.50. Mỗi thành phần đều đóng góp.
- **Kích thước dữ liệu (Hình 3):** DistiLLM-2 thắng GKD và DistiLLM ở 13K/25K/50K (instruction), và ở các cỡ cho toán, code.
- **Capacity gap (Bảng 8):** student Qwen1.5-0.5B, teacher Qwen1.5-Chat 1.8B/7B/14B (WR trên Evol-Instruct): DistiLLM-2 = 66.31, 74.86, 76.78, tăng đơn điệu theo cỡ teacher, vượt DistiLLM +1.08, +2.75, +4.67. Tác giả nói DistiLLM v1 vật lộn với teacher 7B và 14B.
- **DPO sau KD (Bảng 6):** dùng student sau KD làm mô hình tham chiếu cho DPO: WR trung bình Qwen2-1.5B: GKD 56.66, DistiLLM 56.75, DistiLLM-2 60.33 (so với 30.09 của student SFT); Gemma-2-2B: 70.98, 68.58, 76.29 (so với 46.10).
- **VLM (Bảng 7):** teacher LLaVA-1.5-7B, student TinyLLaVA-1.4B, 83K prompt RLAIF-V; VQA accuracy trung bình OK-VQA và TextVQA: teacher 48.81, student 32.61, GKD 37.84, DistiLLM 35.24, DistiLLM-2 39.85.
- **Phục hồi mô hình lượng tử hóa INT4 (Bảng 9, Phi-3.5-mini-instruct):** AVG teacher BF16 85.05; INT4 + SFT 78.94; GKD 82.80; DistiLLM 83.35; DistiLLM-2 83.57 (KD qua LoRA).
- **Speculative decoding (Bảng 10):** draft model Llama-68m, tăng tốc với verifier Phi-3-medium: SFT x1.32, GKD x1.64, DistiLLM x1.71, DistiLLM-2 x1.97; với Phi-3.5-mini: x1.24, x1.58, x1.65, x1.84; tỉ lệ chấp nhận token cao nhất cũng thuộc DistiLLM-2 (0.487 và 0.522).

6. Giới hạn

Tác giả nêu: trong bản trích xuất không thấy mục Limitations; chỉ ghi nhận α tối ưu thay đổi theo cặp teacher-student và theo epoch (là động cơ của curriculum), và rằng DistiLLM v1 vật lộn với capacity gap.

Nhận xét:

- Cần sinh cả TGO lẫn SGO cho mỗi prompt ở mỗi epoch, chi phí sinh gấp đôi so với chỉ một nguồn; paper báo kết quả hiệu quả mẫu nhưng chi phí sinh của teacher không được so rõ trong thân bài.
- Đánh giá chủ yếu bằng LLM-as-a-judge (winning rate), có thể nhiễu và thiên vị; đa số kết quả là một cấu hình huấn luyện (LoRA) chứ không phải full fine-tuning.
- Huấn luyện bằng LoRA với student đã có sẵn khả năng (1.5B-2B); chưa biết hiệu quả trên student bị cắt cấu trúc mạnh.
- Công thức α phụ thuộc xấp xỉ bậc nhất, hợp lệ khi $p - q$ nhỏ; có thể sai lệch ở giai đoạn đầu khi student còn xa teacher.
- Cần cùng tokenizer/vocab cho teacher và student.
- So sánh Speculative KD ở Bảng 2 cho thấy ở một cột nó vượt DistiLLM-2, nên ưu thế không phải tuyệt đối.

7. Liên hệ với các bài khác

- **30 DistiLLM:** nền tảng; DistiLLM-2 giữ SKL/SRKL nhưng gán loss theo loại dữ liệu, thêm curriculum.
- **29 GKD, 32 Speculative KD:** hai baseline chính trong Bảng 2; SKD trộn token teacher/student trong cùng chuỗi, DistiLLM-2 giữ hai chuỗi tách riêng và dùng hai loss.
- **31 Rethinking KL:** hiện tượng head/tail (FKL ưu tiên head, RKL ưu tiên tail) chính là cơ sở cho "pulling-up/pushing-down"; DistiLLM-2 dùng tính chất đó thay vì xem chúng là lỗi.
- **33 TAID:** cũng xử lý capacity gap; TAID nội suy đích theo thời gian, DistiLLM-2 dùng α theo mẫu và β theo thời gian.
- **35-37 (Self-Instruct, step-by-step, Orca):** các bài đó tạo dữ liệu (black-box); DistiLLM-2 chỉ ra rằng phản hồi tốt hơn từ LLM mạnh hơn không nhất thiết giúp khi teacher đã định trước.
- **41-42:** capacity gap law và scaling laws đo lại hiện tượng mà Bảng 8 minh họa.
- **47 ShortOPD:** hướng on-policy cho mô hình đã cắt; DistiLLM-2 cho một kết quả liên quan khi KD dùng để phục hồi mô hình nén (Phần 6.3).

8. Tóm tắt công thức / khái niệm

Khái niệm	Nội dung
TGO / SGO	Phản hồi do teacher / student sinh
Pulling-up / pushing-down	KL thuận tăng q ở vùng p cao; KL ngược giảm q ở vùng p thấp
CALD	$\frac{1}{2}[D_{SKL}^{(\alpha)}(x, y_t) + D_{SRKL}^{(\alpha)}(x, y_s)]$
DistiLLM-2	CALD + curriculum α theo mẫu + β tăng dần cho SRKL
Loss đầy đủ	$(1-\beta)D_{SKL}^{(\alpha_t)}(x, y_t) + \beta D_{SRKL}^{(\alpha_s)}(x, y_s)$
Xấp xỉ	$\log \frac{p}{\tilde{q}_\theta^{(\alpha)}} \simeq (1-\alpha)(p - q_\theta)$
Khác DPKD	Tuyến tính (phân rã token), có trộn p, q nên chặn việc đẩy $q(y_s)$ xuống quá mức
Dữ liệu tối ưu	TGO cho SKL, SGO cho SRKL; LLM mạnh hơn teacher không giúp thêm
Kết quả chính	WR 58.69/56.45/74.83 (3 cặp); toán TB 55.93 / 62.07; code TB 67.79 / 52.47

9. Kết luận

DistiLLM-2 dùng skew KL trên phản hồi teacher và skew reverse KL trên phản hồi student, kết hợp lịch điều chỉnh α và β. Phương pháp vượt GKD, DistiLLM và Speculative KD ở phần lớn, nhưng không phải tất cả, thiết lập; bài báo cũng thử phục hồi mô hình lượng tử hóa.

Nguồn gốc: https://arxiv.org/abs/2503.07067

Bài 35

Self-Instruct: Aligning Language Models with Self-Generated Instructions (2023)

Tác giả: Yizhong Wang, Yeganeh Kordi, Swaroop Mishra, Alisa Liu, Noah A. Smith, Daniel Khashabi, Hannaneh Hajishirzi.
Venue / arXiv: arXiv:2212.10560 (ACL 2023).
Nguồn: https://arxiv.org/abs/2212.10560
Chủ đề: Distillation ở mức dữ liệu: dùng một LM để tự sinh dữ liệu instruction-tuning (instruction, input, output), lọc, rồi finetune.
Vị trí trong lộ trình: Đọc sau nhóm KD cổ điển (02 Hinton KD, 15 Sequence-level KD) và nhóm KD cho LLM (27 survey, 28 MiniLLM). Bài này mở đầu nhánh "teacher sinh dữ liệu, student học trên văn bản": nền cho 36 Distilling step-by-step, 37 Orca, 38 DeepSeek-R1 distillation, 39 Small models struggle with strong reasoners. Điểm đặc biệt: ở bài này teacher và student là **cùng một mô hình** (GPT-3), nên nó là "self-distillation ở mức dữ liệu", chưa phải nén mô hình.

Ghi chú học tập bằng tiếng Việt, diễn giải lại, không phải bản dịch.

1. Bối cảnh

- **Instruction tuning** (finetune LM trên cặp "mệnh lệnh bằng ngôn ngữ tự nhiên -> câu trả lời") giúp LM pretrained làm được task mới ở chế độ zero-shot. Các mô hình như InstructGPT, T0, Tk-Instruct đều dựa vào dữ liệu instruction do **con người** viết.
- Vấn đề: dữ liệu người viết đắt, ít đa dạng (đa số là các task NLP quen thuộc, nghiêng về classification), nên giới hạn khả năng tổng quát hoá. Dữ liệu của InstructGPT thì riêng tư.
- **Self-training** (dùng mô hình gán nhãn cho dữ liệu chưa nhãn) đã có, nhưng thường giả định một task cố định và có sẵn input. Self-Instruct muốn sinh **cả task lẫn dữ liệu** từ đầu.
- Gloss: **bootstrapping** = dùng một tập rất nhỏ làm "hạt giống", cho mô hình tự mở rộng dần, mỗi vòng dùng kết quả vòng trước làm ví dụ.

2. Ý tưởng chính

Bắt đầu từ chỉ **175 task do người viết** (mỗi task 1 instruction + 1 instance). Mỗi vòng, lấy mẫu 8 instruction làm ví dụ few-shot, bảo GPT-3 gốc (chưa instruction-tuned) viết instruction mới, rồi viết input/output cho nó. Lọc bỏ cái trùng/xấu, cho phần còn lại vào "task pool" và lặp. Sau cùng được khoảng 52K instruction và 82K instance, dùng để finetune chính GPT-3 đó. Kết quả: GPT-3 vốn "không làm theo mệnh lệnh" gần ngang InstructGPT-001 mà gần như không tốn nhãn người.

3. Phương pháp chi tiết

3.1 Định nghĩa dữ liệu

Một task t gồm instruction I_t và $n_t \geq 1$ instance $(X_{t,i}, Y_{t,i})$. Mô hình M cần $M(I_t, X_{t,i}) = Y_{t,i}$. Input có thể rỗng (ví dụ "write an essay about school safety" tự nó là instruction hoàn chỉnh).

3.2 Pipeline 4 bước

1. **Sinh instruction.** Task pool khởi tạo 175 task. Mỗi bước lấy 8 instruction: 6 từ người, 2 từ các task do mô hình sinh ở bước trước (khuyến khích đa dạng).

2. **Nhận diện classification.** Few-shot prompt (12 ví dụ classification, 19 không-classification) để phân loại: task có không gian nhãn đầu ra nhỏ, hữu hạn thì coi là classification.
3. **Sinh instance.** Hai cách:
   - *Input-first* (cho non-classification): sinh input trước, rồi output.
   - *Output-first* (cho classification): sinh nhãn lớp trước, rồi sinh input điều kiện theo nhãn. Lý do: input-first hay sinh input lệch về một nhãn (ví dụ detect lỗi ngữ pháp thì toàn sinh câu đúng ngữ pháp).
4. **Lọc.**
   - Instruction mới chỉ vào pool nếu ROUGE-L với **mọi** instruction hiện có < 0.7. (ROUGE-L = độ trùng chuỗi con chung dài nhất, thước đo giống nhau về bề mặt.)
   - Loại instruction chứa từ khoá LM không xử lý được (image, picture, graph).
   - Loại instance trùng hệt, hoặc cùng input khác output; loại cái quá dài/ngắn, output chỉ lặp lại input.

3.3 Finetune

Nối instruction + input thành prompt, huấn luyện sinh output (loss chuẩn cross-entropy trên output). Dùng nhiều template (có/không "Task:", "Input:", "Output:", số dòng trống khác nhau) để mô hình bền với format. GPT-3 "davinci" được finetune qua OpenAI API, prompt loss weight = 0, 2 epoch.

$$\boxed{\mathcal{L} = -\sum_{(I,X,Y)\in\mathcal{D}_{\text{gen}}} \log p_\theta(Y \mid I, X)}$$

(Công thức trên là dạng SFT chuẩn, paper mô tả bằng lời chứ không đánh số công thức.)

3.4 Pseudo-code

```
pool  = seed_tasks (175)
while len(pool) < target:
    demos = sample(6 từ seed, 2 từ pool_generated)
    inst  = LM.generate_instruction(demos)
    if rougeL(inst, pool) >= 0.7 or has_banned_keyword(inst): continue
    is_cls = LM.classify_task(inst)               # few-shot
    if is_cls: instances = LM.output_first(inst)  # nhãn -> input
    else:      instances = LM.input_first(inst)   # input -> output
    instances = filter_dups_and_invalid(instances)
    if instances: pool.add(inst, instances)
SFT(LM, pool_generated)                           # finetune chính LM
```

4. Thí nghiệm và kết quả chính

Dữ liệu sinh ra (GPT-3 davinci, Table 1): 52,445 instruction (11,584 classification, 40,861 non-classification); 82,439 instance, trong đó 35,878 có input rỗng. Độ dài trung bình: instruction 15.9 từ, input không rỗng 12.7 từ, output 18.9 từ.

Chất lượng (Table 2, 200 instruction, một annotator là tác giả):

Câu hỏi	Yes
Instruction mô tả task hợp lệ?	92%
Input phù hợp với instruction?	79%
Output đúng và chấp nhận được?	58%
Tất cả trường đều hợp lệ	54%

Zero-shot trên SuperNI (119 task unseen, ROUGE-L, Table 3):

Mô hình	Params	ROUGE-L
T5-LM (vanilla)	11B	25.7
GPT-3 (vanilla)	175B	6.8
T0	11B	33.1
GPT-3 + T0 training	175B	37.9
GPT-3 Self-Instruct	175B	**39.9**
InstructGPT-001	175B	40.8
Tk-Instruct	11B	46.0
GPT-3 + SuperNI training	175B	49.5
GPT-3 Self-Instruct + SuperNI training	175B	51.6

Kết luận paper: +33.1 điểm so với GPT-3 gốc, gần bằng InstructGPT-001, và vẫn cộng thêm điểm khi kết hợp với dữ liệu SuperNI (dữ liệu bổ trợ).

Đánh giá người trên 252 instruction hướng người dùng (do tác giả viết): Self-Instruct vượt các biến thể GPT-3 finetune trên dữ liệu công khai (T0, SuperNI) khá nhiều, và chỉ kém InstructGPT-001 khoảng 5% (tuyệt đối). Số liệu từng cột trong Figure 6 bị hỏng khi trích PDF nên không chép ở đây.

5. Phân tích / ablation

- **Quy mô dữ liệu (Figure 7):** hiệu năng tăng đều theo số instruction, **gần như bão hoà sau 16K**. Trên SuperNI thì bão hoà sớm hơn nhiều (khoảng vài trăm instruction), nhóm tác giả cho rằng do dữ liệu sinh ra khác phân phối task NLP điển hình.
- **Chất lượng output (đây là "distillation" đúng nghĩa):** dùng InstructGPT-003 sinh lại trường output cho toàn bộ instance rồi finetune GPT-3, kết quả hơn bản dùng output gốc 10% (đánh giá người). Số trong Figure 7: các điểm đọc được gồm 31.0%, 36.9%, 43.7%, 44.4%, 54.4%; không đủ chắc để gán điểm nào cho đường nào nên không ghép.
- **Đa dạng:** chỉ 26,559/52,445 instruction có cấu trúc động từ-danh từ trích được; nhiều instruction có ROUGE-L thấp so với seed, tức là không chỉ sao chép seed.
- **Lỗi vẫn hữu ích:** dù output chỉ đúng 58%, phần lớn lỗi vẫn đúng định dạng hoặc đúng một phần, đủ làm tín hiệu học format.

6. Giới hạn

Do tác giả nêu:

- Thừa hưởng điểm yếu của LM: lợi ích dồn vào các instruction thường gặp (đầu phân phối), có thể mong manh với instruction hiếm/sáng tạo.
- Phụ thuộc mô hình lớn: có thể chỉ hiệu quả với model lớn (chưa khảo sát theo kích thước).
- Khuếch đại thiên kiến xã hội; khó sinh nhãn cân bằng.

Nhận xét:

- Chỉ thử trên một mô hình (GPT-3 175B), nên không biết dữ liệu này có huấn luyện được student nhỏ không.
- Chất lượng output chỉ 58% đúng; không có bước kiểm chứng đáp án (khác với các bài reasoning sau này có verifier).
- Annotator chất lượng là chính tác giả, 200 mẫu; bộ test 252 instruction do tác giả viết nên có thể thiên lệch.

- Metric ROUGE-L trên SuperNI nhạy với format hơn là đúng sai về nghĩa.

7. Liên hệ với các bài khác

- **02 Hinton KD / 15 Sequence-level KD:** KD cổ điển học từ phân phối xác suất (soft label); Seq-KD học từ chuỗi do teacher sinh. Self-Instruct nằm ở chỗ "học từ văn bản teacher sinh", mở rộng thêm: teacher sinh luôn cả **đầu vào** (instruction), không chỉ đầu ra.
- **28 MiniLLM / 29 GKD / 30 DistiLLM:** nhóm logit/phân phối (white-box). Self-Instruct và nhóm 35-39 là nhóm black-box: chỉ cần text từ teacher, không cần logit. Điểm này quan trọng khi teacher chỉ có qua API.
- **36 Distilling step-by-step:** thêm "rationale" vào dữ liệu teacher sinh.
- **37 Orca:** thay output ngắn bằng explanation trace dài của GPT-4.
- **38, 39:** cùng dòng, áp dụng cho reasoning dài.
- **27 LLM KD survey:** phân loại này thuộc "data augmentation / synthetic data" trong survey.

8. Tóm tắt công thức / khái niệm

Khái niệm	Ý nghĩa
Instruction tuning	SFT trên cặp (mệnh lệnh, đáp án)
Seed tasks	175 task người viết để khởi động
Input-first / Output-first	Sinh input rồi output / sinh nhãn rồi input (cho classification)
Lọc ROUGE-L < 0.7	Chống trùng lặp instruction
Dữ liệu cuối	52,445 instruction, 82,439 instance
Kết quả chính	SuperNI ROUGE-L 6.8 -> 39.9 (InstructGPT-001: 40.8)
Data distillation	Dùng InstructGPT-003 sinh lại output: +10% (human eval)

9. Kết luận

Self-Instruct dùng 175 ví dụ mồi để sinh và lọc dữ liệu instruction, rồi fine-tune mô hình từ tập thu được. Kết quả tiến gần InstructGPT-001 trong đánh giá của bài báo. Quy trình làm rõ vai trò của tập mồi, bước sinh và bước lọc trong chưng cất bằng văn bản.

Link gốc: https://arxiv.org/abs/2212.10560

Bài 36

Distilling Step-by-Step! Outperforming Larger Language Models with Less Training Data and Smaller Model Sizes (2023)

Tác giả: Cheng-Yu Hsieh, Chun-Liang Li, Chih-Kuan Yeh, Hootan Nakhost, Yasuhisa Fujii, Alexander Ratner, Ranjay Krishna, Chen-Yu Lee, Tomas Pfister.
Venue / arXiv: arXiv:2305.02301 (bản v2, tháng 7/2023; trang đầu của bản text không ghi tên hội nghị, trích dẫn công khai là Findings of ACL 2023 nhưng paper text ta có không nêu nên không khẳng định).
Nguồn: https://arxiv.org/abs/2305.02301
Chủ đề: Distillation task-specific từ LLM: lấy cả nhãn lẫn **rationale** (lời giải thích từng bước) của teacher, huấn luyện student theo multi-task.
Vị trí trong lộ trình: Đọc sau 35 Self-Instruct (teacher sinh dữ liệu) và 15 Sequence-level KD. Bài này bổ sung ý "teacher không chỉ cho nhãn mà cho cả lý do", và dùng rationale như **tín hiệu phụ chỉ có khi huấn luyện**. Tiếp theo: 37 Orca (explanation trace quy mô lớn), 38 và 39 (reasoning dài, nơi rationale thành phần chính).

> Ghi chú học tập bằng tiếng Việt, diễn giải lại, không phải bản dịch.

1. Bối cảnh

- LLM (PaLM 540B...) rất mạnh few-shot nhưng khó triển khai: nặng, tốn bộ nhớ, độ trễ cao.
- Cách thông thường để có model nhỏ chuyên task: (a) **finetuning** trên nhãn người (đắt), hoặc (b) **task distillation**: dùng LLM gán **nhãn giả** cho dữ liệu chưa nhãn rồi train model nhỏ (cần rất nhiều dữ liệu chưa nhãn).
- Cả hai đều cần nhiều dữ liệu để bắt kịp LLM.
- Gloss: **rationale** = đoạn giải thích trung gian dẫn từ input đến nhãn (ví dụ "Diện tích = dài x rộng; phòng có 11x15..."). **Chain-of-thought (CoT) prompting** = đưa vài ví dụ có lời giải từng bước vào prompt để LLM tự sinh lời giải. **Task-specific model** = model nhỏ chỉ làm một task (khác với model đa dụng).

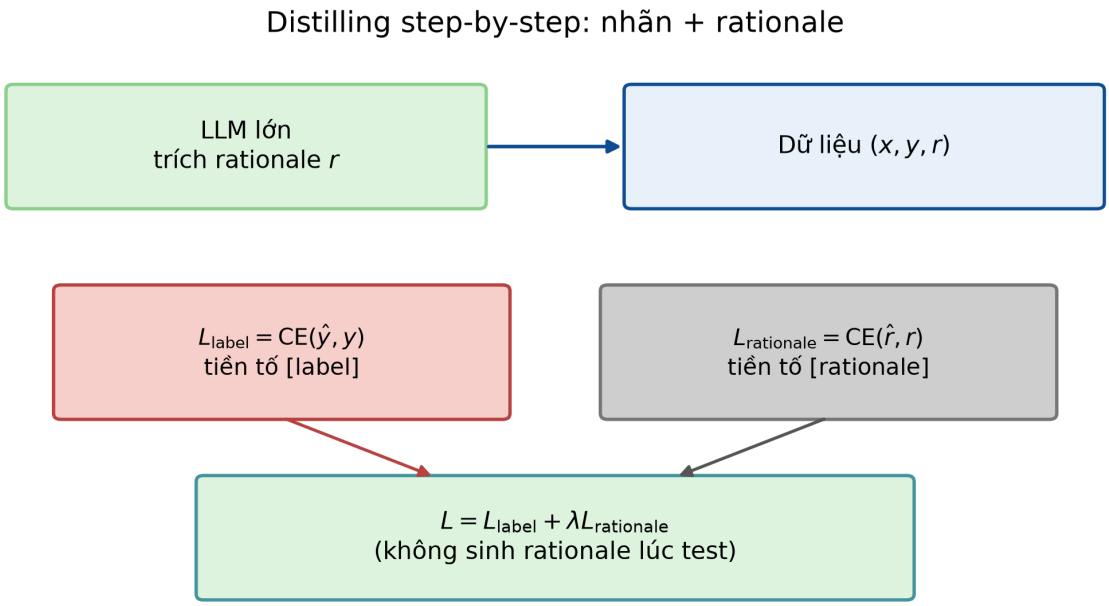
2. Ý tưởng chính

Thay vì coi LLM là nguồn nhãn nhiễu, coi nó là tác nhân biết lập luận: prompt CoT để LLM vừa trả nhãn vừa viết rationale. Sau đó train model nhỏ (T5) với **hai nhiệm vụ**: dự đoán nhãn và sinh rationale, phân biệt bằng tiền tố "[label]" / "[rationale]" ở input. Rationale chứa kiến thức task (ví dụ "diện tích = dài x rộng") mà model nhỏ khó tự suy ra từ ít dữ liệu. Khi inference chỉ cần dự đoán nhãn, không phải sinh rationale, nên không tốn thêm chi phí triển khai.

3. Phương pháp chi tiết

3.1 Bước 1: trích rationale từ LLM

Prompt few-shot gồm các bộ ba (x^p, r^p, y^p) (ví dụ, rationale người viết, nhãn). Với mỗi x_i chưa nhãn, ghép vào prompt, LLM bắt chước và sinh $(\hat{r}_i, \hat{y}_i)$.



Note: T5-770M + 80% ANLI vượt PaLM-540B few-shot CoT

Distilling step-by-step: LLM trích rationale; student multi-task $L_{\text{label}} + \lambda L_{\text{rationale}}$ (minh hoạ, tự tính)

3.2 Bước 2: huấn luyện multi-task

Task distillation chuẩn (nhãn giả $\hat{y}_i$, hoặc nhãn người y_i nếu finetuning):

$$L_{\text{label}} = \frac{1}{N}\sum_{i=1}^{N} \ell\big(f(x_i), \hat{y}_i\big)$$

Có hai cách đưa rationale vào:

- Cách "rationale làm input" (Eq. 2): $L = \frac{1}{N}\sum \ell(f(x_i, \hat{r}_i), \hat{y}_i)$. Nhược điểm: lúc test vẫn cần LLM sinh rationale trước, mất ý nghĩa nén.
- **Cách của bài báo (Eq. 3-4):** rationale làm **đích thứ hai**:

$$\boxed{L = L_{\text{label}} + \lambda L_{\text{rationale}}, \qquad L_{\text{rationale}} = \frac{1}{N}\sum_{i} \ell\big(f(x_i), \hat{r}_i\big)}$$

Cùng một model f, hai đích khác nhau tuỳ tiền tố task. Paper không nêu giá trị λ cụ thể trong phần text ta đọc được (setup trong appendix chỉ nêu learning rate, batch size, số step).

Biến thể đối chứng "single-task" (Eq. 5): gộp thành một đích $[\hat{r}_i, \hat{y}_i]$:

$$L_{\text{single}} = \frac{1}{N}\sum_{i} \ell\big(f(x_i), [\hat{r}_i, \hat{y}_i]\big)$$

3.3 Pseudo-code

```
# Giai đoạn A: sinh rationale (một lần, offline)
for x in D_unlabeled_or_labeled:
    (r_hat, y_hat) = LLM(CoT_prompt + x)          # PaLM 540B, ~10-shot
# Giai đoạn B: train student T5
```

```
for (x, y_hat, r_hat) in batch:
    loss_label = CE(f("[label] " + x),     y_hat)
    loss_rat   = CE(f("[rationale] " + x), r_hat)
    loss = loss_label + lam * loss_rat
# Inference: chỉ f("[label] " + x), không sinh rationale
```

3.4 Ví dụ nhỏ

SVAMP: "Phòng dài 11 ft, rộng 15 ft, đã có 16 sq ft thảm. Cần thêm bao nhiêu?" Đích nhãn: "(11x15)-16". Đích rationale: "Area = length x width. Jesse's room has 11x15 square feet." Model học "công thức diện tích" từ nhiệm vụ rationale, rồi dùng khi dự đoán nhãn.

4. Thí nghiệm và kết quả chính

Setup: LLM = PaLM 540B; student = T5 (220M T5-Base, 770M T5-Large, 11B T5-XXL). 4 dataset: e-SNLI, ANLI (NLI), CQA (commonsense QA), SVAMP (toán lời văn). Số mẫu train: e-SNLI 549,367; ANLI 16,946; CQA 8,766; SVAMP 720. Huấn luyện: lr 5e-5, batch 64 (T5-Base/Large, tối đa 10000 step), batch 32 (T5-XXL, tối đa 4000 step); max input 1024; 4 lần chạy ngẫu nhiên; A100x16.

Kết quả định tính chính (so với finetuning/distillation chuẩn, T5 220M):

- Với nhãn người: chỉ **12.5%** e-SNLI đã vượt finetuning chuẩn dùng 100%. Giảm lượng dữ liệu cần để vượt finetuning chuẩn: 75% (ANLI), 25% (CQA), 20% (SVAMP).
- Chỉ dữ liệu chưa nhãn: 12.5% tập chưa nhãn e-SNLI đã vượt task distillation chuẩn dùng 100%.
- Tóm tắt abstract/intro: dùng ít hơn 50% dữ liệu trung bình (có chỗ trên 85%).

Giảm kích thước model:

- Với nhãn người: vượt Few-shot CoT của PaLM 540B bằng T5 220M trên e-SNLI (nhỏ hơn hơn 2000 lần), T5 770M trên ANLI và SVAMP (hơn 700 lần), T5 11B trên CQA (hơn 45 lần).
- Chỉ dữ liệu chưa nhãn: vượt LLM ở 3/4 dataset (e-SNLI với 220M; 45 lần nhỏ hơn ở ANLI và CQA); SVAMP chỉ bắt kịp sau khi thêm dữ liệu chưa nhãn ASDiv (2,305 mẫu), bằng T5 11B (dưới 3% kích thước PaLM).
- Trên ANLI, cải thiện trung bình so với finetuning chuẩn là 8% và so với distillation chuẩn là 13% (độ chính xác).

Tối thiểu cả dữ liệu lẫn kích thước (Figure 8-9): trên ANLI, T5 770M chỉ cần 80% dữ liệu để vượt PaLM Few-shot CoT, trong khi finetuning chuẩn không bắt kịp ngay cả với 100%. Trên e-SNLI, model 220M và 0.1% dữ liệu đã vượt Few-shot CoT.

5. Phân tích / ablation

Phụ thuộc chất lượng LLM (Table 1, T5 220M, 100% dữ liệu):

Phương pháp	LLM	e-SNLI	ANLI	CQA	SVAMP
Standard finetuning	N/A	88.38	43.58	62.19	62.63
Distilling step-by-step	20B (GPT-NeoX)	89.12	48.15	63.25	63.00
Distilling step-by-step	540B (PaLM)	89.51	49.58	63.29	65.50

LLM nhỏ hơn vẫn cho lợi ích nhưng ít hơn; paper cho rằng do rationale chất lượng thấp hơn (đây là giải thích của tác giả).

Multi-task vs single-task (Table 2):

Phương pháp	e-SNLI	ANLI	CQA	SVAMP
Standard finetuning	88.38	43.58	62.19	62.63
Single-task (rationale + nhãn gộp)	88.88	43.50	61.37	63.00
Multi-task	89.51	49.58	63.29	65.50

Điểm đáng chú ý: gộp rationale vào chuỗi đích có thể **tệ hơn finetuning chuẩn** (ANLI, CQA). Phải tách thành task riêng.

6. Giới hạn

Do tác giả nêu:

- Cần người viết khoảng 10 ví dụ CoT cho mỗi task (có thể giảm bằng zero-shot CoT).
- Tốn thêm chút tính toán khi huấn luyện (không tốn lúc test).
- LLM có thể lập luận kém ở task phức tạp/planning; chất lượng rationale ảnh hưởng chưa được khảo sát kỹ.
- Student thừa hưởng thiên kiến của teacher.

Nhận xét:

- Chỉ 4 benchmark NLP, đầu ra ngắn (nhãn / biểu thức), và student là T5 encoder-decoder; chưa rõ với decoder-only hay đầu ra dài.
- Không có lọc rationale sai (rationale có thể đúng nhãn nhưng sai logic, hoặc nhãn sai); bài báo không đo tỷ lệ này.
- Giá trị λ và độ nhạy với nó không được trình bày trong phần ta đọc.
- Không đo chi phí gọi PaLM 540B để sinh rationale cho toàn bộ dữ liệu so với chi phí thu nhãn người.

7. Liên hệ với các bài khác

- **01 Ba & Caruana / 02 Hinton KD:** "Model compression" gốc dùng logit làm tín hiệu; ở đây tín hiệu phụ là **văn bản giải thích**, một dạng "dark knowledge" ở dạng ngôn ngữ.
- **15 Sequence-level KD:** student học chuỗi do teacher sinh; Distilling step-by-step cũng vậy nhưng dùng cả hai chuỗi (nhãn, rationale) như hai nhiệm vụ.
- **03 FitNets / 04 Attention transfer:** cũng là "tín hiệu phụ chỉ dùng khi huấn luyện" (hint layer / attention map); tương tự về tinh thần, khác ở chỗ ở đây là ngôn ngữ tự nhiên, không cần cùng kiến trúc.
- **35 Self-Instruct:** cùng nhánh black-box; 35 sinh tập instruction, 36 sinh rationale cho task cố định.
- **37 Orca:** tăng quy mô ý tưởng "giải thích từng bước" lên dữ liệu đa task, dùng system prompt để gợi explanation.
- **39 Small models struggle with strong reasoners:** cảnh báo rationale quá dài/phức tạp có thể làm student nhỏ học kém; đối chiếu với ở đây rationale ngắn, rất hữu ích.

8. Tóm tắt công thức / khái niệm

Khái niệm	Ý nghĩa
Rationale	Lời giải thích trung gian do LLM sinh
L_{label}	CE giữa dự đoán và nhãn (người hoặc giả)
$L_{\text{rationale}}$	CE giữa dự đoán và rationale
$L = L_{\text{label}} + \lambda L_{\text{rationale}}$	Mục tiêu multi-task của bài báo
Tiền tố task	"[label]" / "[rationale]" để phân biệt nhiệm vụ

Khái niệm	Ý nghĩa
Lợi ích triển khai	Không sinh rationale lúc test
Kết quả nổi bật	T5 770M + 80% dữ liệu ANLI vượt PaLM 540B few-shot CoT

9. Kết luận

Distilling Step-by-Step dùng rationale của teacher làm tín hiệu bổ sung: student học nhãn và rationale như hai nhiệm vụ. Trong các thí nghiệm, student cần ít dữ liệu hơn và có thể vượt LLM trên tác vụ hẹp mà không phải sinh rationale khi suy luận.

Link gốc: https://arxiv.org/abs/2305.02301

Bài 37

Orca: Progressive Learning from Complex Explanation Traces of GPT-4 (2023)

Tác giả: Subhabrata Mukherjee, Arindam Mitra (đồng tác giả chính), Ganesh Jawahar, Sahaj Agarwal, Hamid Palangi, Ahmed Awadallah (Microsoft Research).
Venue / arXiv: arXiv:2306.02707 (v1, 5/6/2023; bản ghi "Work in progress", không nêu hội nghị).
Nguồn: https://arxiv.org/abs/2306.02707
Chủ đề: Distillation black-box ở quy mô lớn: học từ **explanation trace** của GPT-4/ChatGPT (kích bằng system message), huấn luyện 2 giai đoạn (teacher assistant -> teacher mạnh), đánh giá nghiêm ngặt hơn bằng benchmark lập luận.
Vị trí trong lộ trình: Đọc sau 35 Self-Instruct (instruction đơn giản, response ngắn) và 36 Distilling step-by-step (rationale cho task cố định). Bài này mở rộng thành tập đa task 5M và bàn "teacher assistant" (liên quan 08 Teacher assistant, nền cho 41 Capacity gap law). Tiếp theo: 38 DeepSeek-R1 distillation, 39 Small models struggle with strong reasoners (bàn ngược lại: trace càng dài/mạnh chưa chắc tốt cho student nhỏ).

> Ghi chú học tập bằng tiếng Việt, diễn giải lại, không phải bản dịch.

1. Bối cảnh

Sau ChatGPT, nhiều nhóm train model nhỏ bắt chước nó: Alpaca (Self-Instruct, 52K, text-davinci-003 làm teacher), Vicuna (hội thoại ShareGPT ~70K, ChatGPT), WizardLM (Evol-Instruct, 250K). Paper nêu 4 vấn đề:

1. **Instruction đơn giản, kém đa dạng** (kiểu "thủ đô của Pháp là gì").
2. **Chỉ có cặp (query, response) ngắn**, ít tín hiệu để theo dõi quá trình suy luận của teacher. Với model mã nguồn mở (white-box) thì có logit, hidden state, attention; với LFM đóng thì không (chỉ có text, và top-5 logit qua API).
3. **Dữ liệu nhỏ và lệch** (nghiêng về sáng tác, hỏi đáp thông tin).
4. **Đánh giá yếu**: dùng GPT-4 chấm trên bộ nhỏ (80 / 218 prompt), GPT-4 thiên vị câu dài và vị trí câu trả lời. Vì thế model nhỏ bị "đánh giá quá cao". Paper trích Gudibande et al. "model imitation is a false promise" và muốn phản biện.

Gloss: **LFM** (large foundation model) = ChatGPT/GPT-4. **Imitation learning** = học bắt chước đầu ra của teacher. **System message** = đoạn chỉ dẫn vai trò/phong cách đặt ở đầu prompt (role "system" trong chat API). **Curriculum / progressive learning** = học từ dễ đến khó.

2. Ý tưởng chính

Dùng system message để buộc GPT-4/ChatGPT **giải thích từng bước** (ví dụ "think step-by-step and justify your steps", "explain like I am five"), nên mỗi mẫu huấn luyện chứa cả quá trình lập luận chứ không chỉ đáp án. Lấy câu hỏi từ FLAN-v2 (rất đa dạng, nhiều task), thu 5 triệu phản hồi của ChatGPT, trong đó 1 triệu cũng có phản hồi GPT-4. Train model 13B (nền LLaMA) hai giai đoạn: trước học ChatGPT (dễ, ngắn hơn, rẻ), sau học GPT-4 (khó, dài hơn). Kết quả: lập luận zero-shot cải thiện mạnh so với Vicuna-13B, ngang ChatGPT trên Big-Bench Hard, nhưng vẫn kém GPT-4.

3. Phương pháp chi tiết

3.1 Cấu trúc mẫu

Mỗi mẫu là bộ ba:

$$\langle \text{system message}, \ \text{user query}, \ \text{LFM response} \rangle$$

Phần khác biệt chủ chốt so với instruction tuning thường (chỉ có $\langle \text{query}, \text{response} \rangle$) là system message.

3.2 16 system message

Soạn tay 16 system message (Table 2), ví dụ: rỗng; "Provide a detailed answer so user don't need to search outside"; "Think like you are answering to a five year old"; "think step-by-step and justify your steps"; "Explain how you used the definition to come up with the answer"; "break the definition into small parts...". Chọn system message theo sub-collection; message 8 và 10 chỉ dùng với câu hỏi trắc nghiệm. Mục đích: vừa tạo trace dài, vừa giữ khả năng trả lời ngắn.

3.3 Dữ liệu từ FLAN-v2

FLAN-v2 có 5 sub-collection: CoT, NiV2, T0, Flan 2021, Dialogue. Chỉ lấy câu zero-shot. Lấy mẫu (Table 3):

Sub-collection	Cách lấy	Kích thước gốc	Lấy
CoT	không lấy mẫu	150K	150K
NiV2	phân tầng (tối đa 300 query/task)	5M	440K
Flan 2021	phân tầng	>28.9M (ghi trong bảng)	2.5M
T0	phân tầng, chỉ tập train của T0 (loại Big-Bench)	85.7M	2M
Dialog	bỏ	22.5M	0

Hai tập: **FLAN-5M** (câu hỏi + system message, có phản hồi ChatGPT) và **FLAN-1M** (mẫu con 1M của FLAN-5M, có phản hồi GPT-4). Thuật toán lấy mẫu (Algorithm 1): lặp chọn ngẫu nhiên một task, lấy một query không hoàn lại từ task đó, bỏ task khi cạn.

3.4 ChatGPT làm "teaching assistant"

Huấn luyện tuần tự: FLAN-5M (ChatGPT) rồi FLAN-1M (GPT-4). Hai lý do paper nêu:

- **Capacity gap:** student 13B nhỏ hơn nhiều so với GPT-4; teacher trung gian giảm khoảng cách (tham chiếu ý tưởng teacher assistant, mục 08). Xem như curriculum: phản hồi GPT-4 trung bình dài hơn khoảng 1.5 lần ChatGPT (giả định "dài hơn là khó bắt chước hơn").
- **Chi phí/thời gian:** ChatGPT \$0.002/1K token, 300 request/phút, 120,000 token/phút; GPT-4 8K \$0.03 (prompt) / \$0.06 (completion) trên 1K token, 18 request/phút, 10,000 token/phút (Table 4). Thu dữ liệu mất 2 tuần (ChatGPT) và 3 tuần (GPT-4).

3.5 Huấn luyện

- Tokenizer LLaMA BPE + token [[PAD]] -> vocab 32,001.
- **Packing:** ghép nhiều mẫu vào một chuỗi tối đa 2,048 token, trung bình 2.7 mẫu/chuỗi.
- **Loss chỉ trên token do teacher sinh** (không tính loss trên system message/query):

$$\mathcal{L}(\theta) = -\sum_{t \in \text{response}} \log p_\theta(y_t \mid \text{system}, \ \text{query}, \ y_{<t})$$

- Tính toán: 20 GPU A100 80GB; 160 giờ cho FLAN-5M (4 epoch), 40 giờ cho FLAN-1M (4 epoch).
- Khi đánh giá mặc định dùng system message rỗng (Id 1), nhiệt độ 0.7.

3.6 Pseudo-code

```
# Thu thập dữ liệu
Q5M = sample_flan_v2(CoT 150K, NiV2 440K, Flan2021 2.5M, T0 2M, ...)   # ~5M
for q in Q5M:  s = pick_system_message(q.subcollection)  # 1 trong 16
               D5M.add(s, q, ChatGPT(s, q))
Q1M = random_subset(Q5M, 1M)
for q in Q1M:  D1M.add(s, q, GPT4(s, q))
# Huấn luyện tuần tự (progressive)
Orca = LLaMA13B
Orca = SFT(Orca, D5M, epochs=4)   # giai đoạn 1: ChatGPT
Orca = SFT(Orca, D1M, epochs=4)   # giai đoạn 2: GPT-4
# loss chỉ trên token response
```

4. Thí nghiệm và kết quả chính

Baseline: Text-Davinci-003 (TD-003), ChatGPT, GPT-4, Vicuna-13B. **Benchmark:** Vicuna prompts (80), Awesome prompts (164), WizardLM prompts (218) chấm bằng GPT-4; AGIEval (3546 câu trắc nghiệm tiếng Anh: SAT, LSAT, GRE/GMAT qua AQuA-RAT, LogiQA); Big-Bench Hard (5511 câu trắc nghiệm); TruthfulQA-MC (684 câu); ToxiGen. Đánh giá lập luận ở zero-shot, không exemplar, không CoT.

4.1 Sinh văn mở (GPT-4 chấm, Table 7: điểm của candidate, % so với reference)

Dataset	Reference	Vicuna-13B	Orca-13B
Vicuna Prompts	ChatGPT	92	101.5 (10.4%)
Vicuna Prompts	GPT-4	73.8	87.7 (18.9%)
Awesome Prompts	ChatGPT	86.5	98.1 (13.5%)
Awesome Prompts	GPT-4	77.8	89.3 (14.9%)
WizardLM Prompts	ChatGPT	77.1	84.9 (10.1%)
WizardLM Prompts	GPT-4	69.1	78.4 (13.5%)
Trung bình	ChatGPT	85.2	94.8 (11.3%)
Trung bình	GPT-4	73.6	85.1 (13.5%)

Phần trăm trong ngoặc là mức cải thiện của Orca so với Vicuna. Paper tóm: Orca giữ 95% chất lượng ChatGPT và 85% GPT-4 (do GPT-4 chấm). Lưu ý của chính tác giả: GPT-4 thiên vị câu trả lời đứng đầu, nên các so sánh đều đặt reference ở vị trí đầu như Vicuna (với WizardLM prompts, Vicuna đạt 84.87 nếu đảo vị trí).

4.2 AGIEval (zero-shot, Table 8, độ chính xác %)

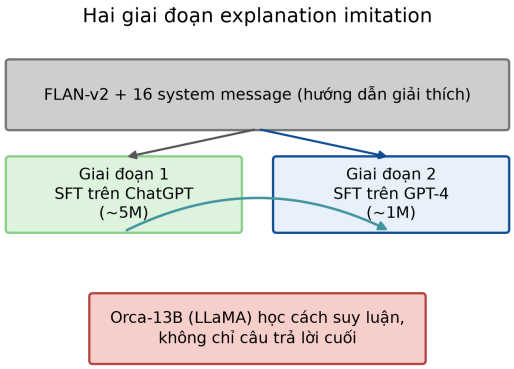
Task	TD-003	ChatGPT	GPT-4	Vicuna-13B	Orca-13B
AQuA-RAT	29.9	31.9	40.6	20.1	27.9
LogiQA	22.7	35	49.3	29.8	35.2
LSAT-AR	21.7	24.4	35.2	20.4	21.3
LSAT-LR	47.5	52.6	80.6	32.6	43.9
LSAT-RC	64.7	65.4	85.9	32.7	57.3
SAT-Math	35.5	42.7	64.6	28.6	32.3
SAT-English	74.8	81.1	88.8	44.2	76.7

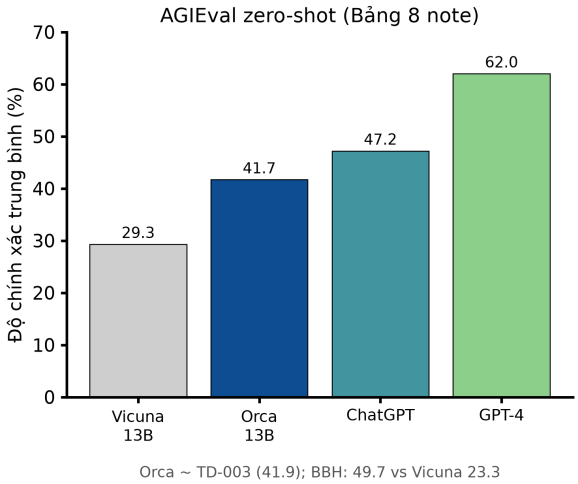
Task	TD-003	ChatGPT	GPT-4	Vicuna-13B	Orca-13B
SAT-English (w/o Psg.)	38.4	44.2	51	26.2	38.8
Trung bình	41.9	47.2	62	29.3	**41.7**

Orca ngang TD-003, giữ 88% chất lượng ChatGPT, hơn Vicuna 42% (tương đối). Vicuna chỉ giữ 62% ChatGPT ở đây, so với 85% ở sinh văn mở.

4.3 Big-Bench Hard (zero-shot không CoT, Table 11, trung bình)

ChatGPT	GPT-4	Vicuna-13B	Orca-13B
48.9	67.4	23.3	**49.7** (+113.7% so với Vicuna)





Orca hai giai đoạn SFT trên trace ChatGPT rồi GPT-4; AGIEval trung bình từ Bảng 8 note (minh hoạ, tự tính)

Orca nhỉnh hơn ChatGPT về trung bình, kém GPT-4 (paper ghi "lags GPT-4 by 26%"). Theo từng nhóm: Orca tốt hơn ChatGPT ở entailment, semantic understanding, temporal reasoning (Temporal Sequences 72.0 vs 35.6); kém hơn ở tri thức thế giới (Sports, Ruin Names), logic/hình học (Geometric Shapes 20.8 vs 25.6, Boolean Expressions 72.0 vs 82.8), hiểu bảng (Penguins in a Table 42.5 vs 45.9). Tác giả lưu ý GPT-4 có thể bị nhiễm Big-Bench.

4.4 An toàn (tóm tắt)

TruthfulQA-MC: Orca trung thực hơn Vicuna nhưng thua ChatGPT/GPT-4 (số liệu nằm trong hình, không đọc được chắc chắn nên không chép). ToxiGen (HateBERT chấm): Orca ít sinh nội dung độc hơn Vicuna; với prompt trung tính, Orca gần ngang GPT-4. Tác giả gán một phần cho căn chỉnh của GPT-4 và content filter của Azure.

5. Phân tích / ablation

- **Teacher assistant / quy mô (Table 10, AGIEval):** chỉ train trên FLAN-1M (GPT-4) có trung bình 37.18; Orca đầy đủ (FLAN-5M rồi FLAN-1M) 41.7, tức +4.5 điểm. Paper quy cho "scale dữ liệu 5x với ChatGPT trung gian". Lưu ý: ablation này gộp hai yếu tố (nhiều dữ liệu hơn + giai đoạn ChatGPT), không tách bạch được tác dụng riêng của "teacher trung gian" so với "chỉ nhiều dữ liệu hơn". Đây là nhận xét bổ sung.

- **System message khi suy luận (Table 9):** system rỗng 41.7 trung bình, "Follow Well" 40.3, "Detailed Answer" 41.6; chọn system tốt nhất cho từng task thì hẹp khoảng cách với ChatGPT còn 4.4 điểm.
- **Phân tích lỗi (100 mẫu mỗi hướng):** ChatGPT thắng Orca nhiều hơn ở ngữ cảnh dài (16% vs 8%), kiến thức miền, và lập luận phức tạp; Orca thắng ở nhiều mẫu LogiQA; LaTeX và hình học chiếm tỷ lệ nhỏ.

6. Giới hạn

Do tác giả nêu:

- Kế thừa thiên kiến, hallucination, rủi ro lạm dụng của LLM/GPT-4; không rõ model nhỏ dễ hallucinate hơn hay không.
- Huấn luyện chỉ trên dữ liệu kiểu zero-shot: chưa kiểm hội thoại nhiều lượt, in-context learning, few-shot, CoT prompting.
- Hiệu năng phụ thuộc phân phối dữ liệu; toán, code, lập luận ít xuất hiện nên yếu.
- Phản hồi có thể thay đổi theo system message do tính ngẫu nhiên; chỉ dùng cho nghiên cứu.
- Chất lượng base model là yếu tố then chốt.

Nhận xét:

- Bản này là "work in progress"; một nửa kết luận (95% ChatGPT) dựa trên GPT-4-làm-giám-khảo, chính là phương pháp mà paper chỉ trích.
- Không công bố ablation tách riêng tác dụng của explanation (so với phản hồi ngắn trên cùng câu hỏi), nên không biết mức đóng góp riêng của "explanation tuning" so với "quy mô + đa dạng của FLAN".
- Câu hỏi FLAN-v2 có trùng lĩnh vực với benchmark (BBH, AGIEval) ở mức nào thì không được phân tích; riêng Big-Bench thì tác giả loại khỏi T0 train.
- Giả định "phản hồi dài = khó hơn" chưa được kiểm chứng; bài 39 sẽ cho thấy điều này có thể sai với student nhỏ.
- Chi phí lớn: hàng triệu lời gọi API đóng, hạn chế khả năng tái lập.

7. Liên hệ với các bài khác

- **08 Teacher assistant (Mirzadeh et al.):** nguồn cảm hứng trực tiếp cho vai trò ChatGPT; trong 08 TA là model trung gian về kích thước, ở đây là teacher yếu hơn nhưng rẻ hơn, và student học tuần tự.
- **41 Capacity gap law, 42 Distillation scaling laws:** khung lý thuyết cho câu hỏi "teacher quá mạnh có hại không".
- **35 Self-Instruct:** thay instruction tự sinh (đơn giản) bằng FLAN-v2 (đa dạng, do người thiết kế) + system message.
- **36 Distilling step-by-step:** cùng niềm tin "giải thích giúp student"; Orca bỏ multi-task, chỉ dùng chuỗi dài.
- **15 Sequence-level KD:** cùng dạng học từ chuỗi do teacher sinh, nhưng teacher là API đóng.
- **38, 39:** reasoning trace dài (R1); 39 chỉ ra student nhỏ học trace dài/khó kém, điều có thể làm cần xem lại giả định "dài hơn = tốt hơn" của Orca.
- **28 MiniLLM, 29 GKD:** nhóm white-box có logit; Orca giải thích rõ vì sao với LFM đóng buộc dùng text.

8. Tóm tắt công thức / khái niệm

Khái niệm	Ý nghĩa
Explanation tuning	SFT trên phản hồi có giải thích (system message kích)
Bộ ba mẫu	$\langle$system, query, response$\rangle$

Khái niệm	Ý nghĩa
16 system message	Điều khiển độ dài/kiểu giải thích
FLAN-5M / FLAN-1M	5M phản hồi ChatGPT / 1M phản hồi GPT-4
Progressive learning	Train ChatGPT-data trước, GPT-4-data sau
Loss	Cross-entropy chỉ trên token phản hồi của teacher
Packing	Ghép ~2.7 mẫu vào chuỗi 2,048 token
Kết quả nhanh	BBH 49.7 (ChatGPT 48.9, Vicuna 23.3); AGIEval 41.7 (ChatGPT 47.2)

9. Kết luận

Orca huấn luyện student 13B trên trace giải thích đa dạng, với tiến trình từ ChatGPT đến GPT-4. Bài báo đánh giá thêm bằng các benchmark lập luận thay vì chỉ dùng GPT-4 làm giám khảo. Các ablation chưa tách bạch đóng góp của từng thành phần.

Link gốc: https://arxiv.org/abs/2306.02707

Bài 38

DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning (2025) — phần distillation

Tác giả: DeepSeek-AI (Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Peiyi Wang, Qihao Zhu, Runxin Xu, Ruoyu Zhang, Shirong Ma, Xiao Bi, Xiaokang Zhang, Xingkai Yu, Yu Wu, Z.F. Wu, Zhibin Gou, Zhihong Shao, Zhuoshu Li, Ziyi Gao... và nhiều người khác). Mục "Author Contributions" ghi Zhibin Gou dẫn dắt loạt R1-distill.
Venue / arXiv: arXiv:2501.12948 (bản v2 ngày 4/1/2026 là bản ta đọc; paper text không nêu tên hội nghị/tạp chí).
Nguồn: https://arxiv.org/abs/2501.12948
Chủ đề: Technical report dài (86 trang). Ghi chú này chỉ giải thích ngắn gọn R1/R1-Zero, rồi **tập trung vào Phụ lục F "DeepSeek-R1 Distillation"**: distill R1 vào các model dày đặc nhỏ và so sánh "distillation vs RL trên model nhỏ". Các phần safety, benchmark tổng quát, thất bại (PRM, MCTS) chỉ nhắc rất ngắn.
Vị trí trong lộ trình: Đọc sau 35-37 (distillation bằng dữ liệu teacher sinh) và 15 Sequence-level KD. Bài này là ví dụ quy mô công nghiệp, dữ liệu là **long chain-of-thought** đã lọc theo tính đúng. Tiếp theo: 39 Small models struggle with strong reasoners (phản biện: không phải student nhỏ nào cũng hưởng lợi), 40 Gemma 2, 44 Qwen3 (distillation trong quy trình hậu huấn luyện).

> Ghi chú học tập bằng tiếng Việt, diễn giải lại, không phải bản dịch. Một số số liệu nằm trong bảng của phụ lục, chép nguyên từ paper.

1. Bối cảnh

- Lập luận của LLM thường dựa trên **chain-of-thought (CoT)** và dữ liệu lập luận do người viết, vừa đắt vừa giới hạn mô hình ở "cách nghĩ của người".
- Gloss: **Reinforcement learning (RL)** = học bằng thưởng/phạt thay vì bắt chước nhãn. **Verifiable reward** = phần thưởng kiểm tra được bằng luật (đáp án toán đúng, code qua test), không cần mô hình chấm. **SFT** = supervised fine-tuning, huấn luyện bắt chước. **Distillation** (ở đây) = SFT student trên văn bản do teacher sinh.
- Câu hỏi của bài báo: nếu RL thuần (không SFT trước) có thể khiến một base model tự phát triển lập luận dài, thì khả năng này có "chuyển" được xuống model nhỏ không, và bằng cách nào rẻ hơn?

2. Ý tưởng chính

R1-Zero: lấy DeepSeek-V3-Base, huấn luyện RL thuần bằng GRPO với thưởng theo luật (đúng/sai + đúng định dạng thẻ `<think>`), model tự học viết lời giải dài với tự kiểm tra và quay lại ("aha moment"). R1: thêm pipeline nhiều giai đoạn (cold-start, RL, rejection sampling + SFT, RL lần hai) để dễ đọc và hữu ích hơn. Phần distillation: dùng R1 sinh khoảng 800K mẫu (600K lập luận + 200K không lập luận), rồi **chỉ SFT** (không RL) các base model Qwen và Llama từ 1.5B đến 70B. Phát hiện chính: SFT trên lời giải của R1 cho model nhỏ kết quả vượt xa việc tự chạy RL cùng cỡ ở thí nghiệm của họ (Qwen-32B).

3. Phương pháp chi tiết

3.1 R1-Zero và R1 (tóm tắt ngắn)

GRPO (Group Relative Policy Optimization): với mỗi câu hỏi q, lấy G đầu ra $\{o_i\}$ từ policy cũ, mục tiêu:

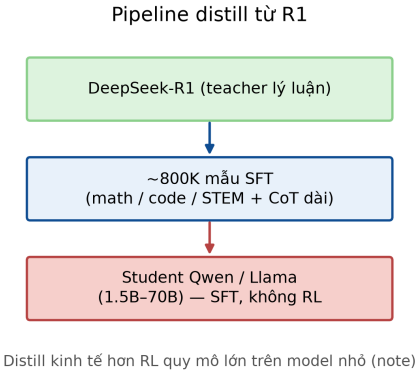
$$\mathcal{J}_{GRPO}(\theta) = \mathbb{E}\Big[\frac{1}{G}\sum_{i=1}^{G}\min\Big(\frac{\pi_\theta(o_i|q)}{\pi_{\theta_{old}}(o_i|q)}A_i,\ \text{clip}(\frac{\pi_\theta(o_i|q)}{\pi_{\theta_{old}}(o_i|q)}, 1-\varepsilon, 1+\varepsilon)A_i\Big) - \beta\,\mathbb{D}_{KL}(\pi_\theta\|\pi_{ref})\Big]$$

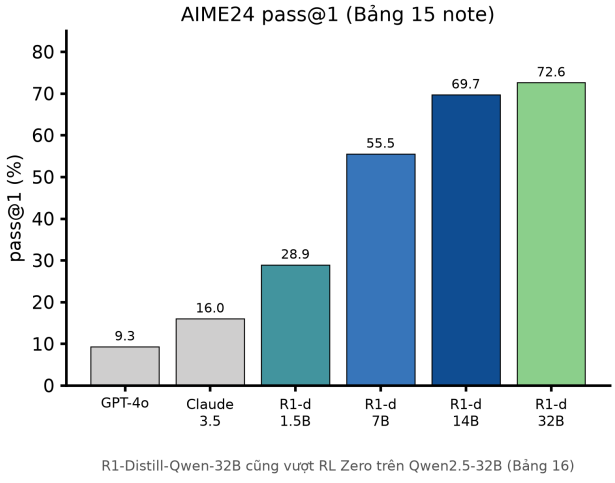
$$\boxed{A_i = \frac{r_i - \text{mean}(\{r_1,\ldots,r_G\})}{\text{std}(\{r_1,\ldots,r_G\})}}$$

Điểm mấu chốt: advantage tính **tương đối trong nhóm**, không cần value network như PPO. Thưởng R1-Zero: $\text{Reward}_{rule} = \text{Reward}_{acc} + \text{Reward}_{format}$, không dùng reward model thần kinh cho tác vụ lập luận vì dễ bị reward hacking. Cài đặt đã nêu: lr 3e-6, hệ số KL 0.001, nhiệt độ rollout 1, mỗi câu 16 mẫu, độ dài tối đa 32,768 token rồi 65,536 sau bước 8.2k, tổng 10,400 bước; AIME 2024 pass@1 từ 15.6% lên 77.9% (86.7% với cons@16 theo paper). Chi tiết thêm của R1 (giai đoạn Dev1-3, thưởng nhất quán ngôn ngữ, reward model có ích/an toàn) xem paper; không phải trọng tâm ở đây.

3.2 Dữ liệu distillation (~800K mẫu, Phụ lục B.3.3)

- **Reasoning data (~600K):** sinh bằng rejection sampling từ checkpoint sau RL giai đoạn 1. Với mỗi prompt lấy nhiều đáp án, **chỉ giữ đáp án đúng**. Đánh giá đúng bằng luật, hoặc (với phần mở rộng) bằng DeepSeek-V3 làm giám khảo sinh. **Lọc** CoT trộn ngôn ngữ, đoạn quá dài, khối code.
- **Non-reasoning data (~200K):** viết lách, hỏi đáp sự kiện, tự nhận thức, dịch; lấy lại từ SFT data của DeepSeek-V3, thêm dữ liệu software engineering (sửa lỗi, front-end). Với một số tác vụ cho V3 sinh CoT trước; câu đơn giản như "hello" thì không có CoT.





Distill R1: ~800K CoT dài → SFT student; AIME24 pass@1 Bảng 15 (1.5B–32B vs GPT-4o/Claude) (minh hoạ, tự tính)

Thống kê SFT (Table 5):

Domain	Số mẫu	Vòng TB	Token TB
Math	395,285	1.0	6,094.2
Code	211,129	1.1	7,435.7
STEM	10,124	1.0	4,928.8
Logic	10,395	1.0	2,739.0
General	177,812	1.1	1,419.8
Tổng	804,745	1.0	5,355.3

Hầu hết là một lượt hội thoại.

3.3 Huấn luyện student

Chỉ SFT (cross-entropy trên phản hồi, như 35-37). 2-3 epoch trên 800K mẫu, lịch lr cosine giảm tới 1/10 giá trị đầu, ngữ cảnh tối đa 32,768 token, batch 64. Base và lr đầu (Table 6):

Model distill	Base	lr đầu
R1-Distill-Qwen-1.5B	Qwen2.5-Math-1.5B	1e-4
R1-Distill-Qwen-7B	Qwen2.5-Math-7B	8e-5
R1-Distill-Qwen-14B	Qwen2.5-14B	7e-5
R1-Distill-Qwen-32B	Qwen2.5-32B	6e-5
R1-Distill-Llama-8B	Llama-3.1-8B	5e-5
R1-Distill-Llama-70B	Llama-3.3-70B-Instruct	2e-5

(Quy tắc rút ra: model càng nhỏ, lr đầu càng lớn.) Không có RL cho student; paper nói RL có thể tăng thêm nhưng để cộng đồng khám phá.

3.4 Pseudo-code

```
# Teacher: R1 (sau RL giai đoạn 1)
for q in reasoning_prompts:
    answers = [R1.sample(q) for _ in range(K)]
    keep    = [a for a in answers if is_correct(a, q) and readable(a)]
    D_reason.extend(keep)             # ~600K tổng
D = D_reason + D_nonreason(~200K, từ V3)      # ~800K
for base in [Qwen2.5-Math-1.5B, ..., Llama-3.3-70B-Instruct]:
    student = SFT(base, D, epochs=2..3, lr=table6[base], max_len=32768, bs=64)
# KHÔNG có giai đoạn RL
```

4. Thí nghiệm và kết quả chính

4.1 R1 và R1-Zero (chỉ vài số, từ Table 3/8)

R1: AIME 2024 pass@1 79.8; MATH-500 97.3; GPQA Diamond 71.5; LiveCodeBench 65.9; Codeforces rating 2029. R1-Zero: AIME 77.9, MATH-500 95.9, GPQA 75.8. Chi phí huấn luyện ước tính (rent H800 \$2/giờ GPU): R1-Zero 101K giờ (\$202K), tạo SFT data 5K giờ $(10K), R141Kgiờ$(82K), tổng 147K giờ (\$294K). Model chính: MoE 671B tổng, 37B kích hoạt/token.

4.2 Model distill so với baseline (Table 15, đánh giá với nhiệt độ 0.6, top-p 0.95, tối đa 32,768 token)

Model	AIME24 pass@1	AIME24 cons@64	MATH-500	GPQA-D	LCB	CF rating
GPT-4o-0513	9.3	13.4	74.6	49.9	32.9	759
Claude-3.5-Sonnet-1022	16.0	26.7	78.3	65.0	38.9	717
R1-Distill-Qwen-1.5B	28.9	52.7	83.9	33.8	16.9	954
R1-Distill-Qwen-7B	55.5	83.3	92.8	49.1	37.6	1189
R1-Distill-Qwen-14B	69.7	80.0	93.9	59.1	53.1	1481
R1-Distill-Qwen-32B	72.6	83.3	94.3	62.1	57.2	1691
R1-Distill-Llama-8B	50.4	80.0	89.1	49.0	39.6	1205

Model	AIME24 pass@1	AIME24 cons@64	MATH-500	GPQA-D	LCB	CF rating
R1-Distill-Llama-70B	70.0	86.7	94.5	65.2	57.5	1633

Paper nhấn mạnh: bản 1.5B đã vượt GPT-4o và Claude-3.5-Sonnet trên các benchmark toán (AIME, MATH-500) nhưng **không** vượt trên GPQA Diamond và LiveCodeBench; hiệu năng tăng dần theo kích thước student. (Phần "vượt ... closed-source" của bài báo chỉ đúng cho toán, đây là đọc kỹ bảng.)

4.3 Distillation vs RL trên model nhỏ (Table 16)

Thí nghiệm: RL quy mô lớn (hơn 10K bước, dữ liệu math, code, STEM) trên Qwen2.5-32B-Base, thu được **Qwen2.5-32B-Zero** (lr 2e-6, KL 0.001, nhiệt độ 1, 16 mẫu/câu, tối đa 32,768 token, batch 512).

Model	AIME24 pass@1	AIME24 cons@64	MATH-500	GPQA-D	LCB
QwQ-32B-Preview	50.0	60.0	90.6	54.5	41.9
Qwen2.5-32B-Zero (RL)	47.0	60.0	91.6	55.0	40.2
R1-Distill-Qwen-32B (SFT từ R1)	**72.6**	**83.3**	**94.3**	**62.1**	**57.2**

Kết luận của tác giả (hai ý): (1) distill từ model mạnh hơn vào model nhỏ cho kết quả rất tốt, còn model nhỏ tự RL quy mô lớn "cần năng lực tính toán khổng lồ và có thể không đạt kết quả của distillation"; (2) distillation kinh tế và hiệu quả, nhưng vượt khỏi ranh giới trí tuệ hiện có có thể vẫn cần base model mạnh hơn và RL quy mô lớn hơn.

Thí nghiệm bổ sung (Table 17): **Qwen2-Math-7B-Zero** (RL ~10,000 bước policy gradient trên base ra mắt trước o1 để đảm bảo base chưa thấy dữ liệu lập luận): AIME 2024 22.3%, AIME 2025 18.1%, so với Qwen2-Math-7B-Instruct 7.9% / 4.6%, GPT-4o-0513 9.3% / "-". Tức RL trên model 7B vẫn làm lập luận nổi lên, nhưng ở mức thấp hơn nhiều so với R1-Distill-Qwen-7B (AIME 55.5, Table 15; lưu ý hai số này không cùng base, Qwen2-Math-7B vs Qwen2.5-Math-7B, nên so sánh chỉ mang tính tham khảo - nhận xét bổ sung).

5. Phân tích / ablation

- **Tầm quan trọng của base model (Phụ lục G.1):** RL từ base trên model 7B dense và MoE 16B "không đem lại cải thiện có ý nghĩa" trên AIME; chúng lặp lại khi độ dài tăng và không tận dụng được long CoT. Chỉ khi chuyển sang 32B dense, 230B MoE, 671B MoE mới thấy tăng đáng kể. Họ kết luận hiệu quả RL từ base phụ thuộc nhiều vào năng lực nền. (Lưu ý đối chiếu: ở Table 17, Qwen2-Math-7B-Zero lại tăng được, nên "7B không RL được" là phát biểu về cấu hình thử ban đầu của họ, không tuyệt đối.)
- **Verifier:** thành công của R1-Zero rất phụ thuộc phần thưởng đáng tin; luật và LLM-judge cho đáp án ngắn hoạt động, nhưng không tổng quát cho viết lách mở.
- **Cả SFT và RL đều cần:** RL khám phá được lập luận mà dữ liệu người không có; SFT cần cho tác vụ khó định nghĩa thưởng.
- **Lọc dữ liệu distillation:** giữ đáp án đúng, bỏ ngôn ngữ trộn, đoạn quá dài, khối code; bài báo không có ablation riêng về từng bước lọc.
- **Chất lượng teacher > dữ liệu người:** paper khẳng định "model distill từ output chất lượng cao của teacher luôn vượt model train trực tiếp trên dữ liệu người" (trích dẫn Busbridge et al., 2025); trong phần ta đọc không có bảng nào trực tiếp so sánh.

6. Giới hạn

Do tác giả nêu:

- Student chỉ dùng SFT, không RL (có thể còn nâng được).
- Distillation không thể "đẩy ranh giới": cần base mạnh hơn + RL lớn hơn.
- Dữ liệu SFT chủ yếu một lượt, hạn chế hội thoại nhiều lượt.
- R1: overthinking với câu đơn giản, nhạy với prompt (few-shot làm giảm hiệu năng), trộn ngôn ngữ, yếu ở tác vụ software engineering.

Nhận xét:

- Thí nghiệm "distillation vs RL" chỉ một cấu hình chính (Qwen2.5-32B), không công bố ngân sách tính toán đồng đều: RL >10K bước so với SFT 800K mẫu có chi phí rất khác, nên "distill thắng" chưa phải so sánh công bằng theo FLOPs. Chi phí sinh 800K mẫu từ R1 671B cũng không được tính vào bên distillation (chỉ có 5K giờ GPU tạo dữ liệu SFT).
- Student 1.5B, 7B dùng base Qwen2.5-**Math**: chuyên toán sẵn, nên lợi ích trên toán có thể phản ánh một phần nền tảng này.
- Không có thí nghiệm student nhỏ đa dạng kích thước cho cùng kỹ thuật RL (ngoài Qwen2-Math-7B); không đo token/độ trễ khi suy luận (long CoT trung bình 5,355 token/mẫu).
- Các số bảng 15 trên model nhỏ ở GPQA và LiveCodeBench vẫn thấp hơn GPT-4o/Claude cho 1.5B, nên cần đọc "vượt closed-source" cẩn thận.

7. Liên hệ với các bài khác

- **35 Self-Instruct, 37 Orca:** cùng họ "teacher sinh văn bản, student SFT". Khác biệt: R1 dùng **lọc theo đúng-sai có kiểm chứng**, và trace là long CoT tự phát sinh bằng RL, không phải prompt kiểu "think step by step".
- **36 Distilling step-by-step:** cũng thêm lời giải của teacher, nhưng dùng multi-task và rationale ngắn; ở R1 là một chuỗi dài làm đích duy nhất.
- **39 Small models struggle with strong reasoners:** phản biện/giới hạn: student nhỏ (<=3B) học từ CoT dài của teacher mạnh có thể kém hơn học CoT ngắn; ở đây R1-Distill-1.5B vẫn lên mạnh nhưng ở toán; cần đọc cùng nhau.
- **41 Capacity gap law, 42 Distillation scaling laws:** khung để giải thích vì sao student nhỏ và teacher quá mạnh có thể không khớp.
- **44 Qwen3 distillation, 40 Gemma 2 distillation:** hai cách dùng distillation trong quy trình huấn luyện thực tế (on-policy/ logit-based) so với SFT thuần ở đây.
- **15 Sequence-level KD:** R1-Distill là sequence-level KD ở quy mô lớn, mẫu dùng hard sampling (đã lọc).
- **28-34 (MiniLLM, GKD, DistiLLM...):** nhóm logit-based, đối lập với phương pháp không dùng logit ở đây.

8. Tóm tắt công thức / khái niệm

Khái niệm	Ý nghĩa
GRPO	RL với advantage chuẩn hoá trong nhóm G mẫu, không cần value net
$A_i = (r_i - \text{mean})/\text{std}$	Advantage nhóm
Rule-based reward	Thưởng đúng đáp án + đúng định dạng
R1-Zero	RL thuần trên base, không SFT
800K SFT data	~600K lập luận (rejection sampling, chỉ đúng) + ~200K khác
R1-Distill-*	SFT-only student: Qwen 1.5B/7B/14B/32B, Llama 8B/70B

Khái niệm	Ý nghĩa
Qwen2.5-32B-Zero	RL >10K bước trên Qwen2.5-32B-Base (đối chứng)
Số then chốt	AIME24 pass@1: Zero 47.0 vs Distill-32B 72.6

9. Kết luận

Trong thí nghiệm DeepSeek, SFT mô hình nhỏ bằng lời giải R1 đã lọc vượt RL trực tiếp trên mô hình cùng cỡ; Qwen-32B đạt 72.6 so với 47.0 trên AIME. Kết quả gắn với toán và code có verifier; ngân sách tính toán giữa các phương án chưa được kiểm soát đồng đều.

Link gốc: https://arxiv.org/abs/2501.12948

Bài 39

Small Models Struggle to Learn from Strong Reasoners (2025)

Tác giả: Yuetai Li, Xiang Yue, Zhangchen Xu, Fengqing Jiang, Luyao Niu, Bill Yuchen Lin, Bhaskar Ramasubramanian, Radha Poovendran.
Venue / arXiv: arXiv:2502.12143 (bản v3, 13/11/2025; paper text không nêu tên hội nghị).
Nguồn: https://arxiv.org/abs/2502.12143
Chủ đề: Distillation lập luận (chain-of-thought) vào model nhỏ: phát hiện "Small Model Learnability Gap" và đề xuất **Mix Distillation**.
Vị trí trong lộ trình: Đọc sau 37 Orca và 38 DeepSeek-R1 distillation (cả hai tin rằng trace dài/teacher mạnh giúp student). Bài này **phản biện có điều kiện**: với student <= 3B, trace dài và teacher lớn không luôn tốt. Nối với 08 Teacher assistant, 09 Efficacy of KD (capacity gap ở phân loại), và dẫn sang 41 Capacity gap law, 42 Distillation scaling laws (khung định lượng).

> Ghi chú học tập bằng tiếng Việt, diễn giải lại, không phải bản dịch. Phạm vi paper chỉ là toán học (MATH và các benchmark toán).

1. Bối cảnh

- Sau Orca (37) và DeepSeek-R1 (38), thói quen phổ biến là: lấy teacher mạnh, sinh lời giải (càng dài, càng "suy nghĩ" càng tốt), SFT student nhỏ. Kỳ vọng ngầm: teacher mạnh hơn thì student tốt hơn.
- Gloss: **Short CoT** = lời giải ngắn gọn đi thẳng tới đáp án. **Long CoT** = chuỗi dài có phản tư, quay lui, thử hướng khác (kiểu QwQ, R1). **Learnability** = mức mà một student nhất định có thể học được từ một kiểu dữ liệu. **Capacity gap** (ở phân loại ảnh, 08/09) = chênh lệch quá lớn giữa teacher và student làm distillation kém đi; bài này khảo sát hiện tượng tương tự ở bài toán sinh lập luận.
- Trước đó đã có công trình (Kim et al. 2024b, được paper trích) cho thấy teacher mạnh không nhất thiết là teacher tốt; bài này quy trách nhiệm chính cho **kích thước student**.

2. Ý tưởng chính

Thí nghiệm có kiểm soát trên 10 student (Qwen2.5 0.5B-32B, Llama 1B-70B) và nhiều cặp teacher: student **nhỏ (<= 3B)** học tốt hơn từ short CoT và từ teacher nhỏ; student **lớn (>= 7B)** hưởng lợi từ long CoT và teacher lớn. Paper gọi đây là **Small Model Learnability Gap**. Gợi ý nguyên nhân: sai khác phân phối giữa student và dữ liệu teacher (student gán perplexity cao cho trace dài/teacher lớn), và tri thức miền hạn chế của model nhỏ. Cách khắc phục đơn giản: **Mix Distillation**, trộn dữ liệu dễ (short/teacher nhỏ) với dữ liệu khó (long/teacher lớn) theo tỷ lệ nhỏ, với tỷ lệ 20% dữ liệu khó (1:4, xem 3.4) cho kết quả tốt hơn dùng riêng từng loại.

3. Phương pháp chi tiết

3.1 Ký hiệu và SFT

Student π_θ học trên $\mathcal{D} = \{(x_i, y^i)\}_{i=1}^N$ với y^i là short CoT, long CoT, CoT của teacher lớn hoặc nhỏ, bằng NLL chuẩn:

$$\mathcal{L}(\theta) = -\frac{1}{N}\sum_i \sum_t \log \pi_\theta(y_t^i \mid x_i, y_{<t}^i)$$

3.2 Hai khoảng cách đo hiện tượng

Với $P_{\text{Long}}, P_{\text{Short}}$ là điểm trung bình (5 benchmark) của student SFT trên long/short CoT:

$$\boxed{\Delta_{\text{Long}} = P_{\text{Long}} - P_{\text{Short}}} \qquad \boxed{\Delta_{\text{Large}} = P_{\text{Large}} - P_{\text{Small}}}$$

$\Delta < 0$: dữ liệu "mạnh" tệ hơn dữ liệu "yếu" cho student đó. "Learnability Gap" xảy ra khi $\Delta < 0$ ở student nhỏ và $\Delta > 0$ ở student lớn.

3.3 Cài đặt

- **Dữ liệu:** 7,500 prompt của MATH.
- **Teacher:** long CoT từ QwQ-32B-Preview; short CoT từ Qwen2.5-32B-Instruct. Cặp lớn/nhỏ trong cùng họ: Qwen2.5-72B vs 3B-Instruct, Llama3.1-70B vs 8B-Instruct, Gemma2-27B vs 9B-it. Thêm DeepSeek-R1-32B (long CoT) làm baseline ở phần Mix.
- **Sinh dữ liệu:** rejection sampling (giữ lời giải đúng), mặc định greedy. Để so teacher lớn/nhỏ công bằng, chỉ giữ các cặp (bài, lời giải) mà **cả hai** teacher đều đúng; độ dài trung bình gần như bằng nhau (432.98 token với teacher 72B, 440.70 với 3B), loại độ dài làm biến gây nhiễu.
- **Huấn luyện:** LLaMA-Factory. Student < 14B: SFT toàn bộ tham số, lr tối đa 1e-5 cosine, 2 epoch. Student > 14B: LoRA lr 1e-4, 2 epoch. Bốn A100 80GB.
- **Đánh giá:** MATH, GSM8K, AMC 2023, AIME 2024, OlympiadBench (phần toán tiếng Anh); zero-shot, greedy, tối đa 16k token; điểm student là trung bình 5 benchmark.

3.4 Mix Distillation

- **Mix-Long:** trộn long CoT (trọng số α) và short CoT ($1-\alpha$).
- **Mix-Large:** trộn CoT của teacher lớn (α) và teacher nhỏ ($1-\alpha$).
- Chọn $\alpha = 0.2$ (Figure 6 cho thấy tại 0.2 điểm trung bình cao nhất với Qwen2.5-3B-Instruct). Trong chú thích Table 3 viết "tỷ lệ 1:4", tức 1 phần long : 4 phần short, khớp với $\alpha = 0.2$.

$$\mathcal{D}_{\text{mix}} = \alpha\,\mathcal{D}_{\text{hard}} \cup (1-\alpha)\,\mathcal{D}_{\text{easy}}, \qquad \alpha = 0.2$$

Pseudo-code:

```
D_hard = teacher_mạnh_sinh(MATH7.5k)    # long CoT hoặc teacher lớn
D_easy = teacher_yếu_sinh(MATH7.5k)     # short CoT hoặc teacher nhỏ
D_mix  = sample(D_hard, 0.2*N) + sample(D_easy, 0.8*N)
student = SFT(student, D_mix)
```

4. Thí nghiệm và kết quả chính

4.1 Long CoT Gap (Table 1, điểm trung bình 5 benchmark)

Student	P_Long	P_Short	Δ_Long	Tốt hơn
Qwen2.5-0.5B	14.8	19.5	-4.7	Short
Qwen2.5-1.5B	27.0	34.2	-7.1	Short
Qwen2.5-3B	40.3	43.4	-3.1	Short
Qwen2.5-7B	48.9	47.2	+1.7	Long
Qwen2.5-14B	59.2	54.3	+4.9	Long
Qwen2.5-32B	73.0	59.3	+13.7	Long
Llama-3.2-1B	15.8	19.5	-3.7	Short

Student	P_Long	P_Short	Δ_Long	Tốt hơn
Llama-3.2-3B	32.5	33.1	-0.6	Short
Llama-3.1-8B	35.2	31.5	+3.7	Long
Llama-3.3-70B	58.2	54.3	+3.8	Long

Điểm chuyển nằm giữa 3B và 7B (Qwen) và giữa 3B và 8B (Llama). Một ví dụ cụ thể ở văn bản: Qwen2.5-1.5B-Instruct thấp hơn hơn 10 điểm trên MATH và AMC khi học long CoT.

4.2 Large Teacher CoT Gap (Table 2; short CoT, teacher lớn Qwen2.5-72B vs nhỏ Qwen2.5-3B)

Student	P_Large	P_Small	Δ_Large	Tốt hơn
Qwen2.5-0.5B	16.9	20.4	-3.5	Weak
Qwen2.5-1.5B	32.2	33.0	-0.8	Weak
Qwen2.5-3B	39.7	39.4	+0.3	Strong
Qwen2.5-7B	48.9	42.3	+6.6	Strong
Qwen2.5-14B	52.9	49.9	+3.0	Strong
Qwen2.5-32B	59.5	53.0	+6.5	Strong
Llama-3.2-1B	16.5	18.5	-1.9	Weak
Llama-3.2-3B	32.8	31.2	+1.6	Strong
Llama-3.2-8B (ghi trong bảng, thực chất Llama-3.1-8B)	25.6	25.1	+0.5	Strong
Llama-3.2-70B (ghi trong bảng)	57.6	53.3	+4.3	Strong

Lưu ý: nhãn “Llama-3.2-8B/70B” là cách ghi trong bảng của bài báo; mục 3.1 liệt kê Llama3.1-8B và Llama3.3-70B. Hiệu ứng ở mức 3B khá mờ (Δ gần 0), và cột “Strong/Weak” do tác giả gán theo dấu Δ.

4.3 Mix Distillation (Table 3, student Qwen2.5-3B-Instruct, MATH 7.5k)

Phương pháp dữ liệu	MATH	AMC	GSM8K	Olympiad	AIME	Trung bình
Long CoT	56.2	37.5	80.0	24.4	3.3	40.3
Short CoT	61.0	37.5	82.0	26.4	10.0	43.4
Strong model CoT	57.5	35.0	80.0	25.9	0.0	39.7
Weak model CoT	60.3	27.5	79.5	26.4	3.3	39.4
DeepSeek-R1-32B (long CoT)	50.7	20.0	81.2	15.7	0.0	33.5
Mix-Long	64.7	45.0	81.4	28.6	10.0	**45.9**
Mix-Large	65.8	42.5	81.7	29.0	10.0	**45.8**

Với Llama3.2-3B-Instruct: Long CoT 32.5, Short CoT 33.1, Strong 32.8, Weak 31.2, R1-32B 33.3, Mix-Long 35.1, Mix-Large 34.7 (trung bình).

Con số trong văn bản: Qwen2.5-3B-Instruct tăng hơn 8 điểm trên MATH và AMC với Mix-Long so với dùng long CoT thuần; tăng hơn 7 điểm trên MATH, AIME và AMC với Mix-Large so với dùng CoT teacher lớn thuần.

5. Phân tích / ablation

- **Tri thức miền (Takeaway 3):** Qwen2.5-Math-1.5B-Instruct (model "chuyên toán" dù nhỏ) có gap nhỏ hơn Qwen2.5-1.5B tổng quát, và hưởng lợi từ teacher lớn nhiều hơn. Tác giả phỏng đoán nguyên nhân quan trọng là tri thức miền hạn chế của student nhỏ.
- **Sai khác phân phối (Takeaway 4, Appendix B.3):** đo perplexity của dữ liệu teacher dưới student: student nhỏ gán perplexity cao hơn đáng kể cho long CoT và CoT teacher lớn; cặp student-teacher "cùng cỡ" khớp nhau hơn. (Bảng PPL chi tiết nằm ở Table 10 của bài báo, ở đây không chép số.)
- **Base vs Instruct (Takeaway 5):** model base nhỏ có gap rõ hơn model Instruct.
- **Thay đổi văn phong (Takeaway 6):** dùng phép đo rank shift của token (Lin et al., 2023): token đổi nhiều nhất sau khi học long CoT/teacher lớn là từ mang tính văn phong như "wait", "But", "Let", chứ không phải nội dung toán.
- **Trọng số trộn (Figure 6):** $\alpha = 0.2$ cho điểm trung bình cao nhất (Qwen2.5-3B-Instruct).
- **Định tính (Figure 7):** model học long CoT thuần "nghĩ vòng vo" lặp lại, không biết dừng; học short thuần sai đáp án; Mix-Long có yếu tố rẽ nhánh ("Alternatively") nhưng ngắn gọn. Độ dài phản hồi trung bình: long CoT 3384.7 token, short 575.7, Mix-Long 1248.9.

6. Giới hạn

Do tác giả nêu:

- Chỉ xét toán; chưa đánh giá instruction following, code, đa phương thức.
- Chưa khảo sát ảnh hưởng của thành phần dữ liệu tiền huấn luyện.

Nhận xét:

- Chỉ SFT trên 7.5K bài MATH; kết luận có thể thay đổi với dữ liệu lớn hơn nhiều (R1 dùng 800K) hoặc với RL sau SFT.
- Nhiều hiệu ứng nhỏ (ví dụ Δ_Large của 3B chỉ +0.3, Llama-3B -0.6), không có thống kê độ tin cậy/seed; AIME chỉ 30 bài nên mỗi bài sai là ±3.3 điểm.
- Trong Table 3, DeepSeek-R1-32B (nhãn trong bài báo) là bản distill 32B, không phải R1 gốc; long CoT của nó thấp hơn cả QwQ ở student 3B, củng cố luận điểm nhưng không cô lập được "độ dài" với "phong cách teacher".
- Nguyên nhân (phân phối, tri thức miền) mới là gợi ý tương quan; chưa có thí nghiệm can thiệp (ví dụ SFT bước đầu để thu hẹp perplexity trước) để chứng minh nhân quả.
- Mix Distillation là heuristic với α chọn trên chính tập đánh giá trung bình; chưa rõ tổng quát.

7. Liên hệ với các bài khác

- **08 Teacher assistant, 09 Efficacy of KD:** cùng lập luận "teacher quá khác student thì khó học"; ở đây chuyển sang bài toán sinh chuỗi.
- **41 Capacity gap law, 42 Distillation scaling laws:** định lượng hoá vấn đề này; Mix Distillation là cách xử lý thực dụng.
- **37 Orca:** Orca dùng ChatGPT làm "teaching assistant" trước GPT-4 vì cùng lý do; paper này là bằng chứng độc lập cho trực giác đó.
- **38 DeepSeek-R1 distillation:** student R1-Distill-1.5B vẫn lên mạnh trên toán (AIME 28.9, MATH 83.9), nhưng dùng 800K mẫu và base Qwen2.5-Math chuyên toán; phù hợp với Takeaway 3 (tri thức miền giúp). Cần cẩn trọng khi so, vì thiết lập khác nhiều.
- **36 Distilling step-by-step:** rationale ngắn, task cố định; hợp với phát hiện "short CoT tốt cho student nhỏ".
- **28-34 (MiniLLM, GKD, DistiLLM, TAID...):** nhóm logit-based/on-policy xử lý khoảng cách phân phối bằng cách học trên mẫu của chính student, một lối giải quyết khác cho cùng vấn đề.

- **44 Qwen3 distillation, 40 Gemma 2:** thực hành trong quy trình sản xuất.

8. Tóm tắt công thức / khái niệm

Khái niệm	Ý nghĩa
Small Model Learnability Gap	Student <= 3B không hưởng lợi từ long CoT / teacher lớn
$\Delta_{\text{Long}} = P_{\text{Long}} - P_{\text{Short}}$	Chênh điểm khi học long so với short CoT
$\Delta_{\text{Large}} = P_{\text{Large}} - P_{\text{Small}}$	Chênh điểm khi học teacher lớn so với nhỏ
Mix-Long / Mix-Large	Trộn long+short / lớn+nhỏ với $\alpha = 0.2$
Điểm chuyển	Giữa ~3B và ~7B (Qwen), 3B và 8B (Llama)
Kết quả nổi bật	Qwen2.5-3B: Mix-Long 45.9, Mix-Large 45.8 so với Long CoT 40.3
Nguyên nhân gợi ý	Sai khác phân phối, tri thức miền, văn phong (“wait”, “But”)

9. Kết luận

Teacher mạnh hơn và lời giải dài hơn không luôn giúp student nhỏ. Với student không quá 3B, short CoT và teacher nhỏ thường hiệu quả hơn; hỗn hợp có 20% dữ liệu khó đạt kết quả cao nhất trong các cấu hình thử. Bằng chứng giới hạn ở SFT trên 7.5 nghìn bài toán.

Link gốc: https://arxiv.org/abs/2502.12143

Bài 40

Gemma 2: Improving Open Language Models at a Practical Size (2024)

Tác giả: Gemma Team (Google DeepMind) — Morgane Riviere, Shreya Pathak, Pier Giuseppe Sessa, Cassidy Hardin, Surya Bhupatiraju, Léonard Hussenot, Thomas Mesnard, Bobak Shahriari, Alexandre Ramé, Johan Ferret và nhiều tác giả khác (danh sách đầy đủ trong bài báo).
Venue / arXiv: arXiv:2408.00118 (v3, 2 Oct 2024); bài báo không nêu venue hội nghị.
Nguồn: https://arxiv.org/abs/2408.00118
Chủ đề: Technical report. Ghi chú này chỉ tập trung vào **knowledge distillation (KD) ở giai đoạn pre-training** cho mô hình 2B/9B và các ablation liên quan (Mục 3.2 và Mục 5 của bài báo). Kiến trúc, an toàn, đánh giá benchmark chỉ nhắc ngắn.
Vị trí trong lộ trình: Đọc trước: 02 (Hinton KD), 15 (Sequence-level KD), 27 (LLM KD survey), 28–29 (MiniLLM, GKD). Bài này cho thấy KD dùng ở **quy mô pre-training công nghiệp**: thay nhãn one-hot bằng phân phối của teacher trên hàng nghìn tỷ token. Tiếp theo: 41 (Capacity gap law), 42 (Distillation scaling laws) giải thích *khi nào* và *vì sao* cách làm này có lợi; 44 (Qwen3) là một công thức công nghiệp khác.

> Tài liệu học tập bằng tiếng Việt, là ghi chú diễn giải, không phải bản dịch paper. Paper là technical report nên mô tả KD rất ngắn và thiếu nhiều chi tiết (không nêu kích thước teacher thật cho 9B/2B ngoài việc mô phỏng bằng teacher 7B).

1. Bối cảnh

Vấn đề. Các mô hình nhỏ (vài tỷ tham số) ngày càng tốt chủ yếu nhờ **train lâu hơn** (nhiều token hơn). Paper nhận xét cách này chỉ cải thiện theo hàm *logarit* của kích thước dữ liệu (theo Hoffmann et al., 2022 — "Chinchilla"), và các mô hình nhỏ mới nhất cần tới 15T token để cải thiện state-of-the-art chưa tới 1–2%. Nghĩa là mô hình nhỏ vẫn "under-trained", nhưng thêm token ngày càng kém hiệu quả.

Chinchilla / compute-optimal (gloss): với ngân sách tính toán cố định, có một cặp (kích thước mô hình, số token) tối ưu; mô hình 2B "compute-optimal" chỉ cần khoảng vài chục tỷ token. Các mô hình hiện đại train *vượt xa* mức này ("over-training") vì chi phí suy luận quan trọng hơn chi phí train.

Ý tưởng thay thế. Thay vì chỉ tăng độ dài train, tăng **chất lượng thông tin mỗi bước train**. Next-token prediction chuẩn dùng nhãn one-hot (đúng một token là 1, còn lại 0). Distillation thay one-hot bằng cả phân phối xác suất của một mô hình lớn (teacher), nên mỗi token cho nhiều bit thông tin hơn ("richer gradients").

Điểm mới của Gemma 2: KD trước đây thường dùng để *rút ngắn thời gian train* mô hình nhỏ. Ở đây họ dùng KD để *train rất nhiều token* ("simulate training beyond the number of available tokens"): 2B train 2T token, 9B train 8T token, tức hơn 50× lượng token compute-optimal theo lý thuyết.

2. Ý tưởng chính

Gemma 2 dùng phân phối token do teacher tạo ra để tiền huấn luyện các student 2B và 9B trên văn bản thô; mô hình 27B được huấn luyện từ đầu. Trong ablation với student 2B và 500B token, chưng cất từ teacher 7B nâng điểm trung bình từ 60.3 lên 67.7. Đây là so sánh trong thiết lập của báo cáo, chưa tính đầy đủ chi phí teacher.

3. Phương pháp chi tiết

3.1 Hàm mục tiêu pre-training

Cho ngữ cảnh x_c (các token trước) và token tiếp theo x. Teacher cho $P_T(x \mid x_c)$, student có tham số θ cho $P_S(x \mid x_c)$. Paper viết mục tiêu là cực tiểu hóa cross-entropy giữa hai phân phối:

$$\boxed{\min_{P_S} \sum_x -P_T(x \mid x_c) \log P_S(x \mid x_c)}$$

Đây là **forward KL** (đến một hằng số không phụ thuộc student): $\mathrm{KL}(P_T \| P_S) = \sum_x P_T \log P_T - \sum_x P_T \log P_S$, số hạng đầu không phụ thuộc P_S. Khi P_T là one-hot ta quay về loss next-token thông thường. Tức SFT/pre-train chuẩn là **trường hợp đặc biệt** với teacher "cứng".

Gloss: *forward KL* khiến student phải "phủ" mọi chế độ (mode) có xác suất đáng kể của teacher (mode-covering), khác với reverse KL của MiniLLM (28) vốn mode-seeking.

Chi tiết paper **không** nêu (cần ghi nhớ khi tái hiện): nhiệt độ softmax, có cắt top-k logit hay không, teacher logit được tính online hay cache, trộn với loss one-hot hay không. Bài 45 (Sparse logit sampling) bàn đúng vấn đề lưu/cắt logit ở quy mô này.

3.2 Pseudo-code

```
for each batch of raw text tokens x_1..x_T:
    # teacher forward (frozen), cho phân phối tại mọi vị trí
    P_T[t] = softmax(teacher(x_<=t))            # kích thước |V| = 256k
    P_S[t] = softmax(student(x_<=t))
    loss   = mean_t( - sum_v P_T[t][v] * log P_S[t][v] )
    update student by gradient of loss
```

Lưu ý quy mô: từ vựng là 256,128 token, nên phân phối đầy đủ mỗi vị trí rất lớn; đây là một trở ngại thực tế của KD ở pre-training (bài báo không bàn cách xử lý).

3.3 Hai chỗ khác dùng distillation (post-training)

Trong SFT, họ làm **hai việc**: (i) behavioral cloning trên prompt thật/tổng hợp với câu trả lời chủ yếu do teacher sinh (tức sequence-level KD kiểu 15); (ii) **distillation trên phân phối của chính student** (on-policy), trích dẫn Agarwal et al. 2024 (GKD, 29) và Gu et al. 2024 (MiniLLM, 28). Paper chỉ nói vậy, không có ablation riêng cho phần này.

3.4 Dữ liệu và kiến trúc kèm theo (tóm tắt)

	2B	9B	27B
Số token train	2T	8T	13T
Cách train	distillation	distillation	từ đầu
d_{model}	2304	3584	4608
Số layer	26	42	46
Tham số non-embedding	2,024,517,888	8,324,201,984	26,047,480,320
Tham số embedding	590,118,912	917,962,752	1,180,237,824

Kiến trúc: xen kẽ local sliding-window attention (cửa sổ 4096) và global attention (span 8192) mỗi hai layer; GQA (num_groups = 2); logit soft-capping (50.0 trong attention, 30.0 ở layer cuối); RMSNorm cả pre-norm và post-norm; từ vựng 256k. Hạ tầng: 2B trên 512 chip TPUv5e, 9B trên 4096 chip TPUv4, 27B trên 6144 chip TPUv5p.

4. Thí nghiệm và kết quả chính

Ablation KD (Mục 5 paper). Teacher 7B; student 2B huấn luyện 500B token (gấp 10× token compute-optimal của mô hình 2B).

Bảng	Thiết lập	From scratch	Distilled
Table 6	2B, 500B token, điểm TB trên 3 benchmark	60.3	67.7

(Paper không liệt kê tên 3 benchmark trong chú thích bảng này.)

Table 7: perplexity trên validation set (thấp hơn là tốt), teacher cố định 7B.

Kích thước student	200M	400M	1B
From scratch	23	19	17
Distilled (7B)	21	17	15

Giải thích của bài báo: họ giữ teacher 7B và huấn luyện student nhỏ hơn để *mô phỏng tỷ lệ kích thước giữa teacher và student thật* (27B → 9B), và thấy lợi ích "vẫn còn khi scale kích thước".

Kết quả chất lượng cuối (pre-trained). Gemma 2 2B (distilled) so với Gemma 1 2B (cùng cỡ, 2T vs 3T token): MMLU 52.2 vs 42.3, GSM8K 24.3 vs 15.1, ARC-C 55.7 vs 48.5, MATH 16.0 vs 11.8. Gemma 2 9B: MMLU 71.3, GSM8K 68.6, MATH 36.6. Gemma 2 27B (không distill): MMLU 75.2, GSM8K 74.0, MATH 42.3. Paper nói mô hình mới tốt hơn bản cũ "tới 10% ở một số benchmark cho 9B" và kết luận distillation cải thiện chất lượng "kể cả khi huấn luyện cùng số token".

Post-training: Gemma 2 27B đạt Elo 1218 trên LMSYS Chatbot Arena (cao hơn Llama 3 70B, 1206), 9B đạt 1187, 2.6B đạt 1126. Đây là kết quả của *toàn bộ pipeline* (pre-train + SFT + RLHF + merging), không tách riêng được đóng góp của KD.

5. Phân tích / ablation

Bài báo cho thấy:

- Distill từ teacher lớn hơn tốt hơn huấn luyện từ đầu ở cùng số token (Table 6, 7).
- Lợi ích không biến mất khi student lớn lên (từ 200M tới 1B), ít nhất trong dải này.
- Các ablation kiến trúc khác (không phải KD): deep 9B hơn wide 9B một chút (52.0 vs 50.8, trung bình 4 benchmark); GQA gần như ngang MHA (50.8 vs 50.3); giảm sliding window từ 4096 xuống 1024 lúc suy luận chỉ làm perplexity từ 1.63 lên 1.64.
- Độ nhạy theo format prompt (độ lệch chuẩn MMLU qua 12 tổ hợp): Gemma 2 2B 2.1, 9B 0.9, 27B 1.0.

Paper KHÔNG ablation (quan sát bổ sung):

- Không so sánh các loss khác (reverse KL, JS, top-k), không thay đổi nhiệt độ.
- Không biến thiên kích thước teacher ở bước pre-training (chỉ một teacher 7B trong ablation, teacher thật cho 9B/2B không được nêu rõ trong bài báo).
- Không tách ảnh hưởng của "nhiều token hơn" khỏi "distill" ở mô hình cuối, vì Gemma 2 2B vs Gemma 1 2B còn khác kiến trúc và dữ liệu. Bằng chứng KD sạch nhất chỉ là Table 6/7 (quy mô nhỏ).
- Không có chi phí tính toán của teacher trong so sánh. Distill ở 500B token đòi hỏi forward teacher 7B trên 500B token, nên "cùng số token" không có nghĩa "cùng compute". Đây chính là điều bài 42 sẽ định lượng.

6. Giới hạn

Do tác giả nêu:

- Ở khâu kiểm tra, paper nói thử nghiệm không bao quát mọi tình huống ứng dụng; người dùng cần tự kiểm tra an toàn.
- Mô hình không multimodal, chủ yếu tiếng Anh, không tối ưu đa ngôn ngữ.

Nhận xét:

- Mô tả KD chỉ nửa trang; thiếu hyperparameter, dữ liệu teacher, chi phí.
- Ablation chỉ ở quy mô nhỏ (≤ 2B, teacher 7B) rồi ngoại suy lên 9B/27B.
- Không có đường cong theo số token hay so sánh với train-from-scratch cho 2B/9B ở token thật.
- Không có kiểm tra khi nào teacher quá lớn gây hại (capacity gap, bài 41).

7. Liên hệ với các bài khác

- **02 Hinton KD:** loss pre-training giống hệt ý tưởng "soft targets", chỉ khác là áp dụng ở mọi token của corpus thô.
- **15 Sequence-level KD, 36–37 (step-by-step, Orca):** phần SFT của Gemma 2 dùng dữ liệu do teacher sinh, là kiểu sequence-level.
- **28 MiniLLM, 29 GKD:** Gemma 2 trích hai bài này cho on-policy distillation trong SFT.
- **41 Capacity gap law:** giải thích vì sao teacher không nên quá lớn so với student; Gemma 2 chọn tỷ lệ 27B → 9B và mô phỏng bằng 7B → 200M–1B.
- **42 Distillation scaling laws:** trả lời định lượng câu hỏi "khi nào distill đáng hơn huấn luyện từ đầu" mà Gemma 2 chỉ trả lời bằng thực nghiệm đơn lẻ; cũng tính chi phí teacher.
- **43 MiniPLM:** cách khác để KD ở pre-training mà không tốn forward teacher online.
- **44 Qwen3:** cũng dùng teacher lớn nhưng tập trung vào post-training (strong-to-weak).
- **45 Sparse logit sampling:** bàn cách lưu và dùng logit teacher ở vocab lớn khi pre-train.

8. Tóm tắt công thức / khái niệm

Khái niệm	Ý nghĩa ngắn
$\min_{P_S} \sum_x -P_T \log P_S$	Cross-entropy với phân phối teacher; tương đương forward KL
Pre-training distillation	KD áp dụng trên corpus thô thay cho next-token one-hot
Over-training	Train nhiều token hơn mức compute-optimal (ở đây >50×)
Soft-capping	$z \leftarrow c \tanh(z/c)$, c=50 (attention), 30 (logit cuối)
Table 6	2B, 500B token, teacher 7B: 60.3 → 67.7
Table 7	PPL 23 → 21, 19 → 17, 17 → 15 cho 200M/400M/1B
On-policy distillation (SFT)	Distill trên mẫu do chính student sinh (GKD/MiniLLM)

9. Kết luận

Trong ablation của Gemma 2, chưng cất khi tiền huấn luyện nâng điểm trung bình student 2B từ 60.3 lên 67.7 với cùng lượng token. Phần KD của technical report chỉ có hai bảng nhỏ, chưa tính chi phí teacher hoặc khảo sát kích thước teacher tối ưu; các bài 41 và 42 bổ sung góc nhìn về giới hạn này.

Link gốc: https://arxiv.org/abs/2408.00118

Bài 41

Towards the Law of Capacity Gap in Distilling Language Models (2023)

Tác giả: Chen Zhang, Qiuchi Li, Dawei Song, Zheyu Ye, Yan Gao, Yan Hu.
Venue / arXiv: arXiv:2311.07052 (v4, 30 Jul 2025); bài báo không nêu venue hội nghị trong văn bản.
Nguồn: https://arxiv.org/abs/2311.07052
Chủ đề: "Curse of capacity gap" (teacher quá lớn làm student kém đi) được biến thành một quy luật tuyến tính: teacher tối ưu ≈ 2,5 × kích thước student. Áp dụng để distill LLaMA2-7B / LLaMA3.1-8B thành MiniMA-3B.
Vị trí trong lộ trình: Đọc trước: 08 (Teacher assistant — nơi "capacity gap" được nêu ở ảnh), 09 (Efficacy of KD), 11 (Patient and consistent teacher), 18–19 (TinyBERT, MiniLM), 40 (Gemma 2). Bài này thêm: **một công thức** chọn kích thước teacher thay vì thử nhiều teacher. Tiếp theo: 42 (Distillation scaling laws) làm lại câu hỏi này nghiêm ngặt và rộng hơn; 39 (Small models struggle with strong reasoners) là biểu hiện ở chuỗi suy luận.

> Tài liệu học tập bằng tiếng Việt, là ghi chú diễn giải, không phải bản dịch paper. Đây là quy luật **thực nghiệm** rút ra từ mô hình nhỏ (<3B) và một cách thức pruning cụ thể; bài 42 sau này đặt nghi vấn về phạm vi của kết luận kiểu "teacher tối ưu tỉ lệ tuyến tính với student".

1. Bối cảnh

Curse of capacity gap (gloss): người ta thường kỳ vọng teacher càng mạnh thì student càng tốt. Thực tế (Mirzadeh et al. 2020 ở thị giác, Zhang et al. 2023b ở LM) student của một cỡ cố định thường *tốt lên rồi xấu đi* khi teacher to dần. Lý do trực giác: student quá nhỏ không đủ sức biểu diễn phân phối phức tạp của teacher khổng lồ ("khoảng cách năng lực" quá lớn), nên teacher vừa phải lại tốt hơn. Hình 1 trong bài báo minh họa bằng GPT-2 và Pythia chưng cất trên OpenWebText, đánh giá perplexity WikiText2.

Hệ quả thực tế. Muốn biết teacher tối ưu phải chạy nhiều lần distill với các teacher cỡ khác nhau. Với LLM việc này quá tốn kém. Paper gọi là **"impossible triangle"**: ba mong muốn không thể đồng thời đạt — (1) kích thước student mong muốn, (2) teacher tối ưu cho nó, (3) chi phí tính toán nhỏ. Mục tiêu: nếu có một **quy luật** nối kích thước student với teacher tối ưu thì chi phí tìm teacher bằng 0.

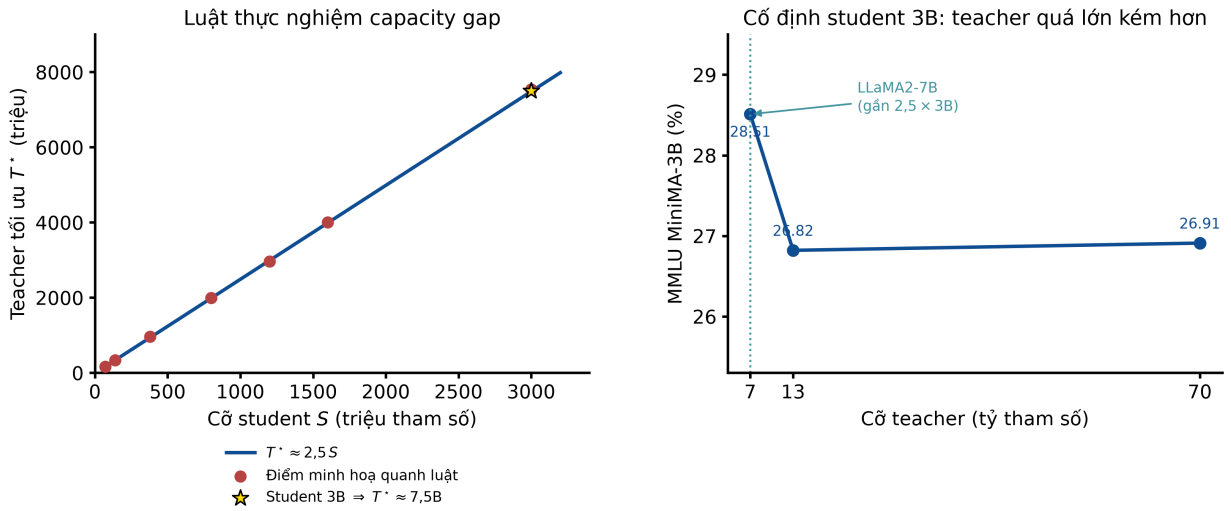
2. Ý tưởng chính

Tác giả làm một nghiên cứu sơ bộ rẻ tiền: lấy các mô hình GPT-2 (140M đến 1.6B) và Pythia (70M đến 2.8B), cắt tỉa mỗi mô hình ở nhiều mức sparsity để tạo ra student, rồi distill từ mỗi teacher trong họ. Với mỗi cỡ student, họ tìm teacher cho kết quả tốt nhất. Các điểm (cỡ student, cỡ teacher tối ưu) nằm gần như trên một đường thẳng: $T^\star \approx 2{,}5\,S$ ($R^2 = 0{,}9957$). Họ kiểm tra bằng cách ngoại suy: distill LLaMA2-7B xuống 3B (tỷ lệ 0,4) và thu được mô hình MiniMA-3B tốt hơn các baseline pruning và layer-dropping cùng ngân sách.

3. Phương pháp chi tiết

3.1 Pipeline thí nghiệm sơ bộ

Với mỗi teacher T trong họ mô hình: (1) prune T tới sparsity p để thành student S; (2) distill từ T sang S; (3) so sánh các teacher khác nhau cho cùng cỡ S, quan sát teacher nào cho student tốt nhất.



Luật thực nghiệm $T^\star \approx 2{,}5\,S$ và MMLU MiniMA-3B theo teacher 7B/13B/70B (Bảng 2 note): teacher ~7B tốt hơn teacher lớn hơn (minh hoạ, tự tính)

3.2 Pruning (structured, theo độ quan trọng)

Mỗi attention head, mỗi neuron FFN và mỗi chiều hidden được gắn biến mặt nạ ξ, ν, μ (hidden state dùng chung giữa các layer):

$$\mathrm{MHA}(x_i) = \sum_{j=1}^{A} \xi_j \,\mathrm{Attn}(\mu \cdot x_{\le i}; W_j^Q, W_j^K)\,\mu\, x_{\le i} W_j^V W_j^O, \qquad \mathrm{FFN}(x_i) = \sum_{k=1}^{I} \nu_k\, g(x_i W_k^I) W_k^O.$$

Độ quan trọng ("expressive score") = kỳ vọng của **giá trị tuyệt đối gradient** của loss theo biến mặt nạ (kiểu Molchanov 2017, gloss: "gradient bậc nhất ước lượng mất mát nếu bỏ thành phần đó"):

$$I_j^{head} = \mathbb{E}\left|\frac{\partial \mathcal{L}_{pruning}}{\partial \xi_j}\right|, \quad I_j^{neuron} = \mathbb{E}\left|\frac{\partial \mathcal{L}_{pruning}}{\partial \nu_j}\right|, \quad I^{hidden} = \mathbb{E}\left|\frac{\partial \mathcal{L}_{pruning}}{\partial \mu}\right|$$

(Công thức trong bài báo bị mất ký hiệu giá trị tuyệt đối do trích xuất PDF, nhưng văn bản mô tả là "accumulated absolute gradients".) Điểm số được chuẩn hóa ℓ_2 để tránh tỷ lệ cắt lệch giữa các layer. Các thành phần cắt riêng với cùng sparsity $p' = 1 - \sqrt{1-p}$ (vì head/neuron và hidden state có hiệu ứng nhân nhau). Chỉ ~5% dữ liệu dùng cho pruning.

3.3 Distillation loss

$$\mathcal{L}^{distill} = \frac{1}{2}\sum_i -y_i^{teacher} \log y_i^{student} \;+\; \frac{1}{2}\sum_i -y_i^{truth} \log y_i^{student}$$

Nghĩa: nửa là cross-entropy với phân phối teacher (token-level KD kiểu Hinton), nửa là CE với nhãn thật. Đây là KD token-level "cơ bản nhất"; họ không dùng loss sequence-level vì kém hiệu quả.

3.4 Quy luật

Ba giả thuyết được kiểm: **H1** capacity gap ảnh hưởng student; **H2** teacher tối ưu tồn tại trong tập hữu hạn; **H3** teacher tối ưu không đổi qua các cấu hình (cho cùng cỡ student). Quan sát H1, H2 hợp lý từ đường cong "tăng → phẳng → giảm" theo cỡ teacher; H3 đúng theo quan sát tại sparsity ~60%. Hồi quy tuyến tính:

$$\boxed{T^\star \approx 2{,}498\,S - 11{,}498 \approx 2{,}5\,S} \quad \text{(đơn vị: triệu tham số, } R^2 = 0{,}9957\text{)}$$

Corollary 1: nếu student có cỡ $0{,}4\,T^\star$ thì teacher cỡ $T^\star$ là tối ưu. Ví dụ: student 3B → teacher ≈ 7,5B, nên LLaMA2-7B là hợp lý (và 13B, 70B thì không).

3.5 Pseudo-code dùng luật

```
given: expected student size S (params)
T_star = 2.5 * S                      # luật
choose teacher closest to T_star       # vd LLaMA2-7B cho S=3B
student = structured_prune(teacher, target=S)   # width pruning, đối xứng giữa layer
distill(student <- teacher) with L = 0.5*CE(teacher)+0.5*CE(truth)
```

3.6 Chi tiết MiniMA

Teacher: LLaMA2-7B (đã mở rộng từ vựng tiếng Trung và train thêm) và LLaMA3.1-8B. Student 3B. Pruning thêm heuristic: chuyển từ ưu tiên toàn cục sang theo từng layer, bỏ 4 layer dưới cùng và 4 layer trên cùng, cắt đối xứng head/neuron/hidden. Distill: dữ liệu 126B token (Pile 57B 45,2%, GitHub 30B 23,8%, WuDao 39B 31,0%), seq 4096, batch 1024 (~4M token), lr 3e-4, weight decay 0,1, 1 epoch, 16 A100 80GB. Sau đó fine-tune thành MiniChat với 1,1M mẫu hướng dẫn.

4. Thí nghiệm và kết quả chính

Nghiên cứu sơ bộ: GPT-2 (base 140M, medium 380M, large 810M, xl 1,6B) và Pythia (70M, 160M, 410M, 1B, 1,4B, 2,8B); dữ liệu OpenWebText (~3,9B token), đánh giá perplexity WikiText2 và độ chính xác LAMBADA; có biến thể ít token hơn. Sparsity thử từ 20% đến 99%. Điểm chung: với mỗi cỡ student, hiệu năng theo cỡ teacher có dạng tăng–ổn định–giảm.

Bảng 2 (trích) — MiniMA-3B vs baseline 3B (126B token trừ khi ghi):

Mô hình	Token	MMLU	CEval	DROP	BBH	GSM8K	HumanEval
LLaMA2-7B (teacher gốc)	2T	46,00	34,40	31,57	32,02	14,10	12,80
ShortGPT 3B	0B	25,57	26,79	8,72	7,53	4,52	0,00
LayerChop 3B	126B	25,76	27,23	10,45	12,23	6,97	3,66
ShearedLLaMA 3B	126B	25,39	27,79	20,41	30,21	5,08	8,54
ShearedLLaMA 3B	252B	26,15	27,81	20,03	30,11	5,26	9,01
MiniMA (teacher LLaMA2-7B)	126B	**28,51**	28,23	22,50	31,61	8,11	10,98
MiniMA (teacher LLaMA2-13B)	126B	26,82	28,23	18,92	30,01	5,31	10,37
MiniMA (teacher LLaMA2-70B)	126B	26,91	24,89	19,03	28,55	3,56	7,32
MiniMA (teacher LLaMA3.1-8B)‡	126B	31,72	29,89	30,46	32,06	10,42	18,46

(‡ dùng corpus lớn hơn do từ vựng lớn hơn, theo chú thích bảng.) Điểm chính: 7B tốt hơn 13B và 70B làm teacher cho student 3B ("hiệu năng giảm đều khi teacher lớn hơn" theo lời paper). Tuy nhiên ngay cả MiniMA-3B tốt nhất (MMLU 28,51) còn kém xa LLaMA2-7B (46,00), tức các student này vẫn yếu nhiều trên các tác vụ khó với chỉ 126B token.

MiniChat vs đối thủ (GPT-4 chấm, Macro avg Vicuna-Bench / BELLE-Bench): vs OpenBuddy-3B 7,64:5,42 và 7,77:6,81; vs BiLLa-7B 7,24:7,41 và 7,73:7,49; vs ChatGLM2-6B 7,35:7,30 và 7,40:8,00.

5. Phân tích / ablation

Bảng 4 (MiniMA 7B→3B):

Biến thể	MMLU	CEval	DROP	BBH	GSM8K	HumanEval
MiniMA	28,51	28,23	22,50	31,61	8,11	10,98
global pruning	28,92	28,52	19,55	32,89	7,26	11,37
layer dropping	27,59	26,95	21,96	30,37	6,85	9,86
patient KD	28,69	28,37	22,51	31,89	8,00	10,64
teacher assistant (5B)	27,69	27,49	21,31	30,02	7,72	9,58

Diễn giải của bài báo: cách pruning đối xứng không hại (có khi lợi) so với global pruning và layer dropping; KD thường "không thua" patient KD nên **loss chưng cất là yếu tố thứ yếu** đối với luật; teacher assistant (chèn teacher 5B giữa) không cải thiện. Nhận xét: chênh lệch trong bảng nhỏ (thường dưới 1 điểm), không có nhiều seed hay khoảng tin cậy, nên các so sánh "tốt hơn" giữa các biến thể này không có ý nghĩa thống kê rõ ràng.

Paper cũng nói luật giữ khi đổi họ mô hình (GPT-2 vs Pythia), đổi lượng dữ liệu, đổi paradigm pruning và đổi mục tiêu distill (tham chiếu Mục 4.3), và nhận xét rằng các SLM distill gần đây (kể cả Gemma 2, bài 40) đều chọn teacher có cỡ gần với luật này. Đó là nhận xét hậu nghiệm của tác giả, không phải thí nghiệm.

6. Giới hạn

Do tác giả nêu: (1) chưa làm đầy đủ thí nghiệm cho MiniMA, trong khi LLaMA3 có mô hình tới 400B; (2) không xét lượng dữ liệu, trộn dữ liệu và kỹ thuật distill như yếu tố scaling chính; (3) không có giai đoạn preference optimization cho MiniChat.

Nhận xét:

- Luật được fit từ student **sinh ra bằng pruning chính teacher đó** (student là phiên bản cắt của teacher cùng họ), nên student thừa hưởng cấu trúc teacher. Không rõ có áp dụng cho student train từ khởi tạo ngẫu nhiên hay teacher/student khác họ.
- Dải fit là 70M đến ~3B, đoạn ngoại suy lên 7B chỉ có một điểm kiểm tra (7B tốt hơn 13B, 70B) với teacher là các LLaMA2 khác cỡ nhưng cùng họ.
- Hệ số 2,5 với $\beta = -11{,}498$ (tức đoạn cắt trục âm) khiến luật không thể đúng cho student rất nhỏ (<5M tham số cho T âm); nên coi là xấp xỉ trong dải đã đo.
- Teacher lớn hơn thua có thể vì **không train đủ lâu** (126B token cho student 3B), chứ không phải do "capacity" theo nghĩa bất biến. Bài 42 cho thấy bức tranh phụ thuộc vào ngân sách train của student và đưa ra mô hình định lượng hơn.
- Chỉ đo qua perplexity/benchmark; student MiniMA có điểm MMLU gần mức ngẫu nhiên (25) ở một số baseline nên chênh lệch dễ nhiễu.

7. Liên hệ với các bài khác

- **08 Teacher assistant (TAKD):** bài gốc nêu curse of capacity gap và đề xuất chèn teacher trung gian. Bài này không cố "chữa" mà đo quy luật; ablation cho thấy TA không giúp thêm.
- **09 / 10 (Efficacy of KD, Does KD really work):** cùng mạch "teacher mạnh hơn chưa chắc tốt hơn".
- **11 Patient and consistent teacher:** patient KD được so trong ablation; kết quả gần như hòa.

- **39 Small models struggle with strong reasoners:** cùng hiện tượng ở chuỗi suy luận dài (student nhỏ học kém từ teacher rất mạnh).
- **40 Gemma 2:** chọn teacher 27B cho student 9B (3:1), phù hợp cỡ "tỷ lệ vài lần" trong luật.
- **42 Distillation scaling laws:** làm lại câu hỏi bằng thiết kế thực nghiệm lớn hơn; giải thích khi nào teacher to hại và khi nào distill đáng giá.
- **18–19 (TinyBERT, MiniLM):** bài toán tương tự trên encoder nhỏ hơn nhiều.

Gợi ý học tập

Cố định student và ngân sách rồi khảo sát một dải teacher thay vì chỉ so hai kích thước. Lặp lại với ngân sách khác để xem vị trí teacher tốt nhất có ổn định; quy tắc trong bài là kết quả thực nghiệm trong phạm vi khảo sát.

8. Tóm tắt công thức / khái niệm

Khái niệm	Ý nghĩa
Curse of capacity gap	Teacher quá lớn so với student làm student kém đi
Impossible triangle	Cỡ student mong muốn – teacher tối ưu – chi phí nhỏ
$T^\star \approx 2{,}498S - 11{,}498$	Luật tuyến tính, R^2=0,9957 (M tham số)
Corollary 1	Student cỡ $0{,}4\,T^\star$ thì teacher $T^\star$ tối ưu
$p' = 1 - \sqrt{1-p}$	Sparsity cho từng thành phần để đạt tổng p
$\mathcal{L} = \frac{1}{2}\text{CE}(teacher) + \frac{1}{2}\text{CE}(truth)$	Loss distill dùng
MiniMA-3B	3B chưng cất từ LLaMA2-7B, 126B token

9. Kết luận

Từ thí nghiệm GPT-2 và Pythia, bài báo đề xuất quy tắc gần đúng "teacher ≈ 2,5 × student" và kiểm tra ngoại suy ở 7B → 3B. Quy tắc có thể thu hẹp phạm vi tìm kiếm teacher, nhưng bằng chứng còn hẹp và chưa giải thích rõ cơ chế. Bài 42 trình bày một mô hình khác tổng quát hơn.

Link gốc: https://arxiv.org/abs/2311.07052

Bài 42

Distillation Scaling Laws (2025)

Tác giả: Dan Busbridge, Amitis Shidani, Floris Weers, Jason Ramapuram, Etai Littwin, Russ Webb (Apple; Shidani thuộc University of Oxford, làm trong kỳ thực tập tại Apple).
Venue / arXiv: arXiv:2502.08606 (v2, 25 Jul 2025); paper ghi "Proceedings of the 42nd International Conference on Machine Learning (ICML), Vancouver, PMLR 267, 2025".
Nguồn: https://arxiv.org/abs/2502.08606
Chủ đề: Một "scaling law" cho student được distill: dự đoán cross-entropy của student từ cỡ student, số token distill và **cross-entropy của teacher**; từ đó suy ra (i) khi nào teacher lớn hơn làm hại, (ii) khi nào distillation đáng hơn train supervised, (iii) cách chia compute giữa teacher và student.
Vị trí trong lộ trình: Đọc trước: 02 (Hinton KD), 08 (Teacher assistant), 11 (Patient and consistent teacher), 40 (Gemma 2), 41 (Capacity gap law). Bài này **tổng quát hóa và định lượng** cả hai: luật tuyến tính của 41 hóa ra là trường hợp đặc biệt khi teacher là compute-optimal. Tiếp theo: 43 (MiniPLM) và 45 (Sparse logit sampling) bàn chi phí teacher; 44 (Qwen3) cho một công thức thực tế.

> Tài liệu học tập bằng tiếng Việt, là ghi chú diễn giải (paper ~41k từ gồm cả phụ lục; phần trình bày tập trung vào thân bài và các phụ lục C, D, E, F, G). Không phải bản dịch. Lưu ý quan trọng: toàn bộ thí nghiệm là **language modeling trên C4 tiếng Anh, transformer 143M–12,6B, loss = cross-entropy**, chưa kiểm tra trên VLM/OCR.

1. Bối cảnh

Scaling law (supervised) (gloss): nhiều thí nghiệm cho thấy cross-entropy L của một LM giảm theo hàm lũy thừa của số tham số N và số token D:

$$L(N, D) = E + \left(\frac{A}{N^{\alpha}} + \frac{B}{D^{\beta}}\right)^{\gamma} \qquad \text{(Eq. 1)}$$

E là "irreducible error" (nhiễu bản chất của dữ liệu), phần trong ngoặc là khả năng của mô hình mô phỏng dữ liệu. Chinchilla (Hoffmann et al. 2022) dùng nó để chọn (N, D) tối ưu cho một ngân sách tính toán (xấp xỉ 20 token mỗi tham số).

Over-training (gloss): vì chi phí suy luận lớn, người ta huấn luyện mô hình nhỏ với nhiều token hơn mức compute-optimal rất nhiều. Hiệu suất khi đó có lợi suất giảm dần.

Distillation pretraining (gloss): dùng teacher cung cấp phân phối đích cho student trong lúc pre-train (như Gemma 2, bài 40). Có công thức công nghiệp nhưng thiếu hiểu biết: kích thước teacher nên là bao nhiêu? Có lúc nào supervised tốt hơn? Teacher đắt thì tính vào đâu? Paper còn nhắc một báo cáo trái chiều (Liu et al. 2024) cho rằng distillation kém supervised pretraining, và hiện tượng capacity gap.

Mục tiêu paper: dùng thí nghiệm có kiểm soát để dự đoán cross-entropy của student distill, rồi dùng luật đó để trả lời các câu hỏi trên.

2. Ý tưởng chính

Student distill có cross-entropy trên dữ liệu thật L_S. Điều quyết định ảnh hưởng của teacher không phải là kích thước hay số token riêng lẻ, mà là **cross-entropy L_T của teacher** (kết quả của cả hai). Student giỏi lên khi L_T giảm, **cho tới một điểm**, sau đó teacher quá mạnh khiến student khó mô phỏng và L_S tăng trở lại: đó là capacity gap, được mô hình hóa bằng một "broken power law" theo L_T. Với luật này, ba kết luận chính: (1) distillation không bao giờ thắng supervised nếu cả hai có đủ nhiều compute/token; (2) distillation thắng

khi ngân sách của student không vượt ngưỡng phụ thuộc cỡ student **và** teacher đã có sẵn hoặc được dùng lại; (3) nếu phải huấn luyện teacher chỉ để distill một student, supervised nói chung tốt hơn.

3. Phương pháp chi tiết

3.1 Hàm loss distillation

Với logit teacher z_T và student z_S, nhiệt độ τ, hệ số trộn λ:

$$\mathcal{L}_{KD}(z_T, z_S) = -\tau^2 \sum_{a=1}^{V} \sigma_a\left(\tfrac{z_T}{\tau}\right) \log \sigma_a\left(\tfrac{z_S}{\tau}\right), \qquad \mathcal{L}_S = (1-\lambda)\mathcal{L}_{NTP} + \lambda \mathcal{L}_{KD} + \lambda_Z \mathcal{L}_Z$$

$\mathcal{L}_{NTP}$ là next-token prediction thường, $\mathcal{L}_Z = \|\log Z\|^2$ là Z-loss ổn định. Paper dùng **pure distillation** ($\lambda = 1$) và $\tau = 1$ trong toàn bộ thí nghiệm chính, và kiểm tra rằng lựa chọn này gần tối ưu (Mục 5).

3.2 Thiết kế thí nghiệm

- Mô hình transformer (nền Gunter et al. 2024), µP đơn giản hóa (cho phép dùng chung learning rate 0,01), seq length 4096, dữ liệu C4 English; teacher và student train trên các split khác nhau, ít lặp dữ liệu.
- Kích thước 143M đến 12,6B, token từ vài tỷ đến 512B.
- Sáu teacher "Chinchilla-optimal" ($M_T = D_T/N_T \approx 20$) cỡ $N_T \in \{546M, 975M, 1{,}82B, 2{,}72B, 4{,}82B, 7{,}75B\}$ (cùng 198M ở đầu dải), distill vào student theo các IsoFLOP profile; thêm IsoFLOP teacher (đổi N_T, D_T với compute cố định) và lưới "fixed-M teacher × fixed-M student" gồm 10 teacher × student 5 cỡ × ít nhất 4 tỷ lệ M_S.
- Tổng cộng **697 run** để fit luật distillation (551 run khi chỉ lấy $L_S \geq 2{,}3$) và 73 run cho luật supervised.

3.3 Luật distillation (Eq. 8)

Gọi $\tilde{L}_S = L(N_S, D_S)$ là cross-entropy mà student đó sẽ đạt nếu train supervised (từ Eq. 1). Luật:

$$\boxed{L_S(N_S, D_S, L_T) = L_T + \frac{1}{L_T^{c_0}} \left(1 + \left(\frac{L_T}{\tilde{L}_S\, d_1}\right)^{1/f_1}\right)^{-c_1 f_1} \left(\frac{A'}{N_S^{\alpha'}} + \frac{B'}{D_S^{\beta'}}\right)^{\gamma'}}$$

Cách đọc từng khối (diễn giải bổ sung dựa trên ba tính chất mà paper yêu cầu):

- **Sàn L_T:** student không thể tốt hơn nhiều so với "mức teacher" cộng một phần dư; một teacher ngẫu nhiên ($L_T \to \infty$) cho student ngẫu nhiên ($L_S \to L_T$).
- **Phần trong ngoặc cuối** $\left(A'/N_S^{\alpha'} + B'/D_S^{\beta'}\right)^{\gamma'}$: khả năng mô phỏng teacher của student, giảm khi tăng N_S hoặc D_S (khi $N_S, D_S \to \infty$ thì $L_S \to L_T$, tức student vô hạn mô hình hóa được mọi teacher).
- **Nhân tử $L_T^{-c_0}\,(1+\dots)^{-c_1 f_1}$:** là **broken power law** theo L_T với điểm gãy khi $L_T/\tilde{L}_S = d_1$. Hai vùng: teacher yếu hơn ngưỡng (student là người học mạnh hơn teacher) và teacher mạnh hơn ngưỡng (student là người học yếu hơn). Với $c_0 > 0$, khi L_T giảm thì số hạng phạt $L_T^{-c_0}(\cdots)$ **tăng**, gây ra capacity gap: tổng L_S có cực tiểu theo L_T ở một giá trị teacher tối ưu $L_T^*(N_S, D_S)$.

Hệ số fit (Table 6, kèm khoảng tin cậy 90% bằng bootstrap):

	Supervised	Distillation
A (A')	3355	2243
B (B')	18186	24181
E	1,220	–

	Supervised	Distillation
α (α')	0,408	0,321
β (β')	0,431	0,637
γ (γ')	0,452	0,764
c_0	–	2,549
c_1	–	522,6
f_1	–	0,090
d_1	–	1,315

Sai số dự đoán khoảng $\lesssim 1\%$ tương đối, kể cả khi ngoại suy từ mô hình yếu (cross-entropy cao) sang mô hình mạnh (Hình 5). Ghi chú của bài báo: các hệ số **đặc thù** cho kiến trúc và dữ liệu của họ, không phổ quát; dạng hàm không suy ra từ nguyên lý đầu tiên.

3.4 Hạch toán compute (Eq. 9)

$$\text{FLOPs} \approx \underbrace{3F(N_S)D_S}_{\text{huấn luyện student}} + F(N_T)\Big(\underbrace{\delta_T^{Lgt} D_S}_{\text{teacher logits}} + \underbrace{\delta_T^{Pre} 3D_T}_{\text{teacher pretrain}} \Big)$$

$F(N) \approx 2N$ FLOPs/token cho một lượt forward (paper dùng công thức chính xác hơn có số hạng attention). Bốn kịch bản (Table 2):

Kịch bản	δ^{Lgt}	δ^{Pre}	Nghĩa
Best case	0	0	Teacher miễn phí hoàn toàn (khấu hao hết)
Teacher inference	1	0	Teacher đã có, chỉ trả tiền forward logit
Teacher pretraining	0	1	Phải huấn luyện teacher nhưng lưu logit dùng lại cho nhiều student
Pretrain + inference	1	1	Train teacher rồi distill đúng một student (xấu nhất)

3.5 Pseudo-code: chọn teacher bằng luật

```
given: student size N_S, student tokens D_S, candidate teachers {(N_T, D_T)}
L_tilde = L(N_S, D_S)                 # supervised scaling law, Eq.1
for each teacher t:
    L_T = L(N_T, D_T)                 # teacher's own pretrain cross-entropy
    L_S[t] = distill_law(N_S, D_S, L_T, L_tilde)     # Eq.8
    cost[t] = flops(N_S, D_S, N_T, D_T; scenario)    # Eq.9
if distill_law(optimal t) >= L_tilde : use supervised
else choose t minimising L_S subject to cost <= budget   # SLSQP in paper
```

4. Thí nghiệm và kết quả chính

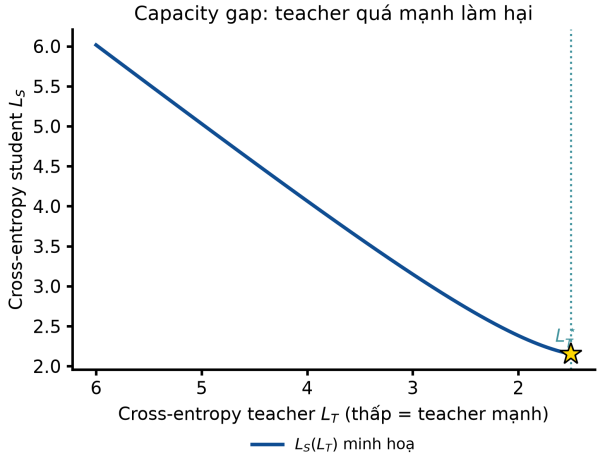
4.1 Khi nào teacher lớn hơn làm hại

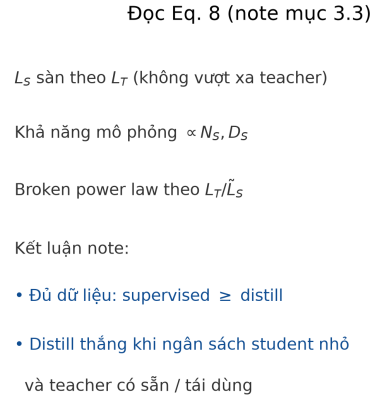
- **Hình 3 và 4:** với student cố định, vẽ L_S theo N_T (teacher compute-optimal): L_S **giảm tới cực trị rồi tăng** khi teacher to dần. Đó là capacity gap. Vẽ theo L_T thay vì N_T thì các teacher IsoFLOP khác nhau (đổi N_T, D_T) rơi vào cùng một đường, chứng tỏ **chỉ** L_T **quan trọng, không phải** N_T **hay** D_T **riêng lẻ**.
- Paper đo KL giữa teacher và student: KL **luôn tăng khi teacher mạnh hơn** (Phụ lục E.3), tức student càng khó mô phỏng. Phụ lục C chứng minh trong kernel regression và một bài MLP tổng hợp rằng khoảng cách giữa không gian giả thuyết của teacher và student gây suy giảm có thể chứng minh được.

- **Capacity gap là gap về learning capacity (không gian giả thuyết và khả năng tối ưu), không chỉ là tỷ lệ kích thước.** Luật tuyến tính của bài 41 chỉ là trường hợp đặc biệt khi các teacher là compute-optimal supervised (Hình 36a).
- Teacher tối ưu $L_T^*(N_S, D_S)$ **giảm theo dạng lũy thừa theo** N_S (teacher cần mạnh hơn khi student lớn hơn), có điểm uốn khi L_S chạm L_T^* (Mục 5.2, Hình 7). Teacher quá yếu cũng hại (student bị kẹt gần L_T).
- Calibration: teacher được kiểm là calibrated tốt (Phụ lục E.8), nên theo paper, miscalibration của teacher "không thể là nguồn duy nhất" của capacity gap.

4.2 Khi nào distillation thắng supervised

Bốn phát biểu của bài báo (tóm lược Mục 1, 5.1, 5.3):





Đường cong định tính Eq. 8: L_S có cực tiểu theo L_T (capacity gap) và các kết luận chọn supervised vs distill (minh hoạ, tự tính)

1. **Distillation không thể cho cross-entropy thấp hơn supervised nếu cả hai có đủ dữ liệu/compute.** Lý do (Phụ lục E.6, D.4.2): distillation không đổi không gian giả thuyết của student (gắn với N_S); với teacher tối ưu và dữ liệu vô hạn, L_S trùng cận dưới supervised $L(N_S, D = \infty)$ (kiểm số học cho $N, N_S \in [100M, 100B]$). Với bất kỳ teacher khác, L_S cao hơn.
2. **Với ngân sách student khiêm tốn, distillation thắng** (vùng xanh trong Hình 6). Ngưỡng compute/token mà supervised bắt đầu thắng **tăng theo kích thước student** (paper: student nhỏ nhạy với việc bị supervised vượt sớm hơn; student lớn được lợi từ distillation lâu hơn).
3. **Chi phí teacher quyết định phần còn lại:** khi teacher đã có (best case hoặc chỉ trả inference), distillation thường rẻ hơn supervised trong vùng loss "vừa phải" (Hình 33: compute tỷ lệ < 1 khi L_S hơi cao hơn cận dưới), nhưng để tạo student *mạnh nhất có thể* thì supervised lại hiệu quả hơn (tỷ lệ > 1 khi L_S tiến sát $L(N_S, \infty)$). Paper gợi ý (chưa thử): dùng distillation ở giai đoạn đầu rồi chuyển sang supervised.
4. **Nếu phải huấn luyện teacher chỉ cho một student** (pretrain + inference, hoặc pretraining tính vào chi phí) **supervised luôn hiệu quả hơn.** Nếu teacher được dùng cho nhiều student (khấu hao), hoặc dùng làm server model, distillation có thể rẻ hơn. (Một đoạn trong Phụ lục D.4.8 nói student 500M đạt loss 2,4 ở ~3/4 compute và student 10B đạt loss 2,2 ở ~1/2 compute của supervised "trong kịch bản pretrain + inference", đoạn này có vẻ mâu thuẫn với câu ngay trước nó; chưa rõ cách giải thích, nên chỉ xem như ghi chú rằng lợi ích lớn hơn với student lớn hơn.)

Phân bổ compute tối ưu khi tính cả teacher (Bảng 3 của bài báo):

Cỡ student	Compute	Chia ngân sách
nhỏ ($\lesssim$ 3B)	nhỏ ($\lesssim 10^{21}$)	chủ yếu huấn luyện teacher
nhỏ ($\lesssim$ 3B)	lớn ($\gtrsim 10^{25}$)	chia đều huấn luyện student và inference teacher, huấn luyện teacher ít hơn nhiều
lớn ($\gtrsim$ 10B)	nhỏ ($\lesssim 10^{21}$)	chủ yếu huấn luyện student chuẩn
lớn ($\gtrsim$ 10B)	lớn ($\gtrsim 10^{25}$)	chia đều huấn luyện student, inference teacher, huấn luyện teacher

Số token student tối ưu và token teacher tối ưu đều tăng theo luật lũy thừa với compute (student nhanh hơn). Cỡ teacher tối ưu tăng đến khi lớn hơn student một chút rồi **đi ngang**, vì inference teacher lớn đắt và với nhiều token student thì nên huấn luyện teacher nhỏ hơn vượt mức tối ưu tính toán.

5. Phân tích / ablation

Tất cả ở Phụ lục G (cỡ 198M–1,82B student, 546M–7,75B teacher):

Yếu tố	Kết quả
Hệ số trộn λ	L_S gần như phẳng với $\lambda > 0{,}5$; λ nhỏ chỉ tốt khi teacher yếu. Pure distillation ($\lambda = 1$) là mặc định đáng tin
Nhiệt độ τ	Tối ưu tại $\tau = 1$ ở mọi cỡ; $\tau > 3$ hại, $\tau < 1$ cũng giảm hiệu quả. Paper giả thuyết lệch với văn liệu trước vì họ không lặp token
Learning rate (µP)	Tối ưu $\eta = 0{,}01$, giống supervised; thay đổi 10× chỉ đổi loss < 0,1 nat
Cắt logit Top-k / Top-p	Hại: $k = 128$ làm loss tăng **0,11 nat** so với full; Top-p $p = 0{,}9$ tăng **0,13 nat**. Bù bằng trộn nhãn thật: $k = 128$ **và** $\lambda = 0{,}7$ giữ trong **0,01 nat**
Forward vs reverse KL	Forward KL tốt hơn trung bình **0,28 nat** (teacher 1,82B → student 546M) và rẻ hơn
Kích thước từ vựng và lưu logit	Với vocab 32.168, lưu đủ logit float32 cho ~2T token C4 khoảng 260 PB, cho thấy cần Top-k hoặc tính online
Weak-to-strong (E.7)	Student lớn hơn teacher có thể vượt teacher, nhưng chỉ trong chế độ dữ liệu distill hữu hạn; nhiều token hơn thì L_S tăng lại về L_T
Downstream (E.1)	Quan hệ cross-entropy ↔ điểm benchmark giống nhau cho supervised và distilled; MMLU/GSM8K gần ngẫu nhiên (do train C4), nên bài **không** đánh giá khả năng suy luận

Mâu thuẫn bề ngoài với "patient teacher" (bài 11): Beyer et al. thấy distill lâu là tốt; ở đây distill lâu cũng cải thiện student, nhưng supervised vẫn vượt ở ngân sách đủ lớn. Paper giải thích (D.1) rằng thiết lập của Beyer (thị giác, lặp dữ liệu nhiều, student không thấy phân phối train của teacher, không có supervised baseline tương ứng) khác với của họ.

6. Giới hạn

Do tác giả nêu (Phụ lục A):

- Chỉ language modeling, chỉ C4 English (token lớn bị lặp dữ liệu, chưa chắc phép "lặp ≤ 4 lần là vô hại" đúng cho distillation); không thể phân tích tốt reasoning.
- Chỉ distillation kiểu Hinton (teacher đưa **toàn phân phối**). Chưa nghiên cứu sequence-level KD / synthetic data (như DeepSeek-R1 distill, bài 38), loss trung gian hay các objective khác; tác giả "kỳ vọng nhưng không chắc" kết luận vẫn đúng.

- Chỉ transformer, teacher và student cùng phân phối dữ liệu (không có lệch miền).
- Hệ số đặc thù cho thiết lập; luật không suy ra từ lý thuyết.

Nhận xét:

- Student dài hơn 512B token và cỡ lớn hơn 12,6B là ngoại suy.
- "Compute" chỉ đếm FLOPs; không đếm băng thông bộ nhớ, thời gian lưu/đọc logit hay chi phí kỹ thuật.
- Teacher trong thí nghiệm luôn được huấn luyện trên cùng loại dữ liệu và cùng loại kiến trúc; kết quả có thể thay đổi nếu teacher là mô hình đa phương thức, đã qua SFT hoặc RL.
- Biến phụ thuộc là loss trên dữ liệu, không phải chất lượng sinh theo task (OCR chẳng hạn).

7. Liên hệ với các bài khác

- **02 Hinton KD:** Eq. 6 là Hinton KD áp dụng cho token; kết luận $\tau = 1$ khác với khuyến nghị nhiệt độ cao trong bài gốc cho phân loại.
- **08 Teacher assistant, 41 Capacity gap law:** cùng hiện tượng; 41 cho luật tuyến tính theo kích thước, bài này nói bản chất là gap về learning capacity và L_T mới là biến đúng. 41 là trường hợp đặc biệt khi teacher là compute-optimal.
- **11 Patient and consistent teacher:** xem mục 5.
- **40 Gemma 2:** bài này cho khung lý thuyết cho việc Gemma 2 distill 27B → 9B và chi phí teacher.
- **43 MiniPLM, 45 Sparse logit sampling:** bài này cho thấy Top-k thô hại (0,11 nat) và cần λ>0 bù; 45 đề xuất cách lấy mẫu logit tốt hơn.
- **44 Qwen3:** compute so sánh distillation với RL ở post-training, bổ sung phần pretraining của bài này.
- **28–31 (MiniLLM, GKD, DistiLLM, Rethinking KL):** forward vs reverse KL; ở đây forward KL thắng trong thiết lập pretraining.
- **38 DeepSeek-R1 distillation:** thuộc loại sequence-level KD mà bài này nói là ngoài phạm vi.

Gợi ý học tập

Khi so sánh supervised với distillation, ghi riêng chi phí huấn luyện teacher và chi phí sử dụng teacher. Thử cả trường hợp teacher được dùng lại và trường hợp chỉ phục vụ một student, vì cách hạch toán ảnh hưởng kết luận.

8. Tóm tắt công thức / khái niệm

Khái niệm	Ý nghĩa
$L(N, D) = E + (A/N^{\alpha} + B/D^{\beta})^{\gamma}$	Scaling law supervised (Eq. 1)
Eq. 8	$L_S = L_T + L_T^{-c_0}(1 + (L_T/\tilde{L}_S d_1)^{1/f_1})^{-c_1 f_1}(A'/N_S^{\alpha'} + B'/D_S^{\beta'})^{\gamma'}$
Capacity gap	Teacher quá mạnh so với learning capacity của student làm L_S tăng
L_T^*	Cross-entropy teacher tối ưu cho (N_S, D_S), giảm theo luật lũy thừa với N_S
$\delta^{Lgt}, \delta^{Pre}$	Có tính chi phí logit teacher / huấn luyện teacher vào compute hay không
Kết luận 1	Đủ dữ liệu thì supervised ≥ distillation
Kết luận 2	Distillation thắng khi ngân sách student ≤ ngưỡng theo N_S và teacher có sẵn / dùng lại
Kết luận 3	Phải huấn luyện teacher cho một student duy nhất → supervised
Cấu hình mặc định	$\lambda = 1$, $\tau = 1$, forward KL, $\eta = 0{,}01$ (µP)
Top-k	$k = 128$: +0,11 nat; kèm $\lambda = 0{,}7$: trong 0,01 nat

9. Kết luận

Luật thực nghiệm mô tả chất lượng student thông qua cross-entropy L_T của teacher, với một điểm gãy biểu diễn capacity gap. Theo phân tích của bài báo, lợi ích chưng cất phụ thuộc ngân sách student và cách tính hoặc khấu hao chi phí teacher.

Link gốc: https://arxiv.org/abs/2502.08606

Bài 43

MiniPLM: Knowledge Distillation for Pre-Training Language Models (2025)

Tác giả: Yuxian Gu, Hao Zhou, Fandong Meng, Jie Zhou, Minlie Huang (Tsinghua CoAI; WeChat AI, Tencent).
Venue / arXiv: arXiv:2410.17215 (v3, 19 Mar 2025); ICLR 2025 (theo đầu trang paper).
Nguồn: https://arxiv.org/abs/2410.17215
Chủ đề: Knowledge distillation ở giai đoạn **pre-training** nhưng làm ở mức **dữ liệu**: teacher chỉ dùng để chọn lại (re-sample) corpus, không cần chạy teacher khi huấn luyện student. Phương pháp "Difference Sampling" so sánh teacher lớn với một mô hình tham chiếu nhỏ.
Vị trí trong lộ trình: Đọc trước: 02 (Hinton KD), 15 (Sequence-level KD), 28 (MiniLLM), 40 (Gemma 2), 41–42 (capacity gap, distillation scaling laws). Bài này thêm: một phương án KD **offline, không phụ thuộc tokenizer**, và đưa ra lý lẽ rằng chi phí teacher online là điểm nghẽn. Tiếp theo: 45 (Sparse logit sampling) giải bài toán lưu logit theo hướng khác.

> Tài liệu học tập bằng tiếng Việt, là ghi chú diễn giải, không phải bản dịch paper.

1. Bối cảnh

Các cách làm KD ở fine-tuning khi đem sang pre-training đều có nhược điểm (Hình 2 của bài báo):

- **Online KD** (Vanilla KD kiểu DistilBERT/Minitron, MiniLLM): teacher phải chạy forward trong khi huấn luyện để cho phân phối từng token. Tốn compute thêm; khi tính theo FLOPs cố định thì lợi ích bị triệt tiêu (các phương pháp KD đều "tương đương hoặc tệ hơn" Pre-Train w/o KD trong thiết lập kiểm soát FLOPs). Ngoài ra teacher và student phải dùng chung tokenizer. Không thể tính sẵn rồi lưu: paper tính 50B token × từ vựng 150K × 4 byte ≈ **30 PB**.
- **Offline KD** (SeqKD, Kim & Rush 2016): teacher sinh sẵn văn bản, student học từ đó. Rẻ và độc lập tokenizer, nhưng văn bản do LM sinh dễ **dễ đoán và thiếu đa dạng** (student overfit mẫu phổ biến), như hiện tượng model collapse (Shumailov et al. 2024).

Reverse KL / mode-seeking (gloss): $\mathrm{KL}(q_\theta \| p)$ phạt student khi đặt xác suất ở nơi teacher cho xác suất thấp, nên student bám vào các "đỉnh" (mode) của teacher. MiniLLM (28) dùng loss này khi fine-tune, bằng PPO.

2. Ý tưởng chính

Thay vì bắt student khớp phân phối token của teacher, MiniPLM **sửa lại phân phối dữ liệu pre-training** theo hiểu biết của teacher: với mỗi đoạn văn trong corpus, tính độ lệch log-xác suất giữa teacher lớn và một mô hình tham chiếu nhỏ (cùng họ, train trên một tập nhỏ), rồi giữ lại những đoạn có hiệu số lớn nhất. Những đoạn mà mô hình lớn dự đoán tốt hơn nhiều so với mô hình nhỏ là kiến thức "khó nhưng có giá trị"; những đoạn cả hai đều đoán dễ (lặp, rập khuôn) bị bỏ bớt; những đoạn teacher lại đoán tệ hơn mô hình nhỏ (nhiễu) bị loại. Student sau đó pre-train như bình thường bằng cross-entropy trên corpus đã lọc. Vì chỉ cần **một con số cho mỗi đoạn văn**, teacher chạy một lần offline và tái sử dụng cho nhiều student, kể cả khác họ.

3. Phương pháp chi tiết

3.1 KD như tối đa hóa phần thưởng

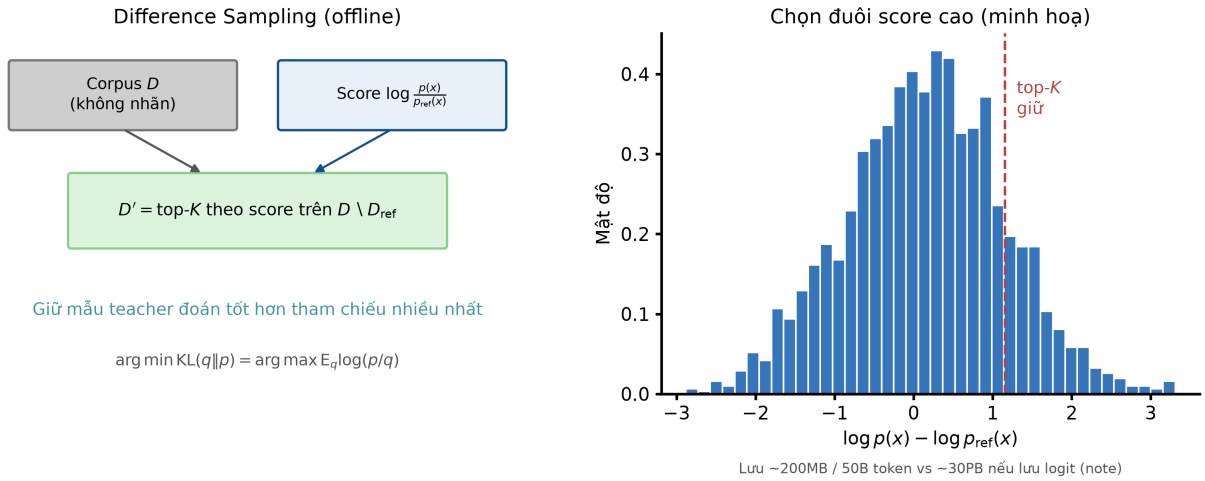
Reverse KL được viết lại thành bài toán reward:

$$\theta^\star = \arg\min_\theta \mathrm{KL}[q_\theta \| p] = \arg\max_\theta \mathbb{E}_{x\sim q_\theta}\, r(p, q_\theta, x), \qquad r(p, q_\theta, x) = \log \frac{p(x)}{q_\theta(x)}$$

Student được thưởng khi sinh văn bản x mà teacher thích (giá trị $\log p(x)$ lớn) và đa dạng (giá trị $\log q_\theta(x)$ nhỏ). Cách tối ưu đơn giản là **Best-of-N**: sinh M ứng viên từ q_θ, giữ K cái có reward cao nhất (Eq. 2), rồi train trên đó. Tuy nhiên cách này cần sinh từ student và tính reward bằng student, nên dữ liệu không dùng lại được cho student khác.

3.2 Difference Sampling

Hai thay đổi để đưa Best-of-N về dạng offline, tái sử dụng được:



MiniPLM Difference Sampling: giữ top-K mẫu theo $\log p/p_{\text{ref}}$ cho offline KD (minh hoạ, tự tính)

1. **Lấy top-K từ corpus gốc D thay vì từ mẫu sinh của student.** Mệnh đề 2.1: với $N, M \to \infty$ và hàm r đơn ánh, xác suất tập top-K của hai mẫu độc lập (từ p_1 và p_2 có hỗ trợ dương mọi nơi) trùng nhau tiến tới 1. Đây là lập luận hợp lý hóa, không phải bảo đảm cho trường hợp thực.
2. **Thay q_θ bằng mô hình tham chiếu nhỏ** p_{ref}, train trên tập con D_{ref} lấy ngẫu nhiên đồng đều từ D. Lý do: độ khó tương đối của các mẫu dữ liệu khá nhất quán giữa các mô hình (Ethayarajh et al. 2022).

Kết quả:

$$\boxed{D' = \text{top-K}\left\{ \log \frac{p(x)}{p_{ref}(x)} \;\middle|\; x \in D - D_{ref} \right\}} \qquad \text{(Eq. 4)}, \quad K = \alpha\,|D - D_{ref}|$$

Student train bằng loss pre-training thông thường trên D':

$$\mathcal{L}(q_\theta, D') = -\frac{1}{|D'|} \sum_{x\in D'} \frac{1}{|x|} \sum_{t=1}^{|x|} \log q_\theta(x_t \mid x_{<t})$$

Ba hiệu ứng (Hình 3b): (1) $\log \frac{p}{p_{ref}} \gtrsim 0 \to$ mẫu dễ và phổ biến, bị down-sample; (2) $\gg 0 \to$ mẫu khó, đa dạng, được up-sample; (3) $< 0 \to$ teacher đoán tệ hơn mô hình nhỏ $\to$ nhiễu/độc hại, bị loại.

3.3 Pseudo-code

```
# Offline, one time per teacher
D_ref  <- uniform_sample(D, small)                  # vd 5B token
```

```
p_ref  <- train_from_scratch(small_LM, D_ref)       # vd 104M tham số
for x in D - D_ref:                                  # x: một sequence 1024 token
    score[x] = logp_teacher(x) - logp_ref(x)        # sequence-level, 1 float / x
D_prime = top_K(score, K = alpha * |D - D_ref|)     # vd alpha = 0.5

# Per student (any size / tokenizer / architecture)
student = pretrain_from_scratch(student_LM, D_prime, loss=next_token_CE)
```

3.4 Ví dụ nhỏ

Giả sử ba đoạn văn có log-xác suất tổng cộng (teacher, tham chiếu): A = (−100, −102) lặp rập khuôn, hiệu số = +2; B = (−120, −170) kiến thức chuyên ngành mà mô hình nhỏ không biết, hiệu = +50; C = (−200, −150) chuỗi ký tự nhiễu mà teacher "ngạc nhiên" hơn, hiệu = −50. Difference Sampling với α đủ nhỏ giữ B, có thể giữ ít A, loại C. (Số này được đặt ra để minh họa, không lấy từ paper; trong bài báo các đoạn đều dài 1024 token nên so sánh loss trung bình tương đương so sánh tổng.)

3.5 Chi phí và lưu trữ

Mỗi sequence chỉ cần một số thực, nên 50B token với sequence 1024 chỉ cần khoảng **200MB**. Bảng 8–9: huấn luyện mô hình tham chiếu 2,6 giờ; inference teacher trên corpus 68 giờ; inference tham chiếu 7 giờ; top-K 10 phút (8×A100 80GB, thiết lập của họ). Student train mất 68 giờ cho Pre-Train và MiniPLM, còn Vanilla KD 139 giờ. Có biến thể dùng "proxy model" (Phụ lục E) hồi quy r từ hidden state của tham chiếu, chỉ cần 0,1B token đã tính nhãn.

4. Thí nghiệm và kết quả chính

Thiết lập. Kiến trúc Qwen-1.5; teacher Qwen-1.5 1,8B; student 200M, 500M, 1,2B huấn luyện từ đầu trên Pile; mỗi student tối đa 50B token (sequence 1024). Tham chiếu 104M train trên 5B token. Thiết lập chính: D gồm 105B token (5B cho D_{ref}, 100B để lấy mẫu), $\alpha = 0{,}5 \rightarrow D'$ 50B token, 1 epoch. Baseline: Pre-Train w/o KD, Vanilla KD (forward KL online), SeqKD (teacher sinh 256 token tiếp sau 768 token prompt), MiniLLM (reverse KL + PPO). Với phương pháp online, số bước train bị giảm để **tổng FLOPs train bằng nhau**. Đánh giá: zero-shot 9 tác vụ (HS, LAM, Wino, OBQA, ARC-e, ARC-c, PIQA, SIQA, Story) và test loss trên một tập con DCLM.

Bảng 1: Điểm trung bình (Avg.) trên 9 tác vụ

Student	Pre-Train w/o KD	Vanilla KD	MiniLLM	SeqKD	**MiniPLM**
200M	39,9	39,9	39,0	39,7	**41,3**
500M	43,2	43,6	41,5	42,3	**44,8**
1,2B	44,9	45,4	43,6	44,3	**46,6**

Nhận xét của bài báo: chỉ Vanilla KD hơn Pre-Train w/o KD (với student đủ lớn) trong baseline; MiniPLM tốt nhất ở hầu hết tác vụ; mức cải thiện so với Vanilla KD giữ ổn khi tăng cỡ student (Hình 1b).

Hiệu quả compute. MiniPLM đạt cùng độ chính xác với Vanilla KD (student 500M) với compute ít hơn **2,2×** (Hình 1a); lợi thế so với Pre-Train w/o KD còn rõ hơn ở 500M và 1,2B.

Loss ngoại suy (Bảng 2, theo Scaling Law của Hoffmann et al.; L_{1T}, L_{10T} là loss khi xử lý 1T và 10T token):

Student	Pre-Train w/o KD	Vanilla KD	MiniPLM
200M	3,35 / 3,32	3,39 / 3,35	3,28 / 3,26

Student	Pre-Train w/o KD	Vanilla KD	MiniPLM
500M	3,12 / 3,08	3,12 / 3,07	3,06 / 3,04
1,2B	2,98 / 2,94	2,95 / 2,91	2,92 / 2,88

(Mỗi ô là L_{1T} / L_{10T}. Đây là **ngoại suy**, không phải train thật ở 10T.)

KD khác họ mô hình (Bảng 3). Teacher/tham chiếu họ Qwen; student Llama3.1 212M và Mamba 140M. Điểm trung bình / loss DCLM: Llama3.1: Pre-Train 41,0 / 3,52, SeqKD 40,8 / 3,54, MiniPLM 41,8 / 3,43. Mamba: Pre-Train 41,6 / 3,24, SeqKD 41,0 / 3,27, MiniPLM 42,6 / 3,15. Vanilla KD và MiniLLM không áp dụng được vì khác tokenizer.

Thiết lập thiếu dữ liệu (Mục 3.4). D cố định 50B token, huấn luyện nhiều epoch trên D': $\alpha = 0{,}25$ với 4 epoch tốt nhất (so với $\alpha = 0{,}5$ ≈2 epoch và $\alpha = 0{,}125$ ≈8 epoch, bị overfit). Paper ước lượng Pre-Train w/o KD cần thêm **68B token** để bằng MiniPLM ($\alpha = 0{,}25$, 4 epoch), tức giảm nhu cầu dữ liệu **2,4×**.

5. Phân tích / ablation

- **Cỡ teacher (Hình 6, student 200M):** teacher lớn hơn không luôn tốt hơn, tương tự capacity gap (bài 41, 42). Với Vanilla KD, teacher lớn tốn thêm compute nên lợi ích giảm. Với MiniPLM, teacher **500M** tốt nhất cho student 200M; teacher 300M quá yếu để phát hiện phần khó; teacher 4B làm thang giá trị $\log p(x)$ quá nhỏ so với tham chiếu nên Difference Sampling suy biến thành lấy mẫu theo p_{ref} thuần, mất vai trò của teacher. Tác giả để ngỏ việc tối ưu cỡ teacher hay bù lệch thang đo.
- **Cỡ mô hình tham chiếu (Hình 8):** tham chiếu lớn hơn (45M, 104M, 200M) cho kết quả tốt hơn nhưng bão hòa. Tương quan Pearson/Spearman giữa $\log \frac{p}{p_{ref}}$ và $\log \frac{p}{q_\theta}$ (student 500M): 45M: 0,743/0,774, độ chính xác lấy mẫu 80,6%; 104M: 0,826/0,840, 87,4%; 200M: 0,856/0,879, 89,5%.
- **Tỷ lệ lấy mẫu α (Hình 9):** α nhỏ cho độ chính xác zero-shot cao hơn trong thiết lập dữ liệu đủ, nhưng cần corpus gốc lớn hơn; họ chọn $\alpha = 0{,}5$ để kiềm chế chi phí offline.
- **Đa dạng ngữ nghĩa (Bảng 4):** corpus gốc 32,25; teacher-generated (SeqKD) 30,16; difference-sampled 36,70. Dù xuất phát từ reverse KL (mode-seeking), corpus lọc lại đa dạng hơn.
- **Kết hợp với Vanilla KD (Bảng 5):** thêm Vanilla KD trên D' tăng thêm ở 500M (44,8 → 44,9) và rõ ở 1,2B (46,6 → 48,1), nhưng **giảm ở 200M** (41,3 → 40,7). Nghĩa là hai cách bổ sung nhau khi student đủ lớn (diễn giải của bài báo: MiniPLM chuyển tri thức "thô" mức sequence, Vanilla KD chuyển tín hiệu mức token).
- **Proxy model (Bảng 12):** MiniPLM$_{prx}$ dùng 9×10¹⁸ FLOPs offline (so với 2×10²⁰ cho MiniPLM chuẩn) vẫn đạt 40,9 so với 41,3 (MiniPLM) và 39,9 (Vanilla KD), student 200M.

6. Giới hạn

Do tác giả nêu: cần xác suất của teacher trên văn bản corpus, nên khó áp dụng cho teacher đóng (API); có thể lấy xác suất từng token qua logit bias nhưng tốn rất nhiều lệnh gọi. Hướng tương lai: tiền huấn luyện mô hình **lớn hơn** teacher (weak-to-strong).

Nhận xét:

- Quy mô nhỏ (≤1,2B student, 50B token, teacher 1,8B). Mọi con số ở 1T/10T là ngoại suy.
- Cải thiện tuyệt đối khiêm tốn (khoảng +1,4 đến +1,7 điểm trung bình so với Pre-Train, chênh lệch giữa các baseline thường dưới 1 điểm); không có nhiều seed hay khoảng tin cậy.
- Chỉ đánh giá zero-shot các tác vụ ngôn ngữ cơ bản và loss DCLM; không có suy luận, code, toán hay chuyển đổi miền.
- Chọn mẫu theo hiệu số loss có thể thiên vị: ưu tiên văn bản mà mô hình lớn "biết" hơn nhưng cũng dễ ưu tiên đoạn dài khó đoán hay nhiễu có cấu trúc lạ; paper chỉ có case study định tính (Phụ lục F).

- Teacher phải cùng loại dữ liệu và có xác suất sequence; không dùng được trực tiếp cho VLM nơi xác suất có điều kiện ảnh.

7. Liên hệ với các bài khác

- **02 / 40 (Hinton KD, Gemma 2):** Vanilla KD chính là KD kiểu Gemma 2; MiniPLM là lựa chọn khi không muốn trả chi phí teacher online.
- **15 Sequence-level KD:** SeqKD là baseline; MiniPLM thắng vì giữ phân phối dữ liệu thật và tăng độ khó/đa dạng thay vì dùng văn bản do teacher sinh.
- **28 MiniLLM:** cùng xuất phát từ reverse KL, nhưng MiniLLM tối ưu bằng PPO online; MiniPLM tối ưu bằng chọn mẫu offline. Kết quả: MiniLLM thấp hơn Pre-Train w/o KD dưới ngân sách FLOPs cố định (39,0 vs 39,9 ở 200M).
- **41 / 42 (Capacity gap, Distillation scaling laws):** Hình 6 là một minh họa khác của capacity gap, với cơ chế mới (thang giá trị log-xác suất). Bài 42 tính chi phí teacher rõ ràng; MiniPLM thuộc nhánh "teacher inference một lần, dùng cho nhiều student".
- **45 Sparse logit sampling:** giải bài toán lưu logit từng token; MiniPLM tránh bài toán đó bằng cách chỉ lưu điểm số sequence.
- **44 Qwen3:** dùng KD ở hậu huấn luyện, không phải pre-training.

8. Tóm tắt công thức / khái niệm

Khái niệm	Ý nghĩa
Online KD	Teacher chạy khi huấn luyện, tốn compute, cần cùng tokenizer
Offline KD (SeqKD)	Student học từ văn bản teacher sinh; dễ thiếu khó và đa dạng
$\arg\min \mathrm{KL}[q_\theta \Vert p] = \arg\max \mathbb{E}_{q_\theta} \log \frac{p}{q_\theta}$	KD như tối đa phần thưởng
Best-of-N	Sinh nhiều mẫu, giữ top-K theo reward
Eq. 4	$D' = \text{top-}K\{\log \frac{p(x)}{p_{ref}(x)}\}$ trên $D - D_{ref}$
Difference Sampling	Giữ mẫu mà teacher đoán tốt hơn tham chiếu nhiều nhất
Lưu trữ	~200MB cho 50B token (một số thực mỗi sequence) so với ~30PB nếu lưu logit
Kết quả chính	200M/500M/1,2B: 41,3 / 44,8 / 46,6; 2,2× ít compute hơn Vanilla KD; 2,4× ít dữ liệu hơn

9. Kết luận

MiniPLM dùng chênh lệch log-xác suất giữa teacher và mô hình tham chiếu nhỏ để chọn dữ liệu tiền huấn luyện. Điểm của từng đoạn được tính một lần và có thể dùng lại cho student khác tokenizer hoặc kiến trúc. Các kết quả hỗ trợ cách tiếp cận trong phạm vi quy mô được khảo sát.

Link gốc: https://arxiv.org/abs/2410.17215

Bài 44

Qwen3 Technical Report (2025)

Tác giả: Qwen Team (An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu và nhiều đồng tác giả; danh sách đầy đủ trong bài báo).
Venue / arXiv: arXiv:2505.09388 (technical report, ngày 14 May 2025); bài báo không nêu venue hội nghị.
Nguồn: https://arxiv.org/abs/2505.09388
Chủ đề: Technical report dài về họ mô hình Qwen3. Ghi chú này **chỉ tập trung vào Mục 4.5 "Strong-to-Weak Distillation"** (distill off-policy rồi on-policy cho mô hình nhỏ) và **phép so sánh chi phí với RL trong Mục 4.7**. Pre-training, kiến trúc, quy trình 4 giai đoạn của mô hình lớn và các benchmark chỉ tóm tắt vừa đủ làm bối cảnh.
Vị trí trong lộ trình: Đọc trước: 02 (Hinton KD), 15 (Sequence-level KD), 28–29 (MiniLLM, GKD: on-policy), 38–39 (DeepSeek-R1 distillation; Small models struggle with strong reasoners). Bài này thêm: một công thức công nghiệp "off-policy rồi on-policy" cho mô hình 0,6B–14B, và một con số so sánh chi phí với RL. Tiếp theo: 45 (Sparse logit sampling) và 47 (ShortOPD).

> Tài liệu học tập bằng tiếng Việt, là ghi chú diễn giải, không phải bản dịch. Phần distillation trong bài báo rất ngắn (một mục khoảng 10 dòng và một bảng); nhiều chi tiết (số mẫu, siêu tham số, tỷ lệ trộn, độ dài rollout) **không được công bố**.

1. Bối cảnh

Họ Qwen3 gồm 6 mô hình dense (0,6B; 1,7B; 4B; 8B; 14B; 32B) và 2 MoE (30B-A3B và 235B-A22B, tức tổng 235B tham số, 22B được kích hoạt mỗi token). Tất cả pre-train trên khoảng **36 nghìn tỷ token**, 119 ngôn ngữ, ba giai đoạn (≈30T token nền tảng; sau đó dữ liệu giàu kiến thức STEM/code; cuối cùng dữ liệu ngữ cảnh dài mở rộng 4.096 → 32.768). Tài liệu này không nói mô hình nhỏ dùng KD ở pre-training.

Thinking mode / non-thinking mode (gloss): một mô hình có hai cách trả lời, "thinking" sinh chuỗi suy luận dài trong khối `<think>...</think>` rồi mới trả lời; "non-thinking" trả lời thẳng với khối think rỗng. Người dùng chọn bằng cờ `/think` và `/no think`.

Quy trình hậu huấn luyện 4 giai đoạn cho mô hình lớn (flagship) (Hình 1 của bài báo): (1) Long-CoT cold start (SFT trên chuỗi suy luận dài đã lọc); (2) Reasoning RL bằng GRPO (3.995 cặp query-verifier; AIME'24 của Qwen3-235B-A22B tăng từ 70,1 lên 85,1 qua 170 bước RL); (3) Thinking Mode Fusion (SFT gộp dữ liệu thinking và non-thinking, nảy sinh khả năng điều khiển "thinking budget"); (4) General RL (hơn 20 loại tác vụ, ba loại reward: rule-based, model-based có đáp án tham chiếu, model-based không có tham chiếu).

Vấn đề với mô hình nhỏ. Chạy cả bốn giai đoạn cho từng mô hình nhỏ rất đắt (nhất là RL). Qwen3 dùng **strong-to-weak distillation** cho 5 mô hình dense (0,6B, 1,7B, 4B, 8B, 14B) và MoE 30B-A3B.

2. Ý tưởng chính

Các mô hình nhỏ được chưng cất qua hai pha. Ở pha off-policy, student học câu trả lời teacher đã sinh trong cả hai chế độ `/think` và `/no think`. Ở pha on-policy, student tự sinh câu trả lời và học khớp logit của teacher bằng KL. Với Qwen3-8B trên các bài toán toán và lập trình, báo cáo cho biết pha on-policy tốt hơn RL trực tiếp từ cùng checkpoint, dùng khoảng một phần mười giờ GPU và cải thiện pass@64.

3. Phương pháp chi tiết

3.1 Hai pha

Pha 1: Off-policy distillation. "Off-policy" (gloss): student học từ mẫu **do teacher sinh**, không phải do chính student sinh; đây là sequence-level KD (bài 15, 37, 38) nhưng với dữ liệu gồm cả hai chế độ `/think` và `/no think`. Mục đích theo paper: giúp student có kỹ năng suy luận cơ bản và khả năng chuyển đổi chế độ, "đặt nền" cho pha sau. Paper không nêu loss cụ thể ở pha này (có thể hiểu là SFT/cross-entropy trên văn bản teacher; đó là suy luận bổ sung).

Pha 2: On-policy distillation. "On-policy" (gloss): student **tự sinh** chuỗi trả lời, rồi nhận phản hồi từ teacher trên chính chuỗi đó, giải quyết lệch phân phối giữa khi huấn luyện (chuỗi của teacher) và lúc chạy (chuỗi của student) như GKD (29) và MiniLLM (28). Theo paper: lấy mẫu prompt; student sinh response ở chế độ `/think` hoặc `/no think`; student được fine-tune "bằng cách căn logit của nó với logit của teacher (Qwen3-32B hoặc Qwen3-235B-A22B) để tối thiểu hóa KL". Viết hình thức (suy luận, bài báo không đưa công thức):

$$\mathcal{L}_{\text{on-policy}}(\theta) = \mathbb{E}_{x\sim\mathcal{D}}\,\mathbb{E}_{y\sim q_\theta(\cdot|x)}\left[\frac{1}{|y|}\sum_{t=1}^{|y|}\mathrm{KL}(p_T(\cdot\mid x, y_{<t})\,\|\,q_\theta(\cdot\mid x, y_{<t}))\right]$$

Hướng KL (forward hay reverse) **không được nêu** trong bài báo; công thức trên chọn forward để minh họa. Với dữ liệu on-policy, mỗi token của chuỗi do student sinh được teacher "chấm" bằng cả phân phối chứ không chỉ một vô hướng thưởng như RL.

3.2 Pseudo-code

```
# Stage A: off-policy (teacher data, both modes)
D_off = { (prompt+flag, teacher_response) : flag in {/think, /no think} }
student <- SFT(student, D_off)          # basic reasoning + mode switching

# Stage B: on-policy (student samples, teacher logits)
for batch of prompts x:
    flag  ~ {/think, /no think}
    y     = student.generate(x, flag)           # on-policy sample
    p_T   = teacher.logits(x, y)                # teacher forward (Qwen3-32B or
    ↪  235B-A22B)
    q     = student.logits(x, y)
    loss  = KL(p_T || q) averaged over tokens of y
    update student
```

3.3 Điều bài báo không nói

Không có: số mẫu mỗi pha, tỷ lệ trộn think/no-think, thang mẫu, nhiệt độ lấy mẫu, độ dài tối đa của rollout, hướng KL, teacher nào cho student nào (chỉ nói 32B hoặc 235B-A22B), có xử lý tokenizer khác hay không (cùng họ Qwen3 nên cùng tokenizer, từ vựng 151.669 token), và vì sao phải làm off-policy trước. Các câu "preliminary experiments suggest" ở Mục 4 cũng không kèm bảng riêng ngoài Bảng 21.

4. Thí nghiệm và kết quả chính

4.1 So sánh với RL trực tiếp (Bảng 21)

Cả hai bắt đầu từ cùng checkpoint Qwen3-8B sau off-policy distillation; chỉ dùng truy vấn toán và code. Các số trong ngoặc là pass@64.

Phương pháp	AIME'24	AIME'25	MATH500	LiveCodeBench v5	MMLU-Redux	GPQA-Diamond	Giờ GPU
Off-policy distillation	55,0 (90,0)	42,8 (83,3)	92,4	42,0	86,4	55,6	–
+ Reinforcement Learning	67,6 (90,0)	55,5 (83,3)	94,8	52,9	86,9	61,3	17.920
+ On-policy Distillation	74,4 (93,3)	65,5 (86,7)	97,0	60,3	88,3	63,3	1.800

Đọc bảng: on-policy distillation tốt hơn RL ở mọi cột, với 1.800 giờ GPU so với 17.920 (tức khoảng 1/10). Pass@64 tăng nhờ distillation (90,0 → 93,3 và 83,3 → 86,7) nhưng đứng nguyên với RL (90,0 và 83,3). Diễn giải của bài báo: distill từ logit teacher giúp student **mở rộng không gian khám phá** và tăng tiềm năng suy luận, còn RL chủ yếu làm tăng pass@1 (độ chính xác khi chỉ thử một lần) mà không nới thêm pass@64.

4.2 Kết quả mô hình nhỏ

Paper kết luận (Mục 4.6) rằng các mô hình nhẹ (30B-A3B, 14B và các dense nhỏ hơn) "luôn vượt các mô hình mã nguồn mở có số tham số tương đương hoặc lớn hơn" và coi đó là bằng chứng thành công của strong-to-weak distillation. Ví dụ so sánh: Qwen3-4B và 8B (thinking) với DeepSeek-R1-Distill-Qwen-14B/32B; Qwen3-1.7B và 0,6B với DeepSeek-R1-Distill-Qwen-1.5B và -Llama-8B (Bảng 17 và 19). Các bảng này bị lệch cột khi trích văn bản nên không sao chép số liệu từng ô; chỉ nên tin kết luận định tính của tác giả và đọc lại bảng gốc nếu cần. Lưu ý: so sánh này là so với các mô hình khác họ, không tách riêng đóng góp của khâu distillation (không có đối chứng "Qwen3-4B train theo 4 giai đoạn đầy đủ" trong bài báo).

5. Phân tích / ablation

Bài báo cho thấy (Mục 4, 4.7):

- Distillation vs RL trên cùng điểm xuất phát: Bảng 21 ở Mục 4.1 trên.
- Tuyên bố tổng quát ở phần mở đầu Mục 4: distill trực tiếp logit teacher vào student "nâng cao hiệu năng và vẫn giữ kiểm soát tinh về quá trình suy luận", cải thiện Pass@1 và Pass@64, và chỉ cần 1/10 số giờ GPU so với quy trình bốn giai đoạn.
- Thinking Mode Fusion cho thấy khả năng điều khiển thinking budget xuất hiện tự nhiên (không được huấn luyện riêng), được minh họa bằng Qwen3-235B-A22B ở Hình 2.

Paper không ablation (nhận xét bổ sung):

- Không tách đóng góp của pha off-policy và pha on-policy (không có cột "chỉ on-policy từ checkpoint ban đầu").
- So sánh RL vs distillation chỉ cho một cỡ student (8B), một teacher chưa nêu rõ, và một họ tác vụ (toán/code); RL ở đây dùng cấu hình riêng nên chưa biết đã được tối ưu cho cùng mức nỗ lực. Một phần của lợi thế distillation có thể đến từ việc teacher 32B/235B đã được RL đầy đủ (chi phí đó **không** tính vào 1.800 giờ).
- Không có thí nghiệm về cỡ teacher (32B vs 235B) cho student cỡ nhỏ.

6. Giới hạn

Do tác giả nêu: phần Conclusion chỉ liệt kê hướng tương lai (mở rộng pre-training, kiến trúc để nén hiệu quả, ngữ cảnh cực dài, tăng compute cho RL cho agent). Paper không có mục "Limitations" riêng về distillation.

Nhận xét:

- Chi phí "1/10" **chỉ** là chi phí của student; teacher (Qwen3-32B/235B-A22B) đã được huấn luyện đầy đủ bốn giai đoạn và còn phải chạy forward trong lúc distill on-policy. Nếu chỉ có một student, phép so sánh công bằng cần cộng chi phí này (tương tự lập luận của bài 42).
- Giờ GPU là số liệu của một lần chạy trong một phân tích; không có khoảng tin cậy hay nhiều seed.
- Thiếu chi tiết nên **khó tái hiện**: không biết số mẫu, độ dài, hướng KL, tỷ lệ trộn.
- Teacher và student cùng họ, cùng tokenizer, cùng dữ liệu pre-train; kết luận chưa chắc đúng khi teacher khác họ hoặc khác miền.
- Các nhiệm vụ đánh giá là suy luận văn bản, không có thị giác hay OCR.

7. Liên hệ với các bài khác

- **15 / 37 / 38 (Sequence-level KD, Orca, DeepSeek-R1 distillation):** Pha 1 giống kiểu distill bằng văn bản teacher sinh. Khác biệt của Qwen3 là thêm pha 2 trên mẫu của student.
- **28 MiniLLM / 29 GKD / 30 DistiLLM / 47 ShortOPD:** Pha 2 là on-policy distillation (student sinh, teacher chấm). Qwen3 không viết thuật toán, nhưng về ý tưởng gần với GKD.
- **39 Small models struggle with strong reasoners:** bài này chỉ ra student nhỏ khó học từ teacher suy luận mạnh khi chỉ dùng SFT; pha off-policy rồi on-policy của Qwen3 là một cách để vượt qua, và RL-so-sánh ở Bảng 21 củng cố hướng đó.
- **40 Gemma 2:** Gemma 2 cũng dùng on-policy distillation trong SFT và KD ở pre-training; Qwen3 chỉ dùng ở hậu huấn luyện (theo paper).
- **41 / 42 (Capacity gap, Distillation scaling laws):** Qwen3 chọn teacher 32B/235B cho student 0,6B–14B mà không bàn capacity gap; bài 42 cảnh báo teacher quá lớn có thể không tối ưu.
- **45 Sparse logit sampling:** on-policy distillation cần logit teacher trên chuỗi mới sinh của student, nên không thể cache offline; liên quan đến chi phí.

8. Tóm tắt công thức / khái niệm

Khái niệm	Ý nghĩa
Strong-to-weak distillation	Dùng mô hình lớn đã hoàn thiện để huấn luyện mô hình nhỏ thay vì chạy lại 4 giai đoạn
Off-policy distillation	Student học từ câu trả lời do teacher sinh (cả `/think`, `/no think`)
On-policy distillation	Student tự sinh, teacher cho logit, tối thiểu KL
Teacher (on-policy)	Qwen3-32B hoặc Qwen3-235B-A22B
Bảng 21	On-policy KD vs RL từ cùng checkpoint 8B: AIME'24 74,4 vs 67,6; 1.800 vs 17.920 giờ GPU
pass@64	KD tăng (90,0 → 93,3; 83,3 → 86,7), RL không đổi
Thinking mode fusion	SFT hợp nhất hai chế độ, cho phép thinking budget
"1/10 GPU hours"	Tỷ lệ so với quy trình bốn giai đoạn (theo paper), chưa tính chi phí teacher

9. Kết luận

Qwen3 mô tả quy trình off-policy rồi on-policy cho các mô hình nhỏ. Trong so sánh ở cỡ 8B, báo cáo cho biết on-policy dùng khoảng một phần mười giờ GPU của RL và cải thiện pass@64. Phần chưng cất ngắn, thiếu ablation và không tính chi phí teacher, nên cần thận trọng khi khái quát.

Link gốc: https://arxiv.org/abs/2505.09388

Bài 45

Sparse Logit Sampling: Accelerating Knowledge Distillation in LLMs (2025)

Tác giả: Anshumann, Mohd Abbas Zaidi, Akhil Kedia, Jinwoo Ahn, Taehwak Kwon, Kangwook Lee, Haejun Lee, Joohyung Lee (Samsung Research, Seoul; ba tác giả đầu đóng góp ngang nhau). **Venue / arXiv:** arXiv:2503.16870 (v2, 24/07/2025); bài không nêu venue trong bản text. **Nguồn:** https://arxiv.org/abs/2503.16870 **Chủ đề:** Distillation "offline": cache logit của teacher rồi huấn luyện student. Vì sao lưu Top-K rồi chuẩn hoá lại là ước lượng **chệch**, và cách lấy mẫu ngẫu nhiên (importance sampling) để có ước lượng **không chệch** với rất ít logit lưu trữ. **Vị trí trong lộ trình:** Đọc trước: 02 (Hinton KD, soft target + KL), 15 (Sequence-level KD), 29 (GKD), 31 (Rethinking KL), 40 (Gemma 2, distillation khi pre-train). Bài này bổ sung: phân tích lý thuyết về việc **lưu thưa** phân phối teacher. Đọc tiếp: 46 (Distil-Whisper, pseudo-label + KL), 47 (ShortOPD, cũng dùng top-100 + một "bin đuôi").

Đây là ghi chú học tập bằng tiếng Việt, không phải bản dịch.

1. Bối cảnh

Distillation theo logit. Student được huấn luyện để khớp *toàn bộ phân phối* xác suất token kế tiếp của teacher (soft target), không chỉ token đúng (hard label). Mất mát chuẩn là KL thuận $\mathrm{KL}(t \,\|\, p)$ với t là xác suất teacher, p là xác suất student (xem 02).

Vấn đề chi phí. Với LLM, mỗi vị trí token có một vector xác suất cỡ $|V|$ (từ vựng, ~100k–128k). Chạy teacher *online* cùng lúc với student tốn GPU. Chạy teacher *một lần*, lưu kết quả ("offline", "cached logits") rồi tái dùng cho nhiều lần huấn luyện student rẻ hơn nhiều, nhưng lưu đủ $|V|$ số là bất khả thi: paper ước tính 128 PB cho 1T token với Llama 3, và 10 PB cho 100B token với từ vựng 100k (Bảng 5).

Giải pháp phổ biến trước đó: lưu Top-K. Chỉ giữ K xác suất lớn nhất (có thể thêm Top-p: giữ đủ khối lượng xác suất p). Các nghiên cứu trước (Raman 2023, Peng 2024) phải lưu rất nhiều logit (cỡ 6400 mỗi token) hoặc thấy hiệu năng *giảm*. Paper hỏi: tại sao Top-K đáng tin về trực giác lại kém, và có cách nào lưu chỉ ~12 token mà vẫn bằng FullKD không?

Thuật ngữ cần biết:

- **FullKD:** distillation dùng toàn bộ phân phối teacher (trần hiệu năng).
- **Calibration / ECE:** mô hình "hiệu chuẩn tốt" khi xác suất nó đưa ra khớp tần suất đúng thực tế. ECE (Expected Calibration Error) đo sai lệch này; thấp là tốt.
- **Estimator không chệch (unbiased):** kỳ vọng của ước lượng bằng đúng giá trị thật.
- **Importance sampling:** lấy mẫu từ phân phối khác q rồi nhân trọng số t/q để vẫn ước lượng đúng kỳ vọng theo t.

2. Ý tưởng chính

Lưu top-K rồi chuẩn hóa lại chia xác suất các token được giữ cho khối lượng $a < 1$, làm chúng tăng lên, đồng thời đặt phần đuôi về 0. Bài báo thay cách cắt này bằng lấy mẫu N token có hoàn lại từ phân phối teacher; tần suất c_i/N tạo thành mục tiêu thưa không chệch. Gradient kỳ vọng theo logit vì vậy bằng gradient của FullKD. Trong thiết lập khảo sát, khoảng 12 token khác nhau trên mỗi vị trí đạt hiệu năng tương đương FullKD, với chi phí huấn luyện cao hơn CE khoảng 10%.

3. Phương pháp chi tiết

3.1 Gradient của KL qua softmax

Ký hiệu logit student x, $p = \text{softmax}(x)$, mục tiêu t. Với $L = \sum_i t_i \log(t_i/p_i)$ (Phụ lục A.1):

$$\frac{\partial L}{\partial x_j} = \Big(\sum_i t_i\Big) p_j - t_j.$$

Khi t là phân phối đầy đủ ($\sum t_i = 1$): $\boxed{\partial L/\partial x_j = p_j - t_j}$. Cực tiểu đạt khi $p = t$. (Cross-entropy cho cùng gradient này, vì chỉ hơn KL một hằng số không phụ thuộc student.)

3.2 Vì sao Top-K chệch: chứng minh ngắn

Gọi K là tập token được giữ, $a = \sum_{j \in K} t_j < 1$.

Biến thể có chuẩn hoá: $t_i^s = t_i/a$ với $i \in K$, 0 ngoài K. Sai khác kỳ vọng so với t: $\mathbb{E}[t_i^s] - t_i = t_i(1/a - 1) > 0$ trong K và $-t_i$ ngoài K. Đây là ước lượng chệch. (Phụ lục A.3 còn chứng minh Top-K là cách chọn tập *giảm tối thiểu sai số L1* cho từng token: $L_1 = 2(1 - a)$, nhỏ nhất khi a lớn nhất. Nghĩa là Top-K "gần nhất" về L1 nhưng vẫn chệch.)

Biến thể không chuẩn hoá: dùng KL chỉ trên K, $L = \sum_{i \in K} t_i \log(t_i/p_i)$. Theo công thức 3.1 với $\sum_{i \in K} t_i = a$:

$$\frac{\partial L}{\partial x_j} = a\, p_j - t_j \ (j \in K), \qquad \frac{\partial L}{\partial x_j} = a\, p_j \ (j \notin K).$$

Đặt gradient bằng 0: $p_j = t_j/a$ trong K (**phóng đại** $1/a$ **lần**) và $p_j = 0$ ngoài K (đẩy đuôi về 0). Vậy dù có chuẩn hoá hay không, student hội tụ tới phân phối "nâng cấp" và quá tự tin (Phụ lục A.4). Đó là cơ chế của mis-calibration ở Bảng 1.

Ví dụ nhỏ (minh họa, không có trong bài báo). $t = (0.5, 0.3, 0.1, 0.05, 0.05)$, $K = 2$: $a = 0.8$, mục tiêu Top-2 chuẩn hoá $= (0.625, 0.375, 0, 0, 0)$. Student học tin token đầu 62.5% trong khi teacher chỉ 50%, và coi 3 token còn lại là bất khả.

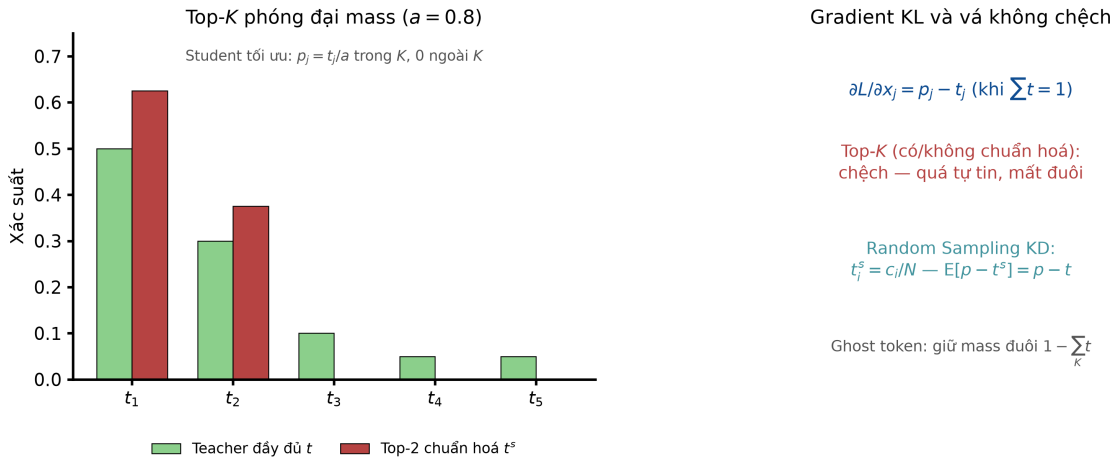
Thêm vấn đề thứ hai: **mất đuôi.** Nếu token thật nằm trong đuôi teacher, Top-K không cho tín hiệu nào, kém cả CE.

3.3 Các cách vá không đủ (Mục 3)

- **Label smoothing:** chia đều phần xác suất còn lại cho mọi lớp. Sai vì phân phối token thực gần Zipf, không đều.
- **Ghost token:** thêm 1 token ảo gom $1 - \sum_K t_i$ ở cả teacher lẫn student. Gradient trong K đúng bằng FullKD; ngoài K gradient là $p_j(\frac{1-\sum_K t}{1-\sum_K p} - 1)$ (A.5). Tốt hơn, nhưng đuôi chỉ được giám sát gián tiếp.
- **Naive fix:** dồn phần xác suất còn lại vào token ground truth. Cải thiện mạnh nhưng vẫn cần ~100 token mới sát FullKD.

3.4 Random Sampling KD

Với phân phối đề xuất $q_i \propto t_i^\tau$ (nhiệt độ lấy mẫu τ), lấy mẫu N lần có lặp; mỗi lần xuất hiện của token i nhận tỷ số hợp lý t_i/q_i; cộng lại rồi chuẩn hoá thành t^s. Với $\tau = 1$ thì $q = t$, tỷ số bằng 1:



Top-K chuẩn hoá phóng đại mass (ví dụ note a=0.8); Random Sampling KD ước lượng không chệch (minh hoạ, tự tính)

$$\boxed{t_i^s = \frac{c_i}{N}}, \qquad \mathbb{E}[t_i^s] = t_i, \qquad \mathrm{Var}[t_i^s] = \frac{t_i(1-t_i)}{N}$$

(công thức phương sai là suy ra từ phân phối nhị thức, bài báo không viết ra). c_i là số lần token i được rút trong N lần. Chỉ tối đa N phần tử khác 0, thực tế ít hơn nhiều (N=50 cho ≈12 token khác nhau, Phụ lục C; quan hệ gần power-law).

Mất mát: $L = \sum_i t_i^s \log(t_i^s/p_i)$ trên các phần tử khác 0; với $\tau = 1$ bằng tổng cross-entropy giữa các token được rút và dự đoán student. Gradient theo logit: $g_j^s = p_j - t_j^s$ nên

$$\boxed{\mathbb{E}[g_j^s] = p_j - \mathbb{E}[t_j^s] = p_j - t_j = g_j^{\text{FullKD}}}$$

(A.7: chỉ cần ước lượng không chệch là gradient được bảo toàn kỳ vọng). Phương sai quan trọng: $\tau = 0$ (lấy đều trên từ vựng) làm train phân kỳ; $q^* \propto t|f|$ là phân phối có phương sai thấp nhất theo lý thuyết, và mô phỏng số cho $\tau \in [0.8, 1.2]$ gần tối ưu. Lưu ý (minh họa): với $\tau \neq 1$ có bước chuẩn hoá nên ước lượng thuần tuý không còn chính xác không chệch; paper dùng $\tau = 1$ cho mọi thí nghiệm chính.

3.5 Pseudo-code (offline)

```
# Giai đoạn 1: chạy teacher một lần, lưu mục tiêu thưa
for batch in corpus:
    t = softmax(teacher(batch))              # [B, T, V]
    ids = multinomial(t, N, replacement=True) # [B, T, N]
    uniq, cnt = unique_with_counts(ids)       # ~12 token khác nhau / vị trí
    save(uniq (17 bit), cnt (7 bit))          # 3 byte / token; cnt/N = t^s

# Giai đoạn 2: huấn luyện student
for batch in corpus:                          # PHẢI cùng thứ tự/ngữ cảnh với giai đoạn 1
    logp = log_softmax(student(batch))
    ts   = cnt / N                            # mục tiêu thưa
    loss = -(ts * logp.gather(uniq)).sum(-1).mean()   # = KL tới hằng số
```

Chi tiết lưu trữ (Phụ lục D.1): từ vựng 100k cần 17 bit cho id, còn 7 bit cho phần xác suất. Vì $N = 50$ nên $c_i \leq 50 < 2^7$, lưu tử số chính xác. Với Top-K phải lượng tử hoá xác suất 7 bit; paper dùng "ratio encoding" trên xác suất đã sắp xếp để giảm sai số lượng tử.

4. Thí nghiệm và kết quả chính

Thiết lập: student LLaMA-style 300M (teacher 3B, 10B token = 1.5x Chinchilla-optimal) và 3B (teacher Llama-3-8B, 100B token trên Fineweb-edu); loss thuận KL; 4 task NLU 0-shot (HellaSwag, ARC-Easy, LAMBADA, PIQA) và instruction following sau SFT.

Bảng 1: Top-K thuần (3B→300M). "% CE→FullKD" là phần khoảng cách giữa CE và FullKD được thu hẹp.

Token duy nhất	LM loss	% CE→FullKD	ECE %
CE	2.81	0%	1.2
3	3.04	−395%	10.6
12	2.87	−99%	4.7
25	2.82	−21%	3.2
50	2.80	5%	2.2
100	2.77	55%	1.1
300	2.76	77%	1.5
FullKD	2.75	100%	0.7

Top-K dưới ~25 token còn **tệ hơn CE thuần**. Dùng Top-p 0.98 (K=100) chỉ đạt 47%.

Bảng 2: cách vá (300M). Label smoothing (50 tok): −73%; Ghost token (50 tok): 59%; Naive fix Top-k 5/10/20/50/100: 44/61/73/83/97% (ECE 4.4/3.3/2.3/1.3/1.2).

Bảng 3: gradient so với FullKD (300M, một batch).

Phương pháp	Góc lệch	Tỷ lệ chuẩn
Top-K 12	58°	2.4
Top-K 50	48°	1.8
Top-K 300	30°	1.3
Random Sampling (12 tok)	4°	1.0

Bảng 6 (3B→300M, 10B token). Random Sampling với 2.4 / 5.0 / 12.1 / 24.5 / 57.0 token: LM loss 2.77 / 2.75 / 2.75 / 2.75 / 2.74; FullKD 2.75; CE 2.81. ECE ≤ 1.1% mọi mức. Speculative acceptance 61.85% (12 tok) so với 62.02% (FullKD).

Bảng 8 (8B→3B, 100B token):

Phương pháp	LM loss	ECE %	Spec. accept %	0-shot	IF SFT
CE	2.37	0.3	71.1	55.6	54.5
Top-K 12	2.50	4.7	73.0	56.6	57.7
Top-K 50	2.40	1.8	73.1	57.1	58.3
Ours (12)	2.35	0.2	73.2	57.5	59.4
Ours (12)+	2.32	1.7	73.5	57.9	59.1
FullKD	2.34	0.2	73.4	57.5	58.4

LLM-as-judge (Bảng 9, 8B→3B): trung bình CE 60.2, Top-K 12 58.0, Top-K 50 60.6, **Ours 65.6**, FullKD 62.2.

Tốc độ (Bảng 4): Random Sampling 2.6x / 1.73x (300M / 3B) so với FullKD = 1.0x; CE 2.9x / 1.77x. Cache implementation nhanh hơn FullKD 1.7–2.6x, chậm hơn CE ~10%.

Lưu trữ (Bảng 5, 100B token): FullKD 10 000 TB; Top-K 300: 90 TB; **Ours (12 logit, 3 byte): 3.6 TB** (gấp 25 lần ít hơn Top-300); CE: 0.3 TB.

Khác: Qwen-style 0.5B (10B token; text nói teacher Llama-3-8B, caption Bảng 14 ghi 3B→0.5B): LM loss 2.99 (CE) → 2.95. Cải thiện so với CE tăng theo kích thước student (100M–3B, Hình 4).

5. Phân tích / ablation

- **Nhiệt độ lấy mẫu (Bảng 12):** $\tau = 0$ phân kỳ; 0.8 / 1.0 / 1.2 đều ~2.74–2.75 LM loss. Chọn 1.0.
- **Hàm mất mát (Bảng 15):** KL thuận 2.75 tốt nhất; MSE 3.37; L1 5.38; KL nghịch tách rời bị ∞.
- **Teacher adaptation:** nếu dữ liệu student khác dữ liệu tiền huấn luyện teacher, nên fine-tune teacher ngắn trước (Bảng 13: 2.99 CE; 2.98 không adapt; 2.96 có adapt).
- **Cải tiến trực giao (Bảng 10):** trộn CE (trọng số 0.1) + tăng LR cho token "khó" (tỷ lệ 2.0) cho 125% CE→FullKD (vượt FullKD), nhưng làm student **under-confident** và giúp ít hơn sau instruction tuning (caveat của tác giả).
- **So với prior work (Bảng 11):** Peng 2024: −78%/−47%; Raman 2023: 57%/5%; Ours: 100%.
- **Căn chỉnh ngữ cảnh teacher/student (Phụ lục D.3, Bảng 16):** nếu seed shuffle khác nhau thì tiền tố ngữ cảnh khác nhau sau ranh giới tài liệu; hiệu năng cache chỉ đạt 79–90% so với online; cùng seed và seq len 4096: 96%. Bài học thực hành: logit cache phải khớp *đúng* ngữ cảnh lúc student train.

6. Giới hạn

Do tác giả nêu: chỉ thử tới 3B và 100B token; chưa thử representation matching (lưu activation trung gian); chưa thử sơ đồ lấy mẫu tinh vi hơn (ví dụ thiết kế thí nghiệm tối ưu).

Nhận xét:

- Toàn bộ thử nghiệm là **pre-training ngôn ngữ**, teacher base (không instruct), được mô tả là hiệu chuẩn gần hoàn hảo. Paper nói alignment có thể làm hỏng calibration (trích dẫn công trình khác), nên với teacher đã SFT/RL kết quả chưa được kiểm chứng.
- Không có thí nghiệm trên sinh chuỗi dài tự hồi quy do student tự lấy mẫu (on-policy); cache offline vốn không sửa được exposure bias.
- Random Sampling cần phân phối **đầy đủ** lúc cache. Nếu dữ liệu cũ đã lưu Top-K thì không thể sửa lại bằng lấy mẫu.
- Chỉ so sánh trên metric loss/NLU; chưa có phân tích chi phí khi sampling nhiều vòng cho chuỗi cực dài.

7. Liên hệ với các bài khác

- **02 (Hinton KD):** nền tảng KL với soft target; bài này chỉ ra cách *nén* soft target để lưu.
- **15 (Sequence-level KD) / 46 (Distil-Whisper):** pseudo-label là mẫu hard từ teacher. Random Sampling KD có thể coi là "lấy mẫu N token tại **mỗi vị trí**" thay vì cả chuỗi.
- **31 (Rethinking KL) / 29 (GKD):** bàn về chọn divergence; bài này (Bảng 15) lại thấy KL thuận tốt nhất trong offline pre-training.
- **40 (Gemma 2):** paper nêu Gemma 3 dùng cách tương tự (lấy mẫu logit teacher theo xác suất teacher, cross-entropy trên token được lấy mẫu) ở quy mô 27B/14T token.
- **47 (ShortOPD):** dùng top-100 logit + **một bin đuôi gộp** (khác với lấy mẫu); xem phần Ứng dụng.
- **42 (Distillation scaling laws):** paper này trích dẫn công trình đó khi nói Top-K gây lệch.

Gợi ý học tập

Đo khối lượng xác suất được giữ bởi top-K và so mục tiêu đã chuẩn hóa với mục tiêu lấy mẫu trên cùng các vị trí. Khi đánh giá, phân biệt sai lệch kỳ vọng của gradient với phương sai do lấy mẫu.

8. Tóm tắt công thức / khái niệm

Khái niệm	Công thức / ý	Ghi chú
Gradient KL tại logit	$\partial L/\partial x_j = (\sum_i t_i)p_j - t_j$	$= p_j - t_j$ nếu t đầy đủ
Top-K có chuẩn hoá	$t_i^s = t_i/a,\ a = \sum_K t$	chệch, quá tự tin
Điểm tối ưu student với Top-K	$p_j = t_j/a$ (trong K), 0 (ngoài K)	từ gradient bằng 0
Ghost token	thêm token đại diện $1 - \sum_K t$	gradient trong K đúng như FullKD
Random Sampling KD	$t_i^s = c_i/N,\ q = t^\tau,\ \tau = 1$	không chệch
Bảo toàn gradient	$\mathbb{E}[p_j - t_j^s] = p_j - t_j$	điều kiện: ước lượng không chệch
Lưu trữ	17 bit id + 7 bit count = 3 byte	N=50 $\Rightarrow$ ~12 token duy nhất
ECE	sai lệch hiệu chuẩn kỳ vọng	thấp = tốt

9. Kết luận

Chuẩn hóa lại top-K tạo mục tiêu chệch và bỏ phần đuôi của phân phối teacher. Lấy mẫu N token tạo mục tiêu thưa không chệch, bảo toàn gradient kỳ vọng; trong thiết lập thử nghiệm, khoảng 12 token mỗi vị trí đạt mức FullKD, chiếm 3.6 TB cho 100B token.

Nguồn: https://arxiv.org/abs/2503.16870

Bài 46

Distil-Whisper: Robust Knowledge Distillation via Large-Scale Pseudo Labelling (2023)

Tác giả: Sanchit Gandhi, Patrick von Platen, Alexander M. Rush (Hugging Face). **Venue / arXiv:** arXiv:2311.00430 (v1, 01/11/2023); bài không nêu venue. **Nguồn:** https://arxiv.org/abs/2311.00430 **Chủ đề:** Distillation mô hình seq2seq nhận dạng tiếng nói (Whisper) bằng pseudo-label quy mô lớn, lọc theo WER, giữ nguyên encoder và cắt decoder còn 2 lớp; đánh giá robust (OOD, nhiễu, ảo giác trên audio dài); dùng làm assistant cho speculative decoding. **Vị trí trong lộ trình:** Đọc trước: 15 (Sequence-level KD, pseudo-label), 16 (DistilBERT, khởi tạo bằng cách bỏ lớp + loss KL/CE), 32 (Speculative KD). Bài này bổ sung: một công thức hoàn chỉnh cho bài toán **"transcription dài, gần tất định"**, gần nhất với OCR trong lộ trình. Đọc tiếp: 47 (ShortOPD, phục hồi mô hình bị cắt bằng on-policy distillation).

> Đây là ghi chú học tập bằng tiếng Việt, không phải bản dịch.

1. Bối cảnh

Whisper là mô hình nhận dạng tiếng nói (ASR) dạng encoder-decoder (seq2seq transformer), 1.5B tham số ở bản large-v2, huấn luyện trên 680 000 giờ dữ liệu "giám sát yếu" cào từ web. Nó khái quát tốt sang nhiều miền nhưng nặng khi cần độ trễ thấp hoặc phần cứng hạn chế.

Distillation đã thành công cho BERT (DistilBERT, TinyBERT: 16, 18) và seq2seq tóm tắt (DistilBART của Shleifer & Rush). Với ASR thì kết quả cũ kém: distill Wav2Vec 2.0 làm WER tăng 6.9%, HuBERT tăng 7.0%, LAS tăng 2.2% (theo paper). Hai thuật ngữ:

- **WER (Word Error Rate):** số từ sai (thay + chèn + xoá) chia số từ tham chiếu. Tương tự CER ở mức ký tự cho OCR.
- **Pseudo-label (PL):** dùng đầu ra sinh của teacher thay cho nhãn thật để huấn luyện student (sequence-level KD, xem 15).
- **OOD (out-of-distribution):** dữ liệu từ miền không có trong tập train; tốt trên OOD = "robust".
- **Ảo giác (hallucination) / lặp:** mô hình sinh text không có trong audio hoặc lặp một đoạn nhiều lần; xảy ra nhiều nhất ở audio dài có khoảng lặng.

Điểm khó: distilled model thường tốt trên dữ liệu cùng phân phối (ID) nhưng sụt mạnh trên OOD (Du et al. 2023, theo paper). Cần dữ liệu train đa dạng.

2. Ý tưởng chính

Distil-Whisper giữ và đóng băng encoder của Whisper, đồng thời giảm decoder từ 32 xuống hai lớp, khởi tạo từ lớp đầu và lớp cuối của teacher. Teacher sinh pseudo-label cho khoảng 22 nghìn giờ âm thanh; các mẫu có WER so với nhãn tham chiếu trên 10% bị loại. Student học bằng tổng có trọng số của KL trên logit và cross-entropy trên pseudo-label. Bài báo báo cáo mô hình nhỏ hơn 51%, nhanh hơn 5.8 lần, WER OOD trên âm thanh ngắn cao hơn một điểm và ít ảo giác hơn trên âm thanh dài.

3. Phương pháp chi tiết

3.1 Kiến trúc và khởi tạo

Teacher: encoder F_{Enc} ánh xạ $X_{1:T} \to H_{1:M}$ ($M \approx T/2$ do conv stem), decoder tự hồi quy dự đoán $P(y_i \mid y_{<i}, H_{1:M})$. Student:

- **Encoder:** sao chép nguyên từ teacher và **đóng băng**; nên chỉ chạy forward 1 lần và backprop chỉ qua decoder.
- **Decoder:** "shrink and fine-tune" theo Shleifer & Rush: copy các lớp cách đều nhau của teacher; với 2 lớp thì lấy lớp 1 và lớp 32.

Mô hình	Lớp enc	Lớp dec	Tham số (M)
distil-medium.en	24	2	394
distil-large-v2	32	2	756

3.2 Hàm mất mát

Pseudo-label: thay nhãn thật y bằng $\hat{y}_{1:N'}$ do teacher sinh (greedy):

$$L_{PL} = -\sum_{i=1}^{N'} \log P(\hat{y}_i \mid \hat{y}_{<i}, H_{1:M}).$$

(Paper viết công thức thiếu log; được sửa theo dạng chuẩn.) KL trên toàn phân phối, mỗi vị trí:

$$L_{KL} = \sum_{i=1}^{N'} \mathrm{KL}(Q_i \,\|\, P_i),$$

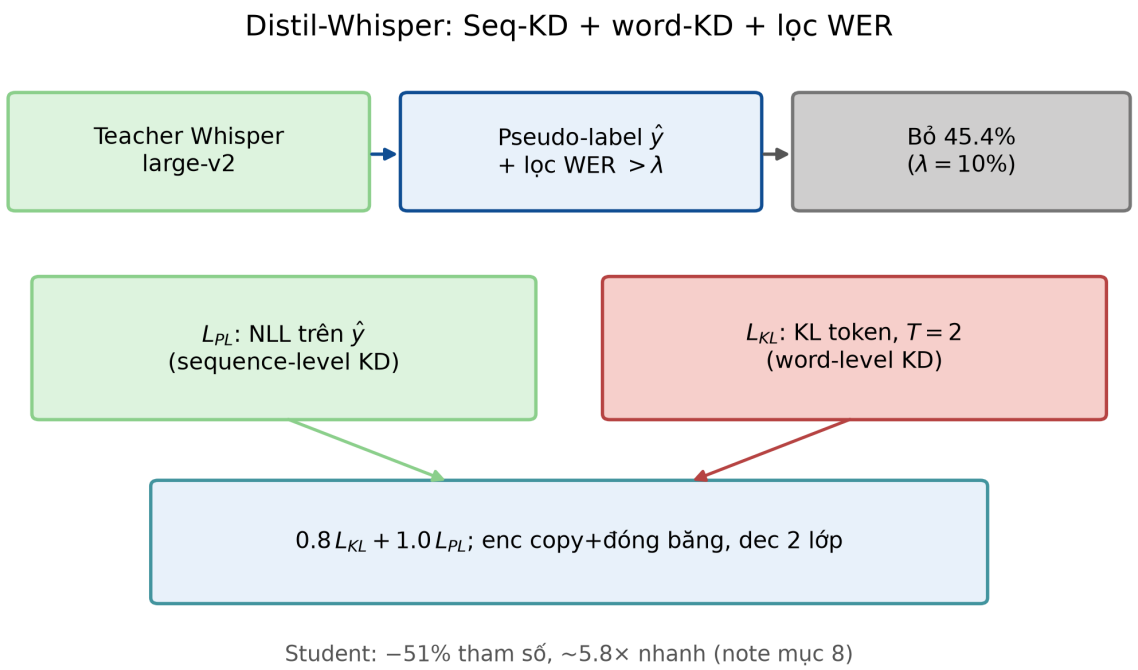
với Q là teacher, P là student, nhiệt độ softmax **2.0** cho cả hai (Phụ lục B.1). Tổng:

$$\boxed{L_{KD} = \alpha_{KL} L_{KL} + \alpha_{PL} L_{PL}, \quad \alpha_{KL} = 0.8,\ \alpha_{PL} = 1.0}$$

(theo Shleifer & Rush). Lý do chọn KL: cho thông tin trên mọi lớp và phương sai gradient thấp hơn CE (Hinton).

3.3 Lọc pseudo-label bằng ngưỡng WER

Chuẩn hoá cả nhãn thật và pseudo-label bằng bộ chuẩn hoá tiếng Anh của Whisper, tính WER giữa hai chuỗi, **loại mẫu nếu**



Distil-Whisper: $0.8L_{KL} + L_{PL}$, lọc WER$>10\%$ (bỏ 45.4%), encoder đóng băng (minh hoạ, tự tính)

$$\boxed{\mathrm{WER}(\mathrm{norm}(y), \mathrm{norm}(\hat{y})) > \lambda}$$

Chọn $\lambda = 10\%$ (tune trên tập validation). Lọc bù cho việc dùng greedy thay vì beam search khi gán nhãn (tác giả thấy hai cách khác biệt nhỏ nên chọn greedy cho nhanh).

3.4 Transcription dài: chunking

Whisper chỉ nhìn 30 giây. Thuật toán gốc dịch cửa sổ tuần tự theo timestamp, cần beam search và "temperature fallback". Paper dùng cách chunk (Patry 2022): cắt audio thành các đoạn có **chồng lấn nhỏ**, giải mã từng đoạn độc lập, rồi ghép tại vùng chồng bằng **chuỗi con chung dài nhất**. Chỉ cần greedy; vì các đoạn độc lập nên batch song song (nhanh tới 9x).

3.5 Speculative decoding

Assistant (Distil-Whisper) sinh vài token ứng viên; teacher kiểm chứng tất cả trong một lần forward; token chỉ được giữ nếu mọi token trước đó khớp dự đoán của teacher. Do đó đầu ra **giống hệt** teacher. Vì dùng chung encoder nên chỉ cần thêm decoder 2 lớp: +8% tham số.

3.6 Pseudo-code

```
# 1. Gán nhãn giả (một lần)
for (audio, y_true) in corpus_21k_hours:
    y_hat = teacher.generate(audio, greedy=True)
    if WER(norm(y_true), norm(y_hat)) <= 0.10:
        keep (audio, y_hat)

# 2. Khởi tạo student
student.encoder = copy(teacher.encoder); freeze(student.encoder)
student.decoder = [teacher.dec[0], teacher.dec[-1]]

# 3. Huấn luyện (80k bước, batch 256, 8 epoch)
```

```
for audio, y_hat in kept:
    H = student.encoder(audio)              # frozen, chạy forward 1 lần
    logits_s = student.decoder(y_hat, H)
    logits_t = teacher(audio, y_hat)        # teacher-forced trên pseudo-label
    L = 0.8 * KL(softmax(logits_t/2), softmax(logits_s/2)) \
      + 1.0 * CE(logits_s, y_hat)
```

Hyperparameter (Bảng 15): AdamW, $\beta = (0.9, 0.999)$, weight decay 0.0, LR đỉnh 1e-4 với 500 bước warmup rồi giảm tuyến tính, max grad norm 1.0, bf16, xác suất giữ timestamp 50%. **Không dùng augmentation hay regularisation** vì chỉ 8 epoch và dữ liệu đa dạng.

4. Thí nghiệm và kết quả chính

Dữ liệu train (Bảng 2), tổng 21 170 giờ, 9 bộ, 10 miền: People's Speech 12 000 h, GigaSpeech 2 500, Common Voice 13 2 400, Fisher 1 960, LibriSpeech 960, VoxPopuli 540, TED-LIUM 450, SwitchBoard 260, AMI 100. Đánh giá: 11 tập ID + 4 tập OOD ngắn (CHiME-4, Earnings-22, FLEURS, SPGISpeech) + 4 tập OOD dài (Earnings-21, Earnings-22, Meanwhile, Rev16). Latency đo trên A100 40GB, fp16.

Bảng 5: WER trung bình OOD và độ trễ tương đối so với large-v2 (ngắn: batch 1; dài: chunked, batch 16).

Mô hình	Tham số (M)	Ngắn: độ trễ	Ngắn: WER	Dài: độ trễ	Dài: WER
tiny.en	39	6.1	18.9	5.4	18.9
small.en	244	2.6	10.8	2.2	14.7
medium.en	769	1.4	9.5	1.3	12.3
large-v2	1550	1.0	9.1	1.0	11.7
distil-medium.en	394	6.8	11.1	8.5	12.4
distil-large-v2	756	5.8	10.1	5.8	**11.6**

(Chú ý: "độ trễ tương đối" ở đây là hệ số tăng tốc, số lớn hơn là nhanh hơn.)

Dài: distil-large-v2 thấp hơn large-v2 0.1 WER. **Bảng 7:** large-v2 tuần tự 10.4 WER; large-v2 chunked 11.7 (nhanh 9.9x); distil-large-v2 chunked 11.6 (nhanh 57.5x so với large-v2 tuần tự).

Ảo giác (Bảng 8, trung bình 4 tập dài):

Mô hình	5-Dup.	IER	SER	DER	WER
wav2vec2-large-960h (CTC)	7971	4.8	18.9	4.6	28.3
large-v2	23792	3.3	3.9	4.5	11.7
distil-large-v2	18503	2.1	5.3	4.2	11.6

(5-Dup. = số 5-gram từ lặp; IER/SER/DER = tỷ lệ chèn/thay/xoá.) Distil-large-v2 có ít lặp hơn 1.3x và IER thấp hơn 1.2 điểm, nhưng SER cao hơn 1.4 điểm. Mô hình CTC chỉ-encoder lặp rất ít (bất lợi chung của seq2seq).

Nhiễu (Hình 2): dưới nhiễu tăng dần, WER của Distil-Whisper suy giảm chậm hơn các mô hình train trên LibriSpeech và tiến gần Whisper, nhờ dùng chung encoder đóng băng. Ở SNR 40 dB nhiều mô hình tốt hơn. (Số chi tiết nằm trong hình, không có trong bản trích.)

Speculative decoding (Bảng 9, so với large-v2 không SD = 1.0): large-v2 + tiny: 2.1x; large-v2 + distil-large-v2: **2.0x** (1669M tham số); medium.en + distil-medium.en: 3.3 (so với 1.4 của medium.en riêng); medium.en + tiny.en: 2.7.

Effective robustness (Bảng 6, macro-trung bình gồm GigaSpeech): large-v2 9.4, distil-large-v2 10.2 (+10.7% tương đối); riêng FLEURS 4.2 → 6.3, SPGISpeech 3.8 → 3.3 (cải thiện).

5. Phân tích / ablation

Ngưỡng WER (Bảng 10, distil-large-v2 train 10 000 bước):

λ	Dữ liệu bị lọc %	WER ID	WER OOD	WER TB
100 (không lọc)	0.0	14.8	9.1	13.4
40	11.9	13.3	7.4	12.0
20	24.2	13.1	7.3	11.7
10	45.4	12.6	7.4	11.4
5	60.3	12.6	7.3	11.4

Lọc quá lỏng để lọt nhãn sai/ảo giác; lọc quá chặt chỉ giữ mẫu dễ và vứt nhiều dữ liệu. Chọn 10% là thoả hiệp; tác giả nói nếu train lâu hơn thì ngưỡng cao hơn có thể có lợi.

Lượng dữ liệu (Bảng 11): WER TB 16.4 (435 h) → 12.0 (3 483 h) → 11.6 (13 933 h) → 11.4 (21 770 h). OOD cải thiện nhanh tới 3 483 h, chậm tới 13 933 h (≈2% dữ liệu pre-train của Whisper), rồi không thêm khi dùng toàn bộ.

Kích thước student (Bảng 12, 10 000 bước):

Enc	Dec	Tham số (M)	Tăng tốc	OOD WER
32	32 (teacher)	1543	1.0	5.3
32	16	1124	1.9	5.4
32	8	914	3.0	6.0
32	4	809	4.3	6.5
32	2	756	5.8	7.4
16	2	441	8.6	10.5

Giảm decoder xuống 16 lớp gần như không mất; xuống 2 lớp mất 2.1 điểm OOD. **Giảm encoder** xuống 16 lớp (giảm thêm 19% tham số) làm OOD WER xấu thêm 3.1 điểm tuyệt đối: encoder sâu rất quan trọng.

6. Giới hạn

Do tác giả nêu: nhược điểm chính là lặp/ảo giác vẫn còn ở mô hình seq2seq (so với CTC); cần nghiên cứu thêm; distil-medium.en kém hơn trên tập Meanwhile (một người nói, nhiều từ hiếm); SER tăng.

Nhận xét:

- Chỉ **tiếng Anh** (bản .en và bộ chuẩn hoá tiếng Anh); đa ngôn ngữ không được thử trong bài báo này.
- Ngưỡng WER dựa vào nhãn thật; ở tình huống không có nhãn (dữ liệu hoàn toàn chưa gán) bộ lọc này không chạy được.
- Lọc 45% dữ liệu có thể làm lệch về mẫu dễ; bài báo không phân tích loại mẫu khó bị loại.
- Ablation train chỉ 10 000 bước (2 epoch), chưa chắc phản ánh mô hình 80 000 bước. Paper không có ablation tách riêng KL và PL (không cho biết mỗi thành phần đóng góp bao nhiêu).
- Thuật toán chunking cho audio dài dùng chung cho teacher lẫn student, nên cải thiện ảo giác có thể một phần đến từ pipeline, và Bảng 7 cho thấy chunking làm teacher kém đi 1.3 WER so với tuần tự.
- Hai con số trung bình trong bài báo (Bảng 5 vs Bảng 6) khác nhau (10.1 vs 10.2 và 9.1 vs 9.4) vì Bảng 6 tính trên bộ tập khác (gồm GigaSpeech).

7. Liên hệ với các bài khác

- **15 (Sequence-level KD):** pseudo-label là chính nó; paper nhấn mạnh PL = sequence-level, KL = word-level, và kết hợp hai.
- **16 (DistilBERT):** cùng ý tưởng trọng số hoá KD + data loss, khởi tạo bằng lớp của teacher; Distil-Whisper kế thừa qua Shleifer & Rush.
- **32 (Speculative KD) / 29 (GKD):** dùng student làm drafter; ở đây student căn chỉnh phân phối với teacher (như DistilSpec).
- **45 (Sparse logit sampling):** bài đó nói cách *lưu* KL; Distil-Whisper tính KL online (teacher chạy trên pseudo-label mỗi bước).
- **47 (ShortOPD):** cùng gặp vấn đề lặp ở suffix; ShortOPD phát hiện và rút ngắn rollout lặp, còn Distil-Whisper giảm lặp một phần nhờ pseudo-label sạch + lọc.
- **36 (Distilling step-by-step) / 37 (Orca):** các cách dùng đầu ra teacher làm dữ liệu; so sánh với lọc theo nhãn thật ở đây.

Gợi ý học tập

Với một bài toán phiên âm có nhãn tham chiếu, có thể so sánh huấn luyện với và không lọc pseudo-label, rồi kiểm tra riêng dữ liệu ngoài miền. Ngưỡng WER 10% là lựa chọn trong nghiên cứu này và cần được khảo sát lại ở thiết lập khác.

8. Tóm tắt công thức / khái niệm

Khái niệm	Công thức / giá trị	Ghi chú
Pseudo-label loss	$L_{PL} = -\sum \log P(\hat{y}_i \mid \hat{y}_{<i}, H)$	sequence-level KD
KL loss	$L_{KL} = \sum_i \mathrm{KL}(Q_i \| P_i)$, $T = 2.0$	word-level KD
Tổng	$0.8 L_{KL} + 1.0 L_{PL}$	theo Shleifer & Rush
Bộ lọc	bỏ nếu $\mathrm{WER}(y, \hat{y}) > \lambda$, $\lambda = 10\%$	lọc 45.4% dữ liệu
Student	enc copy+đóng băng, dec 2 lớp (lớp 1 và N)	5.8x nhanh, −51% tham số
Chunked long-form	chồng lấn + LCS, greedy, batch	tới 9x nhanh
Speculative decoding	assistant đề xuất, teacher kiểm chứng	đầu ra không đổi; 2.0x
Dữ liệu	21 170 h, 9 bộ; lợi ích OOD chậm lại sau ~13 933 h	80k bước, batch 256

9. Kết luận

Distil-Whisper kết hợp encoder đóng băng, decoder rút gọn, pseudo-label được lọc theo WER 10%, và loss KL cộng CE. Mô hình nhỏ hơn 51%, nhanh hơn 5.8 lần và có WER OOD trên âm thanh ngắn cao hơn teacher một điểm. Trên âm thanh dài, kết quả ngang hoặc tốt hơn cùng ít lỗi ảo giác chèn hơn.

Nguồn: https://arxiv.org/abs/2311.00430

Bài 47

ShortOPD: Recovering Pruned LLMs with Short-to-Long On-Policy Distillation (2026)

Tác giả: Qingyu Zhang, Qianhao Yuan, Hongyu Lin, Yaojie Lu, Xianpei Han, Le Sun, Ming Xu, Jiarui Li (ByteDance; Viện Phần mềm, Học viện Khoa học Trung Quốc; UCAS). **Venue / arXiv:** arXiv:2607.13124 (v2, 14/08/2026). Bản text ghi "Work in Progress"; chưa nêu venue. **Nguồn:** https://arxiv.org/abs/2607.13124
Chủ đề: Phục hồi LLM sau structured pruning bằng **on-policy distillation (OPD)** với chính mô hình trước khi cắt làm teacher đóng băng; thêm bộ điều khiển độ dài rollout "ngắn → dài" để tránh phí token vào các đuôi lặp. **Vị trí trong lộ trình:** Đọc trước: 15 (SeqKD), 28 (MiniLLM), 29 (GKD, on-policy KD), 31 (Rethinking KL), 46 (Distil-Whisper). Bài này bổ sung: đánh giá phục hồi sau cắt bằng **sinh tự do** (không chỉ trắc nghiệm) và so sánh trực tiếp OPD với SFT/SeqKD/KD/RLVR. Là bài cuối của lộ trình.

> Đây là ghi chú học tập bằng tiếng Việt, không phải bản dịch.

1. Bối cảnh

Structured pruning (cắt có cấu trúc) loại bỏ nguyên khối tham số (lớp, đầu attention, neuron MLP) nên mô hình vẫn dày và chạy trên kernel thông thường. Tuy nhiên phần lớn nghiên cứu đánh giá bằng bài **trắc nghiệm** (MMLU, HellaSwag): mô hình chỉ cần xếp hạng đáp án. Trên **sinh văn bản tự do** (toán, code, hội thoại) cùng checkpoint ấy sụp đổ. Khoảng trống này gọi là *recognition-generation gap*.

Một vài thuật ngữ:

- **pass@k:** xác suất ít nhất 1 trong k mẫu sinh ra là đúng (toán/code). pass@1 = greedy/mẫu đơn.
- **On-policy:** dữ liệu huấn luyện do chính student sinh (nó tự "chạy" rồi bị chấm), khác **off-policy** là dữ liệu cố định từ nguồn khác (xem 29 GKD).
- **Teacher-forced (KD thường):** student được cho tiền tố *đúng* của chuỗi cố định rồi học khớp phân phối teacher ở mỗi vị trí. Có thể gây *exposure bias*: lúc suy luận student gặp trạng thái do chính nó tạo ra mà khi huấn luyện không thấy.
- **RLVR:** RL với phần thưởng kiểm chứng được (đúng/sai cuối cùng), tín hiệu thưa.
- **Block Influence (BI)** của ShortGPT: lớp càng ít làm đổi hidden state (cosine giữa đầu vào và đầu ra lớp càng cao) thì càng dễ bỏ.

Hai quan sát dẫn đường của bài báo:

1. Năng lực bị **"hạ cấp" chứ không bị xoá**: sau khi bỏ 4/36 lớp của Qwen3-4B-Instruct, greedy gần như sụp nhưng pass@k tăng đều với k; GSM8K đạt 91.2% ở k=64, cao hơn greedy của teacher (88.1%).
2. Lỗi trong vùng có thể phục hồi chủ yếu là **lặp ở đuôi (suffix repetition)**: câu trả lời bắt đầu hợp lý rồi rơi vào vòng lặp.

2. Ý tưởng chính

Vì năng lực vẫn nằm trong phân phối lấy mẫu của mô hình đã cắt, cách phục hồi tốt là cho student **tự sinh** rollout, để teacher (chính mô hình gốc, đóng băng) chấm lại từng token và student khớp phân phối token kế tiếp của teacher tại mọi vị trí (OPD: on-policy + dày đặc + không cần nhãn hay verifier). Nhược điểm: giai đoạn đầu student lặp rất nhiều, rollout dài tốn tiền sinh mà lại ít tín hiệu (các token lặp có JSD với teacher chỉ 0.0014 so với 0.051 ở token thường, thấp ~35 lần). ShortOPD thêm một bộ điều khiển phát hiện đuôi lặp, rút ngắn ngân sách độ dài rollout H khi lặp cao và tăng lại khi sạch, nên đạt chất lượng của rollout dài 8192 token với 1/4 thời gian.

3. Phương pháp chi tiết

3.1 Cắt và OPD cơ bản

BI của lớp i (trên tập hiệu chuẩn giữ riêng):

$$\mathrm{BI}_i = 1 - \mathbb{E}_{X,t}\left[\frac{X_{i,t}^{\top} X_{i+1,t}}{\|X_{i,t}\|_2 \|X_{i+1,t}\|_2}\right]$$

Bỏ các lớp BI thấp nhất tới khi mất ~25% tham số (9/36 lớp) → student π_θ; mô hình gốc là teacher π_T đóng băng. Mất mát OPD:

$$\boxed{\mathcal{L}_{OPD}(\theta) = \mathbb{E}_x \mathbb{E}_{y\sim\pi_\theta(\cdot|x)}\Big[\frac{1}{|y|}\sum_{t=1}^{|y|} D_\alpha(\pi_T(\cdot|x, y_{<t}), \pi_\theta(\cdot|x, y_{<t}))\Big]}$$

D_α là Jensen–Shannon tổng quát, với $m = \alpha p + (1-\alpha)q$:

$$D_\alpha(p,q) = \alpha \sum_i p_i \log \frac{p_i}{m_i} + (1-\alpha)\sum_i q_i \log \frac{q_i}{m_i}$$

Dùng $\alpha = 0.5$ ("thoả hiệp có chặn" giữa KL thuận mode-covering và KL nghịch mode-seeking). Triển khai: **top-100 logit của teacher + 1 bin gộp phần đuôi**, và trọng số importance **bị cắt ở 2.0** so với chính sách sinh rollout. Không có policy-gradient, không phần thưởng.

3.2 Gradient gần điểm khớp (Phụ lục C)

Với logit student z, $q = \mathrm{softmax}(z)$:

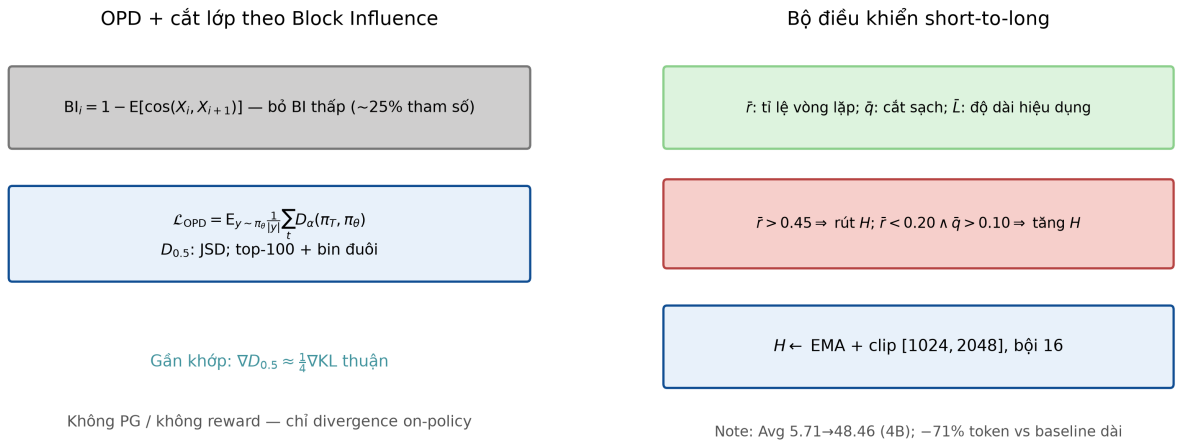
$$\frac{\partial D_\alpha}{\partial z_j} = (1-\alpha)\, q_j \Big[\log \frac{q_j}{m_j} - \sum_i q_i \log \frac{q_i}{m_i}\Big]$$

Khi $q = p$ thì $m = p$ và gradient bằng 0. Viết $q = p + \delta$: gradient $\approx \alpha(1-\alpha)\delta_j$, tức ở $\alpha = 0.5$ chỉ bằng **1/4** gradient KL thuận (δ_j). Hệ quả cho vòng lặp: khi teacher *cũng* tin vòng lặp (NLL teacher 3×10^{-5} so với 0.68 ở token thường) thì $p \approx q$ và gradient gần 0, tín hiệu hầu như bằng không dù tốn token. Lưu ý: vì ánh xạ về "top-100 + bin đuôi", mất mát chỉ nhìn thấy sự khác nhau ở độ phân giải đó (khác biệt trong bin đuôi vô hình).

3.3 Bộ điều khiển short-to-long

Mỗi bước, trên B rollout:

- $a_i = 1$ nếu **bộ dò vòng lặp ở cuối** (Phụ lục B) kết luận rollout i rơi vào vòng lặp nghiêm trọng.
- $b_i = 1$ nếu rollout lấp đầy ngân sách H_s mà không lặp ("cắt cụt sạch").
- ℓ_i: độ dài hiệu dụng (phần tiền tố trước vòng lặp, tinh chỉnh bằng loss OPD và NLL teacher thấp); $\ell_i = |y_i|$ nếu không lặp.



ShortOPD: OPD với $D_{0.5}$ on-policy và bộ điều khiển short-to-long theo tỉ lệ vòng lặp $\bar{r}$ (minh hoạ, tự tính)

$$r_s = \frac{1}{B}\sum a_i, \quad q_s = \frac{1}{B}\sum b_i, \quad L_s = \frac{1}{B}\sum \ell_i, \qquad (\bar{r}_s, \bar{q}_s, \bar{L}_s) = \mathrm{EMA}_\beta(r_s, q_s, L_s)$$

$$H_s^* = \begin{cases} \min(H_s, \lambda \bar{L}_s) & \bar{r}_s > \rho_{high} \text{ (rút ngắn)} \\ \gamma_\uparrow H_s & \bar{r}_s < \rho_{low} \ \wedge \ \bar{q}_s > \tau \text{ (tăng)} \\ H_s & \text{ngược lại} \end{cases} \qquad H_{s+1} = \mathrm{clip}_{[H_{min}, H_{max}]}(\lfloor \beta_H H_s + (1-\beta_H) H_s^* \rceil_{16})$$

Thông số (Bảng 10): $(\rho_{low}, \rho_{high}, \tau) = (0.20, 0.45, 0.10)$; EMA 0.7 cho cả thống kê và ngân sách; $\lambda = 1.15$, $\gamma_\uparrow = 1.25$; làm tròn bội 16; ngân sách khởi tạo 2048, biên $[1024, 2048]$. Dò vòng lặp: cửa sổ cuối $W = 512$, chu kỳ $p \in [1, 10]$, khớp ≥ 90% ở 32 token cuối, ≥ 3 chu kỳ, đuôi tối thiểu 64, "nghiêm trọng" nếu ≥ 128 token hoặc ≥ 30% hậu tố. **Quan trọng:** loss thấp *không bao giờ* tự mở cổng rút ngắn (chỉ dò lặp cấu trúc mới làm được), vì loss thấp nhất đúng lúc lặp. Mất mát OPD vẫn tính trên **mọi token** đã sinh; bộ điều khiển chỉ đổi ngân sách sinh, không che loss, và không tốn thêm forward.

3.4 Pseudo-code

```
H = H_max (=2048); EMAs khởi tạo
for step s in 1..S:
    y_i ~ student(.|x_i), tối đa H token              # on-policy rollout
    a_i = terminal_loop(y_i);  b_i = (len(y_i)==H and a_i==0)
    chạy teacher (đóng băng) trên cùng y_i -> top-100 + bin đuôi
    cập nhật student với L_OPD trên toàn bộ token (JSD alpha=0.5, IS clip 2.0)
    tính r_s, q_s, L_s; cập nhật EMA
    if r_bar > rho_high:                H* = min(H, lam * L_bar)
    elif r_bar < rho_low and q_bar>tau: H* = gamma_up * H
    else:                               H* = H
    H = clip(round16(beta_H*H + (1-beta_H)*H*), H_min, H_max)
```

3.5 Các baseline (Bảng 2: "không gian thiết kế")

Phương pháp	On-policy	Tín hiệu dày	Không nhãn	Không verifier
SFT w/o KD (nhãn vàng)	không	không	không	có

Phương pháp	On-policy	Tín hiệu dày	Không nhãn	Không verifier
SeqKD (phản hồi sinh bởi cùng teacher)	không	không	có	có
KD (teacher-forced, KL thuần T=1, không CE)	không	có	không	có
RLVR (PPO/GRPO)	có	không	không	không
ShortOPD	có	có	có	có

Baseline KD được chọn để mô phỏng quy trình prune-then-distill kiểu Minitron.

4. Thí nghiệm và kết quả chính

Thiết lập. Qwen3-4B-Instruct-2507, Qwen3-4B-Thinking-2507, Qwen3-8B (no-thinking và thinking); 36 lớp; student bỏ 9 lớp BI thấp nhất (25% độ sâu; 3.11B tham số ở 4B, ~6.45B ở 8B cho width/hybrid). Kho phục hồi: trộn toán/code/hội thoại; cùng prompt cho mọi phương pháp; 710 bước, lr 2×10^{-6}, batch 64, 8 rollout/prompt, nhiệt độ rollout 0.8, tối đa 4096 token phản hồi, trong một node 8 GPU. Đánh giá 8 nhiệm vụ sinh tự do: GSM8K, MATH-500, HumanEval, MBPP (chính xác/pass@1), Alpaca/QA/Summarization/MT-Bench (judge GPT-5.5, 1–10). Avg = trung bình 8 nhiệm vụ (điểm judge ×10); T% = % so với teacher.

Bảng 3 (trích): Qwen3-4B-Instruct, cắt 25%.

Phương pháp	GSM8K	MATH	HumanEval	MBPP	Alpaca	QA	Sum	MT-B	Avg	T%
Teacher đặc	88.10	89.40	75.00	80.54	6.64	4.38	8.86	6.95	75.17	100
Đã cắt	1.14	1.60	0.00	0.00	1.14	1.00	1.06	1.09	5.71	7.6
SFT w/o KD	25.93	9.40	26.83	29.18	1.91	1.19	3.12	1.59	21.19	28.2
SeqKD	32.83	12.20	29.88	41.25	2.52	1.38	5.26	2.10	28.60	38.0
KD	29.34	9.60	29.27	39.30	2.94	1.48	6.89	2.36	30.52	40.6
ShortOPD	62.70	42.00	43.90	49.81	4.68	2.04	7.94	4.28	**48.46**	64.5

Avg của ShortOPD thành "~9x giá trị chưa phục hồi" và 18 điểm hơn baseline off-policy tốt nhất. Các khối khác (Avg, T%): 4B Thinking: teacher 73.75; pruned 5.21; SFT/SeqKD/KD 18.75/29.21/35.13; ShortOPD **55.48** (75.2%). 8B no-thinking: teacher 76.05; pruned 5.39; SeqKD 31.86 (tốt nhất off-policy); ShortOPD **42.84** (56.3%). 8B Thinking: teacher 77.34; pruned 5.30; SFT 35.46; ShortOPD **53.71** (69.4%). (Giao thức giải mã khác nhau giữa các khối; chỉ so trong khối.)

Độ lặp theo độ sâu cắt (Hình 2a): ở mốc 25%, 84% rollout có đuôi lặp. Ở cắt mạnh hơn (> khoảng 12–13 lớp) đầu ra thành "salad" token không còn lặp được (Phụ lục E). Vì vậy 25% được coi là "điểm tới hạn" của vùng có thể phục hồi nhẹ.

Hiệu quả của bộ điều khiển (Mục 4.3): vanilla OPD cố định $H = 2048$: 55–75% rollout lặp, loss đạt plateau sau ~80 bước. ShortOPD: ~40–50 bước, ngân sách hạ xuống 1024 rồi quay lại 2048. Thời gian sinh rollout 100 bước đầu giảm 29% (35.4 → 25.1 s/bước); đầu-cuối 11.1 → 8.5 giờ. So sánh 3 lịch (Hình 5): Avg 48.5 đến 50.2; token rollout 250M (ShortOPD) / 337M (cố định 2048) / 869M (cố định 8192); fixed 8192 tốn 35.9 giờ cho +0.6 Avg so với fixed 2048; ShortOPD trong vòng 1.7 điểm Avg của cả hai và ít hơn 71% token rollout so với 8192.

Scaling theo epoch (Bảng 4): Avg 48.46 (710 bước) → 53.45 (2 epoch) → 55.41 (3 epoch, 73.7% teacher).

Dày vs thưa (Bảng 5; Math+Code, 29 585 prompt): Qwen3-4B-Instruct: PPO = 0 trên cả 4 điểm; GRPO 0.91/0.80/0/0; ShortOPD 58.68/38.00/41.46/44.36 (Gen. Avg 38.85 vs ≤ 5.38). 8B no-thinking: PPO 14.78/2.40/0/0, ShortOPD 76.88/39.60/40.24/42.41 (Avg 41.62 vs ≤ 7.68). Tác giả giải thích: student hiếm khi sinh ra đáp án đúng nên thưởng thưa gần như không có mẫu thành công để khuếch đại, còn teacher dày cho tín hiệu trên mọi rollout.

Width / hybrid (Bảng 7, KD vs ShortOPD, cùng 710 bước): gain 6.44–9.01 điểm.

Mô hình, cấu trúc	Đã cắt	KD	ShortOPD	Teacher
4B-Instruct, width (29.5%)	17.23	43.70	51.46	74.58
4B-Instruct, hybrid (29.4%)	19.71	46.15	55.16	74.58
8B no-thinking, width (21.0%)	19.67	52.42	58.86	75.24
8B no-thinking, hybrid (21.2%)	16.17	48.66	56.65	75.24

5. Phân tích / ablation

- **Nguồn quỹ đạo × loại tín hiệu:** cùng teacher, chuyển tín hiệu sang off-policy mất 19.9 điểm Avg (SeqKD); tín hiệu dày nhưng teacher-forced mất 17.9 điểm (KD). Cả hai thành phần (on-policy *và* dày) đều cần.
- **Corpus phục hồi (Bảng 6, leave-one-domain-out, cùng 710 bước):** Full 48.46. Bỏ **code**: HumanEval 0.61, MBPP 2.33, Avg 34.96. Bỏ **toán**: GSM8K 8.04, MATH 6.00, Avg 30.51. Bỏ **hội thoại chung**: Avg 39.02 (code tăng nhẹ nhưng toán và hội thoại tụt). Kết luận: OPD chỉ sửa các trạng thái mà student ghé qua; **độ phủ prompt phải khớp năng lực cần phục hồi**.
- **Trắc nghiệm không đủ làm proxy (Phụ lục F):** mô hình width/hybrid đã cắt giữ MC Avg 51.14 (4B width) / 69.41 (8B hybrid) nhưng Gen. Avg chỉ 17.23 / 16.17; ShortOPD hơn KD 0.87–2.08 điểm MC so với 6.44–9.01 điểm sinh. Trên trắc nghiệm dạng loglik của Qwen3-4B-Instruct (cắt sâu), SFT/KD (74.9/74.4) lại hơn ShortOPD 1 epoch (67.6; 71.6 sau 3 epoch); kết hợp SFT-init + ShortOPD đạt 76.8, ngang teacher 76.7.
- **Bộ điều khiển vs lịch cố định:** xem mục 4; tiết kiệm token mà giữ Avg trong 2 điểm.

6. Giới hạn

Do tác giả nêu: chỉ khảo sát BI depth / width / hybrid trên Qwen3 4B và 8B; cần thêm họ mô hình, tỷ lệ nén và quy mô, và điểm mà nén quá mạnh khiến phục hồi nhẹ không đủ.

Nhận xét:

- Bài còn là bản "work in progress"; nhiều so sánh một lần chạy (không thấy độ lệch chuẩn, không có seed).
- Hiện tượng "lặp ở đuôi" được đo ở **mức 25% độ sâu**; ở cắt mạnh hơn lỗi chuyển sang đầu ra không mạch lạc nên chẩn đoán và bộ điều khiển có thể không còn áp dụng.
- OPD cần teacher (mô hình gốc) **chạy trực tuyến** trên rollout của student: tốn bộ nhớ/tính toán hơn KD với logit cache; không so sánh chi phí tổng quát với KD ngoài giờ rollout.
- Baseline KD không có CE cứng, nhiệt độ 1.0, forward KL; một baseline KD được tinh chỉnh hơn (trộn CE, nhiệt độ, nhiều epoch) có thể thu hẹp khoảng cách. ShortOPD ở 1 epoch, nhưng 3 epoch của baseline không được báo cáo.
- Thực nghiệm Math+Code RLVR chỉ chạy đến bước 462 và mang tính kiểm soát; không loại trừ rằng RL tinh chỉnh hơn (khởi động bằng SFT) làm tốt hơn.
- Độ phủ prompt là yếu tố lớn (Bảng 6) nhưng chỉ khảo sát 3 miền.
- Bảng 1 (pass@k) dùng cắt 4 lớp còn các bảng chính cắt 9 lớp, nên hai quan sát "hạ cấp chứ không xoá" và kết quả phục hồi không đo trên cùng một student.

7. Liên hệ với các bài khác

- **29 (GKD) / 28 (MiniLLM):** nguồn gốc ý tưởng distill trên trạng thái của student, và JSD/KL nghịch; ShortOPD dùng JSD $\alpha = 0.5$ chặn importance weight 2.0.
- **15 (SeqKD) / 46 (Distil-Whisper):** baseline SeqKD ~ pseudo-label; Distil-Whisper thành công nhờ pseudo-label sạch, còn ở đây SeqKD thua OPD vì độ hư hại lớn hơn sau cắt nặng.
- **31 (Rethinking KL):** chọn D_α ở giữa; tác giả lập luận trade-off này độc lập với on/off-policy.
- **45 (Sparse logit sampling):** lưu top-K logit chệch; ShortOPD dùng **top-100 + bin đuôi gộp** (giống ghost token), một cách hợp lý để tránh bias mà không lưu cả từ vựng.
- **40 (Gemma 2), 44 (Qwen3):** các pipeline distillation chính thức; ShortOPD cho thấy phục hồi sau cắt cần xử lý khác hẳn pre-training distillation.
- **39 (Small models struggle with strong reasoners):** góc nhìn bổ sung về khoảng cách năng lực student–teacher.

8. Tóm tắt công thức / khái niệm

Khái niệm	Công thức / giá trị	Ghi chú
Block Influence	$1 - \mathbb{E}[\cos(X_i, X_{i+1})]$	bỏ lớp BI thấp
OPD loss	$\mathbb{E}_{y\sim\pi_\theta} \frac{1}{\lvert y\rvert} \sum_t D_\alpha(\pi_T, \pi_\theta)$	on-policy, dày, không nhãn
JSD tổng quát	$D_\alpha = \alpha \mathrm{KL}(p\Vert m) + (1-\alpha)\mathrm{KL}(q\Vert m)$	$\alpha = 0.5$
Gradient gần khớp	$\approx \alpha(1-\alpha)\delta_j$	1/4 của KL thuận tại $\alpha = 0.5$
Mục tiêu top-K	top-100 + 1 bin đuôi	clip IS = 2.0
Cổng rút ngắn	$\bar{r} > 0.45 \Rightarrow H^* = \min(H, 1.15\bar{L})$	
Cổng tăng	$\bar{r} < 0.20 \wedge \bar{q} > 0.10 \Rightarrow H^* = 1.25H$	
Làm mượt	$H' = 0.7H + 0.3H^*$, bội 16, $[1024, 2048]$	
Kết quả chính	Avg 5.71 → 48.46 (4B-Instruct, −25%)	KD 30.52, SeqKD 28.60
Hiệu quả	8.5 vs 35.9 giờ; 250M vs 869M token	−71% token

9. Kết luận

Sau cắt cấu trúc, bài báo ghi nhận khả năng sinh suy giảm và lỗi lặp ở đuôi. OPD cho student tự sinh rồi học phân phối từ teacher trước khi cắt, dùng JSD trên top-100 và phần đuôi. ShortOPD điều chỉnh độ dài rollout theo mức lặp, tiết kiệm 71% token với chất lượng gần tương đương trong thí nghiệm.

Nguồn: https://arxiv.org/abs/2607.13124